
Apache ShardingSphere document

Apache ShardingSphere

May 14, 2025

1	What is ShardingSphere	1
1.1	Introduction	1
1.1.1	ShardingSphere-JDBC	1
1.1.2	ShardingSphere-Proxy	1
1.2	Product Features	2
1.3	Advantages	2
2	Design Philosophy	4
2.1	Connect: Create database upper level standard	5
2.2	Enhance: Database computing enhancement engine	5
2.3	Pluggable: Building database function ecology	5
2.3.1	L1 Kernel Layer	6
2.3.2	L2 Feature Layer	6
2.3.3	L3 Ecosystem Layer	6
3	Deployment	7
3.1	Using ShardingSphere-JDBC	7
3.2	Using ShardingSphere-Proxy	8
3.3	Hybrid Architecture	9
4	Running Modes	11
4.1	Standalone Mode	11
4.2	Cluster Mode	11
5	Roadmap	12
6	Get Involved	13
7	Quick Start	14
7.1	ShardingSphere-JDBC	14
7.1.1	Scenarios	14
7.1.2	Limitations	14
7.1.3	Requirements	14

7.1.4	Procedure	14
7.2	ShardingSphere-Proxy	16
7.2.1	Scenarios	16
7.2.2	Limitations	16
7.2.3	Requirements	17
7.2.4	Procedure	17
8	Features	19
8.1	Sharding	19
8.1.1	Background	19
	Vertical Sharding	20
	Horizontal Sharding	21
8.1.2	Challenges	22
8.1.3	Goal	22
8.1.4	Application Scenarios	22
	Mass data high concurrency in OLTP scenarios	22
	Mass data real-time analysis in OLAP scenarios	23
8.1.5	Related References	23
8.1.6	Core Concept	23
	Table	23
	Data Nodes	25
	Sharding	26
8.1.7	Limitations	29
	Stable Support	29
	Experimental Support	31
	Do not Support	32
8.1.8	Appendix with SQL operator	33
8.2	Distributed Transaction	33
8.2.1	Background	33
8.2.2	Challenge	34
8.2.3	Goal	34
8.2.4	How it works	34
	LOCAL Transaction	35
	XA Transaction	35
	BASE Transaction	36
8.2.5	Application Scenarios	37
	Application Scenarios for ShardingSphere XA Transactions	37
	Application Scenarios for ShardingSphere BASE Transaction	37
	Application Scenarios for ShardingSphere LOCAL Transaction	37
8.2.6	Related references	37
8.2.7	Core Concept	37
	XA Protocol	37
8.2.8	Limitations	38
	LOCAL Transaction	38
	XA Transaction	38

	BASE Transaction	38
8.2.9	Appendix with SQL operator	38
8.3	Readwrite-splitting	39
8.3.1	Background	39
8.3.2	Challenges	40
8.3.3	Goal	40
8.3.4	Application Scenarios	40
	Complex primary-secondary database architecture	40
8.3.5	Related References	41
8.3.6	Core Concept	41
	Primary database	41
	Secondary database	41
	Primary-Secondary synchronization	41
	Load balancer policy	41
8.3.7	Limitations	41
8.4	DB Gateway	42
8.4.1	Background	42
8.4.2	Challenges	42
8.4.3	Goal	42
8.4.4	Application Scenarios	42
8.4.5	Core Concept	42
	SQL Dialect	42
8.4.6	Limitations	43
8.5	Traffic Governance	43
8.5.1	Background	43
8.5.2	Challenges	43
8.5.3	Goal	43
8.5.4	Application Scenarios	43
	Overloaded compute node protection	43
	Storage node traffic limit	44
8.5.5	Core Concept	44
	Circuit Breaker	44
	Request Limit	44
8.6	Data Migration	44
8.6.1	Background	44
8.6.2	Challenges	44
8.6.3	Goal	45
8.6.4	Application Scenarios	45
8.6.5	Related References	45
8.6.6	Core Concept	45
	Nodes	45
	Cluster	45
	Source	45
	Target	45
	Data Migration Process	46

	Stock Data	46
	Incremental Data	46
8.6.7	Limitations	46
	Procedures Supported	46
	Procedures not supported	46
8.7	Encryption	46
8.7.1	Background	46
8.7.2	Challenges	47
8.7.3	Goal	47
8.7.4	Application Scenarios	47
8.7.5	Related References	47
8.7.6	Core Concept	47
	Logic column	47
	Cipher column	48
	Assisted query column	48
	Like query column	48
8.7.7	Limitations	48
8.7.8	Appendix with SQL operator	48
8.8	Data Masking	49
8.8.1	Background	49
8.8.2	Challenges	49
8.8.3	Goal	49
8.8.4	Application Scenarios	49
8.8.5	Related References	49
8.8.6	Core Concept	50
	Logic column	50
8.8.7	Limitations	50
8.9	Shadow	50
8.9.1	Background	50
8.9.2	Challenges	50
8.9.3	Goal	51
8.9.4	Application Scenario	51
8.9.5	Related References	51
8.9.6	Core Concept	51
	Production Database	51
	Shadow Database	51
	Shadow Algorithm	51
8.9.7	Limitations	52
	Hint based shadow algorithm	52
	Column based shadow algorithm	52
8.10	Observability	53
8.10.1	Background	53
8.10.2	Challenges	55
8.10.3	Goal	55
8.10.4	Application Scenarios	55

	Monitoring panel	55
	Monitoring application performance	55
	Tracing application links	55
8.10.5	Related References	56
8.10.6	Core Concept	56
	Agent	56
	APM	56
	Tracing	56
	Metrics	56
	Logging	56
8.11	SQL Federation	57
8.11.1	Background	57
8.11.2	Challenges	57
8.11.3	Goal	57
8.11.4	Application Scenario	57
8.11.5	Related References	58
8.11.6	Limitations	58
9	User Manual	59
9.1	ShardingSphere-JDBC	59
9.1.1	YAML Configuration	60
	Overview	60
	Usage	60
	YAML Syntax Explanation	61
	Mode	61
	Data Source	63
	Rules	65
	Algorithm	86
	JDBC Driver	88
9.1.2	Java API	98
	Overview	98
	Usage	98
	Mode	99
	Data Source	102
	Rules	103
	Algorithm	131
9.1.3	Special API	133
	Sharding	133
	Readwrite-splitting	136
	Transaction	138
9.1.4	Optional Plugins	157
	ClickHouse	159
	HiveServer2	164
9.1.5	Unsupported Items	177
	Configuration	177

	DataSource Interface	177
	Connection Interface	178
	Statement and PreparedStatement Interface	178
	ResultSet Interface	178
	JDBC 4.1	178
9.1.6	Observability	178
	Agent	178
	Usage	181
	Metrics	183
9.1.7	GraalVM Native Image	184
	Background Information	184
	Usage restrictions	186
	Contribute GraalVM Reachability Metadata	191
9.2	ShardingSphere-Proxy	193
9.2.1	Startup	194
	Use Binary Tar	194
	Use Docker	196
	Build GraalVM Native Image(Alpha)	197
	Use Helm	202
	Add dependencies	209
9.2.2	Yaml Configuration	210
	Authentication & Authorization	210
	Properties	213
	Rules	215
	Data Source	216
9.2.3	DistSQL	217
	Definition	217
	Related Concepts	217
	Impact on the System	218
	Limitations	219
	How it works	219
	Related References	220
	Syntax	220
	Usage	418
9.2.4	Data Migration	425
	Introduction	425
	Build	425
	Manual	428
9.2.5	Observability	443
	Agent	443
	Usage	445
	Metrics	446
9.2.6	Optional Plugins	446
	Seata AT Mode transactions	448
9.2.7	Session Management	458

	Usage	458
9.2.8	Logging Configuration	459
	Background	459
	Procedure	459
9.2.9	CDC	460
	Build	460
	Manual	466
	Precautions	473
9.3	Common Configuration	473
9.3.1	Properties Configuration	473
	Background	473
	Parameters	474
	Procedure	475
	Notes	475
	Sample	475
9.3.2	Builtin Algorithm	475
	Introduction	475
	Usage	475
	Metadata Repository	476
	Sharding Algorithm	479
	Key Generate Algorithm	486
	Load Balance Algorithm	488
	Encryption Algorithm	490
	Shadow Algorithm	491
	SQL Translator	493
	Sharding Audit Algorithm	493
	Data Masking Algorithm	494
	Row Value Expressions	498
9.3.3	SQL Hint	503
	Background	503
	Use specification	504
	Parameters	504
	SQL Hint	505
9.4	Error Code	507
9.4.1	SQL Error Code	507
	Kernel Exception	507
	Feature Exception	512
	Other Exception	515
9.4.2	Server Error Code	515
10	Dev Manual	516
10.1	Mode	516
10.1.1	StandalonePersistRepository	516
	Fully-qualified class name	516
	Definition	516

	Implementation classes	517
10.1.2	ClusterPersistRepository	517
	Fully-qualified class name	517
	Definition	517
	Implementation classes	518
10.2	SQL Parser	518
10.2.1	DatabaseTypedSQLParserFacade	518
	Fully-qualified class name	518
	Definition	519
	Implementation classes	520
10.2.2	SQLStatementVisitorFacade	521
	Fully-qualified class name	521
	Definition	521
	Implementation classes	522
10.3	Data Sharding	523
10.3.1	ShardingAlgorithm	523
	Fully-qualified class name	523
	Definition	523
	Implementation classes	524
10.3.2	ShardingAuditAlgorithm	525
	Fully-qualified class name	525
	Definition	525
	Implementation classes	525
10.3.3	DatetimeService	525
	Fully-qualified class name	525
	Definition	526
	Implementation classes	526
10.3.4	InlineExpressionParser	526
	Fully-qualified class name	526
	Definition	526
	Implementation classes	527
10.4	Infra algorithm	527
10.4.1	LoadBalanceAlgorithm	527
	Fully-qualified class name	527
	Definition	528
	Implementation classes	529
10.4.2	KeyGenerateAlgorithm	530
	Fully-qualified class name	530
	Definition	530
	Implementation classes	530
10.4.3	MessageDigestAlgorithm	531
	Fully-qualified class name	531
	Definition	531
	Implementation classes	531
10.5	SQL Audit	531

10.5.1	SQLAuditor	531
	Fully-qualified class name	531
	Definition	532
	Implementation classes	532
10.6	Encryption	532
10.6.1	EncryptAlgorithm	532
	Fully-qualified class name	532
	Definition	532
	Implementation classes	533
10.7	Data Masking	533
10.7.1	MaskAlgorithm	533
	Fully-qualified class name	533
	Definition	533
	Implementation classes	535
10.8	Shadow DB	536
10.8.1	ShadowAlgorithm	536
	Fully-qualified class name	536
	Definition	536
	Implementation classes	537
10.9	Observability	537
10.9.1	PluginLifecycleService	537
	Fully-qualified class name	537
	Definition	538
	Implementation classes	538
11	Test Manual	539
11.1	Integration Test	539
11.2	Module Test	539
11.3	Performance Test	539
11.4	Sysbench Test	540
11.5	Integration Test	540
11.5.1	Design	540
	Test case	540
	Test environment	540
	Test engine	541
11.5.2	User Guide	541
	Test case configuration	541
	Environment configuration	542
	Run the test engine	543
11.6	Performance Test	545
11.6.1	SysBench ShardingSphere-Proxy Empty Rule Performance Test	545
	Objectives	545
	Set up the test environment	546
	Test phase	547
11.6.2	BenchmarkSQL ShardingSphere-Proxy Sharding Performance Test	549

	Objective	549
	Method	549
	Fine tuning to test tools	549
	Stress testing environment or parameter recommendations	550
	Appendix	552
	BenchmarkSQL 5.0 PostgreSQL statement list	554
11.7	Module Test	563
11.7.1	SQL Parser Test	563
	Prepare Data	563
11.7.2	SQL Rewrite Test	564
	Target	564
11.8	Pipeline E2E Test	566
11.8.1	Objectives	566
11.8.2	Test environment type	566
11.8.3	User guide	566
	Environment setup	566
	Test case	566
	Running the test case	567
12	Reference	569
12.1	Database Compatibility	569
12.2	Database Gateway	570
12.3	Management	570
12.3.1	Data Structure in Registry Center	570
	/rules	573
	/props	573
	/metadata/\${databaseName}/data_sources/units/ds_0/versions/0	573
	/metadata/\${databaseName}/data_sources/nodes/ds_0/versions/0	573
	/metadata/\${databaseName}/rules/sharding/tables/t_order/versions/0	574
	/metadata/databaseName/schemas/{schemaName}/tables/t_order/versions/0	574
	/nodes/compute_nodes	575
	/nodes/qualified_data_sources	575
12.4	Sharding	575
12.4.1	SQL Parser	576
12.4.2	SQL Route	577
12.4.3	SQL Rewrite	577
12.4.4	SQL Execution	577
12.4.5	Result Merger	577
12.4.6	Query Optimization	577
12.4.7	Parse Engine	577
	Abstract Syntax Tree	577
	SQL Parser Engine	578
12.4.8	Route Engine	582
	Sharding Route	582
	Broadcast Route	584

12.4.9	Rewrite Engine	586
	Rewriting for Correctness	586
	Identifier Rewriting	586
	Column Derivation	588
	Pagination Correction	590
	Batch Split	591
	Rewriting for Optimization	592
12.4.10	Execute Engine	593
	Connection Mode	593
	Automatic Execution Engine	595
12.4.11	Merger Engine	599
	Traversal Merger	599
	Order-by Merger	599
	Group-by Merger	601
	Aggregation Merger	604
	Pagination Merger	604
12.5	Transaction	605
12.5.1	Navigation	605
12.5.2	XA Transaction	605
	Transaction Begin	606
	Execute actual sharding SQL	606
	Commit or Rollback	607
12.5.3	Seata BASE transaction	607
	Init Seata Engine	608
	Transaction Begin	608
	Execute actual sharding SQL	608
	Commit or Rollback	609
12.6	Data Migration	609
12.6.1	Explanation	609
12.6.2	Execution Stage Explained	610
	Preparation	610
	Stock data migration	610
	The Synchronization of incremental data	610
	Traffic Switching	610
12.6.3	References	611
12.7	Encryption	611
12.7.1	Overall Architecture	611
12.7.2	Encryption Rules	612
12.7.3	Encryption Process	613
	Detailed Solution	614
	The advantages of Middleware encryption service	616
	Solution	616
12.7.4	EncryptAlgorithm	616
12.8	Mask	617
12.8.1	Overall Architecture	617

12.8.2	Mask Rules	617
12.8.3	Mask Process	619
12.9	Shadow	619
12.9.1	How it works	619
	DML sentence	620
	DDL sentence	621
12.9.2	References	621
12.10	Oberservability	621
12.10.1	How it works	621
12.11	Architecture	622
13	FAQ	624
13.1	JDBC	624
13.1.1	Found a JtaTransactionManager in spring boot project when integrating with XAtransaction.	624
13.1.2	The tableName and columnName configured in yaml or properties leading in-correct result when loading Oracle metadata?	624
13.1.3	SQLException: Unable to unwrap to interface com.mysql.jdbc. Connection exception thrown when using MySQL XA transaction	625
13.2	Proxy	626
13.2.1	In Windows environment, could not find or load main class org.apache.shardingsphere.proxy.Bootstrap, how to solve it?	626
13.2.2	How to add a new logic database dynamically when use ShardingSphere-Proxy?	626
13.2.3	How to use suitable database tools connecting ShardingSphere-Proxy?	626
13.2.4	When using a client to connect to ShardingSphere-Proxy, if ShardingSphere-Proxy does not create a database or does not register a storage unit, the client connection will fail?	627
13.3	Sharding	627
13.3.1	How to solve Cloud not resolve placeholder 'in string value ' error?	627
13.3.2	Why does float number appear in the return result of inline expression?	627
13.3.3	If sharding database is partial, should tables without sharding database & table configured in sharding rules?	627
13.3.4	When generic Long type SingleKeyTableShardingAlgorithm is used, why does the ClassCastException: Integer can not cast to Long exception appear?	628
13.3.5	[Sharding:raw-latex:PROXY] When implementing the StandardShardingAlgorithm custom algorithm, the specific type of Comparable is specified as Long, and the field type in the database table is bigint, a ClassCastException: Integer can not cast to Long exception occurs.	628
13.3.6	Why is the default distributed auto-augment key strategy provided by ShardingSphere not continuous and most of them end with even numbers?	628
13.3.7	How to allow range query with using inline sharding strategy (BETWEEN AND, >, <, >=, <=)?	628

13.3.8	Why does my custom distributed primary key do not work after implementing KeyGenerateAlgorithm interface and configuring type property?	629
13.3.9	In addition to internal distributed primary key, does ShardingSphere support other native auto-increment keys?	629
13.4	Single table	629
13.4.1	Table or view %s does not exist. How to solve the exception?	629
13.5	DistSQL	630
13.5.1	How to set custom JDBC connection properties or connection pool properties when adding a data source using DistSQL?	630
13.5.2	How to solve Storage unit [xxx] is still used by [SingleRule]. exception when dropping a data source using DistSQL?	631
13.5.3	How to solve Failed to get driver instance for jdbcURL=xxx. exception when adding a data source using DistSQL?	631
13.6	Other	631
13.6.1	How to debug when SQL can not be executed rightly in ShardingSphere?	631
13.6.2	Why do some compiling errors appear? Why did not the IDEA index the generated codes?	631
13.6.3	In SQLSever and PostgreSQL, why does the aggregation column without alias throw exception?	632
13.6.4	Why does Oracle database throw “Order by value must implements Comparable” exception when using Timestamp Order By?	632
13.6.5	In Windows environment,when cloning ShardingSphere source code through Git, why prompt filename too long and how to solve it?	633
13.6.6	How to solve Type is required error?	634
13.6.7	How to speed up the metadata loading when service starts up?	634
13.6.8	The ANTLR plugin generates codes in the same level directory as src, which is easy to commit by mistake. How to avoid it?	634
13.6.9	Why is the database sharding result not correct when using Proxool?	635
14	Downloads	636
14.1	Latest Releases	636
14.1.1	Apache ShardingSphere - Version: 5.5.2 (Release Date: January 16th, 2025) . .	636
14.2	All Releases	636
14.3	Verify the Releases	636

What is ShardingSphere

1.1 Introduction

Apache ShardingSphere is an ecosystem to transform any database into a distributed database system, and enhance it with sharding, elastic scaling, encryption features & more.

The project is committed to providing a multi-source heterogeneous, enhanced database platform and further building an ecosystem around the upper layer of the platform. Database Plus, the design philosophy of Apache ShardingSphere, aims at building the standard and ecosystem on the upper layer of the heterogeneous database. It focuses on how to make full and reasonable use of the computing and storage capabilities of existing databases rather than creating a brand new database. It attaches greater importance to the collaboration between multiple databases instead of the database itself.

1.1.1 ShardingSphere-JDBC

ShardingSphere-JDBC is a lightweight Java framework that provides additional services at Java's JDBC layer.

1.1.2 ShardingSphere-Proxy

ShardingSphere-Proxy is a transparent database proxy, providing a database server that encapsulates database binary protocol to support heterogeneous languages.

1.2 Product Features

Feature	Definition
Data Sharding	Data sharding is an effective way to deal with massive data storage and computing. ShardingSphere provides a distributed database solution based on the underlying database, which can scale computing and storage horizontally.
Distributed Transaction	Transactional capability is key to ensuring database integrity and security and is also one of the databases' core technologies. With a hybrid engine based on XA and BASE transactions, ShardingSphere provides distributed transaction capabilities on top of standalone databases, enabling data security across underlying data sources.
Read/Write Splitting	Read/write splitting can be used to cope with business access with high stress. ShardingSphere provides flexible read/write splitting capabilities and can achieve read access load balancing based on the understanding of SQL semantics and the ability to perceive the underlying database topology.
Data Migration	Data migration is the key to connecting data ecosystems. ShardingSphere provides migration capabilities to help users migrate the data from other data sources, while simultaneously performing data sharding.
Federated Query	Federated queries are effective in utilizing data in a complex data environment. ShardingSphere provides complex data query and analysis capabilities across data sources, simplifying the data aggregation from different data locations.
Data Encryption	Data Encryption is a basic way to ensure data security. ShardingSphere provides a complete, transparent, secure, and low-cost data encryption solution.
Shadow Database	In full-link online load testing scenarios, ShardingSphere supports data isolation in complex load testing scenarios through the shadow database function. Execute your load testing scenarios in a production environment without worrying about test data polluting your production data.

1.3 Advantages

- Ultimate Performance

Having been polished for years, the driver is close to a native JDBC in terms of efficiency, with ultimate performance.

- Ecosystem Compatibility

The proxy can be accessed by any application using MySQL/PostgreSQL protocol, and the driver can connect to any database that implements JDBC specifications.

- Zero Business Intrusion

In response to database switchover scenarios, ShardingSphere can achieve smooth business migration without business intrusion.

- Low Ops & Maintenance Cost

ShardingSphere offers a flat learning curve to DBAs and is interaction-friendly while allowing the original technology stack to remain unchanged.

- Security & Stability

It can provide enhancement capability based on mature databases while ensuring security and stability.

- Elastic Extension

It supports computing, storage, and smooth online expansion, which can meet diverse business needs.

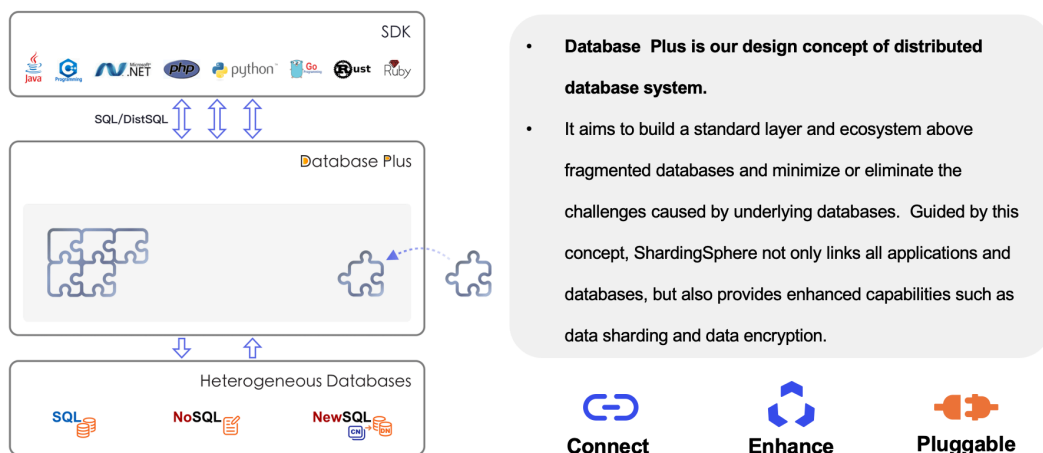
- Open Ecosystem

It can provide users with flexibility thanks to custom systems based on multi-level (kernel, feature, and ecosystem) plugin capabilities.

Design Philosophy

ShardingSphere adopts the database plus design philosophy, which is committed to building the standards and ecology of the upper layer of the database and supplementing the missing capabilities of the database in the ecology.

Design Philosophy: Database Plus



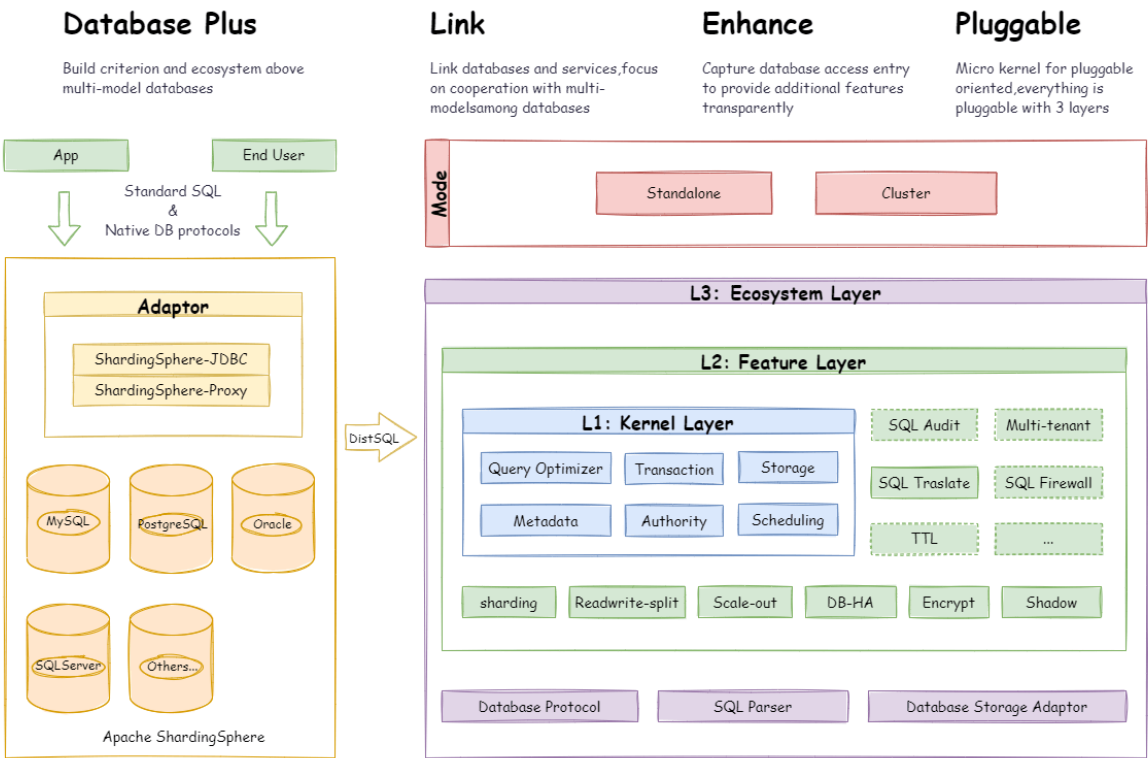
2.1 Connect: Create database upper level standard

Through flexible adaptation of database protocols, SQL dialects, and database storage, it can quickly build standards on top of multi-modal heterogeneous databases, while providing standardized connection mode for applications through built-in DistSQL.

2.2 Enhance: Database computing enhancement engine

It can further provide distributed capabilities and traffic enhancement functions based on native database capabilities. The former can break through the bottleneck of the underlying database in computing and storage, while the latter provides more diversified data application enhancement capabilities through traffic deformation, redirection, governance, authentication, and analysis.

2.3 Pluggable: Building database function ecology



The pluggable architecture of Apache ShardingSphere is composed of three layers - L1 Kernel Layer, L2 Feature Layer and L3 Ecosystem Layer.

2.3.1 L1 Kernel Layer

An abstraction of databases' basic capabilities. All the components are required and the specific implementation method can be replaced thanks to plugins. It includes a query optimizer, distributed transaction engine, distributed execution engine, permission engine and scheduling engine.

2.3.2 L2 Feature Layer

Used to provide enhancement capabilities. All components are optional, allowing you to choose whether to include zero or multiple components. Components are isolated from each other, and multiple components can be used together by overlaying. It includes data sharding, read/write splitting, data encryption and shadow database and so on. The user-defined feature can be fully customized and extended for the top-level interface defined by Apache ShardingSphere without changing kernel codes.

2.3.3 L3 Ecosystem Layer

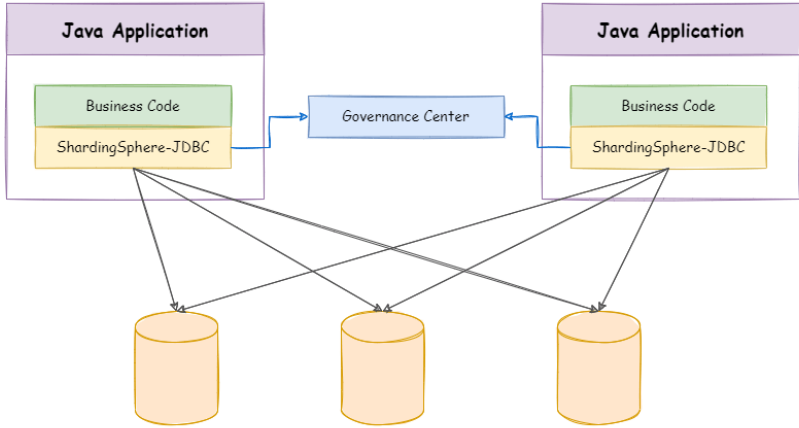
It is used to integrate and merge the current database ecosystems. The ecosystem layer includes database protocol, SQL parser and storage adapter, corresponding to the way in which Apache ShardingSphere provides services by database protocol, the way in which SQL dialect operates data, and the database type that interacts with storage nodes.

Apache ShardingSphere includes two independent clients: ShardingSphere-JDBC & ShardingSphere-Proxy. They all provide functions of data scale-out, distributed transaction and distributed governance, applicable in a variety of scenarios such as Java isomorphism, heterogeneous languages, and a cloud-native environment.

3.1 Using ShardingSphere-JDBC

ShardingSphere-JDBC is a lightweight Java framework that provides additional services at Java's JDBC layer. With the client connecting directly to the database, it provides services in the form of jar and requires no extra deployment and dependence. It can be considered as an enhanced version of the JDBC driver, which is fully compatible with JDBC and all kinds of ORM frameworks.

- Applicable in any ORM framework based on JDBC, such as JPA, Hibernate, Mybatis, Spring JDBC Template, or direct use of JDBC;
- Support any third-party database connection pool, such as DBCP, C3P0, BoneCP, HikariCP;
- Support any kind of JDBC standard database: MySQL, PostgreSQL, Oracle, SQLServer and any JDBC adapted databases.

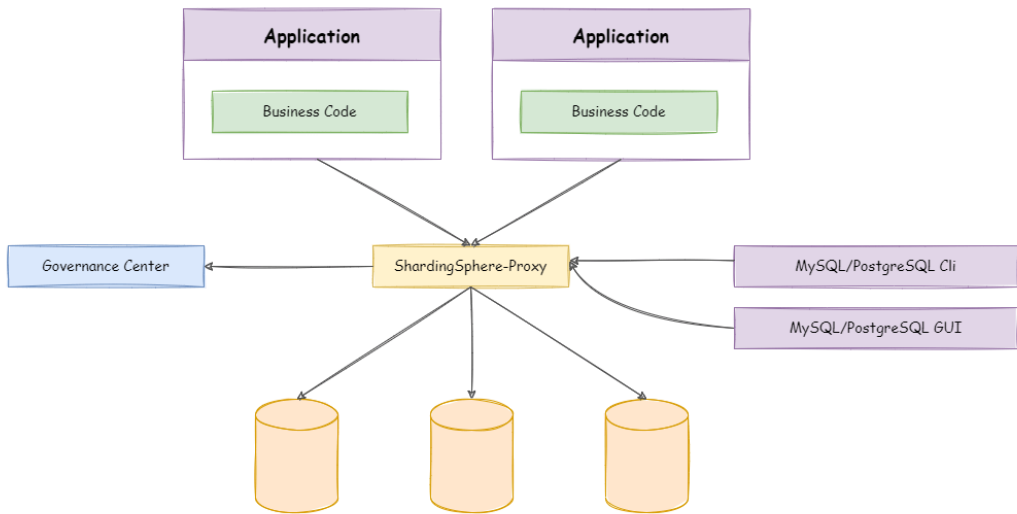


	ShardingSphere-JDBC	ShardingSphere-Proxy
Database	Any	MySQL/PostgreSQL
Connections Count Cost	More	Less
Heterogeneous language	Java Only	Any
Performance	Low loss	Relatively High loss
Decentralization	Yes	No
Static entry	No	Yes

3.2 Using ShardingSphere-Proxy

ShardingSphere-Proxy is a transparent database proxy, providing a database server that encapsulates database binary protocol to support heterogeneous languages. Currently, MySQL and PostgreSQL protocols are provided. It can use any kind of terminal that is compatible with MySQL or PostgreSQL protocol to operate data, which is more friendly to DBAs.

- Transparent to applications, it can be used directly as MySQL/PostgreSQL;
- Compatible with MySQL-based databases, such as MariaDB, and PostgreSQL-based databases, such as openGauss;
- Applicable to any kind of client that is compatible with MySQL/PostgreSQL protocol, such as MySQL Command Client, MySQL Workbench, etc.

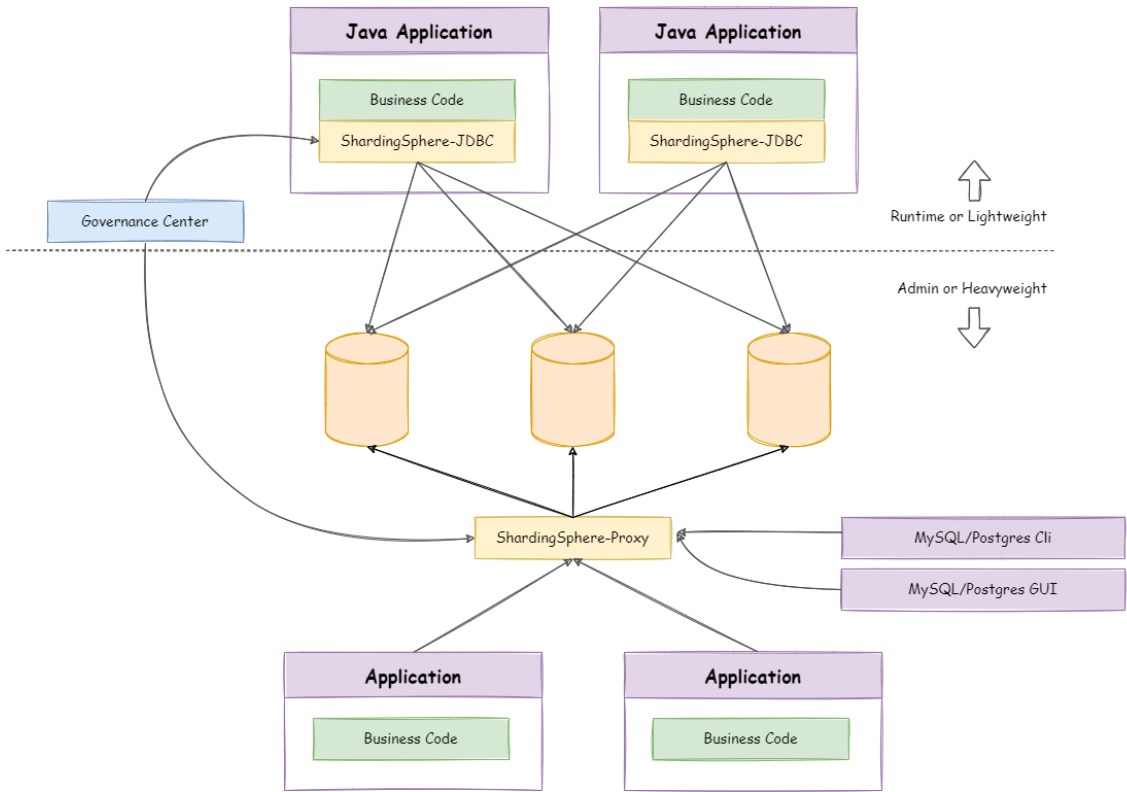


	ShardingSphere-JDBC	ShardingSphere-Proxy
Database	Any	MySQL/PostgreSQL
Connections Count Cost	More	Less
Heterogeneous language	Java Only	Any
Performance	Low loss	Relatively High loss
Decentralization	Yes	No
Static entry	No	Yes

3.3 Hybrid Architecture

ShardingSphere-JDBC adopts a decentralized architecture, applicable to high-performance light-weight OLTP applications developed with Java. ShardingSphere-Proxy provides static entry and supports all languages, applicable to OLAP applications and the sharding databases management and operation situation.

Apache ShardingSphere is an ecosystem composed of multiple access ports. By combining ShardingSphere-JDBC and ShardingSphere-Proxy, and using the same registry to configure sharding strategies, it can flexibly build application systems for various scenarios, allowing architects to freely adjust the system architecture according to the current businesses.



Apache ShardingSphere provides two running modes: standalone mode and cluster mode.

4.1 Standalone Mode

It can achieve data persistence in terms of metadata information such as data sources and rules, but it is not able to synchronize metadata to multiple Apache ShardingSphere instances or be aware of each other in a cluster environment. Updating metadata through one instance causes inconsistencies in other instances because they cannot get the latest metadata.

It is ideal for engineers to build a ShardingSphere environment locally.

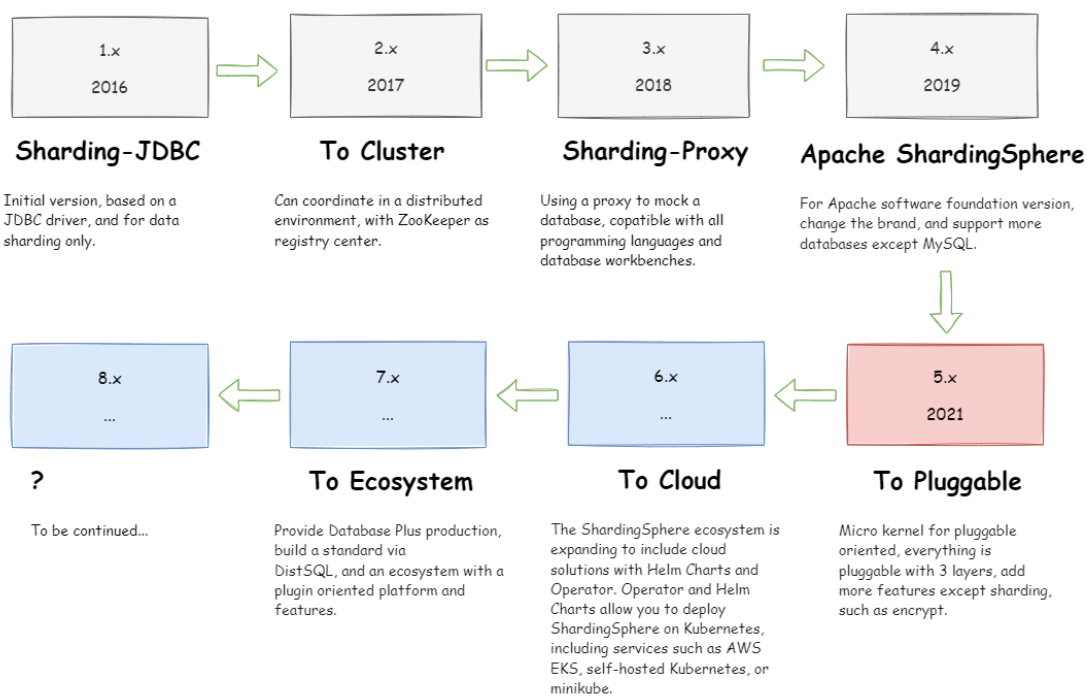
4.2 Cluster Mode

It provides metadata sharing between multiple Apache ShardingSphere instances and the capability to coordinate states in distributed scenarios.

It provides the capabilities necessary for distributed systems, such as horizontal scaling of computing capability and high availability. Clustered environments need to store metadata and coordinate nodes' status through a separately deployed registry center.

We suggest using cluster mode in production environment.

Roadmap



ShardingSphere became an [Apache](#) Top-Level Project on April 16, 2020. You are welcome to check out the mailing list and discuss via [mail](#).

In shortest time, this chapter provides users with a simplest quick start with Apache ShardingSphere.

Example Codes: <https://github.com/apache/shardingsphere/tree/master/examples>

7.1 ShardingSphere-JDBC

7.1.1 Scenarios

There are two ways you can configure Apache ShardingSphere: Java and YAML. Developers can choose the preferred method according to their requirements.

7.1.2 Limitations

Currently only Java language is supported.

7.1.3 Requirements

The development environment requires Java JRE 8 or later.

7.1.4 Procedure

1. Rules configuration.

Please refer to [User Manual](#) for more details.

2. Import Maven dependency

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${latest.release.version}</version>
</dependency>
```

Notice: Please change `${latest.release.version}` to the actual version.

3. Create YAML configuration file

```
# JDBC database name. In cluster mode, use this parameter to connect ShardingSphere-
JDBC and ShardingSphere-Proxy.
# Default: logic_db
databaseName (?):

mode:

dataSources:

rules:
- !FOO_XXX
  ...
- !BAR_XXX
  ...

props:
  key_1: value_1
  key_2: value_2
```

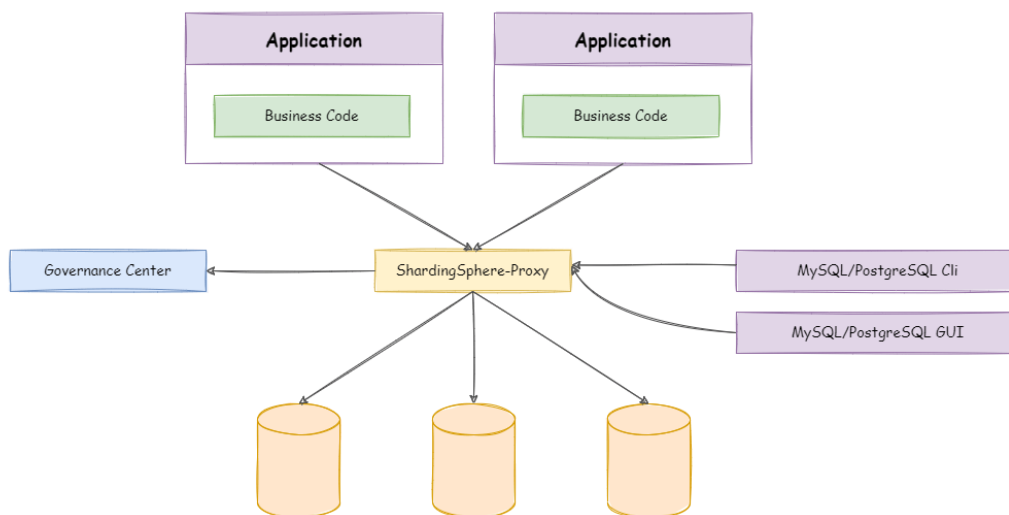
4. Take spring boot as an example, edit application.properties.

```
# Configuring DataSource Drivers
spring.datasource.driver-class-name=org.apache.shardingsphere.driver.
ShardingSphereDriver
# Specify a YAML configuration file
spring.datasource.url=jdbc:shardingsphere:classpath:xxx.yaml
```

For details, see [Spring Boot](#).

7.2 ShardingSphere-Proxy

7.2.1 Scenarios



ShardingSphere-Proxy is positioned as a transparent database proxy. It theoretically supports any client operation data using MySQL, PostgreSQL and openGauss protocols, and is friendly to heterogeneous languages and operation and maintenance scenarios.

7.2.2 Limitations

Proxy provides limited support for system databases / tables (such as `information_schema`, `pg_catalog`). When connecting to Proxy through some graph database clients, the client or proxy may have an error prompt. You can use command-line clients (`mysql`, `psql`, `gsq`, etc.) to connect to the Proxy's authentication function.

7.2.3 Requirements

Starting ShardingSphere-Proxy with Docker requires no additional dependency. To start the Proxy using binary distribution, the environment must have Java JRE 8 or higher.

7.2.4 Procedure

1. Get ShardingSphere-Proxy.

ShardingSphere-Proxy is available at: - [Binary Distribution](#) - [Docker](#) - [Helm](#)

2. Rule configuration.

Edit %SHARDINGSPHERE_PROXY_HOME%/conf/global.yaml.

Edit %SHARDINGSPHERE_PROXY_HOME%/conf/database-xxx.yaml.

%SHARDINGSPHERE_PROXY_HOME% is the proxy extract path. for example: /opt/shardingsphere-proxy-bin/

Please refer to [Configuration Manual](#) for more details.

3. Import dependencies.

If the backend database is PostgreSQL or openGauss, no additional dependencies are required.

If the backend database is MySQL, please download [mysql-connector-java-5.1.49.jar](#) or [mysql-connector-java-8.0.11.jar](#) and put it into the %SHARDINGSPHERE_PROXY_HOME%/ext-lib directory.

4. Start server.

- Use the default configuration to start

```
sh %SHARDINGSPHERE_PROXY_HOME%/bin/start.sh
```

The default port is 3307, while the default profile directory is %SHARDINGSPHERE_PROXY_HOME%/conf/.

- Customize port and profile directory

```
sh %SHARDINGSPHERE_PROXY_HOME%/bin/start.sh ${proxy_port} ${proxy_conf_directory}
```

- Force start

```
sh %SHARDINGSPHERE_PROXY_HOME%/bin/start.sh -f
```

Use the -f parameter to force start the Proxy. This parameter will ignore the abnormal data source during startup and start the Proxy forcibly. After the Proxy is started, you can remove the abnormal data source by DistSQL.

5. Use ShardingSphere-Proxy.

Use MySQL or PostgreSQL or openGauss client to connect ShardingSphere-Proxy.

Use the MySQL client to connect to the ShardingSphere-Proxy:

```
mysql -h${proxy_host} -P${proxy_port} -u${proxy_username} -p${proxy_password}
```

Use the PostgreSQL client to connect to the ShardingSphere-Proxy:

```
psql -h ${proxy_host} -p ${proxy_port} -U ${proxy_username}
```

Use the openGauss client to connect to the ShardingSphere-Proxy:

```
gsql -r -h ${proxy_host} -p ${proxy_port} -U ${proxy_username} -W ${proxy_password}
```


Apache ShardingSphere provides a variety of features, from database kernel and database distributed solution to applications closed features.

There is no boundary for these features, warmly welcome more open source engineers to join the community and provide exciting ideas and features.

8.1 Sharding

8.1.1 Background

The traditional solution that stores all the data in one concentrated node has hardly satisfied the requirement of massive data scenario in three aspects, performance, availability and operation cost.

In performance, the relational database mostly uses B+ tree index. When the data amount exceeds the threshold, deeper index will increase the disk IO access number, and thereby, weaken the performance of query. In the same time, high concurrency requests also make the centralized database to be the greatest limitation of the system.

In availability, capacity can be expanded at a relatively low cost and any extent with stateless service, which can make all the pressure, at last, fall on the database. But the single data node or simple primary-replica structure has been harder and harder to take these pressures. Therefore, database availability has become the key to the whole system.

From the aspect of operation costs, when the data in a database instance has reached above the threshold, DBA's operation pressure will also increase. The time cost of data backup and data recovery will be more uncontrollable with increasing amount of data. Generally, it is a relatively reasonable range for the data in single database case to be within 1TB.

Under the circumstance that traditional relational databases cannot satisfy the requirement of the Internet, there are more and more attempts to store the data in native distributed NoSQL. But its incompatibility with SQL and imperfection in ecosystem block it from defeating the relational database in the competition, so the relational database still holds an unshakable position.

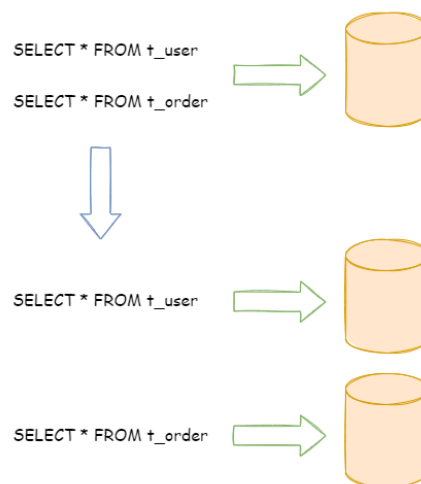
Sharding refers to splitting the data in one database and storing them in multiple tables and databases

according to some certain standard, so that the performance and availability can be improved. Both methods can effectively avoid the query limitation caused by data exceeding affordable threshold. What's more, database sharding can also effectively disperse TPS. Table sharding, though cannot ease the database pressure, can provide possibilities to transfer distributed transactions to local transactions, since cross-database upgrades are once involved, distributed transactions can turn pretty tricky sometimes. The use of multiple primary-replica sharding method can effectively avoid the data concentrating on one node and increase the architecture availability.

Splitting data through database sharding and table sharding is an effective method to deal with high TPS and mass amount data system, because it can keep the data amount lower than the threshold and evacuate the traffic. Sharding method can be divided into vertical sharding and horizontal sharding.

Vertical Sharding

According to business sharding method, it is called vertical sharding, or longitudinal sharding, the core concept of which is to specialize databases for different uses. Before sharding, a database consists of many tables corresponding to different businesses. But after sharding, tables are categorized into different databases according to business, and the pressure is also separated into different databases. The diagram below has presented the solution to assign user tables and order tables to different databases by vertical sharding according to business need.

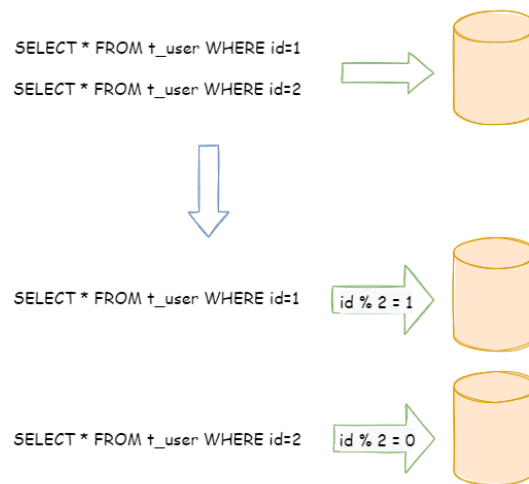


Vertical sharding requires to adjust the architecture and design from time to time. Generally speaking, it is not soon enough to deal with fast changing needs from Internet business and not able to really solve the single-node problem. it can ease problems brought by the high data amount and concurrency

amount, but cannot solve them completely. After vertical sharding, if the data amount in the table still exceeds the single node threshold, it should be further processed by horizontal sharding.

Horizontal Sharding

Horizontal sharding is also called transverse sharding. Compared with the categorization method according to business logic of vertical sharding, horizontal sharding categorizes data to multiple databases or tables according to some certain rules through certain fields, with each sharding containing only part of the data. For example, according to primary key sharding, even primary keys are put into the 0 database (or table) and odd primary keys are put into the 1 database (or table), which is illustrated as the following diagram.



Theoretically, horizontal sharding has overcome the limitation of data processing volume in single machine and can be extended relatively freely, so it can be taken as a standard solution to database sharding and table sharding.

8.1.2 Challenges

Although data sharding solves problems regarding performance, availability, and backup recovery of single points, the distributed architecture has introduced new problems while gaining benefits.

One of the major challenges is that application development engineers and database administrators become extremely overwhelmed with all these operations after such a scattered way of data sharding. They need to know from which specific sub-table can they fetch the data needed.

Another challenge is that SQL that works correctly in one single-node database does not necessarily work correctly in a sharded database. For example, table splitting results in table name changes, or incorrect handling of operations such as paging, sorting, and aggregate grouping.

Cross-library transactions are also tricky for a distributed database cluster. Reasonable use of table splitting can minimize the use of local transactions while reducing the amount of data in a single table, and appropriate use of different tables in the same database can effectively avoid the trouble caused by distributed transactions. In scenarios where cross-library transactions cannot be avoided, some businesses might still be in the need to maintain transaction consistency. The XA-based distributed transactions are not used by Internet giants on a large scale because their performance cannot meet the needs in scenarios with high concurrency, and most of them use flexible transactions with ultimate consistency instead of strong consistent transactions.

8.1.3 Goal

The main design goal of the data sharding modular of Apache ShardingSphere is to try to reduce the influence of sharding, in order to let users use horizontal sharding database group like one database.

8.1.4 Application Scenarios

Mass data high concurrency in OLTP scenarios

Most relational databases use B+ tree indexes, but when the amount of data exceeds the threshold, the increase in index depth will also increase the number of I/O in accessing the disk, which will lower the query performance. Data sharding through ShardingSphere enables data stored in a single database to be dispersed into multiple databases or tables according to a business dimension, which improves performance. The ShardingSphere-JDBC access port can meet the performance requirements of high concurrency in OLTP scenarios.

Mass data real-time analysis in OLAP scenarios

In traditional database architecture, if users want to analyze data, they need to use ETL tools first, synchronize the data to the data platform, and then perform data analysis. However, ETL tools will greatly reduce the effectiveness of data analysis. ShardingSphere-Proxy provides support for static entry and heterogeneous languages, independent of application deployment, which is suitable for real-time analysis in OLAP scenarios.

8.1.5 Related References

- User Guide: [sharding](#)
- Developer Guide: [sharding](#)

8.1.6 Core Concept

Table

Tables are a key concept for transparent data sharding. Apache ShardingSphere adapts to the data sharding requirements under different scenarios by providing diverse table types.

Logic Table

The logical name of the horizontally sharded database (table) of the same structure is the logical identifier of the table in SQL. Example: Order data is split into 10 tables according to the primary key endings, are `t_order_0` to `t_order_9`, and their logical table names are `t_order`.

Actual Table

Physical tables that exist in the horizontally sharded databases. Those are, `t_order_0` to `t_order_9` in the previous example.

Binding Table

Refers to a set of sharded tables with consistent sharding rules. When using binding tables for multi-table associated query, a sharding key must be used for the association, otherwise, Cartesian product association or cross-library association will occur, affecting query efficiency.

For example, if the `t_order` table and `t_order_item` table are both sharded according to `order_id` and are correlated using `order_id`, the two tables are binding tables. The multi-table associated queries between binding tables will not have a Cartesian product association, so the associated queries will be much more effective. Here is an example,

If SQL is:

```
SELECT i.* FROM t_order o JOIN t_order_item i ON o.order_id=i.order_id WHERE o.
order_id in (10, 11);
```

In the case where no binding table relationships are being set, assume that the sharding key `order_id` routes the value 10 to slice 0 and the value 11 to slice 1, then the routed SQL should be 4 items, which are presented as a Cartesian product:

```
SELECT i.* FROM t_order_0 o JOIN t_order_item_0 i ON o.order_id=i.order_id WHERE o.
order_id in (10, 11);
```

```
SELECT i.* FROM t_order_0 o JOIN t_order_item_1 i ON o.order_id=i.order_id WHERE o.
order_id in (10, 11);
```

```
SELECT i.* FROM t_order_1 o JOIN t_order_item_0 i ON o.order_id=i.order_id WHERE o.
order_id in (10, 11);
```

```
SELECT i.* FROM t_order_1 o JOIN t_order_item_1 i ON o.order_id=i.order_id WHERE o.
order_id in (10, 11);
```

After the relationships between binding tables are configured and associated with `order_id`, the routed SQL should then be 2 items:

```
SELECT i.* FROM t_order_0 o JOIN t_order_item_0 i ON o.order_id=i.order_id WHERE o.
order_id in (10, 11);
```

```
SELECT i.* FROM t_order_1 o JOIN t_order_item_1 i ON o.order_id=i.order_id WHERE o.
order_id in (10, 11);
```

The `t_order` table will be used by ShardingSphere as the master table for the entire binding table since it specifies the sharding condition. All routing calculations will use only the policy of the primary table, then the sharding calculations for the `t_order_item` table will use the `t_order` condition.

Note: multiple sharding rules in the binding table need to be configured according to the combination of logical table prefix and sharding suffix, for example:

```
rules:
- !SHARDING
  tables:
    t_order:
      actualDataNodes: ds_${0..1}.t_order_${0..1}
    t_order_item:
      actualDataNodes: ds_${0..1}.t_order_item_${0..1}
  bindingTables:
    - t_order, t_order_item
```

Broadcast data frame

Refers to tables that exist in all data sources. The table structure and its data are identical in each database. Suitable for scenarios where the data volume is small and queries are required to be associated with tables of massive data, e.g., dictionary tables.

Single Table

Refers to the only table that exists in all sharded data sources. Suitable for tables with a small amount of data and do not need to be sharded.

Note: Single tables that meet the following conditions will be automatically loaded: - A single table showing the configuration in rules such as encrypt and mask - A single table created by users executing DDL statements through ShardingSphere

For other single tables that do not meet the above conditions, ShardingSphere will not automatically load them, and users can configure single table rules as needed for management.

Data Nodes

The smallest unit of the data shard, consists of the data source name and the real table. Example: `ds_0.t_order_0`.

The mapping relationship between the logical table and the real table can be classified into two forms: uniform distribution and custom distribution.

Uniform Distribution

refers to situations where the data table exhibits a uniform distribution within each data source. For example:

```
db0
├─ t_order0
└─ t_order1
db1
├─ t_order0
└─ t_order1
```

The configuration of data nodes:

```
db0.t_order0, db0.t_order1, db1.t_order0, db1.t_order1
```

Customized Distribution

Data table exhibiting a patterned distribution. For example:

```
db0
├─ t_order0
└─ t_order1
db1
├─ t_order2
├─ t_order3
└─ t_order4
```

configuration of data nodes:

```
db0.t_order0, db0.t_order1, db1.t_order2, db1.t_order3, db1.t_order4
```

Sharding

Sharding key

A database field is used to split a database (table) horizontally. Example: If the order primary key in the order table is sharded by modulo, the order primary key is a sharded field. If there is no sharded field in SQL, full routing will be executed, of which performance is poor. In addition to the support for single-sharding fields, Apache ShardingSphere also supports sharding based on multiple fields.

Sharding Algorithm

Algorithm for sharding data, supporting =, >=, <=, >, <, BETWEEN and IN. The sharding algorithm can be implemented by the developers themselves or can use the Apache ShardingSphere built-in sharding algorithm, syntax sugar, which is very flexible.

Automatic Sharding Algorithm

Sharding algorithm—syntactic sugar is for conveniently hosting all data nodes without users having to concern themselves with the physical distribution of actual tables. Includes implementations of common sharding algorithms such as modulo, hash, range, and time.

Customized Sharding Algorithm

Provides a portal for application developers to implement their sharding algorithms that are closely related to their business operations, while allowing users to manage the physical distribution of actual tables themselves. Customized sharding algorithms are further divided into: - Standard Sharding Algorithm Used to deal with scenarios where sharding is performed using a single key as the sharding key =, IN, BETWEEN AND, >, <, >=, <=. - Composite Sharding Algorithm Used to cope with scenarios where multiple keys are used as sharding keys. The logic containing multiple sharding keys is very complicated and requires the application developers to handle it on their own. - Hint Sharding Algorithm For scenarios involving Hint sharding.

Sharding Strategy

Consisting of a sharding key and sharding algorithm, which is abstracted independently due to the independence of the sharding algorithm. What is viable for sharding operations is the sharding key + sharding algorithm, known as sharding strategy.

Mandatory Sharding routing

For the scenario where the sharded field is not determined by SQL but by other external conditions, you can use SQL Hint to inject the shard value. Example: Conduct database sharding by employee login primary key, but there is no such field in the database. SQL Hint can be used both via Java API and SQL annotation. See Mandatory Sharding Routing for details.

Row Value Expressions

Row expressions are designed to address the two main issues of configuration simplification and integration. In the cumbersome configuration rules of data sharding, the large number of repetitive configurations makes the configuration itself difficult to maintain as the number of data nodes increases. The data node configuration workload can be effectively simplified by row expressions.

For the common sharding algorithm, using Java code implementation does not help to manage the configuration uniformly. But by writing the sharding algorithm through line expressions, the rule configuration can be effectively stored together, which is easier to browse and store.

A Row Value Expressions consists of two parts as a string, the Type Name part of the corresponding SPI implementation at the beginning of the string and the expression part.

Take <GROOVY>t_order_\${1..3} as sample, the GROOVY substring in the part of the <GROOVY> string is the Type Name used by the corresponding SPI implementation for this Row Value Expressions, which is identified by the <> symbol. And the t_order_\${1..3} string is the expression part of this Row Value Expressions. When a Row Value Expressions does not specify a Type Name, such as t_order_\${1..3}, the Row Value Expressions defaults to parse expressions by GROOVY implementation for InlineExpressionParser SPI.

The following sections describe the syntax rules for the GROOVY implementation.

Row expressions are very intuitive, just use `${ expression }` or `$->{ expression }` in the configuration to identify the row expressions. Data nodes and sharding algorithms are currently supported. The content of row expressions uses Groovy syntax, and all operations supported by Groovy are supported by row expressions. For example:

`${begin..end}` denotes the range interval

`${[unit1, unit2, unit_x]}` denotes the enumeration value

If there are multiple `${ expression }` or `$->{ expression }` expressions in a row expression, the final result of the whole expression will be a Cartesian combination based on the result of each sub-expression.

e.g. The following row expression:

```
${['online', 'offline']}_table${1..3}
```

Finally, it can be parsed as this:

```
online_table1, online_table2, online_table3, offline_table1, offline_table2,
offline_table3
```

Distributed Primary Key

In traditional database software development, automatic primary key generation is a basic requirement. Various databases provide support for this requirement, such as self-incrementing keys of MySQL, self-incrementing sequences of Oracle, etc. After data sharding, it is very tricky to generate global unique primary keys for different data nodes. Self-incrementing keys between different actual tables within the same logical table generate repetitive primary keys because they are not mutually aware. Although collisions can be avoided by constraining the initial value and step size of self-incrementing primary keys, additional operational and maintenance rules are necessary to be introduced, rendering the solution lacking in completeness and scalability.

Many third-party solutions can perfectly solve this problem, such as UUID, which relies on specific algorithms to self-generate non-repeating keys, or by introducing primary key generation services. To facilitate users and meet their demands for different scenarios, Apache ShardingSphere not only provides built-in distributed primary key generators, such as UUID and SNOWFLAKE but also abstracts the interface of distributed primary key generators to enable users to implement their own customized self-extending primary key generators.

8.1.7 Limitations

Compatible with all commonly used SQL that routes to single data nodes; SQL routing to multiple data nodes is divided, because of complexity issues, into three conditions: stable support, experimental support, and no support.

Stable Support

Full support for DML, DDL, DCL, TCL, and common DALs. Support for complex queries such as paging, de-duplication, sorting, grouping, aggregation, table association, etc. Support SCHEMA DDL and DML statements of PostgreSQL and openGauss database. When no schema is specified in SQL, default access to 'public' schema. Other schemas need to declare before the table name, and do not support 'SEARCH_PATH' to modify the schema search path.

Normal Queries

- main statement SELECT

```
SELECT select_expr [, select_expr ...] FROM table_reference [, table_reference ...]
[WHERE predicates]
[GROUP BY {col_name | position} [ASC | DESC], ...]
[ORDER BY {col_name | position} [ASC | DESC], ...]
[LIMIT [{offset,} row_count | row_count OFFSET offset]]
```

- select_expr

```
* |
[DISTINCT] COLUMN_NAME [AS] [alias] |
(MAX | MIN | SUM | AVG)(COLUMN_NAME | alias) [AS] [alias] |
COUNT(* | COLUMN_NAME | alias) [AS] [alias]
```

- table_reference

```
tbl_name [AS] alias [index_hint_list]
| table_reference ([INNER] | {LEFT|RIGHT} [OUTER]) JOIN table_factor [JOIN ON
conditional_expr | USING (column_list)]
```

Sub-query

Stable support is provided by the kernel when both the subquery and the outer query specify a shard key and the values of the slice key remain consistent. e.g:

```
SELECT * FROM (SELECT * FROM t_order WHERE order_id = 1) o WHERE o.order_id = 1;
```

Sub-query for [pagination](#) can be stably supported by the kernel. e.g.:

```
SELECT * FROM (SELECT row_.*, rownum rownum_ FROM (SELECT * FROM t_order) row_ WHERE rownum <= ?) WHERE rownum > ?;
```

Pagination Query

MySQL, PostgreSQL, and openGauss are fully supported, Oracle and SQLServer are only partially supported due to more intricate paging queries.

Pagination for Oracle and SQLServer needs to be handled by subqueries, and ShardingSphere supports paging-related subqueries.

- Oracle Support pagination by rownum

```
SELECT * FROM (SELECT row_.*, rownum rownum_ FROM (SELECT o.order_id as order_id FROM t_order o JOIN t_order_item i ON o.order_id = i.order_id) row_ WHERE rownum <= ?) WHERE rownum > ?
```

- SQL Server Support pagination that coordinates TOP + ROW_NUMBER() OVER

```
SELECT * FROM (SELECT TOP (?) ROW_NUMBER() OVER (ORDER BY o.order_id DESC) AS rownum, * FROM t_order o) AS temp WHERE temp.rownum > ? ORDER BY temp.order_id
```

Support pagination by OFFSET FETCH after SQLServer 2012

```
SELECT * FROM t_order o ORDER BY id OFFSET ? ROW FETCH NEXT ? ROWS ONLY
```

- MySQL, PostgreSQL and openGauss all support LIMIT pagination without the need for sub-query:

```
SELECT * FROM t_order o ORDER BY id LIMIT ? OFFSET ?
```

Aggregation

Support MAX, MIN, SUM, COUNT, AVG, BIT_XOR, GROUP_CONCAT and so on.

Shard keys included in operation expressions

When the sharding key is contained in an expression, the value used for sharding cannot be extracted through the SQL letters and will result in full routing.

For example, assume create_time is a sharding key.

```
SELECT * FROM t_order WHERE to_date(create_time, 'yyyy-mm-dd') = '2019-01-01';
```

LOAD DATA / LOAD XML

Support MySQL LOAD DATA and LOAD XML statements to load data to single table and broadcast table.

View

1. Support create, alter and drop view based on a single table or multiple single tables on the same storage node;
2. Support create, alter and drop view based on any broadcast table;
3. Support create, alter and drop view based on any sharding table. The view must be configured with same sharding rules as sharding table, the view and sharding table must be in same binding table rule;
4. Support create, alter and drop view based on broadcast tables and sharding tables. The sharding table rules are same as create view using sharding tables alone;
5. Support create, alter and drop view based on broadcast tables and single tables;
6. Support MySQL SHOW CREATE TABLE viewName to show create statement of the view.

Experimental Support

Experimental support refers specifically to support provided by implementing Federation execution engine, an experimental product that is still under development. Although largely available to users, it still requires significant optimization.

Sub-query

The Federation execution engine provides support for subqueries and outer queries that do not both specify a sharding key or have inconsistent values for the sharding key.

e.g:

```
SELECT * FROM (SELECT * FROM t_order) o;

SELECT * FROM (SELECT * FROM t_order) o WHERE o.order_id = 1;

SELECT * FROM (SELECT * FROM t_order WHERE order_id = 1) o;

SELECT * FROM (SELECT * FROM t_order WHERE order_id = 1) o WHERE o.order_id = 2;
```

Cross-database Associated query

When multiple tables in an associated query are distributed across different database instances, the Federation execution engine can provide support. Assuming that `t_order` and `t_order_item` are sharded tables with multiple data nodes while no binding table rules are configured, and `t_user` and `t_user_role` are single tables distributed across different database instances, then the Federation execution engine can support the following common associated queries.

```
SELECT * FROM t_order o INNER JOIN t_order_item i ON o.order_id = i.order_id WHERE o.order_id = 1;

SELECT * FROM t_order o INNER JOIN t_user u ON o.user_id = u.user_id WHERE o.user_id = 1;

SELECT * FROM t_order o LEFT JOIN t_user_role r ON o.user_id = r.user_id WHERE o.user_id = 1;

SELECT * FROM t_order_item i LEFT JOIN t_user u ON i.user_id = u.user_id WHERE i.user_id = 1;

SELECT * FROM t_order_item i RIGHT JOIN t_user_role r ON i.user_id = r.user_id WHERE i.user_id = 1;

SELECT * FROM t_user u RIGHT JOIN t_user_role r ON u.user_id = r.user_id WHERE u.user_id = 1;
```

Do not Support

CASE WHEN

The following CASE WHEN statements are not supported: - CASE WHEN contains sub-query - Logic names are used in CASE WHEN(Please use an alias)

Pagination Query

Due to the complexity of paging queries, there are currently some paging queries that are not supported for Oracle and SQLServer, such as: - Oracle The paging method of `rownum + BETWEEN` is not supported at present

- SQLServer Currently, pagination with `WITH xxx AS (SELECT ...)` is not supported. Since the SQLServer paging statement automatically generated by Hibernate uses the `WITH` statement, Hibernate-based SQLServer paging is not supported at this moment. Pagination using two `TOP + subquery` also cannot be supported at this time.

Aggregation

When a query contains multiple aggregate functions at the same time, it does not support mixing aggregate functions with DISTINCT and aggregate functions without DISTINCT.

LOAD DATA / LOAD XML

Not support MySQL LOAD DATA and LOAD XML statements to load data to sharding table.

8.1.8 Appendix with SQL operator

Limited supported SQL:

- When using `getGeneratedKeys` interface of JDBC specification to return auto-increment key, it is necessary to use a distributed key generator that supports auto-increment, and does not support other types of distributed key generators

Unsupported SQL:

- CASE WHEN contains sub-query
- Logical table names are used in CASE WHEN(Please use an alias)
- INSERT INTO tbl_name (col1, col2, ...) SELECT * FROM tbl_name WHERE col3 = ? (The SELECT clause does not support * and the built-in distributed primary key generator)
- REPLACE INTO tbl_name (col1, col2, ...) SELECT * FROM tbl_name WHERE col3 = ? (The SELECT clause does not support * and the built-in distributed primary key generator)
- SELECT MAX(tbl_name.col1) FROM tbl_name (If the query column is a function expression, use the table alias instead of the table name)

Other:

- You should keep actual tables, sharding columns and key generate columns in sharding rule same capitalization with tables and columns in database.

8.2 Distributed Transaction

8.2.1 Background

Database transactions should satisfy the features of ACID (atomicity, consistency, isolation and durability).

- Atomicity: transactions are executed as a whole, and either all or none is executed.
- Consistency: transactions should ensure that the state of data remains consistent after the transaction.

- Isolation: when multiple transactions execute concurrently, the execution of one transaction should not affect the execution of others.
- Durability: when a transaction committed modifies data, the operation will be saved persistently.

In single data node, transactions are only restricted to the access and control of single database resources, called local transactions. Almost all the mature relational databases have provided native support for local transactions. But in distributed application situations based on micro-services, more and more of them require to include multiple accesses to services and the corresponding database resources in the same transaction. As a result, distributed transactions appear.

Though the relational database has provided perfect native ACID support, it can become an obstacle to the system performance under distributed situations. How to make databases satisfy ACID features under distributed situations or find a corresponding substitute solution, is the priority work of distributed transactions.

8.2.2 Challenge

For different application situations, developers need to reasonably weight the performance and the function between all kinds of distributed transactions.

Highly consistent transactions do not have totally the same API and functions as soft transactions, and they cannot switch between each other freely and invisibly. The choice between highly consistent transactions and soft transactions as early as development decision-making phase has sharply increased the design and development cost.

Highly consistent transactions based on XA is relatively easy to use, but is not good at dealing with long transaction and high concurrency situation of the Internet. With a high access cost, soft transactions require developers to transform the application and realize resources lock and backward compensation.

8.2.3 Goal

The main design goal of the distributed transaction modular of Apache ShardingSphere is to integrate existing mature transaction cases to provide an unified distributed transaction interface for local transactions, 2PC transactions and soft transactions; compensate for the deficiencies of current solutions to provide a one-stop distributed transaction solution.

8.2.4 How it works

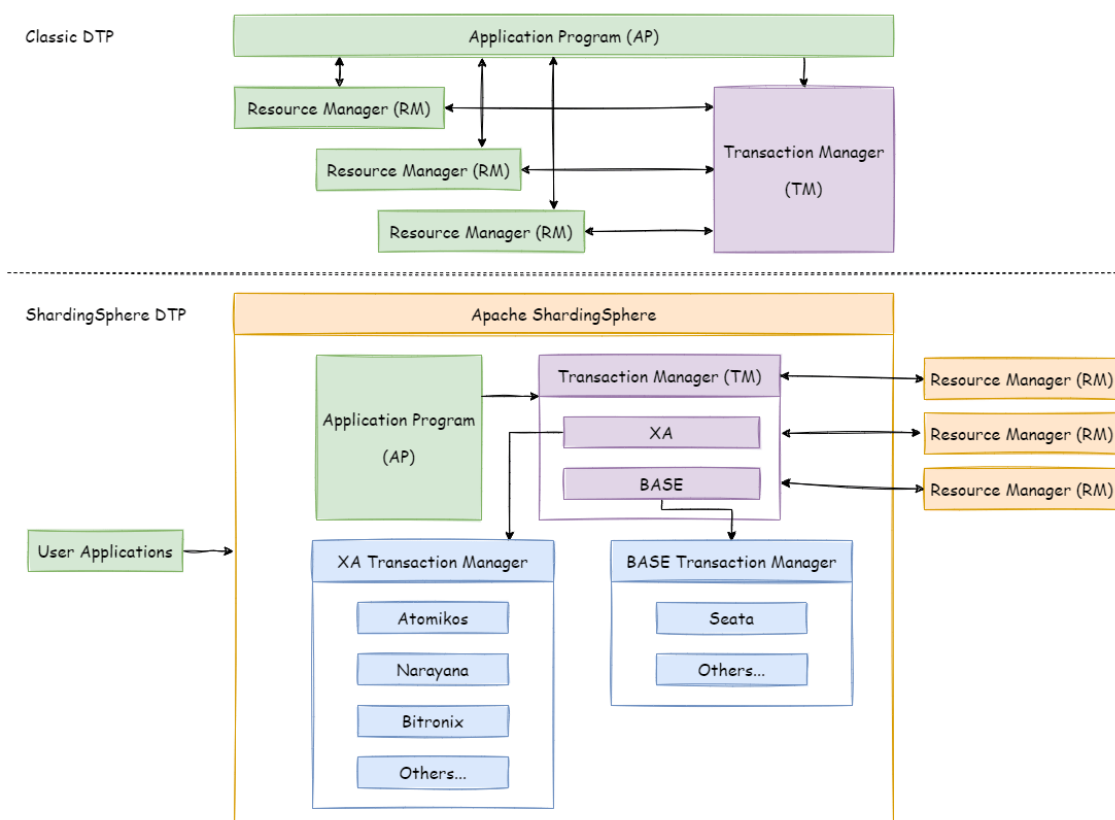
ShardingSphere provides begin/ commit/rollback traditional transaction interfaces externally, and provides distributed transaction capabilities through LOCAL, XA and BASE modes.

LOCAL Transaction

LOCAL mode is implemented based on ShardingSphere's proxy database interfaces, that is begin/commit/rollback. For a logical SQL, ShardingSphere starts transactions on each proxied database with the begin directive, executes the actual SQL, and performs commit/rollback. Since each data node manages its own transactions, there is no coordination and communication between them, and they do not know whether other data node transactions succeed or not. There is no loss in performance, but strong consistency and final consistency cannot be guaranteed.

XA Transaction

XA transaction adopts the concepts including AP(application program), TM(transaction manager) and RM(resource manager) to ensure the strong consistency of distributed transactions. Those concepts are abstracted from **DTP mode** which is defined by X/OPEN group. Among them, TM and RM use XA protocol to carry out both-way communication, which is realized through two-phase commit. Compared to traditional local transactions, XA transaction adds a preparation stage where the database can also inform the caller whether the transaction can be committed, in addition to passively accepting commit instructions. TM can collect the results of all branch transactions and make atomic commit at the end to ensure the strong consistency of transactions.



XA transaction is implemented based on the interface of ShardingSphere's proxy database xa start/end/prepare/commit/rollback/recover.

For a logical SQL, ShardingSphere starts transactions in each proxied database with the xa begin directive, integrates TM internally for coordinating branch transactions, and performs xa commit /rollback.

Distributed transactions based on XA protocol are more suitable for short transactions with fixed execution time because the required resources need to be locked during execution. For long transactions, data exclusivity during the entire transaction will have an impact on performance in concurrent scenarios.

BASE Transaction

If a transaction that implements ACID is called a rigid transaction, then a transaction based on a BASE transaction element is called a flexible transaction. BASE stands for basic availability, soft state, and eventual consistency.

- Basically Available: ensure that distributed transaction parties are not necessarily online at the same time.
- Soft state: system status updates are allowed to have a certain delay, and the delay may not be recognized by customers.
- Eventually consistent: guarantee the eventual consistency of the system by means of messaging.

ACID transaction puts a high demand for isolation, where all resources must be locked during the execution of transactions. Flexible transaction is to move mutex operations from the resource level to the business level through business logic. Reduce the requirement for strong consistency in exchange for higher system throughput.

ACID-based strong consistency transactions and BASE-based final consistency transactions are not a jack of all trades and can fully leverage their advantages in the most appropriate scenarios. Apache ShardingSphere integrates the operational scheme taking SEATA as the flexible transaction. The following table can be used for comparison to help developers choose the suitable technology.

	<i>LOCAL</i>	<i>XA</i>	<i>BASE</i>
Business transformation	None	None	Seata Server needed
Consistency	Not supported	Supported	Final consistency
Isolation	Not supported	Supported	Business side guaranteed
Concurrent performance	no loss	severe loss	slight loss
Applied scenarios	Inconsistent processing by the business side	short transaction & low-level concurrency	long transaction & high concurrency

8.2.5 Application Scenarios

The database's transactions can meet ACID business requirements in a standalone application scenario. However, in distributed scenarios, traditional database solutions cannot manage and control global transactions, and users may find data inconsistency on multiple database nodes.

ShardingSphere distributed transaction makes it easier to process distributed transactions and provides flexible and diverse solutions. Users can select the distributed transaction solutions that best fit their business scenarios among LOCAL, XA, and BASE modes.

Application Scenarios for ShardingSphere XA Transactions

Strong data consistency is guaranteed in a distributed environment in terms of XA transactions. However, its performance may be degraded due to the synchronous blocking problem. It applies to business scenarios that require strong data consistency and low concurrency performance.

Application Scenarios for ShardingSphere BASE Transaction

In terms of BASE transactions, final data consistency is guaranteed in a distributed environment. Unlike XA transactions, resources are not locked during the whole transaction process, so its performance is relatively higher.

Application Scenarios for ShardingSphere LOCAL Transaction

In terms of LOCAL transactions, the data consistency and isolation among database nodes are not guaranteed in a distributed environment. Therefore, the business sides need to handle the inconsistencies by themselves. This applies to business scenarios where users would like to handle data inconsistency in a distributed environment by themselves.

8.2.6 Related references

- [YAML distributed transaction configuration](#)

8.2.7 Core Concept

XA Protocol

The original distributed transaction model of XA protocol is the “X/Open Distributed Transaction Processing (DTP)” model, XA protocol for short, which was proposed by the X/Open international consortium.

8.2.8 Limitations

Although Apache ShardingSphere aims at being compatible with all distributed scenario and providing the best performance, under the CAP theorem guidance, there is no silver bullet with distributed transaction solution.

The Apache ShardingSphere community chose instead to give the users the ability to choose their preferred distributed transaction type and use the most suitable solution according to their scenarios.

LOCAL Transaction

Unsupported

- Does not support the cross-database transactions caused by network or hardware crash. For example, when updating two databases in transaction, if one database crashes before commit, then only the data of the other database can commit.

XA Transaction

Unsupported

- Recover committing and rolling back in other machines after the service is down.
- MySQL, in the transaction block, the SQL execution is abnormal, and run `Commit`, and data remains consistent.
- After XA transactions are configured, the maximum length of the storage unit name cannot exceed 45 characters.

BASE Transaction

Unsupported

- Does not support isolation level.

8.2.9 Appendix with SQL operator

Unsupported SQL:

- RAL and RDL operations of DistSQL that are used in transactions.
- DDL statements that are used in XA transactions.

Privileges required for XA transactions:

In MySQL8, you need to grant the user `XA_RECOVER_ADMIN` privileges, otherwise, the XA transaction manager will report an error when executing the `XA RECOVER` statement.

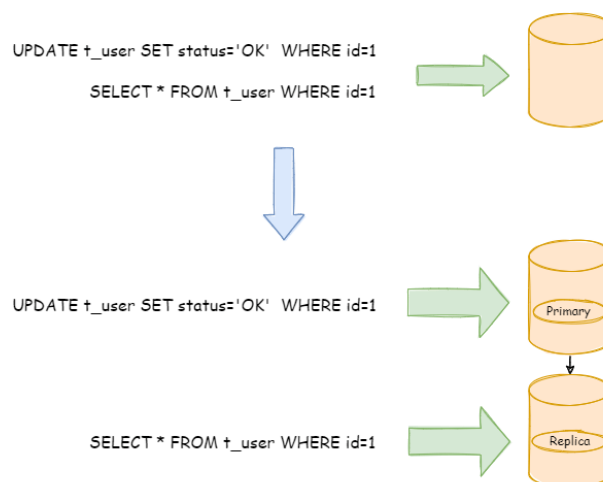
8.3 Readwrite-splitting

8.3.1 Background

Database throughput has faced the bottleneck with increasing TPS. For the application with massive concurrence read but less write in the same time, we can divide the database into a primary database and a replica database. The primary database is responsible for the insert, delete and update of transactions, while the replica database is responsible for queries. It can significantly improve the query performance of the whole system by effectively avoiding row locks.

One primary database with multiple replica databases can further enhance processing capacity by distributing queries evenly into multiple data replicas. Multiple primary databases with multiple replica databases can enhance not only throughput but also availability. Therefore, the system can still run normally, even though any database is down or physical disk destroyed.

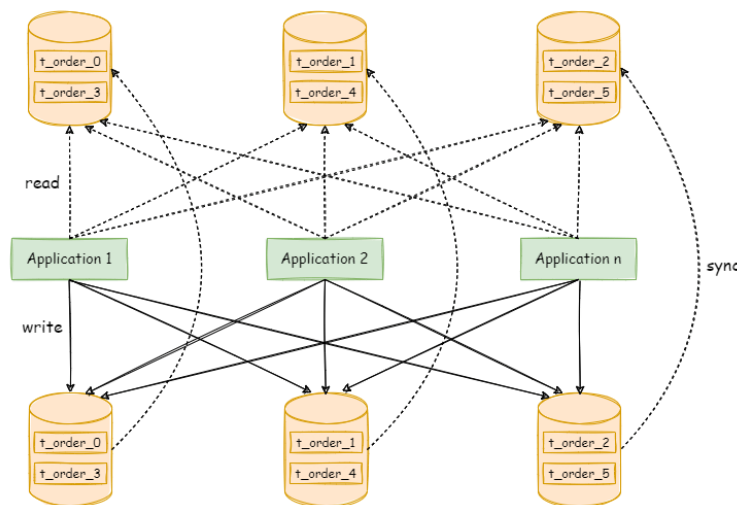
Different from the sharding that separates data to all nodes according to sharding keys, readwrite-splitting routes read and write separately to primary database and replica databases according to SQL analysis.



Data in readwrite-splitting nodes are consistent, whereas that in shards is not. The combined use of sharding and readwrite-splitting will effectively enhance the system performance.

8.3.2 Challenges

Though readwrite-splitting can enhance system throughput and availability, it also brings inconsistent data, including that among multiple primary databases and among primary databases and replica databases. What's more, it also brings the same problem as data sharding, complicating developer and operator's maintenance and operation. The following diagram has shown the complex topological relations between applications and database groups when sharding used together with readwrite-splitting.



8.3.3 Goal

The main design goal of readwrite-splitting of Apache ShardingSphere is to try to reduce the influence of readwrite-splitting, in order to let users use primary-replica database group like one database.

8.3.4 Application Scenarios

Complex primary-secondary database architecture

Many systems rely on the configuration of primary-secondary database architecture to improve the throughput of the whole system. Nevertheless, this configuration can make it more complex to use services.

After accessing ShardingSphere, the read/write splitting feature can be used to manage primary-secondary databases and achieve transparent read/write splitting, enabling users to use databases with primary/secondary architecture just like using one single database.

8.3.5 Related References

[Java API YAML Configuration](#)

8.3.6 Core Concept

Primary database

The primary database is used to add, update, and delete data operations. Currently, only single primary database is supported.

Secondary database

The secondary database is used to query data operations and multi-secondary databases are supported.

Primary-Secondary synchronization

It refers to the operation of asynchronously synchronizing data from a primary database to a secondary database. Due to the asynchronism of primary-secondary synchronization, data from the primary and secondary databases may be inconsistent for a short time.

Load balancer policy

Channel query requests to different secondary databases through load balancer policy.

8.3.7 Limitations

- Data synchronization of primary and secondary databases is not supported.
- Data inconsistency resulting from data synchronization delays between primary and secondary databases is not supported.
- Multi-write of primary database is not supported.
- Transactional consistency between primary and secondary databases is not supported. In the primary-secondary model, both data reads and writes in transactions use the primary database.

8.4 DB Gateway

8.4.1 Background

With the trend of database fragmentation, using multiple types of databases together has become the norm. The scenario of using one SQL dialect to access all heterogeneous databases is increasing.

8.4.2 Challenges

The existence of diversified databases makes it difficult to standardize the SQL dialect accessing the database. Engineers need to use different dialects for different kinds of databases, and there is no unified query platform.

Automatically translate different types of database dialects into the dialects used by the database, so that engineers can use any database dialect to access all heterogeneous databases, which can reduce development and maintenance cost greatly.

8.4.3 Goal

The goal of database gateway for Apache ShardingSphere is translating SQL automatically among various databases.

8.4.4 Application Scenarios

As business scenarios and database products of enterprises become increasingly diversified, the connection between business applications and various database products becomes extremely complex. ShardingSphere database gateway can shield the connection between business applications and the underlying diversified databases. At the same time, it provides a unified access protocol and syntax system for different business scenarios, which can help enterprises quickly build a unified data access platform.

8.4.5 Core Concept

SQL Dialect

SQL dialect means database dialect, and it indicates that some database projects have their own unique syntax in addition to SQL, which are also called dialects. Different database projects may have different SQL dialects.

8.4.6 Limitations

The SQL dialect translation of Apache ShardingSphere is experimental.

Currently, only MySQL/PostgreSQL dialects can be automatically translated. Engineers can use MySQL dialects and protocols to access PostgreSQL databases and vice versa.

8.5 Traffic Governance

8.5.1 Background

As the scale of data continues to expand, a distributed database has become a trend gradually. The unified management ability of cluster perspective, and control ability of individual components are necessary ability in modern database system.

8.5.2 Challenges

The challenge is ability which are unified management of centralized management, and operation in case of single node in failure.

Centralized management is to uniformly manage the state of database storage nodes and middleware computing nodes, and can detect the latest updates in the distributed environment in real time, further provide information with control and scheduling.

In the overload traffic scenario, circuit breaker and request limiting for a node to ensure whole database cluster can run continuously is a challenge to control ability of a single node.

8.5.3 Goal

The goal of Apache ShardingSphere management module is to realize the integrated management ability from database to computing node, and provide control ability for components in case of failure.

8.5.4 Application Scenarios

Overloaded compute node protection

When a compute node in a ShardingSphere cluster exceeds its load, the circuit breaker function is used to block the traffic to the compute node, to ensure that the whole cluster continues to provide stable services.

Storage node traffic limit

In the read-write splitting scenario where a storage node responsible for the read traffic in a ShardingSphere cluster receives overloaded requests, the traffic limit function is used to block traffic from compute nodes to the storage node, to ensure normal response of the storage node cluster.

8.5.5 Core Concept

Circuit Breaker

Fuse connection between Apache ShardingSphere and the database. When an Apache ShardingSphere node exceeds the max load, stop the node's access to the database, so that the database can ensure sufficient resources to provide services for other Apache ShardingSphere nodes.

Request Limit

In the face of overload requests, open request limiting to protect some requests can still respond quickly.

8.6 Data Migration

8.6.1 Background

In a scenario where the business continues to develop and the amount of data and concurrency reaches a certain extent, the traditional single database may face problems in terms of performance, scalability and availability.

Although NoSQL solutions can solve the above problems through data sharding and horizontal scale-out, NoSQL databases generally do not support transactions and SQL.

ShardingSphere can also solve the above problems and supports data sharding and horizontal scale-out, while at the same time, also supporting distributed transactions and SQL.

The data migration scheme provided by ShardingSphere can help the traditional single database smoothly switch to ShardingSphere.

8.6.2 Challenges

The data migration process should not affect the running services. So the first challenge is to minimize the time window during which data is not available.

Next, data migration should not affect existing data. So the second challenge is to ensure the data correctness.

8.6.3 Goal

The major goal of Apache ShardingSphere in performing data migration is to reduce the impact of data migration on services and provide a one-stop universal data migration solution.

8.6.4 Application Scenarios

Application scenario one: when an application system is using a traditional single database, and the amount of data in a single table reaches 100 million and is still growing rapidly, a single database that continues to run with a high load will become the bottleneck of the system.

Once the database becomes the bottleneck, it is useless to scale out the application server. Instead, it is the database that needs to be scaled out.

8.6.5 Related References

- [Configurations of data migration](#)
- [Reference of data migration](#)

8.6.6 Core Concept

Nodes

Instances for running compute or storage tier component processes. These can either be physical machines, virtual machines, or containers, etc.

Cluster

Multiple nodes that are assembled together to provide a specified service.

Source

The storage cluster where the original data resides.

Target

The target storage cluster to which the original data is to be migrated.

Data Migration Process

The entire process of replicating data from one storage cluster to another.

Stock Data

The data that was already in the data node before the data migration operation started.

Incremental Data

New data generated by operational systems during the execution of data migration operations.

8.6.7 Limitations

Procedures Supported

- Migration of peripheral data to databases managed by Apache ShardingSphere.
- Target proxy without rule or configure any rule.
- Migration of single column primary key or unique key table, the first column type could be: integer data type, string data type and part of binary data type (e.g. MySQL VARBINARY).
- Migration of multiple column primary keys or unique keys table.

Procedures not supported

- Migration on top of the current storage node is not supported, so a brand new database cluster needs to be prepared as the migration target cluster.
- Target proxy table rule contains HINT strategy.
- Use different target table schema from source table schema.
- Source table DDL changes during migration.

8.7 Encryption

8.7.1 Background

Security control has always been a crucial link of data governance, data encryption falls into this category. For both Internet enterprises and traditional sectors, data security has always been a highly valued and sensitive topic. Data encryption refers to transforming some sensitive information through encrypt rules to safely protect the private data. Data involves client's security or business sensitivity, such as ID number, phone number, card number, client number and other personal information, requires data encryption according to relevant regulations.

For data encryption requirements, there are the following situations in realistic business scenarios:

- When the new business start to launch, and the security department stipulates that the sensitive information related to users, such as banks and mobile phone numbers, should be encrypted and stored in the database, and then decrypted when used.

8.7.2 Challenges

In the real business scenario, the relevant business development team often needs to implement and maintain a set of encryption and decryption system according to the needs of the company' s security department. When the encryption scenario changes, the encryption system often faces the risk of reconstruction or modification. In addition, for the online business system, it is relatively complex to realize seamless encryption transformation with transparency, security and low risk without modifying the business logic and SQL.

8.7.3 Goal

Provides a security and transparent data encryption solution, which is the main design goal of Apache ShardingSphere data encryption module.

8.7.4 Application Scenarios

For scenarios requiring the quick launch of new services while respecting encryption regulations. The ShardingSphere encryption feature can be used to quickly achieve compliant data encryption, without requiring users to develop complex encryption systems.

At the same time, its flexibility can also help users avoid complex rebuilding and modification risks caused by encryption scenario changes.

8.7.5 Related References

- [Configuration: Data Encryption](#)
- [Developer Guide: Data Encryption](#)

8.7.6 Core Concept

Logic column

It is used to calculate the encryption and decryption columns and it is the logical identifier of the column in SQL. Logical columns contain ciphertext columns (mandatory), query-helper columns (optional), like-query columns (optional), and plaintext columns (optional).

Cipher column

Encrypted data columns.

Assisted query column

It is a helper column used for queries. For some non-idempotent encryption algorithms with higher security levels, irreversible idempotent columns are provided for queries.

Like query column

It is a helper column used for like queries.

8.7.7 Limitations

- You need to process the original data on stocks in the database by yourself.
- The `Like` query supports `%`, `_`, but currently does not support escape.
- Case insensitive queries are not supported for the encrypted fields.
- Comparison operations are not supported for encrypted fields, such as `GREATER THAN`, `LESS THAN`, `ORDER BY`, `BETWEEN`.
- Calculation operations are not supported for encrypted fields, such as `AVG`, `SUM`, and computation expressions.
- When projection subquery contains encrypt column, you must use alias.

8.7.8 Appendix with SQL operator

Unsupported SQL:

- The case-insensitive queries are not supported by encrypted fields.
- Comparison operations are not supported for encrypted fields, such as `GREATER THAN`, `LESS THAN`, `ORDER BY`, `BETWEEN`.
- Calculation operations are not supported for encrypted fields, such as `AVG`, `SUM`, and computation expressions.
- SQL that contains encrypt column in subquery and uses asterisks for outer projection is not supported.
- SQL that contains encrypt column in `WITH` is not supported.
- SQL that contains encrypt column in `INSERT SELECT` is not supported.
- SQL that contains encrypt column in `UNION`, `INTERSECT`, and `EXCEPT` statements is not supported.

Other:

- You should keep encrypt columns, assisted columns and like columns in encrypt rule same capitalization with columns in database.

8.8 Data Masking

8.8.1 Background

With the introduction of laws on user data protection, the protection of personal privacy data has risen to the legal level. Traditional application systems generally lack protection measures for personal privacy data. Data masking can achieve special encryption, masking and replacement of the data returned by the production database according to user-defined masking rules without any changes to the data in the production database to ensure the sensitivity of the production environment data can be protected.

8.8.2 Challenges

In real business scenarios, relevant DevOps teams often need to implement and maintain a set of masking functions by themselves according to data masking requirements, and the masking functions are often coupled in various business logics. Additionally different business systems are difficult to reuse. When the masking scenario changes, the masking function maintained by itself often faces the risk of refactoring or modification.

8.8.3 Goal

According to industry needs for data masking and the pain points of business transformation, it provides a complete, safe, transparent, and low transformation cost data masking integration solution, which is the main design goal of the Apache ShardingSphere data masking module.

8.8.4 Application Scenarios

Whether it is a new business that is launched quickly or a mature business that has already been launched, you can access the data masking function of ShardingSphere to quickly complete the configuration of mask rules. Customers can use data masking function transparently without developing a masking function coupled to the business system, and without changing any business logic and SQL.

8.8.5 Related References

- [Configuration: Data Mask](#)
- [Developer Guide: Data Mask](#)

8.8.6 Core Concept

Logic column

The logical name used to calculate masked column, which is logical identifier of column in SQL.

8.8.7 Limitations

- Masked columns only support string types, not other non-string types.

8.9 Shadow

8.9.1 Background

Under the distributed application architecture based on microservices, business requires multiple services to be completed through a series of services and middleware calls. The pressure testing of a single service can no longer reflect the real scenario.

In the test environment, the cost of rebuild complete set of pressure test environment similar to the production environment is too high. It is usually impossible to simulate the complexity and data of the production environment.

So, it is the better way to use the production environment for pressure test. The test results obtained real capacity and performance of the system accurately.

8.9.2 Challenges

pressure testing on production environment is a complex and huge task. Coordination and adjustments between microservices and middlewares required to cope with the transparent transmission of different flow rates and pressure test tags. Usually we will build a complete set of pressure testing platform for different test plans.

Data isolation have to be done at the database-level, in order to ensure the reliability and integrity of the production data, data generated by pressure testing routed to test database. Prevent test data from polluting the real data in the production database.

This requires business applications to perform data classification based on the transparently transmitted pressure test identification before executing SQL, and route the corresponding SQL to the corresponding data source.

8.9.3 Goal

Apache ShardingSphere focuses on data solutions in pressure testing on production environment.

The main goal of the Apache ShardingSphere shadow Database module is routing pressure testing data to user defined database automatically.

8.9.4 Application Scenario

In order to improve the accuracy of stress testing and reduce the testing cost under the distributed application architecture based on microservices, stress testing is usually carried out in production environments, which will notably increase testing risks. However, the ShardingSphere shadow DB function, combined with the flexible configuration of the shadow algorithm, can address data pollution, improve database performance, and meet the requirements of online stress testing in complex business scenarios.

8.9.5 Related References

- [Java API: shadow DB](#)
- [YAML configuration: shadow DB](#)

8.9.6 Core Concept

Production Database

Database for production data

Shadow Database

The Database for stress test data isolation. Configurations should be the same as the Production Database.

Shadow Algorithm

Shadow Algorithm, which is closely related to business operations, currently has 2 types.

- Column based shadow algorithm Routing to shadow database by recognizing data from SQL. Suitable for stress test scenario that has an emphasis on data list.
- Hint based shadow algorithm Routing to shadow database by recognizing comments from SQL. Suitable for stress test driven by the identification of upstream system passage.

8.9.7 Limitations

Hint based shadow algorithm

No

Column based shadow algorithm

SQL does not support lists: - Does not support DDL.

- Does not support scope, group, subqueries such as BETWEEN, GROUP BY ...HAVING, etc.

SQL support list

- INSERT

SQL	support or not
INSERT INTO table (column,...) VALUES (value,...)	support
INSERT INTO table (column,...) VALUES (value,...),(value,...),...	support
INSERT INTO table (column,...) SELECT column1 from table1 where column1 = value1	do not support

- SELECT/UPDATE/DELETE

condition categories	SQL	support or not
=	SELECT/UPDATE/DELETE ...WHERE column = value	support
LIKE/NOT LIKE	SELECT/UPDATE/DELETE ...WHERE column LIKE/NOT LIKE value	support
IN/NOT IN	SELECT/UPDATE/DELETE ...WHERE column IN/NOT IN (value1,value2,...)	support
BETWEEN	SELECT/UPDATE/DELETE ...WHERE column BETWEEN value1 AND value2	do not support
GROUP BY ... HAVING...	SELECT/UPDATE/DELETE ...WHERE ...GROUP BY column HAVING column > value	do not support
Sub Query	SELECT/UPDATE/DELETE ...WHERE column = (SELECT column FROM table WHERE column = value)	do not support

8.10 Observability

8.10.1 Background

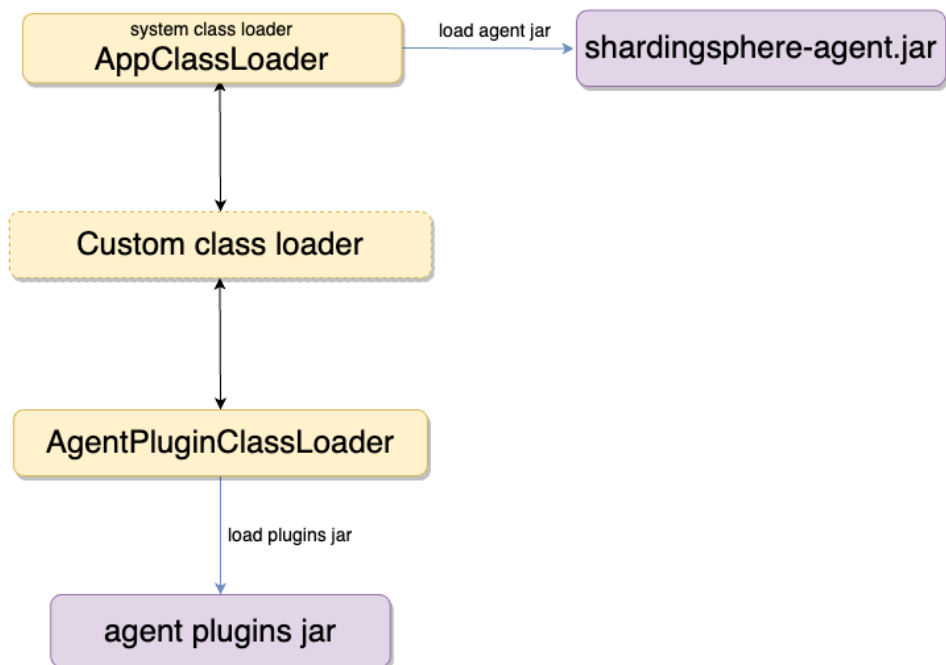
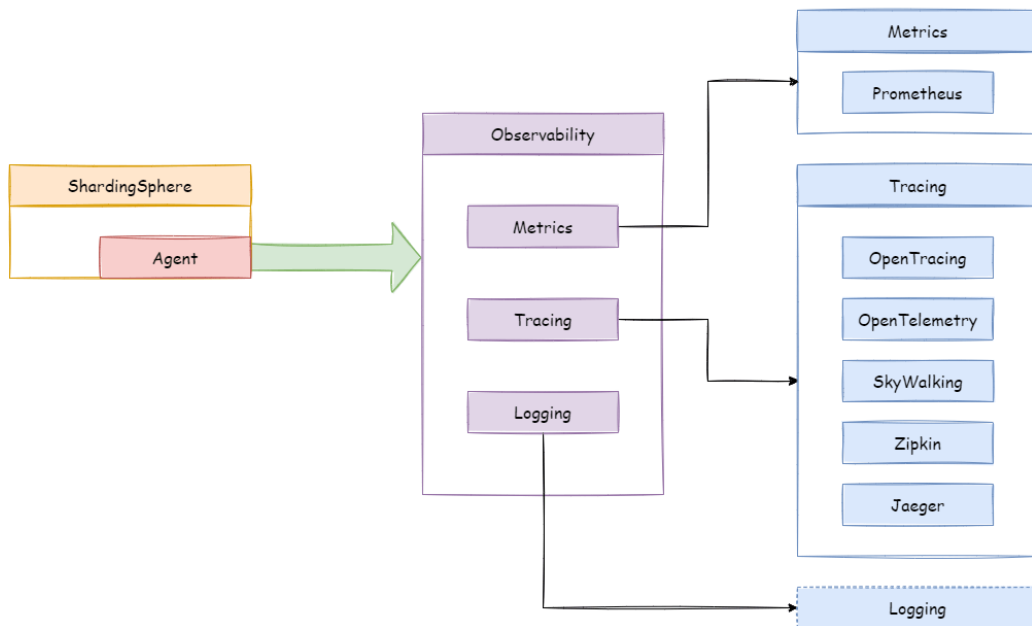
In order to grasp the distributed system status, observe running state of the cluster is a new challenge. The point-to-point operation mode of logging in to a specific server cannot suite to large number of distributed servers. Telemetry through observable data is the recommended operation and maintenance mode for them. Tracking, metrics and logging are important ways to obtain observable data of system status.

APM (application performance monitoring) is to monitor and diagnose the performance of the system by collecting, storing and analyzing the observable data of the system. Its main functions include performance index monitoring, call stack analysis, service topology, etc.

Apache ShardingSphere is not responsible for gathering, storing and demonstrating APM data, but provides the necessary information for the APM. In other words, Apache ShardingSphere is only responsible for generating valuable data and submitting it to relevant systems through standard protocols or plug-ins. Tracing is to obtain the tracking information of SQL parsing and SQL execution. Apache ShardingSphere provides support for OpenTelemetry, SkyWalking by default. It also supports users to develop customized components through plug-in.

- Use OpenTelemetry OpenTelemetry was merged by OpenTracing and OpenCensus in 2019. In this way, you only need to fill in the appropriate configuration in the agent configuration file according to [OpenTelemetry SDK Autoconfigure Guide](#). Data can be exported to Jaeger, Zipkin.
- Use SkyWalking Enable the SkyWalking plug-in in configuration file and need to configure the SkyWalking apm-toolkit.
- Use SkyWalking' s automatic monitor probe Cooperating with [Apache SkyWalking](#) team, Apache ShardingSphere team has realized ShardingSphere automatic monitor probe to automatically send performance data to SkyWalking. Note that automatic probe in this way cannot be used together with Apache ShardingSphere plug-in probe.

Metrics used to collect and display statistical indicator of cluster. Apache ShardingSphere supports Prometheus by default.



8.10.2 Challenges

Tracing and metrics need to collect system information through event tracking. Lots of events tracking make kernel code mess, difficult to maintain, and difficult to customize extend.

8.10.3 Goal

The goal of Apache ShardingSphere observability module is providing as many performance and statistical indicators as possible and isolating kernel code and embedded code.

8.10.4 Application Scenarios

ShardingSphere provides observability for applications through the Agent module, and this feature applies to the following scenarios:

Monitoring panel

The system's static information (such as application version) and dynamic information (such as the number of threads and SQL processing information) are exposed to a third-party application (such as Prometheus) using a standard interface. Administrators can visually monitor the real-time system status.

Monitoring application performance

In ShardingSphere, a SQL statement needs to go through the processes of parsing, routing, rewriting, execution, and result merging before it is finally executed and the response can be output. If a SQL statement is complex and the overall execution takes a long time, how do we know which procedure has room for optimization?

Through Agent plus Tracing, administrators can learn about the time consumption of each step of SQL execution. Thus, they can easily locate performance risks and formulate targeted SQL optimization schemes.

Tracing application links

In a distributed application plus data sharding scenario, it is tricky to figure out which node the SQL statement is issued from and which data source the statement is finally executed on. If an exception occurs during SQL execution, how do we locate the node where the exception occurred?

Agent + Tracing can help users solve the above problems.

Through tracing the full link of the SQL execution process, users can get complete information such as “where the SQL comes from and where it is sent to” .

They can also visually observe the SQL routing situation through the generated topological graph, make timely responses, and quickly locate the root cause of problems.

8.10.5 Related References

- [Usage of observability](#)
- [Dev guide: observability](#)
- [Implementation](#)

8.10.6 Core Concept

Agent

Based on bytecode enhancement and plugin design to provide tracing, metrics and logging features.

Only after the plugin of the Agent is enabled, the monitoring indicator data can be output to the third-party APM for display.

APM

APM is an acronym for Application Performance Monitoring.

Focusing on the performance diagnosis of distributed systems, its main functions include call chain display, application topology analysis, etc.

Tracing

Tracing data between distributed services or internal processes will be collected by agent. It will then be sent to third-party APM systems.

Metrics

System statistical indicators are collected through probes for display by third-party applications.

Logging

The log can be easily expanded through the agent to provide more information for analyzing the system running status.

8.11 SQL Federation

8.11.1 Background

When users use data sharding to horizontally split massive amounts of data, although it can effectively solve database performance bottlenecks, it also brings some new problems in business.

For example, in the following scenarios: cross database association queries, sub queries, pagination, sorting, aggregation queries.

When implementing business operations, it is important to pay attention to the usage range of SQL queries and avoid cross database instance queries as much as possible, which limits the functionality of the business at the database level.

8.11.2 Challenges

User business queries in SQL are often complex and variable, and it is costly to integrate them into ShardingSphere through business SQL transformation.

Convert the original queries on the business side into distributed queries and perform corresponding SQL optimization in distributed query scenarios, which can be completed across database instances: associated queries, sub queries, pagination, sorting, and aggregation queries.

In terms of business implementation, it can enable R&D personnel to no longer care about the scope of SQL usage, focus on business function development, and reduce functional limitations at the business level.

8.11.3 Goal

Implementing distributed SQL for cross database instance queries is the main design goal of Apache ShardingSphere federated queries.

8.11.4 Application Scenario

When cross database association queries, sub queries, and aggregate queries are required. No need to modify SQL, enabling federated queries through configuration can complete the execution of distributed query statements.

8.11.5 Related References

- [SQL Federation Configuration](#)

8.11.6 Limitations

The SQL federation query of Apache ShardingSphere is experimental.

This chapter describes how to use projects of Apache ShardingSphere.

9.1 ShardingSphere-JDBC

Configuration is the only module in ShardingSphere-JDBC that interacts with application developers, through which developers can quickly and clearly understand the functions provided by ShardingSphere-JDBC.

This chapter is a configuration manual for ShardingSphere-JDBC, which can also be referred to as a dictionary if necessary.

ShardingSphere-JDBC has provided 2 kinds of configuration methods for different situations. By configuration, application developers can flexibly use data sharding, readwrite-splitting, data encryption, shadow database or the combination of them.

Mixed rule configurations are very similar to single rule configuration, except for the differences from single rule to multiple rules.

It should be noted that the superposition between rules are data source and table name related. If the previous rule is data source oriented aggregation, the next rule needs to use the aggregated logical data source name configured by the previous rule when configuring the data source; Similarly, if the previous rule is table oriented aggregation, the next rule needs to use the aggregated logical table name configured by the previous rule when configuring the table.

Please refer to [Example](#) for more details.

9.1.1 YAML Configuration

Overview

YAML configuration provides interaction with ShardingSphere JDBC through configuration files. When used with the governance module together, the configuration of persistence in the configuration center is YAML format.

Note: The YAML configuration file supports more than 3MB of configuration content.

YAML configuration is the most common configuration mode, which can omit the complexity of programming and simplify user configuration.

Usage

Import Maven Dependency

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>
```

YAML Format

ShardingSphere-JDBC YAML file consists of database name, mode configuration, data source map, rule configurations and properties.

Note: The example connection pool is HikariCP, which can be replaced with other connection pools according to business scenarios.

```
# JDBC logic database name. Through this parameter to connect ShardingSphere-JDBC
and ShardingSphere-Proxy.
# Default value: logic_db
databaseName (?):

mode:

dataSources:

rules:
- !FOO_XXX
  ...
- !BAR_XXX
  ...

props:
```

```
key_1: value_1
key_2: value_2
```

Please refer to [Mode Configuration](#) for more mode details.

Please refer to [Data Source Configuration](#) for more data source details.

Please refer to [Rules Configuration](#) for more rule details.

Create Data Source

The ShardingSphereDataSource created by YamlShardingSphereDataSourceFactory implements the standard JDBC DataSource interface.

```
File yamlFile = // Indicate YAML file
DataSource dataSource = YamlShardingSphereDataSourceFactory.
createDataSource(yamlFile);
```

Use Data Source

Same with Java API.

YAML Syntax Explanation

!! means instantiation of that class

! means self-defined alias

- means one or multiple can be included

[] means array, can substitutable with - each other

Mode

Parameters

```
mode (?): # Default value is Standalone
  type: # Type of mode configuration. Values could be: Standalone, Cluster
repository (?): # Persist repository configuration
```

Standalone Mode

```
mode:
  type: Standalone
  repository:
    type: # Type of persist repository
    props: # Properties of persist repository
      foo_key: foo_value
      bar_key: bar_value
```

Cluster Mode (recommended)

```
mode:
  type: Cluster
  repository:
    type: # Type of persist repository
    props: # Properties of persist repository
      namespace: # Namespace of registry center
      server-lists: # Server lists of registry center
      foo_key: foo_value
      bar_key: bar_value
```

Notes

1. Cluster mode deployment is recommended for production environment.
2. The ZooKeeper registry center is recommended for cluster mode deployment.
3. If there is configuration information in the ZooKeeper, please refer to the config information there.

Sample

Standalone Mode

```
mode:
  type: Standalone
  repository:
    type: JDBC
```

Cluster Mode (recommended)

```
mode:
  type: Cluster
  repository:
    type: ZooKeeper
    props:
      namespace: governance
      server-lists: localhost:2181
      retryIntervalMilliseconds: 500
      timeToLiveSeconds: 60
```

Using the persistent repository requires additional introduction of the corresponding Maven dependencies. It is recommended to use:

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-cluster-mode-repository-zookeeper</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>
```

Related References

- [Installation and Usage of ZooKeeper Registry Center](#)
- Please refer to [Builtin Persist Repository List](#) for more details about the type of repository.
- Please refer to [ShardingSphere-JDBC Optional Plugins](#) for more implementations of the persistent repository.

Data Source

Background

ShardingSphere-JDBC Supports all JDBC drivers and data source connection pools.

In this example, the database driver is MySQL, and the connection pool is HikariCP, which can be replaced with other database drivers and connection pools. When using ShardingSphere JDBC, the property name of the JDBC pool depends on the definition of the respective JDBC pool and is not defined by ShardingSphere. For related processing, please refer to the class `org.apache.shardingsphere.infra.datasource.pool.creator.DataSourcePoolCreator`. For example, with Alibaba Druid 1.2.9, using `url` instead of `jdbcUrl` in the example below is the expected behavior.

Parameters

```
dataSources: # Data sources configuration, multiple <data-source-name> available
  <data_source_name>: # Data source name
    dataSourceClassName: # Data source class name
    driverClassName: # The database driver class name is subject to the
configuration of the data source connection pool itself
    jdbcUrl: # The database URL connection is subject to the configuration of the
data source connection pool itself
    username: # Database username, subject to the configuration of the data source
connection pool itself
    password: # The database password is subject to the configuration of the data
source connection pool itself
    # ... Other properties of data source pool
```

Sample

```
dataSources:
  ds_1:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/ds_1
    username: root
    password:
  ds_2:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/ds_2
    username: root
    password:

# Configure other data sources
```

Rules

Rules are pluggable part of Apache ShardingSphere. This chapter is a YAML rule configuration manual for ShardingSphere-JDBC.

Sharding

Background

Data sharding YAML configuration is highly readable. The dependencies between sharding rules can be quickly understood through the YAML format. ShardingSphere automatically creates the ShardingSphereDataSource object according to YAML configuration, which can reduce unnecessary coding for users.

Parameters

```
rules:
- !SHARDING
  tables: # Sharding table configuration
    <logic_table_name> (+): # Logic table name
      actualDataNodes (?): # Describe data source names and actual tables (refer to
Inline syntax rules)
      databaseStrategy (?): # Databases sharding strategy, use default databases
sharding strategy if absent. sharding strategy below can choose only one.
      standard: # For single sharding column scenario
        shardingColumn: # Sharding column name
        shardingAlgorithmName: # Sharding algorithm name
      complex: # For multiple sharding columns scenario
        shardingColumns: # Sharding column names, multiple columns separated with
comma
        shardingAlgorithmName: # Sharding algorithm name
      hint: # Sharding by hint
        shardingAlgorithmName: # Sharding algorithm name
      none: # Do not sharding
  tableStrategy: # Tables sharding strategy, same as database sharding strategy
  keyGenerateStrategy: # Key generator strategy
    column: # Column name of key generator
    keyGeneratorName: # Key generator name
  auditStrategy: # Sharding audit strategy
    auditorNames: # Sharding auditor name
      - <auditor_name>
      - <auditor_name>
    allowHintDisable: true # Enable or disable sharding audit hint
  autoTables: # Auto Sharding table configuration
    t_order_auto: # Logic table name
    actualDataSources (?): # Data source names
```

```

    shardingStrategy: # Sharding strategy
      standard: # For single sharding column scenario
        shardingColumn: # Sharding column name
        shardingAlgorithmName: # Auto sharding algorithm name
    bindingTables (+): # Binding tables
      - <logic_table_name_1, logic_table_name_2, ...>
      - <logic_table_name_1, logic_table_name_2, ...>
    defaultDatabaseStrategy: # Default strategy for database sharding
    defaultTableStrategy: # Default strategy for table sharding
    defaultKeyGenerateStrategy: # Default Key generator strategy
    defaultShardingColumn: # Default sharding column name

# Sharding algorithm configuration
shardingAlgorithms:
  <sharding_algorithm_name> (+): # Sharding algorithm name
    type: # Sharding algorithm type
    props: # Sharding algorithm properties
    # ...

# Key generate algorithm configuration
keyGenerators:
  <key_generate_algorithm_name> (+): # Key generate algorithm name
    type: # Key generate algorithm type
    props: # Key generate algorithm properties
    # ...

# Sharding audit algorithm configuration
auditors:
  <sharding_audit_algorithm_name> (+): # Sharding audit algorithm name
    type: # Sharding audit algorithm type
    props: # Sharding audit algorithm properties
    # ...

- !BROADCAST
  tables: # Broadcast tables
    - <table_name>
    - <table_name>

```

Procedure

1. Configure data sharding rules in YAML files, including data source, sharding rules, and global attributes and other configuration items.
2. Call `createDataSource` method of the object `YamlShardingSphereDataSourceFactory`. Create `ShardingSphereDataSource` according to the configuration information in YAML files.

Sample

The YAML configuration sample of data sharding is as follows:

```
dataSources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/demo_ds_0?serverTimezone=UTC&useSSL=false&
useUnicode=true&characterEncoding=UTF-8
    username: root
    password:
  ds_1:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/demo_ds_1?serverTimezone=UTC&useSSL=false&
useUnicode=true&characterEncoding=UTF-8
    username: root
    password:

rules:
- !SHARDING
  tables:
    t_order:
      actualDataNodes: ds_${0..1}.t_order_${0..1}
      tableStrategy:
        standard:
          shardingColumn: order_id
          shardingAlgorithmName: t_order_inline
      keyGenerateStrategy:
        column: order_id
        keyGeneratorName: snowflake
      auditStrategy:
        auditorNames:
          - sharding_key_required_auditor
        allowHintDisable: true
    t_order_item:
      actualDataNodes: ds_${0..1}.t_order_item_${0..1}
      tableStrategy:
        standard:
          shardingColumn: order_id
          shardingAlgorithmName: t_order_item_inline
      keyGenerateStrategy:
        column: order_item_id
        keyGeneratorName: snowflake
    t_account:
      actualDataNodes: ds_${0..1}.t_account_${0..1}
      tableStrategy:
        standard:
```

```

    shardingAlgorithmName: t_account_inline
    keyGenerateStrategy:
      column: account_id
      keyGeneratorName: snowflake
    defaultShardingColumn: account_id
    bindingTables:
      - t_order,t_order_item
    defaultDatabaseStrategy:
      standard:
        shardingColumn: user_id
        shardingAlgorithmName: database_inline
    defaultTableStrategy:
      none:

shardingAlgorithms:
  database_inline:
    type: INLINE
    props:
      algorithm-expression: ds_${user_id % 2}
  t_order_inline:
    type: INLINE
    props:
      algorithm-expression: t_order_${order_id % 2}
  t_order_item_inline:
    type: INLINE
    props:
      algorithm-expression: t_order_item_${order_id % 2}
  t_account_inline:
    type: INLINE
    props:
      algorithm-expression: t_account_${account_id % 2}
keyGenerators:
  snowflake:
    type: SNOWFLAKE
auditors:
  sharding_key_required_auditor:
    type: DML_SHARDING_CONDITIONS

- !BROADCAST
  tables: # Broadcast tables
    - t_address

props:
  sql-show: false

```

Read the YAML configuration to create a data source according to the `createDataSource` method of `YamlShardingSphereDataSourceFactory`.

```
YamlShardingSphereDataSourceFactory.createDataSource(getFile("/META-INF/sharding-databases-tables.yaml"));
```

Related References

- [Core Feature: Data Sharding](#)
- [Developer Guide: Data Sharding](#)

Broadcast Table

Background

Broadcast table YAML configuration is highly readable. The broadcast rules can be quickly understood thanks to the YAML format. ShardingSphere automatically creates the `ShardingSphereDataSource` object according to the YAML configuration, which reduces unnecessary coding for users.

Parameters

```
rules:
- !BROADCAST
  tables: # Broadcast tables
    - <table_name>
    - <table_name>
```

Procedure

1. Configure broadcast table list in the YAML file.
2. Call the `createDataSource` method of the object `YamlShardingSphereDataSourceFactory`. Create `ShardingSphereDataSource` according to the configuration information in YAML files.

Sample

The YAML configuration sample of the broadcast table is as follows:

```
dataSources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/demo_ds_0?serverTimezone=UTC&useSSL=false&useUnicode=true&characterEncoding=UTF-8
    username: root
```

```

    password:
  ds_1:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/demo_ds_1?serverTimezone=UTC&useSSL=false&
useUnicode=true&characterEncoding=UTF-8
    username: root
    password:

rules:
- !BROADCAST
  tables:
    - t_address

```

Read the YAML configuration to create a data source according to the `createDataSource` method of `YamlShardingSphereDataSourceFactory`.

```
YamlShardingSphereDataSourceFactory.createDataSource(getFile("/META-INF/broadcast-
databases-tables.yaml"));
```

Readwrite-splitting

Background

Read/write splitting YAML configuration is highly readable. The YAML format enables you to quickly understand the dependencies between read/write sharding rules. ShardingSphere automatically creates the `ShardingSphereDataSource` object according to the YAML configuration, which reduces unnecessary coding for users.

Parameters

Readwrite-splitting

```

rules:
- !READWRITE_SPLITTING
  dataSourceGroups:
    <data_source_group_name> (+): # Logic data source group name of readwrite-
splitting, which uses Groovy's Row Value Expressions SPI implementation to parse by
default
    write_data_source_name: # Write data source name, which uses Groovy's Row
Value Expressions SPI implementation to parse by default
    read_data_source_names: # Read data source names, multiple data source names
separated with comma, which uses Groovy's Row Value Expressions SPI implementation
to parse by default
    transactionalReadQueryStrategy (?): # Routing strategy for read query within a

```

```

transaction, values include: PRIMARY (to primary), FIXED (to fixed data source),
DYNAMIC (to any data source), default value: DYNAMIC
    loadBalancerName: # Load balance algorithm name

# Load balance algorithm configuration
loadBalancers:
    <load_balancer_name> (+): # Load balance algorithm name
        type: # Load balance algorithm type
        props: # Load balance algorithm properties
        # ...

```

Please refer to [Built-in Load Balance Algorithm List](#) for more details about type of algorithm.

Procedure

1. Add read/write splitting data source.
2. Set the load balancer algorithm.
3. Use read/write data source.

Sample

```

rules:
- !READWRITE_SPLITTING
  dataSourceGroups:
    readwrite_ds:
      writeDataSourceName: write_ds
      readDataSourceNames:
        - read_ds_0
        - read_ds_1
      transactionalReadQueryStrategy: PRIMARY
      loadBalancerName: random
  loadBalancers:
    random:
      type: RANDOM

```

Related References

- [Read-write splitting-Core features](#)
- [Java API: read-write splitting](#)

Distributed Transaction

Background

ShardingSphere provides three modes for distributed transactions LOCAL, XA, BASE.

Parameters

```
transaction:
  defaultType: # Transaction mode, optional value LOCAL/XA/BASE
  providerType: # Specific implementation of the mode
```

Procedure

Use LOCAL Mode

The content of the global.yaml configuration file is as follows:

```
transaction:
  defaultType: LOCAL
```

Use XA Mode

The content of the global.yaml configuration file is as follows:

```
transaction:
  defaultType: XA
  providerType: Narayana/Atomikos
```

To manually add Narayana-related dependencies:

```
jta-5.12.7.Final.jar
arjuna-5.12.7.Final.jar
common-5.12.7.Final.jar
jboss-connector-api_1.7_spec-1.0.0.Final.jar
jboss-logging-3.2.1.Final.jar
jboss-transaction-api_1.2_spec-1.0.0.Alpha3.jar
jboss-transaction-spi-7.6.1.Final.jar
narayana-jts-integration-5.12.7.Final.jar
shardingsphere-transaction-xa-narayana-x.x.x-SNAPSHOT.jar
```

Use BASE Mode

The content of the global.yaml configuration file is as follows:

```
transaction:
  defaultType: BASE
  providerType: Seata
```

Build a Seata Server, add relevant configuration files and Seata dependencies, see [ShardingSphere Integrates Seata Flexible Transactions](#)

Encryption

Background

The YAML configuration approach to data encryption is highly readable, with the YAML format enabling a quick understanding of dependencies between encryption rules. Based on the YAML configuration, ShardingSphere automatically completes the creation of ShardingSphereDataSource objects, reducing unnecessary coding efforts for users.

Parameters

```
rules:
- !ENCRYPT
  tables:
    <table_name> (+): # Encrypt table name
    columns:
      <column_name> (+): # Encrypt logic column name
      cipher:
        name: # Cipher column name
        encryptorName: # Cipher encrypt algorithm name
      assistedQuery (?):
        name: # Assisted query column name
        encryptorName: # Assisted query encrypt algorithm name
      likeQuery (?):
        name: # Like query column name
        encryptorName: # Like query encrypt algorithm name

  # Encrypt algorithm configuration
  encryptors:
    <encrypt_algorithm_name> (+): # Encrypt algorithm name
    type: # Encrypt algorithm type
    props: # Encrypt algorithm properties
    # ...
```

Please refer to [Built-in Encrypt Algorithm List](#) for more details about type of algorithm.

Procedure

1. Configure data encryption rules in the YAML file, including data sources, encryption rules, global attributes, and other configuration items.
2. Using the createDataSource of calling the YamlShardingSphereDataSourceFactory object to create ShardingSphereDataSource based on the configuration information in the YAML file.

Sample

The data encryption YAML configurations are as follows:

```
dataSources:
  unique_ds:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/demo_ds?serverTimezone=UTC&useSSL=false&
useUnicode=true&characterEncoding=UTF-8
    username: root
    password:

rules:
- !ENCRYPT
  tables:
    t_user:
      columns:
        username:
          cipher:
            name: username
            encryptorName: aes_encryptor
          assistedQuery:
            name: assisted_query_username
            encryptorName: assisted_encryptor
          likeQuery:
            name: like_query_username
            encryptorName: like_encryptor
        pwd:
          cipher:
            name: pwd
            encryptorName: aes_encryptor
          assistedQuery:
            name: assisted_query_pwd
            encryptorName: assisted_encryptor
  encryptors:
    aes_encryptor:
      type: AES
      props:
        aes-key-value: 123456abc
```



```

    digest-algorithm-name: SHA-1
  assisted_encryptor:
    type: MD5
  like_encryptor:
    type: CHAR_DIGEST_LIKE

```

Read the YAML configuration to create a data source according to the `createDataSource` method of `YamlShardingSphereDataSourceFactory`.

```
YamlShardingSphereDataSourceFactory.createDataSource(getFile());
```

Related References

- [Core Feature: Data Encryption](#)
- [Developer Guide: Data Encryption](#)

Data Masking

Background

The YAML configuration approach to data masking is highly readable, with the YAML format enabling a quick understanding of dependencies between mask rules. Based on the YAML configuration, ShardingSphere automatically completes the creation of `ShardingSphereDataSource` objects, reducing unnecessary coding efforts for users.

Parameters

```

rules:
- !MASK
  tables:
    <table_name> (+): # Mask table name
      columns:
        <column_name> (+): # Mask logic column name
          maskAlgorithm: # Mask algorithm name

  # Mask algorithm configuration
  maskAlgorithms:
    <mask_algorithm_name> (+): # Mask algorithm name
      type: # Mask algorithm type
      props: # Mask algorithm properties
      # ...

```

Please refer to [Built-in Mask Algorithm List](#) for more details about type of algorithm.

Procedure

1. Configure data masking rules in the YAML file, including data sources, mask rules, global attributes, and other configuration items.
2. Using the createDataSource of calling the YamlShardingSphereDataSourceFactory object to create ShardingSphereDataSource based on the configuration information in the YAML file.

Sample

The data masking YAML configurations are as follows:

```
dataSources:
  unique_ds:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/demo_ds?serverTimezone=UTC&useSSL=false&
useUnicode=true&characterEncoding=UTF-8
    username: root
    password:

rules:
- !MASK
  tables:
    t_user:
      columns:
        password:
          maskAlgorithm: md5_mask
        email:
          maskAlgorithm: mask_before_special_chars_mask
        telephone:
          maskAlgorithm: keep_first_n_last_m_mask

maskAlgorithms:
  md5_mask:
    type: MD5
  mask_before_special_chars_mask:
    type: MASK_BEFORE_SPECIAL_CHARS
    props:
      special-chars: '@'
      replace-char: '*'
  keep_first_n_last_m_mask:
    type: KEEP_FIRST_N_LAST_M
    props:
      first-n: 3
      last-m: 4
      replace-char: '*'
```

Read the YAML configuration to create a data source according to the `createDataSource` method of `YamlShardingSphereDataSourceFactory`.

```
YamlShardingSphereDataSourceFactory.createDataSource(getFile());
```

Related References

- [Core Feature: Data Masking](#)
- [Developer Guide: Data Masking](#)

Shadow DB

Background

Please refer to the following configuration in order to use the ShardingSphere shadow DB feature in ShardingSphere-Proxy.

Parameters

```
rules:
- !SHADOW
  dataSources:
    shadowDataSource:
      productionDataSourceName: # production data source name
      shadowDataSourceName: # shadow data source name
  tables:
    <table_name>:
      dataSourceNames: # shadow table associates shadow data source name list
      - <shadow_data_source>
      shadowAlgorithmNames: # shadow table associates shadow algorithm name list
      - <shadow_algorithm_name>
  defaultShadowAlgorithmName: # default shadow algorithm name (option)
  shadowAlgorithms:
    <shadow_algorithm_name> (+): # shadow algorithm name
      type: # shadow algorithm type
      props: # shadow algorithm attribute configuration
```

Please refer to [Built-in shadow algorithm list](#) for more details.

Procedure

1. Configure shadow DB rules in the YAML file, including data sources, shadow library rules, global properties and other configuration items;
2. Call the `createDataSource()` method of the `YamlShardingSphereDataSourceFactory` object to create a `ShardingSphereDataSource` based on the configuration information in the YAML file.

Sample

The YAML configuration sample of shadow DB is as follows:

```
dataSources:
  ds:
    url: jdbc:mysql://127.0.0.1:3306/ds?serverTimezone=UTC&useSSL=false
    username: root
    password:
    connectionTimeoutMilliseconds: 30000
    idleTimeoutMilliseconds: 60000
    maxLifetimeMilliseconds: 1800000
    maxPoolSize: 50
    minPoolSize: 1
  shadow_ds:
    url: jdbc:mysql://127.0.0.1:3306/shadow_ds?serverTimezone=UTC&useSSL=false
    username: root
    password:
    connectionTimeoutMilliseconds: 30000
    idleTimeoutMilliseconds: 60000
    maxLifetimeMilliseconds: 1800000
    maxPoolSize: 50
    minPoolSize: 1

rules:
- !SHADOW
  dataSources:
    shadowDataSource:
      productionDataSourceName: ds
      shadowDataSourceName: shadow_ds
  tables:
    t_order:
      dataSourceNames:
        - shadowDataSource
      shadowAlgorithmNames:
        - user_id_insert_match_algorithm
        - sql_hint_algorithm
  shadowAlgorithms:
    user_id_insert_match_algorithm:
```

```
type: REGEX_MATCH
props:
  operation: insert
  column: user_id
  regex: "[1]"
sql_hint_algorithm:
  type: SQL_HINT
```

Related References

- [Core Features of Shadow DB](#)
- [JAVA API: Shadow DB Configuration](#)

SQL-parser

Background

The SQL parser YAML configuration is readable and easy to use. The YAML files allow you to separate the code from the configuration, and easily modify the configuration file as needed.

Parameters

```
sqlParser:
  sqlStatementCache: # SQL statement local cache
    initialCapacity: # Initial capacity of local cache
    maximumSize: # Maximum capacity of local cache
  parseTreeCache: # Parse tree local cache
    initialCapacity: # Initial capacity of local cache
    maximumSize: # Maximum capacity of local cache
```

Procedure

1. Set local cache configuration.
2. Set parser configuration.
3. Use a parsing engine to parse SQL.

Sample

```
sqlParser:
  sqlStatementCache:
    initialCapacity: 2000
    maximumSize: 65535
  parseTreeCache:
    initialCapacity: 128
    maximumSize: 1024
```

Related References

- [JAVA API: SQL Parsing](#)

SQL Translator

Background

The SQL translator YAML configuration is readable and easy to use. The YAML files allow you to separate the code from the configuration, and easily modify the configuration file as needed.

Parameters

```
sqlTranslator:
  type: # SQL translator type
  useOriginalSQLWhenTranslatingFailed: # Whether use original SQL when translating
failed
```

Procedure

1. Set SQL translator type.
2. Set useOriginalSQLWhenTranslatingFailed to decide whether use original SQL when translating failed.

Sample

```
sqlTranslator:
  type: Native
  useOriginalSQLWhenTranslatingFailed: true
```

Related References

- [JAVA API: SQL Translator](#)

Mixed Rules

Background

ShardingSphere provides a variety of features, such as data sharding, read/write splitting, and data encryption. These features can be used independently or in combination. Below, you will find the parameters' explanation and configuration samples based on YAML.

Parameters

```
rules:
- !SHARDING
  tables:
    <logic_table_name>: # Logical table name:
      actualDataNodes: # consists of logical data source name plus table name (refer
to Inline syntax rules)
      tableStrategy: # Table shards strategy. The same as database shards strategy
standard:
        shardingColumn: # Sharding column name
        shardingAlgorithmName: # Sharding algorithm name
      keyGenerateStrategy:
        column: # Auto-increment column name. By default, the auto-increment primary
key generator is not used.
        keyGeneratorName: # Distributed sequence algorithm name
      defaultDatabaseStrategy:
        standard:
          shardingColumn: # Sharding column name
          shardingAlgorithmName: # Sharding algorithm name
      shardingAlgorithms:
        <sharding_algorithm_name>: # Sharding algorithm name
          type: INLINE
          props:
            algorithm-expression: # INLINE expression
          t_order_inline:
            type: INLINE
```

```

    props:
      algorithm-expression: # INLINE expression
  keyGenerators:
    <key_generate_algorithm_name> (+): # Distributed sequence algorithm name
      type: # Distributed sequence algorithm type
      props: # Property configuration of distributed sequence algorithm
- !ENCRYPT
  encryptors:
    <encrypt_algorithm_name> (+): # Encryption and decryption algorithm name
      type: # Encryption and decryption algorithm type
      props: # Encryption and decryption algorithm property configuration
    <encrypt_algorithm_name> (+): # Encryption and decryption algorithm name
      type: # Encryption and decryption algorithm type
  tables:
    <table_name>: # Encryption table name
      columns:
        <column_name> (+): # Encrypt logic column name
          cipher:
            name: # Cipher column name
            encryptorName: # Cipher encrypt algorithm name
          assistedQuery (?):
            name: # Assisted query column name
            encryptorName: # Assisted query encrypt algorithm name
          likeQuery (?):
            name: # Like query column name
            encryptorName: # Like query encrypt algorithm name

```

Samples

```

rules:
- !SHARDING
  tables:
    t_order:
      actualDataNodes: replica_ds_${0..1}.t_order_${0..1}
      tableStrategy:
        standard:
          shardingColumn: order_id
          shardingAlgorithmName: t_order_inline
      keyGenerateStrategy:
        column: order_id
        keyGeneratorName: snowflake
      defaultDatabaseStrategy:
        standard:
          shardingColumn: user_id
          shardingAlgorithmName: database_inline
      shardingAlgorithms:
        database_inline:

```



```

    type: INLINE
    props:
      algorithm-expression: replica_ds_${user_id % 2}
  t_order_inline:
    type: INLINE
    props:
      algorithm-expression: t_order_${order_id % 2}
  t_order_item_inline:
    type: INLINE
    props:
      algorithm-expression: t_order_item_${order_id % 2}
keyGenerators:
  snowflake:
    type: SNOWFLAKE
- !ENCRYPT
encryptors:
  aes_encryptor:
    type: AES
    props:
      aes-key-value: 123456abc
      digest-algorithm-name: SHA-1
  assisted_encryptor:
    type: MD5
  like_encryptor:
    type: CHAR_DIGEST_LIKE
tables:
  t_encrypt:
    columns:
      user_id:
        cipher:
          name: user_cipher
          encryptorName: aes_encryptor
      assistedQuery:
        name: assisted_query_user
        encryptorName: assisted_encryptor
      likeQuery:
        name: like_query_user
        encryptorName: like_encryptor
    order_id:
      cipher:
        name: order_cipher
        encryptorName: aes_encryptor

```

Cache for Sharding Route

Background

This feature is **experimental** and needs to be used with the data sharding rule. The cache for sharding route will put the logical SQL, the parameter value of the shard key, and the routing result into the cache, exchange space for time, and reduce CPU usage of the routing logic.

We recommend enabling it only if the following conditions are met: - Pure OLTP scenarios. - The CPU of the machine which deployed the ShardingSphere process has reached the bottleneck. - Most of the CPUs are used by ShardingSphere routing logic. - All SQLs are optimized and each SQL execution could be routed to a single data node.

If the above conditions are not met, the execution delay of SQL may not be significantly improved, and the memory pressure will be increased.

Parameters

```
rules:
- !SHARDING
  tables:
    shardingAlgorithms:
      # ...
    shardingCache:
      allowedMaxSqlLength: 512 # Allow cached SQL length limit
      routeCache:
        initialCapacity: 65536 # Initial capacity
        maximumSize: 262144 # Maximum capacity
        softValues: true # Whether to use soft references
```

Related References

- [Core Feature: Data Sharding](#)

Single Table

Background

Single rule is used to specify which single tables need to be managed by ShardingSphere, or to set the default single table data source.

Parameters

```
rules:
- !SINGLE
  tables:
    # MySQL style
    - ds_0.t_single # Load specified single table
    - ds_1.* # Load all single tables in the specified data source
    - "*.*" # Load all single tables
    # PostgreSQL style
    - ds_0.public.t_config
    - ds_1.public.*
    - ds_2.*.*
    - "*.*.*"
  defaultDataSource: ds_0 # The default data source is used when executing CREATE
  TABLE statement to create a single table. The default value is null, indicating
  random unicast routing.
```

Related References

- [Single Table](#)

SQL Federation

Background

This function is an **experimental one and is currently not suitable for use in core system production environments**. When multiple tables in a join query are distributed across different database instances, enabling federated query allows for cross-database join queries, as well as subqueries.

Parameters

```
sqlFederation:
  sqlFederationEnabled: # SQL federation enabled configuration
  allQueryUseSQLFederation: # all query use SQL federation configuration
  executionPlanCache: # execution plan cache configuration
    initialCapacity: 2000 # execution plan local cache initial capacity
    maximumSize: 65535 # execution plan local cache maximum size
```

Sample

```
sqlFederation:
  sqlFederationEnabled: true
  allQueryUseSQLFederation: false
  executionPlanCache:
    initialCapacity: 2000
    maximumSize: 65535
```

Related References

- [JAVA API: SQL Federation](#)

Algorithm

Sharding

```
shardingAlgorithms:
  # algorithmName is specified by users, and its property has to be consistent with
  # that of shardingAlgorithmName in the sharding strategy.
  <algorithmName>:
    # type and props, please refer to the built-in sharding algorithm: https://
    # shardingsphere.apache.org/document/current/en/user-manual/common-config/builtin-
    # algorithm/sharding/
    type: xxx
    props:
      xxx: xxx
```

Encryption

```
encryptors:
  # encryptorName is specified by users, and its property should be consistent with
  # that of encryptorName in encryption rules.
  <encryptorName>:
    # type and props, please refer to the built-in encryption algorithm: https://
    # shardingsphere.apache.org/document/current/en/user-manual/common-config/builtin-
    # algorithm/encrypt/
    type: xxx
    props:
      xxx: xxx
```

Read/Write Splitting Load Balancer

```
loadBalancers:
  # loadBalancerName is specified by users, and its property has to be consistent
  # with that of loadBalancerName in read/write splitting rules.
  # type and props, please refer to the built-in read/write splitting algorithm
  load balancer: https://shardingsphere.apache.org/document/current/en/user-manual/
  common-config/builtin-algorithm/load-balance/
  type: xxx
  props:
    xxx: xxx
```

Shadow DB

```
shadowAlgorithms:
  # shadowAlgorithmName is specified by users, and its property has to be consistent
  # with that of shadowAlgorithmNames in shadow DB rules.
  <shadowAlgorithmName>:
    # type and props, please refer to the built-in shadow DB algorithm: https://
    shardingsphere.apache.org/document/current/en/user-manual/common-config/builtin-
    algorithm/shadow/
    type: xxx
    props:
      xxx: xxx
```

High Availability

```
discoveryTypes:
  # discoveryTypeName is specified by users, and its property has to be consistent
  # with that of discoveryTypeName in the database discovery rules.
  type: xxx
  props:
    xxx: xxx
```

Data Masking

```
maskAlgorithms:
  # maskAlgorithmName is specified by users, and its property should be consistent
  # with that of maskAlgorithm in mask rules.
  <maskAlgorithmName>:
    # type and props, please refer to the built-in mask algorithm: https://
    shardingsphere.apache.org/document/current/en/user-manual/common-config/builtin-
    algorithm/mask/
```

```

type: xxx
props:
  xxx: xxx

```

JDBC Driver

Background

ShardingSphere-JDBC provides a JDBC Driver, which can be used only through configuration changes without rewriting the code.

Parameters

Driver Class Name

org.apache.shardingsphere.driver.ShardingSphereDriver

URL Configuration and sample

Refer to [known Implementation](#).

Procedure

1. Import Maven Dependency

```

<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>

```

2. Use drive

- Use native drivers:

```

Class.forName("org.apache.shardingsphere.driver.ShardingSphereDriver");
String jdbcUrl = "jdbc:shardingsphere:classpath:config.yaml";

String sql = "SELECT i.* FROM t_order o JOIN t_order_item i ON o.order_id=i.order_id
WHERE o.user_id=? AND o.order_id=?";
try (
    Connection conn = DriverManager.getConnection(jdbcUrl);
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 10);
    ps.setInt(2, 1000);
}

```

```

    try (ResultSet rs = preparedStatement.executeQuery()) {
        while(rs.next()) {
            // ...
        }
    }
}

```

- Use database connection pool:

```

String driverClassName = "org.apache.shardingsphere.driver.ShardingSphereDriver";
String jdbcUrl = "jdbc:shardingsphere:classpath:config.yaml";

// Take HikariCP as an example
HikariDataSource dataSource = new HikariDataSource();
dataSource.setDriverClassName(driverClassName);
dataSource.setJdbcUrl(jdbcUrl);

String sql = "SELECT i.* FROM t_order o JOIN t_order_item i ON o.order_id=i.order_id
WHERE o.user_id=? AND o.order_id=?";
try (
    Connection conn = dataSource.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 10);
    ps.setInt(2, 1000);
    try (ResultSet rs = preparedStatement.executeQuery()) {
        while(rs.next()) {
            // ...
        }
    }
}
}

```

Known Implementation

Background Information

For the driver class of `org.apache.shardingsphere.driver.ShardingSphereDriver`, by implementing the SPI of `org.apache.shardingsphere.infra.url.ShardingSphereURLLoader`, allows YAML configuration files to be fetched from multiple sources and File Systems and parsed into ShardingSphere. If there is no specific statement, the following implementations all use YAML 1.1 as the YAML writing specification. This does not prevent custom implementations of `org.apache.shardingsphere.infra.url.ShardingSphereURLLoader` from being manually converted to YAML from files such as XML or JSON.

After parsing and loading the YAML file into ShardingSphere's metadata, the next behavior will be determined again through the relevant configuration of [Mode Configuration](#). Discuss two situations,

1. ShardingSphere's metadata does not exist in the Metadata Repository, and local metadata will

be stored in the Metadata Repository.

2. The metadata of ShardingSphere already exists in the Metadata Repository. Regardless of whether it is the same as the local metadata, the local metadata will be overwritten by the metadata of the Metadata Repository.

For the configuration of the Metadata Repository, please refer to [Metadata Repository](#).

How to load configuration files

Load configuration files from classpath

The configuration file is `xxx.yaml`. When `placeholder-type` is none or is not specified, the configuration file format is consistent with [YAML configuration](#). When `placeholder-type` exists and is not none, the configuration file format is defined in the JDBC URL Parameters section of this article.

Example:

- `jdbc:shardingsphere:classpath:config.yaml`
- `jdbc:shardingsphere:classpath:config.yaml?placeholder-type=none`
- `jdbc:shardingsphere:classpath:config.yaml?placeholder-type=environment`
- `jdbc:shardingsphere:classpath:config.yaml?placeholder-type=system_props`

Load configuration file from absolute path

The configuration file is `xxx.yaml`. When `placeholder-type` is none or is not specified, the configuration file format is consistent with [YAML configuration](#). When `placeholder-type` exists and is not none, the configuration file format is defined in the JDBC URL Parameters section of this article.

Example:

- `jdbc:shardingsphere:absolutepath:/path/to/config.yaml`
- `jdbc:shardingsphere:absolutepath:/path/to/config.yaml?placeholder-type=none`
- `jdbc:shardingsphere:absolutepath:/path/to/config.yaml?placeholder-type=environment`
- `jdbc:shardingsphere:absolutepath:/path/to/config.yaml?placeholder-type=system_props`

JDBC URL parameters

For implementations of `org.apache.shardingsphere.infra.url.ShardingSphereURLLoader`, not all JDBC URL parameters must be parsed, this involves how to implement `org.apache.shardingsphere.infra.url.ShardingSphereURLLoader.load()`.

placeholder-type

There is a `placeholder-type` attribute for optional loading of configuration files containing dynamic placeholders. The default value of `placeholder-type` is `none`. When `placeholder-type` is set to something other than `none`, allows setting the value of specific YAML properties via dynamic placeholders in the involved YAML file, and configuring optional default values. The name of a dynamic placeholder and its optional default value are separated by `::` and wrapped in the outermost layer by `$$ {` and `}`.

Discuss two situations,

1. When the corresponding dynamic placeholder value does not exist, the value of this YAML attribute will be set to the default value on the right side of `::`.
2. When neither the corresponding dynamic placeholder value nor the default value on the right side of `::` exists, this attribute will be set to empty.

Single dynamic placeholder

none

The configuration file is `xxx.yaml`, and the configuration file format is consistent with [YAML configuration](#).

Example:

- `jdbc:shardingsphere:classpath:config.yaml`
- `jdbc:shardingsphere:classpath:config.yaml?placeholder-type=none`
- `jdbc:shardingsphere:absolutePath:/path/to/config.yaml`
- `jdbc:shardingsphere:absolutePath:/path/to/config.yaml?placeholder-type=none`

environment

When loading a configuration file containing environment variables, users need to set `placeholder-type` to `environment`, which is commonly used in Docker Image deployment scenarios. The configuration file is `xxx.yaml`, and the configuration file format is basically the same as [YAML configuration](#).

Assume that the following set of environment variables exists,

1. The existing environment variable `FIXTURE_JDBC_URL` is `jdbc:h2:mem:foo_ds_1;DB_CLOSE_DELAY=-1;DATABASE_TO_UPPER=false;MODE=MySQL`.
2. The existing environment variable `FIXTURE_USERNAME` is `sa`.

Then for the intercepted fragment of the following YAML file,

```
ds_1:
  dataSourceClassName: com.zaxxer.hikari.HikariDataSource
  driverClassName: ${FIXTURE_DRIVER_CLASS_NAME::org.h2.Driver}
  jdbcUrl: ${FIXTURE_JDBC_URL::jdbc:h2:mem:foo_ds_do_not_use}
  username: ${FIXTURE_USERNAME::}
  password: ${FIXTURE_PASSWORD::}
```

This YAML snippet will be parsed as,

```
ds_1:
  dataSourceClassName: com.zaxxer.hikari.HikariDataSource
  driverClassName: org.h2.Driver
  jdbcUrl: jdbc:h2:mem:foo_ds_1;DB_CLOSE_DELAY=-1;DATABASE_TO_UPPER=false;MODE=MySQL
  username: sa
  password:
```

Example:

- `jdbc:shardingsphere:classpath:config.yaml?placeholder-type=environment`
- `jdbc:shardingsphere:absolutepath:/path/to/config.yaml?placeholder-type=environment`

system_props

When loading a configuration file containing system properties, users need to set `placeholder-type` to `system_props`. The configuration file is `xxx.yaml`, and the configuration file format is basically the same as [YAML configuration](#).

Assume the following set of system properties exists,

1. The existing system property `fixture.config.driver.jdbc-url` is `jdbc:h2:mem:foo_ds_1;DB_CLOSE_DELAY=-1;DATABASE_TO_UPPER=false;MODE=MySQL`.
2. The existing system property `fixture.config.driver.username` is `sa`.

Then for the intercepted fragment of the following YAML file,

```
ds_1:
  dataSourceClassName: com.zaxxer.hikari.HikariDataSource
  driverClassName: ${fixture.config.driver.driver-class-name:org.h2.Driver}
  jdbcUrl: ${fixture.config.driver.jdbc-url:jdbc:h2:mem:foo_ds_do_not_use}
  username: ${fixture.config.driver.username:}
  password: ${fixture.config.driver.password:}
```

This YAML snippet will be parsed as,

```
ds_1:
  dataSourceClassName: com.zaxxer.hikari.HikariDataSource
  driverClassName: org.h2.Driver
  jdbcUrl: jdbc:h2:mem:foo_ds_1;DB_CLOSE_DELAY=-1;DATABASE_TO_UPPER=false;MODE=MySQL
  username: sa
  password:
```

In real situations, system variables are usually defined dynamically. Assume that none of the above system variables are defined, and there is a YAML file `config.yaml` containing the above YAML interception fragment, users can refer to the following method to create a `DataSource` instance using the HikariCP Java API.

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;

import javax.sql.DataSource;

public class ExampleUtils {
    public DataSource createDataSource() {
        HikariConfig config = new HikariConfig();
        config.setDriverClassName("org.apache.shardingsphere.driver.
ShardingSphereDriver");
        config.setJdbcUrl("jdbc:shardingsphere:classpath:config.yaml?placeholder-
type=system_props");
        try {
            assert null == System.getProperty("fixture.config.driver.jdbc-url");
            assert null == System.getProperty("fixture.config.driver.username");
            System.setProperty("fixture.config.driver.jdbc-url", "jdbc:h2:mem:foo_
ds_1;DB_CLOSE_DELAY=-1;DATABASE_TO_UPPER=false;MODE=MySQL");
            System.setProperty("fixture.config.driver.username", "sa");
            return new HikariDataSource(config);
        } finally {
            System.clearProperty("fixture.config.driver.jdbc-url");
            System.clearProperty("fixture.config.driver.username");
        }
    }
}
```

Example:

- jdbc:shardingsphere:classpath:config.yaml?placeholder-type=system_props
- jdbc:shardingsphere:absolutePath:/path/to/config.yaml?
placeholder-type=system_props

multiple dynamic placeholders

On top of a single dynamic placeholder, users can use multiple dynamic placeholders in a single line of YAML. When configuring the value of a YAML attribute, if part of the value of the YAML attribute needs to be replaced dynamically, you can implement this by configuring multiple dynamic placeholders.

Assume the following set of environment variables or system properties exists,

1. The existing environment variable or system property FIXTURE_HOST is 127.0.0.1。
2. The existing environment variable or system property FIXTURE_PORT is 3306。
3. The existing environment variable or system property FIXTURE_DATABASE is test。
4. The existing environment variable or system property FIXTURE_USERNAME is sa。

Then for the intercepted fragment of the following YAML file,

```
ds_1:
  dataSourceClassName: com.zaxxer.hikari.HikariDataSource
  driverClassName: ${FIXTURE_DRIVER_CLASS_NAME::com.mysql.cj.jdbc.Driver}
  jdbcUrl: jdbc:mysql://${FIXTURE_HOST::}:${FIXTURE_PORT::}/${FIXTURE_DATABASE::}
?sslMode=REQUIRED
  username: ${FIXTURE_USERNAME::}
  password: ${FIXTURE_PASSWORD::}
```

This YAML snippet will be parsed as,

```
ds_1:
  dataSourceClassName: com.zaxxer.hikari.HikariDataSource
  driverClassName: com.mysql.cj.jdbc.Driver
  jdbcUrl: jdbc:mysql://127.0.0.1:3306/test?sslMode=REQUIRED
  username: sa
  password:
```

Other implementations

For details, please refer to <https://github.com/apache/shardingsphere-plugin>.

Spring Boot

Overview

ShardingSphere provides a JDBC driver, and developers can configure ShardingSphereDriver in Spring Boot to use ShardingSphere.

Usage

Import Maven Dependency

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>
```

Configure Spring Boot Properties

```
# Configuring DataSource Drivers
spring.datasource.driver-class-name=org.apache.shardingsphere.driver.
ShardingSphereDriver
# Specify a YAML configuration file
spring.datasource.url=jdbc:shardingsphere:classpath:xxx.yaml
```

The YAML configuration file in ‘spring.datasource.url’ currently support in multiple ways, refer to [Known Implementation](#).

Use Data Source

Use this data source directly; or configure ShardingSphereDataSource to be used in conjunction with ORM frameworks such as JPA, Hibernate, and MyBatis.

Handling for Spring Boot OSS 3

Spring Boot OSS 3 has made a “big bang” upgrade to Jakarta EE and Java 17, with all complications involved.

ShardingSphere’s XA distributed transactions are not yet ready on Spring Boot OSS 3. This limitation also applies to other Jakarta EE 9+ based Web Frameworks, such as Quarkus 3, Micronaut Framework 4 and Helidon 3.

Users only need to configure as follows.

```
<project>
  <dependencies>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-jdbc</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
  </dependencies>
</project>
```

Special handling for earlier versions of Spring Boot OSS 2

All features of ShardingSphere are available on Spring Boot OSS 2, but earlier versions of Spring Boot OSS may require manually specifying version 2.2 for SnakeYAML. This is reflected in Maven's `pom.xml` as follows.

```
<project>
  <dependencies>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-jdbc</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
    <dependency>
      <groupId>org.yaml</groupId>
      <artifactId>snakeyaml</artifactId>
      <version>2.2</version>
    </dependency>
  </dependencies>
</project>
```

If the user created the Spring Boot project from <https://start.spring.io/>, users can simplify configuration by following things.

```
<project>
  <properties>
    <snakeyaml.version>2.2</snakeyaml.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-jdbc</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
  </dependencies>
</project>
```

Spring Namespace

Overview

ShardingSphere provides a JDBC driver. To use ShardingSphere, developers can configure ShardingSphereDriver in Spring.

Operation

Import Maven Dependency

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>
```

Configure Spring Bean

Configuration Item Explanation

Name	Type	Description
driverClass	Attribute	Database Driver, need to use ShardingSphereDriver
url	Attribute	YAML configuration file path

Example

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="shardingDataSource" class="com.zaxxer.hikari.HikariDataSource">
    <property name="driverClass" value="org.apache.shardingsphere.driver.
ShardingSphereDriver" />
    <property name="url" value="jdbc:shardingsphere:classpath:xxx.yaml" />
  </bean>
</beans>
```

Use Data Source

Same with Spring Boot.

9.1.2 Java API

Overview

Java API is the basic configuration methods in ShardingSphere-JDBC, and other configurations will eventually be transformed into Java API configuration methods.

The Java API is the most complex and flexible configuration method, which is suitable for the scenarios requiring dynamic configuration through programming.

Usage

Import Maven Dependency

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>
```

Create Data Source

ShardingSphere-JDBC Java API consists of database name, mode configuration, data source map, rule configurations and properties.

The `ShardingSphereDataSource` created by `ShardingSphereDataSourceFactory` implements the standard JDBC `DataSource` interface.

```
String databaseName = "foo_schema"; // Indicate logic database name
ModeConfiguration modeConfig = ... // Build mode configuration
Map<String, DataSource> dataSourceMap = ... // Build actual data sources
Collection<RuleConfiguration> ruleConfigs = ... // Build concentrate rule
configurations
Properties props = ... // Build properties
DataSource dataSource = ShardingSphereDataSourceFactory.
createDataSource(databaseName, modeConfig, dataSourceMap, ruleConfigs, props);
```

Please refer to [Mode Configuration](#) for more mode details.

Please refer to [Data Source Configuration](#) for more data source details.

Please refer to [Rules Configuration](#) for more rule details.

Use Data Source

Developer can choose to use native JDBC or ORM frameworks such as JPA, Hibernate or MyBatis through the DataSource.

Take native JDBC usage as an example:

```
// Create ShardingSphereDataSource
DataSource dataSource = ShardingSphereDataSourceFactory.
createDataSource(databaseName, modeConfig, dataSourceMap, ruleConfigs, props);

String sql = "SELECT i.* FROM t_order o JOIN t_order_item i ON o.order_id=i.order_id
WHERE o.user_id=? AND o.order_id=?";
try (
    Connection conn = dataSource.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 10);
    ps.setInt(2, 1000);
    try (ResultSet rs = preparedStatement.executeQuery()) {
        while(rs.next()) {
            // ...
        }
    }
}
```

Mode

Background

Build the running mode through Java API.

Parameters

Class name: org.apache.shardingsphere.infra.config.mode.ModeConfiguration

Attributes:

Name *	Data Type	Description	Default Value
type	String	Type of mode configurationValues could be: Standalone or Cluster	Standalone
repository	PersistRepositoryConfiguration	Persist repository configurationStandalone type uses StandalonePersistRepositoryConfigurationCluster type uses ClusterPersistRepositoryConfiguration	

Standalone Persist Configuration

Class name: org.apache.shardingsphere.mode.repository.standalone.StandalonePersistRepositoryConfiguration

Attributes:

Name	Data Type	Description
type	String	Type of persist repository
props	Properties	Properties of persist repository

Cluster Persist Configuration

Class name: org.apache.shardingsphere.mode.repository.cluster.ClusterPersistRepositoryConfiguration

Attributes:

Name	Data Type	Description
type	String	Type of persist repository
namespace	String	Namespace of registry center
server-lists	String	Server lists of registry center
props	Properties	Properties of persist repository

Notes

1. Cluster mode deployment is recommended for production environment.
2. The ZooKeeper registry center is recommended for cluster mode deployment.
3. If there is configuration information in the ZooKeeper, please refer to the config information there.

Procedure

Introduce Maven Dependency

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${latest.release.version}</version>
</dependency>
```

Notice: Please change `${latest.release.version}` to the actual version.

Sample

Standalone Mode

```
ModeConfiguration modeConfig = createModeConfiguration();
Map<String, DataSource> dataSourceMap = ... // Building real data sources
Collection<RuleConfiguration> ruleConfigs = ... // Build specific rules
Properties props = ... // Build property configuration
DataSource dataSource = ShardingSphereDataSourceFactory.
createDataSource(databaseName, modeConfig, dataSourceMap, ruleConfigs, props);

private ModeConfiguration createModeConfiguration() {
    return new ModeConfiguration("Standalone", new
    StandalonePersistRepositoryConfiguration("JDBC", new Properties()));
}
```

Cluster Mode (Recommended)

```
ModeConfiguration modeConfig = createModeConfiguration();
Map<String, DataSource> dataSourceMap = ... // Building real data sources
Collection<RuleConfiguration> ruleConfigs = ... // Build specific rules
Properties props = ... // Build property configuration
DataSource dataSource = ShardingSphereDataSourceFactory.
createDataSource(databaseName, modeConfig, dataSourceMap, ruleConfigs, props);
```

```
private ModeConfiguration createModeConfiguration() {
    return new ModeConfiguration("Cluster", new
ClusterPersistRepositoryConfiguration("ZooKeeper", "governance-sharding-db",
"localhost:2181", new Properties()));
}
```

Related References

- [Installation and Usage of ZooKeeper Registry Center](#)
- Please refer to [Builtin Persist Repository List](#) for more details about type of repository.

Data Source

Background

ShardingSphere-JDBC supports all database JDBC drivers and connection pools.

This section describes how to configure data sources through the JAVA API.

Procedure

1. Import Maven dependency.

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${latest.release.version}</version>
</dependency>
```

Notice: Please change `${latest.release.version}` to the actual version.

Sample

```
ModeConfiguration modeConfig = // Build running mode
Map<String, DataSource> dataSourceMap = createDataSources();
Collection<RuleConfiguration> ruleConfigs = ... // Build specific rules
Properties props = ... // Build attribute configuration
DataSource dataSource = ShardingSphereDataSourceFactory.
createDataSource(databaseName, modeConfig, dataSourceMap, ruleConfigs, props);

private Map<String, DataSource> createDataSources() {
    Map<String, DataSource> dataSourceMap = new HashMap<>();
    // Configure the 1st data source
```

```
HikariDataSource dataSource1 = new HikariDataSource();
dataSource1.setDriverClassName("com.mysql.jdbc.Driver");
dataSource1.setJdbcUrl("jdbc:mysql://localhost:3306/ds_1");
dataSource1.setUsername("root");
dataSource1.setPassword("");
dataSourceMap.put("ds_1", dataSource1);

// Configure the 2nd data source
HikariDataSource dataSource2 = new HikariDataSource();
dataSource2.setDriverClassName("com.mysql.jdbc.Driver");
dataSource2.setJdbcUrl("jdbc:mysql://localhost:3306/ds_2");
dataSource2.setUsername("root");
dataSource2.setPassword("");
dataSourceMap.put("ds_2", dataSource2);
}
```

Rules

Rules are pluggable part of Apache ShardingSphere. This chapter is a java rule configuration manual for ShardingSphere-JDBC.

Sharding

Background

The Java API rule configuration for data sharding, which allows users to create ShardingSphereDataSource objects directly by writing Java code, is flexible enough to integrate various types of business systems without relying on additional jar packages.

Parameters

Root Configuration

Class name: org.apache.shardingsphere.sharding.api.config.ShardingRuleConfiguration

Attributes:

Name	DataType	Description	Default Value
tables (+)	Collection<ShardingTableRuleConfiguration>	Sharding table rules	.
autoTables (+)	Collection<ShardingAutoTableRuleConfiguration>	Sharding auto table rules	.
bindingTableGroups (*)	Collection<String>	Binding table rules	Empty
defaultDatabaseShardingStrategy (?)	ShardingStrategyConfiguration	Default database sharding strategy	Not sharding
defaultTableShardingStrategy (?)	ShardingStrategyConfiguration	Default table sharding strategy	Not sharding
defaultKeyGenerateStrategy (?)	KeyGeneratorConfiguration	Default key generator	Snowflake
defaultAuditStrategy (?)	ShardingAuditStrategyConfiguration	Default key auditor	DML_SHARDING_CONDITIONS
defaultShardingColumn (?)	String	Default sharding column name	None
shardingAlgorithms (+)	Map<String, AlgorithmConfiguration>	Sharding algorithm name and configurations	None
keyGenerators (?)	Map<String, AlgorithmConfiguration>	Key generate algorithm name and configurations	None
auditors (?)	Map<String, AlgorithmConfiguration>	Sharding audit algorithm name and configurations	None

Sharding Table Configuration

Class name: org.apache.shardingsphere.sharding.api.config.ShardingTableRuleConfiguration

Attributes:

Name*	Data Type	Description	Default Value
logicTable	String	Name of sharding logic table	.
actualDataNodes (?)	String	Describe data source names and actual tables, delimiter as point. Multiple data nodes split by comma, support inline expression	Broadcast table or databases sharding only
databaseShardingStrategy (?)	ShardingStrategyConfiguration	Databases sharding strategy	Use default databases sharding strategy
tableShardingStrategy (?)	ShardingStrategyConfiguration	Tables sharding strategy	Use default tables sharding strategy
keyGeneratorStrategy (?)	KeyGeneratorConfiguration	Key generator configuration	Use default key generator
auditStrategy (?)	ShardingAuditStrategyConfiguration	Sharding audit strategy configuration	Use default auditor

Sharding Auto Table Configuration

Class name: org.apache.shardingsphere.sharding.api.config.ShardingAutoTableRuleConfiguration

Attributes:

Name	Data Type	Description	Default Value
logicTable	String	Name of sharding logic table	.
actualDataSources (?)	String	Data source names. Multiple data nodes split by comma	Use all configured data sources
shardingStrategy (?)	ShardingStrategyConfiguration	Sharding strategy	Use default sharding strategy
keyGeneratorStrategy (?)	KeyGeneratorConfiguration	Key generator configuration	Use default key generator

Sharding Strategy Configuration

Standard Sharding Strategy Configuration

Class name: org.apache.shardingsphere.sharding.api.config.strategy.sharding.StandardShardingStrategyConfiguration

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
shardingColumn	String	Sharding column name
shardingAlgorithmName	String	Sharding algorithm name

Complex Sharding Strategy Configuration

Class name: org.apache.shardingsphere.sharding.api.config.strategy.sharding.ComplexShardingStrategyConfiguration

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
shardingColumns	String	Sharding column name, separated by commas
shardingAlgorithmName	String	Sharding algorithm name

Hint Sharding Strategy Configuration

Class name: org.apache.shardingsphere.sharding.api.config.strategy.sharding.HintShardingStrategyConfiguration

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
shardingAlgorithmName	String	Sharding algorithm name

None Sharding Strategy Configuration

Class name: org.apache.shardingsphere.sharding.api.config.strategy.sharding.NoneShardingStrategyConfiguration

Attributes: None

Please refer to [Built-in Sharding Algorithm List](#) for more details about type of algorithm.

Distributed Key Strategy Configuration

Class name: org.apache.shardingsphere.sharding.api.config.strategy.keygen.KeyGenerateStrategyConfiguration

Attributes:

Name	DataType	Description
column	String	Column name of key generate
keyGeneratorName	String	key generate algorithm name

Please refer to [Built-in Key Generate Algorithm List](#) for more details about type of algorithm.

Sharding audit Strategy Configuration

Class name: org.apache.shardingsphere.sharding.api.config.strategy.audit.ShardingAuditStrategyConfiguration

Attributes:

Name	DataType	Description
auditorNames	Collection<String>	Sharding audit algorithm name
allowHintDisable	Boolean	Enable or disable sharding audit hint

Please refer to [Built-in Sharding Audit Algorithm List](#) for more details about type of algorithm.

Procedure

1. Create an authentic data source mapping relationship, with key as the logical name of the data source and value as the DataSource object.
2. Create the sharding rule object ShardingRuleConfiguration, and initialize the sharding table objects—ShardingTableRuleConfiguration, the set of bound tables, the set of broadcast tables, and parameters like library sharding strategy and the database sharding strategy, on which the data sharding depends.
3. Using the ShardingSphereDataSource method of calling the ShardingSphereDataSourceFactory subject to create the ShardingSphereDataSource.

Sample

```

public final class ShardingDatabasesAndTablesConfigurationPrecise {

    @Override
    public DataSource getDataSource() throws SQLException {
        return ShardingSphereDataSourceFactory.
createDataSource(createDataSourceMap(), Arrays.
asList(createShardingRuleConfiguration(), createBroadcastRuleConfiguration()), new
Properties());
    }

    private ShardingRuleConfiguration createShardingRuleConfiguration() {
        ShardingRuleConfiguration result = new ShardingRuleConfiguration();
        result.getTables().add(getOrderTableRuleConfiguration());
        result.getTables().add(getOrderItemTableRuleConfiguration());
        result.getBindingTableGroups().add(new
ShardingTableReferenceRuleConfiguration("foo", "t_order, t_order_item"));
        result.setDefaultDatabaseShardingStrategy(new
StandardShardingStrategyConfiguration("user_id", "inline"));
        result.setDefaultTableShardingStrategy(new
StandardShardingStrategyConfiguration("order_id", "standard_test_tbl"));
        Properties props = new Properties();
        props.setProperty("algorithm-expression", "demo_ds_${user_id % 2}");
        result.getShardingAlgorithms().put("inline", new AlgorithmConfiguration(
"INLINE", props));
        result.getShardingAlgorithms().put("standard_test_tbl", new
AlgorithmConfiguration("STANDARD_TEST_TBL", new Properties()));
        result.getKeyGenerators().put("snowflake", new AlgorithmConfiguration(
"SNOWFLAKE", new Properties()));
        result.getAuditors().put("sharding_key_required_auditor", new
AlgorithmConfiguration("DML_SHARDING_CONDITIONS", new Properties()));
        return result;
    }

    private ShardingTableRuleConfiguration getOrderTableRuleConfiguration() {
        ShardingTableRuleConfiguration result = new ShardingTableRuleConfiguration(
"t_order", "demo_ds_${0..1}.t_order_${[0, 1]}");
        result.setKeyGenerateStrategy(new KeyGenerateStrategyConfiguration("order_id
", "snowflake"));
        result.setAuditStrategy(new ShardingAuditStrategyConfiguration(Collections.
singleton("sharding_key_required_auditor"), true));
        return result;
    }

    private ShardingTableRuleConfiguration getOrderItemTableRuleConfiguration() {
        ShardingTableRuleConfiguration result = new ShardingTableRuleConfiguration(
"t_order_item", "demo_ds_${0..1}.t_order_item_${[0, 1]}");
    }

```

```
        result.setKeyGenerateStrategy(new KeyGenerateStrategyConfiguration("order_
item_id", "snowflake"));
        return result;
    }

    private Map<String, DataSource> createDataSourceMap() {
        Map<String, DataSource> result = new HashMap<>();
        result.put("demo_ds_0", DataSourceUtil.createDataSource("demo_ds_0"));
        result.put("demo_ds_1", DataSourceUtil.createDataSource("demo_ds_1"));
        return result;
    }

    private BroadcastRuleConfiguration createBroadcastRuleConfiguration() {
        return new BroadcastRuleConfiguration(Collections.singletonList("t_address
"));
    }
}
```

Related References

- [Core Feature: Data Sharding](#)
- [Developer Guide: Data Sharding](#)

Broadcast Table

Background

The Java API rule configuration for broadcast, which allows users to create `ShardingSphereDataSource` objects directly by writing Java code, is flexible enough to integrate various types of business systems without relying on additional jar packages.

Parameters

Class: `org.apache.shardingsphere.broadcast.config.BroadcastRuleConfiguration`

Attributes:

<i>name</i>	<i>DataType</i>	<i>Description</i>	<i>Default Value</i>
tables (+)	Collection<String>	Broadcast table rules	

Sample

The following is an example of the broadcast table Java API configuration:

```
public final class ShardingDatabasesAndTablesConfigurationPrecise {

    @Override
    public DataSource getDataSource() throws SQLException {
        return ShardingSphereDataSourceFactory.
createDataSource(createDataSourceMap(), Arrays.
asList(createBroadcastRuleConfiguration(), new Properties()));
    }

    private Map<String, DataSource> createDataSourceMap() {
        Map<String, DataSource> result = new HashMap<>();
        result.put("demo_ds_0", DataSourceUtil.createDataSource("demo_ds_0"));
        result.put("demo_ds_1", DataSourceUtil.createDataSource("demo_ds_1"));
        return result;
    }

    private BroadcastRuleConfiguration createBroadcastRuleConfiguration() {
        return new BroadcastRuleConfiguration(Collections.singletonList("t_address
"));
    }
}
```

Related References

- [YAML Configuration: Broadcast](#)

Readwrite-splitting

Background

The read/write splitting configured in Java API form can be easily applied to various scenarios without relying on additional jar packages. Users only need to construct the read/write splitting data source through java code to be able to use the read/write splitting function.

Parameters Explained

Entry

Class name: org.apache.shardingsphere.readwritesplitting.config.ReadwriteSplittingRuleConfiguration

Configurable Properties:

Name	Data Type	Description
dataSources (+)	Collection<ReadwriteSplittingDataSourceRuleConfiguration>	Data sources of write and reads
loadBalancer (*)	Map<String, AlgorithmConfiguration>	Load balance algorithm name and configurations of replica data sources

Primary-secondary Data Source Configuration

Class name: org.apache.shardingsphere.readwritesplitting.config.rule.ReadwriteSplittingDataSourceGroupRuleConfiguration

Configurable Properties:

Name	Data Type	Description	Default Value*
name	String	Readwrite-splitting data source name	.
writeDataSourceName	String	Write data source name	.
readDataSourceNames	List<String>	Read data sources list	.
transactionalReadQueryStrategy (?)	TransactionalReadQueryStrategy	Routing strategy for read query within a transaction, values include: PRIMARY (to primary), FIXED (to fixed data source), DYNAMIC (to any data source)	DYNAMIC
loadBalancerName (?)	String	Load balance algorithm name of replica sources	Round robin load balance algorithm

Please refer to [Built-in Load Balance Algorithm List](#) for details on algorithm types.

Operating Procedures

1. Add read-write splitting data source
2. Set load balancing algorithms
3. Use read-write splitting data source

Configuration Examples

```
public DataSource getDataSource() throws SQLException {
    ReadwriteSplittingDataSourceRuleConfiguration dataSourceConfig = new
    ReadwriteSplittingDataSourceRuleConfiguration(
        "demo_read_query_ds", "demo_write_ds", Arrays.asList("demo_read_ds_0",
        "demo_read_ds_1"), "demo_weight_lb");
    Properties algorithmProps = new Properties();
    algorithmProps.setProperty("demo_read_ds_0", "2");
    algorithmProps.setProperty("demo_read_ds_1", "1");
    Map<String, AlgorithmConfiguration> algorithmConfigMap = new HashMap<>(1);
    algorithmConfigMap.put("demo_weight_lb", new AlgorithmConfiguration("WEIGHT", algorithmProps));
    ReadwriteSplittingRuleConfiguration ruleConfig = new
    ReadwriteSplittingRuleConfiguration(Collections.singleton(dataSourceConfig),
    algorithmConfigMap);
    Properties props = new Properties();
    props.setProperty("sql-show", Boolean.TRUE.toString());
    return ShardingSphereDataSourceFactory.
    createDataSource(createDataSourceMap(), Collections.singleton(ruleConfig), props);
}

private Map<String, DataSource> createDataSourceMap() {
    Map<String, DataSource> result = new HashMap<>(3, 1);
    result.put("demo_write_ds", DataSourceUtil.createDataSource("demo_write_ds"));
    result.put("demo_read_ds_0", DataSourceUtil.createDataSource("demo_read_ds_0"));
    result.put("demo_read_ds_1", DataSourceUtil.createDataSource("demo_read_ds_1"));
    return result;
}
```

References

- [Read-write splitting-Core features](#)
- [YAML Configuration: read-write splitting](#)

Distributed Transaction

Root Configuration

org.apache.shardingsphere.transaction.config.TransactionRuleConfiguration

Attributes:

<i>name</i>	<i>DataType</i>	<i>Description</i>
defaultType	String	Default transaction type
providerType (?)	String	Transaction provider type
props (?)	Properties	Transaction properties

Encryption

Background

The data encryption Java API rule configuration allows users to directly create ShardingSphereDataSource objects by writing java code. The Java API configuration method is very flexible and can integrate various types of business systems without relying on additional jar packages.

Parameters

Root Configuration

Class name: org.apache.shardingsphere.encrypt.config.EncryptRuleConfiguration

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>	<i>Default Value</i>
tables (+)	Collection<EncryptTableRuleConfiguration>	Encrypt table rule configurations	
encryptors (+)	Map<String, Algorithm Configuration>	Encrypt algorithm name and configurations	

Encrypt Table Rule Configuration

Class name: org.apache.shardingsphere.encrypt.config.rule.EncryptTableRuleConfiguration

Attributes:

Name	DataType	Description
name	String	Table name
columns (+)	Collection<EncryptColumnRuleConfiguration>	Encrypt column rule configurations

Encrypt Column Rule Configuration

Class name: org.apache.shardingsphere.encrypt.config.rule.EncryptColumnRuleConfiguration

Attributes:

Name	DataType	Description
name	String	Logic column name
cipher	EncryptColumnItemRuleConfiguration	Cipher column config
assistedQuery (?)	EncryptColumnItemRuleConfiguration	Assisted query column config
likeQuery (?)	EncryptColumnItemRuleConfiguration	Like query column config

Encrypt Column Item Rule Configuration

Class name: org.apache.shardingsphere.encrypt.config.rule.EncryptColumnItemRuleConfiguration

Attributes:

Name	DataType	Description
name	String	encrypt column item name
encryptorName	String	encryptor name

Encrypt Algorithm Configuration

Class name: org.apache.shardingsphere.infra.algorithm.core.config.AlgorithmConfiguration

Attributes:

Name	DataType	Description
name	String	Encrypt algorithm name
type	String	Encrypt algorithm type
properties	Properties	Encrypt algorithm properties

Please refer to [Built-in Encrypt Algorithm List](#) for more details about type of algorithm.

Procedure

1. Create a real data source mapping relationship, where key is the logical name of the data source and value is the datasource object.
2. Create the encryption rule object `EncryptRuleConfiguration`, and initialize the encryption table object `EncryptTableRuleConfiguration`, encryption algorithm and other parameters in the object.
3. Call `createDataSource` of `ShardingSphereDataSourceFactory` to create `ShardingSphereDataSource`.

Sample

```
public final class EncryptDatabasesConfiguration {

    public DataSource getDataSource() throws SQLException {
        Properties props = new Properties();
        props.setProperty("aes-key-value", "123456");
        props.setProperty("digest-algorithm-name", "SHA-1");
        EncryptColumnRuleConfiguration columnConfigAes = new
EncryptColumnRuleConfiguration("username", new EncryptColumnItemRuleConfiguration(
"username", "name_encryptor"));
        EncryptColumnRuleConfiguration columnConfigTest = new
EncryptColumnRuleConfiguration("pwd", new EncryptColumnItemRuleConfiguration("pwd",
"pwd_encryptor"));
        columnConfigTest.setAssistedQuery(new EncryptColumnItemRuleConfiguration(
"assisted_query_pwd", "pwd_encryptor"));
        columnConfigTest.setLikeQuery(new EncryptColumnItemRuleConfiguration("like_
pwd", "like_encryptor"));
        EncryptTableRuleConfiguration encryptTableRuleConfig = new
EncryptTableRuleConfiguration("t_user", Arrays.asList(columnConfigAes,
columnConfigTest));
        Map<String, AlgorithmConfiguration> encryptAlgorithmConfigs = new HashMap<>
();
        encryptAlgorithmConfigs.put("name_encryptor", new AlgorithmConfiguration(
"AES", props));
        encryptAlgorithmConfigs.put("pwd_encryptor", new AlgorithmConfiguration(
"assistedTest", props));
    }
}
```

```

        encryptAlgorithmConfigs.put("like_encryptor", new AlgorithmConfiguration(
"CHAR_DIGEST_LIKE", new Properties()));
        EncryptRuleConfiguration encryptRuleConfig = new
EncryptRuleConfiguration(Collections.singleton(encryptTableRuleConfig),
encryptAlgorithmConfigs);
        return ShardingSphereDataSourceFactory.createDataSource(DataSourceUtil.
createDataSource("demo_ds"), Collections.singleton(encryptRuleConfig), props);
    }
}

```

Related References

- [The feature description of Data Encryption](#)
- [Dev Guide of Data Encryption](#)

Data Masking

Background

The data masking Java API rule configuration allows users to directly create ShardingSphereDataSource objects by writing java code. The Java API configuration method is very flexible and can integrate various types of business systems without relying on additional jar packages.

Parameters

Root Configuration

Class name: org.apache.shardingsphere.mask.config.MaskRuleConfiguration

Attributes:

Name	DataType	Description	Default Value*
tables (+)	Collection<MaskTableRuleConfiguration>	Mask table rule configurations	
maskAlgorithms (+)	Map<String, AlgorithmConfiguration>	Mask algorithm name and configurations	

Mask Table Rule Configuration

Class name: org.apache.shardingsphere.mask.config.rule.MaskTableRuleConfiguration

Attributes:

Name	DataType	Description
name	String	Table name
columns (+)	Collection<MaskColumnRuleConfiguration>	Mask column rule configurations

Mask Column Rule Configuration

Class name: org.apache.shardingsphere.mask.config.rule.MaskColumnRuleConfiguration

Attributes:

Name	DataType	Description
logicColumn	String	Logic column name
maskAlgorithm	String	Mask algorithm name

Mask Algorithm Configuration

Class name: org.apache.shardingsphere.infra.algorithm.core.config.AlgorithmConfiguration

Attributes:

Name	DataType	Description
name	String	Mask algorithm name
type	String	Mask algorithm type
properties	Properties	Mask algorithm properties

Please refer to [Built-in Data Masking Algorithm List](#) for more details about type of algorithm.

Procedure

1. Create a real data source mapping relationship, where key is the logical name of the data source and value is the datasource object.
2. Create the data masking rule object MaskRuleConfiguration, and initialize the mask table object MaskTableRuleConfiguration, mask algorithm and other parameters in the object.
3. Call createDataSource of ShardingSphereDataSourceFactory to create ShardingSphereDataSource.

Sample

```

import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Properties;

public final class MaskDatabasesConfiguration {

    @Override
    public DataSource getDataSource() throws SQLException {
        MaskColumnRuleConfiguration passwordColumn = new
MaskColumnRuleConfiguration("password", "md5_mask");
        MaskColumnRuleConfiguration emailColumn = new MaskColumnRuleConfiguration(
"email", "mask_before_special_chars_mask");
        MaskColumnRuleConfiguration telephoneColumn = new
MaskColumnRuleConfiguration("telephone", "keep_first_n_last_m_mask");
        MaskTableRuleConfiguration maskTableRuleConfig = new
MaskTableRuleConfiguration("t_user", Arrays.asList(passwordColumn, emailColumn,
telephoneColumn));
        Map<String, AlgorithmConfiguration> maskAlgorithmConfigs = new LinkedHashMap
<>(3, 1);
        maskAlgorithmConfigs.put("md5_mask", new AlgorithmConfiguration("MD5", new
Properties()));
        Properties beforeSpecialCharsProps = new Properties();
        beforeSpecialCharsProps.put("special-chars", "@");
        beforeSpecialCharsProps.put("replace-char", "*");
        maskAlgorithmConfigs.put("mask_before_special_chars_mask", new
AlgorithmConfiguration("MASK_BEFORE_SPECIAL_CHARS", beforeSpecialCharsProps));
        Properties keepFirstNLastMProps = new Properties();
        keepFirstNLastMProps.put("first-n", "3");
        keepFirstNLastMProps.put("last-m", "4");
        keepFirstNLastMProps.put("replace-char", "*");
        maskAlgorithmConfigs.put("keep_first_n_last_m_mask", new
AlgorithmConfiguration("KEEP_FIRST_N_LAST_M", keepFirstNLastMProps));
        MaskRuleConfiguration maskRuleConfig = new MaskRuleConfiguration(Collections.
singleton(maskTableRuleConfig), maskAlgorithmConfigs);
        return ShardingSphereDataSourceFactory.createDataSource(DataSourceUtil.
createDataSource("demo_ds"), Collections.singleton(maskRuleConfig), new
Properties());
    }
}

```

Related References

- [The feature description of Data Masking](#)
- [Dev Guide of Data Masking](#)

Shadow DB

Background

In the distributed application architecture based on microservices, businesses require multiple services to be completed through a series of services and middleware, so the stress test of a single service can no longer meet the needs of real scenarios. If we reconstruct a stress test environment similar to the production environment, it is too expensive and often fails to simulate the complexity and traffic of the online environment. For this reason, the industry often chooses the full link stress test, which is performed in the production environment, so that the test results can accurately reflect the true capacity and performance of the system.

Parameters

Root Configuration

Class name: `org.apache.shardingsphere.shadow.config.ShadowRuleConfiguration`

Attributes:

Name	Data Type	Description
dataSources	Map<String, ShadowDataSourceConfiguration>	shadow data source mapping name and configuration
tables	Map<String, ShadowTableConfiguration>	shadow table name and configuration
shadowAlgorithms	Map<String, AlgorithmConfiguration>	shadow algorithm name and configuration
defaultShadowAlgorithmName	String	default shadow algorithm name

Shadow Data Source Configuration

Class name: org.apache.shardingsphere.shadow.config.datasource.ShadowDataSourceConfiguration

Attributes:

Name	Data Type	Description
productionDataSourceName	String	Production data source name
shadowDataSourceName	String	Shadow data source name

Shadow Table Configuration

Class name: org.apache.shardingsphere.shadow.config.table.ShadowTableConfiguration

Attributes:

Name	Data Type	Description
data SourceNames	Collection<String>	shadow table associates shadow data source mapping name list
shadowAlgorithmNames	Collection<String>	shadow table associates shadow algorithm name list

Shadow Algorithm Configuration

Class name: org.apache.shardingsphere.infra.algorithm.core.config.AlgorithmConfiguration

Attributes:

Name	Data Type	Description
type	String	shadow algorithm type
props	Properties	shadow algorithm configuration

Please refer to [Built-in Shadow Algorithm List](#).

Procedure

1. Create production and shadow data source.
2. Configure shadow rule.
 - Configure shadow data source
 - Configure shadow table
 - Configure shadow algorithm

Sample

```

public final class ShadowConfiguration {

    @Override
    public DataSource getDataSource() throws SQLException {
        Map<String, DataSource> dataSourceMap = createDataSourceMap();
        return ShardingSphereDataSourceFactory.createDataSource(dataSourceMap,
createRuleConfigurations(), createShardingSphereProps());
    }

    private Map<String, DataSource> createDataSourceMap() {
        Map<String, DataSource> result = new LinkedHashMap<>();
        result.put("ds", DataSourceUtil.createDataSource("demo_ds"));
        result.put("ds_shadow", DataSourceUtil.createDataSource("shadow_demo_ds"));
        return result;
    }

    private Collection<RuleConfiguration> createRuleConfigurations() {
        Collection<RuleConfiguration> result = new LinkedList<>();
        ShadowRuleConfiguration shadowRule = new ShadowRuleConfiguration();
        shadowRule.setDataSources(createShadowDataSources());
        shadowRule.setTables(createShadowTables());
        shadowRule.setShadowAlgorithms(createShadowAlgorithmConfigurations());
        result.add(shadowRule);
        return result;
    }

    private Map<String, ShadowDataSourceConfiguration> createShadowDataSources() {
        Map<String, ShadowDataSourceConfiguration> result = new LinkedHashMap<>();
        result.put("shadow-data-source", new ShadowDataSourceConfiguration("ds",
"ds_shadow"));
        return result;
    }

    private Map<String, ShadowTableConfiguration> createShadowTables() {
        Map<String, ShadowTableConfiguration> result = new LinkedHashMap<>();
        result.put("t_user", new ShadowTableConfiguration(Collections.singletonList(
"shadow-data-source"), createShadowAlgorithmNames()));
        return result;
    }

    private Collection<String> createShadowAlgorithmNames() {
        Collection<String> result = new LinkedList<>();
        result.add("user-id-insert-match-algorithm");
        result.add("simple-hint-algorithm");
        return result;
    }
}

```

```
private Map<String, AlgorithmConfiguration>
createShadowAlgorithmConfigurations() {
    Map<String, AlgorithmConfiguration> result = new LinkedHashMap<>();
    Properties userIdInsertProps = new Properties();
    userIdInsertProps.setProperty("operation", "insert");
    userIdInsertProps.setProperty("column", "user_type");
    userIdInsertProps.setProperty("value", "1");
    result.put("user-id-insert-match-algorithm", new AlgorithmConfiguration(
"VALUE_MATCH", userIdInsertProps));
    return result;
}
}
```

Related References

[Features Description of Shadow DB](#)

SQL Parser

Background

SQL is the standard language for users to communicate with databases. The SQL parsing engine is responsible for parsing the SQL string into an abstract syntax tree for Apache ShardingSphere to understand and implement its incremental function. Currently, MySQL, PostgreSQL, SQLServer, Oracle, openGauss and SQL dialects conforming to SQL92 specifications are supported. Due to the complexity of SQL syntax, there are still a few unsupported SQLs. By using SQL parsing in the form of Java API, you can easily integrate into various systems and flexibly customize user requirements.

Parameters

Class: org.apache.shardingsphere.parser.config.SQLParserRuleConfiguration

Attributes:

<i>name</i>	<i>DataType</i>	<i>Description</i>
parseTreeCache (?)	CacheOption	Parse syntax tree local cache configuration
sqlStatementCache (?)	CacheOption	sql statement local cache configuration

Cache option Configuration

Class: org.apache.shardingsphere.sql.parser.api.CacheOption

Attributes:

<i>na me</i>	<i>. Data Type *</i>	<i>Des cription</i>	<i>Default Value</i>
ini tia lCa pac ity	i n t	Initial capacity of local cache	parser syntax tree local cache default value 128, SQL statement cache default value 2000
ma xim umS ize (?)	l o n g	Maximum capacity of local cache	The default value of local cache for parsing syntax tree is 1024, and the default value of sql statement cache is 65535

Procedure

1. Set local cache configuration.
2. Set resolution configuration.
3. Use the parsing engine to parse SQL.

Sample

```
CacheOption cacheOption = new CacheOption(128, 1024L);
SQLParserEngine parserEngine = new SQLParserEngine("MySQL", cacheOption);
ParseASTNode parseASTNode = parserEngine.parse("SELECT t.id, t.name, t.age FROM table1 AS t ORDER BY t.id DESC;", false);
SQLStatementVisitorEngine visitorEngine = new SQLStatementVisitorEngine("MySQL");
MySQLStatement sqlStatement = visitorEngine.visit(parseASTNode);
System.out.println(sqlStatement.toString());
```

Related References

- [YAML Configuration: SQL Parser](#)

SQL Translator

Background

By using SQL translator in the form of Java API, you can easily integrate into various systems and flexibly customize user requirements.

Parameters

Class: `org.apache.shardingsphere.sqltranslator.config.SQLTranslatorRuleConfiguration`

Attributes:

<i>name</i>	<i>Data Type</i>	<i>Description</i>
type	String	SQL translator type
useOriginalSQLWhenTranslatingFailed (?)	boolean	Whether use original SQL when translating failed

Procedure

1. Set SQL translator type.
2. Set `useOriginalSQLWhenTranslatingFailed` to decide whether use original SQL when translating failed.

Sample

```
SQLTranslatorRuleConfiguration ruleConfig = new SQLTranslatorRuleConfiguration(
    "Native", new Properties(), false);
String translatedSQL = new SQLTranslatorRule(ruleConfig).translate();
```

Related References

- [YAML Configuration: SQL Translator](#)

Mixed Rules

Background

ShardingSphere provides a variety of features, such as data sharding, read/write splitting, high availability, and data encryption. These features can be used independently or in combination. Below, you will find the configuration samples based on JAVA API.

Samples

```
// Sharding configuration
private ShardingRuleConfiguration createShardingRuleConfiguration() {
    ShardingRuleConfiguration result = new ShardingRuleConfiguration();
    result.getTables().add(getOrderTableRuleConfiguration());
    result.setDefaultDatabaseShardingStrategy(new
StandardShardingStrategyConfiguration("user_id", "inline"));
    result.setDefaultTableShardingStrategy(new
StandardShardingStrategyConfiguration("order_id", "standard_test_tbl"));
    Properties props = new Properties();
    props.setProperty("algorithm-expression", "demo_ds_${user_id % 2}");
    result.getShardingAlgorithms().put("inline", new AlgorithmConfiguration("INLINE
", props));
    result.getShardingAlgorithms().put("standard_test_tbl", new
AlgorithmConfiguration("STANDARD_TEST_TBL", new Properties()));
    result.getKeyGenerators().put("snowflake", new AlgorithmConfiguration("SNOWFLAKE
", new Properties()));
    return result;
}

private ShardingTableRuleConfiguration getOrderTableRuleConfiguration() {
    ShardingTableRuleConfiguration result = new ShardingTableRuleConfiguration("t_
order", "demo_ds_${0..1}.t_order_${[0, 1]}");
    result.setKeyGenerateStrategy(new KeyGenerateStrategyConfiguration("order_id",
"snowflake"));
    return result;
}

// Read/write splitting configuration
private static ReadwriteSplittingRuleConfiguration
createReadwriteSplittingConfiguration() {
    ReadwriteSplittingDataSourceRuleConfiguration dataSourceConfiguration1 = new
ReadwriteSplittingDataSourceRuleConfiguration("replica_ds_0", Arrays.asList(
```

```

"readwrite_ds_0"), true), "");
    ReadwriteSplittingDataSourceRuleConfiguration dataSourceConfiguration2 = new
ReadwriteSplittingDataSourceRuleConfiguration("replica_ds_1", Arrays.asList(
"readwrite_ds_1"), true), "");
    Collection<ReadwriteSplittingDataSourceRuleConfiguration> dataSources = new
LinkedList<>();
    dataSources.add(dataSourceRuleConfiguration1);
    dataSources.add(dataSourceRuleConfiguration2);
    return new ReadwriteSplittingRuleConfiguration(dataSources, Collections.
emptyMap());
}

// Data encryption configuration
private static EncryptRuleConfiguration createEncryptRuleConfiguration() {
    Properties props = new Properties();
    props.setProperty("aes-key-value", "123456");
    props.setProperty("digest-algorithm-name", "SHA-1");
    EncryptColumnRuleConfiguration columnConfigAes = new
EncryptColumnRuleConfiguration("username", new EncryptColumnItemRuleConfiguration(
"username", "name_encryptor"));
    EncryptColumnRuleConfiguration columnConfigTest = new
EncryptColumnRuleConfiguration("pwd", new EncryptColumnItemRuleConfiguration("pwd",
"pwd_encryptor"));
    columnConfigTest.setAssistedQuery(new EncryptColumnItemRuleConfiguration(
"assisted_query_pwd", "pwd_encryptor"));
    columnConfigTest.setLikeQuery(new EncryptColumnItemRuleConfiguration("like_pwd",
"like_encryptor"));
    EncryptTableRuleConfiguration encryptTableRuleConfig = new
EncryptTableRuleConfiguration("t_user", Arrays.asList(columnConfigAes,
columnConfigTest));
    Map<String, AlgorithmConfiguration> encryptAlgorithmConfigs = new HashMap<>();
    encryptAlgorithmConfigs.put("name_encryptor", new AlgorithmConfiguration("AES",
props));
    encryptAlgorithmConfigs.put("pwd_encryptor", new AlgorithmConfiguration(
"assistedTest", props));
    encryptAlgorithmConfigs.put("like_encryptor", new AlgorithmConfiguration("CHAR_
DIGEST_LIKE", new Properties()));
    return new EncryptRuleConfiguration(Collections.
singleton(encryptTableRuleConfig), encryptAlgorithmConfigs);
}

```

Cache for Sharding Route

Background

This feature is **experimental** and needs to be used with the data sharding rule. The cache for sharding route will put the logical SQL, the parameter value of the shard key, and the routing result into the cache, exchange space for time, and reduce CPU usage of the routing logic.

We recommend enabling it only if the following conditions are met: - Pure OLTP scenarios. - The CPU of the machine which deployed the ShardingSphere process has reached the bottleneck. - Most of the CPUs are used by ShardingSphere routing logic. - All SQLs are optimized and each SQL execution could be routed to a single data node.

If the above conditions are not met, the execution delay of SQL may not be significantly improved, and the memory pressure will be increased.

Parameters

Class: `org.apache.shardingsphere.sharding.api.config.cache.ShardingCacheConfiguration`

Attributes:

<i>name</i>	<i>DataType</i>	<i>Description</i>	<i>Default Value</i>
<code>allowedMaxSqlLength</code>	<code>int</code>	允许缓存的SQL长度限制	<code>.</code>
<code>routeCache</code>	<code>org.apache.shardingsphere.sharding.api.config.cache.ShardingCacheOption</code>	路由缓存	<code>.</code>

Class: `org.apache.shardingsphere.sharding.api.config.cache.ShardingCacheOptionsConfiguration`

Attributes:

<i>name</i>	<i>DataType</i>	<i>Description</i>	<i>Default Value</i>
<code>softValues</code>	<code>boolean</code>	是否软引用缓存值	<code>.</code>
<code>initialCapacity</code>	<code>int</code>	缓存初始容量	<code>.</code>
<code>maximumSize</code>	<code>int</code>	缓存最大容量	<code>.</code>

Sample

```

public final class ShardingDatabasesAndTablesConfigurationPrecise {

    @Override
    public DataSource getDataSource() throws SQLException {
        return ShardingSphereDataSourceFactory.
createDataSource(createDataSourceMap(), Arrays.
asList(createShardingRuleConfiguration(), createBroadcastRuleConfiguration())), new
Properties());
    }

    private ShardingRuleConfiguration createShardingRuleConfiguration() {
        ShardingRuleConfiguration result = new ShardingRuleConfiguration();
        result.getTables().add(getOrderTableRuleConfiguration());
        result.getTables().add(getOrderItemTableRuleConfiguration());
        // ...
        result.setShardingCache(new ShardingCacheConfiguration(512, new
ShardingCacheConfiguration.RouteCacheConfiguration(65536, 262144, true)));
        return result;
    }

    private ShardingTableRuleConfiguration getOrderTableRuleConfiguration() {
        ShardingTableRuleConfiguration result = new ShardingTableRuleConfiguration(
"t_order", "demo_ds_${0..1}.t_order_${[0, 1]}");
        result.setKeyGenerateStrategy(new KeyGenerateStrategyConfiguration("order_id
", "snowflake"));
        result.setAuditStrategy(new ShardingAuditStrategyConfiguration(Collections.
singleton("sharding_key_required_auditor"), true));
        return result;
    }

    private ShardingTableRuleConfiguration getOrderItemTableRuleConfiguration() {
        ShardingTableRuleConfiguration result = new ShardingTableRuleConfiguration(
"t_order_item", "demo_ds_${0..1}.t_order_item_${[0, 1]}");
        result.setKeyGenerateStrategy(new KeyGenerateStrategyConfiguration("order_
item_id", "snowflake"));
        return result;
    }

    private Map<String, DataSource> createDataSourceMap() {
        Map<String, DataSource> result = new HashMap<>();
        result.put("demo_ds_0", DataSourceUtil.createDataSource("demo_ds_0"));
        result.put("demo_ds_1", DataSourceUtil.createDataSource("demo_ds_1"));
        return result;
    }

    private BroadcastRuleConfiguration createBroadcastRuleConfiguration() {

```

```

        return new BroadcastRuleConfiguration(Collections.singletonList("t_address
"));
    }
}

```

Related References

- [Core Feature: Data Sharding](#)

Single Table

Background

Single rule is used to specify which single tables need to be managed by ShardingSphere, or to set the default single table data source.

Parameters

Class: `org.apache.shardingsphere.single.config.SingleRuleConfiguration`

Attributes:

<i>name</i>	<i>DataType</i>	<i>Description</i>	<i>Default Value</i>
tables (+)	Collection<String>	single tables	.
defaultDataSource (?)	String	single table default data source	.

Procedure

1. Initialize `SingleRuleConfiguration`;
2. Add a single table to be loaded and configure the default data source.

Sample

```
SingleRuleConfiguration ruleConfig = new SingleRuleConfiguration();
ShardingSphereDataSourceFactory.createDataSource(createDataSourceMap(), Arrays.
asList(ruleConfig), new Properties());
```

Related References

- [Single Table](#)

SQL-federation

Background

This function is an **experimental one and is currently not suitable for use in core system production environments**. When multiple tables in a join query are distributed across different database instances, enabling federated query allows for cross-database join queries, as well as subqueries.

Parameters

Class: org.apache.shardingsphere.sqlfederation.config.SQLFederationRuleConfiguration

Attributes:

<i>name</i>	<i>DataType</i>	<i>Description</i>	<i>De fault V alue</i>
sqlFederationEnabled	boolean	SQL federation enabled configuration	.
allQueryUseSQL Federation	boolean	all query use SQL federation configuration	.
executionPlanCache	org.apache.shardingsphere.sql.parser.api.CacheOption	execution plan cache configuration	.

Cache option Configuration

Class: org.apache.shardingsphere.sql.parser.api.CacheOption

Attributes:

<i>name</i>	<i>Data Type</i>	<i>Description</i>	<i>Default Value</i>
initialCapacity	int	Initial capacity of local cache	execution plan local cache default value of 2000
maximumSize	long	Maximum capacity of local cache	execution plan local cache maximum default value 65535

Sample

```
private SQLFederationRuleConfiguration createSQLFederationRuleConfiguration() {
    CacheOption executionPlanCache = new CacheOption(2000, 65535L);
    return new SQLFederationRuleConfiguration(true, false, executionPlanCache);
}
```

Related References

- [YAML Configuration: SQL Federation](#)

Algorithm

Sharding

```
ShardingRuleConfiguration ruleConfiguration = new ShardingRuleConfiguration();
// algorithmName is specified by users and should be consistent with the sharding
// algorithm in the sharding strategy.
// type and props, please refer to the built-in sharding algorithm: https://
// shardingsphere.apache.org/document/current/en/user-manual/common-config/builtin-
// algorithm/sharding/
ruleConfiguration.getShardingAlgorithms().put("algorithmName", new
AlgorithmConfiguration("xxx", new Properties()));
```

Encryption

```
// encryptorName is specified by users, and its property should be consistent with
// that of encryptorName in encryption rules.
// type and props, please refer to the built-in encryption algorithm: https://
// shardingsphere.apache.org/document/current/en/user-manual/common-config/builtin-
// algorithm/encrypt/
Map<String, AlgorithmConfiguration> algorithmConfigs = new HashMap<>();
algorithmConfigs.put("encryptorName", new AlgorithmConfiguration("xxx", new
Properties()));
```

Read/Write Splitting Load Balancer

```
// loadBalancerName is specified by users, and its property has to be consistent
// with that of loadBalancerName in read/write splitting rules.
// type and props, please refer to the built-in read/write splitting algorithm load
// balancer: https://shardingsphere.apache.org/document/current/en/user-manual/common-
// config/builtin-algorithm/load-balance/
Map<String, AlgorithmConfiguration> algorithmConfigs = new HashMap<>();
algorithmConfigs.put("loadBalancerName", new AlgorithmConfiguration("xxx", new
Properties()));
```

Shadow DB

```
// shadowAlgorithmName is specified by users, and its property has to be consistent
// with that of shadowAlgorithmNames in shadow DB rules.
// type and props, please refer to the built-in shadow DB algorithm: https://
// shardingsphere.apache.org/document/current/en/user-manual/common-config/builtin-
// algorithm/shadow/
Map<String, AlgorithmConfiguration> algorithmConfigs = new HashMap<>();
algorithmConfigs.put("shadowAlgorithmName", new AlgorithmConfiguration("xxx", new
Properties()));
```

High Availability

```
// discoveryTypeName is specified by users, and its property has to be consistent
// with that of discoveryTypeName in database discovery rules.
Map<String, AlgorithmConfiguration> algorithmConfigs = new HashMap<>();
algorithmConfigs.put("discoveryTypeName", new AlgorithmConfiguration("xxx", new
Properties()));
```

Data Masking

```
// maskAlgorithmName is specified by users, and its property should be consistent
// with that of maskAlgorithm in mask rules.
// type and props, please refer to the built-in mask algorithm: https://
// shardingsphere.apache.org/document/current/en/user-manual/common-config/builtin-
// algorithm/mask/
Map<String, AlgorithmConfiguration> algorithmConfigs = new HashMap<>();
algorithmConfigs.put("maskAlgorithmName", new AlgorithmConfiguration("xxx", new
Properties()));
```

9.1.3 Special API

This chapter will introduce the special API of ShardingSphere-JDBC.

Sharding

This chapter will introduce the Sharding API of ShardingSphere-JDBC.

Hint

Background

Apache ShardingSphere uses ThreadLocal to manage sharding key values for mandatory routing. A sharding value can be added by programming to the HintManager that takes effect only within the current thread.

Main application scenarios for Hint: - The sharding fields do not exist in the SQL and database table structure but in the external business logic. - Certain data operations are forced to be performed in given databases.

Procedure

1. Call `HintManager.getInstance()` to obtain an instance of `HintManager`.
2. Use `HintManager.addDatabaseShardingValue`, `HintManager.addTableShardingValue` to set the sharding key value.
3. Execute SQL statements to complete routing and execution.
4. Call `HintManager.close` to clean up the contents of `ThreadLocal`.

Sample

Hint Configuration

Hint algorithms require users to implement the interface of `org.apache.shardingsphere.api.sharding.hint.HintShardingAlgorithm`. `org.apache.shardingsphere.sharding.api.sharding.hint.HintShardingAlgorithm` has two built-in implementations,

- `org.apache.shardingsphere.sharding.algorithm.sharding.hint.HintInlineShardingAlgorithm`
- `org.apache.shardingsphere.sharding.algorithm.sharding.classbased.ClassBasedShardingAlgorithm`

Apache ShardingSphere will acquire sharding values from `HintManager` to route.

Take the following configurations for reference:

```

rules:
- !SHARDING
  tables:
    t_order:
      actualDataNodes: demo_ds_${0..1}.t_order_${0..1}
      databaseStrategy:
        hint:
          shardingColumn: order_id
          shardingAlgorithmName: hint_class_based
      tableStrategy:
        hint:
          shardingColumn: order_id
          shardingAlgorithmName: hint_inline
  shardingAlgorithms:
    hint_class_based:
      type: CLASS_BASED
      props:
        strategy: STANDARD
        algorithmClassName: xxx.xxx.xxx.HintXXXAlgorithm
    hint_inline:
      type: HINT_INLINE
      props:
        algorithm-expression: t_order_${value % 4}
  defaultTableStrategy:
    none:
  defaultKeyGenerateStrategy:
    type: SNOWFLAKE
    column: order_id

props:
  sql-show: true

```

Get HintManager

```
HintManager hintManager = HintManager.getInstance();
```

Add Sharding Value

- Use `hintManager.addDatabaseShardingValue` to add sharding key value of data source.
- Use `hintManager.addTableShardingValue` to add sharding key value of table.

Users can use `hintManager.setDatabaseShardingValue` to set sharding value in hint route to some certain sharding database without sharding tables.

Clean Hint Values

Sharding values are saved in `ThreadLocal`, so it is necessary to use `hintManager.close()` to clean `ThreadLocal`.

``HintManager`` has implemented ``AutoCloseable``. We recommend to close it automatically with ``try with resource``.

Codes:

```
// Sharding database and table with HintManager
String sql = "SELECT * FROM t_order";
try (HintManager hintManager = HintManager.getInstance();
    Connection conn = dataSource.getConnection();
    PreparedStatement preparedStatement = conn.prepareStatement(sql)) {
    hintManager.addDatabaseShardingValue("t_order", 1);
    hintManager.addTableShardingValue("t_order", 2);
    try (ResultSet rs = preparedStatement.executeQuery()) {
        while (rs.next()) {
            // ...
        }
    }
}

// Sharding database and one database route with HintManager
String sql = "SELECT * FROM t_order";
try (HintManager hintManager = HintManager.getInstance();
    Connection conn = dataSource.getConnection();
    PreparedStatement preparedStatement = conn.prepareStatement(sql)) {
    hintManager.setDatabaseShardingValue(3);
    try (ResultSet rs = preparedStatement.executeQuery()) {
        while (rs.next()) {
            // ...
        }
    }
}
```

Related References

- [Core Feature: Data Sharding](#)
- [Developer Guide: Data Sharding](#)

Readwrite-splitting

This chapter will introduce the Readwrite-splitting API of ShardingSphere-JDBC.

Hint

Background

Apache ShardingSphere uses `ThreadLocal` to manage primary database routing marks for mandatory routing. A primary database routing mark can be added to `HintManager` through programming, and this value is valid only in the current thread.

`Hint` is mainly used to perform mandatory data operations in the primary database for read/write splitting scenarios.

Procedure

1. Call `HintManager.getInstance()` to obtain `HintManager` instance.
2. Call `HintManager.setWriteRouteOnly()` method to set the primary database routing marks.
3. Execute SQL statements to complete routing and execution.
4. Call `HintManager.close()` to clear the content of `ThreadLocal`.

Sample

Primary Route with Hint

Get HintManager

The same as sharding based on hint.

Configure Primary Database Route

- Use `hintManager.setWriteRouteOnly` to configure primary database route.

Clean Hint Value

The same as data sharding based on hint.

Code:

```
String sql = "SELECT * FROM t_order";
try (HintManager hintManager = HintManager.getInstance();
     Connection conn = dataSource.getConnection();
     PreparedStatement preparedStatement = conn.prepareStatement(sql)) {
    hintManager.setWriteRouteOnly();
    try (ResultSet rs = preparedStatement.executeQuery()) {
        while (rs.next()) {
            // ...
        }
    }
}
```

Route to the specified database with Hint**Get HintManager**

The same as sharding based on hint.

Configure Database Route

- Use `hintManager.setDataSourceName` to configure database route.

Code:

```
String sql = "SELECT * FROM t_order";
try (HintManager hintManager = HintManager.getInstance();
     Connection conn = dataSource.getConnection();
     PreparedStatement preparedStatement = conn.prepareStatement(sql)) {
    hintManager.setDataSourceName("ds_0");
    try (ResultSet rs = preparedStatement.executeQuery()) {
        while (rs.next()) {
            // ...
        }
    }
}
```

Related References

- [Core Feature: Read/write Splitting](#)
- [Developer Guide: Read/write Splitting](#)

Transaction

Using distributed transaction through Apache ShardingSphere is no different from local transaction. In addition to transparent use of distributed transaction, Apache ShardingSphere can switch distributed transaction types every time the database accesses.

Supported transaction types include local, XA and BASE. It can be set before creating a database connection, and default value can be set when Apache ShardingSphere startup.

Use Java API

Background

With ShardingSphere-JDBC, XA and BASE mode transactions can be used through the API.

Prerequisites

Introducing Maven dependency

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>

<!-- This module is required when using XA transactions -->
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-transaction-xa-core</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>

<!-- This module is required when using the Narayana mode with XA transactions -->
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-transaction-xa-narayana</artifactId>
  <version>${project.version}</version>
</dependency>

<!-- This module is required when using BASE transactions -->
<dependency>
```



```
<groupId>org.apache.shardingsphere</groupId>
<artifactId>shardingsphere-transaction-base-seata-at</artifactId>
<version>${shardingsphere.version}</version>
</dependency>
```

Procedure

Perform the business logic using transactions

Sample

```
// Use ShardingSphereDataSource to get a connection and perform transaction
operations.
try (Connection connection = dataSource.getConnection()) {
    connection.setAutoCommit(false);
    PreparedStatement preparedStatement = connection.prepareStatement("INSERT INTO
t_order (user_id, status) VALUES (?, ?)");
    preparedStatement.setObject(1, 1000);
    preparedStatement.setObject(2, "init");
    preparedStatement.executeUpdate();
    connection.commit();
}
```

Atomikos Transaction

Background

Apache ShardingSphere provides XA transactions, and the default XA transaction manager is Atomikos.

Procedure

1. Configure the transaction type
2. Configure Atomikos

Sample

Configure the transaction type

Yaml:

```
transaction:
  defaultType: XA
  providerType: Atomikos
```

Configure Atomikos

Atomikos configuration items can be customized by adding `jta.properties` to the project's class-path.

See [Atomikos's official documentation](#) for more details.

Data Recovery

`xa_tx.log` is generated in the `logs` directory of the project. This is the log required for recovering XA crash. Do not delete it.

Narayana Transaction

Background

Apache ShardingSphere provides XA transactions that integrate with the Narayana implementation.

Prerequisites

Introducing Maven dependency

```
<properties>
  <narayana.version>5.12.7.Final</narayana.version>
  <jboss-transaction-spi.version>7.6.1.Final</jboss-transaction-spi.version>
  <jboss-logging.version>3.2.1.Final</jboss-logging.version>
</properties>

<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>

<!-- This module is required when using XA transactions -->
```

```

<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-transaction-xa-core</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>

<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-transaction-xa-narayana</artifactId>
  <version>${shardingsphere.version}</version>
</dependency>
<dependency>
  <groupId>org.jboss.narayana.jta</groupId>
  <artifactId>jta</artifactId>
  <version>${narayana.version}</version>
</dependency>
<dependency>
  <groupId>org.jboss.narayana.jts</groupId>
  <artifactId>narayana-jts-integration</artifactId>
  <version>${narayana.version}</version>
</dependency>
<dependency>
  <groupId>org.jboss</groupId>
  <artifactId>jboss-transaction-spi</artifactId>
  <version>${jboss-transaction-spi.version}</version>
</dependency>
<dependency>
  <groupId>org.jboss.logging</groupId>
  <artifactId>jboss-logging</artifactId>
  <version>${jboss-logging.version}</version>
</dependency>

```

Procedure

1. Configure Narayana
2. Set the XA transaction type

Sample

Configure Narayana

Narayana configuration items can be customized by adding `jbossts-properties.xml` to the project's classpath.

See [Narayana's Official Documentation](#) for more details.

For the minimum configuration of `jbosssts-properties.xml`, ShardingSphere requires that Narayana's `CoreEnvironmentBean.nodeIdentifier` property be defined. If Narayana's object store is not shared between different Narayana instances, you can set this value to 1. A possible `jbosssts-properties.xml` configuration is as follows,

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">
<properties>
  <entry key="CoreEnvironmentBean.nodeIdentifier">1</entry>
</properties>
```

In certain cases, you may not want to use XML files, then you need to manually set `CoreEnvironmentBean.nodeIdentifier` in the bootstrap class of your own Java project. You can refer to the following methods to call Narayana Java API.

```
import com.arjuna.ats.arjuna.common.CoreEnvironmentBeanException;
import com.arjuna.ats.arjuna.common.arjPropertyManager;

public class ExampleUtils {
    public void initNarayanaInstance() {
        try {
            arjPropertyManager.getCoreEnvironmentBean().setNodeIdentifier("1");
        } catch (CoreEnvironmentBeanException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Set the XA transaction type

Yaml:

```
transaction:
  defaultType: XA
  providerType: Narayana
```

Seata Transaction

Background

Apache ShardingSphere provides BASE transactions that integrate the Seata implementation. All references to Seata integration in this article refer to Seata AT mode.

Prerequisites

ShardingSphere's Seata integration is only available in `apache/incubator-seata:v2.3.0` or higher. For Seata Client corresponding to the `org.apache.seata:seata-all` Maven module, this limitation applies to both HotSpot VM and GraalVM Native Image. Introduce Maven dependencies and exclude the outdated Maven dependency of `org.antlr:antlr4-runtime:4.8` in `org.apache.seata:seata-all`.

```
<project>
  <dependencies>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-jdbc</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-transaction-base-seata-at</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
    <dependency>
      <groupId>org.apache.seata</groupId>
      <artifactId>seata-all</artifactId>
      <version>2.3.0</version>
      <exclusions>
        <exclusion>
          <groupId>org.antlr</groupId>
          <artifactId>antlr4-runtime</artifactId>
        </exclusion>
      </exclusions>
    </dependency>
  </dependencies>
</project>
```

Affected by Calcite, `commons-lang:commons-lang` and `org.apache.commons:commons-pool2` used by ShardingSphere JDBC have dependency conflicts with Seata Client. Users need to consider whether to resolve dependency conflicts based on actual scenarios. If dependency conflicts are not resolved, build tools such as Maven will randomly use a version of the conflicting dependency in the classpath.

When using ShardingSphere's Seata integration module, the database instance connected to ShardingSphere should implement both ShardingSphere's dialect parsing support and Seata AT mode's dialect parsing support. This type of database includes but is not limited to `mysql`, `gvenzl/oracle-free`, `gvenzl/oracle-xe`, `postgres`, `mcr.microsoft.com/mssql/server` and other Docker Images.

undo_log table restrictions

In each real database instance involved in ShardingSphere, an undo_log table needs to be created. The SQL content of each database is based on the corresponding database in <https://github.com/apache/incubator-seata/tree/v2.3.0/script/client/at/db>.

Related configuration

Write the following content in the YAML configuration file of ShardingSphere of your own project, refer to [Distributed Transaction](#). If Java API is used when initializing ShardingSphere JDBC DataSource, refer to [Distributed Transaction](#).

```
transaction:
  defaultType: BASE
  providerType: Seata
```

Add the seata.conf file to the root directory of the classpath. For the configuration file format, refer to the [JavaDoc](#) of org.apache.seata.config.FileConfiguration.

seata.conf has four properties,

1. shardingsphere.transaction.seata.at.enable, when this value is true, enable ShardingSphere's Seata AT integration. The default value is true
2. shardingsphere.transaction.seata.tx.timeout, global transaction timeout (seconds). The default value is 60
3. client.application.id, application unique primary key, used to set applicationId of Seata Transaction Manager Client and Seata Resource Manager Client
4. client.transaction.service.group, transaction group, used to set transactionServiceGroup of Seata Transaction Manager Client and Seata Resource Manager Client. The default value is default

A fully configured seata.conf is as follows,

```
shardingsphere.transaction.seata.at.enable = true
shardingsphere.transaction.seata.tx.timeout = 60

client {
  application.id = example
  transaction.service.group = default_tx_group
}
```

A minimally configured seata.conf is as follows. In seata.conf managed by ShardingSphere, the default value of client.transaction.service.group is default for historical reasons. Assuming that registry.type and config.type are both file in registry.conf of Seata Server and Seata Client used by the user, then for registry.file.name of registry.conf, the transaction group name in the .conf file configured by config.file.name is de-

fault_tx_group in apache/incubator-seata:v1.5.1 and later, and my_test_tx_group before apache/incubator-seata:v1.5.1.

```
client.application.id = example
```

Modify Seata's registry.conf file according to the actual scenario.

Operation steps

1. Start Seata Server
2. Create undo_log table
3. Add Seata configuration

Configuration Example

Start Seata Server and MySQL Server

Write Docker Compose file to start Seata Server and MySQL Server.

```
services:
  apache-seata-server:
    image: apache/seata-server:2.3.0
    ports:
      - "8091:8091"
  mysql:
    image: mysql:9.1.0
    environment:
      MYSQL_ROOT_PASSWORD: example
    volumes:
      - ./mysql/docker-entrypoint-initdb.d:/docker-entrypoint-initdb.d
    ports:
      - "3306:3306"
```

The ./docker-entrypoint-initdb.d folder contains the file init.sh, the content is as follows,

```
#!/bin/bash
set -e

mysql -uroot -p"$MYSQL_ROOT_PASSWORD" <<EOSQL
CREATE DATABASE demo_ds_0;
CREATE DATABASE demo_ds_1;
CREATE DATABASE demo_ds_2;
EOSQL

for i in "demo_ds_0" "demo_ds_1" "demo_ds_2"
do
```

```
mysql -uroot -p"$MYSQL_ROOT_PASSWORD" "$i" <<'EOSQL'
CREATE TABLE IF NOT EXISTS `undo_log`
(
  `branch_id`      BIGINT      NOT NULL COMMENT 'branch transaction id',
  `xid`            VARCHAR(128) NOT NULL COMMENT 'global transaction id',
  `context`        VARCHAR(128) NOT NULL COMMENT 'undo_log context,such as
serialization',
  `rollback_info`  LONGBLOB    NOT NULL COMMENT 'rollback info',
  `log_status`     INT(11)     NOT NULL COMMENT '0:normal status,1:defense status',
  `log_created`    DATETIME(6) NOT NULL COMMENT 'create datetime',
  `log_modified`   DATETIME(6) NOT NULL COMMENT 'modify datetime',
  UNIQUE KEY `ux_undo_log` (`xid`, `branch_id`)
) ENGINE = InnoDB AUTO_INCREMENT = 1 DEFAULT CHARSET = utf8mb4 COMMENT ='AT
transaction mode undo table';
ALTER TABLE `undo_log` ADD INDEX `ix_log_created` (`log_created`);

CREATE TABLE IF NOT EXISTS t_order (
  order_id BIGINT NOT NULL AUTO_INCREMENT,
  order_type INT(11),
  user_id INT NOT NULL,
  address_id BIGINT NOT NULL,
  status VARCHAR(50),
  PRIMARY KEY (order_id)
);
EOSQL
done
```

Create seata.conf in the classpath of the business project

Create seata.conf in the classpath of the business project, with the following content,

```
service {
  default.grouplist = "127.0.0.1:8091"
  vgroupMapping.default_tx_group = "default"
}
```

Create file.conf in the classpath of the business project

Create file.conf in the classpath of the business project, with the following content,

```
client {
  application.id = test
  transaction.service.group = default_tx_group
}
```


Create registry.conf in the classpath of the business project

Create a registry.conf in the classpath of the business project with the following content:

```
registry {
  type = "file"
  file {
    name = "file.conf"
  }
}
config {
  type = "file"
  file {
    name = "file.conf"
  }
}
```

Add JDBC Driver to the business project and create ShardingSphere configuration file

After the business project introduces the dependencies involved in the prerequisites, add the Maven dependency of MySQL JDBC Driver.

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>9.1.0</version>
</dependency>
```

Write the ShardingSphere data source configuration file demo.yaml on the classpath of the business project.

```
dataSources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.cj.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/demo_ds_0?sslMode=REQUIRED
    username: root
    password: example
  ds_1:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.cj.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/demo_ds_1?sslMode=REQUIRED
    username: root
    password: example
  ds_2:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.cj.jdbc.Driver
    jdbcUrl: jdbc:mysql://localhost:3306/demo_ds_2?sslMode=REQUIRED
```

```

    username: root
    password: example
rules:
  - !SHARDING
    tables:
      t_order:
        actualDataNodes: ds_${0..2}.t_order
        keyGenerateStrategy:
          column: order_id
          keyGeneratorName: snowflake
        defaultDatabaseStrategy:
          standard:
            shardingColumn: user_id
            shardingAlgorithmName: inline
        shardingAlgorithms:
          inline:
            type: INLINE
            props:
              algorithm-expression: ds_${user_id % 2}
        keyGenerators:
          snowflake:
            type: SNOWFLAKE
transaction:
  defaultType: BASE
  providerType: Seata

```

Enjoy integration

You can start enjoying integration on ShardingSphere's data source.

```

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.Connection;
import java.sql.SQLException;
@SuppressWarnings({"SqlNoDataSourceInspection", "AssertWithSideEffects"})
public class ExampleTest {
    void test() throws SQLException {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:shardingsphere:classpath:demo.yaml");
        config.setDriverClassName("org.apache.shardingsphere.driver.
ShardingSphereDriver");
        try (HikariDataSource dataSource = new HikariDataSource(config)) {
            try (Connection conn = dataSource.getConnection()) {
                try {
                    conn.setAutoCommit(false);
                    conn.createStatement().executeUpdate("INSERT INTO t_order (user_
id, order_type, address_id, status) VALUES (2024, 1, 2024, 'INSERT_TEST')");

```

```

        conn.createStatement().executeUpdate("INSERT INTO t_order_does_
not_exist (test_id_does_not_exist) VALUES (2024)");
        conn.commit();
    } catch (final SQLException ignored) {
        conn.rollback();
    } finally {
        conn.setAutoCommit(true);
    }
}
try (Connection conn = dataSource.getConnection()) {
    assert !conn.createStatement().executeQuery("SELECT * FROM t_order_
item WHERE user_id = 2024").next();
}
}
}
}

```

Usage restrictions

ShardingSphere' s Seata integration does not support isolation levels.

ShardingSphere' s Seata integration places the obtained Seata global transaction into the thread' s local variables. And `org.apache.seata.spring.annotation.GlobalTransactionScanner` uses Dynamic Proxy to enhance the method. This means that when using ShardingSphere' s Seata integration, users should avoid using the Java API of `org.apache.seata:seata-all`, unless the user is mixing ShardingSphere' s Seata integration with the TCC mode feature of Seata Client.

For ShardingSphere data source, 7 situations are discussed.

1. Manually obtain the `java.sql.Connection` instance created from the ShardingSphere data source and manually call the `setAutoCommit()`, `commit()` and `rollback()` methods. This is allowed.
2. Use the `javax.transaction.Transactional` annotation of Jakarta EE 8 on the function. This is allowed.
3. Use the `jakarta.transaction.Transactional` annotation of Jakarta EE 9/10 on the function. This is allowed.
4. Use the `org.springframework.transaction.annotation.Transactional` annotation of Spring Framework on the function. This is allowed.
5. Manually obtain an `org.springframework.transaction.support.TransactionTemplate` instance created from an `org.springframework.transaction.PlatformTransactionManager` instance, and use `org.springframework.transaction.support.TransactionTemplate#execute(org.springframework.transaction.support.TransactionCallback)`, which is allowed.
6. Use the `org.apache.seata.spring.annotation.GlobalTransactional` annotation on a function, which is **not allowed**.

7. Manually create an `org.apache.seata.tm.api.GlobalTransaction` instance from an `org.apache.seata.tm.api.GlobalTransactionContext`, and call the `begin()`, `commit()`, and `rollback()` methods of the `org.apache.seata.tm.api.GlobalTransaction` instance, which is **not allowed**.

In actual scenarios where Spring Boot is used, `com.alibaba.cloud:spring-cloud-starter-alibaba-seata` and `org.apache.seata:seata-spring-boot-starter` are often transitively imported by other Maven dependencies. To avoid transaction conflicts, users need to set the property `seata.enable-auto-data-source-proxy` to `false` in the Spring Boot configuration file. A possible dependency relationship is as follows.

```
<project>
  <dependencies>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-jdbc</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-transaction-base-seata-at</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
    <dependency>
      <groupId>org.apache.seata</groupId>
      <artifactId>seata-spring-boot-starter</artifactId>
      <version>2.3.0</version>
      <exclusions>
        <exclusion>
          <groupId>org.antlr</groupId>
          <artifactId>antlr4-runtime</artifactId>
        </exclusion>
      </exclusions>
    </dependency>
  </dependencies>
</project>
```

The corresponding `application.yml` under classpath needs to contain the following configuration. In this case, the equivalent configuration of Seata's `registry.conf` defined in Spring Boot's `application.yml` is still valid. When downstream projects use the Maven module of `org.apache.shardingsphere:shardingsphere-transaction-base-seata-at`, it is always encouraged to use `registry.conf` to configure Seata Client.

```
seata:
  enable-auto-data-source-proxy: false
```

Mixed use with Seata TCC mode features

For the case of setting up ShardingSphere's Seata integration, In business functions unrelated to ShardingSphere JDBC DataSource, if you need to use Seata Client's Seata TCC mode-related features in business functions, you can instantiate a non-proxy ordinary TCC interface implementation class, and then use `org.apache.integration.tx.api.util.ProxyUtil` to create a proxy TCC interface class, and call the functions corresponding to the three stages of the TCC interface implementation class Try, Confirm, and Cancel.

For the `org.apache.seata.spring.annotation.GlobalTransactional` annotation introduced by the Seata TCC mode or the business functions involved in the Seata TCC mode that need to interact with the database instance, ShardingSphere JDBC DataSource should not be used in the business functions marked by this annotation. Instead, a `javax.sql.DataSource` instance should be created manually or obtained from a custom Spring Bean.

Transactional propagation across service calls

Transactional propagation in cross-service call scenarios is not as out-of-the-box as transaction operations within a single microservice. For Seata Server, transactional propagation in cross-service call scenarios requires passing XID to the service provider through service calls and binding it to `org.apache.seata.core.context.RootContext`. Refer to <https://seata.apache.org/docs/user/api/>. This requires discussing two situations,

1. In the scenario of using ShardingSphere JDBC, transaction scenarios across multiple microservices need to consider using `org.apache.seata.core.context.RootContext.getXID()` to obtain Seata XID in the context of the starting microservice, and passing it to the end microservice through HTTP or RPC, and processing it in the Filter or Spring WebMVC HandlerInterceptor of the end microservice. Spring WebMVC HandlerInterceptor is only applicable to Spring Boot microservices and is invalid for Quarkus, Micronaut Framework and Helidon.
2. In the scenario of using ShardingSphere Proxy, multiple microservices operate local transactions against the logical data source of ShardingSphere Proxy. This will be converted into distributed transaction operations on the server side of ShardingSphere Proxy, without considering additional Seata XID.

Introduce a simple scenario to continue discussing the transactional propagation across service calls in the scenario of using ShardingSphere JDBC.

1. MySQL database instance a-mysql, all databases have created UNDO_LOG table and business table.
2. MySQL database instance b-mysql, all databases have created UNDO_LOG table and business table.
3. Seata Server instance a-seata-server using file as configuration center and registration center.
4. Microservice instance a-service. This microservice creates a ShardingSphere JDBC DataSource that only configures the database instance a-mysql. This ShardingSphere JDBC Data-

Source configuration uses the Seata AT integration connected to the Seata Server instance a-seata-server, whose Seata Application Id is service-a, whose Seata transaction group is default_tx_group, and the Seata Transaction Coordinator cluster group pointed to by its Virtual Group Mapping is default. This microservice instance a-service exposes a single Restful API GET endpoint as /hello, and the business function aMethod of this Restful API endpoint uses a common local transaction annotation. If this microservice is based on Spring Boot 2,

```
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class DemoController {
    @Transactional
    @GetMapping("/hello")
    public String aMethod() {
        // ... Perform an UPDATE operation on the database instance `a-mysql`
        return "Hello World!";
    }
}
```

5. Microservice instance b-service. This microservice creates a ShardingSphere JDBC Data-Source that only configures the database instance b-mysql. This ShardingSphere JDBC Data-Source configuration uses the Seata AT integration connected to the Seata Server instance a-seata-server, whose Seata Application Id is service-b, whose Seata transaction group is default_tx_group, and whose Virtual Group Mapping points to the Seata Transaction Coordinator cluster group as default. The business function bMethod of this microservice instance b-service uses a normal local transaction annotation, and calls the /hello Restful API endpoint of the microservice instance a-service through the HTTP Client in bMethod. If this microservice is based on Spring Boot 2,

```
import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.RestTemplate;

@Service
public class DemoService {
    @Transactional
    public void bMethod() {
        RestTemplate restTemplate = new RestTemplateBuilder().build();
        restTemplate.getForEntity("http://a-service/hello", String.class);
        // ... Perform an UPDATE operation on the database instance `b-mysql`
    }
}
```

For this simple scenario, there is a single Seata Server Cluster, which contains a single Virtual Group

as default. This Virtual Group contains a single Seata Server instance as a-seata-server.

Discuss transaction propagation for single service calls. When the business function aMethod of the microservice instance a-service throws an exception, the changes to the MySQL database instance a-mysql in the business function will be rolled back normally.

Discuss transaction propagation for cross-service calls. When the business function bMethod of the microservice instance b-service throws an exception, the changes to the MySQL database instance b-mysql in the business function will be rolled back normally, and the org.apache.seata.core.context.RootContext of the microservice instance a-service is not bound to the Seata XID of the business function bMethod of the microservice instance b-service, so the changes to the MySQL database instance a-mysql in the business function will not be rolled back.

In order to achieve that when the business function bMethod of the microservice instance b-service throws an exception, the changes to the MySQL database instances a-mysql and b-mysql in the business function are rolled back normally, discuss the common processing solutions in different scenarios.

1. The microservice instances a-service and b-service are both Spring Boot 2 microservices based on Jakarta EE 8. Users can use org.springframework.web.client.RestTemplate in the business function bMethod of the microservice instance b-service to pass the XID to the microservice instance a-service through the service call. The possible transformation logic is as follows.

```
import org.apache.seata.core.context.RootContext;
import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.RestTemplate;

@Service
public class DemoService {
    @Transactional
    public void bMethod() {
        RestTemplate restTemplate = new RestTemplateBuilder().
additionalInterceptors((request, body, execution) -> {
            String xid = RootContext.getXID();
            if (null != xid) {
                request.getHeaders().add(RootContext.KEY_XID, xid);
            }
            return execution.execute(request, body);
        })
        .build();
        restTemplate.getForEntity("http://a-service/hello", String.class);
        // ... Perform an UPDATE operation on the database instance `b-mysql`
    }
}
```

At this time, a custom org.springframework.web.servlet.config.annotation.WebMvcConfigurer implementation needs to be added to the microservice instance a-service.

```
import org.apache.seata.integration.http.TransactionPropagationInterceptor;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CustomWebMvcConfigurer implements WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new TransactionPropagationInterceptor());
    }
}
```

At this time, when the business function `bMethod` of the microservice instance `b-service` throws an exception, the changes to the MySQL database instances `a-mysql` and `b-mysql` in the business function are rolled back normally.

2. The microservice instances `a-service` and `b-service` are both Spring Boot 3 microservices based on Jakarta EE 9/10. Users can use `org.springframework.web.client.RestClient` in the business function `bMethod` of the microservice instance `b-service` to pass the XID to the microservice instance `a-service` through a service call. The possible transformation logic is as follows.

```
import org.apache.seata.core.context.RootContext;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.RestClient;

@Service
public class DemoService {
    @Transactional
    public void bMethod() {
        RestClient restClient = RestClient.builder().requestInterceptor((request,
body, execution) -> {
            String xid = RootContext.getXID();
            if (null != xid) {
                request.getHeaders().add(RootContext.KEY_XID, xid);
            }
            return execution.execute(request, body);
        })
        .build();
        restClient.get().uri("http://a-service/hello").retrieve().body(String.
class);
        // ... Perform an UPDATE operation on the database instance `b-mysql`
    }
}
```


At this time, a custom `org.springframework.web.servlet.config.annotation.WebMvcConfigurer` implementation needs to be added to the microservice instance a-service.

```
import org.apache.seata.integration.http.JakartaTransactionPropagationInterceptor;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CustomWebMvcConfigurer implements WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new JakartaTransactionPropagationInterceptor());
    }
}
```

At this time, when the business function `bMethod` of the microservice instance b-service throws an exception, the changes to the MySQL database instances a-mysql and b-mysql in the business function are rolled back normally.

3. The microservice instances a-service and b-service are both Spring Boot microservices, but the API gateway middleware used blocks all HTTP requests containing the HTTP Header of `TX_XID`. The user needs to consider changing the HTTP Header used to pass `XID` to the microservice instance a-service through service calls, or use the RPC framework to pass `XID` to the microservice instance a-service through service calls. Refer to <https://github.com/apache/incubator-seata/tree/v2.3.0/integration>.
4. The microservice instances a-service and b-service are both microservices such as Quarkus, Micronaut Framework and Helidon. In this case, Spring WebMVC `HandlerInterceptor` cannot be used. You can refer to the following Spring Boot 3 custom `WebMvcConfigurer` implementation to implement Filter.

```
import org.apache.seata.common.util.StringUtils;
import org.apache.seata.core.context.RootContext;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.context.annotation.Configuration;
import org.springframework.lang.NonNull;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CustomWebMvcConfigurer implements WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new HandlerInterceptor() {
            @Override
            public boolean preHandle(@NonNull HttpServletRequest request, @NonNull
```

```

HttpServletResponse response, @NonNull Object handler) {
    String rpcXid = request.getHeader(RootContext.KEY_XID);
    String xid = RootContext.getXID();
    if (StringUtils.isBlank(xid) && StringUtils.isNotBlank(rpcXid)) {
        RootContext.bind(rpcXid);
    }
    return true;
}
@Override
public void afterCompletion(@NonNull HttpServletRequest request,
@NonNull HttpServletResponse response, @NonNull Object handler, Exception ex) {
    if (RootContext.inGlobalTransaction()) {
        String rpcXid = request.getHeader(RootContext.KEY_XID);
        String xid = RootContext.getXID();
        if (StringUtils.isNotBlank(xid)) {
            String unbindXid = RootContext.unbind();
            if (!StringUtils.equalsIgnoreCase(rpcXid, unbindXid)) {
                if (StringUtils.isNotBlank(unbindXid)) {
                    RootContext.bind(unbindXid);
                }
            }
        }
    }
}
});
}
}

```

- Both microservice instances a-service and b-service are Spring Boot microservices, but the components used are Spring WebFlux instead of Spring WebMVC. ShardingSphere JDBC cannot handle R2DBC DataSource under the reactive programming API, only JDBC DataSource. Avoid creating ShardingSphere JDBC DataSource in Spring Boot microservices using WebFlux components.

Log Configuration

After starting Seata Client in a business project, you may see the following Error Log.

```

[ERROR] 2024-12-20 11:46:43.878 [ForkJoinPool.commonPool-worker-1] o.a.s.config.
ConfigurationFactory - failed to load non-spring configuration :not found service
provider for : org.apache.seata.config.ConfigurationProvider
org.apache.seata.common.loader.EnhancedServiceNotFoundException: not found service
provider for : org.apache.seata.config.ConfigurationProvider

```

According to <https://github.com/apache/incubator-seata/issues/6886>, throwing this exception is the expected behavior of Seata Client. Users can configure the log of Seata Client by placing `logback.xml` in the classpath of the business project.

9.1.4 Optional Plugins

ShardingSphere only includes the implementation of the core SPI by default, and there is a part of the SPI that contains third-party dependencies in Git Source Implemented plugins are not included. Retrievable at <https://central.sonatype.com/>.

SPI and existing implementation classes of SPI corresponding to all plugins can be retrieved at <https://shardingsphere.apache.org/document/current/cn/dev-manual/>.

All the built-in plugins for ShardingSphere-JDBC are listed below in the form of 'groupId:artifactId'.

- org.apache.shardingsphere:shardingsphere-authority-core, the user authority to load the logical core
- org.apache.shardingsphere:shardingsphere-cluster-mode-core, the persistent definition core of cluster mode configuration information
- org.apache.shardingsphere:shardingsphere-db-discovery-core, high availability core
- org.apache.shardingsphere:shardingsphere-encrypt-core, data encryption core
- org.apache.shardingsphere:shardingsphere-infra-context, the kernel operation and metadata refresh mechanism of Context
- org.apache.shardingsphere:shardingsphere-logging-core, logging core
- org.apache.shardingsphere:shardingsphere-mask-core, data masking core
- org.apache.shardingsphere:shardingsphere-mysql-dialect-exception, MySQL implementation of database gateway
- org.apache.shardingsphere:shardingsphere-parser-core, SQL parsing core
- org.apache.shardingsphere:shardingsphere-postgresql-dialect-exception, PostgreSQL implementation of database
- org.apache.shardingsphere:shardingsphere-readwrite-splitting-core, read-write splitting core
- org.apache.shardingsphere:shardingsphere-shadow-core, shadow library core
- org.apache.shardingsphere:shardingsphere-sharding-core, data sharding core
- org.apache.shardingsphere:shardingsphere-single-core, single-table (only the only table that exists in all sharded data sources) core
- org.apache.shardingsphere:shardingsphere-sql-federation-core, federation query executor core
- org.apache.shardingsphere:shardingsphere-sql-parser-mysql, MySQL dialect implementation of SQL parsing
- org.apache.shardingsphere:shardingsphere-sql-parser-postgresql, PostgreSQL dialect implementation of SQL parsing

- `org.apache.shardingsphere:shardingsphere-sql-parser-opengauss`, OpenGauss dialect implementation of SQL parsing
- `org.apache.shardingsphere:shardingsphere-sql-parser-oracle`, Oracle dialect implementation of SQL parsing
- `org.apache.shardingsphere:shardingsphere-sql-parser-sqlserver`, SQL Server dialect implementation of SQL parsing
- `org.apache.shardingsphere:shardingsphere-sql-parser-doris`, Doris dialect implementation of SQL parsing
- `org.apache.shardingsphere:shardingsphere-sql-parser-presto`, Presto dialect implementation of SQL parsing
- `org.apache.shardingsphere:shardingsphere-sql-parser-sql92`, the SQL 92 dialect implementation of SQL parsing
- `org.apache.shardingsphere:shardingsphere-standalone-mode-core`, the persistence definition core of single-machine mode configuration information
- `org.apache.shardingsphere:shardingsphere-standalone-mode-repository-jdbc-h2`, H2 implementation of persistent definition of configuration information in stand-alone mode
- `org.apache.shardingsphere:shardingsphere-traffic-core`, traffic governance core
- `org.apache.shardingsphere:shardingsphere-transaction-core`, XA Distributed Transaction Manager Core

If ShardingSphere-JDBC needs to use optional plugins, you need to download the JAR containing its SPI implementation and its dependent JARs from Maven Central.

All optional plugins are listed below in the form of `groupId:artifactId`.

- Cluster mode configuration information persistence definition
 - `org.apache.shardingsphere:shardingsphere-cluster-mode-repository-zookeeper`, Zookeeper based persistence
 - `org.apache.shardingsphere:shardingsphere-cluster-mode-repository-etcd`, Etcd based persistence
- XA transaction manager provider definition
 - `org.apache.shardingsphere:shardingsphere-transaction-xa-narayana`, XA distributed transaction manager based on Narayana
- Row Value Expressions definition
 - `org.apache.shardingsphere:shardingsphere-infra-expr-espresso`, Row Value Expressions that uses the Groovy syntax based on GraalVM Truffle's Espresso implementation
- Database type identification
 - `org.apache.shardingsphere:shardingsphere-infra-database-hive`, Adaptation of jdbcURL for JDBC support of Hive, and metadata loading implementation

- `org.apache.shardingsphere:shardingsphere-infra-database-presto`, Adaptation of `jdbcURL` for JDBC support of Presto, and metadata loading implementation
- SQL parsing
 - `org.apache.shardingsphere:shardingsphere-parser-sql-clickhouse`, ClickHouse dialect implementation of SQL parsing
 - `org.apache.shardingsphere:shardingsphere-parser-sql-hive`, Hive dialect implementation of SQL parsing

In addition to the above optional plugins, ShardingSphere community developers have contributed a number of plugin implementations. These plugins can be found in [ShardingSphere Plugins](#) repository. Plugins in ShardingSphere Plugin repository would remain the same release plan with ShardingSphere, you can build plugin jar by yourself, and install into ShardingSphere.

ClickHouse

Background Information

ShardingSphere does not provide support for `driverClassName` of `com.clickhouse.jdbc.ClickHouseDriver` by default. ShardingSphere's support for ClickHouse JDBC Driver is in the optional module.

Prerequisites

To use a `jdbcUrl` like `jdbc:ch://localhost:8123/demo_ds_0` for the data node in the ShardingSphere configuration file, the possible Maven dependencies are as follows,

```
<dependencies>
  <dependency>
    <groupId>org.apache.shardingsphere</groupId>
    <artifactId>shardingsphere-jdbc</artifactId>
    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.shardingsphere</groupId>
    <artifactId>shardingsphere-parser-sql-clickhouse</artifactId>
    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>com.clickhouse</groupId>
    <artifactId>clickhouse-jdbc</artifactId>
    <classifier>http</classifier>
    <version>0.6.3</version>
  </dependency>
</dependencies>
```

Configuration example

Start ClickHouse

Write a Docker Compose file to start ClickHouse.

```
services:
  clickhouse-server:
    image: clickhouse/clickhouse-server:24.11.1.2557
    ports:
      - "8123:8123"
```

Create business tables

Use a third-party tool to create a business database and business table in ClickHouse. Taking DBeaver Community as an example, if you use Ubuntu 22.04.4, you can quickly install it through Snapcraft.

```
sudo apt update && sudo apt upgrade -y
sudo snap install dbeaver-ce
snap run dbeaver-ce
```

In DBeaver Community, use jdbcUrl of jdbc:ch://localhost:8123/default, username of default to connect to ClickHouse, and leave password blank. Execute the following SQL,

```
-- noinspection SqlNoDataSourceInspectionForFile
CREATE DATABASE demo_ds_0;
CREATE DATABASE demo_ds_1;
CREATE DATABASE demo_ds_2;
```

Use jdbcUrl of jdbc:ch://localhost:8123/demo_ds_0, jdbc:ch://localhost:8123/demo_ds_1 and jdbc:ch://localhost:8123/demo_ds_2 to connect to ClickHouse and execute the following SQL.

```
-- noinspection SqlNoDataSourceInspectionForFile
create table IF NOT EXISTS t_order (
  order_id  Int64 NOT NULL,
  order_type Int32,
  user_id   Int32 NOT NULL,
  address_id Int64 NOT NULL,
  status    VARCHAR(50)
) engine = MergeTree
  primary key (order_id)
  order by (order_id);

TRUNCATE TABLE t_order;
```

Create ShardingSphere data source in business project

After the business project introduces the dependencies involved in prerequisites, write the ShardingSphere data source configuration file `demo.yaml` on the classpath of the business project.

```
dataSources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.clickhouse.jdbc.ClickHouseDriver
    jdbcUrl: jdbc:ch://localhost:8123/demo_ds_0
    username: default
    password:
  ds_1:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.clickhouse.jdbc.ClickHouseDriver
    jdbcUrl: jdbc:ch://localhost:8123/demo_ds_1
    username: default
    password:
  ds_2:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.clickhouse.jdbc.ClickHouseDriver
    jdbcUrl: jdbc:ch://localhost:8123/demo_ds_2
    username: default
    password:
rules:
- !SHARDING
  tables:
    t_order:
      actualDataNodes:
      keyGenerateStrategy:
        column: order_id
        keyGeneratorName: snowflake
  defaultDatabaseStrategy:
    standard:
      shardingColumn: user_id
      shardingAlgorithmName: inline
  shardingAlgorithms:
    inline:
      type: INLINE
      props:
        algorithm-expression: ds_${user_id % 2}
  keyGenerators:
    snowflake:
      type: SNOWFLAKE
```

Enjoy integration

Create a ShardingSphere data source to enjoy integration,

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
public class ExampleUtils {
    void test() throws SQLException {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:shardingsphere:classpath:demo.yaml");
        config.setDriverClassName("org.apache.shardingsphere.driver.
ShardingSphereDriver");
        try (HikariDataSource dataSource = new HikariDataSource(config);
            Connection connection = dataSource.getConnection();
            Statement statement = connection.createStatement()) {
            statement.execute("INSERT INTO t_order (user_id, order_type, address_id,
status) VALUES (1, 1, 1, 'INSERT_TEST')");
            statement.executeQuery("SELECT * FROM t_order");
            statement.execute("alter table t_order delete where order_id=1");
        }
    }
}
```

Usage Limitations

SQL Limitations

ShardingSphere JDBC DataSource does not yet support executing ClickHouse's create table, truncate table, and drop table statements. Users should consider submitting a PR containing unit tests for ShardingSphere.

Key Generate restrictions

The column type corresponding to the Key Generate function of ClickHouse itself is UUID, and UUID is received as `java.util.UUID` in ClickHouse JDBC Driver, refer to <https://github.com/ClickHouse/ClickHouse/issues/56228>. The column type corresponding to the Key Generate SPI implementation of ShardingSphere's SNOWFLAKE is `UInt64`, which is received as `java.lang.Long` in ShardingSphere JDBC Driver.

When configuring ShardingSphere to connect to ClickHouse, if ShardingSphere is also configured to use the Key Generate SPI implementation of SNOWFLAKE, the column type in the ClickHouse real database used by ShardingSphere's Key Generate function should not be set to UUID.

Because `com.clickhouse.jdbc.ClickHouseConnection#prepareStatement(String, int)` of `com.clickhouse:clickhouse-jdbc:0.6.3:http` Maven module intentionally throws an exception when `autoGeneratedKeys` is `java.sql.Statement.RETURN_GENERATED_KEYS`, to prevent ShardingSphere from proxying `com.clickhouse.jdbc.internal.ClickHouseConnectionImpl` normally, therefore, if users need to obtain the Key generated by ShardingSphere from the JDBC business code, they need to set `autoGeneratedKeys` to `java.sql.Statement.NO_GENERATED_KEYS`.

A possible example is as follows,

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.*;
public class ExampleTest {
    long test() throws SQLException {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:shardingsphere:classpath:demo.yaml");
        config.setDriverClassName("org.apache.shardingsphere.driver.
ShardingSphereDriver");
        try (HikariDataSource dataSource = new HikariDataSource(config);
            Connection connection = dataSource.getConnection();
            PreparedStatement preparedStatement = connection.prepareStatement(
                "INSERT INTO t_order (user_id, order_type, address_id, status)
VALUES (1, 1, 1, 'INSERT_TEST')",
                Statement.NO_GENERATED_KEYS
            )) {
            preparedStatement.executeUpdate();
            try (ResultSet resultSet = preparedStatement.getGeneratedKeys()) {
                if (resultSet.next()) {
                    return resultSet.getLong(1);
                }
                throw new RuntimeException();
            }
        }
    }
}
```

Transaction Limitations

ClickHouse does not support local transactions at the ShardingSphere integration level, XA transactions, or AT mode transactions for Seata, More discussion is at <https://github.com/ClickHouse/clickhouse-docs/issues/2300>.

This has nothing to do with the Transactions, Commit, and Rollback feature provided by <https://clickhouse.com/docs/en/guides/developer/transactional> for ClickHouse, but only with `com.clickhouse.jdbc.ConnectionImpl` not implementing `java.sql.Connection#rollback()`. See <https://github.com/ClickHouse/clickhouse-java/issues/2023>.

Embedded ClickHouse Limitations

The embedded ClickHouse chDB Java client has not been released yet. ShardingSphere does not do integration testing for the SNAPSHOT version of <https://github.com/chdb-io/chdb-java>. Refer to <https://github.com/chdb-io/chdb/issues/243>.

HiveServer2

Background Information

ShardingSphere does not provide support for `driverClassName` of `org.apache.hive.jdbc.HiveDriver` by default.

ShardingSphere's support for HiveServer2 JDBC Driver is in the optional module.

Prerequisites

To use a `jdbcUrl` like `jdbc:hive2://localhost:10000/` for the data node in the ShardingSphere configuration file, The possible Maven dependencies are as follows.

```
<dependencies>
  <dependency>
    <groupId>org.apache.shardingsphere</groupId>
    <artifactId>shardingsphere-jdbc</artifactId>
    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.shardingsphere</groupId>
    <artifactId>shardingsphere-infra-database-hive</artifactId>
    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.shardingsphere</groupId>
    <artifactId>shardingsphere-parser-sql-hive</artifactId>
    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.hive</groupId>
    <artifactId>hive-jdbc</artifactId>
    <version>4.0.1</version>
  </dependency>
  <dependency>
    <groupId>org.apache.hive</groupId>
    <artifactId>hive-service</artifactId>
    <version>4.0.1</version>
  </dependency>
</dependencies>
```

```

    <groupId>org.apache.hadoop</groupId>
    <artifactId>hadoop-mapreduce-client-core</artifactId>
    <version>3.3.6</version>
    <exclusions>
      <exclusion>
        <groupId>*</groupId>
        <artifactId>*</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
</dependencies>

```

Optional shortcut to resolve dependency conflicts

Using `org.apache.hive:hive-jdbc:4.0.1` directly will cause a large number of dependency conflicts. If users do not want to manually resolve potentially thousands of lines of dependency conflicts, they can use a third-party build of the HiveServer2 JDBC Driver Thin JAR. The following is an example of a possible configuration,

```

<dependencies>
  <dependency>
    <groupId>org.apache.shardingsphere</groupId>
    <artifactId>shardingsphere-jdbc</artifactId>
    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.shardingsphere</groupId>
    <artifactId>shardingsphere-infra-database-hive</artifactId>
    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.shardingsphere</groupId>
    <artifactId>shardingsphere-parser-sql-hive</artifactId>
    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>io.github.linghengqian</groupId>
    <artifactId>hive-server2-jdbc-driver-thin</artifactId>
    <version>1.7.0</version>
    <exclusions>
      <exclusion>
        <groupId>com.fasterxml.woodstox</groupId>
        <artifactId>woodstox-core</artifactId>
      </exclusion>
      <exclusion>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-text</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
</dependencies>

```

```

        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.apache.hadoop</groupId>
    <artifactId>hadoop-mapreduce-client-core</artifactId>
    <version>3.3.6</version>
    <exclusions>
        <exclusion>
            <groupId>*</groupId>
            <artifactId>*</artifactId>
        </exclusion>
    </exclusions>
</dependency>
</dependencies>

```

Configuration Example

Start HiveServer2

Write a Docker Compose file to start HiveServer2.

```

services:
  hive-server2:
    image: apache/hive:4.0.1
    environment:
      SERVICE_NAME: hiveserver2
    ports:
      - "10000:10000"

```

Create business tables

Use a third-party tool to create a business database and business table in HiveServer2. Taking DBeaver Community as an example, if you use Ubuntu 22.04.4, you can quickly install it through Snapcraft.

```

sudo apt update && sudo apt upgrade -y
sudo snap install dbeaver-ce
snap run dbeaver-ce

```

In DBeaver Community, use the jdbcUrl of jdbc:hive2://localhost:10000/ to connect to HiveServer2, and leave username and password blank. Execute the following SQL,

```

-- noinspection SqlNoDataSourceInspectionForFile
CREATE DATABASE demo_ds_0;
CREATE DATABASE demo_ds_1;
CREATE DATABASE demo_ds_2;

```

Use the `jdbcUrl` of `jdbc:hive2://localhost:10000/demo_ds_0`, `jdbc:hive2://localhost:10000/demo_ds_1` and `jdbc:hive2://localhost:10000/demo_ds_2` to connect to HiveServer2 to execute the following SQL,

```
-- noinspection SqlNoDataSourceInspectionForFile
CREATE TABLE IF NOT EXISTS t_order
(
    order_id    BIGINT NOT NULL,
    order_type  INT,
    user_id     INT     NOT NULL,
    address_id  BIGINT NOT NULL,
    status      string,
    PRIMARY KEY (order_id) disable novalidate
) STORED BY ICEBERG STORED AS ORC TBLPROPERTIES ('format-version' = '2');

TRUNCATE TABLE t_order;
```

Create ShardingSphere data source in business projects

After the business project introduces the dependencies involved in prerequisites, write the ShardingSphere data source configuration file `demo.yaml` on the classpath of the business project.

```
dataSources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: jdbc:hive2://localhost:10000/demo_ds_0
  ds_1:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: jdbc:hive2://localhost:10000/demo_ds_1
  ds_2:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: jdbc:hive2://localhost:10000/demo_ds_2
rules:
- !SHARDING
  tables:
    t_order:
      actualDataNodes:
      keyGenerateStrategy:
        column: order_id
        keyGeneratorName: snowflake
      defaultDatabaseStrategy:
        standard:
          shardingColumn: user_id
          shardingAlgorithmName: inline
      shardingAlgorithms:
```

```

inline:
  type: INLINE
  props:
    algorithm-expression: ds_${user_id % 2}
keyGenerators:
  snowflake:
    type: SNOWFLAKE

```

Enjoy the integration

Create a ShardingSphere data source to enjoy the integration.

```

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
public class ExampleUtils {
    void test() throws SQLException {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:shardingsphere:classpath:demo.yaml");
        config.setDriverClassName("org.apache.shardingsphere.driver.
ShardingSphereDriver");
        try (HikariDataSource dataSource = new HikariDataSource(config);
            Connection connection = dataSource.getConnection();
            Statement statement = connection.createStatement()) {
            statement.execute("INSERT INTO t_order (user_id, order_type, address_id,
status) VALUES (1, 1, 1, 'INSERT_TEST')");
            statement.executeQuery("SELECT * FROM t_order");
            statement.execute("DELETE FROM t_order WHERE order_id=1");
        }
    }
}

```

External Integration

Connect to HiveServer2 with ZooKeeper Service Discovery enabled

`jdbcUrl` in the ShardingSphere configuration file can be configured to connect to HiveServer2 with ZooKeeper Service Discovery enabled.

For discussion, assume that there is the following Docker Compose file to start HiveServer2 with ZooKeeper Service Discovery.

```

name: test-1
services:

```

```
zookeeper:
  image: zookeeper:3.9.3-jre-17
  ports:
    - "2181:2181"
apache-hive-1:
  image: apache/hive:4.0.1
  depends_on:
    - zookeeper
  environment:
    SERVICE_NAME: hiveserver2
    SERVICE_OPTS: >-
      -Dhive.server2.support.dynamic.service.discovery=true
      -Dhive.zookeeper.quorum=zookeeper:2181
      -Dhive.server2.thrift.bind.host=0.0.0.0
      -Dhive.server2.thrift.port=10000
  ports:
    - "10000:10000"
```

In DBeaver Community, use jdbcUrl of jdbc:hive2://127.0.0.1:2181/; serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2 to connect to HiveServer2, leave username and password blank. Execute the following SQL,

```
-- noinspection SqlNoDataSourceInspectionForFile
CREATE DATABASE demo_ds_0;
CREATE DATABASE demo_ds_1;
CREATE DATABASE demo_ds_2;
```

Use jdbcUrl of jdbc:hive2://127.0.0.1:2181/demo_ds_0;serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2, jdbc:hive2://127.0.0.1:2181/demo_ds_1; serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2 and jdbc:hive2://127.0.0.1:2181/demo_ds_2;serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2 to connect to HiveServer2 and execute the following SQL,

```
-- noinspection SqlNoDataSourceInspectionForFile
CREATE TABLE IF NOT EXISTS t_order
(
  order_id BIGINT NOT NULL,
  order_type INT,
  user_id INT NOT NULL,
  address_id BIGINT NOT NULL,
  status string,
  PRIMARY KEY (order_id) disable novalidate
) STORED BY ICEBERG STORED AS ORC TBLPROPERTIES ('format-version' = '2');

TRUNCATE TABLE t_order;
```

After the business project introduces the dependencies involved in the prerequisites, write the

ShardingSphere data source configuration file `demo.yaml` on the classpath of the business project.

```
dataSources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: jdbc:hive2://127.0.0.1:2181/demo_ds_0;
serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2
  ds_1:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: jdbc:hive2://127.0.0.1:2181/demo_ds_1;
serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2
  ds_2:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: jdbc:hive2://127.0.0.1:2181/demo_ds_2;
serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2
rules:
- !SHARDING
  tables:
    t_order:
      actualDataNodes:
      keyGenerateStrategy:
        column: order_id
        keyGeneratorName: snowflake
      defaultDatabaseStrategy:
        standard:
          shardingColumn: user_id
          shardingAlgorithmName: inline
      shardingAlgorithms:
        inline:
          type: INLINE
          props:
            algorithm-expression: ds_${user_id % 2}
      keyGenerators:
        snowflake:
          type: SNOWFLAKE
```

At this point, you can create the ShardingSphere data source normally and execute logical SQL on the virtual data source.

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
public class ExampleUtils {
    void test() throws SQLException {
```



```

HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:shardingsphere:classpath:demo.yaml");
config.setDriverClassName("org.apache.shardingsphere.driver.
ShardingSphereDriver");
    try (HikariDataSource dataSource = new HikariDataSource(config);
        Connection connection = dataSource.getConnection();
        Statement statement = connection.createStatement()) {
        statement.execute("INSERT INTO t_order (user_id, order_type, address_id,
status) VALUES (1, 1, 1, 'INSERT_TEST')");
        statement.executeQuery("SELECT * FROM t_order");
        statement.execute("DELETE FROM t_order WHERE order_id=1");
    }
}
}

```

At this point, if the HiveServer2 example with the service named apache-hive-1 is manually destroyed, start the second HiveServer2 instance through another Docker Compose file with the following content,

```

name: test-2
services:
  apache-hive-2:
    image: apache/hive:4.0.1
    environment:
      SERVICE_NAME: hiveserver2
      SERVICE_OPTS: >-
        -Dhive.server2.support.dynamic.service.discovery=true
        -Dhive.zookeeper.quorum=zookeeper:2181
        -Dhive.server2.thrift.bind.host=0.0.0.0
        -Dhive.server2.thrift.port=20000
    ports:
      - "20000:20000"
    networks:
      - test-1_default
networks:
  test-1_default:
    external: true

```

In DBeaver Community, use jdbcUrl of jdbc:hive2://127.0.0.1:2181/; serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2 to connect to HiveServer2, leave username and password blank. Execute the following SQL,

```

-- noinspection SqlNoDataSourceInspectionForFile
CREATE DATABASE demo_ds_0;
CREATE DATABASE demo_ds_1;
CREATE DATABASE demo_ds_2;

```

Use jdbcUrl of jdbc:hive2://127.0.0.1:2181/demo_ds_0;serviceDiscoveryMode=zooKeeper;

zooKeeperNamespace=hiveserver2, jdbc:hive2://127.0.0.1:2181/demo_ds_1;
serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2 and
jdbc:hive2://127.0.0.1:2181/demo_ds_2;serviceDiscoveryMode=zooKeeper;
zooKeeperNamespace=hiveserver2 to connect to HiveServer2 and execute the following SQL,

```
-- noinspection SqlNoDataSourceInspectionForFile
CREATE TABLE IF NOT EXISTS t_order
(
    order_id    BIGINT NOT NULL,
    order_type  INT,
    user_id     INT     NOT NULL,
    address_id  BIGINT NOT NULL,
    status      string,
    PRIMARY KEY (order_id) disable novalidate
) STORED BY ICEBERG STORED AS ORC TBLPROPERTIES ('format-version' = '2');

TRUNCATE TABLE t_order;
```

At this point, the old ShardingSphere JDBC DataSource can still be switched to the HiveServer2 instance named apache-hive-2 in the service to execute the logical SQL without recreating the JDBC DataSource.

```
import com.zaxxer.hikari.HikariDataSource;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
public class ExampleUtils {
    void test(HikariDataSource dataSource) throws SQLException {
        try (Connection connection = dataSource.getConnection();
             Statement statement = connection.createStatement()) {
            statement.execute("INSERT INTO t_order (user_id, order_type, address_id,
status) VALUES (1, 1, 1, 'INSERT_TEST')");
            statement.executeQuery("SELECT * FROM t_order");
            statement.execute("DELETE FROM t_order WHERE order_id=1");
        }
    }
}
```

Usage Restrictions

Version Restrictions

The lifecycle of HiveServer2 2.x and HiveServer2 3.x releases has ended. Refer to <https://lists.apache.org/thread/0mh4hvp1lzv877b1f9srv1c3hlbt9> and <https://lists.apache.org/thread/mpzrv7v1hqo4c1mp0zorswnbvd7ltmbp>. ShardingSphere is only integrated tested for HiveServer2 4.0.1.

Uber JAR Limitation of HiveServer2 JDBC Driver

Affected by <https://issues.apache.org/jira/browse/HIVE-28445>, users should not use `org.apache.hive:hive-jdbc:4.0.1` with classifier as standalone to avoid dependency conflicts.

Embedded HiveServer2 Limitation

Embedded HiveServer2 is no longer considered user-friendly by the Hive community, and users should not try to start embedded HiveServer2 through ShardingSphere's configuration file. Users should always start HiveServer2 through HiveServer2's Docker Image `apache/hive:4.0.1`. Reference <https://issues.apache.org/jira/browse/HIVE-28418>.

Hadoop Limitations

Users can only use Hadoop 3.3.6 as the underlying Hadoop dependency of HiveServer2 JDBC Driver 4.0.1. HiveServer2 JDBC Driver 4.0.1 does not support Hadoop 3.4.1. Reference <https://github.com/apache/hive/pull/5500>.

For HiveServer2 JDBC Driver `org.apache.hive:hive-jdbc:4.0.1` or `org.apache.hive:hive-jdbc:4.0.1` with classifier as standalone, there is actually no additional dependency on `org.apache.hadoop:hadoop-mapreduce-client-core:3.3.6`.

But `org.apache.shardingsphere:shardingsphere-infra-database-hive`'s `org.apache.shardingsphere.infra.database.hive.metadata.data.loader.HiveMetaDataReader` uses `org.apache.hadoop.hive.conf.HiveConf`, which further uses `org.apache.hadoop:hadoop-mapreduce-client-core:3.3.6`'s `org.apache.hadoop.mapred.JobConf` class.

ShardingSphere only needs to use the `org.apache.hadoop.mapred.JobConf` class, so it is reasonable to exclude all additional dependencies of `org.apache.hadoop:hadoop-mapreduce-client-core:3.3.6`.

```
<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-mapreduce-client-core</artifactId>
  <version>3.3.6</version>
  <exclusions>
    <exclusion>
```

```

        <groupId>*</groupId>
        <artifactId>*</artifactId>
    </exclusion>
</exclusions>
</dependency>

```

SQL Limitations

HiveServer2 does not guarantee that every `insert` related DML SQL can be executed successfully, although no exception may be thrown.

ShardingSphere JDBC DataSource does not yet support executing HiveServer2' s `set`, `create table`, `truncate table`, and `drop table` statements. Users should consider submitting a PR containing unit tests for ShardingSphere.

Use `initFile` parameter to partially bypass SQL restrictions

Affected by <https://issues.apache.org/jira/browse/HIVE-28835> , the `initFile` parameter of HiveServer2 JDBC Driver is only available in Linux environment.

SQL statements represented by `set` can be easily configured dynamically at the HiveServer2 Client level. Even though ShardingSphere JDBC does not support executing HiveServer2' s `set` statement on a virtual DataSource, users can directly execute a series of SQLs for the real DataSource through the Hive Session parameter of `initFile`. For discussion, the possible ShardingSphere configuration files are as follows,

```

dataSources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: jdbc:hive2://localhost:10000/demo_ds_0;initFile=/tmp/init.sql
  ds_1:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: jdbc:hive2://localhost:10000/demo_ds_0;initFile=/tmp/init.sql
  ds_2:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: jdbc:hive2://localhost:10000/demo_ds_0;initFile=/tmp/init.sql

```

The possible contents of `/tmp/init.sql` are as follows,

```

-- noinspection SqlNoDataSourceInspectionForFile
set metastore.compactor.initiator.on=true;
set metastore.compactor.cleaner.on=true;
set metastore.compactor.worker.threads=1;

```

```
set hive.support.concurrency=true;
set hive.exec.dynamic.partition.mode=nonstrict;
set hive.txn.manager=org.apache.hadoop.hive.ql.lockmgr.DbTxnManager;
```

Affected by <https://issues.apache.org/jira/browse/HIVE-28317>, the `initFile` parameter can only use absolute paths. However, ShardingSphere JDBC Driver has a `placeholder`-type parameter to dynamically define YAML properties. Further discussion, possible ShardingSphere configuration files are as follows,

```
dataSources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: ${fixture.hive.ds0.jdbc-url::}
  ds_1:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: ${fixture.hive.ds1.jdbc-url::}
  ds_2:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: org.apache.hive.jdbc.HiveDriver
    jdbcUrl: ${fixture.hive.ds2.jdbc-url::}
```

When using ShardingSphere JDBC Driver, user can pass in the absolute path of the file on the classpath of the business project by concatenating strings.

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import javax.sql.DataSource;
import java.nio.file.Paths;
public class ExampleUtils {
    public DataSource createDataSource() {
        HikariConfig config = new HikariConfig();
        config.setDriverClassName("org.apache.shardingsphere.driver.
ShardingSphereDriver");
        config.setJdbcUrl("jdbc:shardingsphere:classpath:demo.yaml?placeholder-
type=system_props");
        try {
            assert null == System.getProperty("fixture.hive.ds0.jdbc-url");
            assert null == System.getProperty("fixture.hive.ds1.jdbc-url");
            assert null == System.getProperty("fixture.hive.ds2.jdbc-url");
            String absolutePath = Paths.get("src/test/resources/init.sql").
toAbsolutePath().toString();
            System.setProperty("fixture.hive.ds0.jdbc-url", "jdbc:hive2://
localhost:10000/demo_ds_0;initFile=" + absolutePath);
            System.setProperty("fixture.hive.ds1.jdbc-url", "jdbc:hive2://
localhost:10000/demo_ds_1;initFile=" + absolutePath);
            System.setProperty("fixture.hive.ds2.jdbc-url", "jdbc:hive2://
```

```
localhost:10000/demo_ds_2;initFile=" + absolutePath);
    return new HikariDataSource(config);
} finally {
    System.clearProperty("fixture.hive.ds0.jdbc-url");
    System.clearProperty("fixture.hive.ds1.jdbc-url");
    System.clearProperty("fixture.hive.ds2.jdbc-url");
}
}
}
```

Prerequisites for using DML SQL statements on ShardingSphere data sources

In order to be able to use DML SQL statements such as delete, users should consider using only ACID-supported tables in ShardingSphere JDBC when connecting to HiveServer2. apache/hive provides multiple transaction solutions.

The first option is to use ACID tables. The possible table creation process is as follows. ACID tables use the outdated directory-based table format.

```
-- noinspection SqlNoDataSourceInspectionForFile
set metastore.compactor.initiator.on=true;
set metastore.compactor.cleaner.on=true;
set metastore.compactor.worker.threads=1;

set hive.support.concurrency=true;
set hive.exec.dynamic.partition.mode=nonstrict;
set hive.txn.manager=org.apache.hadoop.hive.ql.lockmgr.DbTxnManager;

create table IF NOT EXISTS t_order
(
    order_id    BIGINT NOT NULL,
    order_type  INT,
    user_id     INT     NOT NULL,
    address_id  BIGINT NOT NULL,
    status      VARCHAR(50),
    PRIMARY KEY (order_id) disable novalidate
) CLUSTERED BY (order_id) INTO 2 BUCKETS STORED AS ORC TBLPROPERTIES ('transactional' = 'true');
```

The second option is to use Iceberg tables. The possible table creation process is as follows. Apache Iceberg table format is expected to replace the traditional Hive table format in the next few years. Refer to <https://blog.cloudera.com/from-hive-tables-to-iceberg-tables-hassle-free/>.

```
-- noinspection SqlNoDataSourceInspectionForFile
CREATE TABLE IF NOT EXISTS t_order
(
    order_id    BIGINT NOT NULL,
```

```

order_type INT,
user_id    INT    NOT NULL,
address_id BIGINT NOT NULL,
status     string,
PRIMARY KEY (order_id) disable novalidate
) STORED BY ICEBERG STORED AS ORC TBLPROPERTIES ('format-version' = '2');

```

Iceberg table format supports relatively few Hive types. Executing SQL `set iceberg.mr.schema.auto.conversion=true;` for HiveServer2 can help alleviate this problem. SQL `set iceberg.mr.schema.auto.conversion=true;` has the drawbacks mentioned in <https://issues.apache.org/jira/browse/HIVE-26507>.

Transaction Limitations

HiveServer2 does not support local transactions at the ShardingSphere integration level, XA transactions, or Seata's AT mode transactions. For more discussion, please visit <https://cwiki.apache.org/confluence/display/Hive/Hive+Transactions>.

This has nothing to do with the Table rollback feature provided by <https://iceberg.apache.org/docs/1.7.0/hive/#table-rollback> for HiveServer2, but only with `org.apache.hive.jdbc.HiveConnection` not implementing `java.sql.Connection#rollback()`.

DBeaver Community Version Limitations

When users use DBeaver Community to connect to HiveServer2, they need to ensure that the DBeaver Community version is greater than or equal to 24.2.5.

See <https://github.com/dbeaver/dbeaver/pull/35059>.

9.1.5 Unsupported Items

Configuration

- Do not support configuring multiple logic databases

DataSource Interface

- Do not support timeout related operations

Connection Interface

- Do not support operations of stored procedure, function and cursor
- Do not support native SQL
- Do not support savepoint related operations
- Do not support Schema/Catalog operation
- Do not support self-defined type mapping

Statement and PreparedStatement Interface

- Do not support statements that return multiple result sets (stored procedures, multiple pieces of non-SELECT data)
- Do not support the operation of international characters

ResultSet Interface

- Do not support getting result set pointer position
- Do not support changing result pointer position through none-next method
- Do not support revising the content of result set
- Do not support acquiring international characters
- Do not support getting Array

JDBC 4.1

- Do not support new functions of JDBC 4.1 interface

For all the unsupported methods, please read `org.apache.shardingsphere.driver.jdbc.unsupported` package.

9.1.6 Observability

Agent

Compile source code

Download Apache ShardingSphere from GitHub,Then compile.

```
git clone --depth 1 https://github.com/apache/shardingsphere.git
cd shardingsphere
mvn clean install -DskipITs -DskipTests -Prelease,default-dep
```


Agent artifact is `distribution/agent/target/apache-shardingsphere-${latest.release.version}-shardingsphere-agent-bin.tar.gz`

Directory structure

Create agent directory, and unzip agent distribution package to the directory.

```
mkdir agent
tar -zxvf apache-shardingsphere-${latest.release.version}-shardingsphere-agent-bin.tar.gz -C agent
cd agent
tree
├── LICENSE
├── NOTICE
├── conf
│   └── agent.yaml
├── plugins
│   ├── lib
│   │   ├── shardingsphere-agent-metrics-core-${latest.release.version}.jar
│   │   └── shardingsphere-agent-plugin-core-${latest.release.version}.jar
│   ├── logging
│   │   └── shardingsphere-agent-logging-file-${latest.release.version}.jar
│   ├── metrics
│   │   └── shardingsphere-agent-metrics-prometheus-${latest.release.version}.jar
│   └── tracing
│       └── shardingsphere-agent-tracing-opentelemetry-${latest.release.version}.jar
└── shardingsphere-agent-${latest.release.version}.jar
```

Configuration

`conf/agent.yaml` is used to manage agent configuration. Built-in plugins include File, Prometheus, OpenTelemetry.

```
plugins:
# logging:
#   File:
#     props:
#       level: "INFO"
# metrics:
#   Prometheus:
#     host: "localhost"
#     port: 9090
#     props:
#       jvm-information-collector-enabled: "true"
# tracing:
#   OpenTelemetry:
#     props:
```

```
# otel.service.name: "shardingsphere"
# otel.traces.exporter: "jaeger"
# otel.exporter.otlp.traces.endpoint: "http://localhost:14250"
# otel.traces.sampler: "always_on"
```

Plugin description

File

Currently, the File plugin only outputs the time-consuming log output of building metadata, and has no other log output for the time being.

Prometheus

Used for exposure monitoring metrics.

- Parameter description

Name	Description
host	host IP
port	port
jvm-information-collector-enabled	whether to collect JVM indicator information

OpenTelemetry

OpenTelemetry can export tracing data to Jaeger, Zipkin.

- Parameter description

Name	Description
otel.service.name	service name
otel.traces.exporter	traces exporter
otel.exporter.otlp.traces.endpoint	traces endpoint
otel.traces.sampler	traces sampler

Parameter reference [OpenTelemetry SDK Autoconfigure](#)

Usage

- 1 The SpringBoot project ready to integrate ShardingSphere-JDBC, test-project.jar
- 2 Startup project

```
java -javaagent:/agent/shardingsphere-agent-${latest.release.version}.jar -jar test-project.jar
```

- 3 Access to started service
- 4 Check whether the corresponding plug-in is effective

Docker

Local Build

ShardingSphere Agent has a Dockerfile available for easy distribution. You can execute the following command to build a Docker Image,

```
git clone git@github.com:apache/shardingsphere.git
cd ./shardingsphere/
./mvnw -am -pl distribution/agent -Prelease,default-dep,docker -T1C -DskipTests
clean package
```

If you add the following statement in your custom Dockerfile, it will copy the ShardingSphere Agent directory to /shardingsphere-agent/.

```
COPY --from=ghcr.io/apache/shardingsphere-agent:latest /usr/agent/ /shardingsphere-agent/
```

Community Build

Since ShardingSphere 5.5.2, ShardingSphere Agent has released community builds at <https://github.com/apache/shardingsphere/pkgs/container/shardingsphere-agent>. This Docker Image is not part of the ASF distribution, but is provided for convenience.

If you add the following statement in a custom Dockerfile, it will copy the ShardingSphere Agent directory to /shardingsphere-agent/.

```
COPY --from=ghcr.io/apache/shardingsphere-agent:5.5.2 /usr/agent/ /shardingsphere-agent/
```

Nightly Build

ShardingSphere Agent has a nightly built Docker Image at <https://github.com/apache/shardingsphere/pkgs/container/shardingsphere-agent>.

If you add the following statement in your custom Dockerfile, it will copy the ShardingSphere Agent directory to /shardingsphere-agent/.

```
COPY --from=ghcr.io/apache/shardingsphere-agent:latest /usr/agent/ /shardingsphere-agent/
```

Using Dockerfile

Introduce a typical scenario,

1. Assume that the Jaeger All in One Docker Container is deployed through the following Bash command,

```
docker network create example-net
docker run --rm -d \
  --name jaeger \
  -e COLLECTOR_ZIPKIN_HOST_PORT=:9411 \
  --network example-net \
  jaegertracing/all-in-one:1.62.0
```

2. Assume ./custom-agent.yaml contains the configuration of ShardingSphere Agent, and the content may be as follows,

```
plugins:
  tracing:
    OpenTelemetry:
      props:
        otel.service.name: "example"
        otel.exporter.otlp.traces.endpoint: "http://jaeger:4318"
```

3. Assuming ./target/example.jar is an Uber JAR of Spring Boot that will use ShardingSphere Agent, you can use the ShardingSphere Agent in the nightly built Docker Image for a JAR like example.jar through a Dockerfile like the following.

```
FROM ghcr.io/apache/shardingsphere-agent:latest
COPY ./target/example.jar /app.jar
COPY ./custom-agent.yaml /usr/agent/conf/agent.yaml
ENTRYPOINT ["java", "-javaagent:/usr/agent/shardingsphere-agent.jar", "-jar", "/app.jar"]
```

If you build the Docker Image of ghcr.io/apache/shardingsphere-agent:latest locally, the Dockerfile may be as follows,

```
FROM ghcr.io/apache/shardingsphere-agent:latest
COPY ./target/example.jar /app.jar
COPY ./custom-agent.yaml /usr/agent/conf/agent.yaml
ENTRYPOINT ["java","-javaagent:/usr/agent/shardingsphere-agent.jar","-jar","/app.jar"]
```

4. Enjoy it,

```
docker build -t example/gs-spring-boot-docker:latest .
docker run --network example-net example/gs-spring-boot-docker:latest
```

Metrics

Name	Type	Description
build_info	G A U G E	Build information
p arsed_sql_total	C O U N T	Total count of parsed by type (INSERT, UPDATE, DELETE, SE- LECT, DDL, DCL, DAL, TCL, RQL, RDL, RAL, RUL)
r outed_sql_total	C O U N T	Total count of routed by type (INSERT, UPDATE, DELETE, SE- LECT)
rout ed_result_total	C O U N T	Total count of routed result (data source routed, table routed) ER
jdbc_state	G A U G E	Status information of ShardingSphere-JDBC. 0 is OK; 1 is CIR- CUIT BREAK; 2 is LOCK
jdbc_meta_data_info	G A U G E	Meta data information of ShardingSphere-JDBC
jdbc_statemen t_execute_total	G A U G E	Total number of statements executed
jdbc_ state- ment_execu te_errors_total	G A U G E	Total number of statement execution errors
jdbc_st atement_execute tency_millis	ate- _la- GRAM	Statement execution latency
jdbc_tra nsac- tions_total	G A U G E	Total number of transactions, classify by commit and rollback

9.1.7 GraalVM Native Image

Background Information

ShardingSphere JDBC has been validated for availability under GraalVM Native Image.

Build GraalVM Native containing Maven dependencies of `org.apache.shardingsphere:shardingsphere-jdbc:${shardingsphere.version}` Image, you need to resort to GraalVM Native Build Tools. GraalVM Native Build Tools provides Maven Plugin and Gradle Plugin to simplify long list of shell commands for GraalVM CE's `native-image` command line tool.

ShardingSphere JDBC requires GraalVM Native Image to be built with GraalVM CE as follows or higher. Users can quickly switch JDK through SDKMAN!. Same reason applicable to downstream distributions of GraalVM CE such as <https://sdkman.io/jdks#graal>, <https://sdkman.io/jdks#nik> and <https://sdkman.io/jdks#mandrel>.

- GraalVM CE For JDK 22.0.2, corresponding to `22.0.2-graalce` of SDKMAN!

Users can still use the old versions of GraalVM CE such as `21.0.2-graalce` on SDKMAN! to build the GraalVM Native Image product of ShardingSphere. However, this will cause the failure of building the GraalVM Native Image when integrating some third-party dependencies. A typical example is related to the `org.apache.hive:hive-jdbc:4.0.1` HiveServer2 JDBC Driver, which uses AWT-related classes. GraalVM CE only supports AWT for GraalVM CE For JDK22 and higher versions.

```
com.sun.beans.introspect.ClassInfo was unintentionally initialized at build time. To
see why com.sun.beans.introspect.ClassInfo got initialized use --trace-class-
initialization=com.sun.beans.introspect.ClassInfo
java.beans.Introspector was unintentionally initialized at build time. To see why
java.beans.Introspector got initialized use --trace-class-initialization=java.beans.
Introspector
```

Maven Ecology

Users need to actively use the GraalVM Reachability Metadata central repository. The following configuration is for reference to configure additional Maven Profiles for the project, and the documentation of GraalVM Native Build Tools shall prevail.

```
<project>
  <dependencies>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-jdbc</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
```

```

    <plugin>
      <groupId>org.graalvm.buildtools</groupId>
      <artifactId>native-maven-plugin</artifactId>
      <version>0.10.5</version>
      <extensions>true</extensions>
      <configuration>
        <buildArgs>
          <buildArg>-H:+AddAllCharsets</buildArg>
        </buildArgs>
      </configuration>
      <executions>
        <execution>
          <id>build-native</id>
          <goals>
            <goal>compile-no-fork</goal>
          </goals>
          <phase>package</phase>
        </execution>
        <execution>
          <id>test-native</id>
          <goals>
            <goal>test</goal>
          </goals>
          <phase>test</phase>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
</project>

```

Gradle Ecosystem

Users need to actively use the GraalVM Reachability Metadata central repository. The following configuration is for reference to configure additional Gradle Tasks for the project, and the documentation of GraalVM Native Build Tools shall prevail. Due to the limitations of <https://github.com/gradle/gradle/issues/17559>, users need to introduce the JSON file of Metadata Repository through Maven dependency. Reference <https://github.com/graalvm/native-build-tools/issues/572>.

```

plugins {
  id 'org.graalvm.buildtools.native' version '0.10.5'
}

dependencies {
  implementation 'org.apache.shardingsphere:shardingsphere-jdbc:${shardingsphere.version}'
  implementation(group: 'org.graalvm.buildtools', name: 'graalvm-reachability-

```

```

metadata', version: '0.10.5', classifier: 'repository', ext: 'zip')
}

graalvmNative {
    binaries {
        main {
            buildArgs.add('-H:+AddAllCharsets')
        }
        test {
            buildArgs.add('-H:+AddAllCharsets')
        }
    }
    metadataRepository {
        enabled.set(false)
    }
}

```

For build tools such as sbt that are not supported by GraalVM Native Build Tools

Such requirements require opening additional issues at <https://github.com/graalvm/native-build-tools> and providing the Plugin implementation of the corresponding build tool.

Usage restrictions

1. The following algorithm classes are not available under GraalVM Native Image due to the involvement of <https://github.com/oracle/graal/issues/5522>.

- org.apache.shardingsphere.sharding.algorithm.sharding.inline.
InlineShardingAlgorithm
- org.apache.shardingsphere.sharding.algorithm.sharding.inline.
ComplexInlineShardingAlgorithm
- org.apache.shardingsphere.sharding.algorithm.sharding.hint.
HintInlineShardingAlgorithm

For general cases, users can simulate the behavior of GroovyShell by themselves through the CLASS_BASE algorithm. For example, take the following configuration.

```

rules:
- !SHARDING
  defaultDatabaseStrategy:
    standard:
      shardingColumn: user_id
      shardingAlgorithmName: inline
  shardingAlgorithms:
    inline:
      type: INLINE

```



```

props:
  algorithm-expression: ds_${user_id % 2}
  allow-range-query-with-inline-sharding: false

```

You can first define the implementation class of CLASS_BASE.

```

package org.example.test;

import org.apache.shardingsphere.sharding.api.sharding.standard.
PreciseShardingValue;
import org.apache.shardingsphere.sharding.api.sharding.standard.RangeShardingValue;
import org.apache.shardingsphere.sharding.api.sharding.standard.
StandardShardingAlgorithm;

import java.util.Collection;

public final class TestShardingAlgorithmFixture implements StandardShardingAlgorithm
<Integer> {

    @Override
    public String doSharding(final Collection<String> availableTargetNames, final
PreciseShardingValue<Integer> shardingValue) {
        String resultDatabaseName = "ds_" + shardingValue.getValue() % 2;
        for (String each : availableTargetNames) {
            if (each.equals(resultDatabaseName)) {
                return each;
            }
        }
        return null;
    }

    @Override
    public Collection<String> doSharding(final Collection<String>
availableTargetNames, final RangeShardingValue<Integer> shardingValue) {
        throw new RuntimeException("This algorithm class does not support range
queries.");
    }
}

```

Modify the relevant YAML configuration as follows.

```

rules:
- !SHARDING
  defaultDatabaseStrategy:
    standard:
      shardingColumn: user_id
      shardingAlgorithmName: inline
  shardingAlgorithms:
    inline:

```

```

type: CLASS_BASED
props:
  strategy: STANDARD
  algorithmClassName: org.example.test.TestShardingAlgorithmFixture

```

Add the following content to `src/main/resources/META-INF/native-image/exmaple-test-metadata/reflect-config.json` to used normally under GraalVM Native Image.

```

[
{
  "name": "org.example.test.TestShardingAlgorithmFixture",
  "methods": [{"name": "<init>", "parameterTypes": []}]
}
]

```

2. For the `ReadWrite Splitting` feature, you need to use other implementations of `Row Value Expressions SPI` to configure logic database name, `writeDataSourceName` and `readDataSourceNames` when bypassing calls to `GroovyShell`. One possible configuration is to use the `Row Value Expressions SPI` implementation of `LITERAL`.

```

rules:
- !READWRITE_SPLITTING
  dataSourceGroups:
    <LITERAL>readwrite_ds:
      writeDataSourceName: <LITERAL>ds_0
      readDataSourceNames:
        - <LITERAL>ds_1
        - <LITERAL>ds_2

```

The same applies to `actualDataNodes` for the `Sharding` feature.

```

- !SHARDING
  tables:
    t_order:
      actualDataNodes: <LITERAL>ds_0.t_order_0, ds_0.t_order_1, ds_1.t_order_0,
ds_1.t_order_1
      keyGenerateStrategy:
        column: order_id
        keyGeneratorName: snowflake

```

3. Users still need to configure `GraalVM Reachability Metadata` for independent files in the `src/main/resources/META-INF/native-image` folder or `src/test/resources/META-INF/native-image` folder. Users can quickly collect `GraalVM Reachability Metadata` through the `GraalVM Tracing Agent` of `GraalVM Native Build Tools`.
4. Maven modules such as `com.microsoft.sqlserver:mssql-jdbc`, represented by the `JDBC Driver` of `MS SQL Server`, will dynamically load different character sets based on the encoding used in the database, which is unpredictable behavior. When encountering the following Error, users

need to add the buildArg of `-H:+AddAllCharsets` to the configuration of GraalVM Native Build Tools.

```
Caused by: java.io.UnsupportedEncodingException: Codepage Cp1252 is not supported by
the Java environment.
    com.microsoft.sqlserver.jdbc.Encoding.checkSupported(SQLCollation.java:572)
    com.microsoft.sqlserver.jdbc.SQLCollation$SortOrder.getEncoding(SQLCollation.
java:473)
    com.microsoft.sqlserver.jdbc.SQLCollation.encodingFromSortId(SQLCollation.java:501)
    [...]
```

5. To discuss the steps required to use XA distributed transactions under the GraalVM Native Image of ShardingSphere JDBC, additional known prerequisites need to be introduced,

- `org.apache.shardingsphere.transaction.xa.jta.datasource.swapper.DataSourceSwapper#loadXADataSource(String)` will instantiate the `javax.sql.XADataSource` implementation class of each database driver through `java.lang.Class#getDeclaredConstructors`.
- The full class name of the `javax.sql.XADataSource` implementation class of each database driver is stored in the metadata of ShardingSphere by implementing the SPI of `org.apache.shardingsphere.transaction.xa.jta.datasource.properties.XADataSourceDefinition`.

In the GraalVM Native Image, this actually requires the definition of the GraalVM Reachability Metadata of the third-party dependencies, while ShardingSphere itself only provides the corresponding GraalVM Reachability Metadata for `com.h2database:h2`.

GraalVM Reachability Metadata of other database drivers such as `com.mysql:mysql-connector-j` should be defined by themselves, or the corresponding JSON should be submitted to <https://github.com/oracle/graalvm-reachability-metadata>.

Take the `com.mysql.cj.jdbc.MysqlXADataSource` class of `com.mysql:mysql-connector-j:9.0.0` as an example, which is the implementation of `javax.sql.XADataSource` of MySQL JDBC Driver. Users need to define the following JSON in the `reflect-config.json` file in the `/META-INF/native-image/com.mysql/mysql-connector-j/9.0.0/` folder of their own project's classpath, to define the constructor of `com.mysql.cj.jdbc.MysqlXADataSource` inside the GraalVM Native Image.

```
[
{
  "condition":{"typeReachable":"com.mysql.cj.jdbc.MysqlXADataSource"},
  "name":"com.mysql.cj.jdbc.MysqlXADataSource",
  "allPublicMethods": true,
  "methods": [{"name":"<init>","parameterTypes":[]}]
}]
```

6. When using the ClickHouse dialect through ShardingSphere JDBC, users need to manually introduce the relevant optional modules and the ClickHouse JDBC driver with the classifier `http`.

In principle, ShardingSphere's GraalVM Native Image integration does not want to use `com.clickhouse:clickhouse-jdbc` with classifier `all`, because Uber Jar will cause the collection of duplicate GraalVM Reachability Metadata. Possible configuration examples are as follows,

```
<project>
  <dependencies>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-jdbc</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-parser-sql-clickhouse</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
    <dependency>
      <groupId>com.clickhouse</groupId>
      <artifactId>clickhouse-jdbc</artifactId>
      <version>0.6.3</version>
      <classifier>http</classifier>
    </dependency>
  </dependencies>
</project>
```

7. Affected by <https://github.com/grpc/grpc-java/issues/10601>, should users incorporate `org.apache.hive:hive-jdbc` into their project, it is imperative to create a file named `native-image.properties` within the directory `META-INF/native-image/io.grpc/grpc-netty-shaded` of the classpath, containing the following content,

```
Args=--initialize-at-run-time=\
  io.grpc.netty.shaded.io.netty.channel.ChannelHandlerMask,\
  io.grpc.netty.shaded.io.netty.channel.nio.AbstractNioChannel,\
  io.grpc.netty.shaded.io.netty.channel.socket.nio.SelectorProviderUtil,\
  io.grpc.netty.shaded.io.netty.util.concurrent.DefaultPromise,\
  io.grpc.netty.shaded.io.netty.util.internal.MacAddressUtil,\
  io.grpc.netty.shaded.io.netty.util.internal.SystemPropertyUtil,\
  io.grpc.netty.shaded.io.netty.util.NetUtilInitializations,\
  io.grpc.netty.shaded.io.netty.channel.AbstractChannel,\
  io.grpc.netty.shaded.io.netty.util.NetUtil,\
  io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent,\
  io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent0,\
  io.grpc.netty.shaded.io.netty.channel.DefaultChannelPipeline,\
  io.grpc.netty.shaded.io.netty.channel.DefaultChannelId,\
  io.grpc.netty.shaded.io.netty.util.ResourceLeakDetector,\
  io.grpc.netty.shaded.io.netty.channel.AbstractChannelHandlerContext,\
  io.grpc.netty.shaded.io.netty.channel.ChannelOutboundBuffer,\
  io.grpc.netty.shaded.io.netty.util.internal.InternalThreadLocalMap,\
  io.grpc.netty.shaded.io.netty.util.internal.CleanerJava9,\
```

```
io.grpc.netty.shaded.io.netty.util.internal.StringUtil,\
io.grpc.netty.shaded.io.netty.util.internal.CleanerJava6,\
io.grpc.netty.shaded.io.netty.buffer.ByteBufUtil$HexUtil,\
io.grpc.netty.shaded.io.netty.buffer.AbstractByteBufAllocator,\
io.grpc.netty.shaded.io.netty.util.concurrent.FastThreadLocalThread,\
io.grpc.netty.shaded.io.netty.buffer.PoolArena,\
io.grpc.netty.shaded.io.netty.buffer.EmptyByteBuf,\
io.grpc.netty.shaded.io.netty.buffer.PoolThreadCache,\
io.grpc.netty.shaded.io.netty.util.AttributeKey
```

ShardingSphere's unit test only uses the Maven module `io.github.linghengqian:hive-server2-jdbc-driver-thin` to verify the availability under GraalVM Native Image.

8. Due to <https://github.com/oracle/graal/issues/7979>, the Oracle JDBC Driver corresponding to the `com.oracle.database.jdbc:ojdbc8` Maven module cannot be used under GraalVM Native Image.
9. Due to <https://github.com/apache/doris/issues/9426>, when connecting to Apache Doris FE via ShardingSphere JDBC, users need to provide GraalVM Reachability Metadata related to the `apache/doris` integration module.
10. Due to <https://github.com/prestodb/presto/issues/23226>, when connecting to Presto Server via ShardingSphere JDBC, users need to provide GraalVM Reachability Metadata related to the `com.facebook.presto:presto-jdbc` and `prestodb/presto` integration module.

Contribute GraalVM Reachability Metadata

The verification of ShardingSphere's availability under GraalVM Native Image is completed through the Maven Plugin subproject of GraalVM Native Build Tools. By running the unit test under the JVM, label the unit test with `junit-platform-unique-ids*`, and then build it as GraalVM Native Image for `nativeTest` to test Unit Test Coverage under GraalVM Native Image. Please do not use `io.kotest:kotest-runner-junit5-jvm:5.5.4` and some third-party test libraries, they are in `test listener` mode failed to discover tests.

ShardingSphere defines the Maven Module of `shardingsphere-test-native` to provide a small subset of unit tests for native Test. This subset of unit tests avoids the use of third-party libraries such as Mockito that are not available under native Test.

ShardingSphere defines the Maven Profile of `nativeTestInShardingSphere` for executing `nativeTest` for the `shardingsphere-test-native` module.

Contributors must install Docker Engine to execute unit tests related to `testcontainers-java`, as per https://java.testcontainers.org/supported_docker_environment/.

Assuming that the contributor is under a new Ubuntu 22.04.4 LTS instance, Contributors can manage the JDK and tool chain through SDKMAN! through the following bash command, and execute `nativeTest` for the `shardingsphere-test-native` submodule.

```

sudo apt install unzip zip -y
curl -s "https://get.sdkman.io" | bash
source "$HOME/.sdkman/bin/sdkman-init.sh"
sdk install java 22.0.2-graalce
sdk use java 22.0.2-graalce
sudo apt-get install build-essential zlib1g-dev -y

git clone git@github.com:apache/shardingsphere.git
cd ./shardingsphere/
./mvnw -PnativeTestInShardingSphere -e -T 1C clean test

```

When contributors discover that GraalVM Reachability Metadata is missing for a third-party library not related to ShardingSphere, they should open a new issue and submit PR containing the missing third-party library GraalVM Reachability Metadata that depends on <https://github.com/oracle/graalvm-reachability-metadata>. ShardingSphere actively hosts the GraalVM Reachability Metadata of some third-party libraries in the `shardingsphere-infra-reachability-metadata` submodule.

If `nativeTest` execution fails, preliminary GraalVM Reachability Metadata should be generated for unit tests, and manually adjust the contents of the `META-INF/native-image/org.apache.shardingsphere/shardingsphere-infra-reachability-metadata/` folder on the classpath of the `shardingsphere-infra-reachability-metadata` submodule to fix `nativeTest`. If necessary, use the `org.junit.jupiter.api.condition.DisabledInNativeImage` annotation or the `org.graalvm.nativeimage.imagecode` System Property blocks some unit tests from running under GraalVM Native Image.

ShardingSphere defines the Maven Profile of `generateMetadata` to carry the GraalVM Tracing Agent under the GraalVM JIT Compiler to perform unit testing, and generate or overwrite existing GraalVM Reachability Metadata files in the `META-INF/native-image/org.apache.shardingsphere/generated-reachability-metadata/` folder of the classpath of the `shardingsphere-infra-reachability-metadata` submodule. This process can be easily handled with the following bash command. Contributors may still need to manually adjust specific JSON entries and adjust the Filter chain of Maven Profile and GraalVM Tracing Agent as appropriate. For the `shardingsphere-infra-reachability-metadata` submodule, manually added, deleted, and changed JSON entries should be located in the `META-INF/native-image/org.apache.shardingsphere/shardingsphere-infra-reachability-metadata/` folder, entries in `META-INF/native-image/org.apache.shardingsphere/generated-reachability-metadata/` should only be generated by the Maven Profile of `generateMetadata`.

The following command is only an example of using `shardingsphere-test-native` to generate GraalVM Reachability Metadata in Conditional form. Generated GraalVM Reachability Metadata is located under the `shardingsphere-infra-reachability-metadata` submodule.

For GraalVM Reachability Metadata to be used independently by test classes and test files, contributors should place it on the classpath of the `shardingsphere-test-native` submodule `META-INF/native-image/shardingsphere-test-native-test-metadata/`.

```

git clone git@github.com:apache/shardingsphere.git
cd ./shardingsphere/

```

```
./mvnw -PgenerateMetadata -e -T 1C clean test native:metadata-copy
```

Affected by <https://github.com/apache/shardingsphere/issues/33206>, After the contributor executes `./mvnw -PgenerateMetadata -T 1C -e clean test native:metadata-copy`, `infra/reachability-metadata/src/main/resources/META-INF/native-image/org.apache.shardingsphere/generated-reachability-metadata/resource-config.json` will generate unnecessary JSON entries containing absolute paths, similar to the following.

```
{
  "resources":{
    "includes":[{
      "condition":{"typeReachable":"org.apache.shardingsphere.proxy.backend.
config.ProxyConfigurationLoader"},
      "pattern":"\\Qhome/runner/work/shardingsphere/shardingsphere/test/native/
src/test/resources/test-native/yaml/proxy/databases/postgresql//global.yaml\\E"
    }, {
      "condition":{"typeReachable":"org.apache.shardingsphere.proxy.backend.
config.ProxyConfigurationLoader"},
      "pattern":"\\Qhome/runner/work/shardingsphere/shardingsphere/test/native/
src/test/resources/test-native/yaml/proxy/databases/postgresql//\\E"
    }, {
      "condition":{"typeReachable":"org.apache.shardingsphere.proxy.backend.
config.ProxyConfigurationLoader"},
      "pattern":"\\Qhome/runner/work/shardingsphere/shardingsphere/test/native/
src/test/resources/test-native/yaml/proxy/features/sharding//global.yaml\\E"
    }, {
      "condition":{"typeReachable":"org.apache.shardingsphere.proxy.backend.
config.ProxyConfigurationLoader"},
      "pattern":"\\Qhome/runner/work/shardingsphere/shardingsphere/test/native/
src/test/resources/test-native/yaml/proxy/features/sharding//\\E"
    }
  ],
  "bundles":[]
}
```

Contributors should always manually remove these JSON entries containing absolute paths and wait for <https://github.com/oracle/graal/issues/8417> to be resolved.

9.2 ShardingSphere-Proxy

Configuration is the only module in ShardingSphere-Proxy that interacts with application developers, through which developer can quickly and clearly understand the functions provided by ShardingSphere-Proxy.

This chapter is a configuration manual for ShardingSphere-Proxy, which can also be referred to as a dictionary if necessary.

ShardingSphere-Proxy provided YAML configuration, and used DistSQL to communicate. By config-

uration, application developers can flexibly use data sharding, readwrite-splitting, data encryption, shadow database or the combination of them.

Rule configuration keeps consist with YAML configuration of ShardingSphere-JDBC. DistSQL and YAML can be replaced each other.

9.2.1 Startup

This chapter will introduce the deployment and startup of ShardingSphere-Proxy.

Use Binary Tar

Background

This section describes how to start ShardingSphere-Proxy by binary release packages

Premise

Start the Proxy with a binary package requires an environment with Java JRE 8 or later.

Steps

1. Obtain the binary release package of ShardingSphere-Proxy

Obtain the binary release package of ShardingSphere-Proxy on the [download page](#).

2. Configure `conf/global.yaml`

ShardingSphere-Proxy's operational mode is configured on `global.yaml`, and its configuration mode is the same with that of ShardingSphere-JDBC. Refer to [mode of configuration](#).

Please refer to the following links for other configuration items: * [Permission configuration](#) * [Property configuration](#)

3. Configure `conf/database-*.yaml`

Modify files named with the prefix `database-` in the `conf` directory, such as `conf/database-sharding.yaml` file and configure sharding rules and read/write splitting rules. See [Configuration Manual](#) for configuration methods. The `*` part of the `database-*.yaml` file can be named whatever you want.

ShardingSphere-Proxy supports multiple logical data sources. Each YAML configuration file named with the prefix `database-` is a logical data source.

4. Introduce database driver (Optional)

If the backend is connected to a PostgreSQL or openGauss database, no additional dependencies need to be introduced.

If the backend is connected to a MySQL database, please download [mysql-connector-java-5.1.49.jar](#) or [mysql-connector-java-8.0.11.jar](#), and put it into the `ext-lib` directory.

5. Introduce dependencies required by the cluster mode (Optional)

ShardingSphere-Proxy integrates the ZooKeeper Curator client by default. ZooKeeper is used in cluster mode without introducing other dependencies.

If the cluster mode uses Etcd, please copy [vertx-grpc 4.5.1](#) and [vertx-core 4.5.1](#) that Etcd depends on into the `ext-lib` directory.

6. Introduce dependencies required by distributed transactions (Optional)

It is the same with ShardingSphere-JDBC. Please refer to [Distributed Transaction](#) for more details.

7. Introduce custom algorithm (Optional)

If you need to use a user-defined algorithm class, you can configure custom algorithm in the following ways:

1. Implement the algorithm implementation class defined by ``ShardingAlgorithm``.
2. Create a ``META-INF/services`` directory under the project ``resources`` directory.
3. Create file ``org.apache.shardingsphere.sharding.spi.ShardingAlgorithm`` under the directory ``META-INF/services``.
4. Writes the fully qualified class name of the implementation class to a file ``org.apache.shardingsphere.sharding.spi.ShardingAlgorithm``
5. Package the above Java files into jar packages.
6. Copy the above jar package to the ``ext-lib`` directory.
7. Configure the Java file reference of the above custom algorithm implementation class in a YAML file, see [Configuration rule](https://shardingsphere.apache.org/document/current/en/user-manual/shardingsphere-proxy/yaml-config/) for more details.

8. Start ShardingSphere-Proxy

In Linux or macOS, run `bin/start.sh`. In Windows, run `bin/start.bat` to start ShardingSphere-Proxy. The default listening port is 3307 and the default configuration directory is the `conf` directory in Proxy. The startup script can specify the listening port and the configuration file directory by running the following command:

```
bin/start.sh [port] [/path/to/conf]
```

9. Connect ShardingSphere-Proxy with client

Run the MySQL/PostgreSQL/openGauss client command to directly operate ShardingSphere-Proxy.

Connect ShardingSphere-Proxy with MySQL client:

```
mysql -h${proxy_host} -P${proxy_port} -u${proxy_username} -p${proxy_password}
```

Connect ShardingSphere-Proxy with PostgreSQL:

```
psql -h ${proxy_host} -p ${proxy_port} -U ${proxy_username}
```

Connect ShardingSphere-Proxy with openGauss client:

```
gsql -r -h ${proxy_host} -p ${proxy_port} -U ${proxy_username} -W ${proxy_password}
```

Use Docker

Background

This chapter is an introduction about how to start ShardingSphere-Proxy via Docker

Notice

Using Docker to start ShardingSphere-Proxy does not require additional package support.

Steps

1. Acquire Docker Image

- Method 1 (Recommended): Pull from DockerHub

```
docker pull apache/shardingsphere-proxy
```

- Method 2: Acquire latest master branch image master: <https://github.com/apache/shardingsphere-re/pkgs/container/shardingsphere-proxy>
- Method 3: Build your own image

```
git clone https://github.com/apache/shardingsphere
./mvnw clean install
cd shardingsphere-distribution/shardingsphere-proxy-distribution
./mvnw clean package -Prelease,default-dep,docker
```

If the following problems emerge, please make sure Docker daemon Process is running.

```
I/O exception (java.io.IOException) caught when processing request to {}->unix://localhost:80: Connection refused?
```

2. Configure conf/global.yaml and conf/database-*.yaml

Configuration file template can be attained from the Docker container and can be copied to any directory on the host:

```
docker run -d --name tmp --entrypoint=bash apache/shardingsphere-proxy
docker cp tmp:/opt/shardingsphere-proxy/conf /host/path/to/conf
docker rm tmp
```

Since the network conditions inside the container may differ from those of the host, if errors such as “cannot connect to the database” occurs, please make sure that the IP of the database specified in the conf/database-*.yaml configuration file can be accessed from inside the Docker container.

For details, please refer to [ShardingSphere-Proxy quick start manual - binary distribution packages](#).

3. (Optional) Introduce third-party dependencies or customized algorithms

If you have any of the following requirements: * ShardingSphere-Proxy Backend use MySQL Database; * Implement customized algorithms; * Use Etcd as Registry Center in cluster mode.

Please create `ext-lib` directory anywhere inside the host and refer to the steps in [ShardingSphere-Proxy quick start manual - binary distribution packages](#).

4. Start ShardingSphere-Proxy container

Mount the `conf` and `ext-lib` directories from the host to the container. Start the container:

```
docker run -d \
  -v /host/path/to/conf:/opt/shardingsphere-proxy/conf \
  -v /host/path/to/ext-lib:/opt/shardingsphere-proxy/ext-lib \
  -e PORT=3308 -p13308:3308 apache/shardingsphere-proxy:latest
```

`ext-lib` is not necessary during the process. Users can mount it at will. ShardingSphere-Proxy default portal 3307 can be designated according to environment variable `-e PORT` Customized JVM related parameters can be set according to environment variable `JVM_OPTS`

Note:

Support setting environment variable `CGROUP_MEM_OPTS`: used to set related memory parameters in the container environment. The default values in the script are:

```
-XX:InitialRAMPercentage=80.0 -XX:MaxRAMPercentage=80.0 -XX:MinRAMPercentage=80.0
```

5. Use Client to connect to ShardingSphere-Proxy

Please refer to [ShardingSphere-Proxy quick start manual - binary distribution packages](#).

Build GraalVM Native Image(Alpha)

Background

This section mainly introduces it how to build the GraalVM Native Image of ShardingSphere Proxy through the `native-image` command line tool of GraalVM, and the Docker Image containing this GraalVM Native Image.

The GraalVM Native Image of ShardingSphere Proxy refers to ShardingSphere Proxy Native in this article.

For background information about GraalVM Native Image, please refer to <https://www.graalvm.org>.

Notice

All Docker images mentioned in this section are not distributed through ASF official channels such as <https://downloads.apache.org> and <https://repository.apache.org>. Docker images are only provided in downstream channels such as GitHub Packages and Docker Hub for easy use.

Native Image products of Proxy exist in nightly builds at <https://github.com/apache/shardingsphere/pkgs/container/shardingsphere-proxy-native>. Assuming that there is a `conf` folder containing `global.yaml` as `./custom/conf`, you can test it with the following `docker-compose.yml` file.

```
services:
  apache-shardingsphere-proxy-native:
    image: ghcr.io/apache/shardingsphere-proxy-native:latest
    volumes:
      - ./custom/conf:/opt/shardingsphere-proxy-native/conf
    ports:
      - "3307:3307"
```

ShardingSphere Proxy Native can execute DistSQL, which means that no YAML file that defines the logical database is actually required.

By default, the GraalVM Native Image of ShardingSphere Proxy Native only contains,

1. GraalVM Reachability Metadata maintained by ShardingSphere and some third-party dependencies
2. JDBC Driver for H2database, OpenGauss and PostgreSQL
3. HikariCP database connection pool
4. Logback logging framework

If the user needs to use third-party JAR in ShardingSphere Proxy Native, the content of `distribution/proxy-native/pom.xml` needs to be modified to build any of the following outputs,

1. Customized GraalVM Native Image
2. Customized Docker Image containing customized GraalVM Native Image

This section assumes that you are in one of the following system environments:

1. Linux (amd64, aarch64)
2. MacOS (amd64, aarch64/M1)
3. Windows (amd64)

If you are in a system environment that Graal compiler does not support, such as Linux (riscv64), please enable LLVM backend according to the content of <https://medium.com/graalvm/graalvm-native-image-meets-risc-v-899be38eddd9> to use the LLVM compiler.

Users must build the different GraalVM Native Image for each target operating system and target architecture that they need to run the GraalVM Native Image on. Users can consider partially circumventing this limitation by using Docker Image.

This section is still limited by the documented content of the [GraalVM Native Image](#) on the ShardingSphere JDBC side.

Premise

1. Install and configure GraalVM Community Edition or a downstream distribution of GraalVM Community Edition for JDK 22 according to <https://www.graalvm.org/downloads/> . If using SDKMAN!,

```
sdk install java 22.0.2-graalce
sdk use java 22.0.2-graalce
```

2. Install the native toolchain according to <https://www.graalvm.org/jdk23/reference-manual/native-image/#prerequisites> .
3. If you need to build a Docker Image, make sure Docker Engine is installed.

Steps

1. Get Apache ShardingSphere Git Source

Get it from [Download page](#) or <https://github.com/apache/shardingsphere/tree/master> .

2. Build the product in the command line, divided into two cases.

Case 1: No need to use JAR with custom SPI implementation or third-party dependent JAR. Execute the following command in the same directory as Git Source to directly complete the construction of Native Image.

```
cd ./shardingsphere/
./mvnw -am -pl distribution/proxy-native -T1C -Prelease.native,default-dep -
DskipTests clean package
```

Case 2: Need to use JAR with custom SPI implementation or third-party dependent JAR. Add one of the following options to the dependencies of `distribution/proxy-native/pom.xml`:

- (1) JARs with SPI implementations
- (2) JARs with third-party dependencies

The examples are as follows. These JARs should be pre-placed in the local Maven repository or a remote Maven repository such as Maven Central.

```
<dependencies>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>9.0.0</version>
  </dependency>
</dependencies>
```

Then build the GraalVM Native Image through the command line.

```
cd ./shardingsphere/
./mvnw -am -pl distribution/proxy-native -T1C -Prelease.native,default-dep -
DskipTests clean package
```

3. To start Native Image through the command line, you need to bring 4 parameters. The first parameter is the port used by ShardingSphere Proxy Native. The second parameter is the folder containing the `global.yaml` configuration file written by the user. The third parameter is the host to listen to. If it is `0.0.0.0`, any database client is allowed to access ShardingSphere Proxy Native. The fourth parameter is Force Start. If it is `true`, it ensures that ShardingSphere Proxy Native can start normally regardless of whether it can be connected.

Only command line parameters can be set for binaries of built GraalVM Native Images. This means that:

- (1) Users can only set JVM parameters during the process of building a GraalVM Native Image
- (2) Users cannot set JVM parameters for binaries of built GraalVM Native Images

Assuming the folder `/customAbsolutePath/conf` already exists, the example is.

```
cd ./shardingsphere/
cd ./distribution/proxy-native/target/apache-shardingsphere-5.5.2-shardingsphere-
proxy-native-bin/
./proxy-native "3307" "/customAbsolutePath/conf" "0.0.0.0" "false"
```

4. If you need to build a Docker Image, after adding the dependencies that have SPI implementation or third-party dependencies, execute the following command in the command line:

```
cd ./shardingsphere/
./mvnw -am -pl distribution/proxy-native -T1C -Prelease.native,default-dep,docker.
native -DskipTests clean package
```

Assuming that there is a `conf` folder called `./custom/conf` containing `global.yaml`, you can start the Docker Image containing the GraalVM Native Image using the following `docker-compose.yml` file,

```
services:
  apache-shardingsphere-proxy-native:
    image: apache/shardingsphere-proxy-native:latest
    volumes:
      - ./custom/conf:/opt/shardingsphere-proxy-native/conf
    ports:
      - "3307:3307"
```

If the user does not make any changes to the Git Source, the above mentioned command will use `container-registry.oracle.com/os/oraclelinux:9-slim` in <https://yum.oracle.com/oracle-linux-downloads.html> as the Base Docker Image. But if the user wants to use a smaller Docker Image such as `scratch`, `alpine:3`, `gcr.io/distroless/base-debian12`, `gcr.io/distroless/java-base-debian12` or `gcr.io/distroless/static-debian12` as the Base Docker Image, the

user may need to add `--static`, `--libc=musl` or `--static-nolibc` to the Maven Profile of `pom.xml` as required by <https://www.graalvm.org/jdk23/reference-manual/native-image/guides/build-static-executables/> and other operations such as `buildArgs`.

Building a statically linked GraalVM Native Image requires more system dependencies, and currently does not support building statically linked GraalVM Native Images for environments such as Linux (aarch64). Fully statically linked GraalVM Native Images use `musl libc`. Most Linux systems come with outdated `musl`, such as Ubuntu 22.04.5 LTS which uses `musl (1.2.2-4) unstable`. Users always need to build and install a new version of `musl` from source.

Also note that some third-party Maven dependencies will require more system libraries to be installed in the `Dockerfile`, so make sure to adjust the contents of `pom.xml` and `Dockerfile` under `distribution/proxy-native` according to your usage.

Observability

The observability provided by ShardingSphere Proxy in the form of GraalVM Native Image is not consistent with [observability](#).

Users can use a series of command-line tools or visualization tools provided by <https://www.graalvm.org/jdk23/tools/> to observe the internal behavior of GraalVM Native Image, and use VSCode under Linux to complete debugging work according to their requirements. If the user is using IntelliJ IDEA and wants to debug the generated GraalVM Native Image, the user can follow https://blog.jetbrains.com/idea/2022/06/intellij-idea-2022-2-eap-5/#Experimental_GraalVM_Native_Debugger_for_Java and its successors.

If the user is not using Linux, the GraalVM Native Image cannot be debugged. Please follow <https://github.com/oracle/graal/issues/5648> which has not been closed.

For Java Agents such as ShardingSphere Agent, the `native-image` component of GraalVM does not fully support the use of `javaagent` when building Native Image. Users need to pay attention to <https://github.com/oracle/graal/issues/8177> which has not been closed.

If users expect to use such Java Agents under ShardingSphere Proxy Native, they need to pay attention to the changes involved in <https://github.com/oracle/graal/pull/8077>.

Seata AT mode integration

For ShardingSphere Proxy Native in GraalVM Native Image, Users always need to modify the ShardingSphere source code to add the Seata Client and Seata integrated Maven modules and compile them into GraalVM Native Image. ShardingSphere Proxy Native in GraalVM Native Image cannot recognize the additional JAR files.

```
<project>
  <dependencies>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-transaction-base-seata-at</artifactId>
```

```

    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.seata</groupId>
    <artifactId>seata-all</artifactId>
    <version>2.3.0</version>
    <exclusions>
      <exclusion>
        <groupId>org.antlr</groupId>
        <artifactId>antlr4-runtime</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
</dependencies>
</project>

```

Use Helm

Background

Use [Helm](#) to provide guidance for the installation of ShardingSphere-Proxy instance in a Kubernetes cluster. For more details, please checkout [ShardingSphere-on-Cloud](#).

Requirements

- Kubernetes 1.18+
- kubectl
- Helm 3.2.0+
- StorageClass of PV (Persistent Volumes) can be dynamically applied for persistent data (Optional)

Procedure

Online installation

1. Add ShardingSphere-Proxy to the local helm repo:

```
helm repo add shardingsphere https://shardingsphere.apache.org/charts
```

2. Install ShardingSphere-Proxy charts:

```
helm install shardingsphere-proxy shardingsphere/shardingsphere-proxy
```


Source installation

1. Charts will be installed with default configuration if the following commands are executed:

```
git clone https://github.com/apache/shardingsphere-on-cloud.git
cd charts/shardingsphere-proxy/charts/governance
helm dependency build
cd ../..
helm dependency build
cd ..
helm install shardingsphere-proxy shardingsphere-proxy
```

Note:

1. Please refer to the configuration items description below for more details:
2. Execute `helm list` to acquire all installed releases.

Uninstall

1. Delete all release records by default, add `--keep-history` to keep them.

```
helm uninstall shardingsphere-proxy
```

Parameters

Governance-Node parameters

Name	Description	Value
<code>governance.enabled</code>	Switch to enable or disable the governance helm chart	<code>true</code>

Governance-Node ZooKeeper parameters

Name	Description	Value
governance.zookeeper.enabled	Switch to enable or disable the ZooKeeper helm chart	true
governance.zookeeper.replicaCount	Number of ZooKeeper nodes	1
governance.zookeeper.persistence.enabled	Enable persistence on ZooKeeper using PVC(s)	false
governance.zookeeper.persistence.storageClass	Persistent Volume storage class	""
governance.zookeeper.persistence.accessModes	Persistent Volume access modes	["ReadWriteOnce"]
governance.zookeeper.persistence.size	Persistent Volume size	8Gi
governance.zookeeper.resources.limits	The resources limits for the ZooKeeper containers	{}
governance.zookeeper.resources.requests.memory	The requested memory for the ZooKeeper containers	256Mi
governance.zookeeper.resources.requests.cpu	The requested cpu for the ZooKeeper containers	250m

Compute-Node ShardingSphere-Proxy parameters

Name	Description	Value
compute.image.repository	Image name of ShardingSphere-Proxy.	apache/sharding-sphere-proxy
compute.image.pullPolicy	The policy for pulling ShardingSphere-Proxy image	``IfNotPresent``
compute.image.tag	ShardingSphere-Proxy image tag	5.1.2
compute.imagePullSecrets	Specify docker-registry secret names as an array	[]
compute.resources.limits	The resources limits for the ShardingSphere-Proxy containers	{}
compute.resources.requests.memory	The requested memory for the ShardingSphere-Proxy containers	2Gi
compute.resources.requests.cpu	The requested cpu for the ShardingSphere-Proxy containers	200m
compute.replicas	Number of cluster replicas	3
compute.service.type	ShardingSphere-Proxy network mode	ClusterIP
compute.service.port	ShardingSphere-Proxy expose port	3307
compute.mysqlConnector.version	MySQL connector version	5.1.49
compute.startPort	ShardingSphere-Proxy start port	3307
compute.serverConfig	Server Configuration file for ShardingSphere-Proxy	""

Sample

values.yaml

```
#
# Licensed to the Apache Software Foundation (ASF) under one or more
# contributor license agreements. See the NOTICE file distributed with
# this work for additional information regarding copyright ownership.
# The ASF licenses this file to You under the Apache License, Version 2.0
# (the "License"); you may not use this file except in compliance with
# the License. You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
```

```

#

## @section Governance-Node parameters
## @param governance.enabled Switch to enable or disable the governance helm chart
##
governance:
  enabled: true
  ## @section Governance-Node ZooKeeper parameters
  zookeeper:
    ## @param governance.zookeeper.enabled Switch to enable or disable the ZooKeeper
    ## helm chart
    ##
    enabled: true
    ## @param governance.zookeeper.replicaCount Number of ZooKeeper nodes
    ##
    replicaCount: 1
    ## ZooKeeper Persistence parameters
    ## ref: https://kubernetes.io/docs/user-guide/persistent-volumes/
    ## @param governance.zookeeper.persistence.enabled Enable persistence on
    ZooKeeper using PVC(s)
    ## @param governance.zookeeper.persistence.storageClass Persistent Volume
    storage class
    ## @param governance.zookeeper.persistence.accessModes Persistent Volume access
    modes
    ## @param governance.zookeeper.persistence.size Persistent Volume size
    ##
    persistence:
      enabled: false
      storageClass: ""
      accessModes:
        - ReadWriteOnce
      size: 8Gi
    ## ZooKeeper's resource requests and limits
    ## ref: https://kubernetes.io/docs/user-guide/compute-resources/
    ## @param governance.zookeeper.resources.limits The resources limits for the
    ZooKeeper containers
    ## @param governance.zookeeper.resources.requests.memory The requested memory
    for the ZooKeeper containers
    ## @param governance.zookeeper.resources.requests.cpu The requested cpu for the
    ZooKeeper containers
    ##
    resources:
      limits: {}
      requests:
        memory: 256Mi
        cpu: 250m

## @section Compute-Node parameters

```

```

##
compute:
  ## @section Compute-Node ShardingSphere-Proxy parameters
  ## ref: https://kubernetes.io/docs/concepts/containers/images/
  ## @param compute.image.repository Image name of ShardingSphere-Proxy.
  ## @param compute.image.pullPolicy The policy for pulling ShardingSphere-Proxy
image
  ## @param compute.image.tag ShardingSphere-Proxy image tag
  ##
  image:
    repository: "apache/shardingsphere-proxy"
    pullPolicy: IfNotPresent
    ## Overrides the image tag whose default is the chart appVersion.
    ##
    tag: "5.1.2"
  ## @param compute.imagePullSecrets Specify docker-registry secret names as an array
  ## e.g:
  ## imagePullSecrets:
  ##   - name: myRegistryKeySecretName
  ##
  imagePullSecrets: []
  ## ShardingSphere-Proxy resource requests and limits
  ## ref: https://kubernetes.io/docs/concepts/configuration/manage-resources-
containers/
  ## @param compute.resources.limits The resources limits for the ShardingSphere-
Proxy containers
  ## @param compute.resources.requests.memory The requested memory for the
ShardingSphere-Proxy containers
  ## @param compute.resources.requests.cpu The requested cpu for the ShardingSphere-
Proxy containers
  ##
  resources:
    limits: {}
    requests:
      memory: 2Gi
      cpu: 200m
  ## ShardingSphere-Proxy Deployment Configuration
  ## ref: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
  ## ref: https://kubernetes.io/docs/concepts/services-networking/service/
  ## @param compute.replicas Number of cluster replicas
  ##
  replicas: 3
  ## @param compute.service.type ShardingSphere-Proxy network mode
  ## @param compute.service.port ShardingSphere-Proxy expose port
  ##
  service:
    type: ClusterIP
    port: 3307

```

```

## MySQL connector Configuration
## ref: https://shardingsphere.apache.org/document/current/en/quick-start/
shardingsphere-proxy-quick-start/
## @param compute.mysqlConnector.version MySQL connector version
##
mysqlConnector:
  version: "5.1.49"
## @param compute.startPort ShardingSphere-Proxy start port
## ShardingSphere-Proxy start port
## ref: https://shardingsphere.apache.org/document/current/en/user-manual/
shardingsphere-proxy/startup/docker/
##
startPort: 3307
## @section Compute-Node ShardingSphere-Proxy ServerConfiguration parameters
## NOTE: If you use the sub-charts to deploy Zookeeper, the server-lists field
must be "{ { printf \"%s-zookeeper.%s:2181\" .Release.Name .Release.Namespace } }",
## otherwise please fill in the correct zookeeper address
## The global.yaml is auto-generated based on this parameter.
## If it is empty, the global.yaml is also empty.
## ref: https://shardingsphere.apache.org/document/current/en/user-manual/
shardingsphere-jdbc/yaml-config/mode/
## ref: https://shardingsphere.apache.org/document/current/en/user-manual/common-
config/builtin-algorithm/metadata-repository/
##
serverConfig:
  ## @section Compute-Node ShardingSphere-Proxy ServerConfiguration authority
  parameters
  ## NOTE: It is used to set up initial user to login compute node, and authority
  data of storage node.
  ## ref: https://shardingsphere.apache.org/document/current/en/user-manual/
  shardingsphere-proxy/yaml-config/authentication/
  ## @param compute.serverConfig.authority.privilege.type authority provider for
  storage node, the default value is ALL_PERMITTED
  ## @param compute.serverConfig.authority.users[0].password Password for compute
  node.
  ## @param compute.serverConfig.authority.users[0].user Username,authorized host
  for compute node. Format: <username>@<hostname> hostname is % or empty string means
  do not care about authorized host
  ##
  authority:
    privilege:
      type: ALL_PRIVILEGES_PERMITTED
    users:
      - password: root
        user: root@%
  ## @section Compute-Node ShardingSphere-Proxy ServerConfiguration mode
  Configuration parameters
  ## @param compute.serverConfig.mode.type Type of mode configuration. Now only

```

```

support Cluster mode
  ## @param compute.serverConfig.mode.repository.props.namespace Namespace of
registry center
  ## @param compute.serverConfig.mode.repository.props.server-lists Server lists
of registry center
  ## @param compute.serverConfig.mode.repository.props.maxRetries Max retries of
client connection
  ## @param compute.serverConfig.mode.repository.props.
operationTimeoutMilliseconds Milliseconds of operation timeout
  ## @param compute.serverConfig.mode.repository.props.retryIntervalMilliseconds
Milliseconds of retry interval
  ## @param compute.serverConfig.mode.repository.props.timeToLiveSeconds Seconds
of ephemeral data live
  ## @param compute.serverConfig.mode.repository.type Type of persist repository.
Now only support ZooKeeper
  ## @param compute.serverConfig.mode.override Whether overwrite persistent
configuration with local configuration
  ##
mode:
  type: Cluster
  repository:
    type: ZooKeeper
    props:
      maxRetries: 3
      namespace: governance_ds
      operationTimeoutMilliseconds: 5000
      retryIntervalMilliseconds: 500
      server-lists: "{{ printf \"%s-zookeeper.%s:2181\" .Release.Name .Release.
Namespace }}"
      timeToLiveSeconds: 60
    overwrite: true

```

Add dependencies

This chapter mainly introduces how to download optional dependencies of ShardingSphere.

Add Narayana dependencies

Add Narayana dependencies

Adding Narayana dependencies requires downloading the following jar files and adding them under `ext-lib` path.

jar file downloads

- `arjuna-5.12.7.Final.jar`
- `common-5.12.7.Final.jar`
- `jboss-connector-api_1.7_spec-1.0.0.Final.jar`
- `jboss-logging-3.2.1.Final.jar`
- `jboss-transaction-api_1.2_spec-1.0.0.Alpha3.jar`
- `jboss-transaction-spi-7.6.1.Final.jar`
- `jta-5.12.7.Final.jar`
- `narayana-jts-integration-5.12.7.Final.jar`
- `shardingsphere-transaction-xa-narayana.jar`

Please download the corresponding `shardingsphere-transaction-xa-narayana.jar` file according to the proxy version.

9.2.2 Yaml Configuration

The YAML configuration of ShardingSphere-JDBC is the subset of ShardingSphere-Proxy. In `global.yaml` file, ShardingSphere-Proxy can configure authority feature and more properties for Proxy only.

Note: The YAML configuration file supports more than 3MB of configuration content.

This chapter will introduce the extra YAML configuration of ShardingSphere-Proxy.

Authentication & Authorization

Background

In ShardingSphere-Proxy, user authentication and authorization information is configured through `authority`.

Thanks to ShardingSphere's pluggable architecture, Proxy provides two levels of privilege providers, namely:

- `ALL_PERMITTED`: each user has all privileges without special authorization. (Will be removed in a future version)
- `DATABASE_PERMITTED`: grants the user privileges on the specified logical databases, defined by `user-database-mappings`. (Recommended)

The administrator can choose which privilege provider to use as needed when configuring `authority`.

Parameters

```

authority:
  users:
    - user: # Specify the username, and authorized host for logging in to the
      compute node. Format: <username>@<hostname>. When the hostname is % or an empty
      string, it indicates that the authorized host is not limited, username and hostname
      are case-insensitive
      password: # Password
      admin: # Optional, administrator identity. If true, the user has the highest
      authority. The default value is false
      authenticationMethodName: # Optional, used to specify the password
      authentication method for the user
      authenticators: # Optional, not required by default, Proxy will automatically
      choose the authentication method according to the frontend protocol type
      authenticatorName:
        type: # Authentication method type
      defaultAuthenticator: # Optional, specify an authenticator as the default password
      authentication method
      privilege:
        type: # Privilege provider type. The default value is ALL_PERMITTED

```

Sample

Minimalist configuration

```

authority:
  users:
    - user: root@%
      password: root
    - user: sharding
      password: sharding

```

Explanation: - Two users are defined: root@% and sharding; - authenticationMethodName is not specified for root@127.0.0.1, Proxy will automatically choose the authentication method according to the frontend protocol; - Privilege provider is not specified, the default ALL_PERMITTED will be used;

Authentication configuration

The custom authentication configuration allows users to greater leeway to set their own custom configurations according to their scenarios. Taking openGauss as the frontend protocol type as an example, its default authentication method is `scram-sha-256`. If the user sharding needs to use an old version of the `psql` client (which does not support `scram-sha-256`) to connect to the Proxy, the administrator may allow sharding to use the `md5` method for password authentication. The configuration is as follows:

```
authority:
  users:
    - user: root@127.0.0.1
      password: root
    - user: sharding
      password: sharding
      authenticationMethodName: md5
  authenticators:
    md5:
      type: MD5
  privilege:
    type: ALL_PERMITTED
```

Explanation: - Two users are defined: `root@127.0.0.1` and `sharding`; - Use MD5 method for password authentication for `sharding`; - Authentication method is not specified for `root@127.0.0.1`, Proxy will automatically choose one according to the frontend protocol; - The privilege provider `ALL_PERMITTED` is specified.

Authorization configuration

ALL_PERMITTED (Will be removed in a future version)

```
authority:
  users:
    - user: root@127.0.0.1
      password: root
    - user: sharding
      password: sharding
  privilege:
    type: ALL_PERMITTED
```

Explanation: - Two users are defined: `root@127.0.0.1` and `sharding`; - `authenticators` and `authenticationMethodName` are not defined, Proxy will automatically choose the authentication method according to the frontend protocol; - The privilege provider `ALL_PERMITTED` is specified.

DATABASE_PERMITTED (Recommended)

```
authority:
  users:
    - user: root@127.0.0.1
      password: root
      admin: true
    - user: sharding
      password: sharding
    - user: test
      password: sharding
  privilege:
    type: DATABASE_PERMITTED
  props:
    user-database-mappings: sharding@%=*, test@%=test_db, test@%=sharding_db
```

Explanation: - Three users are defined: root@127.0.0.1, sharding and test, which root@127.0.0.1 is an admin user; - authenticators and authenticationMethodName are not defined, Proxy will automatically choose the authentication method according to the frontend protocol; - The privilege provider DATABASE_PERMITTED is specified, authorize sharding@% to access all logical databases (*), and user test can only access test_db and sharding_db.

Properties

Background

Apache ShardingSphere provides a wealth of system configuration properties, which users can configure through `global.yaml`.

Parameters

Name*	Data Type*	Description	Default*	Dynamic Update*
system-log-level (?)	String	System log output level, supports DEBUG, INFO, WARN and ERROR, the default level is INFO.	INFO	True
sql-show (?)	boolean	Whether to print SQL in logs. Printing SQL can help developers quickly locate system problems. Logs contain the following contents: logical SQL, authentic SQL and SQL parsing result. If configuration is enabled, logs will use Topic ShardingSphere-SQL, and log level is INFO.	false	True
sql-simple (?)	boolean	Whether to print simple SQL in logs.	false	True
kernel-executor-size (?)	int	Set the size of the thread pool for task processing. Each ShardingSphere-DataSource uses an independent thread pool, and different data sources on the same JVM do not share thread pools.	infinite	False
max-connections	int	The maximum number of con-	1	True

Properties can be modified online through [DistSQL#RAL](#). Properties that support dynamic change can take effect immediately. For the ones that do not support dynamic change, the effect will be implemented after a restart.

Sample

For a complete sample, please refer to `global.yaml` in ShardingSphere's repository: <https://github.com/apache/shardingsphere/blob/612cd5d8e802d0d712a3a4d89da8fdc048d23879/proxy/bootstrap/src/main/resources/conf/global.yaml#L71-L89>

Rules

Background

This section explains how to configure the ShardingSphere-Proxy rules.

Parameters Explained

Rules configuration for ShardingSphere-Proxy is the same as ShardingSphere-JDBC. For details, please refer to [ShardingSphere-JDBC Rules Configuration](#).

Notice

Unlike ShardingSphere-JDBC, the following rules need to be configured in ShardingSphere-Proxy's `global.yaml`:

- [SQL Parsing](#)

```
sqlParser:
  sqlStatementCache:
    initialCapacity: 2000
    maximumSize: 65535
  parseTreeCache:
    initialCapacity: 128
    maximumSize: 1024
```

- [Distributed Transaction](#)

```
transaction:
  defaultType: XA
  providerType: Atomikos
```

- [SQL Translator](#)

```
sqlTranslator:
  type:
  useOriginalSQLWhenTranslatingFailed:
```

- [SQL Federation](#)

```
sqlFederation:
  sqlFederationEnabled: true
  allQueryUseSQLFederation: false
  executionPlanCache:
    initialCapacity: 2000
    maximumSize: 65535
```

Data Source

Background

ShardingSphere-Proxy supports common data source connection pools: HikariCP, C3P0, DBCP (C3P0, DBCP need download plugin from [shardingsphere-plugins](#) repository).

The connection pool can be specified through the parameter `dataSourceClassName`. When not specified, the default data source connection pool is HikariCP.

Parameters

```
dataSources: # Data sources configuration, multiple <data-source-name> available
  <data_source_name>: # Data source name
    dataSourceClassName: # Data source connection pool full class name
    url: # The database URL connection
    username: # Database username
    password: # The database password
    # ... Other properties of data source pool
```

Sample

```
dataSources:
  ds_1:
    url: jdbc:mysql://localhost:3306/ds_1
    username: root
    password:
  ds_2:
    dataSourceClassName: com.mchange.v2.c3p0.ComboPooledDataSource
    url: jdbc:mysql://localhost:3306/ds_2
    username: root
    password:
```

```
ds_3:
  dataSourceClassName: org.apache.commons.dbcp2.BasicDataSource
  url: jdbc:mysql://localhost:3306/ds_3
  username: root
  password:

# Configure other data sources
```

9.2.3 DistSQL

This chapter will introduce the detailed syntax of DistSQL.

Definition

DistSQL (Distributed SQL) is Apache ShardingSphere's specific SQL, providing additional operation capabilities compared to standard SQL.

Flexible rule configuration and resource management & control capabilities are one of the characteristics of Apache ShardingSphere.

When using 4.x and earlier versions, developers can operate data just like using a database, but they need to configure resources and rules through YAML file (or registry center). However, the YAML file format and the changes brought by using the registry center made it unfriendly to DBAs.

Starting from version 5.x, DistSQL enables users to operate Apache ShardingSphere just like a database, transforming it from a framework and middleware for developers to a database product for DBAs.

Related Concepts

DistSQL is divided into RDL, RQL, RAL and RUL.

RDL

Resource & Rule Definition Language, is responsible for the definition of resources and rules.

RQL

Resource & Rule Query Language, is responsible for the query of resources and rules.

RAL

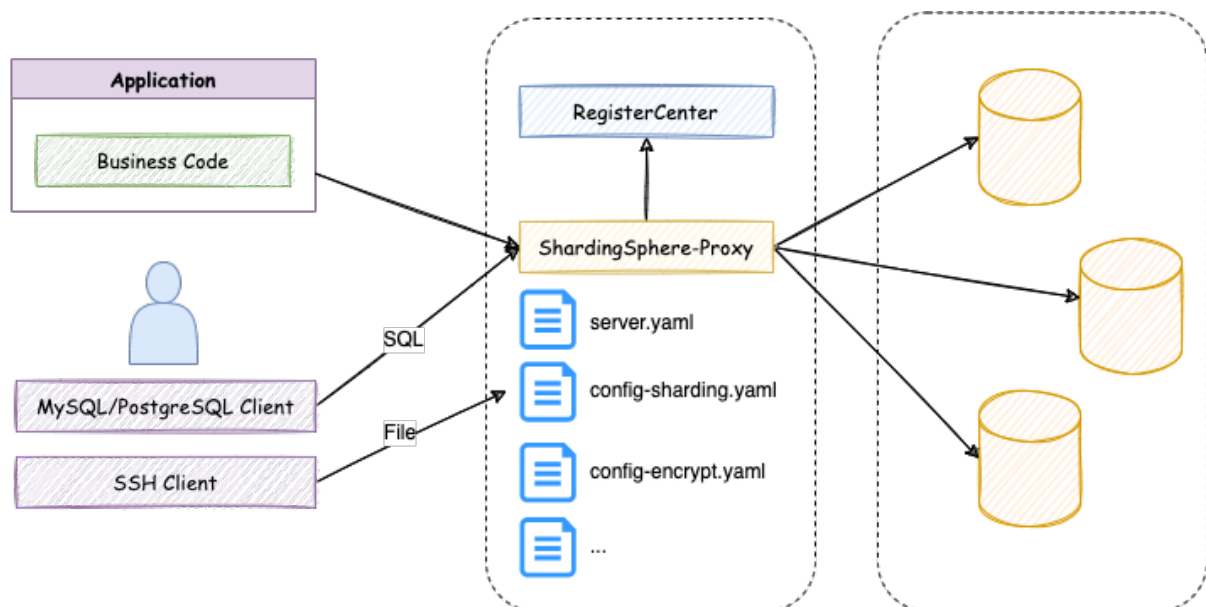
Resource & Rule Administration Language, is responsible for hint, circuit breaker, configuration import and export, scaling control and other management functions.

RUL

Resource & Rule Utility Language, is responsible for SQL parsing, SQL formatting, preview execution plan, etc.

Impact on the System**Before**

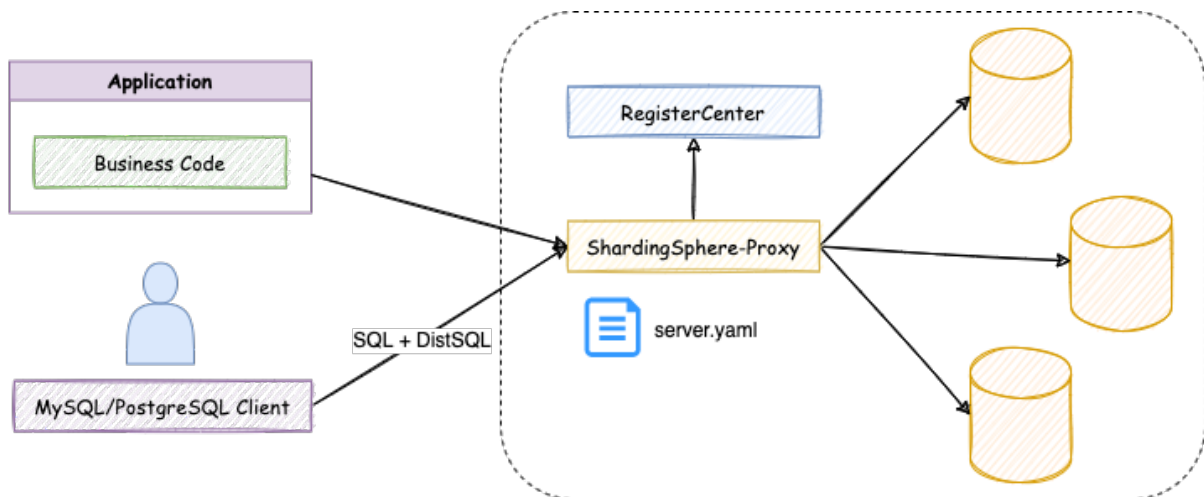
Before having DistSQL, users used SQL to operate data while using YAML configuration files to manage ShardingSphere, as shown below:



At that time, users faced the following problems: - Different types of clients are required to operate data and manage ShardingSphere configuration. - Multiple logical databases require multiple YAML files. - Editing a YAML file requires writing permissions. - Need to restart ShardingSphere after editing YAML.

After

With the advent of DistSQL, the operation of ShardingSphere has also changed:



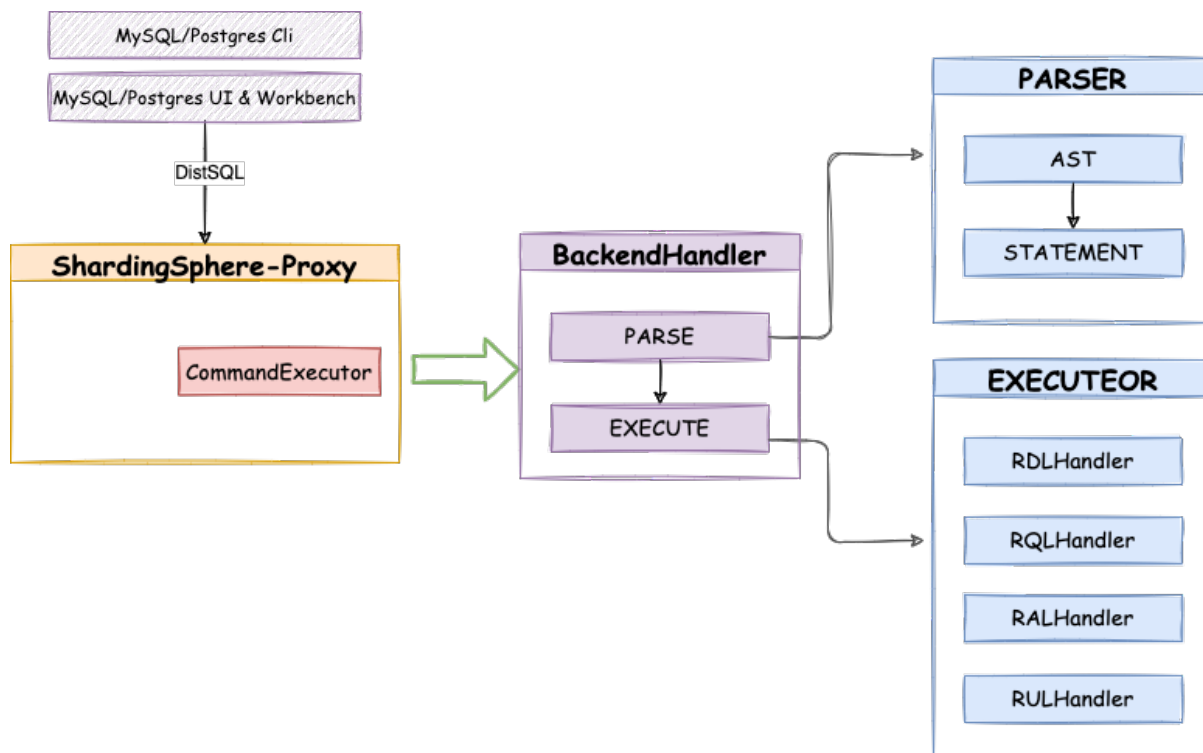
Now, the user experience has been greatly improved: - Uses the same client to operate data and ShardingSphere configuration. - No need for additional YAML files, and the logical databases are managed through DistSQL. - Editing permissions for files are no longer required, and configuration is managed through DistSQL. - Configuration changes take effect in real-time without restarting ShardingSphere.

Limitations

DistSQL can be used only with ShardingSphere-Proxy, not with ShardingSphere-JDBC for now.

How it works

Like standard SQL, DistSQL is recognized by the parsing engine of ShardingSphere. It converts the input statement into an abstract syntax tree and then generates the Statement corresponding to each grammar, which is processed by the appropriate Handler.



Related References

User Manual: DistSQL

Syntax

This chapter describes the syntax of DistSQL in detail, and introduces use of DistSQL with practical examples.

Syntax Rule

In DistSQL statement, except for keywords, the input format of other elements shall conform to the following rules.

Identifier

1. The identifier represents an object in the SQL statement, including:
 - database name
 - table name
 - column name
 - index name
 - resource name

- rule name
 - algorithm name
2. The allowed characters in the identifier are: [a-z, A-Z, 0-9, _] (letters, numbers, under-scores) and should start with a letter.
 3. When keywords or special characters appear in the identifier, use the backticks (`).

Literal

Types of literals include:

- string: enclosed in single quotes (') or double quotes (“)
- int: it is generally a positive integer, such as 0-9;

Note: some DistSQL syntax allows negative values. In this case, a negative sign (-) can be added before the number, such as -1.

- boolean, containing only true & false. Case insensitive.

Special Instructions

- The "" must be used to mark the algorithm type name when specifying a user-defined algorithm type name, for example, NAME="AlgorithmTypeName"
- The "" is not necessary when specifying a ShardingSphere [Built-in algorithm](#) type name, for example, NAME=HASH_MOD

RDL Syntax

RDL (Resource & Rule Definition Language) responsible for definition of resources/rules.

Storage Unit Definition

This chapter describes the syntax of storage unit.

REGISTER STORAGE UNIT

Description

The REGISTER STORAGE UNIT syntax is used to register storage unit for the currently selected logical database.

Syntax

```

RegisterStorageUnit ::=
    'REGISTER' 'STORAGE' 'UNIT' ifNotExists? storageUnitsDefinition (','
checkPrivileges)?

storageUnitsDefinition ::=
    storageUnitDefinition (',' storageUnitDefinition)*

storageUnitDefinition ::=
    storageUnitName '(' ('HOST' '=' hostName ',' 'PORT' '=' port ',' 'DB' '=' dbName |
'URL' '=' url) ',' 'USER' '=' user (',' 'PASSWORD' '=' password)? (','
propertiesDefinition)? ')'

ifNotExists ::=
    'IF' 'NOT' 'EXISTS'

storageUnitName ::=
    identifier

hostname ::=
    string

port ::=
    int

dbName ::=
    string

url ::=
    string

user ::=
    string

password ::=
    string

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
    string

value ::=
    literal

checkPrivileges ::=

```

```
'CHECK_PRIVILEGES' '=' privilegeType (',' privilegeType)*
```

```
privilegeType ::=  
    identifier
```

Supplement

- Before register storage units, please confirm that a database has been created in Proxy, and execute the use command to successfully select a database;
- Confirm that the registered storage unit can be connected normally, otherwise it will not be added successfully;
- storageUnitName is case-sensitive;
- storageUnitName needs to be unique within the current database;
- storageUnitName name only allows letters, numbers and _, and must start with a letter;
- PROPERTIES is optional, used to customize connection pool properties, key must be the same as the connection pool property name;
- CHECK_PRIVILEGES can be specified to check privileges of the storage unit user. The supported types of privilegeType are SELECT, XA, PIPELINE, and NONE. The default value is SELECT. When NONE is included in the type list, the privilege check is skipped.

Example

- Register storage unit using HOST & PORT method

```
REGISTER STORAGE UNIT ds_0 (  
    HOST="127.0.0.1",  
    PORT=3306,  
    DB="db_0",  
    USER="root",  
    PASSWORD="root"  
);
```

- Register storage unit and set connection pool properties using HOST & PORT method

```
REGISTER STORAGE UNIT ds_1 (  
    HOST="127.0.0.1",  
    PORT=3306,  
    DB="db_1",  
    USER="root",  
    PASSWORD="root",  
    PROPERTIES("maximumPoolSize"=10)  
);
```

- Register storage unit and set connection pool properties using URL method

```
REGISTER STORAGE UNIT ds_2 (
  URL="jdbc:mysql://127.0.0.1:3306/db_2?serverTimezone=UTC&useSSL=false&
allowPublicKeyRetrieval=true",
  USER="root",
  PASSWORD="root",
  PROPERTIES("maximumPoolSize"=10,"idleTimeout"=30000)
);
```

- Register storage unit with ifNotExists clause

```
REGISTER STORAGE UNIT IF NOT EXISTS ds_0 (
  HOST="127.0.0.1",
  PORT=3306,
  DB="db_0",
  USER="root",
  PASSWORD="root"
);
```

- Check SELECT, XA and PIPELINE privileges when registering

```
REGISTER STORAGE UNIT ds_3 (
  URL="jdbc:mysql://127.0.0.1:3306/db_3?serverTimezone=UTC&useSSL=false&
allowPublicKeyRetrieval=true",
  USER="root",
  PASSWORD="root",
  PROPERTIES("maximumPoolSize"=10,"idleTimeout"=30000)
), CHECK_PRIVILEGES=SELECT,XA,PIPELINE;
```

Reserved word

REGISTER, STORAGE, UNIT, HOST, PORT, DB, USER, PASSWORD, PROPERTIES, URL, CHECK_PRIVILEGES

Related links

- [Reserved word](#)

ALTER STORAGE UNIT

Description

The ALTER STORAGE UNIT syntax is used to alter storage units for the currently selected logical database.

Syntax

```
AlterStorageUnit ::=
    'ALTER' 'STORAGE' 'UNIT' storageUnitsDefinition (',' checkPrivileges)?

storageUnitsDefinition ::=
    storageUnitDefinition (',' storageUnitDefinition)*

storageUnitDefinition ::=
    storageUnitName '(' ('HOST' '=' hostName ',' 'PORT' '=' port ',' 'DB' '=' dbName |
    'URL' '=' url) ',' 'USER' '=' user (',' 'PASSWORD' '=' password)? (','
    propertiesDefinition)? ')'

storageUnitName ::=
    identifier

hostname ::=
    string

port ::=
    int

dbName ::=
    string

url ::=
    string

user ::=
    string

password ::=
    string

propertiesDefinition ::=
```

```

'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
    string

value ::=
    literal

checkPrivileges ::=
    'CHECK_PRIVILEGES' '=' privilegeType (',' privilegeType)*

privilegeType ::=
    identifier

```

Supplement

- Before altering the storage units, please confirm that a database exists in Proxy, and execute the use command to select a database;
- ALTER STORAGE UNIT is not allowed to change the real data source associated with this storage-Unit (determined by host, port and db);
- ALTER STORAGE UNIT will switch the connection pool. This operation may affect the ongoing business, please use it with caution;
- Please confirm that the storage unit to be altered can be connected successfully, otherwise the altering will fail;
- PROPERTIES is optional, used to customize connection pool properties, key must be the same as the connection pool property name;
- CHECK_PRIVILEGES can be specified to check privileges of the storage unit user. The supported types of privilegeType are SELECT, XA, PIPELINE, and NONE. The default value is SELECT. When NONE is included in the type list, the privilege check is skipped.

Example

- Alter storage unit using HOST & PORT method

```

ALTER STORAGE UNIT ds_0 (
    HOST="127.0.0.1",
    PORT=3306,
    DB="db_0",
    USER="root",
    PASSWORD="root"
);

```

- Alter storage unit and set connection pool properties using HOST & PORT method


```
ALTER STORAGE UNIT ds_1 (
  HOST="127.0.0.1",
  PORT=3306,
  DB="db_1",
  USER="root",
  PASSWORD="root",
  PROPERTIES("maximumPoolSize"=10)
);
```

- Alter storage unit and set connection pool properties using URL method

```
ALTER STORAGE UNIT ds_2 (
  URL="jdbc:mysql://127.0.0.1:3306/db_2?serverTimezone=UTC&useSSL=false&
allowPublicKeyRetrieval=true",
  USER="root",
  PASSWORD="root",
  PROPERTIES("maximumPoolSize"=10,"idleTimeout"=30000)
);
```

- Check SELECT, XA and PIPELINE privileges when altering

```
ALTER STORAGE UNIT ds_2 (
  URL="jdbc:mysql://127.0.0.1:3306/db_2?serverTimezone=UTC&useSSL=false&
allowPublicKeyRetrieval=true",
  USER="root",
  PASSWORD="root",
  PROPERTIES("maximumPoolSize"=10,"idleTimeout"=30000)
), CHECK_PRIVILEGES=SELECT,XA,PIPELINE;
```

Reserved word

ALTER, STORAGE, UNIT, HOST, PORT, DB, USER, PASSWORD, PROPERTIES, URL, CHECK_PRIVILEGES

Related links

- [Reserved word](#)

UNREGISTER STORAGE UNIT

Description

The UNREGISTER STORAGE UNIT syntax is used to unregister storage unit from the current database

Syntax

```

UnregisterStorageUnit ::=
    'UNREGISTER' 'STORAGE' 'UNIT' ifExists? storageUnitName (',' storageUnitName)*
    ignoreTables?

ifExists ::=
    'IF' 'EXISTS'

storageUnitName ::=
    identifier

ignoreTables ::=
    'IGNORE' ('SINGLE')? (',')? ('BROADCAST')? 'TABLES'

```

Supplement

- UNREGISTER STORAGE UNIT will only unregister storage unit in Proxy, the real data source corresponding to the storage unit will not be unregistered;
- Unable to unregister storage unit already used by rules. Storage unit are still in used. will be prompted when removing storage units used by rules;
- The storage unit need to be removed only contains SINGLE RULE, BROADCAST RULE and when the user confirms that this restriction can be ignored, the IGNORE SINGLE TABLES, IGNORE BROADCAST TABLES, IGNORE SINGLE, BROADCAST TABLES keyword can be added to remove the storage unit;
- ifExists clause is used for avoid Storage unit not exists error.

Example

- Unregister a storage unit

```
UNREGISTER STORAGE UNIT ds_0;
```

- Unregister multiple storage units

```
UNREGISTER STORAGE UNIT ds_0, ds_1;
```

- Unregister storage unit and ignore single tables

```
UNREGISTER STORAGE UNIT ds_0 IGNORE SINGLE TABLES;
```

- Unregister storage unit and ignore broadcast tables

```
UNREGISTER STORAGE UNIT ds_0 IGNORE BROADCAST TABLES;
```

- Unregister storage unit, ignore single and broadcast tables

```
UNREGISTER STORAGE UNIT ds_0 IGNORE SINGLE, BROADCAST TABLES;
```

- Unregister storage unit with `ifExists` clause

```
UNREGISTER STORAGE UNIT IF EXISTS ds_0;
```

Reserved word

UNREGISTER, STORAGE, UNIT, IF, EXISTS, IGNORE, SINGLE, BROADCAST, TABLES

Related links

- [Reserved word](#)

Rule Definition

This chapter describes the syntax of rule definition.

Sharding

This chapter describes the syntax of sharding.

CREATE SHARDING TABLE RULE

Description

The `CREATE SHARDING TABLE RULE` syntax is used to add sharding table rule for the currently selected database

Syntax

```
CreateShardingTableRule ::=
    'CREATE' 'SHARDING' 'TABLE' 'RULE' ifNotExists? (tableRuleDefinition |
    autoTableRuleDefinition) (',' (tableRuleDefinition | autoTableRuleDefinition))*

ifNotExists ::=
    'IF' 'NOT' 'EXISTS'

tableRuleDefinition ::=
    ruleName '(' 'DATANODES' '(' dataNode (',' dataNode)* ')' (',' 'DATABASE_STRATEGY'
    '(' strategyDefinition ')')? (',' 'TABLE_STRATEGY' '(' strategyDefinition ')')? (
```

```

',' 'KEY_GENERATE_STRATEGY' '(' keyGenerateStrategyDefinition ')')? (',' 'AUDIT_
STRATEGY' '(' auditStrategyDefinition ')')? ')'

autoTableRuleDefinition ::=
    ruleName '(' 'STORAGE_UNITS' '(' storageUnitName (',' storageUnitName)* ')' ','
'SHARDING_COLUMN' '=' columnName ',' algorithmDefinition (',' 'KEY_GENERATE_STRATEGY
' '(' keyGenerateStrategyDefinition ')')? (',' 'AUDIT_STRATEGY' '('
auditStrategyDefinition ')')? ')'

strategyDefinition ::=
    'TYPE' '=' strategyType ',' ('SHARDING_COLUMN' | 'SHARDING_COLUMNS') '='
columnName ',' algorithmDefinition

keyGenerateStrategyDefinition ::=
    'KEY_GENERATE_STRATEGY' '(' 'COLUMN' '=' columnName ',' algorithmDefinition ')'

auditStrategyDefinition ::=
    'AUDIT_STRATEGY' '(' algorithmDefinition (',' algorithmDefinition)* ')'

algorithmDefinition ::=
    'TYPE' '(' 'NAME' '=' algorithmType (',' propertiesDefinition)? ')'

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
    string

value ::=
    literal

ruleName ::=
    identifier

dataNode ::=
    string

storageUnitName ::=
    identifier

columnName ::=
    identifier

strategyType ::=
    string

algorithmType ::=
    string

```

Supplement

- `tableRuleDefinition` is defined for standard sharding table rule; `autoTableRuleDefinition` is defined for auto sharding table rule. For standard sharding rules and auto sharding rule, refer to [Data Sharding](#);
- use standard sharding table rule:
 - DATANODES can only use resources that have been added to the current database, and can only use INLINE expressions to specify required resources;
 - DATABASE_STRATEGY, TABLE_STRATEGY are the database sharding strategy and the table sharding strategy, which are optional, and the default strategy is used when not configured;
 - The attribute TYPE in `strategyDefinition` is used to specify the type of [Sharding Algorithm](#), currently only supports STANDARD, COMPLEX. Using COMPLEX requires specifying multiple sharding columns with SHARDING_COLUMNS.
- use auto sharding table rule:
 - STORAGE_UNITS can only use storage units that have been registered to the current database, and the required storage units can be specified by enumeration or INLINE expression;
 - Only auto sharding algorithm can be used, please refer to [Auto Sharding Algorithm](#).
- `algorithmType` is the sharding algorithm type, please refer to [Sharding Algorithm](#);
- The auto-generated algorithm naming rule is `tableName _ strategyType _ shardingAlgorithmType`;
- The auto-generated primary key strategy naming rule is `tableName _ strategyType`;
- KEY_GENERATE_STRATEGY is used to specify the primary key generation strategy, which is optional. For the primary key generation strategy, please refer to [Distributed Primary Key](#);
- AUDIT_STRATEGY is used to specify the sharding audit strategy, which is optional. For the sharding audit generation strategy, please refer to [Sharding Audit](#);
- `ifNotExists` clause is used for avoid Duplicate sharding rule error.

Example

1. Standard sharding table rule

```
CREATE SHARDING TABLE RULE t_order_item (
  DATANODES("ds_${0..1}.t_order_item_${0..1}"),
  DATABASE_STRATEGY(TYPE="standard",SHARDING_COLUMN=user_id,SHARDING_
  ALGORITHM(TYPE(NAME="inline",PROPERTIES("algorithm-expression"="ds_${user_id % 2}
  "))),
  TABLE_STRATEGY(TYPE="standard",SHARDING_COLUMN=order_id,SHARDING_
  ALGORITHM(TYPE(NAME="inline",PROPERTIES("algorithm-expression"="t_order_item_
```

```
{order_id % 2}")))),
KEY_GENERATE_STRATEGY(COLUMN=another_id,TYPE(NAME="snowflake")),
AUDIT_STRATEGY (TYPE(NAME="DML_SHARDING_CONDITIONS"),ALLOW_HINT_DISABLE=true)
);
```

2.Auto sharding table rule

```
CREATE SHARDING TABLE RULE t_order (
STORAGE_UNITS(ds_0,ds_1),
SHARDING_COLUMN=order_id,TYPE(NAME="hash_mod",PROPERTIES("sharding-count"="4")),
KEY_GENERATE_STRATEGY(COLUMN=another_id,TYPE(NAME="snowflake")),
AUDIT_STRATEGY (TYPE(NAME="DML_SHARDING_CONDITIONS"),ALLOW_HINT_DISABLE=true)
);
```

3.Create sharding rule with ifNotExists clause

- Standard sharding table rule

```
CREATE SHARDING TABLE RULE IF NOT EXISTS t_order_item (
DATANODES("ds_${0..1}.t_order_item_${0..1}"),
DATABASE_STRATEGY(TYPE="standard",SHARDING_COLUMN=user_id,SHARDING_
ALGORITHM(TYPE(NAME="inline",PROPERTIES("algorithm-expression"="ds_${user_id % 2}
")))),
TABLE_STRATEGY(TYPE="standard",SHARDING_COLUMN=order_id,SHARDING_
ALGORITHM(TYPE(NAME="inline",PROPERTIES("algorithm-expression"="t_order_item_${
order_id % 2}")))),
KEY_GENERATE_STRATEGY(COLUMN=another_id,TYPE(NAME="snowflake")),
AUDIT_STRATEGY (TYPE(NAME="DML_SHARDING_CONDITIONS"),ALLOW_HINT_DISABLE=true)
);
```

- Auto sharding table rule

```
CREATE SHARDING TABLE RULE IF NOT EXISTS t_order (
STORAGE_UNITS(ds_0,ds_1),
SHARDING_COLUMN=order_id,TYPE(NAME="hash_mod",PROPERTIES("sharding-count"="4")),
KEY_GENERATE_STRATEGY(COLUMN=another_id,TYPE(NAME="snowflake")),
AUDIT_STRATEGY (TYPE(NAME="DML_SHARDING_CONDITIONS"),ALLOW_HINT_DISABLE=true)
);
```

Reserved word

CREATE, SHARDING, TABLE, RULE, DATANODES, DATABASE_STRATEGY, TABLE_STRATEGY, KEY_GENERATE_STRATEGY, STORAGE_UNITS, SHARDING_COLUMN, TYPE, SHARDING_COLUMN, KEY_GENERATOR, SHARDING_ALGORITHM, COLUMN, NAME, PROPERTIES, AUDIT_STRATEGY, AUDITORS, ALLOW_HINT_DISABLE

Related links

- [Reserved word](#)
- [CREATE DEFAULT_SHARDING STRATEGY](#)

ALTER SHARDING TABLE RULE

Description

The ALTER SHARDING TABLE RULE syntax is used to alter sharding table rule for the currently selected database

Syntax

```
AlterShardingTableRule ::=
  'ALTER' 'SHARDING' 'TABLE' 'RULE' (tableRuleDefinition | autoTableRuleDefinition)
  (',' (tableRuleDefinition | autoTableRuleDefinition))*

tableRuleDefinition ::=
  ruleName '(' 'DATANODES' '(' dataNode(',' dataNode)* ')' '(' 'DATABASE_STRATEGY'
  '(' strategyDefinition ')')? (',' 'TABLE_STRATEGY' '(' strategyDefinition ')')? (
  ',' 'KEY_GENERATE_STRATEGY' '(' keyGenerateStrategyDefinition ')')? (',' 'AUDIT_
  STRATEGY' '(' auditStrategyDefinition ')')? ')'

autoTableRuleDefinition ::=
  ruleName '(' 'STORAGE_UNITS' '(' storageUnitName(',' storageUnitName)* ')' ','
  'SHARDING_COLUMN' '=' columnName ',' algorithmDefinition (',' 'KEY_GENERATE_STRATEGY'
  '(' keyGenerateStrategyDefinition ')')? (',' 'AUDIT_STRATEGY' '('
  auditStrategyDefinition ')')? ')'

strategyDefinition ::=
  'TYPE' '=' strategyType ',' ('SHARDING_COLUMN' | 'SHARDING_COLUMNS') '='
  columnName ',' algorithmDefinition

keyGenerateStrategyDefinition ::=
  'KEY_GENERATE_STRATEGY' '(' 'COLUMN' '=' columnName ',' algorithmDefinition ')'

auditStrategyDefinition ::=
```

```

'AUDIT_STRATEGY' '(' algorithmDefinition (',' algorithmDefinition)* ')'

algorithmDefinition ::=
  'TYPE' '(' 'NAME' '=' algorithmType (',' propertiesDefinition)? ')'

propertiesDefinition ::=
  'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
  string

value ::=
  literal

ruleName ::=
  identifier

dataNode ::=
  string

storageUnitName ::=
  identifier

columnName ::=
  identifier

strategyType ::=
  string

algorithmType ::=
  string

```

Supplement

- tableRuleDefinition is defined for standard sharding table rule; autoTableRuleDefinition is defined for auto sharding table rule. For standard sharding rules and auto sharding rule, refer to [Data Sharding](#);
- use standard sharding table rule:
 - DATANODES can only use resources that have been added to the current database, and can only use INLINE expressions to specify required resources;
 - DATABASE_STRATEGY, TABLE_STRATEGY are the database sharding strategy and the table sharding strategy, which are optional, and the default strategy is used when not configured;
 - The attribute TYPE in strategyDefinition is used to specify the type of [Sharding Algorithm](#), currently only supports STANDARD, COMPLEX. Using COMPLEX requires specifying

multiple sharding columns with SHARDING_COLUMNS.

- use auto sharding table rule:
 - STORAGE_UNITS can only use storage units that have been registered to the current database, and the required storage units can be specified by enumeration or INLINE expression;
 - Only auto sharding algorithm can be used, please refer to [Auto Sharding Algorithm](#).
- algorithmType is the sharding algorithm type, please refer to [Sharding Algorithm](#);
- The auto-generated algorithm naming rule is tableName _ strategyType _ shardingAlgorithmType;
- The auto-generated primary key strategy naming rule is tableName _ strategyType;
- KEY_GENERATE_STRATEGY is used to specify the primary key generation strategy, which is optional. For the primary key generation strategy, please refer to [Distributed Primary Key](#).
- AUDIT_STRATEGY is used to specify the sharding audit strategy, which is optional. For the sharding audit generation strategy, please refer to [Sharding Audit](#).

Example

1. Standard sharding table rule

```
ALTER SHARDING TABLE RULE t_order_item (
  DATANODES("ds_${0..3}.t_order_item${0..3}"),
  DATABASE_STRATEGY(TYPE="standard", SHARDING_COLUMN=user_id, SHARDING_
  ALGORITHM(TYPE(NAME="inline", PROPERTIES("algorithm-expression"="ds_${user_id % 4}
  "))))),
  TABLE_STRATEGY(TYPE="standard", SHARDING_COLUMN=order_id, SHARDING_
  ALGORITHM(TYPE(NAME="inline", PROPERTIES("algorithm-expression"="t_order_item_${
  order_id % 4}")))),
  KEY_GENERATE_STRATEGY(COLUMN=another_id, TYPE(NAME="snowflake")),
  AUDIT_STRATEGY(TYPE(NAME="ddl_sharding_conditions"), ALLOW_HINT_DISABLE=true)
);
```

2. Auto sharding table rule

```
ALTER SHARDING TABLE RULE t_order (
  STORAGE_UNITS(ds_0, ds_1, ds_2, ds_3),
  SHARDING_COLUMN=order_id, TYPE(NAME="hash_mod", PROPERTIES("sharding-count"="16")),
  KEY_GENERATE_STRATEGY(COLUMN=another_id, TYPE(NAME="snowflake")),
  AUDIT_STRATEGY(TYPE(NAME="ddl_sharding_conditions"), ALLOW_HINT_DISABLE=true)
);
```

Reserved word

ALTER, SHARDING, TABLE, RULE, DATANODES, DATABASE_STRATEGY, TABLE_STRATEGY, KEY_GENERATE_STRATEGY, STORAGE_UNITS, SHARDING_COLUMN, TYPE, SHARDING_COLUMN, KEY_GENERATOR, SHARDING_ALGORITHM, COLUMN, NAME, PROPERTIES, AUDIT_STRATEGY, AUDITORS, ALLOW_HINT_DISABLE

Related links

- [Reserved word](#)
- [ALTER DEFAULT_SHARDING STRATEGY](#)

DROP SHARDING TABLE RULE

Description

The DROP SHARDING TABLE RULE syntax is used to drop sharding table rule for specified database.

Syntax

```
DropShardingTableRule ::=
  'DROP' 'SHARDING' 'TABLE' 'RULE' ifExists? ruleName (',' ruleName)* ('FROM'
  databaseName)?

ifExists ::=
  'IF' 'EXISTS'

ruleName ::=
  identifier

databaseName ::=
  identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted;
- ifExists clause is used to avoid Sharding rule not exists error.

Example

- Drop multiple sharding table rules for specified database

```
DROP SHARDING TABLE RULE t_order, t_order_item FROM sharding_db;
```

- Drop a sharding table rule for current database

```
DROP SHARDING TABLE RULE t_order;
```

- Drop sharding table rule with ifExists clause

```
DROP SHARDING TABLE RULE IF EXISTS t_order;
```

Reserved word

DROP, SHARDING, TABLE, RULE, FROM

Related links

- [Reserved word](#)

CREATE DEFAULT SHARDING STRATEGY

Description

The CREATE DEFAULT SHARDING STRATEGY syntax is used to create a default sharding strategy

Syntax

```
CreateDefaultShardingStrategy ::=
  'CREATE' 'DEFAULT' 'SHARDING' ('DATABASE' | 'TABLE') 'STRATEGY' ifNotExists? '('
shardingStrategy ')'
```

```
ifNotExists ::=
  'IF' 'NOT' 'EXISTS'
```

```
shardingStrategy ::=
  'TYPE' '=' strategyType ',' ('SHARDING_COLUMN' '=' columnName | 'SHARDING_COLUMNS'
  '=' columnNames) ',' 'SHARDING_ALGORITHM' '=' algorithmDefinition
```

```
strategyType ::=
  string
```

```
algorithmDefinition ::=
```

```

'<code>TYPE' '(' 'NAME' '=' algorithmType ',' propertiesDefinition ')'

```

Supplement

- When using the complex sharding algorithm, multiple sharding columns need to be specified using SHARDING_COLUMNS;
- algorithmType is the sharding algorithm type. For detailed sharding algorithm type information, please refer to [Sharding Algorithm](#);
- ifNotExists clause is used for avoid Duplicate default sharding strategy error.

Example

- create a default sharding table strategy

```

-- create a default sharding table strategy
CREATE DEFAULT SHARDING TABLE STRATEGY (
  TYPE="standard", SHARDING_COLUMN=user_id, SHARDING_ALGORITHM(TYPE(NAME="inline",
  PROPERTIES("algorithm-expression"="t_order_${user_id % 2}"))
);

```

- create a default sharding table strategy with ifNotExists clause

```

CREATE DEFAULT SHARDING TABLE STRATEGY IF NOT EXISTS (
  TYPE="standard", SHARDING_COLUMN=user_id, SHARDING_ALGORITHM(TYPE(NAME="inline",
  PROPERTIES("algorithm-expression"="t_order_${user_id % 2}"))
);

```

Reserved word

CREATE, DEFAULT, SHARDING, DATABASE, TABLE, STRATEGY, TYPE, SHARDING_COLUMN, SHARDING_COLUMNS, SHARDING_ALGORITHM, NAME, PROPERTIES

Related links

- [Reserved word](#)

ALTER DEFAULT SHARDING STRATEGY

Description

The ALTER DEFAULT SHARDING STRATEGY syntax is used to alter a default sharding strategy

Syntax

```
AlterDefaultShardingStrategy ::=
  'ALTER' 'DEFAULT' 'SHARDING' ('DATABASE' | 'TABLE') 'STRATEGY' '('
shardingStrategy ')'

shardingStrategy ::=
  'TYPE' '=' strategyType ',' ('SHARDING_COLUMN' '=' columnName | 'SHARDING_COLUMNS'
  '=' columnNames) ',' 'SHARDING_ALGORITHM' '=' algorithmDefinition

strategyType ::=
  string

algorithmDefinition ::=
  'TYPE' '(' 'NAME' '=' algorithmType ',' propertiesDefinition ')'

columnNames ::=
  columnName (',' columnName)+

columnName ::=
  identifier

algorithmType ::=
  string

propertiesDefinition ::=
  'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
  string
```

```
value ::=
    literal
```

Supplement

- When using the complex sharding algorithm, multiple sharding columns need to be specified using SHARDING_COLUMNS;
- `algorithmType` is the sharding algorithm type. For detailed sharding algorithm type information, please refer to [Sharding Algorithm](#).

Example

- Alter a default sharding table strategy

```
ALTER DEFAULT SHARDING TABLE STRATEGY (
    TYPE="standard", SHARDING_COLUMN=user_id, SHARDING_ALGORITHM(TYPE(NAME="inline",
    PROPERTIES("algorithm-expression"="t_order_${user_id % 2}")))
);
```

Reserved word

ALTER, DEFAULT, SHARDING, DATABASE, TABLE, STRATEGY, TYPE, SHARDING_COLUMN, SHARDING_COLUMNS, SHARDING_ALGORITHM, NAME, PROPERTIES

Related links

- [Reserved word](#)

DROP DEFAULT SHARDING STRATEGY

Description

The DROP DEFAULT SHARDING STRATEGY syntax is used to drop default sharding strategy for specified database.

Syntax

```
DropDefaultShardingStrategy ::=
    'DROP' 'DEFAULT' 'SHARDING' ('TABLE' | 'DATABASE') 'STRATEGY' ifExists? ('FROM'
databaseName)?

ifExists ::=
    'IF' 'EXISTS'

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted;
- ifExists clause is used for avoid Default sharding strategy not exists error.

Example

- Drop default sharding table strategy for specified database

```
DROP DEFAULT SHARDING TABLE STRATEGY FROM sharding_db;
```

- Drop default sharding database strategy for current database

```
DROP DEFAULT SHARDING DATABASE STRATEGY;
```

- Drop default sharding table strategy with ifExists clause

```
DROP DEFAULT SHARDING TABLE STRATEGY IF EXISTS;
```

- Drop default sharding database strategy with ifExists clause

```
DROP DEFAULT SHARDING DATABASE STRATEGY IF EXISTS;
```

Reserved word

DROP, DEFAULT, SHARDING, TABLE, DATABASE, STRATEGY, FROM

Related links

- [Reserved word](#)

DROP SHARDING KEY GENERATOR

Description

The `DROP SHARDING KEY GENERATOR` syntax is used to drop sharding key generator for specified database.

Syntax

```
DropShardingKeyGenerator ::=
  'DROP' 'SHARDING' 'KEY' 'GENERATOR' ifExists? keyGeneratorName (keyGeneratorName)*
  ('FROM' databaseName)?

ifExists ::=
  'IF' 'EXISTS'

keyGeneratorName ::=
  identifier

databaseName ::=
  identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used `DATABASE`. If `DATABASE` is not used, No database selected will be prompted;
- `ifExists` clause is used for avoid Sharding key generator not exists error.

Example

- Drop sharding key generator for specified database

```
DROP SHARDING KEY GENERATOR t_order_snowflake FROM sharding_db;
```

- Drop sharding key generator for current database

```
DROP SHARDING KEY GENERATOR t_order_snowflake;
```

- Drop sharding key generator with `ifExists` clause


```
DROP SHARDING KEY GENERATOR IF EXISTS t_order_snowflake;
```

Reserved word

DROP, SHARDING, KEY, GENERATOR, FROM

Related links

- [Reserved word](#)

DROP SHARDING ALGORITHM

Description

The DROP SHARDING ALGORITHM syntax is used to drop sharding algorithm for specified database.

Syntax

```
DropShardingAlgorithm ::=  
    'DROP' 'SHARDING' 'ALGORITHM' algorithmName ifExists? ('FROM' databaseName)?  
  
ifExists ::=  
    'IF' 'EXISTS'  
  
algorithmName ::=  
    identifier  
  
databaseName ::=  
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted;
- ifExists clause used for avoid Sharding algorithm not exists error.

Example

- Drop sharding algorithm for specified database

```
DROP SHARDING ALGORITHM t_order_hash_mod FROM sharding_db;
```

- Drop sharding algorithm for current database

```
DROP SHARDING ALGORITHM t_order_hash_mod;
```

- Drop sharding algorithm with ifExists clause

```
DROP SHARDING ALGORITHM IF EXISTS t_order_hash_mod;
```

Reserved word

DROP, SHARDING, ALGORITHM, FROM

Related links

- [Reserved word](#)

CREATE SHARDING TABLE REFERENCE RULE

Description

The CREATE SHARDING TABLE REFERENCE RULE syntax is used to create reference rule for sharding tables

Syntax

```
CreateShardingTableReferenceRule ::=
    'CREATE' 'SHARDING' 'TABLE' 'REFERENCE' 'RULE' ifNotExists?
    referenceRelationshipDefinition (',' referenceRelationshipDefinition)*

ifNotExists ::=
    'IF' 'NOT' 'EXISTS'

referenceRelationshipDefinition ::=
    ruleName '(' tableName (',' tableName)* ')'

ruleName ::=
    identifier
```

```
tableName ::=
    identifier
```

Supplement

- Sharding table reference rule can only be created for sharding tables;
- A sharding table can only be associated with one sharding table reference rule;
- The referenced sharding tables should be sharded in the same storage units and have the same number of sharding nodes. For example `ds_${0..1}.t_order_${0..1}` and `ds_${0..1}.t_order_item_${0..1}`;
- The referenced sharding tables should use consistent sharding algorithms. For example `t_order_{order_id % 2}` and `t_order_item_{order_item_id % 2}`;
- `ifNotExists` clause used for avoid Duplicate sharding table reference rule error.

Example

1.Create a sharding table reference rule

```
-- Before creating a sharding table reference rule, you need to create sharding
table rules t_order, t_order_item
CREATE SHARDING TABLE REFERENCE RULE ref_0 (t_order,t_order_item);
```

2.Create multiple sharding table reference rules

```
-- Before creating sharding table reference rules, you need to create sharding table
rules t_order, t_order_item, t_product, t_product_item
CREATE SHARDING TABLE REFERENCE RULE ref_0 (t_order,t_order_item), ref_1 (t_product,
t_product_item);
```

3.Create a sharding table reference rule with `ifNotExists` clause

```
CREATE SHARDING TABLE REFERENCE RULE IF NOT EXISTS ref_0 (t_order,t_order_item);
```

Reserved word

CREATE, SHARDING, TABLE, REFERENCE, RULE

Related links

- [Reserved word](#)
- [CREATE SHARDING TABLE RULE](#)

ALTER SHARDING TABLE REFERENCE RULE

Description

The ALTER SHARDING TABLE REFERENCE RULE syntax is used to alter sharding table reference rule.

Syntax

```

AlterShardingTableReferenceRule ::=
    'ALTER' 'SHARDING' 'TABLE' 'REFERENCE' 'RULE' referenceRelationshipDefinition (
    ',' referenceRelationshipDefinition)*

referenceRelationshipDefinition ::=
    ruleName '(' tableName (',' tableName)* ')'

ruleName ::=
    identifier

tableName ::=
    identifier

```

Supplement

- A sharding table can only be associated with one sharding table reference rule;
- The referenced sharding tables should be sharded in the same storage units and have the same number of sharding nodes. For example `ds_${0..1}.t_order_${0..1}` and `ds_${0..1}.t_order_item_${0..1}`;
- The referenced sharding tables should use consistent sharding algorithms. For example `t_order_{order_id % 2}` and `t_order_item_{order_item_id % 2}`;

Example

1. Alter a sharding table reference rule

```
ALTER SHARDING TABLE REFERENCE RULE ref_0 (t_order,t_order_item);
```

2. Alter multiple sharding table reference rules

```
ALTER SHARDING TABLE REFERENCE RULE ref_0 (t_order,t_order_item), ref_1 (t_product,
t_product_item);
```

Reserved word

ALTER, SHARDING, TABLE, REFERENCE, RULE

Related links

- [Reserved word](#)
- [CREATE SHARDING TABLE RULE](#)

DROP SHARDING TABLE REFERENCE RULE

Description

The DROP SHARDING TABLE REFERENCE RULE syntax is used to drop specified sharding table reference rule.

Syntax

```
DropShardingTableReferenceRule ::=
  'DROP' 'SHARDING' 'TABLE' 'REFERENCE' 'RULE' ifExists? ruleName (',' ruleName)*

ifExists ::=
  'IF' 'EXISTS'

ruleName ::=
  identifier
```

Supplement

- `ifExists` clause is used for avoid Sharding reference rule not exists error. ### Example

- Drop a specified sharding table reference rule

```
DROP SHARDING TABLE REFERENCE RULE ref_0;
```

- Drop multiple sharding table reference rules

```
DROP SHARDING TABLE REFERENCE RULE ref_0, ref_1;
```

- Drop sharding table reference rule with `ifExists` clause

```
DROP SHARDING TABLE REFERENCE RULE IF EXISTS ref_0;
```

Reserved word

DROP, SHARDING, TABLE, REFERENCE, RULE

Related links

- [Reserved word](#)

Broadcast Table

This chapter describes the syntax of broadcast table.

CREATE BROADCAST TABLE RULE

Description

The `CREATE BROADCAST TABLE RULE` syntax is used to create broadcast table rules for tables that need to be broadcast (broadcast tables)

Syntax

```
CreateBroadcastTableRule ::=
    'CREATE' 'BROADCAST' 'TABLE' 'RULE' ifNotExists? tableName (',' tableName)*

ifNotExists ::=
    'IF' 'NOT' 'EXISTS'
```

```
tableName ::=  
    identifier
```

Supplement

- `tableName` can use an existing table or a table that will be created;
- `ifNotExists` clause is used for avoid Duplicate Broadcast rule error.

Example

Create broadcast table rule

```
-- Add t_province, t_city to broadcast table rules  
CREATE BROADCAST TABLE RULE t_province, t_city;
```

Create broadcast table rule with `ifNotExists` clause

```
CREATE BROADCAST TABLE RULE IF NOT EXISTS t_province, t_city;
```

Reserved word

CREATE, BROADCAST, TABLE, RULE

Related links

- [Reserved word](#)

DROP BROADCAST TABLE RULE

Description

The DROP BROADCAST TABLE RULE syntax is used to drop broadcast table rule for specified broadcast tables

Syntax

```
DropBroadcastTableRule ::=  
    'DROP' 'BROADCAST' 'TABLE' 'RULE' ifExists? tableName (',' tableName)*  
  
ifExists ::=  
    'IF' 'EXISTS'  
  
tableName ::=  
    identifier
```

Supplement

- tableName can use the table of existing broadcast rules;
- ifExists clause is used for avoid Broadcast rule not exists error.

Example

- Drop broadcast table rule for specified broadcast table

```
DROP BROADCAST TABLE RULE t_province, t_city;
```

- Drop broadcast table rule with ifExists clause

```
DROP BROADCAST TABLE RULE IF EXISTS t_province, t_city;
```

Reserved word

DROP, BROADCAST, TABLE, RULE

Related links

- [Reserved word](#)

Single Table

This chapter describes the syntax of single table.

LOAD SINGLE TABLE

Description

The `LOAD SINGLE TABLE` syntax is used to load single table from storage unit.

Syntax

```
loadSingleTable ::=
    'LOAD' 'SINGLE' 'TABLE' tableDefinition

tableDefinition ::=
    tableIdentifier (',' tableIdentifier)*

tableIdentifier ::=
    '*' | '*'.* | storageUnitName '.' | storageUnitName '.*' | storageUnitName
    '.' schemaName '.' | storageUnitName '.' tableName | storageUnitName '.' schemaName
    '.' tableName

storageUnitName ::=
    identifier

schemaName ::=
    identifier

tableName ::=
    identifier
```

Supplement

- support specifying schemaName in PostgreSQL and OpenGauss protocols

Example

- Load specified single table

```
LOAD SINGLE TABLE ds_0.t_single;
```

- Load all single tables in the specified storage unit

```
LOAD SINGLE TABLE ds_0.*;
```

- Load all single tables

```
LOAD SINGLE TABLE *.*;
```

Reserved word

LOAD, SINGLE, TABLE

Related links

- [Reserved word](#)

UNLOAD SINGLE TABLE

Description

The UNLOAD SINGLE TABLE syntax is used to unload single table.

Syntax

```
unloadSingleTable ::=  
    'UNLOAD' 'SINGLE' 'TABLE' tableNames  
  
tableNames ::=  
    tableName (',' tableName)*  
  
tableName ::=  
    identifier
```

Supplement

- Unlike loading, only the table name needs to be specified when unloading a single table

Example

- Unload specified single table

```
UNLOAD SINGLE TABLE t_single;
```

- Load all single tables

```
UNLOAD SINGLE TABLE *;  
-- or  
UNLOAD ALL SINGLE TABLES;
```

Reserved word

UNLOAD, SINGLE, TABLE, ALL, TABLES

Related links

- [Reserved word](#)

SET DEFAULT SINGLE TABLE STORAGE UNIT

Description

The SET DEFAULT SINGLE TABLE STORAGE UNIT syntax is used to set default single table storage unit.

Syntax

```
SetDefaultSingleTableStorageUnit ::=
    'SET' 'DEFAULT' 'SINGLE' 'TABLE' 'STORAGE' 'UNIT' singleTableDefinition

singleTableDefinition ::=
    '=' (storageUnitName | 'RANDOM')

storageUnitName ::=
    identifier
```

Supplement

- STORAGE UNIT needs to use storage unit managed by RDL. The RANDOM keyword stands for random storage.

Example

- Set a default single table storage unit

```
SET DEFAULT SINGLE TABLE STORAGE UNIT = ds_0;
```

- Set the default single table storage unit to random storage

```
SET DEFAULT SINGLE TABLE STORAGE UNIT = RANDOM;
```

Reserved word

SET, DEFAULT, SINGLE, TABLE, STORAGE, UNIT, RANDOM

Related links

- [Reserved word](#)

Readwrite-Splitting

This chapter describes the syntax of readwrite-splitting.

CREATE READWRITE_SPLITTING RULE

Description

The CREATE READWRITE_SPLITTING RULE syntax is used to create a read/write splitting rule.

Syntax

```

CreateReadwriteSplittingRule ::=
    'CREATE' 'READWRITE_SPLITTING' 'RULE' ifNotExists? readwriteSplittingDefinition (
    ',' readwriteSplittingDefinition)*

ifNotExists ::=
    'IF' 'NOT' 'EXISTS'

readwriteSplittingDefinition ::=
    ruleName '(' dataSourceDefinition (',' transactionalReadQueryStrategyDefinition)?
    (',' loadBalancerDefinition)? ')'

dataSourceDefinition ::=
    'WRITE_STORAGE_UNIT' '=' writeStorageUnitName ',' 'READ_STORAGE_UNITS' '('
    storageUnitName (',' storageUnitName)* ')'

transactionalReadQueryStrategyDefinition ::=
    'TRANSACTIONAL_READ_QUERY_STRATEGY' '=' transactionalReadQueryStrategyType

loadBalancerDefinition ::=
    'TYPE' '(' 'NAME' '=' algorithmType (',' propertiesDefinition)? ')'

ruleName ::=
    identifier

writeStorageUnitName ::=

```

```

    identifier

storageUnitName ::=
    identifier

transactionalReadQueryStrategyType ::=
    string

algorithmType ::=
    string

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
    string

value ::=
    literal

```

Note

- transactionalReadQueryStrategyType specifies the routing strategy for read query within a transaction, please refer to [YAML configuration](#);
- algorithmType specifies the load balancing algorithm type, please refer to [Load Balance Algorithm](#);
- Duplicate ruleName will not be created;
- ifNotExists clause used to avoid the Duplicate readwrite_splitting rule error.

Example

Create a read/write splitting rule

```

CREATE READWRITE_SPLITTING RULE ms_group_0 (
    WRITE_STORAGE_UNIT=write_ds,
    READ_STORAGE_UNITS(read_ds_0,read_ds_1),
    TYPE(NAME="random")
);

```

Create read/write splitting rule with the `ifNotExists` clause

- read/write splitting rule

```
CREATE READWRITE_SPLITTING RULE IF NOT EXISTS ms_group_0 (
    WRITE_STORAGE_UNIT=write_ds,
    READ_STORAGE_UNITS(read_ds_0,read_ds_1),
    TYPE(NAME="random")
);
```

Reserved words

CREATE, READWRITE_SPLITTING, RULE, WRITE_STORAGE_UNIT, READ_STORAGE_UNITS , TYPE, NAME, PROPERTIES, TRUE, FALSE

Related links

- [Reserved words](#)
- [Load Balance Algorithm](#)

ALTER READWRITE_SPLITTING RULE

Description

The ALTER READWRITE_SPLITTING RULE syntax is used to alter a readwrite-splitting rule.

Syntax

```
AlterReadwriteSplittingRule ::=
    'ALTER' 'READWRITE_SPLITTING' 'RULE' readwriteSplittingDefinition (','
readwriteSplittingDefinition)*

readwriteSplittingDefinition ::=
    ruleName '(' dataSourceDefinition (',' transactionalReadQueryStrategyDefinition)?
    (',' loadBalancerDefinition)? ')'

dataSourceDefinition ::=
    'WRITE_STORAGE_UNIT' '=' writeStorageUnitName ',' 'READ_STORAGE_UNITS' '('
storageUnitName (',' storageUnitName)* ')'

transactionalReadQueryStrategyDefinition ::=
    'TRANSACTIONAL_READ_QUERY_STRATEGY' '=' transactionalReadQueryStrategyType

loadBalancerDefinition ::=
```

```

    'TYPE' '(' 'NAME' '=' algorithmType (',' propertiesDefinition)? ')'

ruleName ::=
    identifier

writeStorageUnitName ::=
    identifier

storageUnitName ::=
    identifier

transactionalReadQueryStrategyType ::=
    string

algorithmType ::=
    string

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
    string

value ::=
    literal

```

Supplement

- transactionalReadQueryStrategyType specifies the routing strategy for read query within a transaction, please refer to [YAML configuration](#);
- algorithmType specifies the load balancing algorithm type, please refer to [Load Balance Algorithm](#).

Example

Alter a readwrite-splitting rule

```

ALTER READWRITE_SPLITTING RULE ms_group_0 (
    WRITE_STORAGE_UNIT=write_ds,
    READ_STORAGE_UNITS(read_ds_0,read_ds_1),
    TYPE(NAME="random")
);

```

Reserved word

ALTER, READWRITE_SPLITTING, RULE, WRITE_STORAGE_UNIT, READ_STORAGE_UNITS , TYPE, NAME, PROPERTIES, TRUE, FALSE

Related links

- [Reserved word](#)
- [Load Balance Algorithm](#)

DROP READWRITE_SPLITTING RULE

Description

The DROP READWRITE_SPLITTING RULE syntax is used to drop readwrite-splitting rule for specified database

Syntax

```
DropReadwriteSplittingRule ::=
    'DROP' 'READWRITE_SPLITTING' 'RULE' ifExists? ruleName (',' ruleName)* ('FROM'
databaseName)?

ifExists ::=
    'IF' 'EXISTS'

ruleName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted;
- ifExists clause is used for avoid Readwrite-splitting rule not exists error.

Example

- Drop readwrite-splitting rule for specified database

```
DROP READWRITE_SPLITTING RULE ms_group_1 FROM readwrite_splitting_db;
```

- Drop readwrite-splitting rule for current database

```
DROP READWRITE_SPLITTING RULE ms_group_1;
```

- Drop readwrite-splitting rule with ifExists clause

```
DROP READWRITE_SPLITTING RULE IF EXISTS ms_group_1;
```

Reserved word

DROP, READWRITE_SPLITTING, RULE

Related links

- [Reserved word](#)

Encrypt

This chapter describes the syntax of encrypt.

CREATE ENCRYPT RULE

Description

The CREATE ENCRYPT RULE syntax is used to create encrypt rules.

Syntax

```
CreateEncryptRule ::=
    'CREATE' 'ENCRYPT' 'RULE' ifNotExists? encryptDefinition (',' encryptDefinition)*

ifNotExists ::=
    'IF' 'NOT' 'EXISTS'

encryptDefinition ::=
    ruleName '(' 'COLUMNS' '(' columnDefinition (',' columnDefinition)* ')' ') '

columnDefinition ::=
```

```

    '(' 'NAME' '=' columnName ',' 'CIPHER' '=' cipherColumnName (',' 'ASSISTED_QUERY'
    '=' assistedQueryColumnName)? (',' 'LIKE_QUERY' '=' likeQueryColumnName)? ','
    encryptAlgorithmDefinition (',' assistedQueryAlgorithmDefinition)? (','
    likeQueryAlgorithmDefinition)? ')'

encryptAlgorithmDefinition ::=
    'ENCRYPT_ALGORITHM' '(' 'TYPE' '(' 'NAME' '=' algorithmType (','
    propertiesDefinition)? ')'

assistedQueryAlgorithmDefinition ::=
    'ASSISTED_QUERY_ALGORITHM' '(' 'TYPE' '(' 'NAME' '=' algorithmType (','
    propertiesDefinition)? ')'

likeQueryAlgorithmDefinition ::=
    'LIKE_QUERY_ALGORITHM' '(' 'TYPE' '(' 'NAME' '=' algorithmType (','
    propertiesDefinition)? ')'

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

ruleName ::=
    identifier

columnName ::=
    identifier

cipherColumnName ::=
    identifier

assistedQueryColumnName ::=
    identifier

likeQueryColumnName ::=
    identifier

algorithmType ::=
    string

key ::=
    string

value ::=
    literal

```

Supplement

- CIPHER specifies the cipher column, ASSISTED_QUERY specifies the assisted query column, LIKE_QUERY specifies the like query column;
- algorithmType specifies the encryption algorithm type, please refer to [Encryption Algorithm](#);
- Duplicate ruleName will not be created;
- ifNotExists clause used for avoid Duplicate encrypt rule error.

Example

Create an encrypt rule

```
CREATE ENCRYPT RULE t_encrypt (
  COLUMNS(
    (NAME=user_id,CIPHER=user_cipher,ASSISTED_QUERY=assisted_query_user,LIKE_
    QUERY=like_query_user,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value'=
    '123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_ALGORITHM(TYPE(NAME=
    'MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE'))),
    (NAME=order_id,CIPHER =order_cipher,ASSISTED_QUERY=assisted_query_order,LIKE_
    QUERY=like_query_order,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value
    '='123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_
    ALGORITHM(TYPE(NAME='MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE'))
    )),
  t_encrypt_2 (
    COLUMNS(
      (NAME=user_id,CIPHER=user_cipher,ASSISTED_QUERY=assisted_query_user,LIKE_
      QUERY=like_query_user,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value'=
      '123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_ALGORITHM(TYPE(NAME=
      'MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE'))),
      (NAME=order_id, CIPHER=order_cipher,ASSISTED_QUERY=assisted_query_order,LIKE_
      QUERY=like_query_order,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value
      '='123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_
      ALGORITHM(TYPE(NAME='MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE'))
      ));
```

Create an encrypt rule with ifNotExists clause

```
CREATE ENCRYPT RULE IF NOT EXISTS t_encrypt (
  COLUMNS(
    (NAME=user_id,CIPHER=user_cipher,ASSISTED_QUERY=assisted_query_user,LIKE_
    QUERY=like_query_user,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value'=
    '123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_ALGORITHM(TYPE(NAME=
    'MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE'))),
    (NAME=order_id,CIPHER =order_cipher,ASSISTED_QUERY=assisted_query_order,LIKE_
```

```

QUERY=like_query_order,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value'='123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_
ALGORITHM(TYPE(NAME='MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE')))
)),
t_encrypt_2 (
COLUMNS(
(NAME=user_id,CIPHER=user_cipher,ASSISTED_QUERY=assisted_query_user,LIKE_
QUERY=like_query_user,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value'='123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_ALGORITHM(TYPE(NAME='MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE'))),
(NAME=order_id,CIPHER=order_cipher,ASSISTED_QUERY=assisted_query_order,LIKE_
QUERY=like_query_order,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value'='123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_
ALGORITHM(TYPE(NAME='MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE')))
));

```

Reserved words

CREATE, ENCRYPT, RULE, COLUMNS, NAME, CIPHER, ASSISTED_QUERY, LIKE_QUERY, ENCRYPT_ALGORITHM, ASSISTED_QUERY_ALGORITHM, LIKE_QUERY_ALGORITHM, TYPE, TRUE, FALSE

Related links

- [Reserved word](#)
- [Encryption Algorithm](#)

ALTER ENCRYPT RULE

Description

The ALTER ENCRYPT RULE syntax is used to alter encryption rules.

Syntax

```

AlterEncryptRule ::=
    'ALTER' 'ENCRYPT' 'RULE' encryptDefinition (',' encryptDefinition)*

encryptDefinition ::=
    ruleName '(' 'COLUMNS' '(' columnDefinition (',' columnDefinition)* ')' ')'

columnDefinition ::=
    '(' 'NAME' '=' columnName ',' 'CIPHER' '=' cipherColumnName (',' 'ASSISTED_QUERY'

```

```

'=' assistedQueryColumnName)? (',' 'LIKE_QUERY' '=' likeQueryColumnName)? (','
encryptAlgorithmDefinition (',' assistedQueryAlgorithmDefinition)? (','
likeQueryAlgorithmDefinition)? ')'

encryptAlgorithmDefinition ::=
    'ENCRYPT_ALGORITHM' '(' 'TYPE' '(' 'NAME' '=' algorithmType (','
propertiesDefinition)? ')'

assistedQueryAlgorithmDefinition ::=
    'ASSISTED_QUERY_ALGORITHM' '(' 'TYPE' '(' 'NAME' '=' algorithmType (','
propertiesDefinition)? ')'

likeQueryAlgorithmDefinition ::=
    'LIKE_QUERY_ALGORITHM' '(' 'TYPE' '(' 'NAME' '=' algorithmType (','
propertiesDefinition)? ')'

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

ruleName ::=
    identifier

columnName ::=
    identifier

cipherColumnName ::=
    identifier

assistedQueryColumnName ::=
    identifier

likeQueryColumnName ::=
    identifier

algorithmType ::=
    string

key ::=
    string

value ::=
    literal

```

Supplement

- CIPHER specifies the cipher column, ASSISTED_QUERY specifies the assisted query column, LIKE_QUERY specifies the like query column
- algorithmType specifies the encryption algorithm type, please refer to [Encryption Algorithm](#)

Example

- Alter an encrypt rule

```
ALTER ENCRYPT RULE t_encrypt (
  COLUMNS(
    (NAME=user_id,CIPHER=user_cipher,ASSISTED_QUERY=assisted_query_user,LIKE_
    QUERY=like_query_user,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value'=
    '123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_ALGORITHM(TYPE(NAME=
    'MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE'))),
    (NAME=order_id,CIPHER=order_cipher,ASSISTED_QUERY=assisted_query_order,LIKE_
    QUERY=like_query_order,ENCRYPT_ALGORITHM(TYPE(NAME='AES',PROPERTIES('aes-key-value
    '='123456abc', 'digest-algorithm-name'='SHA-1'))),ASSISTED_QUERY_
    ALGORITHM(TYPE(NAME='MD5')),LIKE_QUERY_ALGORITHM(TYPE(NAME='CHAR_DIGEST_LIKE'))
  ));
```

Reserved words

ALTER, ENCRYPT, RULE, COLUMNS, NAME, CIPHER, ASSISTED_QUERY, LIKE_QUERY, ENCRYPT_ALGORITHM, ASSISTED_QUERY_ALGORITHM, LIKE_QUERY_ALGORITHM, TYPE, TRUE, FALSE

Related links

- [Reserved word](#)
- [Encryption Algorithm](#)

DROP ENCRYPT RULE

Description

The DROP ENCRYPT RULE syntax is used to drop an existing encryption rule.

Syntax

```
DropEncryptRule ::=  
    'DROP' 'ENCRYPT' 'RULE' ifExists? ruleName (',' ruleName)*  
  
ifExists ::=  
    'IF' 'EXISTS'  
  
ruleName ::=  
    identifier
```

Supplement

- ifExists clause is used for avoid Encrypt rule not exists error.

Example

- Drop an encrypt rule

```
DROP ENCRYPT RULE t_encrypt, t_encrypt_2;
```

- Drop encrypt with ifExists clause

```
DROP ENCRYPT RULE IF EXISTS t_encrypt, t_encrypt_2;
```

Reserved words

DROP, ENCRYPT, RULE

Related links

- [Reserved word](#)

Mask

This chapter describes the syntax of mask.

CREATE MASK RULE

Description

The CREATE MASK RULE syntax is used to create a mask rule.

Syntax

```

CreateMaskRule ::=
    'CREATE' 'MASK' 'RULE' ifNotExists? maskRuleDefinition (',' maskRuleDefinition)*

ifNotExists ::=
    'IF' 'NOT' 'EXISTS'

maskRuleDefinition ::=
    ruleName '(' 'COLUMNS' '(' columnDefinition (',' columnDefinition)* ')' ')'

columnDefinition ::=
    '(' 'NAME' '=' columnName ',' maskAlgorithmDefinition ')'

maskAlgorithmDefinition ::=
    'TYPE' '(' 'NAME' '=' algorithmType (',' propertiesDefinition)? ')'

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

ruleName ::=
    identifier

columnName ::=
    identifier

algorithmType ::=
    literal

key ::=
    string

value ::=
    literal

```


Note

- `algorithmType` specifies the data masking algorithm type. For more details, please refer to [Data Masking Algorithm](#);
- Duplicate `ruleName` will not be created;
- `ifNotExists` clause is used for avoid Duplicate mask rule error.

Example

Create a mask rule

```
CREATE MASK RULE t_mask (  
  COLUMNS(  
    (NAME=phone_number, TYPE(NAME='MASK_FROM_X_TO_Y', PROPERTIES("from-x"=1, "to-y"=2, "replace-char"="x"))),  
    (NAME=address, TYPE(NAME='MD5'))  
  );
```

Create mask rule with `ifNotExists` clause

```
CREATE MASK RULE IF NOT EXISTS t_mask (  
  COLUMNS(  
    (NAME=phone_number, TYPE(NAME='MASK_FROM_X_TO_Y', PROPERTIES("from-x"=1, "to-y"=2, "replace-char"="x"))),  
    (NAME=address, TYPE(NAME='MD5'))  
  );
```

Reserved words

CREATE, MASK, RULE, COLUMNS, NAME, TYPE

Related links

- [Reserved word](#)
- [Data Masking Algorithm](#)

ALTER MASK RULE

Description

The ALTER MASK RULE syntax is used to create a mask rule.

Syntax

```
AlterMaskRule ::=
    'ALTER' 'MASK' 'RULE' maskRuleDefinition (',' maskRuleDefinition)*

maskRuleDefinition ::=
    ruleName '(' 'COLUMNS' '(' columnDefinition (',' columnDefinition)* ')' ')'

columnDefinition ::=
    '(' 'NAME' '=' columnName ',' maskAlgorithmDefinition ')'

maskAlgorithmDefinition ::=
    'TYPE' '(' 'NAME' '=' algorithmType (',' propertiesDefinition)? ')'

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

ruleName ::=
    identifier

columnName ::=
    identifier

algorithmType ::=
    literal

key ::=
    string

value ::=
    literal
```

Supplement

- `algorithmType` specifies the data masking algorithm type, please refer to [Data Masking Algorithm](#).

Example

Alter a mask rule

```
ALTER MASK RULE t_mask (
  COLUMNS(
    (NAME=phone_number, TYPE(NAME='MASK_FROM_X_TO_Y', PROPERTIES("from-x"=1, "to-y"=2,
      "replace-char"="*"))),
    (NAME=address, TYPE(NAME='MD5'))
  ));
```

Reserved words

ALTER, MASK, RULE, COLUMNS, NAME, TYPE

Related links

- [Reserved word](#)
- [Data Masking Algorithm](#)

DROP MASK RULE

Description

The DROP MASK RULE syntax is used to drop existing mask rule.

Syntax

```
DropMaskRule ::=
  'DROP' 'MASK' 'RULE' ifExists? ruleName (',' ruleName)*

ifExists ::=
  'IF' 'EXISTS'

ruleName ::=
  identifier
```

Supplement

- `ifExists` clause used for avoid `Mask rule not exists` error.

Example

- Drop mask rule

```
DROP MASK RULE t_mask, t_mask_1;
```

- Drop mask rule with `ifExists` clause

```
DROP MASK RULE IF EXISTS t_mask, t_mask_1;
```

Reserved words

DROP, MASK, RULE

Related links

- [Reserved word](#)

Shadow

This chapter describes the syntax of shadow.

CREATE SHADOW RULE

Description

The `CREATE SHADOW RULE` syntax is used to create a shadow rule.

Syntax

```
CreateShadowRule ::=
    'CREATE' 'SHADOW' 'RULE' ifNotExists? shadowRuleDefinition (',' shadowRuleDefinition)*

ifNotExists ::=
    'IF' 'NOT' 'EXISTS'

shadowRuleDefinition ::=
    ruleName '(' storageUnitMapping shadowTableRule (',' shadowTableRule)* ')'
```

```

storageUnitMapping ::=
  'SOURCE' '=' storageUnitName ',' 'SHADOW' '=' storageUnitName

shadowTableRule ::=
  tableName '(' shadowAlgorithm ')'

shadowAlgorithm ::=
  'TYPE' '(' 'NAME' '=' algorithmType ',' propertiesDefinition ')'

ruleName ::=
  identifier

storageUnitName ::=
  identifier

tableName ::=
  identifier

algorithmName ::=
  identifier

algorithmType ::=
  string

propertiesDefinition ::=
  'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
  string

value ::=
  literal

```

Supplement

- Duplicate ruleName cannot be created;
- storageUnitMapping specifies the mapping relationship between the source database and the shadow library. You need to use the storage unit managed by RDL, please refer to [STORAGE UNIT](#);
- shadowAlgorithm can act on multiple shadowTableRule at the same time;
- If algorithmName is not specified, it will be automatically generated according to ruleName, tableName and algorithmType;
- algorithmType currently supports VALUE_MATCH, REGEX_MATCH and SQL_HINT;

- ifNotExists clause is used for avoid Duplicate shadow rule error.

Example

- Create a shadow rule

```
CREATE SHADOW RULE shadow_rule(
  SOURCE=demo_ds,
  SHADOW=demo_ds_shadow,
  t_order(TYPE(NAME="SQL_HINT")),
  t_order_item(TYPE(NAME="VALUE_MATCH", PROPERTIES("operation"="insert","column"=
"user_id", "value"='1'))))
);
```

- Create a shadow rule with ifNotExists clause

```
CREATE SHADOW RULE IF NOT EXISTS shadow_rule(
  SOURCE=demo_ds,
  SHADOW=demo_ds_shadow,
  t_order(TYPE(NAME="SQL_HINT")),
  t_order_item(TYPE(NAME="VALUE_MATCH", PROPERTIES("operation"="insert","column"=
"user_id", "value"='1'))))
);
```

Reserved word

CREATE, SHADOW, RULE, SOURCE, SHADOW, TYPE, NAME, PROPERTIES

Related links

- [Reserved word](#)
- [STORAGE UNIT](#)

ALTER SHADOW RULE

Description

The ALTER SHADOW RULE syntax is used to alter shadow rule.

Syntax

```
AlterShadowRule ::=
    'ALTER' 'SHADOW' 'RULE' shadowRuleDefinition (',' shadowRuleDefinition)*

shadowRuleDefinition ::=
    ruleName '(' storageUnitMapping shadowTableRule (',' shadowTableRule)* ')'

storageUnitMapping ::=
    'SOURCE' '=' storageUnitName ',' 'SHADOW' '=' storageUnitName

shadowTableRule ::=
    tableName '(' shadowAlgorithm ')'

shadowAlgorithm ::=
    'TYPE' '(' 'NAME' '=' shadowAlgorithmType ',' propertiesDefinition ')'

ruleName ::=
    identifier

storageUnitName ::=
    identifier

tableName ::=
    identifier

algorithmName ::=
    identifier

shadowAlgorithmType ::=
    string

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
    string

value ::=
    literal
```

Supplement

- `storageUnitMapping` specifies the mapping relationship between the source database and the shadow library. You need to use the storage unit managed by RDL, please refer to [STORAGE UNIT](#);
- `shadowAlgorithm` can act on multiple `shadowTableRule` at the same time;
- If `algorithmName` is not specified, it will be automatically generated according to `ruleName`, `tableName` and `shadowAlgorithmType`;
- `shadowAlgorithmType` currently supports `VALUE_MATCH`, `REGEX_MATCH` and `SQL_HINT`.

Example

- Create a shadow rule

```
ALTER SHADOW RULE shadow_rule(
  SOURCE=demo_ds,
  SHADOW=demo_ds_shadow,
  t_order(TYPE(NAME="SQL_HINT")),
  t_order_item(TYPE(NAME="VALUE_MATCH", PROPERTIES("operation"="insert","column"=
"user_id", "value"='1'))))
);
```

Reserved word

ALTER, SHADOW, RULE, SOURCE, SHADOW, TYPE, NAME, PROPERTIES

Related links

- [Reserved word](#)
- [STORAGE UNIT](#)

DROP SHADOW RULE

Description

The `DROP SHADOW RULE` syntax is used to drop shadow rule for specified database

Syntax

```
DropShadowRule ::=
    'DROP' 'SHADOW' 'RULE' ifExists? ruleName ('FROM' databaseName)?

ifExists ::=
    'IF' 'EXISTS'

ruleName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted;
- ifExists clause is used for avoid Shadow rule not exists error.

Example

- Drop shadow rule for specified database

```
DROP SHADOW RULE shadow_rule FROM shadow_db;
```

- Drop shadow rule for current database

```
DROP SHADOW RULE shadow_rule;
```

- Drop shadow rule with ifExists clause

```
DROP SHADOW RULE IF EXISTS shadow_rule;
```

Reserved word

DROP, SHADOW, RULE, FROM

Related links

- [Reserved word](#)

CREATE DEFAULT SHADOW ALGORITHM

Description

The CREATE DEFAULT SHADOW ALGORITHM syntax is used to create a default shadow algorithm.

Syntax

```

CreateDefaultShadowAlgorithm ::=
    'CREATE' 'DEFAULT' 'SHADOW' 'ALGORITHM' ifNotExists? shadowAlgorithm

ifNotExists ::=
    'IF' 'NOT' 'EXISTS'

shadowAlgorithm ::=
    'TYPE' '(' 'NAME' '=' algorithmType ',' propertiesDefinition ')'

algorithmType ::=
    string

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
    string

value ::=
    literal
  
```

Supplement

- algorithmType currently supports VALUE_MATCH, REGEX_MATCH and SQL_HINT;
- ifNotExists clause is used for avoid Duplicate default shadow algorithm error.

Example

- Create default shadow algorithm

```
CREATE DEFAULT SHADOW ALGORITHM TYPE(NAME="SQL_HINT");
```

- Create default shadow algorithm with ifNotExist clause

```
CREATE DEFAULT SHADOW ALGORITHM IF NOT EXISTS TYPE(NAME="SQL_HINT");
```

Reserved word

CREATE, DEFAULT, SHADOW, ALGORITHM, TYPE, NAME, PROPERTIES

Related links

- [Reserved word](#)

ALTER DEFAULT SHADOW ALGORITHM

Description

The ALTER DEFAULT SHADOW ALGORITHM syntax is used to alter a default shadow algorithm.

Syntax

```
AlterDefaultShadowAlgorithm ::=
    'ALTER' 'DEFAULT' 'SHADOW' 'ALGORITHM' shadowAlgorithm

shadowAlgorithm ::=
    'TYPE' '(' 'NAME' '=' algorithmType ',' propertiesDefinition ')'

algorithmType ::=
    string

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
    string

value ::=
    literal
```

Supplement

- `algorithmType` currently supports `VALUE_MATCH`, `REGEX_MATCH` and `SQL_HINT`.

Example

- Alter default shadow algorithm

```
ALTER DEFAULT SHADOW ALGORITHM TYPE(NAME="SQL_HINT");
```

Reserved word

ALTER, DEFAULT, SHADOW, ALGORITHM, TYPE, NAME, PROPERTIES

Related links

- [Reserved word](#)

DROP DEFAULT SHADOW ALGORITHM

Description

The `DROP DEFAULT SHADOW ALGORITHM` syntax is used to drop default shadow algorithm for specified database

Syntax

```
DropDefaultShadowAlgorithm ::=  
    'DROP' 'DEFAULT' 'SHADOW' 'ALGORITHM' ifExists? ('FROM' databaseName)?  
  
ifExists ::=  
    'IF' 'EXISTS'  
  
databaseName ::=  
    identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted;
- `ifExists` clause used for avoid Default shadow algorithm not exists error.

Example

- Drop default shadow algorithm for specified database

```
DROP DEFAULT SHADOW ALGORITHM FROM shadow_db;
```

- Drop default shadow algorithm for current database

```
DROP DEFAULT SHADOW ALGORITHM;
```

- Drop default shadow algorithm with `ifExists` clause

```
DROP DEFAULT SHADOW ALGORITHM IF EXISTS;
```

Reserved word

DROP, DEFAULT, SHADOW, ALGORITHM, FROM

Related links

- [Reserved word](#)

DROP SHADOW ALGORITHM

Description

The DROP SHADOW ALGORITHM syntax is used to drop shadow algorithm for specified database

Syntax

```
DropShadowAlgorithm ::=
  'DROP' 'SHADOW' 'ALGORITHM' ifExists? algorithmName (',' algorithmName)* ('FROM'
  databaseName)?

ifExists ::=
  'IF' 'EXISTS'
```

```
algorithmName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted;
- ifExists clause is used for avoid shadow algorithm not exists error.

Example

- Drop mutiple shadow algorithm for specified database

```
DROP SHADOW ALGORITHM shadow_rule_t_order_sql_hint_0, shadow_rule_t_order_item_sql_
hint_0 FROM shadow_db;
```

- Drop single shadow algorithm for current database

```
DROP SHADOW ALGORITHM shadow_rule_t_order_sql_hint_0;
```

- Drop shadow algorithm with ifExists clause

```
DROP SHADOW ALGORITHM IF EXISTS shadow_rule_t_order_sql_hint_0;
```

Reserved word

DROP, SHADOW, ALGORITHM, FROM

Related links

- [Reserved word](#)

RQL Syntax

RQL (Resource & Rule Query Language) responsible for resources/rules query.

Storage Unit Query

This chapter describes the syntax of storage unit query.

SHOW STORAGE UNITS

Description

The `SHOW STORAGE UNITS` syntax is used to query the storage units that have been added to the specified database.

Syntax

```
ShowStorageUnit ::=  
  'SHOW' 'STORAGE' 'UNITS' ('FROM' databaseName)? showLike?  
  
databaseName ::=  
  identifier  
  
showLike ::=  
  'LIKE' likePattern  
  
likePattern ::=  
  string
```

Supplement

- When `databaseName` is not specified, the default is the currently used `DATABASE`; if `DATABASE` is not used, it will prompt `No database selected`.

Return Value Description

Column	Description
name	Storage unit name
type	Storage unit type
host	Storage unit host
port	Storage unit port
db	Database name
connection_timeout_milliseconds	connection timeout (milliseconds)
idle_timeout_milliseconds	idle timeout (milliseconds)
max_lifetime_milliseconds	max lifetime (milliseconds)
max_pool_size	max pool size
min_pool_size	min pool size
read_only	read-only flag
other_attributes	other attributes

Example

- Query storage units from current database

```
mysql> SHOW STORAGE UNITS;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| name | type | host | port | db | connection_timeout_milliseconds | idle_timeout_milliseconds | max_lifetime_milliseconds | max_pool_size | min_pool_size | read_only | other_attributes |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ds_1 | MySQL | 127.0.0.1 | 3306 | db1 | 30000 | 60000 | 2100000 | 50 | 1 | false | {
"healthCheckProperties":{}, "initializationFailTimeout":1, "validationTimeout":5000,
"keepaliveTime":0, "leakDetectionThreshold":0, "registerMbeans":false,
"allowPoolSuspension":false, "autoCommit":true, "isolateInternalQueries":false} |
| ds_0 | MySQL | 127.0.0.1 | 3306 | db0 | 30000 | 60000 | 2100000 | 50 | 1 | false | {
"healthCheckProperties":{}, "initializationFailTimeout":1, "validationTimeout":5000,
"keepaliveTime":0, "leakDetectionThreshold":0, "registerMbeans":false,
"allowPoolSuspension":false, "autoCommit":true, "isolateInternalQueries":false} |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```



```

-----+-----+-----+-----+
-----+-----+-----+-----+
-----+-----+-----+-----+
-----+-----+-----+-----+
2 rows in set (0.01 sec)

```

- Query storage units from specified database

```

mysql> SHOW STORAGE UNITS FROM sharding_db;
+-----+-----+-----+-----+-----+-----+-----+-----+
| name | type | host | port | db | connection_timeout_milliseconds | idle_timeout_milliseconds | max_lifetime_milliseconds | max_pool_size | min_pool_size | read_only | other_attributes |
+-----+-----+-----+-----+-----+-----+-----+-----+
| ds_1 | MySQL | 127.0.0.1 | 3306 | db1 | 30000 | 60000 | 2100000 | 50 | 1 | false | {
"healthCheckProperties":{}, "initializationFailTimeout":1, "validationTimeout":5000,
"keepaliveTime":0, "leakDetectionThreshold":0, "registerMbeans":false,
"allowPoolSuspension":false, "autoCommit":true, "isolateInternalQueries":false} |
| ds_0 | MySQL | 127.0.0.1 | 3306 | db0 | 30000 | 60000 | 2100000 | 50 | 1 | false | {
"healthCheckProperties":{}, "initializationFailTimeout":1, "validationTimeout":5000,
"keepaliveTime":0, "leakDetectionThreshold":0, "registerMbeans":false,
"allowPoolSuspension":false, "autoCommit":true, "isolateInternalQueries":false} |
+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

```

- Query storage units with like clause

```

mysql> SHOW STORAGE UNITS LIKE '%_0';
+-----+-----+-----+-----+-----+-----+-----+-----+
| name | type | host | port | db | connection_timeout_milliseconds | idle_timeout_milliseconds | max_lifetime_milliseconds | max_pool_size | min_pool_size | read_only | other_attributes |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

```

timeout_milliseconds | max_lifetime_milliseconds | max_pool_size | min_pool_size |
read_only | other_attributes
|
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
| ds_0 | MySQL | 127.0.0.1 | 3306 | db0 | 30000 | 60000 |
| 2100000 | 50 | 1 | false | {
"healthCheckProperties":{}, "initializationFailTimeout":1, "validationTimeout":5000,
"keepaliveTime":0, "leakDetectionThreshold":0, "registerMbeans":false,
"allowPoolSuspension":false, "autoCommit":true, "isolateInternalQueries":false} |
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
1 rows in set (0.01 sec)

```

Reserved word

SHOW, STORAGE, UNITS, FROM, LIKE

Related links

- [Reserved word](#)

Rule Query

This chapter describes the syntax of rule query.

Sharding

This chapter describes the syntax of sharding.

SHOW SHARDING TABLE RULE

Description

The `SHOW SHARDING TABLE RULE` syntax is used to query the sharding table rule in the specified database.

Syntax

```
ShowShardingTableRule ::=
    'SHOW' 'SHARDING' 'TABLE' ('RULE' tableName | 'RULES') ('FROM' databaseName)?

tableName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
table	Logical table name
actual_data_nodes	Actual data node
actual_data_sources	Actual data source (Displayed when creating rules by RDL)
database_strategy_type	Database sharding strategy type
database_sharding_column	Database sharding column
database_sharding_algorithm_type	Database sharding algorithm type
database_sharding_algorithm_props	Database sharding algorithm properties
table_strategy_type	Table sharding strategy type
table_sharding_column	Table sharding column
table_sharding_algorithm_type	Table sharding algorithm type
table_sharding_algorithm_props	Table sharding algorithm properties
key_generate_column	Sharding key generator column
key_generator_type	Sharding key generator type
key_generator_props	Sharding key generator properties

Example

- Query the sharding table rules of the specified logical database

```
SHOW SHARDING TABLE RULES FROM sharding_db;
```

```
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
| table          | actual_data_nodes | actual_data_sources | database_strategy_type |
database_sharding_column | database_sharding_algorithm_type | database_sharding_
algorithm_props | table_strategy_type | table_sharding_column | table_sharding_
algorithm_type | table_sharding_algorithm_props | key_generate_column | key_
generator_type | key_generator_props |
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
| t_order        |                  | ds_0,ds_1          |                  |
|                |                  |                    |                  | mod
| order_id       | mod             |                    | sharding-count=4
|                |                  |                    |                  |
| t_order_item   |                  | ds_0,ds_1          |                  |
|                |                  |                    |                  | mod
| order_id       | mod             |                    | sharding-count=4
|                |                  |                    |                  |
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
2 rows in set (0.12 sec)
```

- Query the sharding table rules of the current logic database

```
SHOW SHARDING TABLE RULES;
```

```
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
| table          | actual_data_nodes | actual_data_sources | database_strategy_type |
database_sharding_column | database_sharding_algorithm_type | database_sharding_
algorithm_props | table_strategy_type | table_sharding_column | table_sharding_
algorithm_type | table_sharding_algorithm_props | key_generate_column | key_
generator_type | key_generator_props |
+-----+-----+-----+-----+
+-----+-----+-----+-----+
```

```

+-----+-----+-----+
+-----+-----+-----+
+-----+
| t_order      |          | ds_0,ds_1      |          |
|              |          |                |          |
| order_id     | mod      |                | sharding-count=4 |
|              |          |                |                |
| t_order_item |          | ds_0,ds_1      |          |
|              |          |                |          |
| order_id     | mod      |                | sharding-count=4 |
|              |          |                |                |
+-----+-----+-----+
+-----+-----+-----+
+-----+-----+-----+
+-----+-----+-----+
+-----+
2 rows in set (0.12 sec)

```

- Query the specified sharding table rule

```
SHOW SHARDING TABLE RULE t_order;
```

```

+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+
| table      | actual_data_nodes | actual_data_sources | database_strategy_type |
database_sharding_column | database_sharding_algorithm_type | database_sharding_
algorithm_props | table_strategy_type | table_sharding_column | table_sharding_
algorithm_type | table_sharding_algorithm_props | key_generate_column | key_
generator_type | key_generator_props |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+
| t_order      |          | ds_0,ds_1      |          |
|              |          |                |          |
| order_id     | mod      |                | sharding-count=4 |
|              |          |                |                |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+
1 row in set (0.12 sec)

```

Reserved word

SHOW, SHARDING, TABLE, RULE, FROM

Related links

- [Reserved word](#)

SHOW SHARDING ALGORITHMS

Description

The SHOW SHARDING ALGORITHMS syntax is used to query the sharding algorithms in the specified database.

Syntax

```
ShowShardingAlgorithms ::=  
    'SHOW' 'SHARDING' 'ALGORITHMS' ('FROM' databaseName)?  
  
databaseName ::=  
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
name	Sharding algorithm name
type	Sharding algorithm type
props	Sharding algorithm properties

Example

- Query the sharding table algorithms of the specified logical database

```
SHOW SHARDING ALGORITHMS FROM sharding_db;
```

```
mysql> SHOW SHARDING ALGORITHMS FROM sharding_db;
+-----+
-----+
| name                | type   | props                                |
+-----+-----+-----+
| t_order_inline      | INLINE | algorithm-expression=t_order_${order_id % 2}
|
| t_order_item_inline | INLINE | algorithm-expression=t_order_item_${order_id %
2}  |
+-----+-----+-----+
-----+
2 rows in set (0.01 sec)
```

- Query the sharding table algorithms of the current logical database

```
SHOW SHARDING ALGORITHMS;
```

```
mysql> SHOW SHARDING ALGORITHMS;
+-----+
-----+
| name                | type   | props                                |
+-----+-----+-----+
| t_order_inline      | INLINE | algorithm-expression=t_order_${order_id % 2}
|
| t_order_item_inline | INLINE | algorithm-expression=t_order_item_${order_id %
2}  |
+-----+-----+-----+
-----+
2 rows in set (0.01 sec)
```

Reserved word

SHOW, SHARDING, ALGORITHMS, FROM

Related links

- [Reserved word](#)

SHOW UNUSED SHARDING ALGORITHMS

Description

The SHOW UNUSED SHARDING ALGORITHMS syntax is used to query the unused sharding algorithms in the specified database.

Syntax

```
ShowShardingAlgorithms::=  
  'SHOW' 'UNUSED' 'SHARDING' 'ALGORITHMS' ('FROM' databaseName)?  
  
databaseName ::=  
  identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
name	Sharding algorithm name
type	Sharding algorithm type
props	Sharding algorithm properties

Example

- Query the unused sharding table algorithms of the specified logical database

```
SHOW UNUSED SHARDING ALGORITHMS;
```

```
mysql> SHOW UNUSED SHARDING ALGORITHMS;
```

```
+-----+-----+-----+
| name      | type  | props                                     |
+-----+-----+-----+
| t1_inline | INLINE | algorithm-expression=t_order_${order_id % 2} |
+-----+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

SHOW, UNUSED, SHARDING, ALGORITHMS, FROM

Related links

- [Reserved word](#)

SHOW DEFAULT SHARDING STRATEGY

Description

The SHOW DEFAULT SHARDING STRATEGY syntax is used to query default sharding strategy in specified database.

Syntax

```
ShowDefaultShardingStrategy::=
  'SHOW' 'DEFAULT' 'SHARDING' 'STRATEGY' ('FROM' databaseName)?

databaseName ::=
  identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
<code>name</code>	Sharding strategy scope
<code>type</code>	Sharding strategy type
<code>sharding_column</code>	Sharding column
<code>sharding_algorithm_name</code>	Sharding algorithm name
<code>sharding_algorithm_type</code>	Sharding algorithm type
<code>sharding_algorithm_props</code>	Sharding algorithm properties

Example

- Query default sharding strategy in specified database.

```
SHOW DEFAULT SHARDING STRATEGY FROM sharding_db;
```

```
mysql> SHOW DEFAULT SHARDING STRATEGY FROM sharding_db;
+-----+-----+-----+-----+-----+
| name      | type      | sharding_column | sharding_algorithm_name | sharding_
algorithm_type | sharding_algorithm_props |
+-----+-----+-----+-----+-----+
| TABLE    | STANDARD  | order_id        | table_inline            | inline
| {algorithm-expression=t_order_item_${order_id % 2}} |
| DATABASE  | STANDARD  | order_id        | table_inline            | inline
| {algorithm-expression=t_order_item_${order_id % 2}} |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

- Query default sharding strategy in current database.

```
SHOW DEFAULT SHARDING STRATEGY;
```

```
mysql> SHOW DEFAULT SHARDING STRATEGY;
+-----+-----+-----+-----+-----+
| name      | type      | sharding_column | sharding_algorithm_name | sharding_
```

```

algorithm_type | sharding_algorithm_props |
+-----+-----+-----+-----+-----+
-----+-----+
| TABLE      | STANDARD | order_id      | table_inline      | inline
| {algorithm-expression=t_order_item_${order_id % 2}} |
| DATABASE    | STANDARD | order_id      | table_inline      | inline
| {algorithm-expression=t_order_item_${order_id % 2}} |
+-----+-----+-----+-----+-----+
-----+-----+
2 rows in set (0.00 sec)

```

Reserved word

SHOW, DEFAULT, SHARDING, STRATEGY, FROM

Related links

- [Reserved word](#)

SHOW SHARDING KEY GENERATORS

Description

SHOW SHARDING KEY GENERATORS syntax is used to query sharding key generators in specified database.

Syntax

```

ShowShardingKeyGenerators ::=
    'SHOW' 'SHARDING' 'KEY' 'GENERATORS' ('FROM' databaseName)?

databaseName ::=
    identifier

```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

column	Description
name	Sharding key generator name
type	Sharding key generator type
props	Sharding key generator properties

Example

- Query the sharding key generators of the specified logical database

```
SHOW SHARDING KEY GENERATORS FROM sharding_db;
```

```
mysql> SHOW SHARDING KEY GENERATORS FROM sharding_db;
+-----+-----+-----+
| name                | type    | props |
+-----+-----+-----+
| snowflake_key_generator | snowflake |      |
+-----+-----+-----+
1 row in set (0.00 sec)
```

- Query the sharding key generators of the current logical database

```
SHOW SHARDING KEY GENERATORS;
```

```
mysql> SHOW SHARDING KEY GENERATORS;
+-----+-----+-----+
| name                | type    | props |
+-----+-----+-----+
| snowflake_key_generator | snowflake |      |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Reserved word

SHOW, SHARDING, KEY, GENERATORS, FROM

Related links

- [Reserved word](#)

SHOW UNUSED SHARDING KEY GENERATORS

Description

SHOW UNUSED SHARDING KEY GENERATORS syntax is used to query sharding key generators that are not used in specified database.

Syntax

```
ShowUnusedShardingKeyGenerators ::=  
    'SHOW' 'UNUSED' 'SHARDING' 'KEY' 'GENERATOR' ('FROM' databaseName)?  
  
databaseName ::=  
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

column	Description
name	Sharding key generator name
type	Sharding key generator type
props	Sharding key generator properties

Example

- Query sharding key generators that are not used in the specified logical database

```
SHOW UNUSED SHARDING KEY GENERATORS FROM sharding_db;
```

```
mysql> SHOW UNUSED SHARDING KEY GENERATORS FROM sharding_db;
+-----+-----+-----+
| name                | type      | props |
+-----+-----+-----+
| snowflake_key_generator | snowflake |       |
+-----+-----+-----+
1 row in set (0.00 sec)
```

- Query sharding key generators that are not used in the current logical database

```
SHOW UNUSED SHARDING KEY GENERATORS;
```

```
mysql> SHOW UNUSED SHARDING KEY GENERATORS;
+-----+-----+-----+
| name                | type      | props |
+-----+-----+-----+
| snowflake_key_generator | snowflake |       |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Reserved word

SHOW, UNUSED, SHARDING, KEY, GENERATORS, FROM

Related links

- [Reserved word](#)

SHOW SHARDING AUDITORS

Description

SHOW SHARDING AUDITORS syntax is used to query sharding auditors in specified database.

Syntax

```
ShowShardingAuditors ::=
    'SHOW' 'SHARDING' 'AUDITORS' ('FROM' databaseName)?

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

column	Description
name	Sharding auditor name
type	Sharding auditor algorithm type
props	Sharding auditor algorithm properties

Example

- Query sharding auditors for the specified logical database

```
SHOW SHARDING AUDITORS FROM sharding_db;
```

```
mysql> SHOW SHARDING AUDITORS FROM sharding_db;
+-----+-----+-----+
| name                | type                | props |
+-----+-----+-----+
| sharding_key_required_auditor | dml_sharding_conditions |      |
+-----+-----+-----+
1 row in set (0.01 sec)
```

- Query sharding auditors for the current logical database

```
SHOW SHARDING AUDITORS;
```

```
mysql> SHOW SHARDING AUDITORS;
+-----+-----+-----+
| name                | type                | props |
+-----+-----+-----+
| sharding_key_required_auditor | dml_sharding_conditions |      |
+-----+-----+-----+
```

```
+-----+-----+-----+
1 row in set (0.00 sec)
```

Reserved word

SHOW, SHARDING, AUDITORS, FROM

Related links

- [Reserved word](#)

SHOW UNUSED SHARDING AUDITORS

Description

SHOW SHARDING AUDITORS syntax is used to query sharding auditors that are not used in specified database.

Syntax

```
ShowUnusedShardingAuditors ::=
    'SHOW' 'UNUSED' 'SHARDING' 'AUDITOR' ('FROM' databaseName)?

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

column	Description
name	Sharding auditor name
type	Sharding auditor algorithm type
props	Sharding auditor algorithm properties

Example

- Query sharding auditors that are not used in the specified logical database

```
SHOW UNUSED SHARDING AUDITORS FROM sharding_db;
```

```
mysql> SHOW UNUSED SHARDING AUDITORS FROM sharding_db;
+-----+-----+-----+
| name                | type                | props |
+-----+-----+-----+
| sharding_key_required_auditor | dml_sharding_conditions |      |
+-----+-----+-----+
1 row in set (0.01 sec)
```

- Query sharding auditors are not used in the current logical database

```
SHOW UNUSED SHARDING AUDITORS;
```

```
mysql> SHOW UNUSED SHARDING AUDITORS;
+-----+-----+-----+
| name                | type                | props |
+-----+-----+-----+
| sharding_key_required_auditor | dml_sharding_conditions |      |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Reserved word

SHOW, UNUSED, SHARDING, AUDITORS, FROM

Related links

- [Reserved word](#)

SHOW SHARDING TABLE NODES

Description

SHOW SHARDING TABLE NODES syntax is used to query sharding table nodes in specified database.

Syntax

```
ShowShardingTableNode ::=
    'SHOW' 'SHARDING' 'TABLE' 'NODES' tableName? ('FROM' databaseName)?

tableName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Columns	Descriptions
name	Sharding rule name
nodes	Sharding nodes

Example

- Query sharding table nodes for specified table in the specified logical database

```
SHOW SHARDING TABLE NODES t_order_item FROM sharding_db;
```

```
mysql> SHOW SHARDING TABLE NODES t_order_item FROM sharding_db;
+-----+-----+
| name          | nodes                                                                 |
+-----+-----+
| t_order_item | resource_0.t_order_item_0, resource_0.t_order_item_1, resource_1.t_order_item_0, resource_1.t_order_item_1 |
+-----+-----+
1 row in set (0.00 sec)
```

- Query sharding table nodes for specified table in the current logical database

```
SHOW SHARDING TABLE NODES t_order_item;
```

```
mysql> SHOW SHARDING TABLE NODES t_order_item;
+-----+
| name          | nodes
+-----+
| t_order_item | resource_0.t_order_item_0, resource_0.t_order_item_1, resource_1.t_order_item_0, resource_1.t_order_item_1 |
+-----+
1 row in set (0.00 sec)
```

- Query sharding table nodes for all tables in the specified logical database

```
SHOW SHARDING TABLE NODES FROM sharding_db;
```

```
mysql> SHOW SHARDING TABLE NODES FROM sharding_db;
+-----+
| name          | nodes
+-----+
| t_order_item | resource_0.t_order_item_0, resource_0.t_order_item_1, resource_1.t_order_item_0, resource_1.t_order_item_1 |
+-----+
1 row in set (0.00 sec)
```

- Query sharding table nodes for all tables in the current logical database

```
SHOW SHARDING TABLE NODES;
```

```
mysql> SHOW SHARDING TABLE NODES;
+-----+
| name          | nodes
+-----+
| t_order_item | resource_0.t_order_item_0, resource_0.t_order_item_1, resource_1.t_order_item_0, resource_1.t_order_item_1 |
+-----+
1 row in set (0.00 sec)
```

Reserved word

SHOW, SHARDING, TABLE, NODES, FROM

Related links

- [Reserved word](#)

SHOW SHARDING TABLE RULES USED ALGORITHM

Description

SHOW SHARDING TABLE RULES USED ALGORITHM syntax is used to query sharding rules used specified sharding algorithm in specified logical database

Syntax

```
ShowShardingTableRulesUsedAlgorithm::=
  'SHOW' 'SHARDING' 'TABLE' 'RULES' 'USED' 'ALGORITHM' algorithmName ('FROM'
  databaseName)?

algorithmName ::=
  identifier

databaseName ::=
  identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Columns	Descriptions
type	Sharding rule type
name	Sharding rule name

Example

- Query sharding table rules for the specified sharding algorithm in specified logical database

```
SHOW SHARDING TABLE RULES USED ALGORITHM table_inline FROM sharding_db;
```

```
mysql> SHOW SHARDING TABLE RULES USED ALGORITHM table_inline FROM sharding_db;
+-----+-----+
| type | name      |
+-----+-----+
| table | t_order_item |
+-----+-----+
1 row in set (0.00 sec)
```

- Query sharding table rules for specified sharding algorithm in the current logical database

```
SHOW SHARDING TABLE RULES USED ALGORITHM table_inline;
```

```
mysql> SHOW SHARDING TABLE RULES USED ALGORITHM table_inline;
+-----+-----+
| type | name      |
+-----+-----+
| table | t_order_item |
+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

SHOW, SHARDING, TABLE, RULES, USED, ALGORITHM, FROM

Related links

- [Reserved word](#)

SHOW SHARDING TABLE RULES USED KEY GENERATOR

Description

SHOW SHARDING TABLE RULES USED KEY GENERATOR syntax is used to query sharding rules used specified sharding key generator in specified logical database

Syntax

```
ShowShardingTableRulesUsedKeyGenerator ::=
    'SHOW' 'SHARDING' 'TABLE' 'RULES' 'USED' 'KEY' 'GENERATOR' keyGeneratorName ('FROM'
    ' databaseName)?

keyGeneratorName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Columns	Descriptions
type	Sharding rule type
name	Sharding rule name

Example

- Query sharding table rules for the specified sharding key generator in spicified logical database

```
SHOW SHARDING TABLE RULES USED KEY GENERATOR snowflake_key_generator FROM sharding_db;
```

```
mysql> SHOW SHARDING TABLE RULES USED KEY GENERATOR snowflake_key_generator FROM
sharding_db;
+-----+-----+
| type | name          |
+-----+-----+
| table | t_order_item |
+-----+-----+
1 row in set (0.00 sec)
```

- Query sharding table rules for specified sharding key generator in the current logical database

```
SHOW SHARDING TABLE RULES USED KEY GENERATOR snowflake_key_generator;
```

```
mysql> SHOW SHARDING TABLE RULES USED KEY GENERATOR snowflake_key_generator;
+-----+-----+
| type | name      |
+-----+-----+
| table | t_order_item |
+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

SHOW, SHARDING, TABLE, USED, KEY, GENERATOR, FROM

Related links

- [Reserved word](#)

SHOW SHARDING TABLE RULES USED AUDITOR

Description

SHOW SHARDING TABLE RULES USED AUDITOR syntax is used to query sharding rules used specified sharding auditor in specified logical database

Syntax

```
ShowShardingTableRulesUsedAuditor ::=
    'SHOW' 'SHARDING' 'TABLE' 'RULES' 'USED' 'AUDITOR' AuditorName ('FROM'
databaseName)?

AuditorName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Columns	Descriptions
type	Sharding rule type
name	Sharding rule name

Example

- Query sharding table rules for the specified sharding auditor in spicified logical database

```
SHOW SHARDING TABLE RULES USED AUDITOR sharding_key_required_auditor FROM sharding_db;
```

```
mysql> SHOW SHARDING TABLE RULES USED AUDITOR sharding_key_required_auditor FROM sharding_db;
+-----+-----+
| type | name |
+-----+-----+
| table | t_order |
+-----+-----+
1 row in set (0.00 sec)
```

- Query sharding table rules for specified sharding auditor in the current logical database

```
SHOW SHARDING TABLE RULES USED AUDITOR sharding_key_required_auditor;
```

```
mysql> SHOW SHARDING TABLE RULES USED AUDITOR sharding_key_required_auditor;
+-----+-----+
| type | name |
+-----+-----+
| table | t_order |
+-----+-----+
1 row in set (0.00 sec)
```


Reserved word

SHOW, SHARDING, TABLE, RULES, USED, AUDITOR, FROM

Related links

- [Reserved word](#)

SHOW SHARDING TABLE REFERENCE RULE

Description

SHOW SHARDING TABLE REFERENCE RULE syntax is used to query specified sharding table reference rule in the specified logical database.

Syntax

```
ShowShardingBindingTableRules ::=
    'SHOW' 'SHARDING' 'TABLE' 'REFERENCE' ('RULE' ruleName | 'RULES') ('FROM'
databaseName)?

ruleName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Columns	Descriptions
name	Sharding table reference rule name
sharding_table_reference	sharding table reference

Example

- Query sharding table reference rules for the specified logical database

```
SHOW SHARDING TABLE REFERENCE RULES FROM sharding_db;
```

```
mysql> SHOW SHARDING TABLE REFERENCE RULES FROM sharding_db;
+-----+-----+
| name | sharding_table_reference |
+-----+-----+
| ref_0 | t_a,t_b                  |
| ref_1 | t_c,t_d                  |
+-----+-----+
2 rows in set (0.00 sec)
```

- Query sharding table reference rules for the current logical database

```
SHOW SHARDING TABLE REFERENCE RULES;
```

```
mysql> SHOW SHARDING TABLE REFERENCE RULES;
+-----+-----+
| name | sharding_table_reference |
+-----+-----+
| ref_0 | t_a,t_b                  |
| ref_1 | t_c,t_d                  |
+-----+-----+
2 rows in set (0.00 sec)
```

- Query specified sharding table reference rule for the specified logical database

```
SHOW SHARDING TABLE REFERENCE RULE ref_0 FROM sharding_db;
```

```
mysql> SHOW SHARDING TABLE REFERENCE RULE FROM sharding_db;
+-----+-----+
| name | sharding_table_reference |
+-----+-----+
| ref_0 | t_a,t_b                  |
+-----+-----+
1 row in set (0.00 sec)
```

- Query specified sharding table reference rule for the current logical database

```
SHOW SHARDING TABLE REFERENCE RULE ref_0;
```

```
mysql> SHOW SHARDING TABLE REFERENCE RULE ref_0;
+-----+-----+
| name | sharding_table_reference |
+-----+-----+
```

```
| ref_0 | t_a,t_b |
+-----+-----+
1 row in set (0.00 sec)
```

Reserved word

SHOW, SHARDING, TABLE, REFERENCE, RULE, RULES, FROM

Related links

- [Reserved word](#)

COUNT SHARDING RULE

Description

The COUNT SHARDING RULE syntax is used to query the number of sharding rules for specified database.

Syntax

```
CountShardingRule ::=
    'COUNT' 'SHARDING' 'RULE' ('FROM' databaseName)?

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
rule_name	rule type
database	the database to which the rule belongs
count	the number of the rule

Example

- Query the number of sharding rules for specified database.

```
COUNT SHARDING RULE FROM sharding_db;
```

```
mysql> COUNT SHARDING RULE FROM sharding_db;
```

rule_name	database	count
sharding_table	sharding_db	2
sharding_table_reference	sharding_db	2

2 rows in set (0.00 sec)

- Query the number of sharding rules for current database.

```
COUNT SHARDING RULE;
```

```
mysql> COUNT SHARDING RULE;
```

rule_name	database	count
sharding_table	sharding_db	2
sharding_table_reference	sharding_db	2

2 rows in set (0.00 sec)

Reserved word

COUNT, SHARDING, RULE, FROM

Related links

- [Reserved word](#)

Broadcast Table

This chapter describes the syntax of broadcast table.

SHOW BROADCAST TABLE RULE

Description

The `SHOW BROADCAST TABLE RULE` syntax is used to broadcast tables for specified database.

Syntax

```
ShowBroadcastTableRule ::=  
  'SHOW' 'BROADCAST' 'TABLE' 'RULES' ('FROM' databaseName)?  
  
databaseName ::=  
  identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
<code>broadcast_table</code>	Broadcast table name

Example

- Query broadcast tables for specified database.

```
SHOW BROADCAST TABLE RULES FROM sharding_db;
```

```
mysql> SHOW BROADCAST TABLE RULES FROM sharding_db;  
+-----+  
| broadcast_table |  
+-----+  
| t_a            |  
| t_b            |  
| t_c            |  
+-----+  
3 rows in set (0.00 sec)
```

- Query broadcast table for current database.

```
SHOW BROADCAST TABLE RULES;
```

```
mysql> SHOW BROADCAST TABLE RULES;
```

```
+-----+
| broadcast_table |
+-----+
| t_a            |
| t_b            |
| t_c            |
+-----+
3 rows in set (0.00 sec)
```

Reserved word

SHOW, BROADCAST, TABLE, RULES

Related links

- [Reserved word](#)

COUNT BROADCAST RULE

Description

The COUNT BROADCAST RULE syntax is used to query the number of broadcast table rules for specified database.

Syntax

```
CountBroadcastRule ::=
    'COUNT' 'BROADCAST' 'RULE' ('FROM' databaseName)?

databaseName ::=
    identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
<code>rule_name</code>	rule type
<code>database</code>	the database to which the rule belongs
<code>count</code>	the number of the rule

Example

- Query the number of broadcast table rules for specified database.

```
COUNT BROADCAST RULE FROM sharding_db;
```

```
mysql> COUNT BROADCAST RULE FROM sharding_db;
```

```
+-----+-----+-----+
| rule_name          | database      | count |
+-----+-----+-----+
| broadcast_table    | sharding_db   | 0     |
+-----+-----+-----+
1 rows in set (0.00 sec)
```

- Query the number of broadcast table rules for current database.

```
COUNT BROADCAST RULE;
```

```
mysql> COUNT BROADCAST RULE;
```

```
+-----+-----+-----+
| rule_name          | database      | count |
+-----+-----+-----+
| broadcast_table    | sharding_db   | 0     |
+-----+-----+-----+
1 rows in set (0.00 sec)
```

Reserved word

COUNT, BROADCAST, RULE, FROM

Related links

- [Reserved word](#)

Single Table

This chapter describes the syntax of single table.

SHOW SINGLE TABLE

Description

The `SHOW SINGLE TABLE` syntax is used to query single tables for specified database.

Syntax

```
ShowSingleTable ::=  
    'SHOW' 'SINGLE' ('TABLES' ('LIKES' likeLiteral)? | 'TABLE' tableName) ('FROM'  
databaseName)?  
  
tableName ::=  
    identifier  
  
databaseName ::=  
    identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
table_name	Single table name
storage_unit_name	The storage unit name where the single table is located

Example

- Query specified single table for specified database.

```
SHOW SINGLE TABLE t_user FROM sharding_db;
```

```
mysql> SHOW SINGLE TABLE t_user FROM sharding_db;
+-----+-----+
| table_name | storage_unit_name |
+-----+-----+
| t_user    | ds_0              |
+-----+-----+
1 row in set (0.00 sec)
```

- Query specified single table for current database.

```
SHOW SINGLE TABLE t_user;
```

```
mysql> SHOW SINGLE TABLE t_user;
+-----+-----+
| table_name | storage_unit_name |
+-----+-----+
| t_user    | ds_0              |
+-----+-----+
1 row in set (0.00 sec)
```

- Query single tables for specified database.

```
SHOW SINGLE TABLES FROM sharding_db;
```

```
mysql> SHOW SINGLE TABLES FROM sharding_db;
+-----+-----+
| table_name | storage_unit_name |
+-----+-----+
| t_user    | ds_0              |
+-----+-----+
1 row in set (0.00 sec)
```

- Query single tables for current database.

```
SHOW SINGLE TABLES;
```

```
mysql> SHOW SINGLE TABLES;
+-----+-----+
| table_name | storage_unit_name |
+-----+-----+
| t_user     | ds_0               |
+-----+-----+
1 row in set (0.00 sec)
```

- Query the single tables whose table name end with order_5 for the specified logic database.

```
SHOW SINGLE TABLES LIKE '%order_5' FROM sharding_db;
```

```
mysql> SHOW SINGLE TABLES LIKE '%order_5' FROM sharding_db;
+-----+-----+
| table_name | storage_unit_name |
+-----+-----+
| t_order_5  | ds_1               |
+-----+-----+
1 row in set (0.11 sec)
```

- Query the single tables whose table name end with order_5 for the current logic database

```
SHOW SINGLE TABLES LIKE '%order_5';
```

```
mysql> SHOW SINGLE TABLES LIKE '%order_5';
+-----+-----+
| table_name | storage_unit_name |
+-----+-----+
| t_order_5  | ds_1               |
+-----+-----+
1 row in set (0.11 sec)
```

Reserved word

SHOW, SINGLE, TABLE, TABLES, LIKE, FROM

Related links

- [Reserved word](#)

SHOW DEFAULT SINGLE TABLE STORAGE UNIT

Description

The SHOW DEFAULT SINGLE TABLE STORAGE UNIT syntax is used to query storage units for specified database.

Syntax

```
ShowDefaultSingleTableStorageUnit ::=
    'SHOW' 'DEFAULT' 'SINGLE' 'TABLE' 'STORAGE' 'UNIT' ('FROM' databaseName)?

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return Value Description

Column	Description
storage_unit_name	Storage unit name

Example

- Query storage units for specified database.

```
SHOW DEFAULT SINGLE TABLE STORAGE UNIT
```

```
sql> SHOW DEFAULT SINGLE TABLE STORAGE UNIT;
+-----+
| storage_unit_name |
+-----+
| ds_0              |
+-----+
1 row in set (0.01 sec)
```

Reserved word

SHOW, DEFAULT, SINGLE, TABLE, STORAGE, UNIT

Related links

- [Reserved word](#)

COUNT SINGLE_TABLE RULE

Description

The COUNT SINGLE_TABLE syntax is used to query number of single table for specified database.

Syntax

```
CountSingleTable ::=
    'COUNT' 'SINGLE' 'TABLE' ('FROM' databaseName)?

databaseName ::=
    identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return Value Description

Column	Description
database	The database name where the single table is located
count	The count of single table

Example

- Query the number of single rules for specified database.

```
COUNT SINGLE TABLE
```

```
mysql> COUNT SINGLE TABLE;
+-----+-----+
| database | count |
+-----+-----+
| ds      | 2     |
+-----+-----+
1 row in set (0.02 sec)
```

Reserved word

COUNT, SINGLE, TABLE, FROM

Related links

- [Reserved word](#)

SHOW UNLOADED SINGLE TABLES

Description

The `SHOW UNLOADED SINGLE TABLES` syntax is used to query unloaded single tables.

Syntax

```
showUnloadedSingleTables::=  
  'SHOW' 'UNLOADED' 'SINGLE' 'TABLES'
```

Return value description

Column	Description
table_name	Single table name
storage_unit_name	The storage unit name where the single table is located

Example

- Query unloaded single tables.

```
SHOW UNLOADED SINGLE TABLES;
```

```
mysql> SHOW UNLOADED SINGLE TABLES;  
+-----+-----+  
| table_name | storage_unit_name |  
+-----+-----+  
| t_single   | ds_1               |  
+-----+-----+  
1 row in set (0.01 sec)
```

Reserved word

SHOW, UNLOADED, SINGLE, TABLES

Related links

- [Reserved word](#)

Readwrite-Splitting

This chapter describes the syntax of readwrite-splitting.

SHOW READWRITE_SPLITTING RULE

Description

The SHOW READWRITE_SPLITTING RULE syntax is used to query specified readwrite-splitting rules for specified database.

Syntax

```
ShowReadWriteSplittingRule ::=  
  'SHOW' 'READWRITE_SPLITTING' ('RULE' ruleName | 'RULES') ('FROM' databaseName)?  
  
ruleName ::=  
  identifier  
  
databaseName ::=  
  identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
name	Readwrite-splitting rule name
write_data_source_name	Write data source name
read_data_source_names	Read data source name list
transactional_read_query_strategy	Routing strategy for read query within a transaction
load_balancer_type	Load balance algorithm type
load_balancer_props	Load balance algorithm parameter

Example

- Query readwrite-splitting rules for specified database.

```
SHOW READWRITE_SPLITTING RULES FROM readwrite_splitting_db;
```

```
mysql> SHOW READWRITE_SPLITTING RULES FROM readwrite_splitting_db;
```

name	write_storage_unit_name	read_storage_unit_names	transactional_read_query_strategy	load_balancer_type	load_balancer_props
ms_group_0	write_ds	read_ds_0,read_ds_1		DYNAMIC	
random					

1 row in set (0.01 sec)

- Query readwrite-splitting rules for current database.

```
SHOW READWRITE_SPLITTING RULES;
```

```
mysql> SHOW READWRITE_SPLITTING RULES;
```

name	write_storage_unit_name	read_storage_unit_names	transactional_read_query_strategy	load_balancer_type	load_balancer_props
ms_group_0	write_ds	read_ds_0,read_ds_1		DYNAMIC	
random					

1 row in set (0.01 sec)

- Query specified readwrite-splitting rule for specified database.

```
SHOW READWRITE_SPLITTING RULE ms_group_0 FROM readwrite_splitting_db;
```

```
mysql> SHOW READWRITE_SPLITTING RULE ms_group_0 FROM readwrite_splitting_db;
```

name	write_storage_unit_name	read_storage_unit_names	transactional_read_query_strategy	load_balancer_type	load_balancer_props
ms_group_0	write_ds	read_ds_0,read_ds_1		DYNAMIC	


```
| random          |
+-----+-----+-----+
-----+-----+
1 row in set (0.01 sec)
```

- Query specified readwrite-splitting rule for current database.

```
SHOW READWRITE_SPLITTING RULE ms_group_0;
```

```
mysql> SHOW READWRITE_SPLITTING RULE ms_group_0;
+-----+-----+-----+-----+
-----+-----+-----+-----+
| name          | write_storage_unit_name | read_storage_unit_names | transactional_
read_query_strategy | load_balancer_type | load_balancer_props |
+-----+-----+-----+-----+
| ms_group_0    | write_ds              | read_ds_0,read_ds_1    | DYNAMIC
| random        |                        |                          |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

SHOW, READWRITE_SPLITTING, RULE, RULES, FROM

Related links

- [Reserved word](#)

COUNT READWRITE_SPLITTING RULE

Description

The COUNT READWRITE_SPLITTING RULE syntax is used to query the number of readwrite-splitting rules for specified database.

Syntax

```
CountReadwriteSplittingRule ::=
    'COUNT' 'READWRITE_SPLITTING' 'RULE' ('FROM' databaseName)?

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
rule_name	rule type
database	the database to which the rule belongs
count	the number of the rule

Example

- Query the number of readwrite-splitting rules for specified database.

```
COUNT READWRITE_SPLITTING RULE FROM readwrite_splitting_db;
```

```
mysql> COUNT READWRITE_SPLITTING RULE FROM readwrite_splitting_db;
+-----+-----+-----+
| rule_name          | database          | count |
+-----+-----+-----+
| readwrite_splitting | readwrite_splitting_db | 1     |
+-----+-----+-----+
1 row in set (0.02 sec)
```

- Query the number of readwrite-splitting rules for current database.

```
COUNT READWRITE_SPLITTING RULE;
```

```
mysql> COUNT READWRITE_SPLITTING RULE;
+-----+-----+-----+
| rule_name          | database          | count |
+-----+-----+-----+
| readwrite_splitting | readwrite_splitting_db | 1     |
+-----+-----+-----+
```

```
+-----+-----+-----+
1 row in set (0.00 sec)
```

Reserved word

COUNT, READWRITE_SPLITTING, RULE, FROM

Related links

- [Reserved word](#)

Encrypt

This chapter describes the syntax of encrypt.

SHOW ENCRYPT RULES

Description

The SHOW ENCRYPT RULES syntax is used to query encryption rules for a specified database.

Syntax

```
ShowEncryptRule ::=
    'SHOW' 'ENCRYPT' ('RULES' | 'TABLE' 'RULE' ruleName) ('FROM' databaseName)?

ruleName ::=
    identifier

databaseName ::=
    identifier
```

Note

- When databaseName is not specified, then DATABASE is currently used as the default name. If DATABASE is not used, you will receive a No database selected prompt.

Return value description

Column	Description
table	Logical table name
logic_column	Logical column name
cipher_column	Ciphertext column name
assisted_query_column	Assisted query column name
like_query_column	Like query column name
encryptor_type	Encryption algorithm type
encryptor_props	Encryption algorithm parameter
assisted_query_type	Assisted query algorithm type
assisted_query_props	Assisted query algorithm parameter
like_query_type	Like query algorithm type
like_query_props	Like query algorithm parameter

Example

- Query encrypt rules for specified database.

```
SHOW ENCRYPT RULES FROM encrypt_db;
```

```
mysql> SHOW ENCRYPT RULES FROM encrypt_db;
+-----+-----+-----+-----+-----+-----+-----+-----+
| table      | logic_column | cipher_column | assisted_query_column | like_query_ |
| column    | encryptor_type | encryptor_props | assisted_query_type | assisted_ |
| query_props | like_query_type | like_query_props |
+-----+-----+-----+-----+-----+-----+-----+
| t_user     | pwd          | pwd_cipher    |
| AES        | aes-key-value=123456abc, digest-algorithm-name=SHA-1 |
|
| t_encrypt  | pwd          | pwd_cipher    |
| AES        | aes-key-value=123456abc, digest-algorithm-name=SHA-1 |
|
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

- Query encrypt rules for current database.

```
SHOW ENCRYPT RULES;
```

```
mysql> SHOW ENCRYPT RULES;
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
| table      | logic_column | cipher_column | assisted_query_column | like_query_
column | encryptor_type | encryptor_props          | assisted_query_type | assisted_
query_props | like_query_type | like_query_props |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
| t_user      | pwd          | pwd_cipher      |                      |
| AES         | aes-key-value=123456abc, digest-algorithm-name=SHA-1 |
|             |              |                  |                      |
| t_encrypt   | pwd          | pwd_cipher      |                      |
| AES         | aes-key-value=123456abc, digest-algorithm-name=SHA-1 |
|             |              |                  |                      |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

- Query specified encrypt rule in specified database.

```
SHOW ENCRYPT TABLE RULE t_encrypt FROM encrypt_db;
```

```
mysql> SHOW ENCRYPT TABLE RULE t_encrypt FROM encrypt_db;
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
| table      | logic_column | cipher_column | assisted_query_column | like_query_
column | encryptor_type | encryptor_props          | assisted_query_type | assisted_
query_props | like_query_type | like_query_props |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
| t_encrypt   | pwd          | pwd_cipher      |                      |
| AES         | aes-key-value=123456abc, digest-algorithm-name=SHA-1 |
|             |              |                  |                      |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

- Query specified encrypt rule in current database.

```
SHOW ENCRYPT TABLE RULE t_encrypt;
```

```
mysql> SHOW ENCRYPT TABLE RULE t_encrypt;
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
| table      | logic_column | cipher_column | assisted_query_column | like_query_
column | encryptor_type | encryptor_props      | assisted_query_type | assisted_
query_props | like_query_type | like_query_props |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
| t_encrypt | pwd          | pwd_cipher      |                  |
| AES       | aes-key-value=123456abc, digest-algorithm-name=SHA-1 |
|           |              |                 |                  |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

SHOW, ENCRYPT, TABLE, RULE, RULES, FROM

Related links

- [Reserved word](#)

COUNT ENCRYPT RULE

Description

The COUNT ENCRYPT RULE syntax is used to query the number of encrypt rules for specified database.

Syntax

```
CountEncryptRule ::=
    'COUNT' 'ENCRYPT' 'RULE' ('FROM' databaseName)?

databaseName ::=
    identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
<code>rule_name</code>	rule type
<code>database</code>	the database to which the rule belongs
<code>count</code>	the number of the rule

Example

- Query the number of encrypt rules for specified database.

```
COUNT ENCRYPT RULE FROM encrypt_db;
```

```
mysql> COUNT ENCRYPT RULE FROM encrypt_db;
+-----+-----+-----+
| rule_name | database | count |
+-----+-----+-----+
| encrypt  | encrypt_db | 2     |
+-----+-----+-----+
1 row in set (0.01 sec)
```

- Query the number of encrypt rules for current database.

```
COUNT ENCRYPT RULE;
```

```
mysql> COUNT ENCRYPT RULE;
+-----+-----+-----+
| rule_name | database | count |
+-----+-----+-----+
| encrypt  | encrypt_db | 2     |
+-----+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

COUNT, ENCRYPT, RULE, FROM

Related links

- [Reserved word](#)

Mask

This chapter describes the syntax of mask.

SHOW MASK RULES

Description

The SHOW MASK RULES syntax is used to query mask rules for specified database.

Syntax

```
ShowMaskRule ::=
    'SHOW' 'MASK' ('RULES' | 'RULE' ruleName) ('FROM' databaseName)?

ruleName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
table	Table name
column	Column name
algorithm_type	Mask algorithm type
algorithm_props	Mask algorithm properties

Example

- Query mask rules for specified database

```
SHOW MASK RULES FROM mask_db;
```

```
mysql> SHOW MASK RULES FROM mask_db;
+-----+-----+-----+-----+
| table | column | algorithm_type | algorithm_props |
+-----+-----+-----+-----+
| t_mask | phoneNum | MASK_FROM_X_TO_Y | to-y=2,replace-char=*,from-x=1 |
| t_mask | address | MD5 | |
| t_order | order_id | MD5 | |
| t_user | user_id | MASK_FROM_X_TO_Y | to-y=2,replace-char=*,from-x=1 |
+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

- Query mask rules for current database

```
SHOW MASK RULES;
```

```
mysql> SHOW MASK RULES;
+-----+-----+-----+-----+
| table | column | algorithm_type | algorithm_props |
+-----+-----+-----+-----+
| t_mask | phoneNum | MASK_FROM_X_TO_Y | to-y=2,replace-char=*,from-x=1 |
| t_mask | address | MD5 | |
| t_order | order_id | MD5 | |
| t_user | user_id | MASK_FROM_X_TO_Y | to-y=2,replace-char=*,from-x=1 |
+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

- Query specified mask rule for specified database

```
SHOW MASK RULE t_mask FROM mask_db;
```

```
mysql> SHOW MASK RULE t_mask FROM mask_db;
+-----+-----+-----+-----+
| table | logic_column | mask_algorithm | props |
+-----+-----+-----+-----+
| t_mask | phoneNum | MASK_FROM_X_TO_Y | to-y=2,replace-char=*,from-x=1 |
| t_mask | address | MD5 | |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

- Query specified mask rule for current database

```
SHOW MASK RULE t_mask;
```

```
mysql> SHOW MASK RULE t_mask;
```

```
+-----+-----+-----+-----+
| table | logic_column | mask_algorithm | props |
+-----+-----+-----+-----+
| t_mask | phoneNum     | MASK_FROM_X_TO_Y | to-y=2,replace-char=*,from-x=1 |
| t_mask | address      | MD5              | |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Reserved word

SHOW, MASK, RULE, RULES, FROM

Related links

- [Reserved word](#)

COUNT MASK RULE

Description

The COUNT MASK RULE syntax is used to query the number of mask rules for specified database.

Syntax

```
CountMaskRule ::=
    'COUNT' 'MASK' 'RULE' ('FROM' databaseName)?
```

```
databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
rule_name	rule type
database	the database to which the rule belongs
count	the number of the rule

Example

- Query the number of mask rules for specified database.

```
COUNT MASK RULE FROM mask_db;
```

```
mysql> COUNT MASK RULE FROM mask_db;
+-----+-----+-----+
| rule_name | database | count |
+-----+-----+-----+
| mask      | mask_db  | 3      |
+-----+-----+-----+
1 row in set (0.50 sec)
```

- Query the number of mask rules for current database.

```
COUNT MASK RULE;
```

```
mysql> COUNT MASK RULE;
+-----+-----+-----+
| rule_name | database | count |
+-----+-----+-----+
| mask      | mask_db  | 3      |
+-----+-----+-----+
1 row in set (0.50 sec)
```

Reserved word

COUNT, MASK, RULE, FROM

Related links

- [Reserved word](#)

Shadow

This chapter describes the syntax of shadow.

SHOW SHADOW RULE

Description

The SHOW SHADOW RULE syntax is used to query shadow rules for specified database.

Syntax

```
ShowShadowRule ::=
    'SHOW' 'SHADOW' ('RULES' | 'RULE' shadowRuleName) ('FROM' databaseName)?

shadowRuleName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
rule_name	Shadow rule name
source_name	Data source name
shadow_name	Shadow data source name
shadow_table	Shadow table

Example

- Query specified shadow rule in specified database.

```
SHOW SHADOW RULE shadow_rule FROM shadow_db;
```

```
mysql> SHOW SHADOW RULE shadow_rule FROM shadow_db;
+-----+-----+-----+-----+
| rule_name | source_name | shadow_name | shadow_table |
+-----+-----+-----+-----+
| shadow_rule | ds_0 | ds_1 | t_order_item,t_order |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

- Query specified shadow rule in current database.

```
SHOW SHADOW RULE shadow_rule;
```

```
mysql> SHOW SHADOW RULE shadow_rule;
+-----+-----+-----+-----+
| rule_name | source_name | shadow_name | shadow_table |
+-----+-----+-----+-----+
| shadow_rule | ds_0 | ds_1 | t_order_item,t_order |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

- Query shadow rules for specified database.

```
SHOW SHADOW RULES FROM shadow_db;
```

```
mysql> SHOW SHADOW RULES FROM shadow_db;
+-----+-----+-----+-----+
| rule_name | source_name | shadow_name | shadow_table |
+-----+-----+-----+-----+
| shadow_rule | ds_0 | ds_1 | t_order_item,t_order |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

- Query shadow rules for current database.

```
SHOW SHADOW RULES;
```

```
mysql> SHOW SHADOW RULES;
+-----+-----+-----+-----+
| rule_name | source_name | shadow_name | shadow_table |
+-----+-----+-----+-----+
| shadow_rule | ds_0 | ds_1 | t_order_item,t_order |
+-----+-----+-----+-----+
```

```
1 row in set (0.00 sec)
```

Reserved word

SHOW, SHADOW, RULE, RULES, FROM

Related links

- [Reserved word](#)

SHOW SHADOW TABLE RULES

Description

The SHOW SHADOW TABLE RULES syntax is used to query shadow table rules for specified database.

Syntax

```
ShowShadowTableRule ::=  
    'SHOW' 'SHADOW' 'TABLE' 'RULES' ('FROM' databaseName)?  
  
databaseName ::=  
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
shadow_table	Shadow table
shadow_algorithm_name	Shadow algorithm name

Example

- Query shadow table rules for specified database.

```
SHOW SHADOW TABLE RULES FROM shadow_db;
```

```
mysql> SHOW SHADOW TABLE RULES FROM shadow_db;
```

```
+-----+-----+
| shadow_table | shadow_algorithm_name |
+-----+-----+
| t_order_item | shadow_rule_t_order_item_value_match |
| t_order      | sql_hint_algorithm,shadow_rule_t_order_regex_match |
+-----+-----+
2 rows in set (0.00 sec)
```

- Query shadow table rules for current database.

```
SHOW SHADOW TABLE RULES;
```

```
mysql> SHOW SHADOW TABLE RULES;
```

```
+-----+-----+
| shadow_table | shadow_algorithm_name |
+-----+-----+
| t_order_item | shadow_rule_t_order_item_value_match |
| t_order      | sql_hint_algorithm,shadow_rule_t_order_regex_match |
+-----+-----+
2 rows in set (0.01 sec)
```

Reserved word

SHOW, SHADOW, TABLE, RULES, FROM

Related links

- [Reserved word](#)

SHOW SHADOW ALGORITHMS

Description

The SHOW SHADOW ALGORITHMS syntax is used to query shadow algorithms for specified database.

Syntax

```
ShowShadowAlgorithm::=
  'SHOW' 'SHADOW' 'ALGORITHMS' ('FROM' databaseName)?

databaseName ::=
  identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
shadow_algorithm_name	Shadow algorithm name
type	Shadow algorithm type
props	Shadow algorithm properties
is_default	Default

Example

- Query shadow algorithms for specified database.

```
SHOW SHADOW ALGORITHMS FROM shadow_db;
```

```
mysql> SHOW SHADOW ALGORITHMS FROM shadow_db;
+-----+-----+-----+
+-----+
| shadow_algorithm_name | type          | props                                     |
| is_default           |               |                                         |
+-----+-----+-----+
+-----+
| user_id_match_algorithm | VALUE_MATCH | column=user_id,operation=insert,value=1 |
| false                 |             |                                         |
+-----+-----+-----+
1 row in set (0.00 sec)
```

- Query shadow algorithms for current database.


```
SHOW SHADOW ALGORITHMS;
```

```
mysql> SHOW SHADOW ALGORITHMS;
```

```
+-----+-----+-----+
+-----+
| shadow_algorithm_name | type      | props                                |
| is_default           |           |                                     |
+-----+-----+-----+
+-----+
| user_id_match_algorithm | VALUE_MATCH | column=user_id,operation=insert,value=1 |
| false                  |           |                                     |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Reserved word

SHOW, SHADOW, ALGORITHMS, FROM

Related links

- [Reserved word](#)

SHOW DEFAULT SHADOW ALGORITHM

Description

The SHOW DEFAULT SHADOW ALGORITHM syntax is used to query default shadow algorithm for specified database.

Syntax

```
ShowDefaultShadowAlgorithm ::=
    'SHOW' 'DEFAULT' 'SHADOW' 'ALGORITHM' ('FROM' databaseName)?

databaseName ::=
    identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
<code>shadow_algorithm_name</code>	Shadow algorithm name
<code>type</code>	Shadow algorithm type
<code>props</code>	Shadow algorithm properties

Example

- Query default shadow algorithm for specified database.

```
SHOW DEFAULT SHADOW ALGORITHM FROM shadow_db;
```

```
mysql> SHOW DEFAULT SHADOW ALGORITHM FROM shadow_db;
```

```
+-----+-----+-----+
| shadow_algorithm_name | type          | props                                     |
+-----+-----+-----+
| user_id_match_algorithm | VALUE_MATCH  | column=user_id,operation=insert,value=1 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

- Query default shadow algorithm for current database.

```
SHOW DEFAULT SHADOW ALGORITHM;
```

```
mysql> SHOW DEFAULT SHADOW ALGORITHM;
```

```
+-----+-----+-----+
| shadow_algorithm_name | type          | props                                     |
+-----+-----+-----+
| user_id_match_algorithm | VALUE_MATCH  | column=user_id,operation=insert,value=1 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Reserved word

SHOW, DEFAULT, SHADOW, ALGORITHM, FROM

Related links

- [Reserved word](#)

COUNT SHADOW RULE

Description

The COUNT SHADOW RULE syntax is used to query the number of shadow rules for specified database.

Syntax

```
CountShadowRule ::=  
    'COUNT' 'SHADOW' 'RULE' ('FROM' databaseName)?  
  
databaseName ::=  
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return value description

Column	Description
rule_name	rule type
database	the database to which the rule belongs
count	the number of the rule

Example

- Query the number of shadow rules for specified database.

```
COUNT SHADOW RULE FROM shadow_db;
```

```
mysql> COUNT SHADOW RULE FROM shadow_db;
```

```
+-----+-----+-----+
| rule_name | database | count |
+-----+-----+-----+
| shadow    | shadow_db | 1      |
+-----+-----+-----+
1 row in set (0.00 sec)
```

- Query the number of shadow rules for current database.

```
COUNT SHADOW RULE;
```

```
mysql> COUNT SHADOW RULE;
```

```
+-----+-----+-----+
| rule_name | database | count |
+-----+-----+-----+
| shadow    | shadow_db | 1      |
+-----+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

COUNT, SHADOW, RULE, FROM

Related links

- [Reserved word](#)

RAL Syntax

RAL (Resource & Rule Administration Language) responsible for the added-on feature of transaction type switch, scaling and so on.

GLOBAL RULE

This chapter describes the syntax of Global Rule.

SHOW AUTHORITY RULE

Description

The SHOW AUTHORITY RULE syntax is used to query authority rule configuration.

Syntax

```
ShowAuthorityRule ::=
    'SHOW' 'AUTHORITY' 'RULE'
```

Return Value Description

Column	Description
users	users
provider	privilege provider type
props	privilege properties

Example

- Query authority rule configuration

```
SHOW AUTHORITY RULE;
```

```
mysql> SHOW AUTHORITY RULE;
+-----+-----+-----+
| users          | provider      | props |
+-----+-----+-----+
| root@%; sharding@% | ALL_PERMITTED |      |
+-----+-----+-----+
1 row in set (0.07 sec)
```

Reserved word

SHOW, AUTHORITY, RULE

Related links

- [Reserved word](#)

SHOW TRANSACTION RULE

Description

The SHOW TRANSACTION RULE syntax is used to query transaction rule configuration.

Syntax

```
ShowTransactionRule ::=
  'SHOW' 'TRANSACTION' 'RULE'
```

Return Value Description

Column	Description
users	users
provider	privilege provider type
props	privilege properties

Example

- Query transaction rule configuration

```
SHOW TRANSACTION RULE;
```

```
mysql> SHOW TRANSACTION RULE;
+-----+-----+-----+
| default_type | provider_type | props |
+-----+-----+-----+
| LOCAL       |               |       |
+-----+-----+-----+
1 row in set (0.05 sec)
```

Reserved word

SHOW, TRANSACTION, RULE

Related links

- [Reserved word](#)

ALTER TRANSACTION RULE

Description

The ALTER TRANSACTION RULE syntax is used to alter transaction rule configuration.

Syntax

```
AlterTransactionRule ::=
  'ALTER' 'TRANSACTION' 'RULE' '(' 'DEFAULT' '=' defaultTransactionType ',' 'TYPE'
  '(' 'NAME' '=' transactionManager ',' propertiesDefinition ')' ')'

propertiesDefinition ::=
  'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

defaultTransactionType ::=
  string

transactionManager ::=
  string

key ::=
  string

value ::=
  literal
```

Supplement

- defaultTransactionType support LOCAL, XA, BASE
- transactionManager support Atomikos and Narayana

Example

- Alter transaction rule

```
ALTER TRANSACTION RULE(
  DEFAULT="XA", TYPE(NAME="Narayana")
);
```

Reserved word

ALTER, TRANSACTION, RULE, DEFAULT, TYPE, NAME, PROPERTIES

Related links

- [Reserved word](#)

SHOW SQL_PARSER RULE

Description

The SHOW SQL_PARSER RULE syntax is used to query sql parser rule configuration.

Syntax

```
ShowSqlParserRule ::=
  'SHOW' 'SQL_PARSER' 'RULE'
```

Return Value Description

Column	Description
parse_tree_cache	parse tree cache configuration
sql_statement_cache	SQL statement cache configuration

Example

- Query sql parser rule configuration

```
SHOW SQL_PARSER RULE;
```

```
mysql> SHOW SQL_PARSER RULE;
+-----+-----+
---+
| parse_tree_cache                | sql_statement_cache                |
+-----+-----+
---+
| initialCapacity: 128, maxSize: 1024 | initialCapacity: 2000, maxSize: 65535 |
+-----+-----+
1 row in set (0.05 sec)
```

Reserved word

SHOW, SQL_PARSER, RULE

Related links

- [Reserved word](#)

Alter SQL_PARSER Rule

Description

The ALTER SQL_PARSER RULE syntax is used to alter the SQL parser rule configuration.

Syntax

```
AlterSqlParserRule ::=
  'ALTER' 'SQL_PARSER' 'RULE' '(' sqlParserRuleDefinition ')'

sqlParserRuleDefinition ::=
  parseTreeCacheDefinition? (',' sqlStatementCacheDefinition)?

parseTreeCacheDefinition ::=
  'PARSE_TREE_CACHE' '(' cacheOption ')'

sqlStatementCacheDefinition ::=
```

```

'SQL_STATEMENT_CACHE' '(' cacheOption ')'

cacheOption ::=
  ('INITIAL_CAPACITY' '=' initialCapacity)? ('?' 'MAXIMUM_SIZE' '=' maximumSize)?

initialCapacity ::=
  int

maximumSize ::=
  int

```

Note

- `PARSE_TREE_CACHE`: local cache configuration of the syntax tree.
- `SQL_STATEMENT_CACHE`: the local cache of SQL statement.

Example

- Alter SQL parser rule

```

ALTER SQL_PARSER RULE (
  PARSE_TREE_CACHE(INITIAL_CAPACITY=128, MAXIMUM_SIZE=1024),
  SQL_STATEMENT_CACHE(INITIAL_CAPACITY=2000, MAXIMUM_SIZE=65535)
);

```

Reserved word

ALTER, SQL_PARSER, RULE, PARSE_TREE_CACHE, INITIAL_CAPACITY, MAXIMUM_SIZE, SQL_STATEMENT_CACHE

Related links

- [Reserved word](#)

SHOW TRAFFIC RULE**Description**

The `SHOW TRAFFIC RULE` syntax is used to query specified dual routing rule.

Syntax

```
ShowTrafficRule ::=
    'SHOW' 'TRAFFIC' ('RULES' | 'RULE' ruleName)?

ruleName ::=
    identifier
```

Supplement

- When ruleName not specified, the default is show all traffic rules

Return Value Description

Column	Description
name	traffic rule name
labels	compute node labels
algorithm_type	traffic algorithm type
algorithm_props	traffic algorithm properties
load_balancer_type	load balancer type
load_balancer_props	load balancer properties

Example

- Query specified traffic rule

```
SHOW TRAFFIC RULE sql_match_traffic;
```

```
mysql> SHOW TRAFFIC RULE sql_match_traffic;
+-----+-----+-----+-----+
| name           | labels | algorithm_type | algorithm_props |
| load_balancer_type | load_balancer_props |
+-----+-----+-----+-----+
| sql_match_traffic | OLTP   | SQL_MATCH     | sql=SELECT * FROM t_order WHERE order_id = 1; UPDATE t_order SET order_id = 5; |
| RANDOM          |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

- Query all traffic rules

```
SHOW TRAFFIC RULES;
```

```
mysql> SHOW TRAFFIC RULES;
+-----+-----+-----+-----+
| name          | labels | algorithm_type | algorithm_props |
| load_balancer_type | load_balancer_props |
+-----+-----+-----+-----+
| sql_match_traffic | OLTP   | SQL_MATCH      | sql=SELECT * FROM t_order WHERE order_id = 1; UPDATE t_order SET order_id = 5; | RANDOM |
+-----+-----+-----+-----+
1 row in set (0.04 sec)
```

Reserved word

SHOW, TRAFFIC, RULE, RULES

Related links

- [Reserved word](#)

ALTER TRAFFIC RULE

Description

The ALTER TRAFFIC RULE syntax is used to alter dual routing rule.

Syntax

```
AlterTrafficRule ::=
  'ALTER' 'TRAFFIC' 'RULE' '(' 'LABELS' '(' lableName ')' ','
  trafficAlgorithmDefinition ',' loadBalancerDefinition ')'

lableName ::=
  identifier

trafficAlgorithmDefinition ::=
```

```

    'TRAFFIC_ALGORITHM' '(' 'TYPE' '(' 'NAME' '=' trafficAlgorithmTypeName (','
propertiesDefinition)? ')' ')'

loadBalancerDefinition ::=
    'LOAD_BALANCER' '(' 'TYPE' '(' 'NAME' '=' loadBalancerName (','
propertiesDefinition)? ')' ')'

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

trafficAlgorithmTypeName ::=
    string

loadBalancerTypeName ::=
    string

key ::=
    string

value ::=
    literal

```

Supplement

- TRAFFIC_ALGORITHM support SQL_MATCH and SQL_HINT two types;
- LOAD_BALANCER support RANDOM and ROUND_ROBIN two types.

Example

- Alter dual routing rule

```

TRAFFIC RULE sql_match_traffic (
  LABELS (OLTP),
  TRAFFIC_ALGORITHM(TYPE(NAME="SQL_MATCH",PROPERTIES("sql" = "SELECT * FROM t_order
WHERE order_id = 1; UPDATE t_order SET order_id = 5;"))),
  LOAD_BALANCER(TYPE(NAME="RANDOM"));

```

Reserved word

ALTER, TRAFFIC, RULE, LABELS, TYPE, NAME, PROPERTIES, TRAFFIC_ALGORITHM, LOAD_BALANCER

Related links

- [Reserved word](#)

SHOW SQL_FEDERATION RULE

Description

The SHOW SQL_FEDERATION RULE syntax is used to query the federated query configuration.

Syntax

```
ShowSQLFederationRule ::=  
  'SHOW' 'SQL_FEDERATION' 'RULE'
```

Return Value Description

Column	Description
sql_federation_enabled	SQL federation enabled configuration
all_query_use_sql_federation	all query use SQL federation configuration
execution_plan_cache	execution plan cache configuration

Example

- Query sql federation rule configuration

```
SHOW SQL_FEDERATION RULE;
```

```
mysql> show sql_federation rule;  
+-----+-----+-----+  
| sql_federation_enabled | all_query_use_sql_federation | execution_plan_cache  
|  
+-----+-----+-----+  
| true                  | false                        | initialCapacity: 2000,
```

```
maximumSize: 65535 |
+-----+-----+-----+
-----+
1 row in set (0.31 sec)
```

Reserved word

SHOW、SQL_FEDERATION、RULE

Related links

- [Related links](#)

ALTER SQL_FEDERATION RULE

Description

The ALTER SQL_FEDERATION RULE syntax is used to modify the federated query configuration.

Syntax

```
AlterSQLFederationRule ::=
    'ALTER' 'SQL_FEDERATION' 'RULE' sqlFederationRuleDefinition

sqlFederationRuleDefinition ::=
    '(' sqlFederationEnabled? allQueryUseSQLFederation? ('?' executionPlanCache)? ')'

sqlFederationEnabled ::=
    'SQL_FEDERATION_ENABLED' '=' boolean_

allQueryUseSQLFederation ::=
    'ALL_QUERY_USE_SQL_FEDERATION' '=' boolean_

executionPlanCache ::=
    'EXECUTION_PLAN_CACHE' '(' cacheOption ')'

cacheOption ::=
    ('INITIAL_CAPACITY' '=' initialCapacity)? ('?' 'MAXIMUM_SIZE' '=' maximumSize)?

initialCapacity ::=
    int

maximumSize ::=
    int
```

```
boolean_ ::=
    TRUE | FALSE
```

Example

- Alter SQL Federation rule

```
ALTER SQL_FEDERATION RULE (SQL_FEDERATION_ENABLED=TRUE ALL_QUERY_USE_SQL_
FEDERATION=TRUE EXECUTION_PLAN_CACHE(INITIAL_CAPACITY=1024 MAXIMUM_SIZE=65535));
```

Reserved word

ALTER,SQL_FEDERATION,RULE,SQL_FEDERATION_ENABLED,ALL_QUERY_USE_SQL_FEDERATION,EXECUTION_PLAN_CACHE、INITIAL_CAPACITY、MAXIMUM_SIZE

Related links

- [Related links](#)

CIRCUIT BREAKER

This chapter describes the syntax of Circuit Breaker.

ALTER READWRITE_SPLITTING RULE ENABLE/DISABLE

Description

The ALTER READWRITE_SPLITTING RULE ENABLE/DISABLE syntax is used enable/disable a specified read source for specified readwrite-splitting rule.

Syntax

```
AlterReadwriteSplittingRule ::=
    'ALTER' 'READWRITE_SPLITTING' 'RULE' groupName ('ENABLE' | 'DISABLE')
storageUnitName 'FROM' databaseName

groupName ::=
    identifier

storageUnitName ::=
    identifier
```



```
databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Example

- Disable a specified read source for specified readwrite-splitting rule in specified database

```
ALTER READWRITE_SPLITTING RULE ms_group_0 DISABLE read_ds_0 FROM sharding_db;
```

- Enable a specified read source for specified readwrite-splitting rule in specified database

```
ALTER READWRITE_SPLITTING RULE ms_group_0 ENABLE read_ds_0 FROM sharding_db;
```

- Disable a specified read source for specified readwrite-splitting rule in current database

```
ALTER READWRITE_SPLITTING RULE ms_group_0 DISABLE read_ds_0;
```

- Enable a specified read source for specified readwrite-splitting rule in current database

```
ALTER READWRITE_SPLITTING RULE ms_group_1 ENABLE read_ds_0;
```

Reserved word

ALTER, READWRITE_SPLITTING, RULE, ENABLE, DISABLE

Related links

- [Reserved word](#)

SHOW STATUS FROM READWRITE_SPLITTING RULE

Description

The SHOW STATUS FROM READWRITE_SPLITTING RULE syntax is used to query readwrite-splitting storage unit status for specified readwrite-splitting rule in specified database.

Syntax

```
ShowStatusFromReadwriteSplittingRule ::=
    'SHOW' 'STATUS' 'FROM' 'READWRITE_SPLITTING' ('RULES' | 'RULE' groupName) ('FROM'
databaseName)?

groupName ::=
    identifier

databaseName ::=
    identifier
```

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Return Value Description

Columns	Description
storage_unit	storage unit name
status	storage unit status

Example

- Query readwrite-splitting storage unit status for specified readwrite-splitting rule in specified database.

```
SHOW STATUS FROM READWRITE_SPLITTING RULE ms_group_0 FROM sharding_db;
```

```
mysql> SHOW STATUS FROM READWRITE_SPLITTING RULE ms_group_0 FROM sharding_db;
+-----+-----+
| storage_unit | status |
+-----+-----+
| ds_0        | disabled |
+-----+-----+
1 rows in set (0.01 sec)
```

- Query all readwrite-splitting storage unit from specified database

```
SHOW STATUS FROM READWRITE_SPLITTING RULES FROM sharding_db;
```

```
mysql> SHOW STATUS FROM READWRITE_SPLITTING RULES FROM sharding_db;
+-----+-----+
| storage_unit | status |
+-----+-----+
| ds_0         | disabled |
+-----+-----+
1 rows in set (0.01 sec)
```

- Query readwrite-splitting storage unit status for specified readwrite-splitting rule in current database

```
SHOW STATUS FROM READWRITE_SPLITTING RULE ms_group_0;
```

```
mysql> SHOW STATUS FROM READWRITE_SPLITTING RULE ms_group_0;
+-----+-----+
| storage_unit | status |
+-----+-----+
| ds_0         | disabled |
+-----+-----+
1 rows in set (0.01 sec)
```

- Query all readwrite-splitting storage unit from current database

```
mysql> SHOW STATUS FROM READWRITE_SPLITTING RULES;
```

```
mysql> SHOW STATUS FROM READWRITE_SPLITTING RULES;
+-----+-----+
| storage_unit | status |
+-----+-----+
| ds_0         | disabled |
+-----+-----+
1 rows in set (0.01 sec)
```

Reserved word

SHOW, STATUS, FROM, READWRITE_SPLITTING, RULE, RULES

Related links

- [Reserved word](#)

SHOW COMPUTE NODES

Description

The SHOW COMPUTE NODES syntax is used to query compute nodes information. ### Syntax

```
ShowComputeNodes ::=
    'SHOW' 'COMPUTE' 'NODES'
```

Return Value Description

Columns	Description
instance_id	instance id
instance_type	instance type
host	host
port	port
status	status
mode_type	mode type
worker_id	worker id
labels	labels
version	version
database_name	database name (Only valid for JDBC)

Example

```
mysql> SHOW COMPUTE NODES;
+-----+-----+-----+-----+-----+-----+
| instance_id | instance_type | host | port | status | mode_type | worker_id | labels | version | database_name |
+-----+-----+-----+-----+-----+-----+
| 3e84d33e-cb97-42f2-b6ce-f78fea0ded89 | PROXY | 127.0.0.1 | 3307 | OK | Cluster | -1 | | 5.4.2 | logic_db |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

Dedicated Terminology

SHOW, COMPUTE, NODES

Related links

- [Reserved word](#)

ENABLE/DISABLE COMPUTE NODE

Description

The ENABLE/DISABLE COMPUTE NODE syntax is used enable/disable a specified proxy instance

Syntax

```
EnableDisableComputeNode ::=  
  ('ENABLE' | 'DISABLE') 'COMPUTE' 'NODE' instanceId  
  
instanceId ::=  
  string
```

Supplement

- instanceId needs to be obtained through [SHOW COMPUTE NODES](#) syntax query
- The currently in-use proxy instance cannot be disabled

Example

- Disable a specified proxy instance

```
DISABLE COMPUTE NODE '734bb086-b15d-4af0-be87-2372d8b6a0cd';
```

- Enable a specified proxy instance

```
ENABLE COMPUTE NODE '734bb086-b15d-4af0-be87-2372d8b6a0cd';
```

Reserved word

ENABLE, DISABLE, COMPUTE, NODE

Related links

- [Reserved word](#)
- [SHOW COMPUTE NODES](#)

LABEL|RELABEL COMPUTE NODES

Description

The LABEL|RELABEL COMPUTE NODES syntax is used to label PROXY instance.

Syntax

```

LabelRelabelComputeNodes ::=
  ('LABEL' | 'RELABEL') 'COMPUTE' 'NODE' instance_id 'WITH' labelName

instance_id ::=
  string

labelName ::=
  identifier

```

Supplement

- needs to be obtained through [SHOW COMPUTE NODES](#) syntax query
- RELABEL is used to relabel PROXY instance

Example

- Label PROXY instance

```
LABEL COMPUTE NODE "0699e636-ade9-4681-b37a-65240c584bb3" WITH label_1;
```

- Relabel PROXY instance

```
RELABEL COMPUTE NODE "0699e636-ade9-4681-b37a-65240c584bb3" WITH label_2;
```

Reserved word

LABEL, RELABEL, COMPUTE, NODES, WITH

Related links

- [Reserved word](#)
- [SHOW COMPUTE NODES](#)

UNLABEL COMPUTE NODES

Description

The UNLABEL COMPUTE NODES syntax is used to remove specified label from PROXY instance.

Syntax

```
UnlabelComputeNode ::=
    'UNLABEL' 'COMPUTE' 'NODE' instance_id 'WITH' labelName

instance_id ::=
    string

labelName ::=
    identifier
```

Supplement

- needs to be obtained through [SHOW COMPUTE NODES](#) syntax query

Example

- Remove specified label from PROXY instance

```
UNLABEL COMPUTE NODE "0699e636-ade9-4681-b37a-65240c584bb3" WITH label_1;
```

Reserved word

UNLABEL, COMPUTE, NODES, WITH

Related links

- [Reserved word](#)
- [SHOW COMPUTE NODES](#)

MIGRATION

This chapter describes the syntax of migration.

SHOW MIGRATION RULE

Description

The SHOW MIGRATION RULE syntax is used to query migration rule.

Syntax

```
ShowMigrationRule ::=
    'SHOW' 'MIGRATION' 'RULE'
```

Return Value Description

Column	Description
read	Data reading configuration
write	Data writing configuration
stream_channel	Data channel

Example

- Query migration rule

```
SHOW MIGRATION RULE;
```

```
mysql> SHOW MIGRATION RULE;
```

```
+-----+-----+
+-----+-----+
```



```
| read | write
| stream_channel |
+-----+-----+
+-----+-----+
| {"workerThread":20,"batchSize":1000,"shardingSize":10000000} | {"workerThread":20,
"batchSize":1000} | {"type":"MEMORY","props":{"block-queue-size":"2000"}} |
+-----+-----+
+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

SHOW, MIGRATION, RULE

Related links

- [Reserved word](#)

ALTER MIGRATION RULE

Description

The ALTER MIGRATION RULE syntax is used to alter migration rule.

Syntax

```
AlterMigrationRule ::=
  'ALTER' 'MIGRATION' 'RULE' '(' (readConfiguration ',')? (writeConfiguration ',')? (dataChannel)? ')'?

readConfiguration ::=
  'READ' '(' ('WORKER_THREAD' '=' workerThreadPoolSize ',')? ('BATCH_SIZE' '=' batchSize ',')? ('SHARDING_SIZE' '=' shardingSize ',')? (rateLimiter)? ')'

writeConfiguration ::=
  'WRITE' '(' ('WORKER_THREAD' '=' workerThreadPoolSize ',')? ('BATCH_SIZE' '=' batchSize ',')? ('SHARDING_SIZE' '=' shardingSize ',')? (rateLimiter)? ')'

dataChannel ::=
  'STREAM_CHANNEL' '(' 'TYPE' '(' 'NAME' '=' algorithmName ',' propertiesDefinition ')' ')'

workerThreadPoolSize ::=
  int
```

```

batchSize ::=
    int

shardingSize ::=
    int

rateLimiter ::=
    'RATE_LIMITER' '(' 'TYPE' '(' 'NAME' '=' algorithmName ',' propertiesDefinition ')'
    ' )'

algorithmName ::=
    string

propertiesDefinition ::=
    'PROPERTIES' '(' key '=' value (',' key '=' value)* ')'

key ::=
    string

value ::=
    literal

```

Example

```

ALTER MIGRATION RULE (
    READ( WORKER_THREAD=20, BATCH_SIZE=1000, SHARDING_SIZE=10000000, RATE_LIMITER
    (TYPE(NAME='QPS',PROPERTIES('qps'='500')))),
    WRITE( WORKER_THREAD=20, BATCH_SIZE=1000, RATE_LIMITER (TYPE(NAME='TPS',
    PROPERTIES('tps'='2000')))),
    STREAM_CHANNEL ( TYPE(NAME='MEMORY',PROPERTIES('block-queue-size'='2000'))))
);

```

Reserved word

ALTER, MIGRATION, RULE, READ, WRITE, WORKER_THREAD, BATCH_SIZE, SHARDING_SIZE, STREAM_CHANNEL, TYPE, NAME, PROPERTIES

Related links

- [Reserved word](#)

REGISTER MIGRATION SOURCE STORAGE UNIT

Description

The REGISTER MIGRATION SOURCE STORAGE UNIT syntax is used to register migration source storage unit for the currently connection.

Syntax

```
RegisterStorageUnit ::=
    'REGISTER' 'MIGRATION' 'SOURCE' 'STORAGE' 'UNIT' storageUnitDefinition (','
storageUnitDefinition)*

storageUnitDefinition ::=
    StorageUnitName '(' 'URL' '=' url ',' 'USER' '=' user ',' 'PASSWORD' '='
password)? (',' propertiesDefinition)? ')'

storageUnitName ::=
    identifier

url ::=
    string

user ::=
    string

password ::=
    string

propertiesDefinition ::=
    'PROPERTIES' '(' ( key '=' value ) ( ',' key '=' value )* ')'

key ::=
    string

value ::=
    literal
```

Supplement

- Confirm that the registered migration source storage unit can be connected normally, otherwise it will not be added successfully;
- `storageUnitName` is case-sensitive;
- `storageUnitName` needs to be unique within the current connection;
- `storageUnitName` name only allows letters, numbers and `_`, and must start with a letter;
- `poolProperty` is used to customize connection pool parameters, key must be the same as the connection pool parameter name, value supports `int` and `String` types;
- When password contains special characters, it is recommended to use the string form; For example, the string form of `password@123` is `"password@123"`.
- The data migration source storage unit currently only supports registration using URL, and temporarily does not support using `HOST` and `PORT`.

Example

- Register migration source storage unit

```
REGISTER MIGRATION SOURCE STORAGE UNIT ds_0 (
    URL="jdbc:mysql://127.0.0.1:3306/migration_ds_0?serverTimezone=UTC&useSSL=false",
    USER="root",
    PASSWORD="root"
);
```

- Register migration source storage unit and set connection pool parameters

```
REGISTER MIGRATION SOURCE STORAGE UNIT ds_0 (
    URL="jdbc:mysql://127.0.0.1:3306/migration_ds_0?serverTimezone=UTC&useSSL=false",
    USER="root",
    PASSWORD="root",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
);
```

Reserved word

REGISTER, MIGRATION, SOURCE, STORAGE, UNIT, USER, PASSWORD, PROPERTIES, URL

Related links

- [Reserved word](#)

UNREGISTER MIGRATION SOURCE STORAGE UNIT

Description

The UNREGISTER MIGRATION SOURCE STORAGE UNIT syntax is used to unregister migration source storage unit from the current connection

Syntax

```
UnregisterMigrationSourceStorageUnit ::=
    'UNREGISTER' 'MIGRATION' 'SOURCE' 'STORAGE' 'UNIT' storageUnitName (','
storageUnitName)*

storageUnitName ::=
    identifier
```

Supplement

- UNREGISTER MIGRATION SOURCE STORAGE UNIT will only unregister storage unit in Proxy, the real data source corresponding to the storage unit will not be dropped;

Example

- Drop a migration source storage unit

```
UNREGISTER MIGRATION SOURCE STORAGE UNIT ds_0;
```

- Drop multiple migration source storage units

```
UNREGISTER MIGRATION SOURCE STORAGE UNIT ds_1, ds_2;
```

Reserved word

UNREGISTER、MIGRATION、SOURCE、STORAGE、UNIT

Related links

- [Reserved word](#)

SHOW MIGRATION SOURCE STORAGE UNITS

Description

The `SHOW MIGRATION SOURCE STORAGE UNITS` syntax is used to query the registered migration source storage units

Syntax

```
ShowStorageUnit ::=
'SHOW' 'MIGRATION' 'SOURCE' 'STORAGE' 'UNITS'
```

Return Value Description

Column	Description
name	Storage unit name
type	Storage unit type
host	Storage unit host
port	Storage unit port
db	Database name
attribute	Storage unit attribute

Example

- Query registered migration source storage units

```
SHOW MIGRATION SOURCE STORAGE UNITS;
```

```
mysql> SHOW MIGRATION SOURCE STORAGE UNITS;
+-----+-----+-----+-----+-----+-----+-----+-----+
| name | type | host      | port | db           | connection_timeout_milliseconds |
| idle_timeout_milliseconds | max_lifetime_milliseconds | max_pool_size | min_pool_size | read_only | other_attributes |
+-----+-----+-----+-----+-----+-----+-----+-----+
| ds_1 | MySQL | 127.0.0.1 | 3306 | migration_ds_0 |
```

+	+	+	+	+	+
+	+	+	+	+	+
+	+	+	+	+	+
1 row in set (0.01 sec)					

Reserved word

SHOW, MIGRATION, SOURCE, STORAGE, UNITS

Related links

- [Reserved word](#)

MIGRATE TABLE INTO

Description

MIGRATE TABLE INTO syntax is used to migration table from source to target

Syntax

```
MigrateTableInto ::=
    'MIGRATE' 'TABLE' migrationSource '.' tableName 'INTO' (databaseName '.')?
    tableName

migrationSource ::=
    identifier

databaseName ::=
    identifier

tableName ::=
    identifier
```

Supplement

- When `databaseName` is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Example

- Migrate table from source to current database

```
MIGRATE TABLE ds_0.t_order INTO t_order;
```

- Migrate table from source to specified database

```
UNREGISTER MIGRATION SOURCE STORAGE UNIT ds_1, ds_2;
```

Reserved word

MIGRATE, TABLE, INTO

Related links

- [Reserved word](#)

SHOW MIGRATION LIST

Description

The SHOW MIGRATION LIST syntax is used to query migration job list.

Syntax

```
ShowMigrationList ::=  
  'SHOW' 'MIGRATION' 'LIST'
```


Return Values Description

Columns	Description
id	migration job id
tables	migration tables
job_item_count	migration job sharding number
active	migration job states
create_time	migration job create time
stop_time	migration job stop time
job_sharding_nodes	migration job sharding nodes

Example

- Query migration job list

```
SHOW MIGRATION LIST;
```

```
mysql> SHOW MIGRATION LIST;
+-----+-----+-----+-----+-----+-----+
| id                                     | tables               | active | create_ |
time          | stop_time | job_item_count | job_sharding_nodes |
+-----+-----+-----+-----+-----+-----+
| j0102p00001d029afca1fd960d567fed6cddc9b4a2 | source_ds.t_order   | true   | 2022-  |
10-31 18:18:24 |          | 1          | 10.7.5.76@-@27808 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.06 sec)
```

Reserved word

SHOW, MIGRATION, LIST

Related links

- [Reserved word](#)

SHOW MIGRATION STATUS

Description

The `SHOW MIGRATION STATUS` syntax is used to query migration job status for specified migration job.

Syntax

```
ShowMigrationStatus ::=
    'SHOW' 'MIGRATION' 'STATUS' migrationJobId

migrationJobId ::=
    string
```

Supplement

- `migrationJobId` needs to be obtained through `SHOW MIGRATION LIST` syntax query

Return Value Description

column	Description
item	migration job sharding serial number
data source	migration source
status	migration job status
processed_records_count	number of processed rows
inventory_finished_percentage	finished percentage of migration job
incremental_idle_seconds	incremental idle time
error_message	error message

Example

- Query migration job status

```
SHOW MIGRATION STATUS 'j010180026753ef0e25d3932d94d1673ba551';
```

```
mysql> SHOW MIGRATION STATUS 'j010180026753ef0e25d3932d94d1673ba551';
+-----+-----+-----+-----+-----+
| item | data_source | status | active | processed_records_count |
inventory_finished_percentage | incremental_idle_seconds | error_message |
```

+-----+-----+-----+-----+-----+					
+-----+-----+-----+-----+-----+					
0	ds_1	EXECUTE_INCREMENTAL_TASK	true	6	
100		25			
+-----+-----+-----+-----+-----+					
+-----+-----+-----+-----+-----+					
1 row in set (0.01 sec)					

Reserved word

SHOW, MIGRATION, STATUS

Related links

- [Reserved word](#)

SHOW MIGRATION CHECK ALGORITHM

Description

The SHOW MIGRATION RULE syntax is used to query migration check algorithm.

Syntax

```
ShowMigrationCheckAlgorithm ::=
  'SHOW' 'MIGRATION' 'CHECK' 'ALGORITHMS'
```

Return Value Description

Column	Description
type	migration check algorithm type
supported_database_types	supported database type
description	Description of migration check algorithm

Example

- Query migration check algorithm

```
SHOW MIGRATION CHECK ALGORITHMS;
```

```
mysql> SHOW MIGRATION CHECK ALGORITHMS;
+-----+-----+-----+
| type          | supported_database_types          |
description          |
+-----+-----+-----+
| CRC32_MATCH   | MySQL                             | Match
CRC32 of records.   |
| DATA_MATCH   | SQL92,MySQL,MariaDB,PostgreSQL,openGauss,Oracle,SQLServer,H2 | Match
raw data of records. |
+-----+-----+-----+
2 rows in set (0.03 sec)
```

Reserved word

SHOW, MIGRATION, CHECK, ALGORITHMS

Related links

- [Reserved word](#)

CHECK MIGRATION

Description

The CHECK MIGRATION LIST syntax is used to check data consistency in migration job.

Syntax

```
ShowMigrationList ::=
    'CHECK' 'MIGRATION' migrationJobId 'BY' 'TYPE' '(' 'NAME' '='
migrationCheckAlgorithmType ')'

migrationJobId ::=
    string
```

```
migrationCheckAlgorithmType ::=  
    string
```

Supplement

- migrationJobId needs to be obtained through [SHOW MIGRATION LIST](#) syntax query
- migrationCheckAlgorithmType needs to be obtained through [SHOW MIGRATION CHECK ALGORITHMS](#) syntax query

Example

- check data consistency in migration job

```
CHECK MIGRATION 'j01016e501b498ed1bdb2c373a2e85e2529a6' BY TYPE (NAME='CRC32_MATCH  
';
```

Reserved word

CHECK, MIGRATION, BY, TYPE

Related links

- [Reserved word](#)
- [SHOW MIGRATION LIST](#)
- [SHOW MIGRATION CHECK ALGORITHMS](#)

SHOW MIGRATION CHECK STATUS

Description

The `SHOW MIGRATION CHECK STATUS` syntax is used to query migration check status for specified migration job.

Syntax

```
ShowMigrationCheckStatus ::=
    'SHOW' 'MIGRATION' 'CHECK' 'STATUS' migrationJobId

migrationJobId ::=
    string
```

Supplement

- migrationJobId needs to be obtained through `SHOW MIGRATION LIST` syntax query

Return Value Description

Columns	Description
tables	migration check table
result	check result
finished_percentage	check finished finished_percentage
remaining_seconds	check remaining time
check_begin_time	check begin time
check_end_time	check end time
error_message	error message

Example

- Query migration check status

```
SHOW MIGRATION CHECK STATUS 'j010180026753ef0e25d3932d94d1673ba551';
```

```
mysql> SHOW MIGRATION CHECK STATUS 'j010180026753ef0e25d3932d94d1673ba551';
+-----+-----+-----+-----+-----+
| tables | result | finished_percentage | remaining_seconds | check_begin_time |
| check_end_time | duration_seconds | error_message |
+-----+-----+-----+-----+-----+
| t_order | true | 100 | 0 | 2022-11-01 17:57:39.940 |
| 2022-11-01 17:57:40.587 | 0 | | |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

SHOW, MIGRATION, CHECK, STATUS

Related links

- [Reserved word](#)
- [SHOW MIGRATION LIST](#)

START MIGRATION CHECK

Description

The START MIGRATION CHECK syntax is used to stop migration check process.

Syntax

```
StartMigrationCheck ::=  
    'START' 'MIGRATION' 'CHECK' migrationJobId  
  
migrationJobId ::=  
    string
```

Supplement

- migrationJobId needs to be obtained through [SHOW MIGRATION LIST](#) syntax query

Example

- Stop migration check process

```
START MIGRATION CHECK 'j010180026753ef0e25d3932d94d1673ba551';
```

Reserved word

START, MIGRATION, CHECK

Related links

- [Reserved word](#)
- [SHOW MIGRATION LIST](#)

STOP MIGRATION CHECK

Description

The STOP MIGRATION CHECK syntax is used to stop migration check process.

Syntax

```
StopMigrationCheck ::=  
    'STOP' 'MIGRATION' 'CHECK' migrationJobId  
  
migrationJobId ::=  
    string
```

Supplement

- migrationJobId needs to be obtained through [SHOW MIGRATION LIST](#) syntax query

Example

- Stop migration check process

```
STOP MIGRATION CHECK 'j010180026753ef0e25d3932d94d1673ba551';
```

Reserved word

STOP, MIGRATION, CHECK

Related links

- [Reserved word](#)
- [SHOW MIGRATION LIST](#)

START MIGRATION

Description

The `START MIGRATION` syntax is used to start migration process.

Syntax

```
StartMigration ::=  
  'START' 'MIGRATION' migrationJobId  
  
migrationJobId ::=  
  string
```

Supplement

- migrationJobId needs to be obtained through `SHOW MIGRATION LIST` syntax query

Example

- Start migration process

```
START MIGRATION 'j010180026753ef0e25d3932d94d1673ba551';
```

Reserved word

START, MIGRATION

Related links

- [Reserved word](#)
- [SHOW MIGRATION LIST](#)

STOP MIGRATION

Description

The `STOP MIGRATION` syntax is used to stop migration process.

Syntax

```
StopMigration ::=  
    'STOP' 'MIGRATION' migrationJobId  
  
migrationJobId ::=  
    string
```

Supplement

- migrationJobId needs to be obtained through [SHOW MIGRATION LIST](#) syntax query

Example

- Stop migration process

```
STOP MIGRATION 'j010180026753ef0e25d3932d94d1673ba551';
```

Reserved word

STOP, MIGRATION

Related links

- [Reserved word](#)
- [SHOW MIGRATION LIST](#)

COMMIT MIGRATION

Description

The COMMIT MIGRATION syntax is used to commit migration process.

Syntax

```
CommitMigration ::=  
    'COMMIT' 'MIGRATION' migrationJobId  
  
migrationJobId ::=  
    string
```

Supplement

- migrationJobId needs to be obtained through `SHOW MIGRATION LIST` syntax query

Example

- Commit migration process

```
COMMIT MIGRATION 'j010180026753ef0e25d3932d94d1673ba551';
```

Reserved word

COMMIT, MIGRATION

Related links

- [Reserved word](#)
- [SHOW MIGRATION LIST](#)

ROLLBACK MIGRATION

Description

The ROLLBACK MIGRATION syntax is used to rollback migration process.

Syntax

```
RollbackMigration ::=  
    'ROLLBACK' 'MIGRATION' migrationJobId  
  
migrationJobId ::=  
    string
```

Supplement

- migrationJobId needs to be obtained through `SHOW MIGRATION LIST` syntax query
- After the statement is executed, the target will be cleaned up

Example

- Rollback migration process

```
ROLLBACK MIGRATION 'j010180026753ef0e25d3932d94d1673ba551';
```

Reserved word

ROLLBACK, MIGRATION

Related links

- [Reserved word](#)
- [SHOW MIGRATION LIST](#)

PLUGIN

This chapter describes the syntax of plugin.

SHOW PLUGINS OF SPI

Description

The `SHOW PLUGINS OF interfaceClass` syntax is used to query all the implementations of an interface.

Syntax

```
showPluginImplementations ::=  
    'SHOW' 'PLUGINS' 'OF' interfaceClass  
  
interfaceClass ::=  
    string
```

Return Value Description

Columns	Description
type	type
type_aliases	type aliases
description	description

Example

- Query all the implementations for `org.apache.shardingsphere.sharding.spi.ShardingAlgorithm` interface

```
SHOW PLUGINS OF 'org.apache.shardingsphere.sharding.spi.ShardingAlgorithm'
```

```
SHOW PLUGINS OF 'org.apache.shardingsphere.sharding.spi.ShardingAlgorithm';
```

type	type_aliases	description
MOD		
HASH_MOD		
VOLUME_RANGE		
BOUNDARY_RANGE		
AUTO_INTERVAL		
INTERVAL		
CLASS_BASED		
INLINE		
COMPLEX_INLINE		
HINT_INLINE		

10 rows in set (0.52 sec)

Supplement

For some commonly used interface implementations, ShardingSphere provides syntax sugar functions to simplify operations.

The currently provided syntax sugar are as follows:

- Show implementations of `org.apache.shardingsphere.sharding.spi.ShardingAlgorithm`: `SHOW SHARDING ALGORITHM PLUGINS`
- Show implementations of `org.apache.shardingsphere.infra.algorithm.loadbalancer.core.LoadBalanceAlgorithm`: `SHOW LOAD BALANCE ALGORITHM PLUGINS`

- Show implementations of `org.apache.shardingsphere.encrypt.spi.EncryptAlgorithm`: [SHOW ENCRYPT ALGORITHM PLUGINS](#)
- Show implementations of `org.apache.shardingsphere.mask.spi.MaskAlgorithm`: [SHOW MASK ALGORITHM PLUGINS](#)
- Show implementations of `org.apache.shardingsphere.shadow.spi.ShadowAlgorithm`: [SHOW SHADOW ALGORITHM PLUGINS](#)
- Show implementations of `org.apache.shardingsphere.keygen.core.algorithm.KeyGenerateAlgorithm`: [SHOW KEY GENERATE ALGORITHM PLUGINS](#)

Reserved word

SHOW, PLUGINS, OF

Related links

- [Reserved word](#)

SHOW SHARDING ALGORITHM PLUGINS

Description

The `SHOW SHARDING ALGORITHM PLUGINS` syntax is used to query all the plugins of the interface `org.apache.shardingsphere.sharding.spi.ShardingAlgorithm`.

Syntax

```
showShardingAlgorithmPlugins ::=
    'SHOW' 'SHARDING' 'ALGORITHM' 'PLUGINS'
```

Return Value Description

Columns	Description
type	type
type_aliases	type aliases
description	description

Example

- Query all the plugins for `org.apache.shardingsphere.sharding.spi.ShardingAlgorithm` interface

```
SHOW SHARDING ALGORITHM PLUGINS
```

```
SHOW SHARDING ALGORITHM PLUGINS;
+-----+-----+-----+
| type          | type_aliases | description |
+-----+-----+-----+
| MOD           |              |             |
| HASH_MOD      |              |             |
| VOLUME_RANGE  |              |             |
| BOUNDARY_RANGE|              |             |
| AUTO_INTERVAL|              |             |
| INTERVAL      |              |             |
| CLASS_BASED   |              |             |
| INLINE        |              |             |
| COMPLEX_INLINE|              |             |
| HINT_INLINE   |              |             |
+-----+-----+-----+
10 rows in set (0.27 sec)
```

Reserved word

SHOW, SHARDING, ALGORITHM, PLUGINS

Related links

- [Reserved word](#)

SHOW LOAD BALANCE ALGORITHM PLUGINS

Description

The `SHOW LOAD BALANCE ALGORITHM PLUGINS` syntax is used to query all the implementations of the interface `org.apache.shardingsphere.infra.algorithm.loadbalancer.core.LoadBalanceAlgorithm`.

Syntax

```
showLoadBalanceAlgorithmPlugins ::=
    'SHOW' 'LOAD' 'BALANCE' 'ALGORITHM' 'PLUGINS'
```

Return Value Description

Columns	Description
type	type
type_aliases	type aliases
description	description

Example

- Query all the implementations for `org.apache.shardingsphere.infra.algorithm.loadbalancer.core.LoadBalanceAlgorithm` interface

```
SHOW LOAD BALANCE ALGORITHM PLUGINS
```

```
SHOW LOAD BALANCE ALGORITHM PLUGINS;
+-----+-----+-----+
| type      | type_aliases | description |
+-----+-----+-----+
| ROUND_ROBIN |              |              |
| RANDOM      |              |              |
| WEIGHT      |              |              |
+-----+-----+-----+
3 rows in set (0.03 sec)
```

Reserved word

SHOW, LOAD, BALANCE, ALGORITHM, PLUGINS

Related links

- [Reserved word](#)

SHOW ENCRYPT ALGORITHM PLUGINS

Description

The `SHOW ENCRYPT ALGORITHM PLUGINS` syntax is used to query all the implementations of the interface `org.apache.shardingsphere.encrypt.spi.EncryptAlgorithm`.

Syntax

```
showEncryptAlgorithmPlugins ::=  
    'SHOW' 'ENCRYPT' 'ALGORITHM' 'PLUGINS'
```

Return Value Description

Columns	Description
type	type
type_aliases	type aliases
description	description

Example

- Query all the implementations for `org.apache.shardingsphere.encrypt.spi.EncryptAlgorithm` interface

```
SHOW ENCRYPT ALGORITHM PLUGINS
```

```
SHOW ENCRYPT ALGORITHM PLUGINS;  
+-----+-----+-----+  
| type | type_aliases | description |  
+-----+-----+-----+  
| AES  |              |             |  
| MD5  |              |             |  
+-----+-----+-----+  
2 rows in set (0.06 sec)
```

Reserved word

SHOW, ENCRYPT, ALGORITHM, PLUGINS

Related links

- [Reserved word](#)

SHOW MASK ALGORITHM PLUGINS

Description

The SHOW MASK ALGORITHM PLUGINS syntax is used to query all the implementations of the interface `org.apache.shardingsphere.mask.spi.MaskAlgorithm`.

Syntax

```
showMaskAlgorithmPlugins ::=
    'SHOW' 'MASK' 'ALGORITHM' 'PLUGINS'
```

Return Value Description

Columns	Description
type	type
type_aliases	type aliases
description	description

Example

- Query all the implementations for `org.apache.shardingsphere.mask.spi.MaskAlgorithm` interface

```
SHOW MASK ALGORITHM PLUGINS
```

```
SHOW MASK ALGORITHM PLUGINS;
```

```
+-----+-----+-----+
| type           | type_aliases | description |
+-----+-----+-----+
| MD5            |              |            |
| KEEP_FIRST_N_LAST_M |            |            |
| KEEP_FROM_X_TO_Y |            |            |
```

MASK_AFTER_SPECIAL_CHARS			
MASK_BEFORE_SPECIAL_CHARS			
MASK_FIRST_N_LAST_M			
MASK_FROM_X_TO_Y			
GENERIC_TABLE_RANDOM_REPLACE			
+-----+-----+-----+			
8 rows in set (0.13 sec)			

Reserved word

SHOW, MASK, ALGORITHM, PLUGINS

Related links

- [Reserved word](#)

SHOW SHADOW ALGORITHM PLUGINS

Description

The SHOW SHADOW ALGORITHM PLUGINS syntax is used to query all the implementations of the interface `org.apache.shardingsphere.shadow.spi.ShadowAlgorithm`.

Syntax

```
showShadowAlgorithmPlugins ::=
    'SHOW' 'SHADOW' 'ALGORITHM' 'PLUGINS'
```

Return Value Description

Columns	Description
type	type
type_aliases	type aliases
description	description

Example

- Query all the implementations for `org.apache.shardingsphere.shadow.spi.ShadowAlgorithm` interface

```
SHOW SHADOW ALGORITHM PLUGINS
```

```
SHOW SHADOW ALGORITHM PLUGINS;
+-----+-----+-----+
| type      | type_aliases | description |
+-----+-----+-----+
| SQL_HINT  |              |              |
| REGEX_MATCH |              |              |
| VALUE_MATCH |              |              |
+-----+-----+-----+
3 rows in set (0.37 sec)
```

Reserved word

SHOW, SHADOW, ALGORITHM, PLUGINS

Related links

- [Reserved word](#)

SHOW KEY GENERATE ALGORITHM PLUGINS

Description

The "SHOW KEY GENERATE ALGORITHM PLUGINS" syntax is used to query all the implementations of the interface `org.apache.shardingsphere.keygen.core.algorithm.KeyGenerateAlgorithm`.

Syntax

```
showKeyGenerateAlgorithmPlugins ::=
    'SHOW' 'KEY' 'GENERATE' 'ALGORITHM' 'PLUGINS'
```

Return Value Description

Columns	Description
type	type
type_aliases	type aliases
description	description

Example

- Query all the implementations for `org.apache.shardingsphere.keygen.core.algorithm.KeyGenerateAlgorithm` interface

```
SHOW KEY GENERATE ALGORITHM PLUGINS
```

```
SHOW KEY GENERATE ALGORITHM PLUGINS;
+-----+-----+-----+
| type      | type_aliases | description |
+-----+-----+-----+
| UUID      |              |             |
| SNOWFLAKE |              |             |
+-----+-----+-----+
2 rows in set (0.05 sec)
```

Reserved word

SHOW, KEY, GENERATE, ALGORITHM, PLUGINS

Related links

- [Reserved word](#)

SHOW COMPUTE NODE INFO

Description

The `SHOW COMPUTE NODE INFO` syntax is used to query current proxy instance information. ### Syntax

```
ShowComputeNodeInfo ::=
    'SHOW' 'COMPUTE' 'NODE' 'INFO'
```

Return Value Description

Columns	Description
instance_id	proxy instance id
host	host address
port	port number
status	proxy instance status
mode_type	proxy instance mode
worker_id	worker id
labels	labels
version	version

Example

- Query current proxy instance information

```
SHOW COMPUTE NODE INFO;
```

```
mysql> SHOW COMPUTE NODES;
```

```
+-----+-----+-----+-----+
+-----+-----+-----+
| instance_id          | instance_type | host      | port | status |
mode_type | worker_id | labels | version |
+-----+-----+-----+-----+
+-----+-----+-----+
| 3e84d33e-cb97-42f2-b6ce-f78fea0ded89 | PROXY          | 127.0.0.1 | 3307 | OK      |
Cluster    | -1        |          | 5.4.2  |
+-----+-----+-----+-----+
+-----+-----+-----+
1 row in set (0.01 sec)
```

Reserved word

SHOW, COMPUTE, NODE, INFO

Related links

- [Reserved word](#)

SHOW COMPUTE NODE MODE

Description

The SHOW COMPUTE NODE MODE syntax is used to query current proxy instance mode configuration information. ### Syntax

```
ShowComputeNodeMode ::=
    'SHOW' 'COMPUTE' 'NODE' 'MODE'
```

Return Value Description

Columns	Description
type	type of proxy mode configuration
repository	type of persist repository
props	properties of persist repository

Example

- Query current proxy instance mode configuration information

```
SHOW COMPUTE NODE MODE;
```

```
mysql> SHOW COMPUTE NODE MODE;
+-----+-----+-----+
| type   | repository | props |
+-----+-----+-----+
| Cluster | ZooKeeper | {"operationTimeoutMilliseconds":500,"timeToLiveSeconds":60,
"maxRetries":3,"namespace":"governance_ds","server-lists":"localhost:2181",
```

- Reserved word

```
SetDistVariable ::=
    'SET' 'DIST' 'VARIABLE' (proxyPropertyName '=' proxyPropertyValue | 'agent_
plugins_enabled' '=' agentPluginsEnabled)
```

```
proxyPropertyName ::=
    identifier
```

```
proxyPropertyValue ::=
  literal
```

```
agentPluginsEnabled ::=
    boolean
```

- `proxy_property_name` is one of `properties configuration` of PROXY, name is split by underscore
- `agent_plugins_enabled` is use to set the agent plugins enable status, the default value is `FALSE`
- `system_log_level` is the system log level, only affects the log printing of PROXY, the default value is `INFO`

Example

- Set property configuration of Proxy

```
SET DIST VARIABLE sql_show = true;
```

- Set agent plugin enable status

```
SET DIST VARIABLE agent_plugins_enabled = TRUE;
```

Reserved word

SET, DIST, VARIABLE

Related links

- [Reserved word](#)

SHOW DIST VARIABLE

Description

The SHOW DIST VARIABLE syntax is used to query PROXY system variables configuration.

Syntax

```
ShowDistVariable ::=
    'SHOW' 'DIST' ('VARIABLES' ('LIKE' likePattern)?| 'VARIABLE' 'WHERE' 'NAME' '='
variableName)

likePattern ::=
    string

variableName ::=
    identifier
```

Return Value Description

Columns	Description
variable_name	system variable name
variable_value	system variable value

Supplement

- When variableName is not specified, the default is query all PROXY variables configuration.

Example

- Query all system variables configuration of PROXY

```
SHOW DIST VARIABLES;
```

```
mysql> SHOW DIST VARIABLES;
```

```
+-----+-----+
| variable_name                | variable_value |
+-----+-----+
| agent_plugins_enabled        | true           |
| cached_connections           | 0              |
| cdc_server_port              | 33071          |
| check_table_metadata_enabled | false          |
| kernel_executor_size         | 0              |
| max_connections_size_per_query | 1              |
| proxy_backend_query_fetch_size | -1             |
| proxy_default_port           | 3307           |
| proxy_frontend_database_protocol_type |               |
| proxy_frontend_executor_size | 0              |
| proxy_frontend_flush_threshold | 128            |
| proxy_frontend_max_connections | 0              |
| proxy_frontend_ssl_cipher     |               |
| proxy_frontend_ssl_enabled    | false          |
| proxy_frontend_ssl_version    | TLSv1.2,TLSv1.3 |
| proxy_meta_data_collector_enabled | true           |
| proxy_netty_backlog           | 1024           |
| sql_federation_type          | NONE           |
| sql_show                     | false          |
| sql_simple                    | false          |
| system_log_level              | INFO           |
+-----+-----+
21 rows in set (0.01 sec)
```

- Query specified system variable configuration of PROXY

```
SHOW DIST VARIABLE WHERE NAME = sql_show;
```

```
mysql> SHOW DIST VARIABLE WHERE NAME = sql_show;
```

```
+-----+-----+
| variable_name | variable_value |
+-----+-----+
| sql_show      | false          |
+-----+-----+
1 row in set (0.00 sec)
```

Reserved word

SHOW, DIST, VARIABLE, VARIABLES, NAME

Related links

- [Reserved word](#)

REFRESH TABLE METADATA

Description

The REFRESH TABLE METADATA syntax is used to refresh table metadata.

Syntax

```
RefreshTableMetadata ::=
  'REFRESH' 'TABLE' 'METADATA' (tableName | tableName 'FROM' 'STORAGE' 'UNIT'
storageUnitName ('SCHEMA' schemaName)?)?

tableName ::=
  identifier

storageUnitName ::=
  identifier

schemaName ::=
  identifier
```

Supplement

- When `tableName` and `storageUnitName` is not specified, the default is to refresh all table metadata.
- refresh table metadata need to use `DATABASE`. If `DATABASE` is not used, No database selected will be prompted.
- If there are no tables in the schema, the schema will be deleted.

Example

- Refresh specified table's metadata in specified schema of a specified storage unit

```
REFRESH TABLE METADATA t_order FROM STORAGE UNIT ds_1 SCHEMA db_schema;
```

- Refresh all tables' metadata in specified schema of a specified storage unit

```
REFRESH TABLE METADATA FROM STORAGE UNIT ds_1 SCHEMA db_schema;
```

- Refresh metadata for specified table in specified storage unit

```
REFRESH TABLE METADATA t_order FROM STORAGE UNIT ds_1;
```

- Refresh metadata for specified table

```
REFRESH TABLE METADATA t_order;
```

- Refresh all table metadata

```
REFRESH TABLE METADATA;
```

Reserved word

REFRESH, TABLE, METADATA, FROM, STORAGE, UNIT

Related links

- [Reserved word](#)

REFRESH DATABASE METADATA FROM GOVERNANCE CENTER

Description

The REFRESH DATABASE METADATA syntax is used to refresh the metadata of the local logic database.

Syntax

```
RefreshDatabaseMetadata ::=  
    'FORCE'? 'REFRESH' 'DATABASE' 'METADATA' databaseName?  
  
databaseName ::=  
    identifier
```

Supplement

- When databaseName is not specified, the default is to refresh all database metadata.
- When using FORCE to refresh metadata, the latest metadata will be obtained locally and written to the governance center. If without FORCE, it will be pulled from the governance center.

Example

- Refresh metadata for specified database

```
REFRESH DATABASE METADATA sharding_db;
```

- Refresh all database metadata

```
REFRESH DATABASE METADATA;
```

- Force refresh all database metadata

```
FORCE REFRESH DATABASE METADATA;
```

Reserved word

FORCE, REFRESH, DATABASE, METADATA

Related links

- [Reserved word](#)

SHOW TABLE METADATA

Description

The SHOW TABLE METADATA syntax is used to query table metadata.

Syntax

```
ShowTableMetadata ::=  
    'SHOW' 'TABLE' 'METADATA' tableName (',' tableName)* ('FROM' databaseName)?  
  
tableName ::=  
    identifier  
  
databaseName ::=  
    identifier
```

Return Value Description

Columns	Description
schema_name	database name
table_name	table name
type	metadata type
name	metadata name

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Example

- Query metadata of multiple tables from specified database

```
SHOW TABLE METADATA t_order, t_order_1 FROM sharding_db;
```

```
mysql> SHOW TABLE METADATA t_order, t_order_1 FROM sharding_db;
+-----+-----+-----+-----+
| schema_name | table_name | type   | name   |
+-----+-----+-----+-----+
| sharding_db | t_order_1  | COLUMN | order_id |
| sharding_db | t_order_1  | COLUMN | user_id  |
| sharding_db | t_order_1  | COLUMN | status   |
| sharding_db | t_order_1  | INDEX  | PRIMARY  |
| sharding_db | t_order    | COLUMN | order_id |
| sharding_db | t_order    | COLUMN | user_id  |
| sharding_db | t_order    | COLUMN | status   |
| sharding_db | t_order    | INDEX  | PRIMARY  |
+-----+-----+-----+-----+
8 rows in set (0.01 sec)
```

- Query metadata of one table from specified database

```
SHOW TABLE METADATA t_order FROM sharding_db;
```

```
mysql> SHOW TABLE METADATA t_order FROM sharding_db;
+-----+-----+-----+-----+
| schema_name | table_name | type   | name   |
+-----+-----+-----+-----+
| sharding_db | t_order    | COLUMN | order_id |
| sharding_db | t_order    | COLUMN | user_id  |
| sharding_db | t_order    | COLUMN | status   |
| sharding_db | t_order    | INDEX  | PRIMARY  |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

- Query metadata of multiple tables from current database

```
SHOW TABLE METADATA t_order, t_order_1;
```

```
mysql> SHOW TABLE METADATA t_order, t_order_1;
+-----+-----+-----+-----+
| schema_name | table_name | type   | name   |
+-----+-----+-----+-----+
| sharding_db | t_order_1  | COLUMN | order_id |
| sharding_db | t_order_1  | COLUMN | user_id  |
| sharding_db | t_order_1  | COLUMN | status   |
| sharding_db | t_order_1  | INDEX  | PRIMARY  |
+-----+-----+-----+-----+
```

```
| sharding_db      | t_order  | COLUMN | order_id |
| sharding_db      | t_order  | COLUMN | user_id  |
| sharding_db      | t_order  | COLUMN | status   |
| sharding_db      | t_order  | INDEX  | PRIMARY  |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

- Query metadata of one table from current database

```
SHOW TABLE METADATA t_order;
```

```
mysql> SHOW TABLE METADATA t_order;
+-----+-----+-----+-----+
| schema_name      | table_name | type   | name     |
+-----+-----+-----+-----+
| sharding_db      | t_order   | COLUMN | order_id |
| sharding_db      | t_order   | COLUMN | user_id  |
| sharding_db      | t_order   | COLUMN | status   |
| sharding_db      | t_order   | INDEX  | PRIMARY  |
+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Reserved word

SHOW, TABLE, METADATA, FROM

Related links

- [Reserved word](#)

SHOW RULES USED STORAGE UNIT

Description

The SHOW RULES USED STORAGE UNIT syntax is used to query the rules for using the specified storage unit in specified database.

Syntax

```
ShowRulesUsedStorageUnit ::=
    'SHOW' 'RULES' 'USED' 'STORAGE' 'UNIT' storageUnitName ('FROM' databaseName)?

storageUnitName ::=
    identifier

databaseName ::=
    identifier
```

Return Value Description

Columns	Description
type	rule type
name	rule name

Supplement

- When databaseName is not specified, the default is the currently used DATABASE. If DATABASE is not used, No database selected will be prompted.

Example

- Query the rules for using the specified storage unit in specified database

```
SHOW RULES USED STORAGE UNIT ds_1 FROM sharding_db;
```

```
mysql> SHOW RULES USED STORAGE UNIT ds_1 FROM sharding_db;
+-----+-----+
| type          | name          |
+-----+-----+
| readwrite_splitting | ms_group_0 |
| readwrite_splitting | ms_group_0 |
+-----+-----+
2 rows in set (0.01 sec)
```

- Query the rules for using the specified storage unit in current database

```
SHOW RULES USED STORAGE UNIT ds_1;
```

```
mysql> SHOW RULES USED STORAGE UNIT ds_1;
+-----+-----+
```

```

| type          | name          |
+-----+-----+
| readwrite_splitting | ms_group_0 |
| readwrite_splitting | ms_group_0 |
+-----+-----+
2 rows in set (0.01 sec)

```

Reserved word

SHOW, RULES, USED, STORAGE, UNIT, FROM

Related links

- [Reserved word](#)

EXPORT DATABASE CONFIGURATION

Description

The `EXPORT DATABASE CONFIGURATION` syntax is used to export storage units and rule configurations to YAML format.

Syntax

```

ExportDatabaseConfiguration ::=
  'EXPORT' 'DATABASE' 'CONFIGURATION' ('FROM' databaseName)? ('TO' 'FILE' filePath)?

databaseName ::=
  identifier

filePath ::=
  string

```

Supplement

- When `databaseName` is not specified, the currently used logical database will be exported; if no database is used, No database selected will be prompted;
- When `filePath` is not specified, the exported information will be output through the result set;
- When `filePath` is specified, the file will be automatically created. If the file already exists, it will be overwritten.

Example

- Export currently used logical database

```
mysql> EXPORT DATABASE CONFIGURATION;
```

```
+-----+
| result
|
+-----+

+-----+
| databaseName: sharding_db
dataSources:
  ds_1:
    password: 123456
    url: jdbc:mysql://127.0.0.1:3306/db0
    username: root
    minPoolSize: 1
    connectionTimeoutMilliseconds: 30000
    maxLifetimeMilliseconds: 2100000
    readOnly: false
    idleTimeoutMilliseconds: 60000
    maxPoolSize: 50
  ds_2:
    password: 123456
    url: jdbc:mysql://127.0.0.1:3306/db1
    username: root
    minPoolSize: 1
    connectionTimeoutMilliseconds: 30000
    maxLifetimeMilliseconds: 2100000
    readOnly: false
    idleTimeoutMilliseconds: 60000
    maxPoolSize: 50
rules:
|
+-----+
```

```
1 row in set (0.01 sec)
```

- Export the specified logical database and output it to file

```
mysql> EXPORT DATABASE CONFIGURATION FROM sharding_db TO FILE '/xxx/config_sharding_
db.yaml';
+-----+
| result |
+-----+
| Successfully exported to: '/xxx/config_sharding_db.yaml' |
+-----+
1 row in set (0.02 sec)
```

Reserved word

EXPORT, DATABASE, CONFIGURATION, FROM, TO, FILE

Related links

- [Reserved word](#)

IMPORT DATABASE CONFIGURATION

Description

The IMPORT DATABASE CONFIGURATION syntax is used to import a database from the configuration in YAML.

Syntax

```
ImportDatabaseConfiguration ::=
    'IMPORT' 'DATABASE' 'CONFIGURATION' 'FROM' 'FILE' filePath

filePath ::=
    string
```

Supplement

- When a database with the same name already exists in the metadata, it cannot be imported;
- When databaseName in YAML is empty, it cannot be imported;
- When dataSources in YAML is empty, only empty database will be imported.

Example

```
IMPORT DATABASE CONFIGURATION FROM FILE "/xxx/config_sharding_db.yaml";
```

Reserved word

IMPORT, DATABASE, CONFIGURATION, FROM, FILE

Related links

- [Reserved word](#)

CONVERT YAML CONFIGURATION

Description

The CONVERT YAML CONFIGURATION syntax is used to convert YAML configuration to DistSQL RDL statements.

Syntax

```
convertYamlConfiguration ::=  
    'CONVERT' 'YAML' 'CONFIGURATION' 'FROM' 'FILE' filePath  
  
filePath ::=  
    string
```



```
TYPE(NAME='mod', PROPERTIES('sharding-count'='4')),
KEY_GENERATE_STRATEGY(COLUMN=order_id, TYPE(NAME='snowflake'))
);

|
+-----+
|
|
|
|
|
|
|
|
+-----+
1 row in set (0.02 sec)
```

Reserved word

CONVERT, YAML, CONFIGURATION, FROM, FILE

Related links

- [Reserved word](#)

LOCK CLUSTER WITH

Description

The LOCK CLUSTER WITH syntax is utilized to apply a lock with a specific algorithm to the CLUSTER.

Syntax

```
LockClusterWith ::=
    'LOCK' 'CLUSTER' 'WITH' lockStrategy ('TIMEOUT' timeoutMillis)?

timeoutMillis ::=
    long
```

{{% /tab %}} {{% tab name= "Railroad diagram" %}}

{{% /tab %}} {{< /tabs >}}

Supplement

- When the CLUSTER is already locked, it is impossible to re-lock it, otherwise an exception will be thrown.
- Currently, the lockStrategy supports two lock strategies, namely the exclusive lock WRITE and the read-write lock READ_WRITE .
- The timeoutMillis is used to indicate the timeout period for attempting to acquire the lock, with the unit being milliseconds. When not specified, the default value is 3,000 milliseconds.

Example

- Lock the CLUSTER with an exclusive lock without setting the timeout.

-

```
LOCK CLUSTER WITH WRITE;
```

- Lock the CLUSTER with a read-write lock and set the timeout to 2000 milliseconds.

-

```
LOCK CLUSTER WITH READ_WRITE TIMEOUT 2000;
```

Reserved words

LOCK,CLUSTER,WITH

Related links

- [Reserved word](#)

UNLOCK CLUSTER

Description

The UNLOCK CLUSTER syntax is used to release the lock applied to the CLUSTER by the LOCK CLUSTER WITH statement.

Syntax

```
UnlockCluster ::=
    'UNLOCK' 'CLUSTER' ('TIMEOUT' timeoutMillis)?

timeoutMillis ::=
    long
```

```
{{% /tab %}} {{% tab name= "Railroad diagram" %}}
```

```
{{% /tab %}}{< /tabs >}}
```

Supplement

- When the CLUSTER is not in a locked state, it is impossible to release the lock; otherwise, an exception will be thrown.
- `timeoutMillis` is used to indicate the timeout duration for attempting to unlock, with the unit being milliseconds. When not specified, the default value is 3,000 milliseconds.

Example

- Unlock the CLUSTER without setting a timeout.
-

```
UNLOCK CLUSTER;
```

- Unlock the CLUSTER and set the timeout to 2000 milliseconds.

```
UNLOCK CLUSTER TIMEOUT 2000;
```

Reserved words

UNLOCK,CLUSTER

Related links

- [Reserved word](#)


```
"NONE"}}},"parameterCount":0,"parameterMarkerSegments":[],"commentSegments":[] } |
+-----+-----+
-----
-----
-----+
1 row in set (0.01 sec)
```

Reserved word

PARSE

Related links

- [Reserved word](#)

FORMAT SQL

Description

The FORMAT SQL syntax is used to parse SQL and output formatted SQL statement.

Syntax

```
FormatSql ::=
    'FORMAT' sqlStatement
```

Return Value Description

Column	Description
formatted_result	formatted SQL statement

Example

- Parse SQL and output formatted SQL statement

```
FORMAT SELECT * FROM t_order;
```

```
mysql> FORMAT SELECT * FROM t_order;
+-----+
| formatted_result |
```

```
+-----+
| SELECT *
FROM t_order; |
+-----+
1 row in set (0.00 sec)
```

Reserved word

FORMAT

Related links

- [Reserved word](#)

PREVIEW SQL

Description

The PREVIEW SQL syntax is used to preview SQL execution plan.

Syntax

```
PreviewSql ::=
    'PREVIEW' sqlStatement
```

Return Value Description

Column	Description
data_source_name	storage unit name
actual_sql	actual excute SQL statement

Example

- Preview SQL execution plan

```
PREVIEW SELECT * FROM t_order;
```

```
mysql> PREVIEW SELECT * FROM t_order;
+-----+-----+
| data_source_name | actual_sql |
+-----+-----+
```

```
+-----+-----+
| su_1          | SELECT * FROM t_order |
+-----+-----+
1 row in set (0.18 sec)
```

Reserved word

PREVIEW

Related links

- [Reserved word](#)

Reserved word

RDL

Basic Reserved Words

CREATE, ALTER, DROP, TABLE, RULE, TYPE, NAME, PROPERTIES, TRUE, FALSE, IF, NOT, EXISTS

Storage Unit Definition

ADD, RESOURCE, IF, EXISTS, HOST, PORT, DB, USER, PASSWORD, URL , IGNORE, SINGLE, TABLES

Rule Definition

Sharding

DEFAULT, SHARDING, BROADCAST, REFERENCE, DATABASE, STRATEGY, RULES, ALGORITHM, DATANODES, DATABASE_STRATEGY, TABLE_STRATEGY, KEY_GENERATE_STRATEGY, RESOURCES, SHARDING_COLUMN, KEY , GENERATOR, SHARDING_COLUMNS, KEY_GENERATOR, SHARDING_ALGORITHM, COLUMN, AUDIT_STRATEGY , AUDITORS, ALLOW_HINT_DISABLE

Broadcast table

BROADCAST

Single Table

SET, DEFAULT, SINGLE, STORAGE, UNIT, RANDOM

Readwrite-Splitting

READWRITE_SPLITTING, WRITE_STORAGE_UNIT, READ_STORAGE_UNITS, AUTO_AWARE_RESOURCE

Encrypt

ENCRYPT, COLUMNS, CIPHER, ENCRYPT_ALGORITHM

Database Discovery

DB_DISCOVERY, STORAGE_UNITS, HEARTBEAT

Shadow

SHADOW, DEFAULT, SOURCE, SHADOW

MASK

MASK, COLUMNS

RQL

Basic Reserved Words

SHOW, COUNT, DEFAULT, RULE, RULES, TABLE, DATABASE, FROM, UNUSED, USED

Resource Definition

RESOURCES, UNUSED, USED

Rule Query

SHARDING

DEFAULT, SHARDING, BROADCAST, REFERENCE, STRATEGY, ALGORITHM, ALGORITHMS, AUDITORS ,
KEY, GENERATOR, GENERATORS, AUDITOR, AUDITORS, NODES

Single Table

SINGLE, STORAGE, UNIT

Readwrite-Splitting

READWRITE_SPLITTING

Encrypt

ENCRYPT

Database Discovery

DB_DISCOVERY, TYPES, HEARTBEATS

Shadow

SHADOW, ALGORITHMS

MASK

MASK

RAL

ALTER, READWRITE_SPLITTING, RULE, RULES, FROM, ENABLE, DISABLE, SHOW, COMPUTE, NODES, NODE , STATUS, LABEL, RELABEL, WITH, UNLABEL, AUTHORITY, TRANSACTION, SQL_PARSER, DEFAULT, TYPE , NAME, PROPERTIES, PARSE_TREE_CACHE, INITIAL_CAPACITY, MAXIMUM_SIZE , CONCURRENCY_LEVEL, SQL_STATEMENT_CACHE, TRAFFIC, TRAFFIC_ALGORITHM, LOAD_BALANCER, CREATE , DATABASE_VALUE, TABLE_VALUE, CLEAR, MIGRATION, READ, WRITE, WORKER_THREAD, BATCH_SIZE , SHARDING_SIZE, STREAM_CHANNEL, REGISTER, URL, UNREGISTER, UNITS, INTO, LIST, CHECK, BY , STOP, START, ROLLBACK, COMMIT, INFO, MODE, DIST, VARIABLE, VARIABLES, WHERE, DROPSET , SET, HINT, SOURCE, ADD, SHARDING, STORAGE, UNIT, USER, PASSWORD, REFRESH, METADATA, TABLE , DATABASE, GOVERNANCE, CENTER, EXPORT, CONFIGURATION, TO, FILE, IMPORT, USED, IMPLEMENTATIONS, OF , KEY, GENERATE, ALGORITHM, QUERY, LOAD, BALANCE, FEDERATION, SQL_FEDERATION_ENABLED , ALL_QUERY_USE_SQL_FEDERATION, EXECUTION_PLAN_CACHE

RUL

PARSE, FORMAT, PREVIEW

Supplement

- The above reserved words are not case-sensitive

Usage

This chapter will introduce how to use DistSQL to manage resources and rules in a distributed database.

Pre-work

Use MySQL as example, can replace to other databases.

1. Start the MySQL service;
2. Create to be registered MySQL databases;
3. Create role and user in MySQL with creation permission for ShardingSphere-Proxy;
4. Start Zookeeper service;
5. Add mode and authentication configurations to `global.yaml`;
6. Start ShardingSphere-Proxy;
7. Use SDK or terminal connect to ShardingSphere-Proxy.

Create Logic Database

1. Create logic database

```
CREATE DATABASE foo_db;
```

2. Use newly created logic database

```
USE foo_db;
```

Resource Operation

More details please see concentrate rule examples.

Rule Operation

More details please see concentrate rule examples.

Notice

1. Currently, DROP DATABASE will only remove the logical distributed database, not the user's actual database;
2. DROP TABLE will delete all logical fragmented tables and actual tables in the database;
3. CREATE DATABASE will only create a logical distributed database, so users need to create actual databases in advance.

Sharding

Storage unit Operation

- Configure data source information

```
REGISTER STORAGE UNIT ds_0 (  
  HOST="127.0.0.1",  
  PORT=3306,  
  DB="ds_1",  
  USER="root",  
  PASSWORD="root"  
) , ds_1 (  
  HOST="127.0.0.1",  
  PORT=3306,  
  DB="ds_2",  
  USER="root",
```

```
PASSWORD="root"
);
```

Rule Operation

- Create sharding rule

```
CREATE SHARDING TABLE RULE t_order(
  STORAGE_UNITS(ds_0,ds_1),
  SHARDING_COLUMN=order_id,
  TYPE(NAME="hash_mod",PROPERTIES("sharding-count"="4")),
  KEY_GENERATE_STRATEGY(COLUMN=order_id,TYPE(NAME="snowflake"))
);
```

- Create sharding table

```
CREATE TABLE `t_order` (
  `order_id` int NOT NULL,
  `user_id` int NOT NULL,
  `status` varchar(45) DEFAULT NULL,
  PRIMARY KEY (`order_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
```

- Drop sharding table

```
DROP TABLE t_order;
```

- Drop sharding rule

```
DROP SHARDING TABLE RULE t_order;
```

- Unregister storage unit

```
UNREGISTER STORAGE UNIT ds_0, ds_1;
```

- Drop distributed database

```
DROP DATABASE foo_db;
```

Readwrite_splitting

Storage unit Operation

```
REGISTER STORAGE UNIT write_ds (
    HOST="127.0.0.1",
    PORT=3306,
    DB="ds_0",
    USER="root",
    PASSWORD="root"
),read_ds (
    HOST="127.0.0.1",
    PORT=3307,
    DB="ds_0",
    USER="root",
    PASSWORD="root"
);
```

Rule Operation

- Create readwrite_splitting rule

```
CREATE READWRITE_SPLITTING RULE group_0 (
    WRITE_STORAGE_UNIT=write_ds,
    READ_STORAGE_UNITS(read_ds),
    TYPE(NAME="random")
);
```

- Alter readwrite_splitting rule

```
ALTER READWRITE_SPLITTING RULE group_0 (
    WRITE_STORAGE_UNIT=write_ds,
    READ_STORAGE_UNITS(read_ds),
    TYPE(NAME="random",PROPERTIES("read_weight"="2:0"))
);
```

- Drop readwrite_splitting rule

```
DROP READWRITE_SPLITTING RULE group_0;
```

- Unregister storage unit

```
UNREGISTER STORAGE UNIT write_ds,read_ds;
```

- Drop distributed database

```
DROP DATABASE readwrite_splitting_db;
```

Encrypt

Storage unit Operation

```
REGISTER STORAGE UNIT ds_0 (
    HOST="127.0.0.1",
    PORT=3306,
    DB="ds_0",
    USER="root",
    PASSWORD="root"
);
```

Rule Operation

- Create encrypt rule

```
CREATE ENCRYPT RULE t_encrypt (
    COLUMNS(
        (NAME=user_id,CIPHER=user_cipher,ENCRYPT_ALGORITHM(TYPE(NAME='AES',
        PROPERTIES('aes-key-value'='123456abc', 'digest-algorithm-name'='SHA-1')))),
        (NAME=order_id,CIPHER =order_cipher,ENCRYPT_ALGORITHM(TYPE(NAME='AES',
        PROPERTIES('aes-key-value'='123456abc', 'digest-algorithm-name'='SHA-1'))))
    ));
```

- Create encrypt table

```
CREATE TABLE `t_encrypt` (
    `id` int(11) NOT NULL,
    `user_id` varchar(45) DEFAULT NULL,
    `order_id` varchar(45) DEFAULT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

- Alter encrypt rule

```
ALTER ENCRYPT RULE t_encrypt (
    COLUMNS(
        (NAME=user_id,CIPHER=user_cipher,ENCRYPT_ALGORITHM(TYPE(NAME='AES',
        PROPERTIES('aes-key-value'='123456abc', 'digest-algorithm-name'='SHA-1'))))
    ));
```

- Drop encrypt rule

```
DROP ENCRYPT RULE t_encrypt;
```

- Unregister storage unit

```
UNREGISTER STORAGE UNIT ds_0;
```

- Drop distributed database

```
DROP DATABASE encrypt_db;
```

MASK

Storage unit Operation

```
REGISTER STORAGE UNIT ds_0 (
    HOST="127.0.0.1",
    PORT=3306,
    DB="ds_0",
    USER="root",
    PASSWORD="root"
);
```

Rule Operation

- Create mask rule

```
CREATE MASK RULE t_mask (
    COLUMNS(
        (NAME=phone_number, TYPE(NAME='MASK_FROM_X_TO_Y', PROPERTIES("from-x"=1, "to-y"=2, "replace-char"="*"))),
        (NAME=address, TYPE(NAME='MD5'))
    )
);
```

- Create mask table

```
CREATE TABLE `t_mask` (
    `id` int(11) NOT NULL,
    `user_id` varchar(45) DEFAULT NULL,
    `phone_number` varchar(45) DEFAULT NULL,
    `address` varchar(45) DEFAULT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

- Alter mask rule

```
ALTER MASK RULE t_mask (
  COLUMNS(
    (NAME=user_id,TYPE(NAME='MD5'))
  ));
```

- Drop mask rule

```
DROP MASK RULE t_mask;
```

- Unregister storage unit

```
UNREGISTER STORAGE UNIT ds_0;
```

- Drop distributed database

```
DROP DATABASE mask_db;
```

Shadow

Storage unit Operation

```
REGISTER STORAGE UNIT ds_0 (
  HOST="127.0.0.1",
  PORT=3306,
  DB="ds_0",
  USER="root",
  PASSWORD="root"
),ds_1 (
  HOST="127.0.0.1",
  PORT=3306,
  DB="ds_1",
  USER="root",
  PASSWORD="root"
),ds_2 (
  HOST="127.0.0.1",
  PORT=3306,
  DB="ds_2",
  USER="root",
  PASSWORD="root"
);
```

Rule Operation

- Create shadow rule

```
CREATE SHADOW RULE group_0(
SOURCE=ds_0,
SHADOW=ds_1,
t_order(TYPE(NAME="SQL_HINT"),TYPE(NAME="REGEX_MATCH", PROPERTIES("operation"=
"insert","column"="user_id", "regex"='[1]'))),
t_order_item(TYPE(NAME="SQL_HINT")));
```

- Alter shadow rule

```
ALTER SHADOW RULE group_0(
SOURCE=ds_0,
SHADOW=ds_2,
t_order_item(TYPE(NAME="SQL_HINT")));
```

- Drop shadow rule

```
DROP SHADOW RULE group_0;
```

- Unregister storage unit

```
UNREGISTER STORAGE UNIT ds_0,ds_1,ds_2;
```

- Drop distributed database

```
DROP DATABASE foo_db;
```

9.2.4 Data Migration

Introduction

ShardingSphere provides solution of migrating data since **4.1.0**.

Build

Background

For systems running on a single database that urgently need to securely and simply migrate data to a horizontally sharded database.

Prerequisites

- Proxy is developed in JAVA, and JDK version 1.8 or later is recommended.
- Data migration adopts the cluster mode, and ZooKeeper is currently supported as the registry.

Procedure

1. Get ShardingSphere-Proxy. Please refer to [proxy startup guide](#) for details.
2. Modify the configuration file `conf/global.yaml`. Please refer to [mode configuration](#) for details.

Currently, mode must be Cluster, and the corresponding registry must be started in advance.

Configuration sample:

```
mode:
  type: Cluster
  repository:
    type: ZooKeeper
  props:
    namespace: governance_ds
    server-lists: localhost:2181
    retryIntervalMilliseconds: 500
    timeToLiveSeconds: 60
    maxRetries: 3
    operationTimeoutMilliseconds: 500
```

3. Introduce JDBC driver.

Proxy has included JDBC driver of PostgreSQL and openGauss.

If the backend is connected to the following databases, download the corresponding JDBC driver jar package and put it into the `${shardingsphere-proxy}/ext-lib` directory.

Database	JDBC Driver
MySQL	mysql-connector-j-8.3.0.jar

If you are migrating to a heterogeneous database, then you could use more types of database. Introduce JDBC driver as above too.

4. Start ShardingSphere-Proxy:

```
sh bin/start.sh
```

5. View the proxy log `logs/stdout.log`. If you see the following statements:

```
[INFO ] [main] o.a.s.p.frontend.ShardingSphereProxy - ShardingSphere-Proxy start success
```


The startup will have been successful.

6. Configure and migrate on demand.

6.1. Query configuration.

```
SHOW MIGRATION RULE;
```

The default configuration is as follows.

```
+-----+-----+
| read                                     | write
| stream_channel                         |
+-----+-----+
| {"workerThread":20,"batchSize":1000,"shardingSize":100000000} | {"workerThread":20,
"batchSize":1000} | {"type":"MEMORY","props":{"block-queue-size":"2000"}} |
+-----+-----+
```

6.2. Alter configuration (Optional).

Since the migration rule has default values, there is no need to create it, only the ALTER statement is provided.

A completely configured DistSQL is as follows.

```
ALTER MIGRATION RULE (
READ(
  WORKER_THREAD=20,
  BATCH_SIZE=1000,
  SHARDING_SIZE=100000000,
  RATE_LIMITER (TYPE(NAME='QPS',PROPERTIES('qps'='500'))))
),
WRITE(
  WORKER_THREAD=20,
  BATCH_SIZE=1000,
  RATE_LIMITER (TYPE(NAME='TPS',PROPERTIES('tps'='2000'))))
),
STREAM_CHANNEL (TYPE(NAME='MEMORY',PROPERTIES('block-queue-size'='2000'))))
);
```

Configuration item description:

```
ALTER MIGRATION RULE (
READ( -- Data reading configuration. If it is not configured, part of the parameters
will take effect by default.
  WORKER_THREAD=20, -- Obtain the thread pool size of all the data from the source
side. If it is not configured, the default value is used.
  BATCH_SIZE=1000, -- The maximum number of records returned by a query operation.
```

If it is not configured, the default value is used.

SHARDING_SIZE=100000000, -- Sharding size of all the data. If it is not configured, the default value is used.

RATE_LIMITER (-- Traffic limit algorithm. If it is not configured, traffic is not limited.

TYPE(-- Algorithm type. Option: QPS

NAME='QPS',

PROPERTIES(-- Algorithm property

'qps'='500'

)))

),

WRITE(-- Data writing configuration. If it is not configured, part of the parameters will take effect by default.

WORKER_THREAD=20, -- The size of the thread pool on which data is written into the target side. If it is not configured, the default value is used.

BATCH_SIZE=1000, -- The maximum number of records for a batch write operation. If it is not configured, the default value is used.

RATE_LIMITER (-- Traffic limit algorithm. If it is not configured, traffic is not limited.

TYPE(-- Algorithm type. Option: TPS

NAME='TPS',

PROPERTIES(-- Algorithm property.

'tps'='2000'

)))

),

STREAM_CHANNEL (-- Data channel. It connects producers and consumers, used for reading and writing procedures. If it is not configured, the MEMORY type is used by default.

TYPE(-- Algorithm type. Option: MEMORY

NAME='MEMORY',

PROPERTIES(-- Algorithm property

'block-queue-size'='2000' -- Property: blocking queue size.

)))

);

Manual

MySQL user guide

Environment

Supported MySQL versions: 5.1.15 to 8.0.x.

Authority required

1. Enable binlog in source

MySQL 5.7 my.cnf configuration sample:

```
[mysqld]
server-id=1
log-bin=mysql-bin
binlog-format=row
binlog-row-image=full
max_connections=600
```

Run the following command and check whether binlog is enabled.

```
show variables like '%log_bin%';
show variables like '%binlog%';
```

If the following information is displayed, binlog is enabled.

Variable_name	Value
log_bin	ON
binlog_format	ROW
binlog_row_image	FULL

2. Grant Replication-related permissions for source MySQL account.

Run the following command to check whether the user has migration permission.

```
SHOW GRANTS FOR 'migration_user';
```

Result sample:

```
+-----+
| Grants for ${username}@${host} |
+-----+
| GRANT REPLICATION SLAVE, REPLICATION CLIENT ON *.* TO ${username}@${host} |
| ..... |
+-----+
```

3. Grant DDL DML permissions for MySQL account

Source MySQL account needs SELECT permission. Example:

```
GRANT SELECT ON migration_ds_0.* TO `migration_user`@`%`;
```

Target MySQL account needs part of DDL and all DML permissions. Example:

```
GRANT CREATE, DROP, INDEX, SELECT, INSERT, UPDATE, DELETE ON *.* TO `migration_
user`@`%`;
```

Please refer to [MySQL GRANT](#)

Complete procedure example

Requirements

1. Prepare the source database, table, and data in MySQL.

```
DROP DATABASE IF EXISTS migration_ds_0;
CREATE DATABASE migration_ds_0 DEFAULT CHARSET utf8;

USE migration_ds_0

CREATE TABLE t_order (order_id INT NOT NULL, user_id INT NOT NULL, status
VARCHAR(45) NULL, PRIMARY KEY (order_id));

INSERT INTO t_order (order_id, user_id, status) VALUES (1,2,'ok'),(2,4,'ok'),(3,6,
'ok'),(4,1,'ok'),(5,3,'ok'),(6,5,'ok');
```

2. Prepare the target database in MySQL.

```
DROP DATABASE IF EXISTS migration_ds_10;
CREATE DATABASE migration_ds_10 DEFAULT CHARSET utf8;

DROP DATABASE IF EXISTS migration_ds_11;
CREATE DATABASE migration_ds_11 DEFAULT CHARSET utf8;

DROP DATABASE IF EXISTS migration_ds_12;
CREATE DATABASE migration_ds_12 DEFAULT CHARSET utf8;
```

Procedure

1. Create a new logical database in proxy and configure storage units and rules.

```
CREATE DATABASE sharding_db;

USE sharding_db

REGISTER STORAGE UNIT ds_2 (
  URL="jdbc:mysql://127.0.0.1:3306/migration_ds_10?serverTimezone=UTC&useSSL=false
",
  USER="root",
  PASSWORD="root",
```

```

    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
), ds_3 (
    URL="jdbc:mysql://127.0.0.1:3306/migration_ds_11?serverTimezone=UTC&useSSL=false",
    USER="root",
    PASSWORD="root",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
), ds_4 (
    URL="jdbc:mysql://127.0.0.1:3306/migration_ds_12?serverTimezone=UTC&useSSL=false",
    USER="root",
    PASSWORD="root",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
);

CREATE SHARDING TABLE RULE t_order(
    STORAGE_UNITS(ds_2,ds_3,ds_4),
    SHARDING_COLUMN=order_id,
    TYPE(NAME="hash_mod",PROPERTIES("sharding-count"="6")),
    KEY_GENERATE_STRATEGY(COLUMN=order_id,TYPE(NAME="snowflake"))
);

```

If you are migrating to a heterogeneous database, you need to execute the table-creation statements in proxy.

2. Configure the source storage units in proxy.

```

REGISTER MIGRATION SOURCE STORAGE UNIT ds_0 (
    URL="jdbc:mysql://127.0.0.1:3306/migration_ds_0?serverTimezone=UTC&useSSL=false",
    USER="root",
    PASSWORD="root",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
);

```

3. Start data migration.

```
MIGRATE TABLE ds_0.t_order INTO t_order;
```

Or you can specify a target logical database.

```
MIGRATE TABLE ds_0.t_order INTO sharding_db.t_order;
```

4. Check the data migration job list.

```
SHOW MIGRATION LIST;
```

Result example:

```

+-----+-----+-----+-----+
| id | tables | job_item_count | active |
| create_time | stop_time |
+-----+-----+-----+-----+
| j0102p00002333dcb3d9db141cef14bed6fbf1ab54 | ds_0.t_order | 1 | true |
| 2023-09-20 14:41:32 | NULL |
+-----+-----+-----+-----+

```

5. View the data migration details.

```
SHOW MIGRATION STATUS 'j0102p00002333dcb3d9db141cef14bed6fbf1ab54';
```

Result example:

```

+-----+-----+-----+-----+-----+-----+
| item | data_source | tables | status | active | processed_ |
| records_count | inventory_finished_percentage | incremental_idle_seconds | error_ |
| message |
+-----+-----+-----+-----+-----+-----+
| 0 | ds_0 | ds_0.t_order | EXECUTE_INCREMENTAL_TASK | true | 6 |
| 100 | | | | | |
+-----+-----+-----+-----+-----+-----+

```

6. Verify data consistency.

```
CHECK MIGRATION 'j0102p00002333dcb3d9db141cef14bed6fbf1ab54' BY TYPE (NAME='DATA_MATCH');
```

Data consistency check algorithm list:

```
SHOW MIGRATION CHECK ALGORITHMS;
```

Result example:

```

+-----+-----+-----+
| type | type_aliases | supported_database_types |
| description |
+-----+-----+-----+

```

```
| CRC32_MATCH | | MySQL,MariaDB,H2
| Match CRC32 of records. |
| DATA_MATCH | | SQL92,MySQL,PostgreSQL,openGauss,Oracle,SQLServer,
MariaDB,H2 | Match raw data of records. |
+-----+-----+-----+
-----+-----+-----+
```

If encrypt rule is configured in target proxy, then DATA_MATCH could be used.

If you are migrating to a heterogeneous database, then DATA_MATCH could be used.

Query data consistency check progress:

```
SHOW MIGRATION CHECK STATUS 'j0102p00002333dcb3d9db141cef14bed6fbf1ab54';
```

Result example:

```
+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
---+-----+-----+-----+-----+
-+-----+
| tables      | result | check_failed_tables | active | inventory_finished_
percentage | inventory_remaining_seconds | incremental_idle_seconds | check_begin_
time       | check_end_time       | duration_seconds | algorithm_type |
algorithm_props | error_message |
+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
---+-----+-----+-----+-----+
-+-----+
| ds_0.t_order | true   | | false | 100 | 0
| | | 2023-09-20 14:45:31.992 | 2023-09-20 14:45:33.519 | 1
| DATA_MATCH | | | |
+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
---+-----+-----+-----+-----+
-+-----+
```

7. Commit the job.

```
COMMIT MIGRATION 'j0102p00002333dcb3d9db141cef14bed6fbf1ab54';
```

Please refer to [RAL#Migration](#) for more details.

PostgreSQL user guide

Environment

Supported PostgreSQL version: 9.4 or later.

Authority required

1. Enable `test_decoding` in source.
2. Modify WAL configuration in source.

`postgresql.conf` configuration sample:

```
wal_level = logical
max_wal_senders = 10
max_replication_slots = 10
wal_sender_timeout = 0
max_connections = 600
```

Please refer to [Write Ahead Log](#) and [Replication](#) for details.

3. Grant replication permission for source PostgreSQL account.

`pg_hba.conf` instance configuration:

```
host replication repl_acct 0.0.0.0/0 md5
```

Please refer to [The pg_hba.conf File](#) for details.

4. Grant DDL DML permissions for PostgreSQL account.

If you are using a non-super admin account for migration, you need to GRANT CREATE and CONNECT privileges on the database used for migration.

```
GRANT CREATE, CONNECT ON DATABASE migration_ds_0 TO migration_user;
```

The account also needs to have access to the migrated tables and schema. Take the `t_order` table under `test` schema as an example.

```
\c migration_ds_0

GRANT USAGE ON SCHEMA test TO GROUP migration_user;
GRANT SELECT ON TABLE test.t_order TO migration_user;
```

PostgreSQL has the concept of OWNER, and if the account is the OWNER of a database, SCHEMA, or table, the relevant steps can be omitted.

Please refer to [PostgreSQL GRANT](#)

Complete procedure example

Requirements

1. Prepare the source database, table, and data in PostgreSQL.

```
DROP DATABASE IF EXISTS migration_ds_0;
CREATE DATABASE migration_ds_0;

\c migration_ds_0

CREATE TABLE t_order (order_id INT NOT NULL, user_id INT NOT NULL, status
VARCHAR(45) NULL, PRIMARY KEY (order_id));

INSERT INTO t_order (order_id, user_id, status) VALUES (1,2,'ok'),(2,4,'ok'),(3,6,
'ok'),(4,1,'ok'),(5,3,'ok'),(6,5,'ok');
```

2. Prepare the target database in PostgreSQL.

```
DROP DATABASE IF EXISTS migration_ds_10;
CREATE DATABASE migration_ds_10;

DROP DATABASE IF EXISTS migration_ds_11;
CREATE DATABASE migration_ds_11;

DROP DATABASE IF EXISTS migration_ds_12;
CREATE DATABASE migration_ds_12;
```

Procedure

1. Create a new logical database in proxy and configure storage units and rules.

```
CREATE DATABASE sharding_db;

\c sharding_db

REGISTER STORAGE UNIT ds_2 (
    URL="jdbc:postgresql://127.0.0.1:5432/migration_ds_10",
    USER="postgres",
    PASSWORD="root",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
), ds_3 (
    URL="jdbc:postgresql://127.0.0.1:5432/migration_ds_11",
    USER="postgres",
    PASSWORD="root",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
), ds_4 (
    URL="jdbc:postgresql://127.0.0.1:5432/migration_ds_12",
```

```

    USER="postgres",
    PASSWORD="root",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
);

CREATE SHARDING TABLE RULE t_order(
    STORAGE_UNITS(ds_2,ds_3,ds_4),
    SHARDING_COLUMN=order_id,
    TYPE(NAME="hash_mod",PROPERTIES("sharding-count"="6")),
    KEY_GENERATE_STRATEGY(COLUMN=order_id,TYPE(NAME="snowflake"))
);

```

If you are migrating to a heterogeneous database, you need to execute the table-creation statements in proxy.

2. Configure the source storage units in proxy.

```

REGISTER MIGRATION SOURCE STORAGE UNIT ds_0 (
    URL="jdbc:postgresql://127.0.0.1:5432/migration_ds_0",
    USER="postgres",
    PASSWORD="root",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
);

```

3. Enable data migration.

```

MIGRATE TABLE ds_0.t_order INTO t_order;

```

Or you can specify a target logical database.

```

MIGRATE TABLE ds_0.t_order INTO sharding_db.t_order;

```

Or you can specify a source schema name.

```

MIGRATE TABLE ds_0.public.t_order INTO sharding_db.t_order;

```

4. Check the data migration job list.

```

SHOW MIGRATION LIST;

```

Result example:

```

+-----+-----+-----+-----+
--+-----+-----+
| id                                     | tables      | job_item_count | active
| create_time       | stop_time   |                |
+-----+-----+-----+-----+
--+-----+-----+
| j0102p00002333dcb3d9db141cef14bed6fbf1ab54 | ds_0.t_order | 1              | true
| 2023-09-20 14:41:32 | NULL        |                |

```

```
+-----+-----+-----+-----+
--+
```

5. View the data migration details.

```
SHOW MIGRATION STATUS 'j0102p00002333dcb3d9db141cef14bed6fbf1ab54';
```

Result example:

```
+-----+-----+-----+-----+-----+-----+
--+
```

item	data_source	tables	status	active	processed_records_count	inventory_finished_percentage	incremental_idle_seconds	error_message
0	ds_0	ds_0.t_order	EXECUTE_INCREMENTAL_TASK	true	6			
100								

```
+-----+-----+-----+-----+-----+-----+
--+
```

6. Verify data consistency.

```
CHECK MIGRATION 'j0102p00002333dcb3d9db141cef14bed6fbf1ab54';
Query OK, 0 rows affected (0.09 sec)
```

Query data consistency check progress:

```
SHOW MIGRATION CHECK STATUS 'j0102p00002333dcb3d9db141cef14bed6fbf1ab54';
```

Result example:

```
+-----+-----+-----+-----+-----+-----+
--+
```

tables	result	check_failed_tables	active	inventory_finished_percentage	inventory_remaining_seconds	incremental_idle_seconds	check_begin_time	check_end_time	duration_seconds	algorithm_type	algorithm_props	error_message
ds_0.t_order	true		false	100								
					2023-09-20 14:45:31.992	2023-09-20 14:45:33.519	1					

```
+-----+-----+-----+-----+-----+-----+
--+
```

DATA_MATCH			
+-----+	+-----+	+-----+	+-----+
-----+	-----+	-----+	-----+
-----+	-----+	-----+	-----+
-----+	-----+	-----+	-----+

7. Commit the job.

```
COMMIT MIGRATION 'j0102p00002333dcb3d9db141cef14bed6fbf1ab54';
```

Please refer to [RAL#Migration](#) for more details.

openGauss user guide

Environment

Supported openGauss version: 2.0.1 to 3.0.0.

Authority required

1. Modify WAL configuration in source.

postgresql.conf configuration sample:

```
wal_level = logical
max_wal_senders = 10
max_replication_slots = 10
wal_sender_timeout = 0
max_connections = 600
```

Please refer to [Write Ahead Log](#) and [Replication](#) for details.

2. Grant replication permission for source openGauss account.

pg_hba.conf instance configuration:

```
host replication repl_acct 0.0.0.0/0 md5
```

Please refer to [Configuring Client Access Authentication](#) and [Example: Logic Replication Code](#) for details.

3. Grant DDL DML permissions for openGauss account.

If you are using a non-super admin account for migration, you need to GRANT CREATE and CONNECT privileges on the database used for migration.

```
GRANT CREATE, CONNECT ON DATABASE migration_ds_0 TO migration_user;
```

The account also needs to have access to the migrated tables and schema. Take the t_order table under test schema as an example.

```
\c migration_ds_0
```

```
GRANT USAGE ON SCHEMA test TO GROUP migration_user;
GRANT SELECT ON TABLE test.t_order TO migration_user;
```

openGauss has the concept of OWNER, and if the account is the OWNER of a database, SCHEMA, or table, the relevant steps can be omitted.

openGauss does not allow normal accounts to operate in public schema, so if the migrated table is in public schema, you need to authorize additional.

Please refer to [openGauss GRANT](#)

```
GRANT ALL PRIVILEGES TO migration_user;
```

Complete procedure example

Requirements

1. Prepare the source database, table, and data.

1.1. Isomorphic database.

```
DROP DATABASE IF EXISTS migration_ds_0;
CREATE DATABASE migration_ds_0;
```

```
\c migration_ds_0
```

```
CREATE TABLE t_order (order_id INT NOT NULL, user_id INT NOT NULL, status
VARCHAR(45) NULL, PRIMARY KEY (order_id));
```

```
INSERT INTO t_order (order_id, user_id, status) VALUES (1,2,'ok'),(2,4,'ok'),(3,6,
'ok'),(4,1,'ok'),(5,3,'ok'),(6,5,'ok');
```

1.2. Heterogeneous database.

MySQL example:

```
DROP DATABASE IF EXISTS migration_ds_0;
CREATE DATABASE migration_ds_0 DEFAULT CHARSET utf8;
```

```
USE migration_ds_0
```

```
CREATE TABLE t_order (order_id INT NOT NULL, user_id INT NOT NULL, status
VARCHAR(45) NULL, PRIMARY KEY (order_id));
```

```
INSERT INTO t_order (order_id, user_id, status) VALUES (1,2,'ok'),(2,4,'ok'),(3,6,
'ok'),(4,1,'ok'),(5,3,'ok'),(6,5,'ok');
```

2. Prepare the target database in openGauss.

```
DROP DATABASE IF EXISTS migration_ds_10;
CREATE DATABASE migration_ds_10;

DROP DATABASE IF EXISTS migration_ds_11;
CREATE DATABASE migration_ds_11;

DROP DATABASE IF EXISTS migration_ds_12;
CREATE DATABASE migration_ds_12;
```

Procedure

1. Create a new logical database and configure storage units and rules.

- 1.1. Create logic database.

```
CREATE DATABASE sharding_db;

\c sharding_db
```

- 1.2. Register storage units.

```
REGISTER STORAGE UNIT ds_2 (
    URL="jdbc:opengauss://127.0.0.1:5432/migration_ds_10",
    USER="gaussdb",
    PASSWORD="Root@123",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
), ds_3 (
    URL="jdbc:opengauss://127.0.0.1:5432/migration_ds_11",
    USER="gaussdb",
    PASSWORD="Root@123",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
), ds_4 (
    URL="jdbc:opengauss://127.0.0.1:5432/migration_ds_12",
    USER="gaussdb",
    PASSWORD="Root@123",
    PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
);
```

- 1.3. Create sharding table rule.

```
CREATE SHARDING TABLE RULE t_order(
    STORAGE_UNITS(ds_2,ds_3,ds_4),
    SHARDING_COLUMN=order_id,
    TYPE(NAME="hash_mod",PROPERTIES("sharding-count"="6")),
    KEY_GENERATE_STRATEGY(COLUMN=order_id,TYPE(NAME="snowflake"))
);
```

1.4. Create target table.

If you are migrating to a heterogeneous database, you need to execute the table-creation statements in proxy.

MySQL example:

```
CREATE TABLE t_order (order_id INT NOT NULL, user_id INT NOT NULL, status
VARCHAR(45) NULL, PRIMARY KEY (order_id));
```

2. Configure the source storage units in proxy.

2.1. Isomorphic database.

```
REGISTER MIGRATION SOURCE STORAGE UNIT ds_0 (
  URL="jdbc:opengauss://127.0.0.1:5432/migration_ds_0",
  USER="gaussdb",
  PASSWORD="Root@123",
  PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
);
```

2.2. Heterogeneous database.

MySQL example:

```
REGISTER MIGRATION SOURCE STORAGE UNIT ds_0 (
  URL="jdbc:mysql://127.0.0.1:3306/migration_ds_0?serverTimezone=UTC&useSSL=false",
  USER="root",
  PASSWORD="root",
  PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
);
```

3. Enable data migration.

```
MIGRATE TABLE ds_0.t_order INTO t_order;
```

Or you can specify a target logical database.

```
MIGRATE TABLE ds_0.t_order INTO sharding_db.t_order;
```

Or you can specify a source schema name.

```
MIGRATE TABLE ds_0.public.t_order INTO sharding_db.t_order;
```

4. Check the data migration job list.

```
SHOW MIGRATION LIST;
```

Result example:


```

├─ LICENSE
├─ NOTICE
├─ conf
│   └─ agent.yaml
├─ plugins
│   ├── lib
│   │   ├── shardingsphere-agent-metrics-core-${latest.release.version}.jar
│   │   └── shardingsphere-agent-plugin-core-${latest.release.version}.jar
│   ├── logging
│   │   └── shardingsphere-agent-logging-file-${latest.release.version}.jar
│   ├── metrics
│   │   └── shardingsphere-agent-metrics-prometheus-${latest.release.version}.jar
│   └── tracing
│       └── shardingsphere-agent-tracing-opentelemetry-${latest.release.version}.jar
└─ shardingsphere-agent-${latest.release.version}.jar

```

Configuration

conf/agent.yaml is used to manage agent configuration. Built-in plugins include File, Prometheus, OpenTelemetry.

```

plugins:
#  logging:
#    File:
#      props:
#        level: "INFO"
#  metrics:
#    Prometheus:
#      host: "localhost"
#      port: 9090
#      props:
#        jvm-information-collector-enabled: "true"
#  tracing:
#    OpenTelemetry:
#      props:
#        otel.service.name: "shardingsphere"
#        otel.traces.exporter: "jaeger"
#        otel.exporter.otlp.traces.endpoint: "http://localhost:14250"
#        otel.traces.sampler: "always_on"

```

Plugin description

File

Currently, the File plugin only outputs the time-consuming log output of building metadata, and has no other log output for the time being.

Prometheus

Used for exposure monitoring metrics.

- Parameter description

Name	Description
host	host IP
port	port
jvm-information-collector-enabled	whether to collect JVM indicator information

OpenTelemetry

OpenTelemetry can export tracing data to Jaeger, Zipkin.

- Parameter description

Name	Description
otel.service.name	service name
otel.traces.exporter	traces exporter
otel.exporter.otlp.traces.endpoint	traces endpoint
otel.traces.sampler	traces sampler

Parameter reference [OpenTelemetry SDK Autoconfigure](#)

Usage

Start ShardingSphere-Proxy

```
tar -zxvf apache-shardingsphere-${latest.release.version}-shardingsphere-proxy-bin.tar.gz
cd apache-shardingsphere-${latest.release.version}-shardingsphere-proxy-bin
./bin/start.sh -g
```

After startup, you can find the plugin info in the log of ShardingSphere-Proxy, Metric and Tracing data can be viewed through the configured monitoring address.

Metrics

Name	Type	Description
build_info	G A U G E	Build information
parsed_sql_total	C O U N T E R	Total count of parsed by type (INSERT, UPDATE, DELETE, SELECT, DDL, DCL, DAL, TCL, RQL, RDL, RAL, RUL)
routed_sql_total	C O U N T E R	Total count of routed by type (INSERT, UPDATE, DELETE, SELECT)
routed_result_total	C O U N T E R	Total count of routed result (data source routed, table routed)
proxy_state	G A U G E	Status information of ShardingSphere-Proxy. 0 is OK; 1 is CIRCUIT BREAK; 2 is LOCK
proxy_meta_data_info	G A U G E	Meta data information of ShardingSphere-Proxy. database_count is logic number of databases; storage_unit_count is number of storage units
proxy_current_connections	G A U G E	Current connections of ShardingSphere-Proxy
proxy_requests_total	C O U N T E R	Total requests of ShardingSphere-Proxy
proxy_transactions_total	C O U N T E R	Total transactions of ShardingSphere-Proxy, classify by commit, rollback
proxy_execute_latency_millis	H I S T O G R A M	Execute latency millis histogram of ShardingSphere-Proxy
proxy_executor_errors_total	C O U N T E R	Total executor errors of ShardingSphere-Proxy

9.2.6 Optional Plugins

ShardingSphere only includes the implementation of the core SPI by default, and there is a part of the SPI that contains third-party dependencies in Git Source Implemented plugins are not included. Retrievable at <https://central.sonatype.com/>.

SPI and existing implementation classes of SPI corresponding to all plugins can be retrieved at <https://shardingsphere.apache.org/document/current/cn/dev-manual/>.

All the built-in plugins for ShardingSphere-Proxy are listed below in the form of 'groupId:artifactId'.

- org.apache.shardingsphere:shardingsphere-cluster-mode-repository-etcd, etcd implementation of persistent definition of cluster mode configuration information
- org.apache.shardingsphere:shardingsphere-cluster-mode-repository-zookeeper,

the zookeeper implementation of the persistent definition of cluster mode configuration information

- `org.apache.shardingsphere:shardingsphere-jdbc`, JDBC module
- `org.apache.shardingsphere:shardingsphere-db-protocol-core`, database protocol core
- `org.apache.shardingsphere:shardingsphere-mysql-protocol`, the MySQL implementation of the database protocol
- `org.apache.shardingsphere:shardingsphere-postgresql-protocol`, the PostgreSQL implementation of the database protocol
- `org.apache.shardingsphere:shardingsphere-opengauss-protocol`, the OpenGauss implementation of the database protocol
- `org.apache.shardingsphere:shardingsphere-proxy-frontend-core`, used by ShardingSphere-Proxy to parse and adapt the protocol for accessing the database
- `org.apache.shardingsphere:shardingsphere-proxy-frontend-mysql`, a MySQL implementation for ShardingSphere-Proxy to parse and adapt the protocol for accessing the database
- `org.apache.shardingsphere:shardingsphere-proxy-frontend-postgresql`, a PostgreSQL implementation for ShardingSphere-Proxy to parse and adapt the protocol for accessing the database
- `org.apache.shardingsphere:shardingsphere-proxy-frontend-opengauss`, an openGauss implementation for ShardingSphere-Proxy to parse and adapt the protocol for accessing the database
- `org.apache.shardingsphere:shardingsphere-proxy-backend-core`, the backend core for ShardingSphere Proxy
- `org.apache.shardingsphere:shardingsphere-standalone-mode-core`, the persistence definition core of single-machine mode configuration information

For the core `org.apache.shardingsphere:shardingsphere-jdbc`, its built-in plugins reference [ShardingSphere-JDBC Optional Plugins](#).

If ShardingSphere Proxy needs to use optional plugins, you need to download the JAR containing its SPI implementation and its dependent JARs from Maven Central.

All optional plugins are listed below in the form of `groupId:artifactId`.

- Standalone mode configuration information persistence definition
 - `org.apache.shardingsphere:shardingsphere-standalone-mode-repository-jdbc`, JDBC based persistence
- XA transaction manager provider definition
 - `org.apache.shardingsphere:shardingsphere-transaction-xa-narayana`, XA distributed transaction manager based on Narayana
- Row Value Expressions definition

- `org.apache.shardingsphere:shardingsphere-infra-expr-espresso`, Row Value Expressions that uses the Groovy syntax based on GraalVM Truffle's Espresso implementation
- Database type identification
 - `org.apache.shardingsphere:shardingsphere-infra-database-hive`, Adaptation of `jdbcURL` for JDBC support of Hive, and metadata loading implementation
 - `org.apache.shardingsphere:shardingsphere-infra-database-presto`, Adaptation of `jdbcURL` for JDBC support of Presto, and metadata loading implementation
- SQL parsing
 - `org.apache.shardingsphere:shardingsphere-parser-sql-clickhouse`, ClickHouse dialect implementation of SQL parsing
 - `org.apache.shardingsphere:shardingsphere-parser-sql-hive`, Hive dialect implementation of SQL parsing

In addition to the above optional plugins, ShardingSphere community developers have contributed a number of plugin implementations. These plugins can be found in [ShardingSphere Plugins](#) repository. Plugins in ShardingSphere Plugin repository would remain the same release plan with ShardingSphere, you can build plugin jar by yourself, and install into ShardingSphere.

Seata AT Mode transactions

Background Information

ShardingSphere Proxy or ShardingSphere Proxy Native in the form of GraalVM Native Image do not provide support for Seata's AT mode by default. Both support Seata's AT mode in optional modules.

This section is still limited by the documented content of [Seata transaction](#) on the ShardingSphere JDBC side, but there are some differences,

1. If the user uses ShardingSphere JDBC in a hybrid deployment architecture, this scenario does not directly interact with ShardingSphere Proxy and is therefore not relevant to this article. This article only discusses the scenario where the business project does not use ShardingSphere JDBC
2. Seata Client only exists in ShardingSphere Proxy, and the business project does not need to rely on Seata Client
3. The R2DBC DataSource of the business project can be normally connected to the ShardingSphere Proxy with Seata integration turned on
4. For ShardingSphere Proxy with Seata integration turned on, it is not possible to establish a transaction propagation across services operation to propagate transactions to other ShardingSphere Proxy instances using Seata integration or other microservices using Seata integration. If users have such needs, they should consider submitting a PR for ShardingSphere
5. The assumptions made by ShardingSphere JDBC on Seata's TCC mode are invalid on ShardingSphere Proxy

The following discussion takes ShardingSphere Proxy using Seata Client 2.3.0 as an example.

Operation steps

1. Confirm the JAR and dependency list of Seata Client
2. Start Seata Server
3. Create the undo_log table for the real database involved
4. Create ShardingSphere Proxy containing Seata Client and Seata integration module
5. Add Seata configuration to ShardingSphere Proxy

Configuration example

Confirm the JAR and dependency list of Seata Client

For Ubuntu 22.04.4 with SDKMAN! installed, you can confirm all compile scope dependencies of Seata Client with the following command:

```
sdk install java 23-open
sdk use java 23-open
sdk install maven 3.9.9
sdk use maven 3.9.9
mvn dependency:get -Dartifact=org.apache.seata:seata-all:2.3.0
mvn -f ~/.m2/repository/org/apache/seata/seata-all/2.3.0/seata-all-2.3.0.pom
dependency:tree | grep -v ':provided' | grep -v ':runtime'
```

Compared with the pom.xml of org.apache.shardingsphere:shardingsphere-proxy-distribution, it is not difficult to find the differences listed as follows:

```
org.apache.seata:seata-all:jar:2.3.0
org.springframework:spring-context:jar:5.3.39
org.springframework:spring-expression:jar:5.3.39
org.springframework:spring-core:jar:5.3.39
org.springframework:spring-jcl:jar:5.3.39
org.springframework:spring-beans:jar:5.3.39
org.springframework:spring-aop:jar:5.3.39
org.springframework:spring-webmvc:jar:5.3.39
org.springframework:spring-web:jar:5.3.39
org.springframework:spring-tx:jar:5.3.39
io.netty:netty-all:jar:4.1.101.Final
io.netty:netty-codec-dns:jar:4.1.101.Final
io.netty:netty-codec-haproxy:jar:4.1.101.Final
io.netty:netty-codec-memcache:jar:4.1.101.Final
io.netty:netty-codec-mqtt:jar:4.1.101.Final
io.netty:netty-codec-redis:jar:4.1.101.Final
io.netty:netty-codec-smtp:jar:4.1.101.Final
```

```
io.netty:netty-codec-stomp:jar:4.1.101.Final
io.netty:netty-codec-xml:jar:4.1.101.Final
io.netty:netty-handler-ssl-ocsp:jar:4.1.101.Final
io.netty:netty-resolver-dns:jar:4.1.101.Final
io.netty:netty-transport-rxtx:jar:4.1.101.Final
io.netty:netty-transport-sctp:jar:4.1.101.Final
io.netty:netty-transport-udt:jar:4.1.101.Final
io.netty:netty-transport-classes-kqueue:jar:4.1.101.Final
io.netty:netty-resolver-dns-classes-macos:jar:4.1.101.Final
org.antlr:antlr4:jar:4.8
org.antlr:antlr-runtime:jar:3.5.2
org.antlr:ST4:jar:4.3
org.abego.treelayout:org.abego.treelayout.core:jar:1.0.3
org.glassfish:javax.json:jar:1.0.4
com.ibm.icu:icu4j:jar:61.1
com.alibaba:fastjson:jar:1.2.83
com.alibaba:druid:jar:1.2.20
com.typesafe:config:jar:1.2.1
commons-pool:commons-pool:jar:1.6
org.apache.dubbo.extensions:dubbo-filter-seata:jar:1.0.2
aopalliance:aopalliance:jar:1.0
```

Obviously, users should always avoid paying attention to the outdated `org.antlr:antlr4-runtime:4.8`. This list will be used to recreate the Docker Image of ShardingSphere Proxy.

In addition, users can always add Seata Client to the locally built Docker Image of ShardingSphere Proxy by modifying the source code of ShardingSphere.

Start Seata Server, Postgres Server and ShardingSphere Proxy

Write the Docker Compose file to start Seata Server and Postgres Server.

```
services:
  postgres:
    image: postgres:17.2-bookworm
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - ./docker-entypoint-initdb.d:/docker-entypoint-initdb.d
  apache-seata-server:
    image: apache/seata-server:2.3.0
    healthcheck:
      test: [ "CMD", "sh", "-c", "curl -s apache-seata-server:7091/health | grep -q '^ok$'" ]
  shardingsphere-proxy-custom:
    image: example/shardingsphere-proxy-custom:latest
    pull_policy: build
```



```

build:
  context: .
  dockerfile_inline: |
    FROM apache/shardingsphere-proxy:latest
    RUN wget https://repo1.maven.org/maven2/org/apache/shardingsphere/
shardingsphere-transaction-base-seata-at/5.5.2/shardingsphere-transaction-base-
seata-at-5.5.2.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/apache/seata/seata-all/2.3.
0/seata-all-2.3.0.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/springframework/spring-
context/5.3.39/spring-context-5.3.39.jar --directory-prefix=/opt/shardingsphere-
proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/springframework/spring-
expression/5.3.39/spring-expression-5.3.39.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/springframework/spring-core/
5.3.39/spring-core-5.3.39.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/springframework/spring-jcl/
5.3.39/spring-jcl-5.3.39.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/springframework/spring-beans/
5.3.39/spring-beans-5.3.39.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/springframework/spring-aop/
5.3.39/spring-aop-5.3.39.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/springframework/spring-
webmvc/5.3.39/spring-webmvc-5.3.39.jar --directory-prefix=/opt/shardingsphere-
proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/springframework/spring-web/
5.3.39/spring-web-5.3.39.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/springframework/spring-tx/5.
3.39/spring-tx-5.3.39.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-all/4.1.101.Final/
netty-all-4.1.101.Final.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-codec-dns/4.1.
101.Final/netty-codec-dns-4.1.101.Final.jar --directory-prefix=/opt/shardingsphere-
proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-codec-haproxy/4.
1.101.Final/netty-codec-haproxy-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-codec-memcache/4.
1.101.Final/netty-codec-memcache-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-codec-mqtt/4.1.
101.Final/netty-codec-mqtt-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-codec-redis/4.1.
101.Final/netty-codec-redis-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-codec-smtp/4.1.

```

```

101.Final/netty-codec-smtp-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-codec-stomp/4.1.
101.Final/netty-codec-stomp-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-codec-xml/4.1.
101.Final/netty-codec-xml-4.1.101.Final.jar --directory-prefix=/opt/shardingsphere-
proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-handler-ssl-ocsp/
4.1.101.Final/netty-handler-ssl-ocsp-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-resolver-dns/4.1.
101.Final/netty-resolver-dns-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-transport-rxtx/4.
1.101.Final/netty-transport-rxtx-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-transport-sctp/4.
1.101.Final/netty-transport-sctp-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-transport-udt/4.
1.101.Final/netty-transport-udt-4.1.101.Final.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-transport-
classes-kqueue/4.1.101.Final/netty-transport-classes-kqueue-4.1.101.Final.jar --
directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/io/netty/netty-resolver-dns-
classes-macos/4.1.101.Final/netty-resolver-dns-classes-macos-4.1.101.Final.jar --
directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/antlr/antlr4/4.8/antlr4-4.8.
jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/antlr/antlr-runtime/3.5.2/
antlr-runtime-3.5.2.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/antlr/ST4/4.3/ST4-4.3.jar --
directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/abego/treelayout/org.abego.
treelayout.core/1.0.3/org.abego.treelayout.core-1.0.3.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/org/glassfish/javax.json/1.0.4/
javax.json-1.0.4.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/com/ibm/icu/icu4j/61.1/icu4j-61.
1.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/com/alibaba/fastjson/1.2.83/
fastjson-1.2.83.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo1.maven.org/maven2/com/alibaba/druid/1.2.20/druid-
1.2.20.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    RUN wget https://repo.akka.io/maven/com/typesafe/config/1.2.1/config-1.
2.1.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib

```

```

        RUN wget https://repo1.maven.org/maven2/commons-pool/commons-pool/1.6/
commons-pool-1.6.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
        RUN wget https://repo1.maven.org/maven2/org/apache/dubbo/extensions/
dubbo-filter-seata/1.0.2/dubbo-filter-seata-1.0.2.jar --directory-prefix=/opt/
shardingsphere-proxy/ext-lib
        RUN wget https://repo1.maven.org/maven2/aopalliance/aopalliance/1.0/
aopalliance-1.0.jar --directory-prefix=/opt/shardingsphere-proxy/ext-lib
    volumes:
      - ./conf:/opt/shardingsphere-proxy/conf
    ports:
      - "3308:3308"
    environment:
      PORT: 3308
    depends_on:
      apache-seata-server:
        condition: service_healthy

```

In addition, users can always build the Docker Image of ShardingSphere Proxy in advance using Dockerfile instead of dynamically building the Docker Image in Docker Compose.

The `./conf` folder contains the file `global.yaml` with the following content,

```

authority:
  users:
    - user: root@%
      password: root
      admin: true
  privilege:
    type: ALL_PERMITTED
transaction:
  defaultType: BASE
  providerType: Seata
props:
  proxy-frontend-database-protocol-type: PostgreSQL

```

The `./conf` folder contains the file `file.conf` with the following content,

```

service {
  vgroupMapping.default_tx_group = "default"
  default.grouplist = "apache-seata-server:8091"
}

```

The `./conf` folder contains the file `registry.conf` with the following contents,

```

registry {
  type = "file"
  file {
    name = "file.conf"
  }
}

```

```

}
config {
  type = "file"
  file {
    name = "file.conf"
  }
}

```

The `./conf` folder contains the file `seata.conf`, the content of which is as follows,

```

client {
  application.id = test
  transaction.service.group = default_tx_group
}

```

The `./docker-entrypoint-initdb.d` folder contains the file `init.sh` with the following content,

```

#!/bin/bash
set -e

psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" <<-EOSQL
  CREATE DATABASE demo_ds_0;
  CREATE DATABASE demo_ds_1;
  CREATE DATABASE demo_ds_2;
EOSQL

for i in "demo_ds_0" "demo_ds_1" "demo_ds_2"
do
psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname "$i" <<-EOSQL
  CREATE TABLE IF NOT EXISTS public.undo_log
  (
    id          SERIAL          NOT NULL,
    branch_id   BIGINT          NOT NULL,
    xid         VARCHAR(128) NOT NULL,
    context     VARCHAR(128) NOT NULL,
    rollback_info BYTEA         NOT NULL,
    log_status  INT             NOT NULL,
    log_created  TIMESTAMP(0) NOT NULL,
    log_modified  TIMESTAMP(0) NOT NULL,
    CONSTRAINT pk_undo_log PRIMARY KEY (id),
    CONSTRAINT ux_undo_log UNIQUE (xid, branch_id)
  );
  CREATE INDEX ix_log_created ON undo_log(log_created);
  COMMENT ON TABLE public.undo_log IS 'AT transaction mode undo table';
  COMMENT ON COLUMN public.undo_log.branch_id IS 'branch transaction id';
  COMMENT ON COLUMN public.undo_log.xid IS 'global transaction id';
  COMMENT ON COLUMN public.undo_log.context IS 'undo_log context,such as
serialization';
  COMMENT ON COLUMN public.undo_log.rollback_info IS 'rollback info';

```

```

COMMENT ON COLUMN public.undo_log.log_status IS '0:normal status,1:defense status
';
COMMENT ON COLUMN public.undo_log.log_created IS 'create datetime';
COMMENT ON COLUMN public.undo_log.log_modified IS 'modify datetime';
CREATE SEQUENCE IF NOT EXISTS undo_log_id_seq INCREMENT BY 1 MINVALUE 1 ;

CREATE TABLE IF NOT EXISTS t_order (
    order_id BIGSERIAL NOT NULL PRIMARY KEY,
    order_type INTEGER,
    user_id INTEGER NOT NULL,
    address_id BIGINT NOT NULL,
    status VARCHAR(50)
);
EOSQL
done

```

Create ShardingSphere virtual database

Use third-party tools to create ShardingSphere virtual database in ShardingSphere Proxy. Taking DBeaver Community as an example, if you use Ubuntu 22.04.4, you can quickly install it through Snapcraft.

```

sudo apt update && sudo apt upgrade -y
sudo snap install dbeaver-ce
snap run dbeaver-ce

```

In DBeaver Community, use the `jdbcUrl` of `jdbc:postgresql://127.0.0.1:3308/postgres` to connect to ShardingSphere Proxy, and the username and password are both root. The required JDBC Driver corresponds to the `proxy-front-end-database-protocol-type` set by ShardingSphere Proxy. Execute the following SQL,

```

-- noinspection SqlNoDataSourceInspectionForFile
CREATE DATABASE sharding_db;

```

In DBeaver Community, use the `jdbcUrl` of `jdbc:postgresql://127.0.0.1:3308/sharding_db` to connect to ShardingSphere Proxy, and the username and password are both root. Execute the following SQL,

```

-- noinspection SqlNoDataSourceInspectionForFile
REGISTER STORAGE UNIT ds_0 (
    URL="jdbc:postgresql://postgres:5432/demo_ds_0",
    USER="postgres",
    PASSWORD="example"
), ds_1 (
    URL="jdbc:postgresql://postgres:5432/demo_ds_1",
    USER="postgres",
    PASSWORD="example"
)

```

```

),ds_2 (
  URL="jdbc:postgresql://postgres:5432/demo_ds_2",
  USER="postgres",
  PASSWORD="example"
);

CREATE DEFAULT SHARDING DATABASE STRATEGY (
  TYPE="standard",
  SHARDING_COLUMN=user_id,
  SHARDING_ALGORITHM(
    TYPE(
      NAME=INLINE,
      PROPERTIES(
        "algorithm-expression"="ds_${user_id % 2}"
      )
    )
  )
);

CREATE SHARDING TABLE RULE t_order (
  DATANODES("ds_${0..2}.t_order"),
  KEY_GENERATE_STRATEGY(COLUMN=order_id,TYPE(NAME="SNOWFLAKE"))
);

```

Introduce Postgres JDBC Driver in the business project

Introduce Postgres JDBC Driver in the business project. The required JDBC Driver corresponds to the proxy-frontend-database-protocol-type set by ShardingSphere Proxy.

```

<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.5</version>
</dependency>

```

Enjoy the integration

Create ShardingSphere's data source through Postgres JDBC Driver to enjoy the integration.

```

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
@SuppressWarnings("SqlNoDataSourceInspection")
public class ExampleUtils {

```

```

void test() throws SQLException {
    HikariConfig config = new HikariConfig();
    config.setJdbcUrl("jdbc:postgresql://127.0.0.1:3308/sharding_db");
    config.setDriverClassName("org.postgresql.Driver");
    try (HikariDataSource dataSource = new HikariDataSource(config);
        Connection connection = dataSource.getConnection();
        Statement statement = connection.createStatement()) {
        statement.execute("INSERT INTO t_order (user_id, order_type, address_id,
status) VALUES (1, 1, 1, 'INSERT_TEST')");
        statement.executeQuery("SELECT * FROM t_order");
        statement.execute("DELETE FROM t_order WHERE order_id=1");
    }
}
}

```

Usage restrictions

ShardingSphere Proxy Native for GraalVM Native Image

For ShardingSphere Proxy Native in GraalVM Native Image, Users always need to modify the ShardingSphere source code to add the Seata Client and Seata integrated Maven modules and compile them into GraalVM Native Image. ShardingSphere Proxy Native in GraalVM Native Image cannot recognize the additional JAR files.

```

<project>
  <dependencies>
    <dependency>
      <groupId>org.apache.shardingsphere</groupId>
      <artifactId>shardingsphere-transaction-base-seata-at</artifactId>
      <version>${shardingsphere.version}</version>
    </dependency>
    <dependency>
      <groupId>org.apache.seata</groupId>
      <artifactId>seata-all</artifactId>
      <version>2.2.0</version>
      <exclusions>
        <exclusion>
          <groupId>org.antlr</groupId>
          <artifactId>antlr4-runtime</artifactId>
        </exclusion>
      </exclusions>
    </dependency>
  </dependencies>
</project>

```

9.2.7 Session Management

ShardingSphere supports session management. You can view the current session or terminate the SQL executed in the session through the SQL of the native database. At present, this function is only available when the storage node is MySQL. MySQL `SHOW PROCESSLIST` and `KILL QUERY` commands are supported.

Usage

View Session

Different methods of viewing sessions are supported for different associated databases. The `SHOW PROCESSLIST` command can be used to view sessions for associated MySQL databases. ShardingSphere will automatically generate a unique UUID ID as the ID, and store the SQL execution information in each instance. When this command is executed, ShardingSphere will collect and synchronize the SQL execution information of each computing node through the governance center, and then summarize and return it to the user.

```
mysql> show processlist;
```

Id	Info	User	Host	db	Command	Time
05ede3bd584fd4a429dcaac382be2973	root	127.0.0.1	sharding_db	Execute	2	
Executing 0/1	select sleep(10)					
f9e5c97431567415fe10badc5fa46378	root	127.0.0.1	sharding_db	Sleep	690	

- Output Description

Simulates the output of native MySQL, but the Id field is a special random string.

KILL THE SQL IN THE SESSION

The user determines whether the `KILL QUERY` statement needs to be executed according to the results returned by `SHOW PROCESSLIST`. ShardingSphere cancels the SQL being executed according to the ID in the `KILL QUERY` statement.

```
-- Cancel the SQL statement currently executing in the specified session.
mysql> KILL QUERY 05ede3bd584fd4a429dcaac382be2973;
Query OK, 0 rows affected (0.04 sec)
```



```
mysql> SHOW PROCESSLIST;
Empty set (0.02 sec)
```

9.2.8 Logging Configuration

Background

ShardingSphere uses Logback for log management, and the Java SPI internally to provide default log configuration. Users can use XML files to configure customized log output. Proxy will preferentially read the log configuration provided in `logback.xml` in the `/conf` directory.

The following steps describe how to customize the log configuration.

Procedure

1. Create file `conf/logback.xml`

Customize the logger level and pattern, etc. according to your needs. > It is recommended to make modifications based on the configuration example

2. View logs

After ShardingSphere-Proxy starts, the log will be output to the `logs` directory, select the target log file to view.

Sample

```
<?xml version="1.0"?>
<!--
  ~ Licensed to the Apache Software Foundation (ASF) under one or more
  ~ contributor license agreements. See the NOTICE file distributed with
  ~ this work for additional information regarding copyright ownership.
  ~ The ASF licenses this file to You under the Apache License, Version 2.0
  ~ (the "License"); you may not use this file except in compliance with
  ~ the License. You may obtain a copy of the License at
  ~
  ~ http://www.apache.org/licenses/LICENSE-2.0
  ~
  ~ Unless required by applicable law or agreed to in writing, software
  ~ distributed under the License is distributed on an "AS IS" BASIS,
  ~ WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  ~ See the License for the specific language governing permissions and
  ~ limitations under the License.
-->

<configuration>
  <appender name="console" class="ch.qos.logback.core.ConsoleAppender">
```

```

        <encoder>
            <pattern>[%-5level] %d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %logger{36} -
            %msg%n</pattern>
        </encoder>
    </appender>
    <logger name="org.apache.shardingsphere" level="info" additivity="false">
        <appender-ref ref="console" />
    </logger>

    <logger name="com.zaxxer.hikari" level="error" />

    <logger name="com.atomikos" level="error" />

    <logger name="io.netty" level="error" />

    <root>
        <level value="info" />
        <appender-ref ref="console" />
    </root>
</configuration>

```

9.2.9 CDC

CDC (Change Data Capture) captures incremental data changes. CDC can monitor data changes in the storage nodes of ShardingSphere-Proxy, capture data operation events, filter and extract useful information, and finally send these changed data to a specified target.

CDC can be used for data synchronization, data backup and recovery, it currently supports openGauss, MySQL, and PostgreSQL.

Build

Background Information

ShardingSphere CDC is divided into two parts, one is the CDC Server, and the other is the CDC Client. The CDC Server and ShardingSphere-Proxy are currently deployed together.

Users can introduce the CDC Client into their own projects to implement data consumption logic.

Constraints

- Pure JAVA development, JDK recommended 1.8 or above.
- CDC Server requires ShardingSphere-Proxy to use cluster mode, currently supports ZooKeeper as the registry center.
- CDC only synchronizes data, does not synchronize table structure, and currently does not support DDL statement synchronization.
- CDC incremental task will not split transaction data of the database shards. If you want to enable XA transaction compatibility, both openGauss and ShardingSphere-Proxy need the GLT module.

CDC Server Deployment Steps

Here, the openGauss database is used as an example to introduce the deployment steps of the CDC Server.

Since the CDC Server is built into ShardingSphere-Proxy, you need to get ShardingSphere-Proxy. For details, please refer to the [proxy startup manual](#).

Configure GLT Module (Optional)

The official website's released binary package does not include the GLT module by default, if you are using the openGauss database with GLT functionality, you can additionally introduce the GLT module to ensure the integrity of XA transactions.

There are currently two ways to introduce the GLT module, and corresponding configurations need to be made in `global.yaml`.

1. Source code compilation and installation

1.1 Prepare the code environment, download in advance or use Git clone to download the [ShardingSphere](#) source code from Github.

1.2 Delete the `<scope>provided</scope>` tag of the `shardingsphere-global-clock-tso-provider-redis` dependency in `kernel/global-clock/type/tso/core/pom.xml` and the `<scope>provided</scope>` tag of `jedis` in `kernel/global-clock/type/tso/provider/redis/pom.xml`

1.3 Compile ShardingSphere-Proxy, for specific compilation steps, please refer to the [ShardingSphere Compilation Manual](#).

2. Directly introduce GLT dependencies

Can be introduced from the maven repository

2.1. `shardingsphere-global-clock-tso-provider-redis`, download the same version as ShardingSphere-Proxy

2.2. `jedis-4.3.1`

CDC Server User Manual

Modify the configuration file `conf/global.yaml` and turn on the CDC function. Currently, mode must be `Cluster`, and the corresponding registry center needs to be started in advance. If the GLT provider uses Redis, Redis needs to be started in advance.

Configuration example:

1. Enable CDC function in `global.yaml`.

```
mode:
  type: Cluster
  repository:
    type: ZooKeeper
    props:
      namespace: cdc_demo
      server-lists: localhost:2181
      retryIntervalMilliseconds: 500
      timeToLiveSeconds: 60
      maxRetries: 3
      operationTimeoutMilliseconds: 500

authority:
  users:
    - user: root@%
      password: root
  privilege:
    type: ALL_PERMITTED

# When using GLT, you also need to enable distributed transactions, GLT is only
# supported by the openGauss database currently.
#transaction:
#  defaultType: XA
#  providerType: Atomikos
#
#globalClock:
#  enabled: true
#  type: TSO
#  provider: redis
#  props:
```

```
# host: 127.0.0.1
# port: 6379

props:
  system-log-level: INFO
  proxy-default-port: 3307 # Proxy default port.
  cdc-server-port: 33071 # CDC Server port, must be configured
  proxy-frontend-database-protocol-type: openGauss # Consistent with the type of
  backend database
```

2. Introduce JDBC driver.

Proxy has included JDBC driver of PostgreSQL and openGauss.

If the backend is connected to the following databases, download the corresponding JDBC driver jar package and put it into the `${shardingsphere-proxy}/ext-lib` directory.

Database	JDBC Driver
MySQL	<code>mysql-connector-j-8.3.0.jar</code>

4. Start ShardingSphere-Proxy:

```
sh bin/start.sh
```

5. View the proxy log `logs/stdout.log`. If you see the following statements:

```
[INFO ] [main] o.a.s.p.frontend.ShardingSphereProxy - ShardingSphere-Proxy Cluster
mode started successfully
```

The startup will have been successful.

6. Configure CDC on demand.

6.1. Query configuration.

```
SHOW STREAMING RULE;
```

The default configuration is as follows:

```
+-----+-----+
| read                                     | write
| stream_channel                         |
+-----+-----+
| {"workerThread":20,"batchSize":1000,"shardingSize":10000000} | {"workerThread":20,
"batchSize":1000} | {"type":"MEMORY","props":{"block-queue-size":"2000"}} |
+-----+-----+
```

6.2. Alter configuration (optional).

Since the streaming rule has default values, there is no need to create it, only the ALTER statement is provided.

A completely configured DistSQL is as follows.

```
ALTER STREAMING RULE (
  READ(
    WORKER_THREAD=20,
    BATCH_SIZE=1000,
    SHARDING_SIZE=10000000,
    RATE_LIMITER (TYPE(NAME='QPS',PROPERTIES('qps'='500'))
  ),
  WRITE(
    WORKER_THREAD=20,
    BATCH_SIZE=1000,
    RATE_LIMITER (TYPE(NAME='TPS',PROPERTIES('tps'='2000'))
  ),
  STREAM_CHANNEL (TYPE(NAME='MEMORY',PROPERTIES('block-queue-size'='2000'))
);
```

Configuration item description:

```
ALTER STREAMING RULE (
  READ( -- Data reading configuration. If it is not configured, part of the parameters
    will take effect by default.
    WORKER_THREAD=20, -- Affects full and incremental tasks, obtain the thread pool
    size of all the data from the source side. If it is not configured, the default
    value is used. It needs to ensure that this value is not lower than the number of
    database shards.
    BATCH_SIZE=1000, -- Affects full and incremental tasks, the maximum number of
    records returned by a query operation. If it is not configured, the default value is
    used. If the amount of data in a transaction is greater than this value, the
    incremental situation may exceed the set value.
    SHARDING_SIZE=10000000, -- Affects full tasks, sharding size of all the data. If
    it is not configured, the default value is used.
    RATE_LIMITER ( -- Affects full and incremental tasks, traffic limit algorithm. If
    it is not configured, traffic is not limited.
      TYPE( -- Algorithm type. Option: QPS
        NAME='QPS',
        PROPERTIES( -- Algorithm property
          'qps'='500'
        )))
  ),
  WRITE( -- Data writing configuration. If it is not configured, part of the
    parameters will take effect by default.
    WORKER_THREAD=20, -- Affects full and incremental tasks, the size of the thread
    pool on which data is written into the target side. If it is not configured, the
    default value is used.
```

```

    BATCH_SIZE=1000, -- Affects full and incremental tasks, the maximum number of
    records for a batch write operation. If it is not configured, the default value is
    used. If the amount of data in a transaction is greater than this value, the
    incremental situation may exceed the set value.
    RATE_LIMITER ( -- Traffic limit algorithm. If it is not configured, traffic is not
    limited.
    TYPE( -- Algorithm type. Option: TPS
    NAME='TPS',
    PROPERTIES( -- Algorithm property.
    'tps'='2000'
    )))
),
STREAM_CHANNEL ( -- Data channel. It connects producers and consumers, used for
reading and writing procedures. If it is not configured, the MEMORY type is used by
default.
TYPE( -- Algorithm type. Option: MEMORY
NAME='MEMORY',
PROPERTIES( -- Algorithm property
'block-queue-size'='2000' -- Property: blocking queue size.
)))
);

```

CDC Client Manual

The CDC Client does not need to be deployed separately, just need to introduce the dependency of the CDC Client through maven to use it in the project. Users can interact with the server through the CDC Client.

If necessary, users can also implement a CDC Client themselves to consume data and ACK.

```

<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-data-pipeline-cdc-client</artifactId>
  <version>${version}</version>
</dependency>

```

CDC Client Introduction

`org.apache.shardingsphere.data.pipeline.cdc.client.CDCClient` is the entry class of the CDC Client. Users can interact with the CDC Server through this class. The main new methods are as follows.

Method Name		R e t u r n V a l u e	Description
connect(Consumer<List> data-Consumer, ExceptionHandler exceptionHandler, Se rverError-ResultHandler errorHandler)		v o i d	Connect with the server, when connecting, you need to specify 1. Data consumption processing function 2. Exception handling logic during consumption 3. Server error exception handling function
login(CDCLoginParameter parameter)		v o i d	CDC login, parameters username: username password: password
startStreaming(S tartStreamingParameter parameter)		s t r e a m i n g I d	Start CDC subscription StartStreamingParameter parameters database: logical database name schemaTables: subscribed table name full: whether to subscribe to full data
r estartStreaming(String streamingId)		v o i d	Restart subscription
stopStreaming(String streamingId)		v o i d	Stop subscription
dropStreaming(String streamingId)		v o i d	Delete subscription
await()		v o i d	Block the CDC thread and wait for the channel to close
close()		v o i d	Close the channel, the process ends

Manual

Introduction to CDC Function

CDC only synchronizes data, it does not synchronize table structures, and currently does not support the synchronization of DDL statements.

Introduction to CDC Protocol

The CDC protocol uses Protobuf, and the corresponding Protobuf types are mapped based on the types in Java.

Here, taking openGauss as an example, the mapping relationship between the data types of the CDC protocol and the database types is as follows.

openGauss type	Java data type	CDC corresponding protobuf type	Remarks
tinyint, smallint, integer	Integer	int32	
bigint	Long	int64	
numeric	BigDecimal	string	
real, float4	Float	float	
binary_double, double precision	Double	double	
boolean	Boolean	bool	
char, varchar, text, clob	String	string	
blob, bytea, raw	byte[]	bytes	
date, timestamp, timestampz, small-datetime	java.sql.Timestamp	Timestamp	The Timestamp type of protobuf only contains seconds and nanoseconds, so it is irrelevant to the time zone
time, timetz	java.sql.Time	int64	Represents the number of nanoseconds of the day, irrelevant to the time zone
interval, reftime, abstime	String	string	
point, lseg, box, path, polygon, circle	String	string	
cidr, inet, macaddr	String	string	
tsvector	String	string	
tsquery	String	String	
uuid	String	string	
json, jsonb	String	string	
hll	String	string	
int4range, date-range, tsrange, tstzrange	String	string	
hash16, hash32	String	string	
bit, bit varying	String	string	Returns Boolean type when bit(1)

openGauss User Manual

Environmental Requirements

Supported openGauss versions: 2.x ~ 3.x.

Permission Requirements

1. Adjust the source end WAL configuration.

Example configuration for `postgresql.conf`:

```
wal_level = logical
max_wal_senders = 10
max_replication_slots = 10
wal_sender_timeout = 0
max_connections = 600
```

For details, please refer to [Write Ahead Log and Replication](#).

2. Grant replication permission to the source end openGauss account.

Example configuration for `pg_hba.conf`:

```
host replication repl_acct 0.0.0.0/0 md5
# 0.0.0.0/0 means allowing access from any IP address, which can be adjusted to the
IP address of the CDC Server according to the actual situation
```

For details, please refer to [Configuring Client Access Authentication](#) and [Example: Logic Replication Code](#).

3. Grant DDL DML permissions to the openGauss account.

If a non-super administrator account is used, it is required that this account has CREATE and CONNECT permissions on the database used.

Example:

```
GRANT CREATE, CONNECT ON DATABASE source_ds TO cdc_user;
```

The account also needs to have access permissions to the table and schema to be subscribed, taking the `t_order` table under the `test` schema as an example.

```
\c source_ds

GRANT USAGE ON SCHEMA test TO GROUP cdc_user;
GRANT SELECT ON TABLE test.t_order TO cdc_user;
```

openGauss has the concept of OWNER. If it is the OWNER of the database, SCHEMA, or table, the corresponding authorization steps can be omitted.

openGauss does not allow ordinary accounts to operate under the public schema. So if the table to be migrated is under the public schema, additional authorization is needed.

```
GRANT ALL PRIVILEGES TO cdc_user;
```

For details, please refer to [openGauss GRANT](#)

Complete Process Example

Prerequisites

1. Prepare the database, table, and data of the CDC source end.

```
DROP DATABASE IF EXISTS ds_0;
CREATE DATABASE ds_0;

DROP DATABASE IF EXISTS ds_1;
CREATE DATABASE ds_1;
```

Configure CDC Server

1. Create a logical database.

```
CREATE DATABASE sharding_db;

\c sharding_db
```

2. Register storage unit.

```
REGISTER STORAGE UNIT ds_0 (
  URL="jdbc:opengauss://127.0.0.1:5432/ds_0",
  USER="gaussdb",
  PASSWORD="Root@123",
  PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
), ds_1 (
  URL="jdbc:opengauss://127.0.0.1:5432/ds_1",
  USER="gaussdb",
  PASSWORD="Root@123",
  PROPERTIES("minPoolSize"="1","maxPoolSize"="20","idleTimeout"="60000")
);
```

3. Create sharding rules.

```
CREATE SHARDING TABLE RULE t_order(
  STORAGE_UNITS(ds_0,ds_1),
  SHARDING_COLUMN=order_id,
  TYPE(NAME="hash_mod",PROPERTIES("sharding-count"="2")),
```

```
KEY_GENERATE_STRATEGY(COLUMN=order_id,TYPE(NAME="snowflake"))
);
```

4. Create tables

Execute the creation table statement in the proxy.

```
CREATE TABLE t_order (order_id INT NOT NULL, user_id INT NOT NULL, status
VARCHAR(45) NULL, PRIMARY KEY (order_id));
```

Start CDC Client

Currently, the CDC Client only provides a Java API, and users need to implement the data consumption themselves.

Below is a simple example of starting the CDC Client.

```
import lombok.SneakyThrows;
import lombok.extern.slf4j.Slf4j;
import org.apache.shardingsphere.data.pipeline.cdc.client.CDCClient;
import org.apache.shardingsphere.data.pipeline.cdc.client.config.
CDCClientConfiguration;
import org.apache.shardingsphere.data.pipeline.cdc.client.handler.
RetryStreamingExceptionHandler;
import org.apache.shardingsphere.data.pipeline.cdc.client.parameter.
CDCLoginParameter;
import org.apache.shardingsphere.data.pipeline.cdc.client.parameter.
StartStreamingParameter;
import org.apache.shardingsphere.data.pipeline.cdc.protocol.request.
StreamDataRequestBody.SchemaTable;

import java.util.Collections;

@Slf4j
public final class Bootstrap {

    @SneakyThrows(InterruptedException.class)
    public static void main(final String[] args) {
        String address = "127.0.0.1";
        // Construct CDCClient, pass in CDCClientConfiguration,
        CDCClientConfiguration contains the address and port of the CDC Server, as well as
        the timeout time
        try (CDCClient cdcClient = new CDCClient(new CDCClientConfiguration(address,
        33071, 10000))) {
            // First call connect to the CDC Server, you need to pass in 1. Data
            consumption processing logic 2. Exception handling logic during consumption 3.
            Server error exception handling logic
            cdcClient.connect(records -> log.info("records: {}", records), new
```

```

RetryStreamingExceptionHandler(cdcClient, 5, 5000),
    (ctx, result) -> log.error("Server error: {}", result.
getErrorMessage()));
    cdcClient.login(new CDCLoginParameter("root", "root"));
    // Start CDC data synchronization, the returned streamingId is the unique
    identifier of this CDC task, the basis for the CDC Server to generate a unique
    identifier is the name of the subscribed database + the subscribed table + whether
    it is full synchronization
    String streamingId = cdcClient.startStreaming(new
StartStreamingParameter("sharding_db", Collections.singleton(SchemaTable.
newBuilder().setTable("t_order").build()), true));
    log.info("Streaming id={}", streamingId);
    // Prevent the main thread from exiting
    cdcClient.await();
}
}
}

```

There are mainly 4 steps 1. Construct CDCClient, pass in CDCClientConfiguration 2. Call CDCClient.connect(), this step is to establish a connection with the CDC Server 3. Call CDCClient.login(), log in with the username and password configured in global.yaml 4. Call CDCClient.startStreaming(), start subscribing, you need to ensure that the subscribed database and table exist in ShardingSphere-Proxy, otherwise an error will be reported

CDCClient.await is to block the main thread, it is not a necessary step, other methods can also be used, as long as the CDC thread is always working.

If you need more complex data consumption implementation, such as writing to the database, you can refer to [DataSourceRecordConsumer](#)

Write Data

When write data through a proxy, the CDC Client is notified of the data change.

```

INSERT INTO t_order (order_id, user_id, status) VALUES (1,1,'ok1'),(2,2,'ok2'),(3,3,
'ok3');
UPDATE t_order SET status='updated' WHERE order_id = 1;
DELETE FROM t_order WHERE order_id = 2;

```

Bootstrap will output a similar log.

```

records: [before {
name: "order_id"
value {
  type_url: "type.googleapis.com/google.protobuf.Empty"
}
.....

```

View the Running Status of the CDC Task

The start and stop of the CDC task can only be controlled by the CDC Client. You can view the status of the CDC task by executing DistSQL in the proxy

1. View the CDC task list

```
SHOW STREAMING LIST;
```

Running result

```
sharding_db=> SHOW STREAMING LIST;
      id | database | tables | job_item_count |
active | create_time | stop_time
-----+-----+-----+-----
j0302p0000702a83116fcee83f70419ca5e2993791 | sharding_db | t_order | 1
| true | 2023-10-27 22:01:27 |
(1 row)
```

2. View the details of the CDC task

```
SHOW STREAMING STATUS j0302p0000702a83116fcee83f70419ca5e2993791;
```

Running result

```
sharding_db=> SHOW STREAMING STATUS j0302p0000702a83116fcee83f70419ca5e2993791;
 item | data_source | status | active | processed_records_count |
inventory_finished_percentage | incremental_idle_seconds | confirmed_position |
current_position | error_message
-----+-----+-----+-----+-----+-----+-----
+-----+-----+-----+-----+-----+-----+-----
0 | ds_0 | EXECUTE_INCREMENTAL_TASK | false | 2 | 100
| 115 | 5/597E43D0 | 5/597E4810 |
1 | ds_1 | EXECUTE_INCREMENTAL_TASK | false | 3 | 100
| 115 | 5/597E4450 | 5/597E4810 |
(2 rows)
```

3. Drop CDC task

```
DROP STREAMING j0302p0000702a83116fcee83f70419ca5e2993791;
```

The CDC task can only be deleted when there are no subscriptions. At this time, the replication slots on the openGauss physical database will also be deleted.

```
sharding_db=> DROP STREAMING j0302p0000702a83116fcee83f70419ca5e2993791;
SUCCESS
```

Precautions

Explanation of incremental data push

1. The CDC incremental push is currently transactional, and the transactions of the physical database will not be split. Therefore, if there are data changes in multiple tables in a transaction, these data changes will be pushed together. If you want to support XA transactions (currently only supports openGauss), both openGauss and Proxy need the GLT module.
2. The conditions for push are met when a certain amount of data is met or a certain time interval is reached (currently 300ms). When processing XA transactions, if the received multiple physical database incremental events exceed 300ms, it may cause the XA transaction to be split and pushed.

Handling of large transactions

Currently, large transactions are fully parsed, which may cause the CDC Server process to OOM. In the future, forced truncation may be considered.

Recommended configuration

There is no fixed value for the performance of CDC, you can focus on the batchSize of read/write in the configuration, and the size of the memory queue, and tune it according to the actual situation.

9.3 Common Configuration

This chapter mainly introduces general configuration, including property configuration and built-in algorithm configuration.

9.3.1 Properties Configuration

Background

Apache ShardingSphere provides the way of property configuration to configure system level configuration.

Parameters

Name*	Data Type*	Description	DefaultValue*
sql-show (?)	boolean	Whether show SQL or not in log. Print SQL details can help developers debug easier. The log details include: logic SQL, actual SQL and SQL parse result. Enable this property will log into log topic ShardingSphere-SQL, log level is INFO	false
sql-simple (?)	boolean	Whether show SQL details in simple style	false
kernel-executor-size (?)	int	The max thread size of worker group to execute SQL. One ShardingSphereDataSource will use a independent thread pool, it does not share thread pool even different data source in same JVM	infinite
max-connections-size-per-query (?)	int	Max opened connection size for each query	1
check-table-metadata-enabled (?)	boolean	Whether validate table meta data consistency when application startup or updated	false
load-table-metadata-batch-size (?)	int	The number of table metadata loaded at a time when application startup or refreshes table metadata	1000

Procedure

1. Properties configuration is directly configured in the profile used by ShardingSphere-JDBC. The format is as follows:

```
props:
  sql-show: true
```

Notes

The default value of the `max-connections-size-per-query` configuration is 1, meaning each query request can only use one connection per database instance. If you adjust this parameter to enable memory-restricted mode (see [Memory-Strictly Mode](#) for details), ensure that your database's JDBC implementation supports streaming queries or can enable them. For example, in MySQL, you need to set `statement.setFetchSize(Integer.MIN_VALUE)` to achieve streaming queries.

Sample

The example of ShardingSphere warehouse contains property configurations of various scenarios. Please refer to: <https://github.com/apache/shardingsphere/tree/master/examples>

9.3.2 Builtin Algorithm

Introduction

Apache ShardingSphere allows developers to implement algorithms via SPI; At the same time, Apache ShardingSphere also provides a couple of builtin algorithms for simplify developers.

Usage

The builtin algorithms are configured by type and props. Type is defined by the algorithm in SPI, and props is used to deliver the customized parameters of the algorithm.

No matter which configuration type is used, the configured algorithm is named and passed to the corresponding rule configuration. This chapter distinguishes and lists all the builtin algorithms of Apache ShardingSphere according to its functions for developers' reference.

Metadata Repository

Background

Apache ShardingSphere provides different metadata persistence methods for different running modes. Users can freely choose the most appropriate way to store metadata while configuring the running mode.

Parameters

Database Repository

The optional values of `provider` are H2, MySQL, EmbeddedDerby, DerbyNetworkServer and HSQLDB. Since third-party Vulnerability Reports often misreport H2 Database, avoiding the use of H2 Database in ShardingSphere Standalone Mode may be an option. Discuss the case where `provider` is not the default value H2.

1. If `provider` is set to MySQL, a ready MySQL Server is required. The classpath should contain the Maven dependency of `com.mysql:mysql-connector-j:9.0.0`.
2. If `provider` is set to EmbeddedDerby, the Derby database engine will run in the same JVM as the application. The classpath should contain Maven dependencies of `org.apache.derby:derby:10.17.1.0` and `org.apache.derby:derbytools:10.17.1.0`, and the JDK version required to compile or run the downstream project is greater than or equal to JDK19. Possible configurations are as follows.

```
mode:
  type: Standalone
  repository:
    type: JDBC
    props:
      provider: EmbeddedDerby
      jdbc_url: jdbc:derby:memory:config;create=true
      username:
```

3. If `provider` is set to DerbyNetworkServer, a ready Derby Network Server is required. There is no available Docker Image for Derby Network Server, and users may need to start Derby Network Server manually. The classpath should contain Maven dependencies of `org.apache.derby:derbyclient:10.17.1.0` and `org.apache.derby:derbytools:10.17.1.0`, and the JDK version required to compile or run the downstream project is greater than or equal to JDK19.
4. If `provider` is set to HSQLDB, a ready HyperSQL using Server Modes is required, or a database is created as an in-process database. The classpath should contain the Maven dependency of `org.hsqldb:hsqldb:2.7.3` with `classifier` as `jdk8`. There is no available Docker Image for HyperSQL using Server Modes, and users may need to manually start HyperSQL using Server Modes. If HyperSQL using `mem:` protocol is used, the possible configuration is as follows,

```
mode:
  type: Standalone
  repository:
    type: JDBC
    props:
      provider: HSQLDB
      jdbc_url: jdbc:hsqldb:mem:config
      username: SA
```

Type: JDBC

Mode: Standalone

Attributes:

Name	Type	Description	Default Value
provider	String	Type for metadata persist	H2
jdbc_url	String	JDBC URL	jdbc:h2:mem:config;DB_CLOSE_DELAY=-1;DATABASE_TO_UPPER=false;MODE=Memory
username	String	username	sa
password	String	password	

ZooKeeper Repository

Type: ZooKeeper

Mode: Cluster

Attributes:

Name	Type	Description	Default Value
retryIntervalMilliseconds	int	Milliseconds of retry interval	500
maxRetries	int	Max retries of client connection	3
timeToLiveSeconds	int	Seconds of ephemeral data live	60
operationTimeoutMilliseconds	int	Milliseconds of operation timeout	500
digest	String	Password of login	

Etcd Repository

Type: Etcd

Mode: Cluster

Attributes:

Name	Type	Description	Default Value
timeToLiveSeconds	long	Seconds of ephemeral data live	30
connectionTimeout	long	Seconds of connection timeout	30

Procedure

1. Configure running mode in global.yaml.
2. Configure metadata persistence warehouse type.

Sample

- Standalone mode configuration method.

```
mode:
  type: Standalone
  repository:
    type: JDBC
    props:
      provider: H2
      jdbc_url: jdbc:h2:mem:config;DB_CLOSE_DELAY=-1;DATABASE_TO_UPPER=false;
MODE=MYSQL
  username: test
  password: Test@123
```

- Cluster mode.

```
mode:
  type: Cluster
  repository:
    type: zookeeper
    props:
      namespace: governance_ds
      server-lists: localhost:2181
      retryIntervalMilliseconds: 500
      timeToLiveSeconds: 60
      maxRetries: 3
      operationTimeoutMilliseconds: 500
```

Sharding Algorithm

Background

ShardingSphere built-in algorithms provide a variety of sharding algorithms, which can be divided into automatic sharding algorithms, standard sharding algorithms, composite sharding algorithms, and hint sharding algorithms, and can meet the needs of most business scenarios of users.

Additionally, considering the complexity of business scenarios, the built-in algorithm also provides a way to customize the sharding algorithm. Users can complete complex sharding logic by writing java code.

It should be noted that the sharding logic of the automatic sharding algorithm is automatically managed by ShardingSphere and needs to be used by configuring the autoTables sharding rules.

Parameters

Auto Sharding Algorithm

Modulo Sharding Algorithm

Type: MOD

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
sharding-count	int	Sharding count

Hash Modulo Sharding Algorithm

Type: HASH_MOD

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
sharding-count	int	Sharding count

Volume Based Range Sharding Algorithm

Type: VOLUME_RANGE

Attributes:

Name	Data Type	Description
range-lower	long	Range lower bound, throw exception if lower than bound
range-upper	long	Range upper bound, throw exception if upper than bound
sharding-volume	long	Sharding volume

Boundary Based Range Sharding Algorithm

Type: BOUNDARY_RANGE

Attributes:

Name	Data Type	Description
sharding-ranges	String	Range of sharding border, multiple boundaries separated by commas

Auto Interval Sharding Algorithm

Type: AUTO_INTERVAL

Attributes:

Name	Data Type *	Description
datetime-lower	String	Shard datetime begin boundary, pattern: yyyy-MM-dd HH:mm:ss
datetime-upper	String	Shard datetime end boundary, pattern: yyyy-MM-dd HH:mm:ss
sharding-seconds	long	Max seconds for the data in one shard, allows sharding key timestamp format seconds with time precision, but time precision after seconds is automatically erased

Standard Sharding Algorithm

Apache ShardingSphere built-in standard sharding algorithm are:

Inline Sharding Algorithm

With Groovy expressions that uses the default implementation of the `InlineExpressionParser` SPI, `InlineShardingStrategy` provides single-key support for the sharding operation of `=` and `IN` in SQL. Simple sharding algorithms can be used through a simple configuration to avoid laborious Java code developments. For example, `t_user_${u_id % 8}` means table `t_user` is divided into 8 tables according to `u_id`, with table names from `t_user_0` to `t_user_7`. Please refer to [Inline Expression](#) for more details.

Type: `INLINE`

Attributes:

Name	Data Type *	Description	Default Value
algorithm-expression	String	Inline expression sharding algorithm	.
allow-range-query-with-inline-sharding (?)	boolean	Whether range query is allowed. Note: range query will ignore sharding strategy and conduct full routing	false

Interval Sharding Algorithm

This algorithm actively ignores the time zone information of `datetime-pattern`. This means that when `datetime-lower`, `datetime-upper` and the incoming shard key contain time zone information, time zone conversion will not occur due to time zone inconsistencies.

When the shard key passed in is `java.time.Instant` or `java.util.Date`, there is a special case. It will carry the system's time zone information and convert it into a string format of `datetime-pattern` before the next sharding.

Type: `INTERVAL`

Attributes:

Name	DataType*	Description	DefaultValue*
date time -pat tern	String	Timestamp pattern of sharding value, must can be transformed to Java LocalDateTime. For example: yyyy-MM-dd HH:mm:ss, yyyy-MM-dd or HH:mm:ss etc. But GGGy-MM etc. related to java.time.chrono. JapaneseDate are not supported	.
da teti me-l ower	String	Datetime sharding lower boundary, pattern is defined datetime-pattern	.
da teti me-u pper (?)	String	Datetime sharding upper boundary, pattern is defined datetime-pattern	N o w
sha rdin g-su ffix -pat tern	String	Suffix pattern of sharding data sources or tables, must can be transformed to Java LocalDateTime, must be consistent with date-time-interval-unit For example: yyyyMM	.
date time -int erval-am ount (?)	int	Interval of sharding value, after which the next shard will be entered	1
da teti me-i nter val-unit (?)	String	Unit of sharding value interval, must can be transformed to Java ChronoUnit's Enum value. For example: MONTHS	D A Y S

Complex Sharding Algorithm

Complex Inline Sharding Algorithm

Please refer to [Inline Expression](#) for more details.

Type: COMPLEX_INLINE

Name	Data Type*	Description	Default Value
sharding-columns (?)	String	sharding column names	.
algorithm-expression	String	Inline expression sharding algorithm	.
allow-range-query-with-inline-sharding (?)	boolean	Whether range query is allowed. Note: range query will ignore sharding strategy and conduct full routing	false

Hint Sharding Algorithm

Hint Inline Sharding Algorithm

Please refer to [Inline Expression](#) for more details.

Type: COMPLEX_INLINE

Name	Data Type	Description	Default Value
algorithm-expression	String	Inline expression sharding algorithm	\${value}

Class Based Sharding Algorithm

Realize custom extension by configuring the sharding strategy type and algorithm class name. CLASS_BASED allows additional custom properties to be passed into the algorithm class. The passed properties can be retrieved through the `java.util.Properties` class instance with the property name props. Refer to Git's `org.apache.shardingsphere.example.extension.sharding.algorithm.classbased.fixture.ClassBasedStandardShardingAlgorithmFixture`.

Type: CLASS_BASED

Attributes:

Name	Data Type	Description
strategy	String	Sharding strategy type, support STANDARD, COMPLEX or HINT (case insensitive)
algorithmName	String	Fully qualified name of sharding algorithm

Procedure

1. When using data sharding, configure the corresponding data sharding algorithm under the shardingAlgorithms attribute.

Sample

```
rules:
- !SHARDING
  tables:
    t_order:
      actualDataNodes: ds_${0..1}.t_order_${0..1}
      tableStrategy:
        standard:
          shardingColumn: order_id
          shardingAlgorithmName: t_order_inline
      keyGenerateStrategy:
        column: order_id
        keyGeneratorName: snowflake
    t_order_item:
      actualDataNodes: ds_${0..1}.t_order_item_${0..1}
      tableStrategy:
        standard:
          shardingColumn: order_id
          shardingAlgorithmName: t_order_item_inline
      keyGenerateStrategy:
        column: order_item_id
        keyGeneratorName: snowflake
    t_account:
      actualDataNodes: ds_${0..1}.t_account_${0..1}
      tableStrategy:
        standard:
          shardingAlgorithmName: t_account_inline
      keyGenerateStrategy:
        column: account_id
        keyGeneratorName: snowflake
  defaultShardingColumn: account_id
```

```

bindingTables:
  - t_order,t_order_item
defaultDatabaseStrategy:
  standard:
    shardingColumn: user_id
    shardingAlgorithmName: database_inline
defaultTableStrategy:
  none:

shardingAlgorithms:
  database_inline:
    type: INLINE
    props:
      algorithm-expression: ds_${user_id % 2}
  t_order_inline:
    type: INLINE
    props:
      algorithm-expression: t_order_${order_id % 2}
  t_order_item_inline:
    type: INLINE
    props:
      algorithm-expression: t_order_item_${order_id % 2}
  t_account_inline:
    type: INLINE
    props:
      algorithm-expression: t_account_${account_id % 2}
keyGenerators:
  snowflake:
    type: SNOWFLAKE

- !BROADCAST
tables:
  - t_address

```

Related References

- [Core Feature: Data Sharding](#)
- [Developer Guide: Data Sharding](#)

Key Generate Algorithm

Background

In traditional database software development, automatic primary key generation is a basic requirement and various databases provide support for this requirement, such as MySQL's self-incrementing keys, Oracle's self-incrementing sequences, etc.

After data sharding, it is a very tricky problem to generate global unique primary keys from different data nodes. Self-incrementing keys between different actual tables within the same logical table generate duplicate primary keys because they are not mutually perceived.

Although collisions can be avoided by constraining the initial value and step size of self-incrementing primary keys, additional O&M rules must be introduced, making the solution lack completeness and scalability.

There are many third-party solutions that can perfectly solve this problem, such as UUID, which relies on specific algorithms to generate non-duplicate keys, or by introducing primary key generation services.

In order to cater to the requirements of different users in different scenarios, Apache ShardingSphere not only provides built-in distributed primary key generators, such as UUID, SNOWFLAKE, but also abstracts the interface of distributed primary key generators to facilitate users to implement their own customized primary key generators.

Parameters

Snowflake

Type: SNOWFLAKE

Attributes:

<i>Name</i>	<i>Data Type *</i>	<i>Description</i>	<i>DefaultValue *</i>
worker-id (?)	long	The unique ID for working machine	0
max-tolerance-milliseconds (?)	long	The max tolerate time for different server's time difference in milliseconds	10
max-vibration-offset (?)	int	The max upper limit value of vibrate number, range [0, 4096). Notice: To use the generated value of this algorithm as sharding value, it is recommended to configure this property. The algorithm generates $\text{key} \bmod 2^n$ (2^n is usually the sharding amount of tables or databases) in different milliseconds and the result is always 0 or 1. To prevent the above sharding problem, it is recommended to configure this property, its value is $(2^n) - 1$	1

Note: worker-id is optional 1. In standalone mode, support user-defined configuration, if the user does not configure the default value of 0. 2. In cluster mode, it will be automatically generated by the system, and duplicate values will not be generated in the same namespace.

UUID

Type: UUID

Attributes: None

Procedure

1. Policy of distributed primary key configurations is for columns when configuring data sharding rules.

Sample

- Snowflake Algorithms

```
keyGenerators:
  snowflake:
    type: SNOWFLAKE
```

- UUID

```
keyGenerators:
  uuid:
    type: UUID
```

Load Balance Algorithm

Background

ShardingSphere built-in provides a variety of load balancer algorithms, including polling algorithm, random access algorithm and weight access algorithm, which can meet users' needs in most business scenarios.

Moreover, considering the complexity of the business scenario, the built-in algorithm also provides an extension mode. Users can implement the load balancer algorithm they need based on SPI interface.

Parameters

Round-robin Load Balance Algorithm

Type: ROUND_ROBIN

Random Load Balance Algorithm

Type: RANDOM

Weight Load Balance Algorithm

Type: WEIGHT

Attributes:

<i>N a m e</i>	<i>. Data Type *</i>	<i>Description</i>
<code>\${replica-name}</code>	<code>double</code>	Attribute name uses the name of the replica, and the parameter fills in the weight value corresponding to the replica. Weight parameter range min > 0, total <= Double.MAX_VALUE.

Procedure

1. Configure a load balancer algorithm for the loadBalancers attribute to use read/write splitting.

Sample

```
rules:
- !READWRITE_SPLITTING
  dataSourceGroups:
    readwrite_ds:
      writeDataSourceName: write_ds
      readDataSourceNames:
        - read_ds_0
        - read_ds_1
      transactionalReadQueryStrategy: PRIMARY
      loadBalancerName: random
  loadBalancers:
    random:
      type: RANDOM
```

Related References

- [Core Feature: Read/Write Splitting](#)
- [Developer Guide: Read/Write Splitting](#)

Encryption Algorithm

Background

Encryption algorithms are by the encryption features of Apache ShardingSphere. A variety of algorithms are built-in to make it easy for users to fully leverage the feature.

Parameters

Standard Encrypt Algorithm

AES Encrypt Algorithm

Type: AES

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
aes-key-value	String	AES KEY
digest-algorithm-name	String	AES KEY DIGEST ALGORITHM

Assisted Encrypt Algorithm

MD5 Assisted Encrypt Algorithm

Type: MD5

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
salt	String	Salt value(optional)

Operating Procedure

1. Configure encryptors in an encryption rule.
2. Use relevant algorithm types in encryptors.

Configuration Examples

```
rules:
- !ENCRYPT
  tables:
    t_user:
      columns:
        username:
          cipher:
            name: username
            encryptorName: name_encryptor
          assistedQuery:
            name: assisted_username
            encryptorName: assisted_encryptor
      encryptors:
        name_encryptor:
          type: AES
          props:
            aes-key-value: 123456abc
            digest-algorithm-name: SHA-1
        assisted_encryptor:
          type: MD5
          props:
            salt: 123456
```

Related References

- [Core Feature: Data Encrypt](#)
- [Developer Guide: Data Encrypt](#)

Shadow Algorithm

Background

The shadow DB feature carries out shadow measurement to SQL statements executed. Shadow measurement supports two types of algorithms, and users can choose one or a combination of them based on actual business needs.

Parameters

Column-based shadow algorithm

Column value matching shadow algorithm

Type: VALUE_MATCH

Attribute Name	Data Type	Description
column	String	shadow column
operation	String	SQL operation type (INSERT, UPDATE, DELETE, SELECT)
value	String	value matched by shadow column

Column-based Regex matching algorithm

Type: REGEX_MATCH

Attribute Name	Data Type	Description
column	String	match a column
operation	String	SQL operation type (INSERT, UPDATE, DELETE, SELECT)
regex	String	shadow column matching Regex

Hint-based shadow algorithm

SQL HINT shadow algorithm

Type: SQL_HINT

```
/* SHARDINGSPHERE_HINT: SHADOW=true */
```

Configuration sample

- Java API

```
public final class ShadowConfiguration {
    // ...

    private AlgorithmConfiguration createShadowAlgorithmConfiguration() {
        Properties userIdInsertProps = new Properties();
        userIdInsertProps.setProperty("operation", "insert");
        userIdInsertProps.setProperty("column", "user_id");
        userIdInsertProps.setProperty("value", "1");
    }
}
```

```

        return new AlgorithmConfiguration("VALUE_MATCH", userIdInsertProps);
    }

    // ...
}

```

- YAML:

```

shadowAlgorithms:
  user-id-insert-algorithm:
    type: VALUE_MATCH
    props:
      column: user_id
      operation: insert
      value: 1

```

SQL Translator

Native SQL translator

Type: NATIVE

Attributes:

None

Default SQL translator, does not implement yet.

Sharding Audit Algorithm

Background

The sharding audit is to audit the SQL statements in the sharding database. Sharding audit not only intercept illegal SQL statements, but also gather the SQL statistics.

Parameters

DML_SHARDING_CONDITIONS algorithm

Type: DML_SHARDING_CONDITIONS

Procedure

1. when configuring data sharding rules, create sharding audit configurations.

Sample

- DML_SHARDING_CONDITIONS

```
auditors:
  sharding_key_required_auditor:
    type: DML_SHARDING_CONDITIONS
```

Data Masking Algorithm

Background

Data masking algorithms are by the mask features of Apache ShardingSphere. A variety of algorithms are built-in to make it easy for users to fully leverage the feature.

Parameters

Hash Data Masking Algorithm

MD5 Data Masking Algorithm

Type: MD5

Attributes:

Name	Data Type	Description
salt	String	Salt value (optional)

Mask Data Masking Algorithm

Keep First N Last M Data Masking Algorithm

Type: KEEP_FIRST_N_LAST_M

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
first-n	int	first n substring
last-m	int	last m substring
replace-char	String	replace char

Keep From X To Y Data Masking Algorithm

Type: KEEP_FROM_X_TO_Y

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
from-x	int	start position (from 0)
to-y	int	end position (from 0)
replace-char	String	replace char

Mask First N Last M Data Masking Algorithm

Type: MASK_FIRST_N_LAST_M

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
first-n	int	first n substring
last-m	int	last m substring
replace-char	String	replace char

Mask From X To Y Data Masking Algorithm

Type: MASK_FROM_X_TO_Y

Attributes:

<i>Name</i>	<i>DataType</i>	<i>Description</i>
from-x	int	start position (from 0)
to-y	int	end position (from 0)
replace-char	String	replace char

Mask Before Special Chars Data Masking Algorithm

Type: MASK_BEFORE_SPECIAL_CHARS

Attributes:

Name	DataType	Description
special-chars	String	Special chars (first appearance)
replace-char	String	replace char

Mask After Special Chars Data Masking Algorithm

Type: MASK_AFTER_SPECIAL_CHARS

Attributes:

Name	DataType	Description
special-chars	String	Special chars (first appearance)
replace-char	String	replace char

Replace Data Masking Algorithm

Generic table random replace algorithm.

Type: GENERIC_TABLE_RANDOM_REPLACE

Attributes:

Name	DataType	Description
uppercase-letter-codes	String	Uppercase letter codes (separate with comma, default value: A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z)
lowercase-letter-codes	String	Lowercase-letter codes (separate with comma, default value: a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,z)
digital-random-codes	String	Numbers (separate with comma, default value: 0,1,2,3,4,5,6,7,8,9)
special-codes	String	Special codes (separate with comma, default value: ~,!,@,#,\$,%,^,&,*,;,<,>,)

Operating Procedure

1. Configure maskAlgorithms in a mask rule.
2. Use relevant algorithm types in maskAlgorithms.

Configuration Examples

```
rules:
- !MASK
  tables:
    t_user:
      columns:
        password:
          maskAlgorithm: md5_mask
        email:
          maskAlgorithm: mask_before_special_chars_mask
        telephone:
          maskAlgorithm: keep_first_n_last_m_mask

  maskAlgorithms:
    md5_mask:
      type: MD5
    mask_before_special_chars_mask:
      type: MASK_BEFORE_SPECIAL_CHARS
      props:
        special-chars: '@'
        replace-char: '*'
    keep_first_n_last_m_mask:
      type: KEEP_FIRST_N_LAST_M
      props:
        first-n: 3
        last-m: 4
        replace-char: '*'
```

Related References

- [Core Feature: Data Masking](#)
- [Developer Guide: Data Masking](#)

Row Value Expressions

Row Value Expressions that uses the Groovy syntax

Type: GROOVY

Just use `${ expression }` or `$->{ expression }` in the configuration to identify the row expressions. The content of row expressions uses Groovy syntax, and all operations supported by Groovy are supported by row expressions. `${begin..end}` denotes the range interval, `${[unit1, unit2, unit_x]}` denotes the enumeration value. If there are multiple `${ expression }` or `$->{ expression }` expressions in a row expression, the final result of the whole expression will be a Cartesian combination based on the result of each sub-expression.

Example:

- `<GROOVY>t_order_${1..3}` will be converted to `t_order_1`, `t_order_2`, `t_order_3`
- `<GROOVY>${['online', 'offline']}_table${1..3}` will be converted to `online_table1`, `online_table2`, `online_table3`, `offline_table1`, `offline_table2`, `offline_table3`

Row Value Expressions that uses a standard list

The LITERAL implementation will not convert any symbols to the expression part, and will directly obtain the output of the standard list from the input of the standard list. This helps address the issue that Groovy expressions are inconvenient to use under GraalVM Native Image.

Type: LITERAL

Example:

- `<LITERAL>t_order_1, t_order_2, t_order_3` will be converted to `t_order_1`, `t_order_2`, `t_order_3`
- `<LITERAL>t_order_${1..3}` will be converted to `t_order_${1..3}`

Row Value Expressions based on fixed interval that uses the Key-Value syntax

The INTERVAL implementation introduces a Key-Value style property syntax to define a set of time ranges of strings via a single line string. This is often used to simplify the definition of actualDataNodes for Sharding feature. The string generated by INTERVAL is only applicable to multiple tables in a single real database. If you need to process multiple tables in multiple real databases, consider creating your own SPI implementation class of `org.apache.shardingsphere.infra.expr.spi.InlineExpressionParser`.

INTERVAL implements the method of defining multiple attributes as `Key1=Value1;Key2=Value2`, using `;` to separate key-value pairs, and `=` to separate Key values and Value values.

This implementation actively ignores the time zone information of SP, which means that when DL and DU contain time zone information, no time zone conversion will occur due to inconsistent time zones.

This implementation is not sensitive to the order of key-value pairs, and the line expression does not carry the `;` sign at the end.

The INTERVAL implementation introduces the following Key values:

1. P stands for the abbreviation of prefix, which means the prefix of the result list unit, usually representing the prefix format of the real table.
2. SP stands for the abbreviation of suffix pattern, which means the timestamp format of the suffix of the result list unit. It usually represents the suffix format of the real table and must follow the format of Java `DateTimeFormatter`. For example: `yyyyMMdd`, `yyyyMMdd` or `yyyy` etc.
3. DIA stands for the abbreviation of datetime interval amount, which means the time interval of the result list unit.
4. DIU stands for the abbreviation of datetime interval unit, which means the shard key time interval unit. It must follow the enumeration value of Java `java.time.temporal.ChronoUnit#toString()`. For example: `Months`.
5. DL stands for the abbreviation of datetime lower, which means the lower bound of time. The format is consistent with the timestamp format defined by SP.
6. DU stands for the abbreviation of datetime upper, which means the upper bound value of time. The format is consistent with the timestamp format defined by SP.
7. C stands for the abbreviation of chronology, which means calendar system and must follow the format of Java `java.time.chrono.Chronology#getId()`. For example: `Japanese`, `Min-guo`, `ThaiBuddhist`. There is a default value of `ISO`.

Whether the Value corresponding to the Key of C is available depends on the system environment in which the JVM is located. This means that if the user needs to set `C=Japanese`, they may need to call `java.util.Locale.setDefault(java.util.Locale.JAPAN)`; in the application's startup class to modify the system environment. Discuss two JVM environments.

1. Hotspot JVM determines the return value of `java.util.Locale.getDefault()` at RunTime.
2. GraalVM Native Image determines the return value of `java.util.Locale.Locale.getDefault()` at BuildTime, which is inconsistent with the performance of Hotspot JVM.

Refer to <https://github.com/oracle/graal/issues/8022>.

Type: INTERVAL

Example:

- `<INTERVAL>P=t_order_;SP=yyyy_MMdd;DIA=1;DIU=Days;DL=2023_1202;DU=2023_1204` will be converted to `t_order_2023_1202`, `t_order_2023_1203`, `t_order_2023_1204`
- `<INTERVAL>P=t_order_;SP=yyyy_MM;DIA=1;DIU=Months;DL=2023_10;DU=2023_12` will be converted to `t_order_2023_10`, `t_order_2023_11`, `t_order_2023_12`
- `<INTERVAL>P=t_order_;SP=yyyy;DIA=1;DIU=Years;DL=2021;DU=2023` will be converted to `t_order_2021`, `t_order_2022`, `t_order_2023`
- `<INTERVAL>P=t_order_;SP=HH_mm_ss_SSS;DIA=1;DIU=Millis;DL=22_48_52_131;DU=22_48_52_133` will be converted to `t_order_22_48_52_131`, `t_order_22_48_52_132`, `t_order_22_48_52_133`
- `<INTERVAL>P=t_order_;SP=yyyy_MM_dd_HH_mm_ss_SSS;DIA=1;DIU=Days;DL=2023_12_04_22_48_52_131;DU=2023_12_06_22_48_52_131` will be converted to `t_order_2023_12_04_22_48_52_131`, `t_order_2023_12_05_22_48_52_131`, `t_order_2023_12_06_22_48_52_131`
- `<INTERVAL>P=t_order_;SP=MM;DIA=1;DIU=Months;DL=10;DU=12` will be converted to `t_order_10`, `t_order_11`, `t_order_12`
- `<INTERVAL>P=t_order_;SP=GGGGyyyy_MM_dd;DIA=1;DIU=Days;DL=平成 0001_12_05;DU=平成 0001_12_06;C=Japanese` will be converted to `t_order_平成 0001_12_05`, `t_order_平成 0001_12_06`
- `<INTERVAL>P=t_order_;SP=GGGGyyy_MM_dd;DIA=1;DIU=Days;DL=平成 001_12_05;DU=平成 001_12_06;C=Japanese` will be converted to `t_order_平成 001_12_05`, `t_order_平成 001_12_06`
- `<INTERVAL>P=t_order_;SP=GGGGy_MM_dd;DIA=1;DIU=Days;DL=平成 1_12_05;DU=平成 1_12_06;C=Japanese` will be converted to `t_order_平成 1_12_05`, `t_order_平成 1_12_06`

Row Value Expressions that uses the Groovy syntax based on GraalVM Truffle' s Espresso implementation

This is an optional implementation. You need to actively declare the following dependencies in the `pom.xml` of your own project. And please make sure your own projects are compiled with OpenJDK 21+ or its downstream distribution.

Due to the limitation of <https://www.graalvm.org/jdk21/reference-manual/java-on-truffle/faq/#does-java-running-on-truffle-run-on-hotspot-too>, when this module is used in a non-GraalVM Native Image environment, it is only ready on Linux with System Property `os.arch` set to `amd64`.

Truffle' s backward compatibility matrix with the JDK is located at <https://medium.com/graalvm/40027a59c401>.

```
<dependencies>
  <dependency>
    <groupId>org.apache.shardingsphere</groupId>
    <artifactId>shardingsphere-infra-expr-espresso</artifactId>
    <version>${shardingsphere.version}</version>
  </dependency>
  <dependency>
    <groupId>org.graalvm.polyglot</groupId>
    <artifactId>polyglot</artifactId>
    <version>24.1.0</version>
  </dependency>
  <dependency>
    <groupId>org.graalvm.polyglot</groupId>
    <artifactId>java</artifactId>
    <version>24.1.0</version>
    <type>pom</type>
  </dependency>
</dependencies>
```

ESPRESSO is still an experimental module that allows the use of Row Value Expressions with Groovy syntax under GraalVM Native Image through the Espresso implementation of GraalVM Truffle.

The syntax part is the same as the GROOVY implementation rules.

Type: ESPRESSO

Example:

- <ESPRESSO>t_order_\${1..3} will be converted to t_order_1, t_order_2, t_order_3
- <ESPRESSO>\${[['online', 'offline']]_table\${1..3}} will be converted to online_table1, online_table2, online_table3, offline_table1, offline_table2, offline_table3

Custom Implementation

Users can always create their own implementation class of `org.apache.shardingsphere.infra.expr.spi.InlineExpressionParser`, to cover more complex scenarios, including connecting to a remote Elasticsearch cluster to execute ES|QL to obtain `java.util.List<String>`.

Consider a simple SPI implementation class,

```
import org.apache.shardingsphere.infra.expr.spi.InlineExpressionParser;
import java.util.Arrays;
import java.util.List;
import java.util.Properties;
public final class CustomInlineExpressionParserFixture implements
InlineExpressionParser {
    private String inlineExpression;
    @Override
```

```

public void init(final Properties props) {
    inlineExpression = props.getProperty(INLINE_EXPRESSION_KEY);
}
@Override
public List<String> splitAndEvaluate() {
    if ("spring".equals(inlineExpression)) {
        return Arrays.asList("t_order_2024_01", "t_order_2024_02");
    }
    return Arrays.asList("t_order_2024_03", "t_order_2024_04");
}
@Override
public Object getType() {
    return "CUSTOM.FIXTURE";
}
}

```

And add the file META-INF/services/org.apache.shardingsphere.infra.expr.spi.InlineExpressionParser to the project classpath,

```
org.example.CustomInlineExpressionParserFixture
```

At this time, for actualDataNodes in the ShardingSphere configuration file, 1. If configured as <CUSTOM.FIXTURE>spring, it will be converted to t_order_2024_01, t_order_2024_02. 2. If configured as <CUSTOM.FIXTURE>summer, it will be converted to t_order_2024_03, t_order_2024_04.

Procedure

When using attributes that require the use of Row Value Expressions, such as in the data sharding feature, it is sufficient to indicate the Type Name of the specific SPI implementation under the actualDataNodes attribute.

If the Row Value Expressions does not indicate the Type Name of the SPI, the SPI implementation of GROOVY will be used by default.

Sample

```

rules:
- !SHARDING
  tables:
    t_order:
      actualDataNodes: <LITERAL>ds_0.t_order_0, ds_0.t_order_1, ds_1.t_order_0, ds_1.t_order_1
      tableStrategy:
        standard:
          shardingColumn: order_id

```

```

        shardingAlgorithmName: t_order_inline
    keyGenerateStrategy:
        column: order_id
        keyGeneratorName: snowflake
    defaultDatabaseStrategy:
        standard:
            shardingColumn: user_id
            shardingAlgorithmName: database_inline
    shardingAlgorithms:
        database_inline:
            type: INLINE
            props:
                algorithm-expression: <GROOVY>ds_${user_id % 2}
        t_order_inline:
            type: INLINE
            props:
                algorithm-expression: t_order_${order_id % 2}
    keyGenerators:
        snowflake:
            type: SNOWFLAKE

```

Related References

- [Core Concept](#)
- [Data Sharding](#)

9.3.3 SQL Hint

Background

At present, most relational databases basically provide SQL Hint as a supplement to SQL syntax. SQL Hint allows users to intervene in the execution process of SQL through the built-in Hint syntax of the database, to complete some special functions or realize optimization of SQL execution. ShardingSphere also provides SQL Hint syntax, allowing users to perform force route for sharding and read-write splitting, and data source pass through.

Use specification

The SQL Hint syntax of ShardingSphere needs to be written in SQL in the form of comments. The SQL Hint syntax format only supports `/* */` temporarily, and the Hint content needs to start with `SHARDINGSPHERE_HINT:`, and then define the attribute key/value pairs corresponding to different features, separated by commas when there are multiple attributes. The SQL Hint syntax format of ShardingSphere is as follows:

```
/* SHARDINGSPHERE_HINT: {key} = {value}, {key} = {value} */ SELECT * FROM t_order;
```

If you use the MySQL client to connect, you need to add the `-c` option to retain comments, and the client defaults to `--skip-comments` to filter comments.

Parameters

The following attributes can be defined in ShardingSphere SQL Hint. In order to be compatible with the lower version SQL Hint syntax, the attributes defined in the alias can also be used:

Name	Alias	Data Type	Description	Default Value
SHARDING_DATABASE_VALUE (?)	shardingDatabaseValue	Comparable	Database sharding value, used when config Hint sharding strategy	.
SHARDING_TABLE_VALUE (?)	shardingTableValue	Comparable	Table sharding value, used when config Hint sharding strategy	.
WRITE_ROUTE_ONLY (?)	writeRouteOnly	boolean	Route to the write datasource when use readwrite-splitting	false
DATA_SOURCE_NAME (?)	dataSourceName	String	Data source pass through, route SQL directly to the specified data source	.
SKIP_SQL_REWRITE (?)	skipSQL Rewrite	boolean	Skip the SQL rewrite phase	false
SKIP_METADATA_VALIDATE (?)	skipMetadataValidate	boolean	Skip the SQL metadata validate	false
DISABLE_AUDIT_NAMES (?)	disableAuditNames	String	Disable the specified SQL audit algorithm	.
SHADOW (?)	shadow	boolean	Route to the shadow datasource when use shadow	false

SQL Hint

Sharding

The optional attributes of sharding SQL Hint include:

- {table}.SHARDING_DATABASE_VALUE: used to add data source sharding value corresponding to the {table} table, multiple attributes are separated by commas;
- {table}.SHARDING_TABLE_VALUE: used to add table sharding value corresponding to the {table} table, multiple attributes are separated by commas.

In the case of only database sharding, when forcing routing to a certain datasource, you can use the SHARDING_DATABASE_VALUE method to set the sharding value without specifying

{table}.

An example of using the SQL Hint of sharding:

```
/* SHARDINGSPHERE_HINT: t_order.SHARDING_DATABASE_VALUE=1, t_order.SHARDING_TABLE_VALUE=1 */ SELECT * FROM t_order;
```

ReadwriteSplitting

The optional attribute of read-write splitting SQL Hint is WRITE_ROUTE_ONLY, and true means that the current SQL is forced to be routed to write datasource for execution.

An example of using the SQL Hint for read-write splitting:

```
/* SHARDINGSPHERE_HINT: WRITE_ROUTE_ONLY=true */ SELECT * FROM t_order;
```

DataSource Pass Through

The optional attribute of datasource pass through SQL Hint is DATA_SOURCE_NAME, which needs to specify the name of the data source registered in the ShardingSphere logic database.

An example of using the SQL Hint of data source pass through:

```
/* SHARDINGSPHERE_HINT: DATA_SOURCE_NAME=ds_0 */ SELECT * FROM t_order;
```

SKIP SQL REWRITE

The optional attribute of skip SQL rewriting SQL Hint is SKIP_SQL_REWRITE, and true means skipping the current SQL rewriting stage.

An example of skipping SQL rewrite SQL Hint:

```
/* SHARDINGSPHERE_HINT: SKIP_SQL_REWRITE=true */ SELECT * FROM t_order;
```

SKIP METADATA VALIDATE

The optional attribute of skip SQL metadata validate SQL Hint is SKIP_METADATA_VALIDATE, and true means skipping the current SQL metadata validate.

An example of skipping SQL metadata validate SQL Hint:

```
/* SHARDINGSPHERE_HINT: SKIP_METADATA_VALIDATE=true */ SELECT * FROM t_order;
```


DISABLE SQL AUDIT

The optional attribute of disable SQL audit is `DISABLE_AUDIT_NAMES`, you need to specify names of SQL audit algorithm that needs to be disabled, and multiple SQL audit algorithms need to be separated by commas.

An example of disable sql audit SQL Hint:

```
/* SHARDINGSPHERE_HINT: DISABLE_AUDIT_NAMES=sharding_key_required_auditor */ SELECT
* FROM t_order;
```

SHADOW

The optional attribute of the shadow database pressure test SQL Hint is `SHADOW`, and `true` means that the current SQL will be routed to the shadow database data source for execution.

An example of using shadow SQL Hint:

```
/* SHARDINGSPHERE_HINT: SHADOW=true */ SELECT * FROM t_order;
```

9.4 Error Code

This chapter lists error codes of Apache ShardingSphere. They include SQL error codes and server error codes.

All contents of this chapter are draft, the error codes maybe need to adjust.

9.4.1 SQL Error Code

SQL error codes provide by standard SQL State, Vendor Code and Reason, which return to client when SQL execute error.

the error codes are draft, still need to be adjusted.

Kernel Exception

Meta data

Vendor Code	SQL State	Reason
10000	42S02	Database is required.
10001	42S02	Schema '%s' does not exist.
10002	42S02	Table or view '%s' does not exist.
10003	42S02	Unknown column '%s' in '%s'.

continues on next page

Table 1 – continued from previous page

Vendor Code	SQL State	Reason
10004	4 2S02	Index ‘%s’ does not exist.
10005	4 2S01	Index ‘%s’ already exists.
10010	H Y000	Rule and storage meta data mismatched, reason is: %s.
10100	H Y000	Can not %s storage units ‘%s’ .
10012	H Y000	Load table meta data failed for database ‘%s’ and tables ‘%s’ .
10101	4 2S02	There is no storage unit in database ‘%s’ .
10102	4 4000	Storage units ‘%s’ do not exist in database ‘%s’ .
10103	4 4000	Storage unit ‘%s’ still used by ‘%s’ .
10104	4 2S01	Duplicate storage unit names ‘%s’ .
10110	0 8000	Storage units can not connect, error messages are: %s.
10111	0 A000	Can not alter connection info in storage units: ‘%s’ .
10120	4 4000	Invalid storage unit status, error message is: %s.
10200	4 4000	Invalid ‘%s’ rule ‘%s’ , error message is: %s
10201	4 2S02	There is no rule in database ‘%s’ .
10202	4 2S02	%s rules ‘%s’ do not exist in database ‘%s’ .
10203	4 4000	%s rules ‘%s’ in database ‘%s’ are still in used.
10204	4 2S01	Duplicate %s rule names ‘%s’ in database ‘%s’ .
10210	4 2S02	%s strategies ‘%s’ do not exist.
10300	H Y000	Invalid format for actual data node ‘%s’ .
10301	0 A000	Can not support 3-tier structure for actual data node ‘%s’ with JDBC ‘%s’ .
10400	4 4000	Algorithm ‘%s’ initialization failed, reason is: %s.
10401	4 2S02	‘%s’ algorithm on %s is required.
10402	4 2S02	‘%s’ algorithm ‘%s’ on %s is unregistered.
10403	4 4000	%s algorithms ‘%s’ in database ‘%s’ are still in used.
10404	4 4000	Invalid %s algorithm configuration ‘%s’ .
10410	0 A000	Unsupported %s.%s with database type ‘%s’ .
10440	H Y000	Algorithm ‘%s’ execute failed, reason is: %s.
10500	4 4000	Invalid single rule configuration, reason is: %s.
10501	4 2S02	Single table ‘%s’ does not exist.
10502	H Y000	Can not load table with database name ‘%s’ and data source name ‘%s’ , reason is: %s.
10503	0 A000	Can not drop schema ‘%s’ because of contains tables.

Data

Vendor Code	SQL State	Reason
11000	HY004	Unsupported conversion data type ‘%s’ for value ‘%s’ .
11001	HY004	Unsupported conversion stream charset ‘%s’ .

Syntax

Vendor Code	SQL State	Reason
12 000	4 20 00	SQL String can not be NULL or empty.
12 010	4 40 00	Can not support variable ‘%s’ .
12 011	H Y0 04	Invalid variable value ‘%s’ .
12 020	H V0 08	Column index ‘%d’ is out of range.
12 021	4 2S 02	Can not find column label ‘%s’ .
12 022	H Y0 00	Column ‘%s’ in %s is ambiguous.
12 100	4 20 00	You have an error in your SQL syntax: %s
12 101	4 20 00	Can not accept SQL type ‘%s’ .
12 200	4 20 00	Hint data source ‘%s’ does not exist.
12 300	0 A0 00	DROP TABLE ...CASCADE is not supported.
12 500	4 20 00	Not unique table/alias: ‘%s’ .
12 600	H Y0 00	In definition of view, derived table or common table expression, SELECT list and column names list have different column counts.

Connection

Vend orCo de	SQL Sta te	Reason
13000	0800	Can not get %d connections one time, partition succeed connection(%d) have re-leased. Please consider increasing the 'maxPoolSize' of the data sources or decreasing the 'max-connections-size-per-query' in properties.
13001	0800	SQL execution has been interrupted.
13010	0100	Circuit break open, the request has been ignored.
13100	0A00	Unsupported storage type of URL '%s' .
13101	0800	The URL '%s' is not recognized, please refer to the pattern '%s' .
13200	0800	Can not register driver.
13201	0800	Connection has been closed.
13202	0800	Result set has been closed.
13400	H Y 0 000	Load datetime from database failed, reason: %s

Transaction

V endor Code	SQL St ate	Reason
14000	25 000	Switch transaction type failed, please terminate the current transaction.
14001	42 S02	Can not find transaction manager of '%s' .
14002	44 000	Max length of unique resource name '%s' exceeded, should be less than 45.
14003	25 000	Transaction timeout should more than 0.
14004	25 000	Close transaction manager failed.
14200	25 000	Failed to create '%s' XA data source.
14201	25 000	Can not start new XA transaction in a active transaction.
14202	25 000	Check XA transaction privileges failed on data source, please grant '%s' to current user.
14400	44 000	No application id within 'seata.conf' file.
14401	25 000	Seata-AT transaction has been disabled.

Lock

Vendor Code	SQL State	Reason
15030	HY000	Cluster is already locked.
15031	HY000	Cluster is not locked.

Cluster

Vendor Code	SQL State	Reason
17000	44000	Mode must be 'cluster' .
17001	HY000	Worker ID assigned failed, which should be in [0, %s).
17010	HY000	Cluster persist repository error, reason is: %s
17011	HY000	Failed to reload meta data context.
17020	HY000	The cluster status is %s, can not support SQL statement '%s' .
17100	42S02	Cluster persist repository configuration is required.

Data Pipeline

Vendor Code	SQL State	Reason
18000	22023	There is invalid parameter value ‘%s’ .
18100	42S02	Target database ‘%s’ does not exist.
18101	42S02	Can not find pipeline job ‘%s’ .
18102	44000	Sharding count of job ‘%s’ is 0.
18103	42S02	Can not get meta data for table ‘%s’ when split by range.
18104	HY000	Can not split by unique key ‘%s’ for table ‘%s’ .
18105	HY000	Target table ‘%s’ is not empty.
18106	01007	Source data source lacks ‘%s’ privilege(s).
18107	HY000	Source data source required ‘%s = %s’ , now is ‘%s’ .
18108	42S02	User ‘%s’ does exist.
18109	08000	Check privileges failed on source data source.
18110	HY000	Importer job write data failed.
18111	08000	Get binlog position failed by job ‘%s’ .
18112	HY000	Can not find consistency check job of ‘%s’ .
18113	HY000	Uncompleted consistency check job ‘%s’ exists, progress ‘%s’ .
18114	HY000	Failed to get DDL for table ‘%s’ .
18200	HY000	Before data record is ‘%s’ , after data record is ‘%s’ .
18201	08000	Data check table ‘%s’ failed.
18202	0A000	Unsupported pipeline database type ‘%s’ .
18400	42S02	Can not find stream data source table.
18401	42S02	Database ‘%s’ does not exist.
18410	42S02	CDC Login request body is empty.
18411	08004	Illegal username or password.

Feature Exception

Data Sharding

Vendor Code	SQL State	Reason
20000	42S02	%s configuration does not exist in database ‘%s’ .
20001	42S02	Can not find table rule with logic tables ‘%s’ .
20002	42S02	Can not find data source in sharding rule, invalid actual data node ‘%s’ .
20003	42S02	Data nodes is required for sharding table ‘%s’ .
20004	42S02	Actual table ‘%s.%s’ is not in table rule configuration.
20005	42S02	Can not find binding actual table, data source is ‘%s’ , logic table is ‘%s’ , other act
20006	44000	Actual tables ‘%s’ are in use.
20009	42S01	View name has to bind to %s tables.

Table 2 – continued from previous page

Vendor Code	SQL State	Reason
2 00 10	4 40 00	Invalid binding table configuration.
2 00 11	4 40 00	Only allowed 0 or 1 sharding strategy configuration.
2 00 12	4 2S 01	Same actual data node cannot be configured in multiple logic tables in same database, logic table.
2 00 20	4 40 00	Sharding value can not be null in SQL statement.
2 00 21	H Y0 04	Found different types for sharding value ‘%s’ .
2 00 22	H Y0 04	Invalid %s, datetime pattern should be ‘%s’ , value is ‘%s’ .
2 00 23	4 40 00	Sharding value %s subtract stop offset %d can not be less than start offset %d.
2 00 24	4 40 00	%s value ‘%s’ must implements Comparable.
2 00 30	0 A0 00	Can not support operation ‘%s’ with sharding table ‘%s’ .
2 00 31	4 40 00	Can not update sharding value for table ‘%s’ .
2 00 32	0 A0 00	The CREATE VIEW statement contains unsupported query statement.
2 00 33	4 40 00	PREPARE statement can not support sharding tables route to same data sources.
2 00 34	4 40 00	The table inserted and the table selected must be the same or bind tables.
2 00 35	0 A0 00	Can not support DML operation with multiple tables ‘%s’ .
2 00 36	4 20 00	%s ...LIMIT can not support route to multiple data nodes.
2 00 37	4 40 00	Can not find actual data source intersection for logic tables ‘%s’ .
2 00 38	4 20 00	INSERT INTO ...SELECT can not support applying key generator with absent generate keys.
2 00 39	0 A0 00	Alter view rename .. to .. statement should have same config for ‘%s’ and ‘%s’ .
2 00 40	H Y0 00	‘%s %s’ can not route correctly for %s ‘%s’ .
2 00 41	4 2S 02	Can not get route result, please check your sharding rule configuration.
2 00 42	3 40 00	Can not get cursor name from fetch statement.
2 00 50	H Y0 00	Sharding algorithm class ‘%s’ should be implement ‘%s’ .
2 00 51	H Y0 00	Routed target ‘%s’ does not exist, available targets are ‘%s’ .
2 00 52	4 40 00	Inline sharding algorithms expression ‘%s’ and sharding column ‘%s’ do not match.
2 00 53	4 40 00	Complex inline algorithm need %d sharding columns, but only found %d.
2 00 54	4 40 00	No sharding database route info.
2 00 55	4 40 00	Some routed data sources do not belong to configured data sources. routed data sources.
2 00 56	4 40 00	Please check your sharding conditions ‘%s’ to avoid same record in table ‘%s’ routed.
2 00 57	4 40 00	Can not find routing table factor, data source ‘%s’ , actual table ‘%s’ .
2 00 60	H Y0 00	Invalid %s strategy ‘%s’ , strategy does not match data nodes.
2 00 90	4 20 00	Not allow DML operation without sharding conditions.

SQL Federation

Vendor Code	SQL State	Reason
20100	42000	Unsupported SQL node conversion for SQL statement ‘%s’ .
20101	42000	SQL federation does not support SQL ‘%s’ .
20102	42S02	SQL federation schema ‘%s’ not found in SQL ‘%s’ .

Readwrite-splitting

Vendor Code	SQL State	Reason
20200	4 2S02	Readwrite-splitting data source rule name is required in database ‘%s’ .
20201	4 2S02	Can not find readwrite-splitting data source rule ‘%s’ in database ‘%s’ .
20202	4 2S02	Readwrite-splitting [READ/WRITE] data source is required in %s.
20203	4 2S02	Can not find readwrite-splitting [READ/WRITE] data source ‘%s’ in %s.
20204	4 2S01	Readwrite-splitting [READ/WRITE] data source ‘%s’ is duplicated in %s.
20205	4 4000	Readwrite-splitting [READ/WRITE] data source inline expression error in %s.

SQL Dialect Translator

Vendor Code	SQL State	Reason
20400	0A000	Can not support database ‘%s’ in SQL translation.

Traffic Management

Vendor Code	SQL State	Reason
20500	42S02	Can not get traffic execution unit.

Data Encrypt

Vendor Code	SQL State	Reason
2 1000	42 S02	%s column is required in %s.
2 1001	42 S02	Can not find encrypt table ‘%s’ .
2 1002	42 S02	Can not find logic encrypt column by ‘%s’ .
2 1003	42 S02	Can not find encrypt column ‘%s’ from table ‘%s’ .
2 1004	HY 000	‘%s’ column’s encrypt algorithm ‘%s’ should support %s in database ‘%s’ .
2 1005	HY 000	Column ‘%s’ of table ‘%s’ is not configured with %s query algorithm.
2 1010	44 000	Altered column ‘%s’ must use same encrypt algorithm with previous column ‘%s’ in table ‘%s’ .
2 1020	0A 000	The SQL clause ‘%s’ is unsupported in encrypt feature.
2 1030	22 000	Failed to decrypt the ciphertext ‘%s’ in the column ‘%s’ of table ‘%s’ .

Shadow Database

Vendor Code	SQL State	Reason
22000	42S02	Production data source configuration does not exist in database ‘%s’ .
22001	42S02	Shadow data source configuration does not exist in database ‘%s’ .
22002	42S02	No available shadow data sources mappings in shadow table ‘%s’ .
22003	44000	Default shadow algorithm class should be implement HintShadowAlgorithm.
22010	HY004	Shadow column ‘%s’ of table ‘%s’ does not support ‘%s’ type.
22020	42000	Insert value of index ‘%d’ can not support for shadow.

Other Exception

Vendor Code	SQL State	Reason
30000	HY000	Unknown exception: %s
30001	0A000	Unsupported SQL operation: %s
30002	HY000	Database protocol exception: %s
30003	0A000	Unsupported command: %s
30004	HY000	Server exception: %s
30010	HY000	Can not find plugin class ‘%s’ .
30020	HY000	File access failed, file is: %s

9.4.2 Server Error Code

Unique codes provided when server exception occur, which printed by Proxy backend or JDBC startup logs.

Error Code	Reason
SPI-00001	No implementation class load from SPI ‘%s’ with type ‘%s’ .
DATA-SOURCE-00001	Data source ‘%s’ is unavailable.
PROPS-00001	Properties convert failed, details are: %s.
PROXY-00001	Load database server info failed.

Apache ShardingSphere provides dozens of SPI based extensions. it is very convenient to customize the functions for developers.

This chapter lists all SPI extensions of Apache ShardingSphere. If there is no special requirement, users can use the built-in implementation provided by Apache ShardingSphere; advanced users can refer to the interfaces for customized implementation.

Apache ShardingSphere community welcomes developers to feed back their implementations to the [open-source community](#), so that more users can benefit from it.

10.1 Mode

10.1.1 StandalonePersistRepository

Fully-qualified class name

```
`org.apache.shardingsphere.mode.repository.standalone.  
StandalonePersistRepository <https://github.com/apache/shardingsphere/blob/master/mode/type/standalone/repository/api/src/main/java/org/apache/shardingsphere/mode/repository/standalone/StandalonePersistRepository.java>`__
```

Definition

Standalone mode configuration information persistence definition

Implementation classes

<i>Configuration Type</i>	<i>Description*</i>	<i>Fully-qualified class name</i>
JDBC	JDBC-based persistence	<code>org.apache.shardingsphere.mode.repository.standalone.jdbc.JDBCRepository</code> https://github.com/apache/shardingsphere/blob/master/mode/type/standalone/repository/provider/jdbc/src/main/java/org/apache/shardingsphere/mode/repository/standalone/jdbc/JDBCRepository.java

10.1.2 ClusterPersistRepository

Fully-qualified class name

`org.apache.shardingsphere.mode.repository.cluster.ClusterPersistRepository`
<https://github.com/apache/shardingsphere/blob/master/mode/type/cluster/repository/api/src/main/java/org/apache/shardingsphere/mode/repository/cluster/ClusterPersistRepository.java>

Definition

Cluster mode configuration information persistence definition

Implementation classes

ConfigurationType*	Description*	Fully-qualified class name
ZooKeeper	ZooKeeper based persistence	org.apache.shardingsphere.mode.repository.cluster.zookeeper.ZookeeperRepository < https://github.com/apache/shardingsphere/blob/master/mode/type/cluster/repository-provider/zookeeper/src/main/java/org/apache/shardingsphere-mode/repository-cluster/zookeeper/ZookeeperRepository.java >
etcd	Etcd based persistence	org.apache.shardingsphere.mode.repository.cluster.etcd.EtcdRepository < https://github.com/apache/shardingsphere/blob/master/mode/type/provider/etcd/src/main/java/org/apache/shardingsphere-mode/repository-cluster/etcd/EtcdRepository.java >

10.2 SQL Parser

10.2.1 DatabaseTypedSQLParserFacade

Fully-qualified class name

org.apache.shardingsphere.sql.parser.spi.DialectSQLParserFacade <<https://github.com/apache/shardingsphere/blob/master/parser/sql/spi/src/main/java/org/apache/shardingsphere/sql/parser/spi/DialectSQLParserFacade.java>>`__

Definition

Database typed SQL parser facade service definition

Implementation classes

<code>ConfigurationType</code> *	Description	Fully-qualified class name
MySQL	SQL parser entry based on MySQL	<code>org.apache.shardingsphere.sql.parser.mysql.parser.MySQLParserFacade</code> <https://github.com/apache/shardingsphere/blob/master/sql/dialect/mysql/src/main/java/org/apache/shardingsphere/sql/parser/mysql/parser/MySQLParserFacade.java>
PostgreSQL	SQL parser entry based on PostgreSQL	<code>org.apache.shardingsphere.sql.parser.postgresql.parser.PostgreSQLParserFacade</code> <https://github.com/apache/shardingsphere/blob/master/parser/sql/postgresql/src/main/java/org/apache/shardingsphere/sql/parser/postgresql/parser/PostgreSQLParserFacade.java>
openGauss	SQL parser entry based on openGauss	<code>org.apache.shardingsphere.sql.parser.opengauss.parser.OpenGaussParserFacade</code> <https://github.com/apache/shardingsphere/blob/master/parser/sql/opengauss/src/main/java/org/apache/shardingsphere/sql/parser/opengauss/parser/OpenGaussParserFacade.java>
Oracle	SQL parser entry based on Oracle	<code>org.apache.shardingsphere.sql.parser.oracle.parser.OracleParserFacade</code> <https://github.com/apache/shardingsphere/blob/master/sql/dialect/oracle/src/main/java/org/apache/shardingsphere/sql/parser/oracle/parser/OracleParserFacade.java>
SQLServer	SQL parser entry based on SQLServer	<code>org.apache.shardingsphere.sql.parser.sqlserver.parser.SQLServerParserFacade</code> <https://github.com/apache/shardingsphere/blob/master/parser/sqlserver/src/main/java/org/apache/shardingsphere/sqlserver/parser/SQLServerParserFacade.java>

10.2. SQL Parser

10.2.2 SQLStatementVisitorFacade

Fully-qualified class name

`org.apache.shardingsphere.sql.parser.spi.SQLStatementVisitorFacade` <<https://github.com/apache/shardingsphere/blob/master/parser/sql/spi/src/main/java/org/apache/shardingsphere/sql/parser/spi/SQLStatementVisitorFacade.java>>`__

Definition

SQL visitor facade class definition

Implementation classes

ConfigurationType*	Description*	Fully-qualified class name
MySQL	MySQL syntax tree visitor entry	org.apache.shardingsphere.sql.parser.mysql.visitor.statement.MySQLStatementVisitorFacade < https://github.com/apache/shardingsphere/blob/master/parser/sql/parser/src/main/java/org/apache/shardingsphere/sql/parser/visitor/statement/MySQLStatementVisitorFacade >
PostgreSQL	PostgreSQL syntax tree visitor entry	org.apache.shardingsphere.sql.parser.postgresql.visitor.statement.PostgreSQLStatementVisitorFacade < https://github.com/apache/shardingsphere/blob/master/parser/sql/dialect/postgresql/src/main/java/org/apache/shardingsphere/sql/parser/postgresql/visitor/statement/PostgreSQLStatementVisitorFacade >
SQLServer	SQLServer syntax tree visitor entry	org.apache.shardingsphere.sql.parser.sqlserver.visitor.statement.SQLServerStatementVisitorFacade < https://github.com/apache/shardingsphere/blob/master/parser/sql/dialect/sqlserver/src/main/java/org/apache/shardingsphere/sql/parser/sqlserver/visitor/statement/SQLServerStatementVisitorFacade >
Oracle	Oracle syntax tree visitor entry	org.apache.shardingsphere.sql.parser.oracle.visitor.statement.OracleStatementVisitorFacade < https://github.com/apache/shardingsphere/blob/master/parser/sql/dialect/oracle/src/main/java/org/apache/shardingsphere/sql/parser/oracle/visitor/statement/OracleStatementVisitorFacade >
SQL92	SQL92 syntax tree visitor entry	org.apache.shardingsphere.sql.parser.sql92.visitor.statement.SQL92StatementVisitorFacade

10.3 Data Sharding

10.3.1 ShardingAlgorithm

Fully-qualified class name

```
`org.apache.shardingsphere.sharding.spi.ShardingAlgorithm` <https://github.com/apache/shardingsphere/blob/master/features/sharding/api/src/main/java/org/apache/shardingsphere/sharding/spi/ShardingAlgorithm.java>`__
```

Definition

Sharding Algorithm definition

Implementation classes

ConfigurationType*	AutoCreateTables*	Description	Fully-qualified class name
MOD	Y	Modulo sharding algorithm	`org.apache.shardingsphere.sharding.algorithm.sharding.mod.ModShardingAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/sharding/core/src/main/java/org/apache/shardingsphere/sharding/mod/ModShardingAlgorithm.java
HASH_MOD	Y	Hash modulo sharding algorithm	`org.apache.shardingsphere.sharding.algorithm.sharding.mod.HashModShardingAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/sharding/core/src/main/java/org/apache/shardingsphere/sharding/mod/HashModShardingAlgorithm.java
BOUNDARY_RANGE	Y	Boundary based range sharding algorithm	`org.apache.shardingsphere.sharding.algorithm.range.BoundaryBasedRangeShardingAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/sharding/core/src/main/java/org/apache/shardingsphere/sharding/algorithm/range/BoundaryBasedRangeShardingAlgorithm.java
VOLUME_RANGE	Y	Volume based range sharding algorithm	`org.apache.shardingsphere.sharding.algorithm.range.VolumeBasedRangeShardingAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/sharding/core/src/main/java/org/apache/shardingsphere/sharding/algorithm/range/VolumeBasedRangeShardingAlgorithm.java

10.3.2 ShardingAuditAlgorithm

Fully-qualified class name

``org.apache.shardingsphere.sharding.spi.ShardingAuditAlgorithm <https://github.com/apache/shardingsphere/blob/master/features/sharding/api/src/main/java/org/apache/shardingsphere/sharding/spi/ShardingAuditAlgorithm.java>`__`

Definition

Sharding audit algorithm definition

Implementation classes

<code>ConfigurationType*</code>	Description	Fully-qualified class name
<code>DML_SHARDING_CONDITIONS</code>	Prohibit DML auditing algorithm without sharding conditions	<code>`org.apache.shardingsphere.sharding.algorithm.audit.DMLShardingConditionsShardingAuditAlgorithm <https://github.com/apache/shardingsphere/blob/master/features/sharding/core/src/main/java/org/apache/shardingsphere/sharding/algorithm/audit/DMLShardingConditionsShardingAuditAlgorithm.java>`__</code>

10.3.3 DatetimeService

Fully-qualified class name

``org.apache.shardingsphere.timeservice.spi.TimestampService <https://github.com/apache/shardingsphere/blob/master/kernel/time-service/api/src/main/java/org/apache/shardingsphere/timeservice/spi/TimestampService.java>`__`

Definition

Obtain the current date for routing definition

Implementation classes

<i>Configuration</i>	<i>Description</i>	<i>Fully-qualified class name</i>
DatabaseTimestampService	Get the current time from the database for routing	<code>`org.apache.shardingsphere.timeservice.type.database.DatabaseTimestampService <https://github.com/apache/shardingsphere/blob/master/kernel/time-service/type/database/src/main/java/org/apache/shardingsphere/timeservice/type/database/DatabaseTimestampService.java>`</code>
SystemTimestampService	Get the current time from the application system for routing	<code>`org.apache.shardingsphere.timeservice.type.system.SystemTimestampService <https://github.com/apache/shardingsphere/blob/master/kernel/time-service/type/system/src/main/java/org/apache/shardingsphere/timeservice/type/system/SystemTimestampService.java>`</code>

10.3.4 InlineExpressionParser

Fully-qualified class name

`org.apache.shardingsphere.infra.expr.core.InlineExpressionParser`

Definition

Row Value Expressions definition

Implementation classes

Configuration type	Description	Fully-qualified class name
GROUP	Row Value Expressions that uses the Groovy syntax	<code>org.apache.shardingsphere.infra.expr.groovy.GroovyInlineExpressionParser</code>
LITERAL	Row Value Expressions that uses a standard list	<code>org.apache.shardingsphere.infra.expr.literal.LiteralInlineExpressionParser</code>
INTERVAL	Row Value Expressions based on fixed interval that uses the Key-Value syntax	<code>org.apache.shardingsphere.infra.expr.interval.IntervalInlineExpressionParser</code>
ESPRESSO	Row Value Expressions that uses the Groovy syntax based on GraalVM Truffle's Espresso implementation	<code>org.apache.shardingsphere.infra.expr.espresso.EspressoInlineExpressionParser</code>

10.4 Infra algorithm

10.4.1 LoadBalanceAlgorithm

Fully-qualified class name

```
org.apache.shardingsphere.infra.algorithm.loadbalancer.core.
LoadBalanceAlgorithm <https://github.com/apache/shardingsphere/blob/master/infra/algorithm/type/load-balancer/core/src/main/java/org/apache/shardingsphere/infra/algorithm/loadbalancer/core/LoadBalanceAlgorithm.java>`__
```

Definition

Load balance algorithms, they can be used in readwrite-splitting and traffic features.

Implementation classes

ConfigurationType*	Description*	Fully-qualified class name
ROUND_ROBIN	load balancer algorithm based on polling	org.apache.shardingsphere.infra.algorithm.loadbalancer.round robin.RoundRobinLoadBalanceAlgorithm < https://github.com/apache/shardingsphere/blob/master/infra/algorithm/type/load-balancer/type/round-robin/src/main/java/org/apache/shardingsphere/infra/algorithm/loadbalancer/round/robin/RoundRobinLoadBalanceAlgorithm.java
RANDOM	load balancer algorithm based on random	org.apache.shardingsphere.infra.algorithm.loadbalancer.random.RandomLoadBalanceAlgorithm < https://github.com/apache/shardingsphere/blob/master/infra/algorithm/type/load-balancer/type/random/src/main/java/org/apache/shardingsphere/infra/algorithm/loadbalancer/random/RandomLoadBalanceAlgorithm.java
WEIGHT	load balancer algorithm based on weight	org.apache.shardingsphere.infra.algorithm.loadbalancer.weight.WeightLoadBalanceAlgorithm < https://github.com/apache/shardingsphere/blob/master/infra/algorithm/type/load-balancer/type/weight/src/main/java/org/apache/shardingsphere/infra/algorithm/loadbalancer/weight/WeightLoadBalanceAlgorithm.java

10.4.2 KeyGenerateAlgorithm

Fully-qualified class name

``org.apache.shardingsphere.keygen.core.algorithm.KeyGenerateAlgorithm`` <<https://github.com/apache/shardingsphere/blob/master/infra/algorithm/type/key-generator/core/src/main/java/org/apache/shardingsphere/infra/algorithm/keygen/core/KeyGenerateAlgorithm.java>>`__

Definition

Distributed key generated algorithms, they can be used in sharding feature.

Implementation classes

ConfigurationType*	Description*	Fully-qualified class name
SNOWFLAKE	Snowflake key generate algorithm	<code>`org.apache.shardingsphere.keygen.snowflake.algorithm.SnowflakeKeyGenerateAlgorithm`</code> < https://github.com/apache/shardingsphere/blob/master/infra/algorithm/type/key-generator/type/snowflake/src/main/java/org/apache/shardingsphere/infra/algorithm/keygen/snowflake/SnowflakeKeyGenerateAlgorithm.java >
UUID	UUID key generate algorithm	<code>`org.apache.shardingsphere.keygen.uuid.algorithm.UUIDKeyGenerateAlgorithm`</code> < https://github.com/apache/shardingsphere/blob/master/infra/algorithm/type/key-generator/type/uuid/src/main/java/org/apache/shardingsphere/infra/algorithm/keygen/uuid/UUIDKeyGenerateAlgorithm.java >

10.4.3 MessageDigestAlgorithm

Fully-qualified class name

```
`org.apache.shardingsphere.infra.algorithm.messagedigest.core.  
MessageDigestAlgorithm <https://github.com/apache/shardingsphere/blob/master/infra/algorithm/type/message-digest/core/src/main/java/org/apache/shardingsphere/infra/algorithm/messagedigest/core/MessageDigestAlgorithm.java>`__
```

Definition

Message digest algorithms, they can be used in encrypt and mask feature.

Implementation classes

ConfigurationType*	Description*	Fully-qualified class name
MD5	MD5 message digest algorithm	`org.apache.shardingsphere.infra.algorithm.messagedigest.md5.MD5MessageDigestAlgorithm < https://github.com/apache/shardingsphere/blob/master/infra/algorithm/type/message-digest/type/md5/src/main/java/org/apache/shardingsphere/infra/algorithm/messagedigest/md5/MD5MessageDigestAlgorithm.java >`__

10.5 SQL Audit

10.5.1 SQLAuditor

Fully-qualified class name

```
`org.apache.shardingsphere.infra.executor.audit.SQLAuditor <https://github.com/apache/shardingsphere/blob/master/infra/executor/src/main/java/org/apache/shardingsphere/infra/executor/audit/SQLAuditor.java>`__
```

Definition

SQL auditor class definition

Implementation classes

<i>Configuration Type</i>	<i>Description*</i>	<i>Fully-qualified class name</i>
Sharding	Sharding auditor	<code>org.apache.shardingsphere.sharding.auditor.ShardingSQLAuditor</code> < https://github.com/apache/shardingsphere/blob/master/features/sharding/core/src/main/java/org/apache/shardingsphere/sharding/auditor/ShardingSQLAuditor.java >__

10.6 Encryption

10.6.1 EncryptAlgorithm

Fully-qualified class name

`org.apache.shardingsphere.encrypt.spi.EncryptAlgorithm` <<https://github.com/apache/shardingsphere/blob/master/features/encrypt/api/src/main/java/org/apache/shardingsphere/encrypt/spi/EncryptAlgorithm.java>>__

Definition

Data encrypt algorithm definition

Implementation classes

ConfigurationType*	Description*	Fully-qualified class name
AES	AES data encrypt algorithm	org.apache.shardingsphere.encrypt.algorithm.encrypt.AESEncryptAlgorithm <https://github.com/apache/shardingsphere/blob/master/encrypt/core/src/main/java/org/apache/shardingsphere/encrypt/algorithm/standard/AESEncryptAlgorithm.java>
MD5	MD5 assisted query encrypt algorithm	org.apache.shardingsphere.encrypt.algorithm.encrypt.MD5EncryptAlgorithm <https://github.com/apache/shardingsphere/blob/master/features/encrypt/core/src/main/java/org/apache/shardingsphere/encrypt/algorithm/assisted/MD5AssistedEncryptAlgorithm.java>

10.7 Data Masking

10.7.1 MaskAlgorithm

Fully-qualified class name

org.apache.shardingsphere.mask.spi.MaskAlgorithm<https://github.com/apache/shardingsphere/blob/master/features/mask/api/src/main/java/org/apache/shardingsphere/mask/spi/MaskAlgorithm.java>

Definition

Data masking algorithm definition

Implementation classes

Configuration Type	Description	Fully-qualified class name
MD5	Data masking algorithm based on MD5	`org.apache.shardingsphere.mask.algorithm.hash.MD5MaskAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/mask/core/src/main/java/org/apache/shardingsphere/mask/algorithm/hash/MD5MaskAlgorithm.java
KEEP_FIRST_N_LAST_MASKING_ALGORITHM	Keep first n last m data masking algorithm	`org.apache.shardingsphere.mask.algorithm.cover.KeepFirstNLastMMaskAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/mask/core/src/main/java/org/apache/shardingsphere/mask/algorithm/cover/KeepFirstNLastMMaskAlgorithm.java
KEEP_FROM_X_TO_Y_MASKING_ALGORITHM	Keep from x to y data masking algorithm	`org.apache.shardingsphere.mask.algorithm.cover.KeepFromXToYMaskAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/mask/core/src/main/java/org/apache/shardingsphere/mask/algorithm/cover/KeepFromXToYMaskAlgorithm.java
MASK_FIRST_N_LAST_MASKING_ALGORITHM	Mask first n last m data masking algorithm	`org.apache.shardingsphere.mask.algorithm.cover.MaskFirstNLastMMaskAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/mask/core/src/main/java/org/apache/shardingsphere/mask/algorithm/cover/MaskFirstNLastMMaskAlgorithm.java
MASK_FROM_X_TO_Y_MASKING_ALGORITHM	Mask from x to y data masking algorithm	`org.apache.shardingsphere.mask.algorithm.cover.MaskFromXToYMaskAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/mask/core/src/main/java/org/apache/shardingsphere/mask/algorithm/cover/MaskFromXToYMaskAlgorithm.java
MASK_BEFORE_SPECIAL_CHARS_DATA_MASKING_ALGORITHM	Mask before special chars data masking algorithm	`org.apache.shardingsphere.mask.algorithm.cover.MaskBeforeSpecialCharsAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/mask/core/src/main/java/org/apache/shardingsphere/mask/algorithm/cover/MaskBeforeSpecialCharsAlgorithm.java
MASK_AFTER_SPECIAL_CHARS_DATA_MASKING_ALGORITHM	Mask after special chars data masking algorithm	`org.apache.shardingsphere.mask.algorithm.cover.MaskAfterSpecialCharsAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/mask/core/src/main/java/org/apache/shardingsphere/mask/algorithm/cover/MaskAfterSpecialCharsAlgorithm.java
GENERIC_TABLE_RANDOM_REPLACE_ALGORITHM	Generic table random replace algorithm	`org.apache.shardingsphere.mask.algorithm.replace.GenericTableRandomReplaceAlgorithm` https://github.com/apache/shardingsphere/blob/master/features/mask/core/src/main/java/org/apache/shardingsphere/mask/algorithm/replace/GenericTableRandomReplaceAlgorithm.java

10.8 Shadow DB

10.8.1 ShadowAlgorithm

Fully-qualified class name

`org.apache.shardingsphere.shadow.spi.ShadowAlgorithm` <<https://github.com/apache/shardingsphere/blob/master/features/shadow/api/src/main/java/org/apache/shardingsphere/shadow/spi/ShadowAlgorithm.java>>`__

Definition

Shadow algorithm's definition

Implementation classes

<code>.</code> ConfigurationType*	Description	Fully-qualified class name
VALUE_MATCH	Match shadow algorithms based on field values	<code>`org.apache.shardingsphere.shadow.algorithm.shadow.column.ColumnValueMatchedShadowAlgorithm`</code> https://github.com/apache/shardingsphere/blob/master/features/shadow/core/src/main/java/org/apache/shardingsphere/shadow/algorithm/shadow/column/ColumnValueMatchedShadowAlgorithm.java
REGEX_MATCH	Regular matching shadow algorithm based on field value	<code>`org.apache.shardingsphere.shadow.algorithm.shadow.column.ColumnRegexMatchedShadowAlgorithm`</code> https://github.com/apache/shardingsphere/blob/master/features/shadow/core/src/main/java/org/apache/shardingsphere/shadow/algorithm/shadow/column/ColumnRegexMatchedShadowAlgorithm.java
SQL_HINT	Shadow algorithm on sql hint	<code>`org.apache.shardingsphere.shadow.algorithm.shadow.hint.SQLHintShadowAlgorithm`</code> https://github.com/apache/shardingsphere/blob/master/features/shadow/core/src/main/java/org/apache/shardingsphere/shadow/algorithm/shadow/hint/SQLHintShadowAlgorithm.java

10.9 Observability

10.9.1 PluginLifecycleService

Fully-qualified class name

``org.apache.shardingsphere.agent.spi.PluginLifecycleService`` <https://github.com/apache/shardingsphere/blob/master/agent/api/src/main/java/org/apache/shardingsphere/agent/spi>

/PluginLifecycleService.java>`__

Definition

Plug lifecycle management interface

Implementation classes

ConfigurationType*	Description*	Fully-qualified class name
File	Logging plug lifecycle management class	`org.apache.shardingsphere.agent.plugin.logging.file.FileLoggingPluginLifecycleService < https://github.com/apache/shardingsphere/blob/master/agent/plugins/logging/type/file/src/main/java/org/apache/shardingsphere/agent/plugin/logging/file/FileLoggingPluginLifecycleService.java >
Prometheus	Prometheus plug lifecycle management class	`org.apache.shardingsphere.agent.plugin.metrics.prometheus.PrometheusPluginLifecycleService < https://github.com/apache/shardingsphere/blob/master/agent/plugins/metrics/type/prometheus/src/main/java/org/apache/shardingsphere/agent/plugin/metrics/prometheus/PrometheusPluginLifecycleService >
OpenTelemetry	OpenTelemetryTracing plug lifecycle management class	`org.apache.shardingsphere.agent.plugin.tracing.opentelemetry.OpenTelemetryTracingPluginLifecycleService < https://github.com/apache/shardingsphere/blob/master/agent/plugins/tracing/type/opentelemetry/src/main/java/org/apache/shardingsphere/agent/plugin/tracing/opentelemetry/OpenTelemetryTracingPluginLifecycleService >

Apache ShardingSphere provides test engines for integration, module and performance.

11.1 Integration Test

Provide point to point test which connect real ShardingSphere and database instances.

They define SQLs in XML files, engine run for each database independently. All test engines designed to modify the configuration files to execute all assertions without any **Java code** modification. It does not depend on any third-party environment, ShardingSphere-Proxy and database used for testing are provided by docker image.

11.2 Module Test

Provide module test engine for complex modules.

They define SQLs in XML files, engine run for each database independently too It includes SQL parser and SQL rewriter modules.

11.3 Performance Test

Provide multiple performance test methods, includes Sysbench, JMH or TPCC and so on.

11.4 Sysbench Test

11.5 Integration Test

11.5.1 Design

The integration testing consists of three modules: test case, test environment and test engine.

Test case

It is used to define the SQL to be tested and the assertion data of the test results.

Each case defines one SQL, which can define multiple database execution types.

Test environment

It is used to set up the database and ShardingSphere-Proxy environment for running test cases. The environment is classified into environment preparation mode, database type, and scenario.

Environment preparation mode is divided into Native and Docker, and Embed type will be supported in the future. - Native environment is used for test cases to run directly in the test environment provided by the developer, suitable for debugging scenarios; - Docker environment is directly built when Maven runs the Docker-Compose plug-in. It is suitable for cloud compilation environment and testing ShardingSphere-Proxy, such as GitHub Action; - Embed environment is built when the test framework automatically builds embedded MySQL. It is suitable for the local environment test of ShardingSphere-JDBC.

Currently, the Native environment is adopted by default, and ShardingSphere-JDBC + H2 database is used to run test cases. Maven's `-pit`. `Env.docker` parameter specifies how the Docker environment is run. In the future, ShardingSphere-JDBC + MySQL of the Embed environment will be adopted to replace the default environment type used when Native executes test cases.

Database types currently support MySQL, PostgreSQL, SQLServer, and Oracle, and test cases can be executed using ShardingSphere-JDBC or ShardingSphere-Proxy.

Scenarios are used to test the supporting rules of ShardingSphere. Currently, data sharding and read/write splitting and other related scenarios are supported, and the combination of scenarios will be improved continuously in the future.

Test engine

It is used to read test cases in batches and execute and assert test results line by line.

The test engine arranges test cases and environments to test as many scenarios as possible with the fewest test cases.

Each SQL generates a test report in the combination of database type * access port type * SQL execution mode * JDBC execution mode * Scenario. Currently, each dimension is supported as follows:

- Database types: H2, MySQL, PostgreSQL, SQLServer, and Oracle;
- Access port types: ShardingSphere-JDBC and ShardingSphere-Proxy;
- SQL execution modes: Statement and PreparedStatement;
- JDBC execution modes: execute and executeQuery/executeUpdate;
- Scenarios: database shards, table shards, read/write splitting and sharding + read/write splitting

Therefore, one SQL will drive Database type (5) * Access port type (2) * SQL execution mode (2) * JDBC execution mode (2) * Scenario (4) = 160 test cases to be run to achieve the pursuit of high quality.

11.5.2 User Guide

Module path: test/e2e/sql

Test case configuration

SQL test case is in resources/cases/\${SQL-TYPE}/\${SQL-TYPE}-integration-test-cases.xml.

The case file format is as follows:

```
<integration-test-cases>
  <test-case sql="${SQL}">
    <assertion parameters="${value_1}:${type_1}, ${value_2}:${type_2}" expected-
data-file="${dataset_file_1}.xml" />
    <!-- ... more assertions -->
    <assertion parameters="${value_3}:${type_3}, ${value_4}:${type_4}" expected-
data-file="${dataset_file_2}.xml" />
  </test-case>

  <!-- ... more test cases -->
</integration-test-cases>
```

The lookup rule of expected-data-file is as follows: 1. Find the file dataset\\${SCENARIO_NAME}\\${DATABASE_TYPE}\\${dataset_file}.xml in the same level directory; 2. Find the file dataset\\${SCENARIO_NAME}\\${dataset_file}.xml in the same level directory;

3. Find the file `dataset\${dataset_file}.xml` in the same level directory; 4. Report an error if none of them are found.

The assertion file format is as follows:

```
<dataset>
  <metadata>
    <column name="column_1" />
    <!-- ... more columns -->
    <column name="column_n" />
  </metadata>
  <row values="value_01, value_02" />
  <!-- ... more rows -->
  <row values="value_n1, value_n2" />
</dataset>
```

e2e operation module is E2E test, does not contains the assertion for `</dataset>` tag

Environment configuration

`${SCENARIO-TYPE}` Refers to the scenario name used to identify a unique scenario during the test engine run. `${DATABASE-TYPE}` refers to the database types.

Native environment configuration

Directory: `src/test/resources/env/${SCENARIO-TYPE}`

- `scenario-env.properties`: data source configuration;
- `rules.yaml`: rule configuration;
- `databases.xml`: name of the real database;
- `dataset.xml`: initialize the data;
- `init-sql\${DATABASE-TYPE}\init.sql`: initialize the database and table structure;
- `authority.xml`: to be supplemented.

Docker environment configuration

Directory: `src/test/resources/docker/${SCENARIO-TYPE}`

- `docker-compose.yaml`: Docker-Compose config files, used for Docker environment startup;
- `proxy/conf/database-${SCENARIO-TYPE}.yaml`: rule configuration。

The Docker environment configuration provides a remote debugging port for ShardingSphere-Proxy. You can find the second exposed port for remote debugging in ``shardingsphere-proxy`` of the ``docker-comemage.yaml`` file.

Run the test engine

Configure the running environment of the test engine

Control the test engine by configuring `src/test/resources/env/engine-env.properties`.

All attribute values can be dynamically injected via Maven command line `-D`.

```
# Scenario type. Multiple values can be separated by commas. Optional values: db,
tbl, dbtbl_with_replica_query, replica_query
it.scenarios=db,tbl,dbtbl_with_replica_query,replica_query

# Whether to run additional test cases
it.run.additional.cases=false

# Configure the environment type. Only one value is supported. Optional value:
docker or null. The default value: null.
it.cluster.env.type=${it.env}
# Access port types to be tested. Multiple values can be separated by commas.
Optional value: jdbc, proxy. The default value: jdbc
it.cluster.adapters=jdbc

# Scenario type. Multiple values can be separated by commas. Optional value: H2,
MySQL, Oracle, SQLServer, PostgreSQL
it.cluster.databases=H2,MySQL,Oracle,SQLServer,PostgreSQL
```

Run debugging mode

- Standard test engine Run `org.apache.shardingsphere.test.integration.engine. ${SQL-TYPE}.General${SQL-TYPE}E2EIT` to start the test engines of different SQL types.
- Batch test engine Run `org.apache.shardingsphere.test.integration.engine.dml. BatchDMLE2EIT` to start the batch test engine for the test `addBatch()` provided for DML statements.
- Additional test engine Run `org.apache.shardingsphere.test.integration.engine. ${SQL-TYPE}.Additional${SQL-TYPE}E2EIT` to start the test engine with more JDBC method calls. Additional test engines need to be enabled by setting `it.run.additional.cases=true`.

Run Docker mode

```
./mvnw -B clean install -f test/e2e/pom.xml -Pit.env.docker -Dit.cluster.
adapters=proxy,jdbc -Dit.scenarios=${scenario_name_1,scenario_name_2,scenario_name_
n} -Dit.cluster.databases=MySQL
```

Run the above command to build a Docker mirror `apache/shardingsphere-proxy-test:latest` used for integration testing. If you only modify the test code, you can reuse the existing test mirror without rebuilding it. Skip the mirror building and run the integration testing directly with the following command:

```
./mvnw -B clean install -f test/e2e/sql/pom.xml -Pit.env.docker -Dit.cluster.
adapters=proxy,jdbc -Dit.scenarios=${scenario_name_1,scenario_name_2,scenario_name_
n} -Dit.cluster.databases=MySQL
```

Remote debug Proxy code in Docker container

First of all, you need to modify the configuration file `it-env.properties`, set `function.it.env.type` to `docker`, and then set the corresponding database image version like `transaction.it.docker.mysql.version=mysql:5.7`. Then generate the test image through the command, for example:

```
# for operation, replace ${operation} with transaction, pipeline or showprocesslist
./mvnw -B clean install -am -pl test/e2e/operation/${operation} -Pit.env.docker -
DskipTests

# for e2e sql
./mvnw -B clean install -am -pl test/e2e/sql -Pit.env.docker -DskipTests -Dspotless.
apply.skip=true
```

Remote debug Proxy started by docker image

E2E Test Proxy image opens the 3308 port by default for remote debugging of the instance in the container. Use the following method to connect and debug the Proxy code in the container with IDE tools such as IDEA:

IDEA -> Run -> Edit Configurations -> Add New Configuration -> Remote JVM Debug

Edit the corresponding information: - Name: A descriptive name, such as `e2e-debug`. - Host: A IP that can access docker, such as `127.0.0.1` - Port: debugging port(will set in next step). - use module classpath: The root directory of the project `shardingsphere`.

After editing the above information, run `Run -> Run -> e2e-debug` in IDEA to start the remote debug of IDEA.

Remote debug Proxy started by Testcontainer

Note: If the Proxy container is started by Testcontainer, because the 3308 port is not exposed before Testcontainer starts, it cannot be debugged by the Remote debug Proxy started by docker image method. Debug Testcontainer started Proxy container by the following method: - Set a breakpoint in the relevant startup class of Testcontainer, for example, after the line `containerComposer.start()`; in `E2EContainerComposer` in the suite test, at this time, the relevant containers must have been started. - Access breakpoint debugging mode through shortcut key `Alt + F8`, and get mapped port by `docker ps` for the 3308 mapping of the Proxy object under the `containerComposer` (the external mapping port of Testcontainer is random). - See the Remote debug Proxy started by docker image method, set the Name, Host, Port, and use the port got in previous step, e.g. 51837.

After editing the above information, run `Run -> Run -> e2e-debug -> debug` in IDEA to start the remote debug of IDEA.

Notice

1. To test Oracle, add an Oracle driver dependency to `pom.xml`.
2. In order to ensure the integrity and legibility of the test data, 10 database shards and 10 table shards are used in the sharding of the integration testing, which takes a long time to run the test cases completely.

11.6 Performance Test

Provides result for each performance test tools.

11.6.1 SysBench ShardingSphere-Proxy Empty Rule Performance Test

Objectives

Compare the performance of ShardingSphere-Proxy and MySQL 1. Sysbench directly carries out stress testing on the performance of MySQL. 1. Sysbench directly carries out stress testing on ShardingSphere-Proxy (directly connect MySQL).

Based on the above two groups of experiments, we can figure out the loss of MySQL when using ShardingSphere-Proxy.

Set up the test environment

Server information

1. Db-related configuration: it is recommended that the memory is larger than the amount of data to be tested, so that the data is stored in the memory hot block, and the rest can be adjusted.
2. ShardingSphere-Proxy-related configuration: it is recommended to use a high-performance, multi-core CPU, and other configurations can be customized.
3. Disable swap partitions on all servers involved in the stress testing.

Database

```
[mysqld]
innodb_buffer_pool_size=${MORE_THAN_DATA_SIZE}
innodb_log_file_size=3000000000
innodb_log_files_in_group=5
innodb_flush_log_at_trx_commit=0
innodb_change_buffer_max_size=40
back_log=900
innodb_max_dirty_pages_pct=75
innodb_open_files=20480
innodb_buffer_pool_instances=8
innodb_page_cleaners=8
innodb_purge_threads=2
innodb_read_io_threads=8
innodb_write_io_threads=8
table_open_cache=102400
log_timestamps=system
thread_cache_size=16384
transaction_isolation=READ-COMMITTED

# Appropriate tuning can be considered to magnify the underlying DB performance, so
that the experiment doesn't subject to DB performance bottleneck.
```


Stress testing tool

Refer to [sysbench](#)'s GitHub

ShardingSphere-Proxy

bin/start.sh

```
-Xmx16g -Xms16g -Xmn8g # Adjust JVM parameters
```

config.yaml

```

databaseName: sharding_db

dataSources:
  ds_0:
    url: jdbc:mysql://***.***.***.***:****/test?serverTimezone=UTC&useSSL=false #
    Parameters can be adjusted appropriately
    username: test
    password:
    connectionTimeoutMilliseconds: 30000
    idleTimeoutMilliseconds: 60000
    maxLifetimeMilliseconds: 1800000
    maxPoolSize: 200 # The maximum ConnPool is set to ${the number of concurrencies
in stress testing}, which is consistent with the number of concurrencies in stress
testing to shield the impact of additional connections in the process of stress
testing.
    minPoolSize: 200 # The minimum ConnPool is set to ${the number of concurrencies
in stress testing}, which is consistent with the number of concurrencies in stress
testing to shield the impact of connections initialization in the process of stress
testing.

rules: []

```

Test phase

Environment setup

```

sysbench oltp_read_write --mysql-host=${DB_IP} --mysql-port=${DB_PORT} --mysql-user=
${USER} --mysql-password=${PASSWD} --mysql-db=test --tables=10 --table-size=1000000
--report-interval=10 --time=100 --threads=200 cleanup
sysbench oltp_read_write --mysql-host=${DB_IP} --mysql-port=${DB_PORT} --mysql-user=
${USER} --mysql-password=${PASSWD} --mysql-db=test --tables=10 --table-size=1000000
--report-interval=10 --time=100 --threads=200 prepare

```

Stress testing command

```
sysbench oltp_read_write --mysql-host=${DB/PROXY_IP} --mysql-port=${DB/PROXY_PORT} --mysql-user=${USER} --mysql-password=${PASSWD} --mysql-db=test --tables=10 --table-size=1000000 --report-interval=10 --time=100 --threads=200 run
```

Stress testing report analysis

```
sysbench 1.0.20 (using bundled LuaJIT 2.1.0-beta2)
Running the test with following options:
Number of threads: 200
Report intermediate results every 10 second(s)
Initializing random number generator from current time
Initializing worker threads...
Threads started!
# Report test results every 10 seconds, and the number of tps, reads per second,
writes per second, and the total response time of more than 95th percentile.
[ 10s ] thds: 200 tps: 11161.70 qps: 223453.06 (r/w/o: 156451.76/44658.51/22342.80)
lat (ms,95%): 27.17 err/s: 0.00 reconn/s: 0.00
...
[ 120s ] thds: 200 tps: 11731.00 qps: 234638.36 (r/w/o: 164251.67/46924.69/23462.00)
lat (ms,95%): 24.38 err/s: 0.00 reconn/s: 0.00
SQL statistics:
  queries performed:
    read:                19560590                # number of reads
    write:               5588740                 # number of writes
    other:               27943700                 # number of
other operations (COMMIT etc.)
    total:               27943700                 # the total number
  transactions:         1397185 (11638.59 per sec.) # number of
transactions (per second)
  queries:              27943700 (232771.76 per sec.) # number of
statements executed (per second)
  ignored errors:        0      (0.00 per sec.)      # number of
ignored errors (per second)
  reconnects:            0      (0.00 per sec.)      # number of
reconnections (per second)

General statistics:
  total time:            120.0463s                # total time
  total number of events: 1397185                 # total number
of transactions

Latency (ms):
  min:                   5.37                     # minimum latency
  avg:                   17.13                     # average latency
  max:                   109.75                    # maximum latency
```

```

          95th percentile:                24.83                # average
response time of over 95th percentile.
          sum:                            23999546.19

Threads fairness:
  events (avg/stddev):                   6985.9250/34.74        # On average,
6985.9250 events were completed per thread, and the standard deviation is 34.74
  execution time (avg/stddev):           119.9977/0.01          # The average
time of each thread is 119.9977 seconds, and the standard deviation is 0.01

```

Noticeable features

1. CPU utilization ratio of the server where ShardingSphere-Proxy resides. It is better to make full use of CPU.
2. I/O of the server disk where the DB resides. The lower the physical read value is, the better.
3. Network IO of the server involved in the stress testing.

11.6.2 BenchmarkSQL ShardingSphere-Proxy Sharding Performance Test

Objective

BenchmarkSQL tool is used to test the sharding performance of ShardingSphere-Proxy.

Method

ShardingSphere-Proxy supports the TPC-C test through [BenchmarkSQL 5.0](#). In addition to the content described in this document, BenchmarkSQL is operated according to the original document `HOW-TO-RUN.txt`.

Fine tuning to test tools

Unlike stand-alone database stress testing, distributed database solutions inevitably face trade-offs in functions. It is recommended to make the following adjustments when using BenchmarkSQL to carry out stress testing on ShardingSphere-Proxy.

Remove the foreign key and extraHistID

Modify `run/runDatabaseBuild.sh` in the `BenchmarkSQL` directory at line 17.

Before modification:

```
AFTER_LOAD="indexCreates foreignKeys extraHistID buildFinish"
```

After modification:

```
AFTER_LOAD="indexCreates buildFinish"
```

Stress testing environment or parameter recommendations

Note: None of the parameters mentioned in this section are absolute values and need to be adjusted based on actual test results.

It is recommended to run ShardingSphere using Java 17

ShardingSphere can be compiled using Java 8.

When using Java 17, maximize the ShardingSphere performance by default.

ShardingSphere data sharding recommendations

The data sharding of BenchmarkSQL can use the warehouse id in each table as the sharding key.

One of the tables `bmsql_item` has no warehouse id and has a fixed data volume of 100,000 rows: - You can take `i_id` as a sharding key. However, the same Proxy connection may hold connections to multiple different data sources at the same time. - Or you can give up sharding and store it in a single data source. But a data source may be under great pressure. - Or you may choose range-based sharding for `i_id`, such as 1-50000 for data source 0 and 50001-100000 for data source 1.

BenchmarkSQL has the following SQL involving multiple tables:

```
SELECT c_discount, c_last, c_credit, w_tax
FROM bmsql_customer
      JOIN bmsql_warehouse ON (w_id = c_w_id)
WHERE c_w_id = ? AND c_d_id = ? AND c_id = ?
```

```
SELECT o_id, o_entry_d, o_carrier_id
FROM bmsql_oorder
WHERE o_w_id = ? AND o_d_id = ? AND o_c_id = ?
      AND o_id = (
        SELECT max(o_id)
        FROM bmsql_oorder
        WHERE o_w_id = ? AND o_d_id = ? AND o_c_id = ?
      )
```

If the warehouse id is used as the sharding key, the tables involved in the above SQL can be configured as bindingTable:

```
rules:
  - !SHARDING
    bindingTables:
      - bmsql_warehouse, bmsql_customer
      - bmsql_stock, bmsql_district, bmsql_order_line
```

For the data sharding configuration with warehouse id as the sharding key, refer to the appendix of this document.

PostgreSQL JDBC URL parameter recommendations

Adjust the JDBC URL in the configuration file used by BenchmarkSQL, that is, the value of the parameter name conn: - Adding the parameter defaultRowFetchSize=50 may reduce the number of fetch for multi-row result sets. You need to increase or decrease the number according to actual test results. - Adding the parameter reWriteBatchedInserts=true may reduce the time spent on bulk inserts, such as preparing data or bulk inserts for the New Order business. Whether to enable the operation depends on actual test results.

props.pg file excerpt. It is suggested to change the parameter value of conn in line 3.

```
db=postgres
driver=org.postgresql.Driver
conn=jdbc:postgresql://localhost:5432/postgres?defaultRowFetchSize=50&
reWriteBatchedInserts=true
user=benchmarksql
password=PWbmsql
```

ShardingSphere-Proxy global.yaml parameter recommendations

The default value of proxy-backend-query-fetch-size is -1. Changing it to about 50 can minimize the number of fetch for multi-row result sets.

The default value of proxy-frontend-executor-size is CPU * 2 and can be reduced to about CPU * 0.5 based on actual test results. If NUMA is involved, set this parameter to the number of physical cores per CPU based on actual test results.

global.yaml file excerpt:

```
props:
  proxy-backend-query-fetch-size: 50
  # proxy-frontend-executor-size: 32 # 4*32C aarch64
  # proxy-frontend-executor-size: 12 # 2*12C24T x86
```

Appendix

BenchmarkSQL data sharding reference configuration

Adjust pool size according to the actual stress testing process.

```
databaseName: bmsql_sharding
datasources:
  ds_0:
    url: jdbc:postgresql://db0.ip:5432/bmsql
    username: postgres
    password: postgres
    connectionTimeoutMilliseconds: 3000
    idleTimeoutMilliseconds: 60000
    maxLifetimeMilliseconds: 1800000
    maxPoolSize: 1000
    minPoolSize: 1000
  ds_1:
    url: jdbc:postgresql://db1.ip:5432/bmsql
    username: postgres
    password: postgres
    connectionTimeoutMilliseconds: 3000
    idleTimeoutMilliseconds: 60000
    maxLifetimeMilliseconds: 1800000
    maxPoolSize: 1000
    minPoolSize: 1000
  ds_2:
    url: jdbc:postgresql://db2.ip:5432/bmsql
    username: postgres
    password: postgres
    connectionTimeoutMilliseconds: 3000
    idleTimeoutMilliseconds: 60000
    maxLifetimeMilliseconds: 1800000
    maxPoolSize: 1000
    minPoolSize: 1000
  ds_3:
    url: jdbc:postgresql://db3.ip:5432/bmsql
    username: postgres
    password: postgres
    connectionTimeoutMilliseconds: 3000
    idleTimeoutMilliseconds: 60000
    maxLifetimeMilliseconds: 1800000
    maxPoolSize: 1000
    minPoolSize: 1000
rules:
  - !SHARDING
    bindingTables:
      - bmsql_warehouse, bmsql_customer
```

```

    - bmsql_stock, bmsql_district, bmsql_order_line
defaultDatabaseStrategy:
  none:
defaultTableStrategy:
  none:
keyGenerators:
  snowflake:
    type: SNOWFLAKE
tables:
  bmsql_config:
    actualDataNodes: ds_0.bmsql_config

  bmsql_warehouse:
    actualDataNodes: ds_${0..3}.bmsql_warehouse
    databaseStrategy:
      standard:
        shardingColumn: w_id
        shardingAlgorithmName: mod_4

  bmsql_district:
    actualDataNodes: ds_${0..3}.bmsql_district
    databaseStrategy:
      standard:
        shardingColumn: d_w_id
        shardingAlgorithmName: mod_4

  bmsql_customer:
    actualDataNodes: ds_${0..3}.bmsql_customer
    databaseStrategy:
      standard:
        shardingColumn: c_w_id
        shardingAlgorithmName: mod_4

  bmsql_item:
    actualDataNodes: ds_${0..3}.bmsql_item
    databaseStrategy:
      standard:
        shardingColumn: i_id
        shardingAlgorithmName: mod_4

  bmsql_history:
    actualDataNodes: ds_${0..3}.bmsql_history
    databaseStrategy:
      standard:
        shardingColumn: h_w_id
        shardingAlgorithmName: mod_4

  bmsql_oorder:

```

```

actualDataNodes: ds_${0..3}.bmsql_oorder
databaseStrategy:
  standard:
    shardingColumn: o_w_id
    shardingAlgorithmName: mod_4

bmsql_stock:
actualDataNodes: ds_${0..3}.bmsql_stock
databaseStrategy:
  standard:
    shardingColumn: s_w_id
    shardingAlgorithmName: mod_4

bmsql_new_order:
actualDataNodes: ds_${0..3}.bmsql_new_order
databaseStrategy:
  standard:
    shardingColumn: no_w_id
    shardingAlgorithmName: mod_4

bmsql_order_line:
actualDataNodes: ds_${0..3}.bmsql_order_line
databaseStrategy:
  standard:
    shardingColumn: ol_w_id
    shardingAlgorithmName: mod_4

shardingAlgorithms:
  mod_4:
    type: MOD
    props:
      sharding-count: 4

```

BenchmarkSQL 5.0 PostgreSQL statement list

Create tables

```

create table bmsql_config (
  cfg_name    varchar(30) primary key,
  cfg_value   varchar(50)
);

create table bmsql_warehouse (
  w_id        integer not null,
  w_ytd       decimal(12,2),
  w_tax       decimal(4,4),
  w_name      varchar(10),

```



```
w_street_1  varchar(20),
w_street_2  varchar(20),
w_city      varchar(20),
w_state     char(2),
w_zip       char(9)
);

create table bmsql_district (
  d_w_id      integer      not null,
  d_id        integer      not null,
  d_ytd       decimal(12,2),
  d_tax       decimal(4,4),
  d_next_o_id integer,
  d_name      varchar(10),
  d_street_1  varchar(20),
  d_street_2  varchar(20),
  d_city      varchar(20),
  d_state     char(2),
  d_zip       char(9)
);

create table bmsql_customer (
  c_w_id      integer      not null,
  c_d_id      integer      not null,
  c_id        integer      not null,
  c_discount  decimal(4,4),
  c_credit    char(2),
  c_last      varchar(16),
  c_first     varchar(16),
  c_credit_lim decimal(12,2),
  c_balance   decimal(12,2),
  c_ytd_payment decimal(12,2),
  c_payment_cnt integer,
  c_delivery_cnt integer,
  c_street_1  varchar(20),
  c_street_2  varchar(20),
  c_city      varchar(20),
  c_state     char(2),
  c_zip       char(9),
  c_phone     char(16),
  c_since     timestamp,
  c_middle    char(2),
  c_data      varchar(500)
);

create sequence bmsql_hist_id_seq;

create table bmsql_history (
```

```
hist_id integer,
h_c_id integer,
h_c_d_id integer,
h_c_w_id integer,
h_d_id integer,
h_w_id integer,
h_date timestamp,
h_amount decimal(6,2),
h_data varchar(24)
);

create table bmsql_new_order (
  no_w_id integer not null,
  no_d_id integer not null,
  no_o_id integer not null
);

create table bmsql_oorder (
  o_w_id integer not null,
  o_d_id integer not null,
  o_id integer not null,
  o_c_id integer,
  o_carrier_id integer,
  o_ol_cnt integer,
  o_all_local integer,
  o_entry_d timestamp
);

create table bmsql_order_line (
  ol_w_id integer not null,
  ol_d_id integer not null,
  ol_o_id integer not null,
  ol_number integer not null,
  ol_i_id integer not null,
  ol_delivery_d timestamp,
  ol_amount decimal(6,2),
  ol_supply_w_id integer,
  ol_quantity integer,
  ol_dist_info char(24)
);

create table bmsql_item (
  i_id integer not null,
  i_name varchar(24),
  i_price decimal(5,2),
  i_data varchar(50),
  i_im_id integer
);
```

```
create table bmsql_stock (  
  s_w_id      integer      not null,  
  s_i_id      integer      not null,  
  s_quantity  integer,  
  s_ytd       integer,  
  s_order_cnt integer,  
  s_remote_cnt integer,  
  s_data      varchar(50),  
  s_dist_01   char(24),  
  s_dist_02   char(24),  
  s_dist_03   char(24),  
  s_dist_04   char(24),  
  s_dist_05   char(24),  
  s_dist_06   char(24),  
  s_dist_07   char(24),  
  s_dist_08   char(24),  
  s_dist_09   char(24),  
  s_dist_10   char(24)  
);
```

Create indexes

```
alter table bmsql_warehouse add constraint bmsql_warehouse_pkey  
  primary key (w_id);  
  
alter table bmsql_district add constraint bmsql_district_pkey  
  primary key (d_w_id, d_id);  
  
alter table bmsql_customer add constraint bmsql_customer_pkey  
  primary key (c_w_id, c_d_id, c_id);  
  
create index bmsql_customer_idx1  
  on bmsql_customer (c_w_id, c_d_id, c_last, c_first);  
  
alter table bmsql_oorder add constraint bmsql_oorder_pkey  
  primary key (o_w_id, o_d_id, o_id);  
  
create unique index bmsql_oorder_idx1  
  on bmsql_oorder (o_w_id, o_d_id, o_carrier_id, o_id);  
  
alter table bmsql_new_order add constraint bmsql_new_order_pkey  
  primary key (no_w_id, no_d_id, no_o_id);  
  
alter table bmsql_order_line add constraint bmsql_order_line_pkey  
  primary key (ol_w_id, ol_d_id, ol_o_id, ol_number);
```

```
alter table bmsql_stock add constraint bmsql_stock_pkey
primary key (s_w_id, s_i_id);

alter table bmsql_item add constraint bmsql_item_pkey
primary key (i_id);
```

New Order business

stmtNewOrderSelectWhseCust

```
UPDATE bmsql_district
SET d_next_o_id = d_next_o_id + 1
WHERE d_w_id = ? AND d_id = ?
```

stmtNewOrderSelectDist

```
SELECT d_tax, d_next_o_id
FROM bmsql_district
WHERE d_w_id = ? AND d_id = ?
FOR UPDATE
```

stmtNewOrderUpdateDist

```
UPDATE bmsql_district
SET d_next_o_id = d_next_o_id + 1
WHERE d_w_id = ? AND d_id = ?
```

stmtNewOrderInsertOrder

```
INSERT INTO bmsql_oorder (
o_id, o_d_id, o_w_id, o_c_id, o_entry_d,
o_ol_cnt, o_all_local)
VALUES (?, ?, ?, ?, ?, ?, ?)
```

stmtNewOrderInsertNewOrder

```
INSERT INTO bmsql_new_order (
no_o_id, no_d_id, no_w_id)
VALUES (?, ?, ?)
```

stmtNewOrderSelectStock

```
SELECT s_quantity, s_data,
s_dist_01, s_dist_02, s_dist_03, s_dist_04,
s_dist_05, s_dist_06, s_dist_07, s_dist_08,
s_dist_09, s_dist_10
FROM bmsql_stock
```

```
WHERE s_w_id = ? AND s_i_id = ?
FOR UPDATE
```

stmtNewOrderSelectItem

```
SELECT i_price, i_name, i_data
FROM bmsql_item
WHERE i_id = ?
```

stmtNewOrderUpdateStock

```
UPDATE bmsql_stock
SET s_quantity = ?, s_ytd = s_ytd + ?,
    s_order_cnt = s_order_cnt + 1,
    s_remote_cnt = s_remote_cnt + ?
WHERE s_w_id = ? AND s_i_id = ?
```

stmtNewOrderInsertOrderLine

```
INSERT INTO bmsql_order_line (
    ol_o_id, ol_d_id, ol_w_id, ol_number,
    ol_i_id, ol_supply_w_id, ol_quantity,
    ol_amount, ol_dist_info)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

Payment business

stmtPaymentSelectWarehouse

```
SELECT w_name, w_street_1, w_street_2, w_city,
       w_state, w_zip
FROM bmsql_warehouse
WHERE w_id = ?
```

stmtPaymentSelectDistrict

```
SELECT d_name, d_street_1, d_street_2, d_city,
       d_state, d_zip
FROM bmsql_district
WHERE d_w_id = ? AND d_id = ?
```

stmtPaymentSelectCustomerListByLast

```
SELECT c_id
FROM bmsql_customer
WHERE c_w_id = ? AND c_d_id = ? AND c_last = ?
ORDER BY c_first
```

stmtPaymentSelectCustomer

```
SELECT c_first, c_middle, c_last, c_street_1, c_street_2,
       c_city, c_state, c_zip, c_phone, c_since, c_credit,
       c_credit_lim, c_discount, c_balance
FROM bmsql_customer
WHERE c_w_id = ? AND c_d_id = ? AND c_id = ?
FOR UPDATE
```

stmtPaymentSelectCustomerData

```
SELECT c_data
FROM bmsql_customer
WHERE c_w_id = ? AND c_d_id = ? AND c_id = ?
```

stmtPaymentUpdateWarehouse

```
UPDATE bmsql_warehouse
SET w_ytd = w_ytd + ?
WHERE w_id = ?
```

stmtPaymentUpdateDistrict

```
UPDATE bmsql_district
SET d_ytd = d_ytd + ?
WHERE d_w_id = ? AND d_id = ?
```

stmtPaymentUpdateCustomer

```
UPDATE bmsql_customer
SET c_balance = c_balance - ?,
    c_ytd_payment = c_ytd_payment + ?,
    c_payment_cnt = c_payment_cnt + 1
WHERE c_w_id = ? AND c_d_id = ? AND c_id = ?
```

stmtPaymentUpdateCustomerWithData

```
UPDATE bmsql_customer
SET c_balance = c_balance - ?,
    c_ytd_payment = c_ytd_payment + ?,
    c_payment_cnt = c_payment_cnt + 1,
    c_data = ?
WHERE c_w_id = ? AND c_d_id = ? AND c_id = ?
```

stmtPaymentInsertHistory

```
INSERT INTO bmsql_history (
    h_c_id, h_c_d_id, h_c_w_id, h_d_id, h_w_id,
    h_date, h_amount, h_data)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)
```

Order Status business

stmtOrderStatusSelectCustomerListByLast

```
SELECT c_id
  FROM bmsql_customer
 WHERE c_w_id = ? AND c_d_id = ? AND c_last = ?
 ORDER BY c_first
```

stmtOrderStatusSelectCustomer

```
SELECT c_first, c_middle, c_last, c_balance
  FROM bmsql_customer
 WHERE c_w_id = ? AND c_d_id = ? AND c_id = ?
```

stmtOrderStatusSelectLastOrder

```
SELECT o_id, o_entry_d, o_carrier_id
  FROM bmsql_oorder
 WHERE o_w_id = ? AND o_d_id = ? AND o_c_id = ?
    AND o_id = (
        SELECT max(o_id)
          FROM bmsql_oorder
         WHERE o_w_id = ? AND o_d_id = ? AND o_c_id = ?
    )
```

stmtOrderStatusSelectOrderLine

```
SELECT ol_i_id, ol_supply_w_id, ol_quantity,
       ol_amount, ol_delivery_d
  FROM bmsql_order_line
 WHERE ol_w_id = ? AND ol_d_id = ? AND ol_o_id = ?
 ORDER BY ol_w_id, ol_d_id, ol_o_id, ol_number
```

Stock level business

stmtStockLevelSelectLow

```
SELECT count(*) AS low_stock FROM (
  SELECT s_w_id, s_i_id, s_quantity
    FROM bmsql_stock
   WHERE s_w_id = ? AND s_quantity < ? AND s_i_id IN (
        SELECT ol_i_id
          FROM bmsql_district
         JOIN bmsql_order_line ON ol_w_id = d_w_id
            AND ol_d_id = d_id
            AND ol_o_id >= d_next_o_id - 20
            AND ol_o_id < d_next_o_id
    )
)
```

```

        WHERE d_w_id = ? AND d_id = ?
    )
) AS L

```

Delivery BG business

stmtDeliveryBGSelectOldestNewOrder

```

SELECT no_o_id
FROM bmsql_new_order
WHERE no_w_id = ? AND no_d_id = ?
ORDER BY no_o_id ASC

```

stmtDeliveryBGDeleteOldestNewOrder

```

DELETE FROM bmsql_new_order
WHERE no_w_id = ? AND no_d_id = ? AND no_o_id = ?

```

stmtDeliveryBGSelectOrder

```

SELECT o_c_id
FROM bmsql_oorder
WHERE o_w_id = ? AND o_d_id = ? AND o_id = ?

```

stmtDeliveryBGUpdateOrder

```

UPDATE bmsql_oorder
SET o_carrier_id = ?
WHERE o_w_id = ? AND o_d_id = ? AND o_id = ?

```

stmtDeliveryBGSelectSumOLAmount

```

SELECT sum(ol_amount) AS sum_ol_amount
FROM bmsql_order_line
WHERE ol_w_id = ? AND ol_d_id = ? AND ol_o_id = ?

```

stmtDeliveryBGUpdateOrderLine

```

UPDATE bmsql_order_line
SET ol_delivery_d = ?
WHERE ol_w_id = ? AND ol_d_id = ? AND ol_o_id = ?

```

stmtDeliveryBGUpdateCustomer

```

UPDATE bmsql_customer
SET c_balance = c_balance + ?,
    c_delivery_cnt = c_delivery_cnt + 1
WHERE c_w_id = ? AND c_d_id = ? AND c_id = ?

```


11.7 Module Test

Provides test engine with each complex modules.

11.7.1 SQL Parser Test

Prepare Data

Not like Integration test, SQL parse test does not need a specific database environment, just define the sql to parse, and the assert data:

SQL Data

As mentioned `sql-case-id` in Integration test, `test-case-id` could be shared in different module to test, and the file is in `shardingsphere-test-it-parser` module, at `test/it/parser/src/main/resources/sql/supported/${SQL-TYPE}/*.xml`

Assert Data

The assert data is at `test/it/parser/src/main/resources/case/${SQL-TYPE}/*.xml` in that xml file, it could assert against the table name, token or sql condition and so on. For example:

```
<parser-result-sets>
  <parser-result sql-case-id="insert_with_multiple_values">
    <tables>
      <table name="t_order" />
    </tables>
    <tokens>
      <table-token start-index="12" table-name="t_order" length="7" />
    </tokens>
    <sharding-conditions>
      <and-condition>
        <condition column-name="order_id" table-name="t_order" operator=
"EQUAL">
          <value literal="1" type="int" />
        </condition>
        <condition column-name="user_id" table-name="t_order" operator=
"EQUAL">
          <value literal="1" type="int" />
        </condition>
      </and-condition>
      <and-condition>
        <condition column-name="order_id" table-name="t_order" operator=
"EQUAL">
          <value literal="2" type="int" />
        </condition>
      </and-condition>
    </sharding-conditions>
  </parser-result>
</parser-result-sets>
```

```

        </condition>
        <condition column-name="user_id" table-name="t_order" operator=
"EQUAL">
            <value literal="2" type="int" />
        </condition>
    </and-condition>
</sharding-conditions>
</parser-result>
</parser-result-sets>

```

When these configs are ready, launch the test engine in `test/it/parser` to test SQL parse.

11.7.2 SQL Rewrite Test

Target

Facing logic databases and tables cannot be executed directly in actual databases. SQL rewrite is used to rewrite logic SQL into rightly executable ones in actual databases, including two parts, correctness rewrite and optimization rewrite. rewrite tests are for these targets.

Test

The rewrite tests are in the test folder under `test/it/rewriter`. Followings are the main part for rewrite tests:

- test engine
- environment configuration
- assert data

Test engine is the entrance of rewrite tests, just like other test engines, through Junit [Parameterized](#), read every and each data in the xml file under the target test type in `test/resources`, and then assert by the engine one by one

Environment configuration is the yaml file under test type under `test/resources/yaml`. The configuration file contains `dataSources`, `shardingRule`, `encryptRule` and other info. for example:

```

dataSources:
  db: !!com.zaxxer.hikari.HikariDataSource
    driverClassName: org.h2.Driver
    jdbcUrl: jdbc:h2:mem:db;DB_CLOSE_DELAY=-1;DATABASE_TO_UPPER=false;MODE=MYSQL
    username: sa
    password:

## sharding Rules
rules:
- !SHARDING
  tables:

```

```

t_account:
  actualDataNodes: db.t_account_${0..1}
  tableStrategy:
    standard:
      shardingColumn: account_id
      shardingAlgorithmName: account_table_inline
  keyGenerateStrategy:
    column: account_id
    keyGeneratorName: snowflake
t_account_detail:
  actualDataNodes: db.t_account_detail_${0..1}
  tableStrategy:
    standard:
      shardingColumn: order_id
      shardingAlgorithmName: account_detail_table_inline
bindingTables:
  - t_account, t_account_detail
shardingAlgorithms:
  account_table_inline:
    type: INLINE
    props:
      algorithm-expression: t_account_${account_id % 2}
  account_detail_table_inline:
    type: INLINE
    props:
      algorithm-expression: t_account_detail_${account_id % 2}
keyGenerators:
  snowflake:
    type: SNOWFLAKE

```

Assert data are in the xml under test type in test\resources. In the xml file, yaml-rule means the environment configuration file path, input contains the target SQL and parameters, output contains the expected SQL and parameters. The db-type described the type for SQL parse, default is SQL92. For example:

```

<rewrite-assertions yaml-rule="yaml/sharding/sharding-rule.yaml">
  <!-- to change SQL parse type, change db-type -->
  <rewrite-assertion id="create_index_for_mysql" db-type="MySQL">
    <input sql="CREATE INDEX index_name ON t_account ('status')" />
    <output sql="CREATE INDEX index_name ON t_account_0 ('status')" />
    <output sql="CREATE INDEX index_name ON t_account_1 ('status')" />
  </rewrite-assertion>
</rewrite-assertions>

```

After set up the assert data and environment configuration, rewrite test engine will assert the corresponding SQL without any Java code modification.

11.8 Pipeline E2E Test

11.8.1 Objectives

Verify the functional correctness of pipeline scenarios.

11.8.2 Test environment type

Currently, NATIVE and DOCKER are available. 1. NATIVE : Run on developer local machine. Need to start ShardingSphere-Proxy instance and database instance by developer. It could be used for local debugging. 2. DOCKER : Run on docker started by Maven plugin. It could be used for GitHub Actions, and it could be used for local debugging too.

Supported databases: MySQL, PostgreSQL and openGauss.

11.8.3 User guide

Module path: test/e2e/operation/pipeline.

Environment setup

`${DOCKER-IMAGE}` refers to the name of a docker mirror, such as `mysql:5.7`. `${DATABASE-TYPE}` refers to database types.

Directory: `src/test/resources/env/` - `it-env.properties`: Environment setup configuration file. - `${DATABASE-TYPE}/global.yaml`: ShardingSphere-Proxy configuration file. - `${DATABASE-TYPE}/initdb.sql`: Database initialization SQL file. - `${DATABASE-TYPE}/*.cnf`, `*.conf`: Database configuration files. - `common/*.xml`: DistSQL files. - `scenario/`: SQL files for different scenarios.

Test case

Test case example: `MySQLMigrationGeneralE2EIT`. Functions included: - Database-level migration (all tables). - Table-level migration (any number). - Verify migration data consistency. - Support restart during data migration. - Support integer primary keys during data migration. - Support string primary keys during data migration. - A non-administrator account can be used to migrate data.

Running the test case

Any property of `it-env.properties` could be defined by Maven command line parameter `-D`, and its priority is higher than configuration file.

NATIVE environment setup

1. Start ShardingSphere-Proxy (port should be 3307): refer to [proxy startup guide](#), or run `org.apache.shardingsphere.proxy.Bootstrap` in IDE after modifying `proxy/bootstrap/src/main/resources/conf/global.yaml`.

Refer to following files for proxy `global.yaml` configuration:

- `test/e2e/operation/pipeline/src/test/resources/env/mysql/server-8.yaml`
- `test/e2e/operation/pipeline/src/test/resources/env/postgresql/global.yaml`
- `test/e2e/operation/pipeline/src/test/resources/env/opengauss/global.yaml`

2. Start registry center (e.g. ZooKeeper) and database.
3. Take MySQL as an example, `it-env.properties` could be configured as follows:

```
pipeline.it.env.type=NATIVE
pipeline.it.native.database=mysql
pipeline.it.native.mysql.username=root
pipeline.it.native.mysql.password=root
pipeline.it.native.mysql.port=3306
```

4. Find test class and start it on IDE.

DOCKER environment setup

Refer to `.github/workflows/e2e-pipeline.yml` for more details.

1. Build docker image.

```
./mvnw -B clean install -am -pl test/e2e/operation/pipeline -Pit.env.docker -DskipTests
```

Running the above command will build a docker image `apache/shardingsphere-proxy-test:latest`.

The docker image has port 3308 for remote debugging.

If only test code is modified, you could reuse existing docker image.

2. Configure `it-env.properties`.

```
pipeline.it.env.type=DOCKER
pipeline.it.docker.mysql.version=mysql:5.7
```

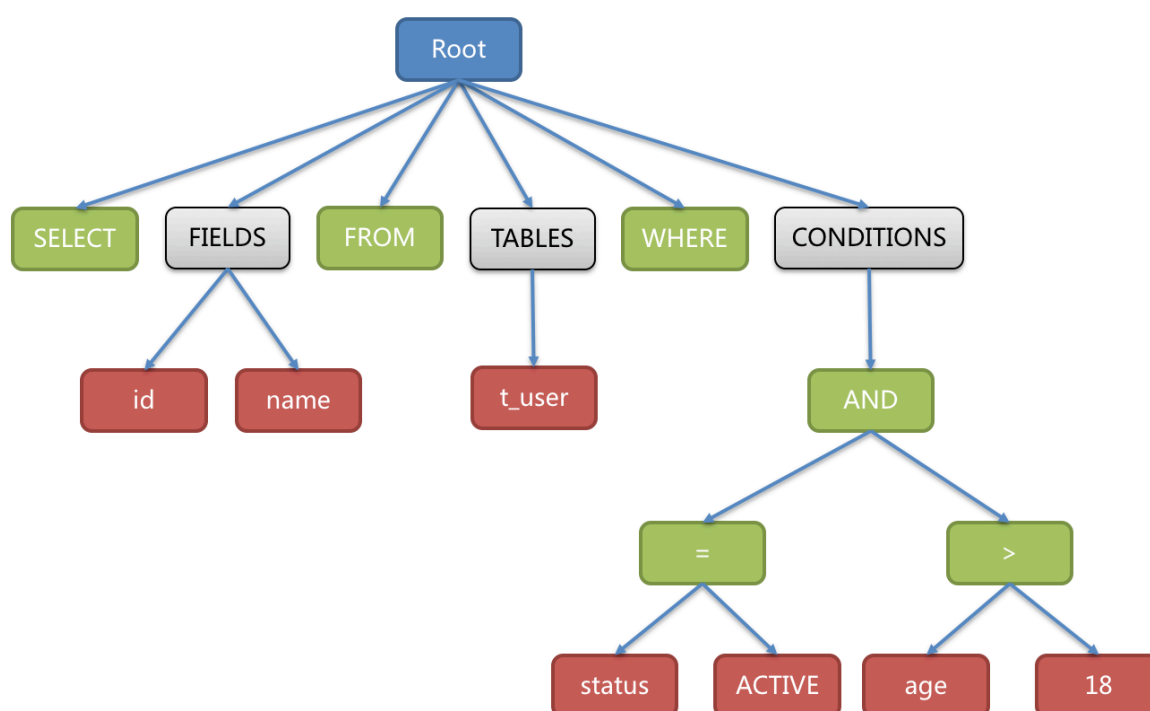
3. Run test cases.

Take MySQL as an example:

```
./mvnw -nsu -B install -f test/e2e/operation/pipeline/pom.xml -Dpipeline.it.env.  
type=docker -Dpipeline.it.docker.mysql.version=mysql:5.7
```

This chapter contains a section of technical implementation with Apache ShardingSphere, which provide the reference with users and developers.

12.1 Database Compatibility



- SQL compatibility

SQL is the standard language for users to communicate with databases. The SQL parsing engine is responsible for parsing SQL strings into abstract syntax trees so that Apache ShardingSphere can understand and implement its incremental function. ShardingSphere currently supports MySQL, PostgreSQL, SQLServer, Oracle, openGauss, ClickHouse, Doris, Hive, Presto and SQL dialects conforming to the SQL92 standard. Due to the complexity of SQL syntax, a few SQL are not supported for now.

- Database protocol compatibility

Apache ShardingSphere currently implements MySQL and PostgreSQL protocols according to different data protocols.

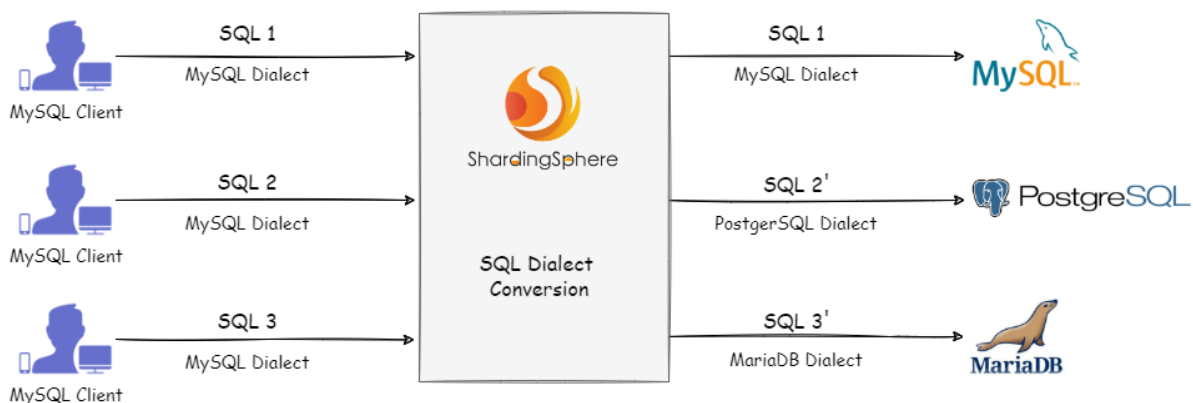
- Supported features

Apache ShardingSphere provides distributed collaboration capabilities for databases. At the same time, it abstracts some database features to the upper layer for unified management, so as to facilitate users.

Therefore, native SQL will not deliver the features provided uniformly to the database, and a message will be displayed indicating that the operation is not supported. Users can replace it with methods provided by ShardingSphere.

12.2 Database Gateway

Apache ShardingSphere provides the ability for SQL dialect translation to achieve automatic conversion between database dialects. For example, users can use MySQL client to connect ShardingSphere and send SQL based on MySQL dialect. ShardingSphere can automatically identify user protocol and storage node type, automatically complete SQL dialect conversion, and access heterogeneous storage nodes such as PostgreSQL.



12.3 Management

12.3.1 Data Structure in Registry Center

Under a defined namespace, rules, props and metadata nodes persist in YAML. Modifying nodes can dynamically refresh configurations. nodes persist the runtime node of the database access object, to distinguish different database access instances. statistics persist data records in system tables.

```

namespace
  |—rules                                     # Global rule configuration
  |   |—transaction
  |   |   |—active_version
  |   |   |—versions
  
```



```

|         |         |         |         |         tables
|         |         |         |         |         |-- ${tableName}           # logic table name
|         |         |         |         |         |         |-- active_verison # Active version
|         |         |         |         |         |         |-- versions        # version list
|         |         |         |         |         |         |         --0
|         |         |         |         |         |         ...
|-- nodes
|   |-- compute_nodes
|     |-- online
|       |-- proxy
|         |-- UUID                # Proxy instance identifier
|         |-- ....
|       |-- jdbc
|         |-- UUID                # JDBC instance identifier
|         |-- ....
|     |-- status
|       |-- UUID
|       |-- ....
|     |-- worker_id
|       |-- UUID
|       |-- ....
|     |-- show_process_list_trigger
|       |-- process_id:UUID
|       |-- ....
|     |-- labels
|       |-- UUID
|       |-- ....
|   |-- qualified_data_sources
|     |-- ${databaseName.groupName.dataSourceName}
|     |-- ${databaseName.groupName.dataSourceName}
|-- statistics
|   |-- shardingsphere
|     |-- schemas
|       |-- shardingsphere
|         |-- tables              # system tables
|         |-- sharding_table_statistics # sharding statistics
table
|   |   |   |   |   |   |   |   |---8a2dc b0d97c3d86ef77b3d4651a1d7d0 # md5
|   |   |   |   |   |   |   |---cluster_information    # cluster information table
```

/rules

These are the global rule configurations, transaction configuration.

```
transaction:
  defaultType: XA
  providerType: Atomikos
```

/props

These are the properties' configurations. Please refer to the [Configuration Manual](#) for more details.

```
kernel-executor-size: 20
sql-show: true
```

/metadata/\${databaseName}/data_sources/units/ds_0/versions/0

Database connection pools, whose properties (e.g. HikariCP) are to be configured by the user.

```
ds_0:
  initializationFailTimeout: 1
  validationTimeout: 5000
  maxLifetime: 1800000
  leakDetectionThreshold: 0
  minimumIdle: 1
  password: root
  idleTimeout: 60000
  jdbcUrl: jdbc:mysql://127.0.0.1:3306/ds_0?serverTimezone=UTC&useSSL=false
  dataSourceClassName: com.zaxxer.hikari.HikariDataSource
  maximumPoolSize: 50
  connectionTimeout: 30000
  username: root
  poolName: HikariPool-1
```

/metadata/\${databaseName}/data_sources/nodes/ds_0/versions/0

Database connection pools, whose properties (e.g. HikariCP) are to be configured by the user.

```
ds_0:
  initializationFailTimeout: 1
  validationTimeout: 5000
  maxLifetime: 1800000
  leakDetectionThreshold: 0
  minimumIdle: 1
  password: root
  idleTimeout: 60000
```

```
jdbcUrl: jdbc:mysql://127.0.0.1:3306/ds_0?serverTimezone=UTC&useSSL=false
dataSourceClassName: com.zaxxer.hikari.HikariDataSource
maximumPoolSize: 50
connectionTimeout: 30000
username: root
poolName: HikariPool-1
```

/metadata/\${databaseName}/rules/sharding/tables/t_order/versions/0

Sharding configuration.

```
actualDataNodes: ds_${0..1}.t_order_${0..1}
auditStrategy:
  allowHintDisable: true
  auditorNames:
    - t_order_dml_sharding_conditions_0
databaseStrategy:
  standard:
    shardingAlgorithmName: t_order_database_inline
    shardingColumn: user_id
keyGenerateStrategy:
  column: another_id
  keyGeneratorName: t_order_snowflake
logicTable: t_order
tableStrategy:
  standard:
    shardingAlgorithmName: t_order_table_inline
    shardingColumn: order_id
```

/metadata/databaseName/schemas/{schemaName}/tables/t_order/versions/0

Use separate node storage for each table.

```
name: t_order # Table name
columns: # Columns
  id: # Column name
    caseSensitive: false
    dataType: 0
    generated: false
    name: id
    primaryKey: true
  order_id:
    caseSensitive: false
    dataType: 0
    generated: false
    name: order_id
```

```
primaryKey: false
indexes:                                     # Index
  t_user_order_id_index:                     # Index name
    name: t_user_order_id_index
```

/nodes/compute_nodes

It includes running instance information of database access object, with sub-nodes as the identifiers of the currently running instance, which is automatically generated at each startup using UUID.

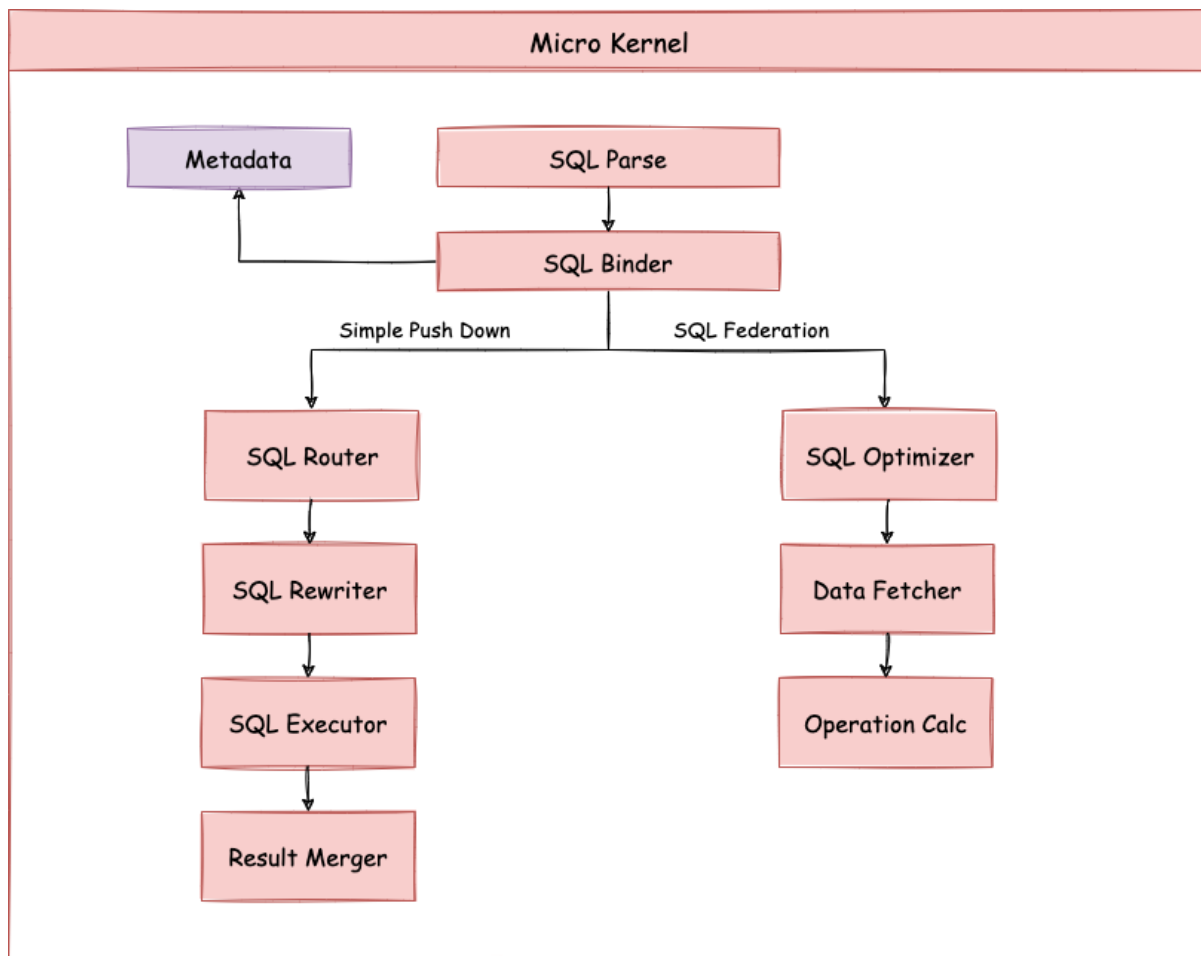
The identifiers are temporary nodes, which are registered when instances are online and cleared when instances are offline. The registry center monitors the change of those nodes to govern the database access of running instances and other things.

/nodes/qualified_data_sources

It can orchestrate a replica database on readwrite-splitting feature, disable data dynamically.

12.4 Sharding

The figure below shows how sharding works. According to whether query and optimization are needed, it can be divided into the Simple Push Down process and SQL Federation execution engine process. Simple Push Down process consists of SQL parser => SQL binder => SQL router => SQL rewriter => SQL executor => result merger, mainly used to deal with SQL execution in standard sharding scenarios. SQL Federation execution engine consists of SQL parser => SQL binder => logical optimization => physical optimization => data fetcher => operator calculation. This process performs logical optimization and physical optimization internally, during which the standard kernel procedure is adopted to route, rewrite, execute and merge the optimized logical SQL.



12.4.1 SQL Parser

It is divided into the lexical parser and syntactic parser. SQL is first split into indivisible words through a lexical parser.

The syntactic parser is then used to analyze SQL and ultimately extract the parsing context, which can include tables, options, ordering items, grouping items, aggregation functions, pagination information, query conditions, and placeholders that may be modified.

12.4.2 SQL Route

The sharding strategy configured by the user is matched according to the parsing context and the routing path is generated. Currently, sharding router and broadcast router are supported.

12.4.3 SQL Rewrite

Rewrite SQL into statements that can be executed correctly in a real database. SQL rewriting is divided into rewriting for correctness and rewriting for optimization.

12.4.4 SQL Execution

It executes asynchronously through a multithreaded executor.

12.4.5 Result Merger

It merges multiple execution result sets to achieve output through the unified JDBC interface. The result merger includes the stream merger, memory merger and appended merger using decorator mode.

12.4.6 Query Optimization

Supported by the experimental Federation Execution Engine, it optimizes complex queries such as associated queries and sub-queries and supports distributed queries across multiple database instances. It internally optimizes query plans using relational algebra to query results through optimal plans.

12.4.7 Parse Engine

SQL is relatively simple compared with other programming languages, but it's still a complete programming language. Therefore, there's no essential difference between parsing SQL syntax and parsing other languages (such as Java, C and Go, etc.).

Abstract Syntax Tree

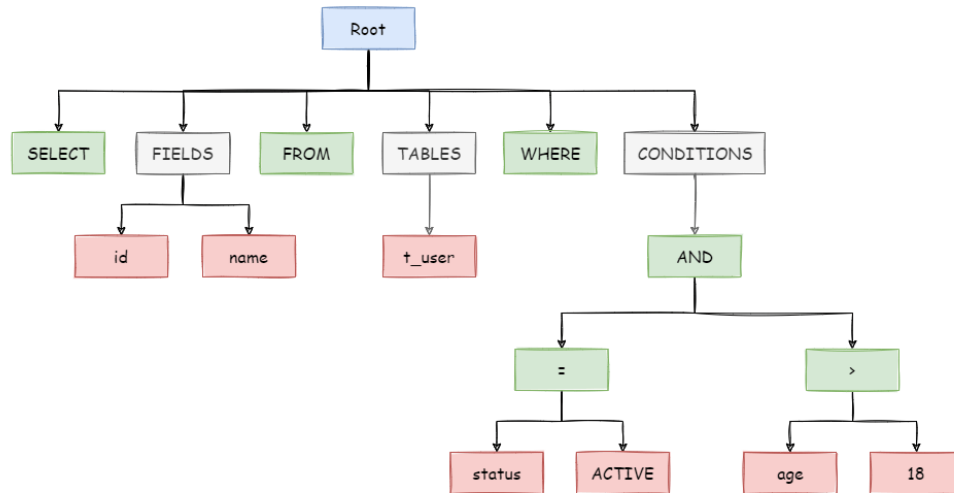
The parsing process is divided into lexical parsing and syntactic parsing. The lexical parser is used to split SQL into indivisible atomic symbols called Tokens.

Tokens are classified into keywords, expressions, literals, and operators based on the dictionaries provided by different database dialects. The syntactic parser is then used to convert the output of the lexical parser into an abstract syntax tree.

For example:

```
SELECT id, name FROM t_user WHERE status = 'ACTIVE' AND age > 18
```

After the above SQL is parsed, its AST (Abstract Syntax Tree) is as follows:



The tokens for keywords in the AST are green, while the tokens for variables are red, and gray ones indicate that further splitting is required.

Finally, the domain model is traversed through the abstract syntax tree by visitor; the context required for sharding is extracted through the domain model (SQLStatement); and then, mark locations that may need rewriting.

The parsing context for sharding includes select items, table, sharding condition, auto-increment primary key, and Order By, Group By, and pagination information (Limit, Rownum, Top). The SQL parsing process is irreversible.

Each Token is parsed in the original SQL order, providing high performance. Taking the similarities and differences of SQL dialects of various databases into consideration, the SQL dialect dictionary of various databases is provided in the parsing module.

SQL Parser Engine

Iteration

SQL parsing is the core of sharding solutions, and its performance and compatibility are the most important indicators. ShardingSphere's SQL parser has undergone three iterations and upgrades.

To achieve high performance and fast implementation, the first generation of SQL parsers used Druid prior to V1.4.x. In practical tests, its performance far exceeds that of other parsers.

The second generation of SQL parsers started from V1.5.x. ShardingSphere uses a completely self-developed SQL parsing engine. Owing to different purposes, ShardingSphere does not need to convert SQL into a complete abstract syntax tree, nor does it require a second traversal through the accessor pattern. It uses a half-parsing method to extract only the context required by data sharding, thus further improving the performance and compatibility of SQL parsing.

The third generation of SQL parsers, starting with V3.0.x, attempts to use ANTLR as a generator of SQL parsing engines and uses Visit to obtain SQL statements from the AST. Since V5.0.x, the architecture of the parsing engine has been restructured and adjusted. Moreover, the AST obtained from the first parsing is stored in the cache so that the parsing results of the same SQL can be directly obtained next time to improve parsing efficiency. Therefore, it is recommended that you use PreparedStatement, a SQL-precompiled method, to improve performance.

Features

- Independent SQL parsing engine
- The syntax rules can be easily expanded and modified (using ANTLR)
- Support multiple dialects

Database	Status
MySQL	perfect supported
PostgreSQL	perfect supported
SQLServer	supported
Oracle	supported
SQL92	supported
openGauss	supported
ClickHouse	supported
Doris	supported
Hive	supported
Presto	supported

API Usage

- Introducing Maven dependency

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-parser-sql-engine</artifactId>
  <version>${project.version}</version>
</dependency>
<!-- According to the needs, introduce the parsing module of the specified dialect
(take MySQL as an example), you can add all the supported dialects, or just what you
need -->
<dependency>
```

```

<groupId>org.apache.shardingsphere</groupId>
<artifactId>shardingsphere-parser-sql-mysql</artifactId>
<version>${project.version}</version>
</dependency>

```

- Obtain AST

```

CacheOption cacheOption = new CacheOption(128, 1024L);
SQLParserEngine parserEngine = new SQLParserEngine("MySQL", cacheOption);
ParseASTNode parseASTNode = parserEngine.parse(sql, useCache);

```

- Obtain SQLStatement

```

CacheOption cacheOption = new CacheOption(128, 1024L);
SQLParserEngine parserEngine = new SQLParserEngine("MySQL", cacheOption);
ParseASTNode parseASTNode = parserEngine.parse(sql, useCache);
SQLVisitorEngine sqlVisitorEngine = new SQLVisitorEngine(sql, "STATEMENT", useCache,
new Properties());
SQLStatement sqlStatement = sqlVisitorEngine.visit(parseASTNode);

```

- SQL Formatting

```

new SQLFormatEngine(TypedSPILoader.getService(DatabaseType.class, "Mysql"),
cacheOption)
    .format(sql, false, null);

```

Example:

Original SQL	Formatted SQL
select a+1 as b, name n from table1 join table2 where id=1 and name= 'lu' ;	SELECT a + 1 AS b, name n FROM table1 JOIN table2 WHERE id = 1 and name = 'lu' ;
select id, name, age, sex, ss, yy from table1 where id=1;	SELECT id , name , age , sex , ss , yy FROM table1 WHERE id = 1;
select id, name, age, count(*) as n, (select id, name, age, sex from table2 where id=2) as sid, yyyy from table1 where id=1;	SELECT id , name , age , COUNT(*) AS n, (SELECT id , name , age , sex FROM table2 WHERE id = 2) AS sid, yyyy FROM table1 WHERE id = 1;
select id, name, age, sex, ss, yy from table1 where id=1 and name=1 and a=1 and b=2 and c=4 and d=3;	SELECT id , name , age , sex , ss , yy FROM table1 WHERE id = 1 and name = 1 and a = 1 and b = 2 and c = 4 and d = 3;
ALTER TABLE t_order ADD column4 DATE, ADD column5 DATETIME, engine ss max_rows 10,min_rows 2, ADD column6 TIMESTAMP, ADD column7 TIME;	ALTER TABLE t_order ADD column4 DATE, ADD column5 DATETIME, ENGINE ss MAX_ROWS 10, MIN_ROWS 2, ADD column6 TIMESTAMP, ADD column7 TIME
CREATE TABLE IF NOT EXISTS runoob_tbl(runoob_id INT UNSIGNED AUTO_INCREMENT,runoob_title VARCHAR(100) NOT NULL,runoob_author VARCHAR(40) NOT NULL,runoob_test NATIONAL CHAR(40), submission_date DATE,PRIMARY KEY (runoob_id))ENGINE=InnoDB DEFAULT CHARSET=utf8;	CREATE TABLE IF NOT EXISTS runoob_tbl (runoob_id INT UNSIGNED AUTO_INCREMENT, runoob_title VARCHAR(100) NOT NULL, runoob_author VARCHAR(40) NOT NULL, runoob_test NATIONAL CHAR(40), submission_date DATE, PRIMARY KEY (runoob_id)) ENGINE = InnoDB DEFAULT CHARSET = utf8;
INSERT INTO t_order_item(order_id, user_id, status, creation_date) values (1, 1, 'insert' , '2017-08-08'), (2, 2, 'insert' , '2017-08-08') ON DUPLICATE KEY UPDATE status = 'init' ;	INSERT INTO t_order_item (order_id , user_id , status , creation_date) VALUES (1, 1, 'insert' , '2017-08-08'), (2, 2, 'insert' , '2017-08-08') ON DUPLICATE KEY UPDATE status = 'init' ;
INSERT INTO t_order SET order_id = 1, user_id = 1, status = convert(to_base64(aes_encrypt(1, 'key')) USING utf8) ON DUPLICATE KEY UPDATE status = VALUES(status);	INSERT INTO t_order SET order_id = 1, user_id = 1, status = CONVERT(to_base64(aes_encrypt(1 , 'key')) USING utf8) ON DUPLICATE KEY UPDATE status = VALUES(status);
INSERT INTO t_order (order_id, user_id, status) SELECT order_id, user_id, status FROM t_order WHERE order_id = 1;	INSERT INTO t_order (order_id , user_id , status) SELECT order_id , user_id , status FROM t_order WHERE order_id = 1;

12.4.8 Route Engine

Sharding strategies for databases and tables are matched based on the parsing context, and routing paths are generated. SQL with shard keys can be divided into the single-shard router (the shard key operator is equal), multi-shard router (the shard key operator is IN), and range router (the shard key operator is BETWEEN). SQL that does not carry shard keys adopts broadcast routing.

Sharding strategies can usually be configured either by the built-in database or by the user. The built-in database scheme is relatively simple, and the built-in sharding strategy can be roughly divided into mantissa modulo, hash, range, label, time, etc.

The sharding strategies configured by the user are more flexible. You can customize the compound sharding strategy based on the user's requirements. If it is used with automatic data migration, users do not need to work on the sharding strategies.

Sharding and data balancing can be automatically achieved by the middle layer of the database, and distributed databases can achieve elastic scalability. In the planning of ShardingSphere, the elastic scaling function will be available at V4.x.

Sharding Route

The scenario that is routed based on shard keys is divided into three types: direct route, standard route, and Cartesian route.

Direct Route

The requirement for direct route is relatively harsh. It needs to be sharded by Hint (using HintAPI to specify routes to databases and tables), and it can avoid SQL parsing and subsequent result merge on the premise of having database shards but not table shards.

Therefore, it is the most compatible one and can execute any SQL in complex scenarios including sub-queries and custom functions. The direct route can also be used when shard keys are not in SQL. For example, set the key for database sharding to 3,

```
hintManager.setDatabaseShardingValue(3);
```

If the routing algorithm is value % 2, when a logical database t_order corresponds to two physical databases t_order_0 and t_order_1, the SQL will be executed on t_order_1 after routing. The following is a sample code using the API.

```
String sql = "SELECT * FROM t_order";
try (
    HintManager hintManager = HintManager.getInstance();
    Connection conn = dataSource.getConnection();
    PreparedStatement pstmt = conn.prepareStatement(sql)) {
    hintManager.setDatabaseShardingValue(3);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
```

```

        //...
    }
}
}

```

Standard Route

The standard route is the most recommended sharding method, and it is applicable to SQL that does not contain an associated query or only contains the associated query between binding tables.

When the sharding operator is equal, the routing result will fall into a single database (table). When the sharding operator is BETWEEN or IN, the routing result will not necessarily fall into a unique database (table).

Therefore, logical SQL may eventually be split into multiple real SQL to be executed. For example, if the data sharding is carried out according to the odd and even numbers of order_id, the SQL for a single table query is as follows:

```
SELECT * FROM t_order WHERE order_id IN (1, 2);
```

Then the routing result should be:

```
SELECT * FROM t_order_0 WHERE order_id IN (1, 2);
SELECT * FROM t_order_1 WHERE order_id IN (1, 2);
```

An associated query for a binding table is as complex as a single table query and they have the same performance. For example, if the SQL of an associated query that contains binding tables is as follows:

```
SELECT * FROM t_order o JOIN t_order_item i ON o.order_id=i.order_id WHERE order_id
IN (1, 2);
```

Then the routing result should be:

```
SELECT * FROM t_order_0 o JOIN t_order_item_0 i ON o.order_id=i.order_id WHERE
order_id IN (1, 2);
SELECT * FROM t_order_1 o JOIN t_order_item_1 i ON o.order_id=i.order_id WHERE
order_id IN (1, 2);
```

As you can see, the number of SQL splits is consistent with that of a single table.

Cartesian Route

The Cartesian route is the most complex one because it cannot locate sharding rules according to the relationship between binding tables, so associated queries between unbound tables need to be disassembled and executed as cartesian product combinations. If the SQL in the previous example was not configured with binding table relationships, the routing result would be:

```
SELECT * FROM t_order_0 o JOIN t_order_item_0 i ON o.order_id=i.order_id WHERE
order_id IN (1, 2);
SELECT * FROM t_order_0 o JOIN t_order_item_1 i ON o.order_id=i.order_id WHERE
order_id IN (1, 2);
SELECT * FROM t_order_1 o JOIN t_order_item_0 i ON o.order_id=i.order_id WHERE
order_id IN (1, 2);
SELECT * FROM t_order_1 o JOIN t_order_item_1 i ON o.order_id=i.order_id WHERE
order_id IN (1, 2);
```

The Cartesian route query has low performance, so think carefully when you use it.

Broadcast Route

For SQL that does not carry shard keys, broadcast routes are used. According to the SQL type, it can be further divided into five types: full database and table route, full database route, full instance route, unicast route, and block route.

Full database and table route

The full database table route is used to handle operations on all real tables related to its logical tables in the database, including DQL and DML without shard keys, as well as DDL, etc. For example:

```
SELECT * FROM t_order WHERE good_prority IN (1, 10);
```

All tables in all databases will be traversed, matching logical tables and real table names one by one. The table that can be matched will be executed. The routing result would be:

```
SELECT * FROM t_order_0 WHERE good_prority IN (1, 10);
SELECT * FROM t_order_1 WHERE good_prority IN (1, 10);
SELECT * FROM t_order_2 WHERE good_prority IN (1, 10);
SELECT * FROM t_order_3 WHERE good_prority IN (1, 10);
```

Full database route

The full database route is used to handle operations on the database, including database management commands of type SET for database settings and transaction control statements such as TCL.

In this case, all real database matching names are traversed based on the logical database name, and the command is executed in the real database. For example:

```
SET autocommit=0;
```

If the command is executed in `t_order`, `t_order` which has two real databases, it is actually executed on both `t_order_0` and `t_order_1`.

Full instance route

Full instance route is used for DCL operations, and authorized statements are used for database instances.

No matter how many schemas are contained in an instance, each database instance is executed only once. For example:

```
CREATE USER customer@127.0.0.1 identified BY '123';
```

This command will be executed on all real database instances to ensure that users can access each instance.

Unicast Route

The unicast route is used to obtain the information of a real table. It only needs to obtain data from any real table in any database. For example:

```
DESCRIBE t_order;
```

`t_order_0` and `t_order_1`, the two real tables of `t_order`, have the same description structure, so this command is executed only once on any real table.

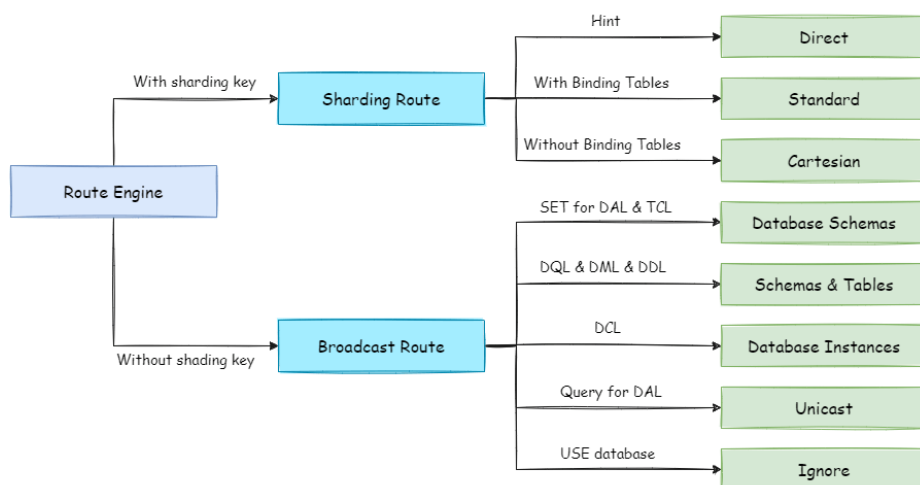
Block Route

Block route is used to block SQL operations on the database, for example:

```
USE order_db;
```

This command will not be executed in a real database because ShardingSphere uses the logical Schema and there is no need to send the Schema shift command to the database.

The overall structure of the routing engine is as follows.



12.4.9 Rewrite Engine

SQL written by engineers for logical databases and tables cannot be directly executed in real databases.

SQL rewriting is used to rewrite logical SQL into SQL that can be executed correctly in real databases. It includes rewriting for correctness and rewriting for optimization.

Rewriting for Correctness

In a scenario with table shards, you need to rewrite the logical table name in the table shards configuration to the real table name obtained after routing.

Only database shards do not require rewriting table names. Additionally, it also includes column derivation and pagination information correction.

Identifier Rewriting

The identifiers that need to be overwritten include table names, index names, and Schema names.

Rewriting table names is the process of finding the location of the logical table in the original SQL and rewriting it into a real table.

Table name rewriting is a typical scenario that requires SQL parsing. For example, if logical SQL is:


```
SELECT order_id FROM t_order WHERE order_id=1;
```

Assume that the SQL is configured with the shard key `order_id` and `order_id=1`, it will be routed to shard table 1. Then the rewritten SQL should be:

```
SELECT order_id FROM t_order_1 WHERE order_id=1;
```

In the simplest SQL scenario, it doesn't seem to matter whether or not the SQL is parsed into an abstract syntax tree.

SQL can be rewritten correctly only by finding and replacing strings. However, it is impossible to achieve the same effect in the following scenarios.

```
SELECT order_id FROM t_order WHERE order_id=1 AND remarks=' t_order xxx';
```

The correct rewritten SQL would be:

```
SELECT order_id FROM t_order_1 WHERE order_id=1 AND remarks=' t_order xxx';
```

Instead of:

```
SELECT order_id FROM t_order_1 WHERE order_id=1 AND remarks=' t_order_1 xxx';
```

Because there may be characters similar to the table name, you cannot rewrite SQL simply by replacing strings.

Let's look at a more complex scenario:

```
SELECT t_order.order_id FROM t_order WHERE t_order.order_id=1 AND remarks=' t_order xxx';
```

The above SQL uses the table name as an identifier of the field, so it needs to be modified when SQL is rewritten:

```
SELECT t_order_1.order_id FROM t_order_1 WHERE t_order_1.order_id=1 AND remarks=' t_order xxx';
```

If a table alias is defined in SQL, the alias does not need to be modified, even if it is the same as the table name. For example:

```
SELECT t_order.order_id FROM t_order AS t_order WHERE t_order.order_id=1 AND remarks=' t_order xxx';
```

Rewriting the table name is enough for SQL rewriting.

```
SELECT t_order.order_id FROM t_order_1 AS t_order WHERE t_order.order_id=1 AND remarks=' t_order xxx';
```

The index name is another identifier that can be rewritten. In some databases (such as MySQL and SQLServer), indexes are created in the dimension of tables.

Indexes in different tables can have the same name. In other databases (such as PostgreSQL and Oracle), indexes are created in the dimension of databases, and even indexes on different tables should have unique names.

In ShardingSphere, schemas are managed in the same way as tables. Logical Schemas are used to manage a set of data sources.

Therefore, ShardingSphere needs to replace the logical Schema written by the user in SQL with the real database Schema.

Currently, ShardingSphere does not support the use of Schema in DQL and DML statements. It only supports the use of Schema in database management statements. For example:

```
SHOW COLUMNS FROM t_order FROM order_ds;
```

Schema rewriting refers to the rewriting of a logical Schema using unicast routing to a correct and real Schema that is randomly found.

Column Derivation

There are two cases that need to complement columns in a query statement. In the first case, ShardingSphere needs to get the data during the result merge, but the data is not returned by the queried SQL.

In this case, it mainly applies to GROUP BY and ORDER BY. When merging the results, you need to group and order the field items according to GROUP BY and ORDER BY, but if the original SQL does not contain grouping or ordering items in the selections, you need to rewrite the original SQL. Let's look at a scenario where the original SQL has the required information for result merge.

```
SELECT order_id, user_id FROM t_order ORDER BY user_id;
```

Since user_id is used for sorting, the data of user_id needs to be retrieved in the result merge. And the above SQL can obtain the data of user_id, so there is no need to add columns.

If the selection does not contain the columns required to merge the results, you need to fill the columns, as in the following SQL:

```
SELECT order_id FROM t_order ORDER BY user_id;
```

Since the original SQL does not contain the user_id required in the result merge, you need to fill in and rewrite the SQL. Then SQL would be:

```
SELECT order_id, user_id AS ORDER_BY_DERIVED_0 FROM t_order ORDER BY user_id;
```

It should be noted that only missing columns are complemented instead of all columns. And SQL that contains * in the SELECT statement will also selectively complement columns based on the metadata information of the table. Here is a relatively complex column derivation scenario of SQL:

```
SELECT o.* FROM t_order o, t_order_item i WHERE o.order_id=i.order_id ORDER BY user_id, order_item_id;
```

We assume that only the table `t_order_item` contains the column `order_item_id`. According to the metadata information of the table, when the result is merged, the `user_id` in the ordering items exists on the table `t_order`, so there is no need to add columns. `order_item_id` is not in `t_order`, so column derivation is required. Then SQL would become:

```
SELECT o.*, order_item_id AS ORDER_BY_DERIVED_0 FROM t_order o, t_order_item i WHERE
o.order_id=i.order_id ORDER BY user_id, order_item_id;
```

The second case of column derivation is the use of AVG aggregate functions. In distributed scenarios, using $(avg1 + avg2 + avg3)/3$ to calculate the average is incorrect and should be rewritten as $(sum1 + sum2 + sum3)/(count1 + count2 + count3)$. In this case, rewriting the SQL containing AVG to SUM and COUNT is required, and recalculating the average when the results are merged. For example:

```
SELECT AVG(price) FROM t_order WHERE user_id=1;
```

The above SQL should be rewritten as:

```
SELECT COUNT(price) AS AVG_DERIVED_COUNT_0, SUM(price) AS AVG_DERIVED_SUM_0 FROM t_order
WHERE user_id=1;
```

Then you can calculate the average correctly by merging the results.

The last type of column derivation is the one that does not need to write the primary key field if the database auto-increment primary key is used during executing an INSERT SQL statement. However, the auto-increment primary key of the database cannot meet the unique primary key in distributed scenarios. Therefore, ShardingSphere provides the generation strategy of the distributed auto-increment primary key. Users can replace the existing auto-increment primary key transparently with the distributed auto-increment primary key without changing the existing code through column derivation. The generation strategy for distributed auto-increment primary keys is described below, and here only SQL rewriting is illustrated. For example, if the primary key of table `t_order` is `order_id`, the original SQL would be:

```
INSERT INTO t_order (`field1`, `field2`) VALUES (10, 1);
```

As you can see, the above SQL does not contain the auto-increment primary key, which requires the database itself to fill. After ShardingSphere is configured with the auto-increment primary key, SQL will be rewritten as:

```
INSERT INTO t_order (`field1`, `field2`, order_id) VALUES (10, 1, xxxxx);
```

The rewritten SQL will add column names of the primary key and auto-increment primary key values generated automatically at the end of the INSERT FIELD and INSERT VALUE. The xxxxx in the above SQL represents the auto-increment primary key value generated automatically.

If the INSERT SQL does not contain the column name of the table, ShardingSphere can also compare the number of parameters and the number of columns in the table meta information and automatically generate auto-increment primary keys. For example, the original SQL is:

```
INSERT INTO t_order VALUES (10, 1);
```

The rewritten SQL will simply add the auto-increment primary key in the column order in which the primary key locates:

```
INSERT INTO t_order VALUES (xxxxx, 10, 1);
```

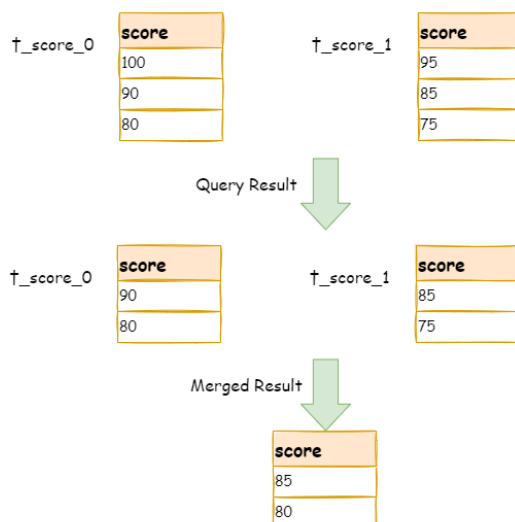
If you use placeholders to write SQL, you only need to rewrite the parameter list, not the SQL itself.

Pagination Correction

The scenario of acquiring pagination data from multiple databases is different from that of one single database. If every 10 pieces of data are taken as one page, the user wants to take the second page of data. It is not correct to acquire `LIMIT 10, 10` under sharding situations, or take out the first 10 pieces of data according to sorting conditions after merging. For example, if SQL is:

```
SELECT score FROM t_score ORDER BY score DESC LIMIT 1, 2;
```

The following picture shows the pagination execution results without SQL rewriting.

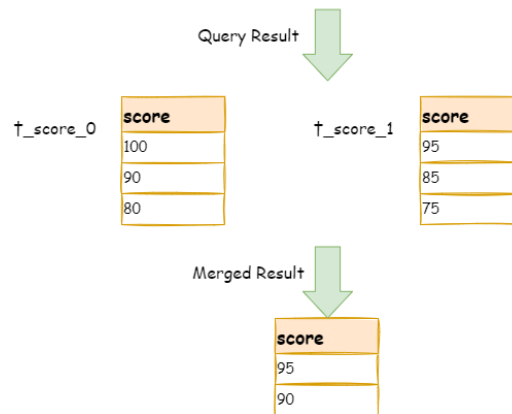


As shown in the picture, if you want to acquire the second and the third piece of data sorted by score in both tables, and they are supposed to be 95 and 90.

Since executed SQL can only acquire the second and the third piece of data from each table, i.e., 90 and 80 from `t_score_0`, 85 and 75 from `t_score_1`. When merging results, it can only merge from 90, 80, 85 and 75 already acquired, so the right result cannot be acquired anyway.

The right way is to rewrite pagination conditions as `LIMIT 0, 3`, take out all the data from the first two pages and calculate the right data based on sorting conditions. The following picture shows the execution results of pagination after SQL rewrite.

SELECT score FROM t_score ORDER BY score DESC LIMIT 0, 3



The latter the offset position is, the lower the efficiency of using `LIMIT` pagination will be. There are many ways to avoid using `LIMIT` as pagination method, such as constructing a secondary index to the number of line records and line offsets or using the end ID of the last pagination data as a condition for the next query.

When revising pagination information, if the users use the placeholder to write SQL, they only need to rewrite the parameter list rather than SQL itself.

Batch Split

When using bulk inserted SQL, if the inserted data crosses shards, the SQL needs to be rewritten to prevent excess data from being written to the database.

The insertion operation differs from the query operation in that the query statement does not affect the data even if it uses the shard key that does not exist in the current shard. In contrast, insertion operations must remove excess shard keys. For example, see the following SQL:

```
INSERT INTO t_order (order_id, xxx) VALUES (1, 'xxx'), (2, 'xxx'), (3, 'xxx');
```

If the database is still divided into two parts according to the odd and even number of `order_id`, this SQL will be executed after its table name is revised. Then, both shards will be written with the same record.

Though only the data that satisfies sharding conditions can be retrieved from the query statement, it is not reasonable for the schema to have excessive data. So SQL should be rewritten as:

```
INSERT INTO t_order_0 (order_id, xxx) VALUES (2, 'xxx');
INSERT INTO t_order_1 (order_id, xxx) VALUES (1, 'xxx'), (3, 'xxx');
```

IN query is similar to batch insertion, but IN operation will not lead to wrong data query result. Through rewriting IN query, the query performance can be further improved. See the following SQL:

```
SELECT * FROM t_order WHERE order_id IN (1, 2, 3);
```

The SQL is rewritten as:

```
SELECT * FROM t_order_0 WHERE order_id IN (2);
SELECT * FROM t_order_1 WHERE order_id IN (1, 3);
```

The query performance will be further improved. For now, ShardingSphere has not realized this rewrite strategy, so the current rewrite result is:

```
SELECT * FROM t_order_0 WHERE order_id IN (1, 2, 3);
SELECT * FROM t_order_1 WHERE order_id IN (1, 2, 3);
```

Though the execution result of SQL is right, it did not achieve the highest query efficiency.

Rewriting for Optimization

Its purpose is to effectively improve performance without influencing the correctness of the query. It can be divided into single node optimization and stream merger optimization.

Single Node Optimization

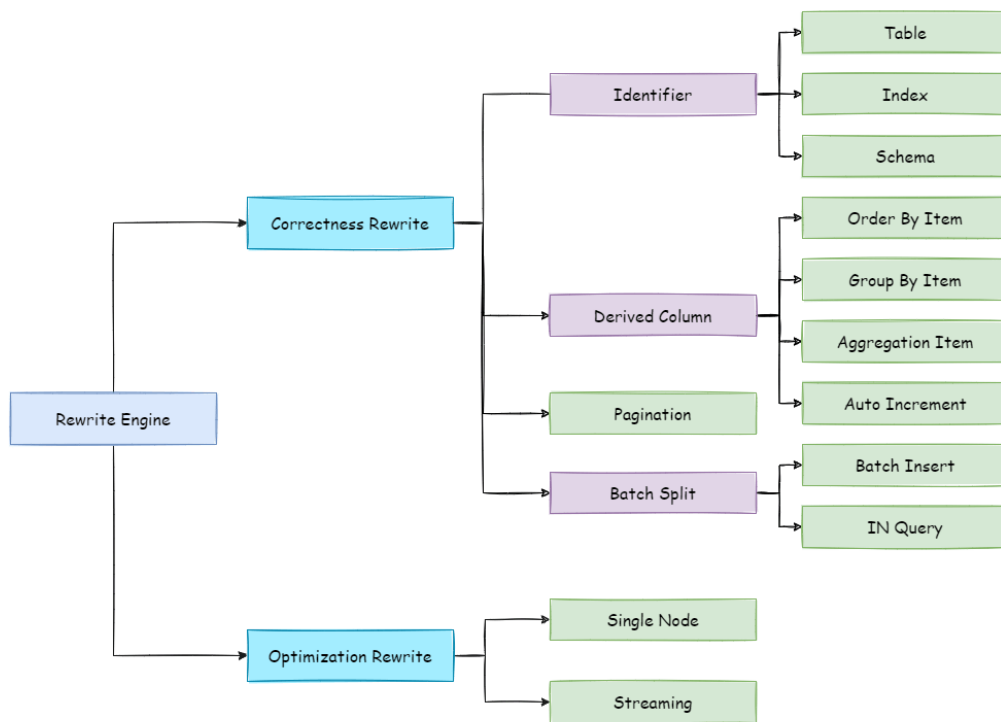
It refers to the optimization that stops the SQL rewrite from the route to the single node. After acquiring one route result, if it is routed to a single data node, there is no need to involve result merger, as well as rewrites such as column derivation and pagination information correction.

In particular, there is no need to read from the first piece of information, which reduces the pressure on the database to a large extent and saves meaningless consumption of the network bandwidth.

Stream Merger Optimization

It only adds ORDER BY and ordering items and sorting orders identical with grouping items to SQL that contains GROUP BY. And it is used to transfer memory merger to stream merger. Stream merger and memory merger will be explained in detail in the result merger section.

The overall structure of the rewrite engine is shown in the following picture.



12.4.10 Execute Engine

ShardingSphere uses an automated execution engine to safely and efficiently send the real SQL, which has been routed and rewritten, to the underlying data source for execution.

It does not simply send SQL directly to the data source for execution via JDBC, nor are execution requests placed directly into a thread pool for concurrent execution.

It focuses more on the creation of a balanced data source connection, the consumption generated by the memory usage, and the maximum utilization of the concurrency. The objective of the execution engine is to automatically balance resource control with execution efficiency.

Connection Mode

From the perspective of resource control, the connection number a business can make to the database should be limited. It can effectively prevent certain business operations from occupying excessive resources, exhausting database connection resources, and influencing the normal access of other businesses.

Especially when one database instance contains many sub-tables, a logical SQL that does not contain any shard key will produce a large number of real SQLs that fall into different tables in one database. If each real SQL takes an independent connection, a query will undoubtedly take up excessive resources.

From the perspective of execution efficiency, maintaining an independent database connection for each shard query can make more effective use of multi-thread to improve execution efficiency.

Creating a separate thread for each database connection allows I/O consumption to be processed in parallel. Maintaining a separate database connection for each shard also prevents premature loading of query result data into memory.

It is enough for independent database connections to maintain result set quotation and cursor position, and move the cursor when acquiring corresponding data.

Merging the result set by moving down its cursor is called the stream merger. It does not need to load all the query results into the memory, which can effectively save memory resources effectively and reduce the frequency of garbage collection.

If each shard query cannot be guaranteed to have an independent database connection, the current query result set needs to be loaded into memory before reusing the database connection to obtain the query result set of the next shard table. Therefore, though the stream merger can be used, it will also degenerate into the memory merger in this scenario.

On the one hand, we need to control and protect database connection resources; on the other hand, it is important to save middleware memory resources by adopting a better merging mode. How to deal with the relationship between the two is a problem that the ShardingSphere execution engine needs to solve. Specifically, if an SQL is sharded through the ShardingSphere, it needs to operate on 200 tables under a database instance. So, should we choose to create 200 connections in parallel, or one connection in sequence? How to choose between efficiency and resource control? For the above scenario, ShardingSphere provides a solution. It introduces the concept of Connection Mode, which is divided into MEMORY_STRICTLY and CONNECTION_STRICTLY.

MEMORY_STRICTLY Mode

The prerequisite to using this mode is that ShardingSphere does not restrict the connection number of one operation. If the actual executed SQL needs to operate 200 tables in some database instance, it will create a new database connection for each table and deal with them concurrently through multi-thread to maximize the execution efficiency. When SQL meets the conditions, stream merger is preferred to avoid memory overflow or frequent garbage recycling.

CONNECTION_STRICTLY Mode

The prerequisite to using this mode is that ShardingSphere strictly restricts the connection consumption number of one operation. If the SQL to be executed needs to operate 200 tables in a database instance, it will create one database connection and operate them serially. If shards exist in different databases, it will still adopt multi-thread operations for different databases, but with only one database connection being created for each operation in each database. It prevents the problem of consuming too many database connections for one request. The mode chooses memory merger all the time.

The MEMORY_STRICTLY mode applies to OLAP operation and can increase the system throughput by removing database connection restrictions. It is also applicable to OLTP operation, which usually has shard keys and can be routed to a single shard. So it is a wise choice to control database connections strictly to make sure that database resources in an online system can be used by more applications.

Automatic Execution Engine

ShardingSphere initially leaves the decision of which mode to use up to the users and they can choose to use MEMORY_STRICTLY mode or CONNECTION_STRICTLY mode according to their actual business scenarios.

This solution gives users the right to choose, who must understand the pros and cons of the two modes and make a choice based on the requirements of the business scenarios. No doubt, it is not the best solution as it increases users' learning and use costs.

This dichotomy solution, which leaves the switching of the two modes to static initialization, lacks flexibility. In practical scenarios, the routing result varies with SQL and placeholder indexes. This means that some operations may need to use memory merger, while others may prefer stream merger. Connection modes should not be set by the user before ShardingSphere is started, but should be determined dynamically based on the SQL and placeholder indexes scenarios.

In order to reduce the usage cost for users and achieve a dynamic connection mode, ShardingSphere has extracted the concept of the automatic execution engine to eliminate the connection mode concept internally. The user does not need to know what the MEMORY_STRICTLY mode and CONNECTION_STRICTLY mode are, but the execution engine automatically selects the best execution scheme according to the current scenario.

The automatic execution engine chooses the connection mode based on each SQL operation. For each SQL request, the automatic execution engine will do real-time calculations and evaluations according to its route result and execute the appropriate connection mode automatically to strike the optimal balance between resource control and efficiency. For the automatic execution engine, users only need to configure `maxConnectionSizePerQuery`, which represents the maximum connection number allowed by each database for one query.

The execution engine is divided into two phases: preparation and execution.

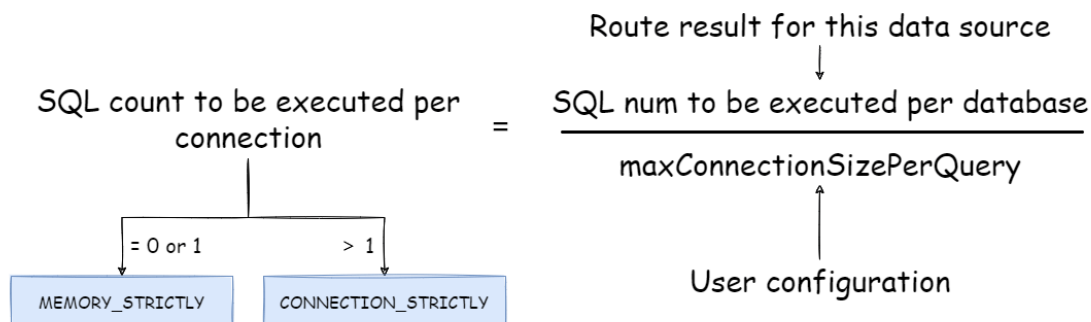
Preparation Phase

As indicated by its name, this phrase is used to prepare the data to be executed. It can be divided into two steps: result set grouping and unit creation.

Result set grouping is the key to realizing the internal connection model concept. According to the configuration items of `maxConnectionSizePerQuery`, the execution engine will choose an appropriate connection mode based on the current route result.

Detailed steps are as follow:

1. Group SQL route results according to data source names.
2. As we can see in the following formula, users can acquire the SQL route result set to be executed by each database instance within the `maxConnectionSizePerQuery` permission range and calculate the optimal connection mode of this request.

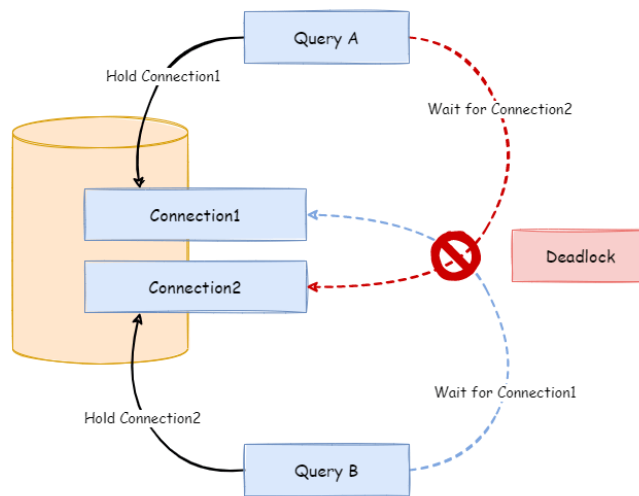


Within the scope of the `maxConnectionSizePerQuery` allowed, when the request number that one connection needs to execute is more than 1, the current database connection cannot hold the corresponding data result set, so it must use memory merger. On the contrary, when the number equals 1, the current database connection can hold the corresponding data result set, and it can use stream merger.

Each connection mode selection is specific to each physical database. That is, if you route to more than one database in the same query, the connection mode of each database may not be the same, and they may be mixed. Users can use the route grouping result acquired from the last step to create the execution unit. When the data source uses technologies, such as the database connection pool, to control database connection numbers, there is a chance that a deadlock will occur if concurrency is not handled properly while retrieving database connections. As multiple requests wait for each other to release database connection resources, starvation occurs, causing the crossing deadlock.

For example, suppose that a query requires obtaining two database connections at a data source and routing queries to two sub-tables of the same database. It is possible that query A has obtained one database connection from this data source and is waiting to obtain another database connection.

Query B has also acquired a database connection at the data source and is also waiting for another database connection to be acquired. If the maximum number of connections allowed in the database connection pool is 2, then the two query requests will wait forever. The following diagram depicts a deadlock situation.



ShardingSphere synchronizes database connections to avoid deadlocks. When it creates the execution unit, it atomically obtains all the database connections required by the SQL request at one time, eliminating the possibility of obtaining partial resources in each query request.

Because the operation on the database is very frequent, locking a database connection each time when acquiring it will reduce the concurrency of ShardingSphere. Therefore, ShardingSphere has improved two aspects here:

1. Locking can be avoided and only one database connection needs to be obtained each time. Because under this circumstance, two requests waiting for each other will not happen, so there is no need for locking. Most OLTP operations use shard keys to route to the unique data node, which makes the system completely unlocked and further improves the concurrency efficiency. In addition to routing to a single shard, read/write-splitting also belongs to this category.
2. Locking resources only happens in `MEMORY_STRICTLY` mode. When using `CONNECTION_STRICTLY` mode, all the query result sets will release database connection resources after loading them to the memory, so deadlock wait will not appear.

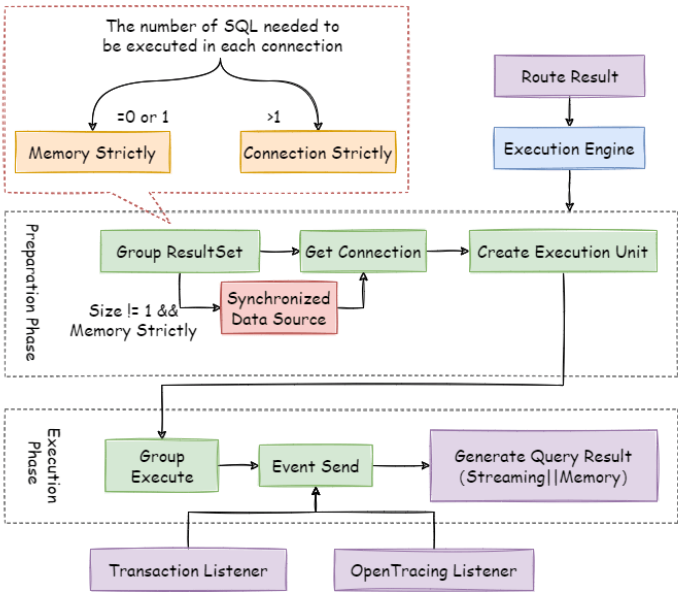
Execution Phase

This stage is used to actually execute SQL and is divided into two steps: group execution and merger result generation.

Group execution can distribute execution unit groups generated in the preparation phase to the underlying concurrency engine and send events for each key step during the execution process, such as starting, successful and failed execution events. The execution engine only focuses on sending events rather than subscribers to the event. Other ShardingSphere modules, such as distributed transactions, call linked tracing and so on, will subscribe to the events of interest and process them accordingly.

ShardingSphere generates memory merger result sets or stream merger result sets through the connection mode acquired in the preparation phase. And then it passes the result set to the result merger engine for the next step.

The overall structure of the execution engine is divided as shown below.



12.4.11 Merger Engine

Result merger refers to merging multi-data result sets acquired from all the data nodes as one result set and returning it to the requesting client correctly.

The result merger supported by ShardingSphere can be divided into five functional types: traversal, order-by, group-by, pagination and aggregation, which are combined rather than mutually exclusive. From the perspective of structure, it can be divided into stream merger, memory merger and decorator merger, among which stream merger and memory merger are mutually exclusive, and decorator merger can be further processed based on stream merger and memory merger.

Since the result set is returned from the database one by one instead of being loaded to the memory all at a time, the method of merging the result sets returned from the database can greatly reduce memory consumption and is the preferred method of merging.

Stream merger means that each time the data is obtained from the result set is able to return the correct single piece of data line by line. It is the best fit with the native method of returning the result set of the database. Traversal, order-by, and stream group-by are all examples of the stream merger.

Memory merger needs to traverse all the data in the result set and store it in the memory first. After unified grouping, ordering, aggregation and other calculations, the data is packaged into the data result set accessed one by one and returned.

Decorator merger merges and reinforces all the result sets function uniformly. Currently, decorator merger has two types: pagination merger and aggregation merger.

Traversal Merger

As the simplest merger method, traversal merger only requires the combination of multiple data result sets into a one-way linked table. After traversing current data result sets in the linked table, it only needs to move the elements of the linked table back one bit and continue traversing the next data result set.

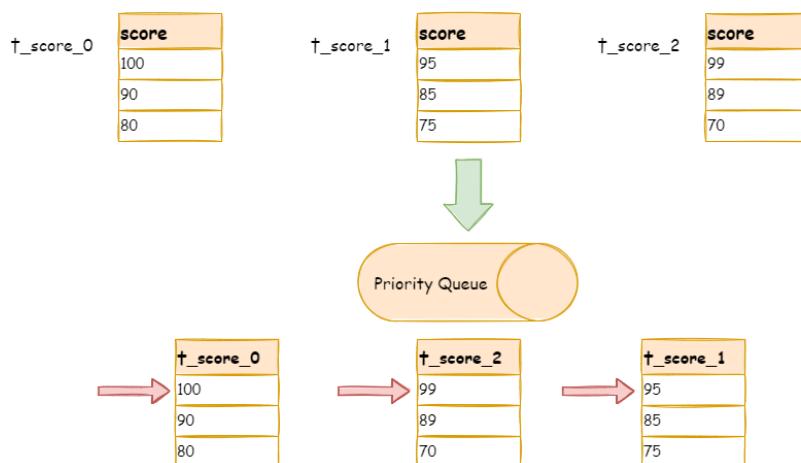
Order-by Merger

Because there is an ORDER BY statement in SQL, each data result has its own order. So it only needs to sort data value that the result set cursor currently points to, which is equal to sorting multiple ordered arrays. Therefore, order-by merger is the most suitable sorting algorithm in this scenario.

When merging ordered queries, ShardingSphere will compare current data values in each result set (which is realized by the Java Comparable interface) and put them into the priority queue. Each time when acquiring the next piece of data, it only needs to move down the result set cursor at the top of the queue, reenter the priority order according to the new cursor and relocate its own position.

Here is an instance to explain ShardingSphere's order-by merger. The following picture is an illustration of ordering by the score. Data result sets returned by 3 tables are shown in the example and each of them has already been ordered according to the score, but there is no order between the 3 data result sets. Order the data value that the result set cursor currently points to in these 3 result sets. Then put them into the priority queue. The first data value of `t_score_0` is the biggest, followed by that

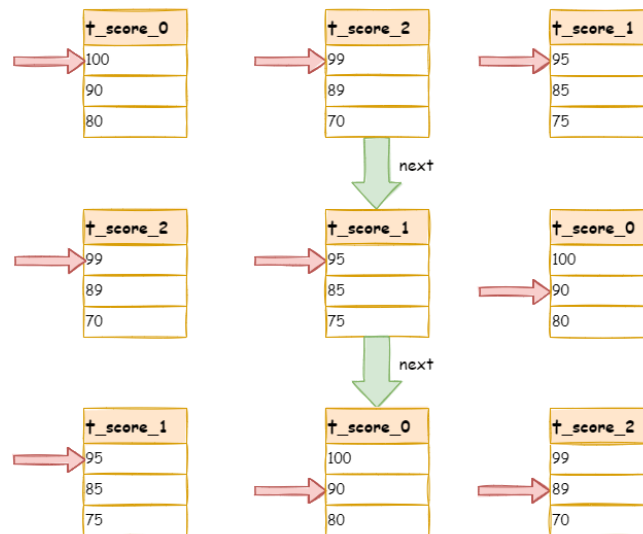
of `t_score_2` and `t_score_1` in sequence. Thus, the priority queue is ordered by the sequence of `t_score_0`, `t_score_2` and `t_score_1`.



The following diagram illustrates how the order-by merger works when using next call. We can see from the diagram that when using the next call, `t_score_0` at the first of the queue will be popped out. After returning the data value currently pointed by the cursor (i.e., 100) to the requesting client, the cursor will be moved down and `t_score_0` will be put back into the queue.

While the priority queue will also be ordered according to the `t_score_0` data value (90 here) pointed by the cursor of the current data result set. According to the current value, `t_score_0` is at the end of the queue, and the data result set of `t_score_2`, originally in the second place of the queue, automatically moves to the first place of the queue.

In the second next call, `t_score_2` in the first place is popped out. Its value pointed by the cursor of the data result set is returned to the client end, with its cursor moved down to rejoin the queue, and the following will be the same way. If there is no data in the result set, it will not rejoin the queue.



It can be seen that when data in each result set is ordered, but multiple result sets are disordered, ShardingSphere can still order them with no need to upload all the data to the memory. In the stream merger method, each `next` operation only acquires the right piece of data each time, which saves memory consumption to a large extent.

On the other hand, the order-by merger has maintained the orderliness on the horizontal axis and vertical axis of the data result set. Naturally ordered, the vertical axis refers to each data result set itself, which is acquired by SQL with `ORDER BY`. The horizontal axis refers to the current value pointed by each data result set, and its order needs to be maintained by the priority queue. Each time when the current cursor moves down, it requires putting the result set in the priority order again, which means only the cursor of the first data result set can be moved down.

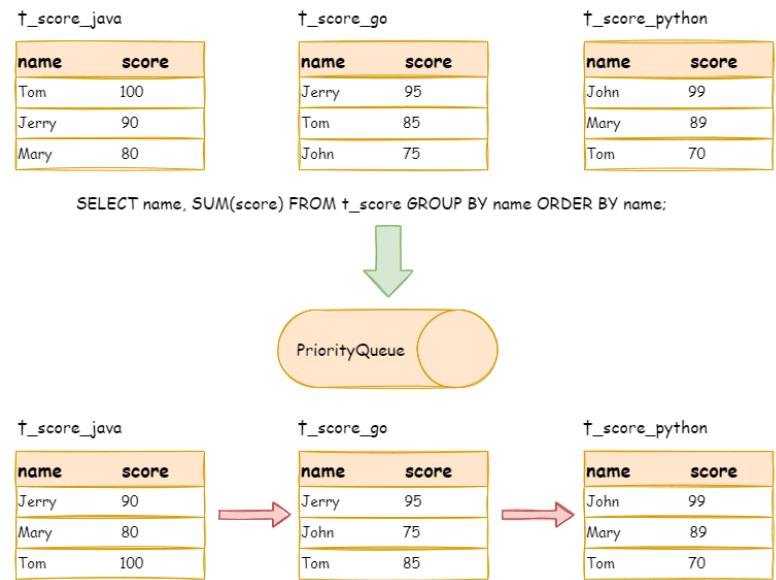
Group-by Merger

Group-by merger is the most complex one and can be divided into stream group-by merger and memory group-by merger. Stream group-by merger requires that the SQL's ordering items must be consistent with the field and ordering types (ASC or DESC) of the group-by item; otherwise, data correctness can only be guaranteed by memory merger.

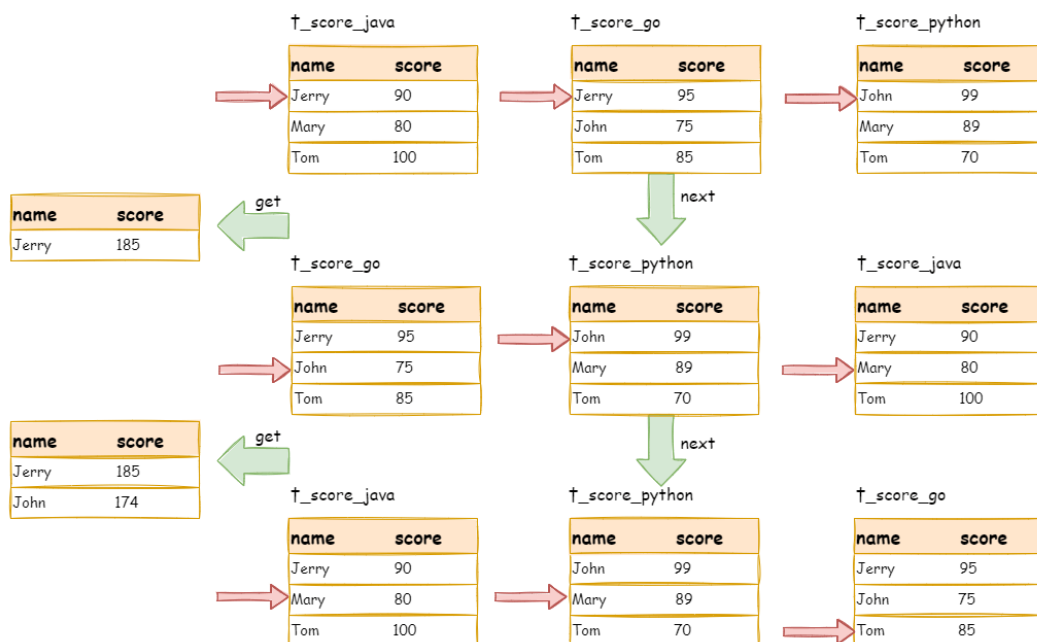
For instance, if it is sharded based on subject, the table structure contains the examinees' name (to simplify, name repetition is not taken into consideration) and score. The following SQL is used to acquire each examinee's total score:

```
SELECT name, SUM(score) FROM t_score GROUP BY name ORDER BY name;
```

When order-by item and group-by item are totally consistent, the data obtained is continuous. The data required by group-by is all stored in the data value that the data result set cursor currently points to. Thus, stream group-by merger can be used, as illustrated by the diagram:



The merging logic is similar to that of order-by merger. The following picture shows how the stream group-by merger works in the next call.



We can see from the picture that, in the first `next` call, `t_score_java` in the first place will be popped out of the queue, along with other result set data having the same grouping value of “Jerry”. After acquiring all the students’ scores with the name of “Jerry”, the accumulation operation will proceed. Hence, after the first `next` call is finished, the result set acquired is the sum of Jerry’ s scores. At the same time, all the cursors in data result sets will be moved down to a different data value next to “Jerry” and reordered according to the current result set value. Thus, the data that contains the second name “John” will be put at the beginning of the queue.

Stream group-by merger is different from order-by merger only in two aspects:

1. It will take out all the data with the same group item from multiple data result sets at once.
2. It carried out the aggregation calculation according to the aggregation function type.

For the inconsistency between the grouping item and ordering item, it requires uploading all the data to the memory to group and aggregate, since the relevant data value needed to acquire group information is not continuous, and stream merger is not available. For example, acquire each examinee’ s total score through the following SQL and order them from the highest to the lowest:

```
SELECT name, SUM(score) FROM t_score GROUP BY name ORDER BY score DESC;
```

Then, stream merger is not able to use, for the data taken out from each result set is the same as the original data of the order-by merger diagram in the upper half part structure.

When SQL only contains the group-by statement, according to different database implementations, its sorting order may not be the same as the group order. The lack of an ordering statement indicates the order is not important in this SQL. Therefore, through the optimization of SQL rewriting, Sharding-

Sphere can automatically add the ordering item the same as the grouping item, converting it from the memory merger that consumes memory to the stream merger.

Aggregation Merger

Whether it is stream group-by merger or memory group-by merger, they process the aggregation function in the same way. In addition to grouped SQL, ungrouped SQL can also use aggregate functions. Therefore, aggregation merger is an additional merging ability based on what has been introduced above, i.e., the decorator mode. The aggregation function can be categorized into three types: comparison, sum and average.

The comparison aggregation function refers to MAX and MIN. They need to compare all the result set data of each group and simply return the maximum or minimum value.

The sum aggregation function refers to SUM and COUNT. They need to sum up all the result set data of each group.

The average aggregation function refers only to AVG. It must be calculated through SUM and COUNT rewritten by SQL, which has been mentioned in the SQL rewriting section.

Pagination Merger

All the merger types above can be paginated. Pagination is the decorator added to other kinds of mergers. ShardingSphere strengthens its ability to paginate the data result set through decorator mode. The pagination merger is responsible for filtering unnecessary data.

ShardingSphere's pagination function can be misleading to users in that they may think it will take a large amount of memory. In distributed scenarios, it can only guarantee the data correctness by rewriting `LIMIT 10000000, 10` to `LIMIT 0, 10000010`. Users can easily misunderstand that ShardingSphere uploads a large amount of meaningless data to the memory and has the risk of memory overflow. Actually, it can be known from the principle of stream merger that only memory group-by merger will upload all the data to the memory. Generally speaking, SQL used for OLAP grouping, is often applied to massive calculations or small result generation, and it won't generate vast result data. Except for memory group-by merger, other scenarios all use stream merger to acquire data result set. So ShardingSphere would skip unnecessary data through the next call method in the result set, rather than storing it in the memory.

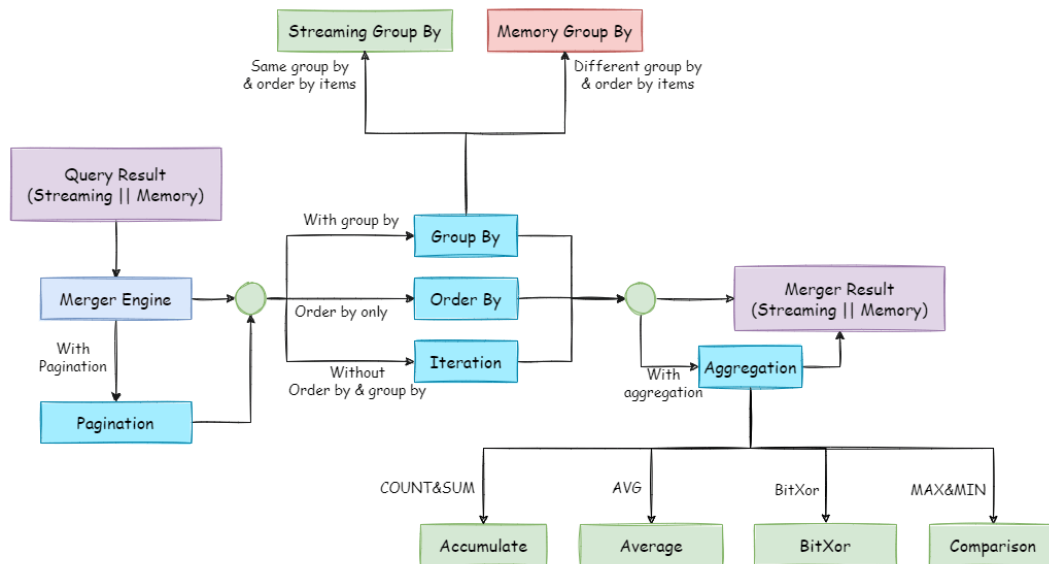
But it should be noted that pagination with LIMIT is not the best practice, because a large amount of data still needs to be transmitted to ShardingSphere's memory space for ordering. LIMIT cannot query data by index, so paginating with ID is a better solution if ID continuity can be guaranteed. For example:

```
SELECT * FROM t_order WHERE id > 100000 AND id <= 100010 ORDER BY id;
```

Or query the next page through the ID of the last query result, for example:

```
SELECT * FROM t_order WHERE id > 10000000 LIMIT 10;
```

The overall structure of the merger engine is shown in the following diagram:



12.5 Transaction

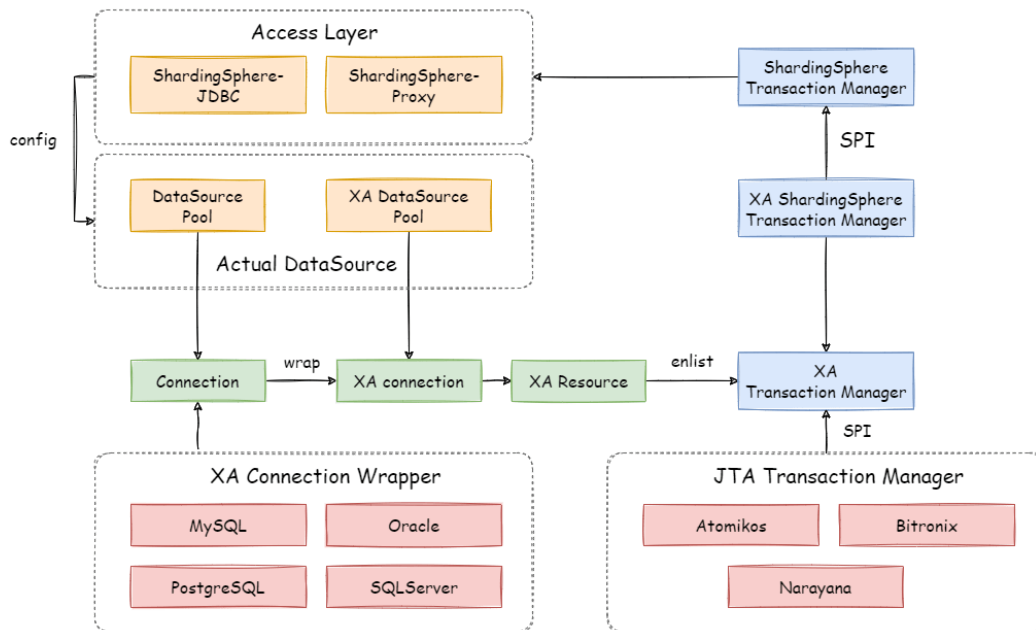
12.5.1 Navigation

This chapter mainly introduces the principles of the distributed transactions:

- 2PC transaction with XA
- BASE transaction with Seata

12.5.2 XA Transaction

XAShardingSphereTransactionManager is XA transaction manager of Apache ShardingSphere. Its main responsibility is manage and adapt multiple data sources, and sent corresponding transactions to concrete XA transaction manager.



Transaction Begin

When receiving set `autoCommit=0` from client, `XAShardingSphereTransactionManager` will use XA transaction managers to start overall XA transactions, which is marked by `XID`.

Execute actual sharding SQL

After `XAShardingSphereTransactionManager` register the corresponding `XAResource` to the current XA transaction, transaction manager will send `XAResource.start` command to databases. After databases received `XAResource.end` command, all SQL operator will mark as XA transaction.

For example:

```

XAResource1.start          ## execute in the enlist phase
statement.execute("sql1");
statement.execute("sql2");
XAResource1.end            ## execute in the commit phase
    
```

`sql1` and `sql2` in example will be marked as XA transaction.

Commit or Rollback

After `XAShardingSphereTransactionManager` receives the commit command in the access, it will delegate it to the actual XA manager. It will collect all the registered `XAResource` in the thread, before sending `XAResource.end` to mark the boundary for the XA transaction. Then it will send prepare command one by one to collect votes from `XAResource`. If all the `XAResource` feedback is OK, it will send commit command to finally finish it; If there is any No `XAResource` feedback, it will send rollback command to roll back. After sending the commit command, all `XAResource` exceptions will be submitted again according to the recovery log to ensure the atomicity and high consistency.

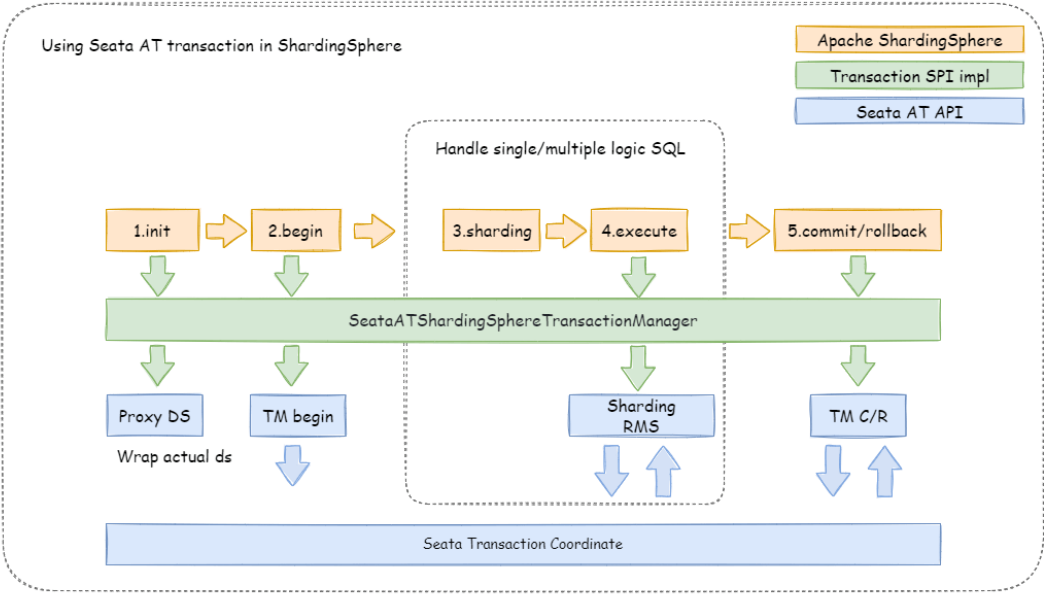
For example:

```
XAResource1.prepare      ## ack: yes
XAResource2.prepare      ## ack: yes
XAResource1.commit
XAResource2.commit

XAResource1.prepare      ## ack: yes
XAResource2.prepare      ## ack: no
XAResource1.rollback
XAResource2.rollback
```

12.5.3 Seata BASE transaction

When integrating Seata AT transaction, we need to integrate TM, RM and TC component into ShardingSphere transaction manager. Seata have proxied `DataSource` interface in order to RPC with TC. Similarly, Apache ShardingSphere faced to `DataSource` interface to aggregate data sources too. After Seata `DataSource` encapsulation, it is easy to put Seata AT transaction into Apache ShardingSphere sharding ecosystem.



Init Seata Engine

When an application containing `ShardingSphereTransactionBaseSeataAT` startup, the user-configured `DataSource` will be wrapped into `seata DataSourceProxy` through `seata.conf`, then registered into RM.

Transaction Begin

TM controls the boundaries of global transactions. TM obtains the global transaction ID by sending `Begin` instructions to TC. All branch transactions participate in the global transaction through this global transaction ID. The context of the global transaction ID will be stored in the thread local variable.

Execute actual sharding SQL

Actual SQL in Seata global transaction will be intercepted to generate undo snapshots by RM and sends participate instructions to TC to join global transaction. Since actual sharding SQLs executed in multi-threads, global transaction context should transfer from main thread to child thread, which is exactly the same as context transfer between services.

Commit or Rollback

When submitting a seata transaction, TM sends TC the commit and rollback instructions of the global transaction. TC coordinates all branch transactions for commit and rollback according to the global transaction ID.

12.6 Data Migration

12.6.1 Explanation

The current data migration solution uses a completely new database cluster as the migration target.

This implementation has the following advantages:

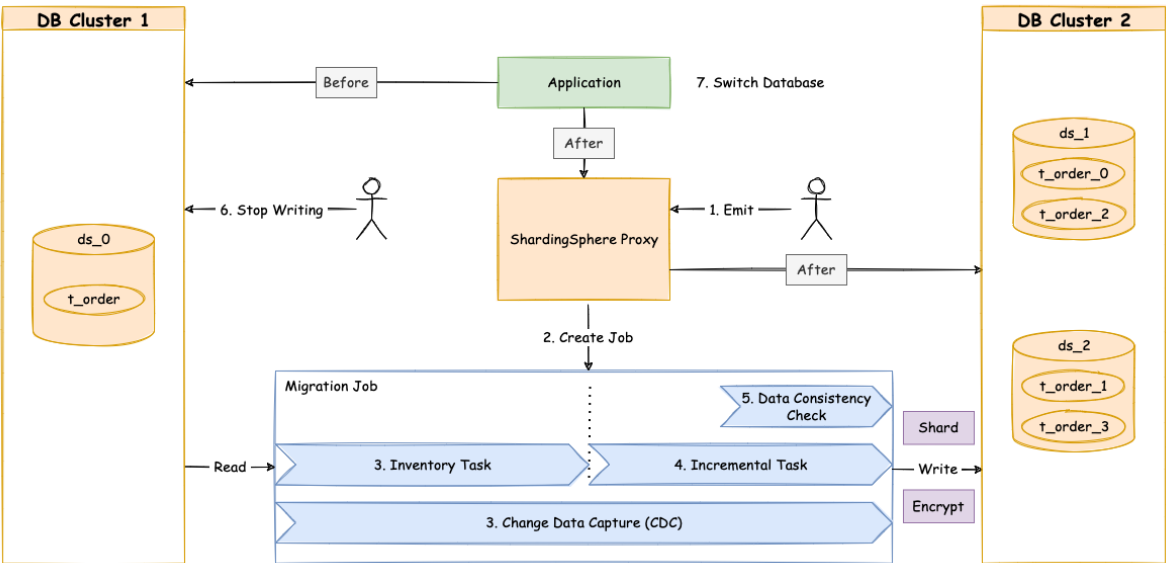
- 1. No impact on the original data during migration.
- 2. No risk in case of migration failure.
- 3. Freedom from sharding strategy limitations.

The implementation has the following disadvantages:

- 1. Redundant servers can exist for a certain period of time.
- 2. All data needs to be moved.

A single data migration mainly consists of the following phases:

- 1. Preparation.
- 2. Stock data migration.
- 3. The synchronization of incremental data.
- 4. Traffic switching .



12.6.2 Execution Stage Explained

Preparation

In the preparation stage, the data migration module verifies data source connectivity and permissions, counts stock data statistics, records the log and finally shards the tasks according to data volume and parallelism set by the users.

Stock data migration

Execute the stock data migration tasks that have been sharded during preparation stage. The stock migration stage uses JDBC queries to read data directly from the source and write into the target based on the sharding rules and other configurations.

The Synchronization of incremental data

Since the duration of stock data migration depends on factors such as data volume and parallelism, it is necessary to synchronize the data added to the business operations during this period. Different databases differ in technical details, but in general they are all based on replication protocols or WAL logs to achieve the capture of changed data.

- MySQL: subscribe and parse binlog.
- PostgreSQL: uses official logical replication `test_decoding`.

The incremental data captured is also written into the new data nodes by the data migration modules. When synchronization of incremental data is completed (the incremental data flow is not interrupted since the business system is still in function), you can then move to the traffic switching stage.

Traffic Switching

During this stage, there may be a read-only period of time, where data in the source data nodes is allowed to be in static mode for a short period of time to ensure that the incremental synchronization can be fully completed. Users can set this by shifting the database to read-only status or by controlling the traffic flow generated from the source.

The length of this read-only window depends on whether users need to perform consistency checks on the data and the exact amount of data in this scenario. Consistency check is an independent task. It supports separate start/stop and breakpoint resume.

Once confirmed, the data migration is complete. Users can then switch the read traffic or write traffic to Apache ShardingSphere.

12.6.3 References

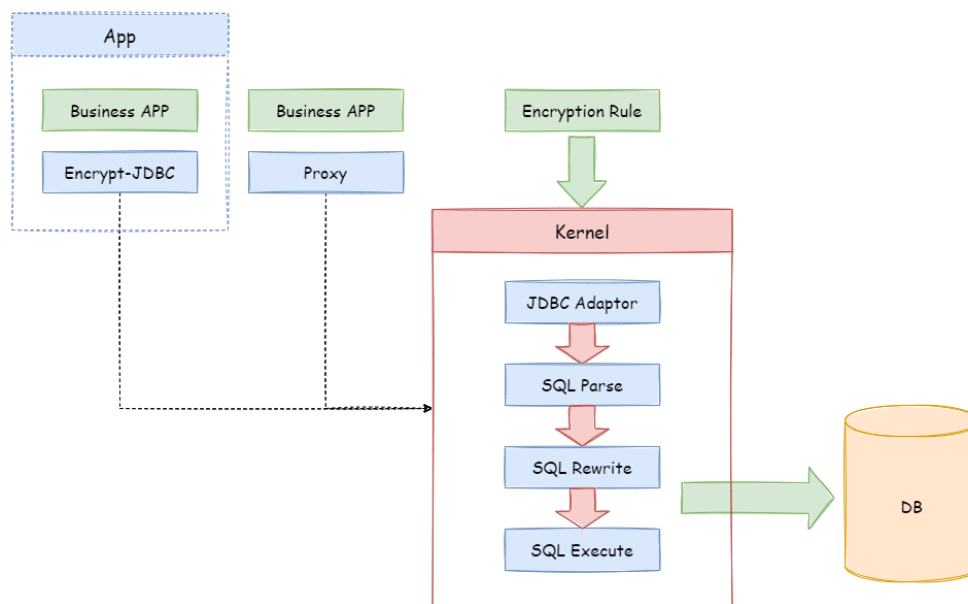
Configurations of data migration

12.7 Encryption

Apache ShardingSphere parses the SQL entered by users and rewrites the SQL according to the encryption rules provided by users.

When a user queries data, it only retrieves ciphertext data from the database, decrypts it, and finally returns the decrypted source data to the user. Apache ShardingSphere achieves a transparent and automatic data encryption process. Users can use encrypted data as normal data without paying attention to the implementation details of data encryption.

12.7.1 Overall Architecture



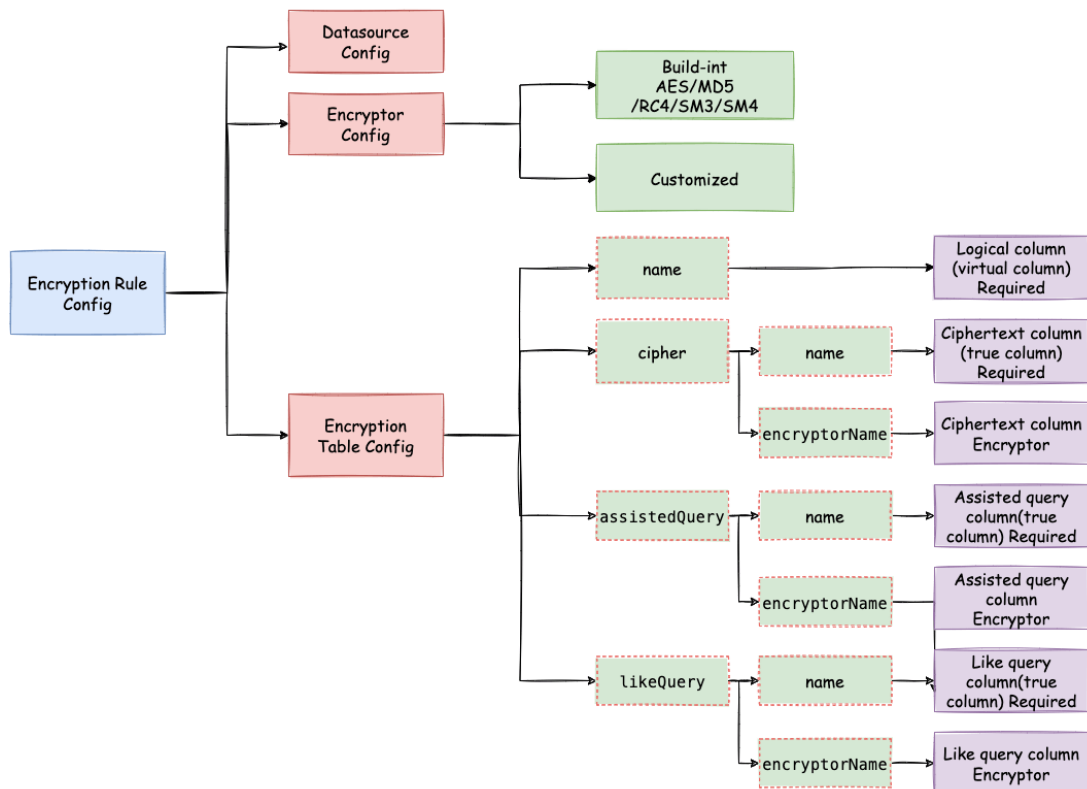
The encrypted module intercepts the SQL initiated by the user and parses and understands the SQL behavior through the SQL syntactic parser. Then it finds out the fields to be encrypted and the encryption and decryption algorithm according to the encryption rules introduced by the user and interacts with the underlying database.

Apache ShardingSphere will encrypt the plaintext requested by users and store it in the underlying database. When the user queries, the ciphertext is extracted from the database, decrypted, and re-

turned to the terminal user. By shielding the data encryption process, users do not need to operate the SQL parsing process, data encryption, and data decryption.

12.7.2 Encryption Rules

Before explaining the whole process, we need to understand the encryption rules and configuration. Encryption configuration is mainly divided into three parts: data source configuration, encryptor configuration, encryption table configuration, as shown in the figure below:



Data source configuration: the configuration of the data source.

Encryptor configuration: refers to the encryption algorithm used for encryption and decryption. Currently, ShardingSphere has three built-in encryption and decryption algorithms: AES, MD5 and RC4. Users can also implement a set of encryption and decryption algorithms by implementing the interfaces provided by ShardingSphere.

Encryption table configuration: it is used to tell ShardingSphere which column in the data table is used to store ciphertext data (`cipherColumn`), and which column the user would like to use for SQL writing (`logicColumn`).

What does it mean by “which column the user would like to use for SQL writing (`logicColumn`)”? We have to know first why the encrypted module exists. The goal of the encrypted module is to shield the underlying data encryption process, which means we don’t want users to know how data is encrypted and decrypted, and how to store ciphertext data into `cipherColumn`. In other words, we don’t want users to know there is a `cipherColumn` or how they are used. Therefore, we need to provide the user with a conceptual column

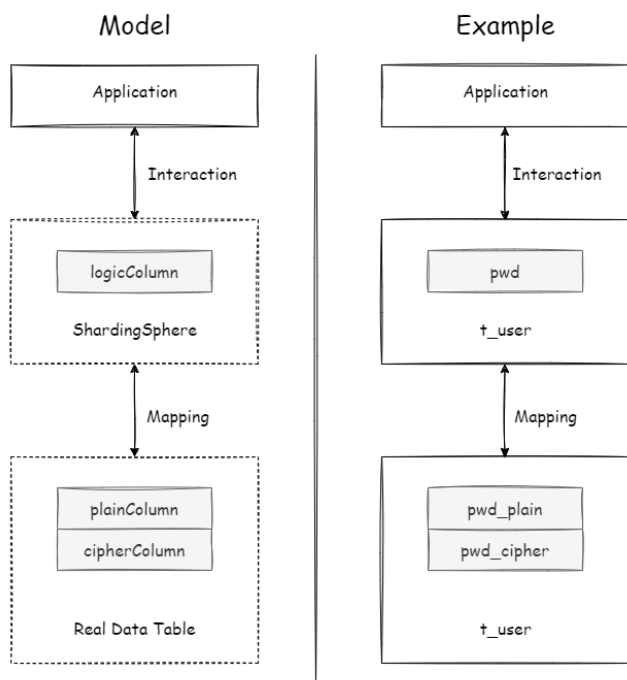
that can be separated from the real column in the underlying database. It may or may not be a real column in the database table so that users can change the column names of cipherColumn of the underlying database at will. The only thing we have to ensure is that the user's SQL is written towards the logical column, and the correct mapping relation between logicColumn and cipherColumn can be seen in the encryption rules.

Query attribute configuration: if both plaintext and ciphertext data are stored in the underlying database table, this attribute can be used to determine whether to query the plaintext data in the database table and return it directly, or query the ciphertext data and return it after decryption through Apache ShardingSphere. This attribute can be configured at the table level and the entire rule level. The table-level has the highest priority.

12.7.3 Encryption Process

For example, if there is a table named `t_user` in the database, and they're two fields in the table: `pwd_cipher` for storing ciphertext data, and `logicColumn` is defined as `pwd`, then users should write SQL for `logicColumn`, that is `INSERT INTO t_user SET pwd = '123'`. Apache ShardingSphere receives the SQL and finds that the `pwd` is the `logicColumn` based on the encryption configuration provided by the user. Therefore, it encrypts the logical column and its corresponding plaintext data.

Apache ShardingSphere transforms the column names and data encryption mapping between the logical columns facing users and cipher columns facing the underlying database. As shown in the figure below:

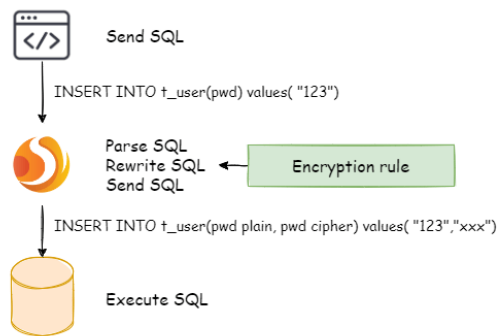


The user's SQL is separated from the underlying data table structure according to the encryption rules

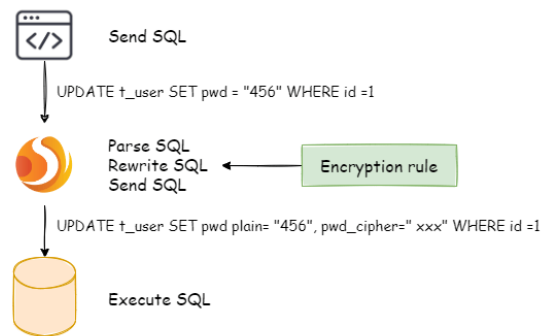
provided by the user so that the user's SQL writing does not depend on the real database table structure. The connection, mapping, and transformation between the user and the underlying database are handled by Apache ShardingSphere.

The picture below shows the processing flow and conversion logic when the encryption module is used to add, delete, change and check, as shown in the figure below.

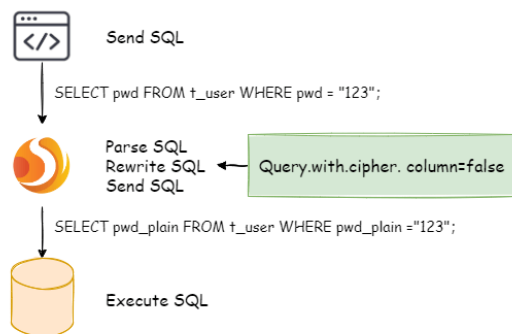
Insert-use logical column



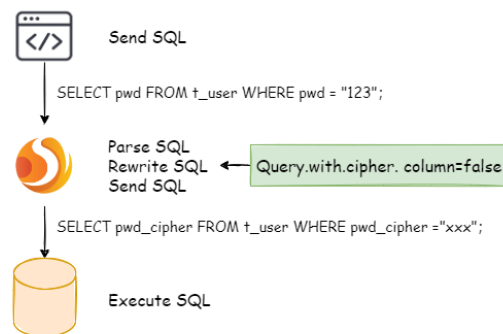
Update-use logical column



Query-use plaintext Column



Query-use ciphertext Column



Detailed Solution

After understanding Apache ShardingSphere's encryption process, you can combine the encryption configuration and encryption process according to your scenario. The entire design & development was conceived to address the pain points encountered in business scenarios. So, how to use Apache ShardingSphere to meet the business requirements mentioned before?

Business scenario analysis: the newly launched business is relatively simple because it starts from scratch and there's no need to clean up historical data.

Solution description: after selecting the appropriate encryption algorithm, such as AES, you only need to configure the logical column (write SQL for users) and the ciphertext column (the data table stores the ciphertext data). The logical columns and ciphertext columns can also be different. The following configurations are recommended (in YAML format):

```

- !ENCRYPT
  encryptors:
    aes_encryptor:

```

```

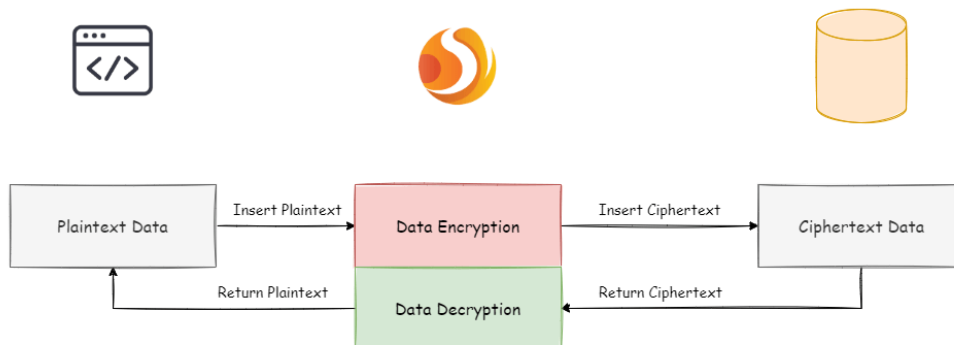
type: AES
props:
  aes-key-value: 123456abc
  digest-algorithm-name: SHA-1
tables:
  t_user:
    columns:
      pwd:
        cipher:
          name: pwd_cipher
          encryptorName: aes_encryptor
        assistedQuery:
          name: pwd_assisted_query
          encryptorName: pwd_assisted_query_cipher

```

With the above configuration, Apache ShardingSphere only needs to convert logicColumn, cipher-Column, and assistedQueryColumn.

The underlying data table does not store plaintext, and only ciphertext is stored, which is also the requirement of the security audit. The overall processing flow is shown in the figure below:

New online service



The advantages of Middleware encryption service

1. Automatic and transparent data encryption process. Encryption implementation details are no longer a concern for users.
2. It provides a variety of built-in and third-party (AKS) encryption algorithms, which are available through simple configurations.
3. It provides an encryption algorithm API interface. Users can implement the interface to use a custom encryption algorithm for data encryption.
4. It can switch among different encryption algorithms.

Solution

Apache ShardingSphere provides an encryption algorithm for data encryption, namely `EncryptAlgorithm`.

On the one hand, Apache ShardingSphere provides users with built-in implementation classes for encryption and decryption, which are available through configurations by users.

On the other hand, in order to be applicable to different scenarios, we also opened the encryption and decryption interfaces, and users can provide specific implementation classes according to these two types of interfaces.

After simple configuration, Apache ShardingSphere can call user-defined encryption and decryption schemes for data encryption.

12.7.4 EncryptAlgorithm

The solution provides two methods, `encrypt()` and `decrypt()`, to encrypt or decrypt data. When users perform INSERT, DELETE and UPDATE operations, ShardingSphere will parse, rewrite and route SQL according to the configuration.

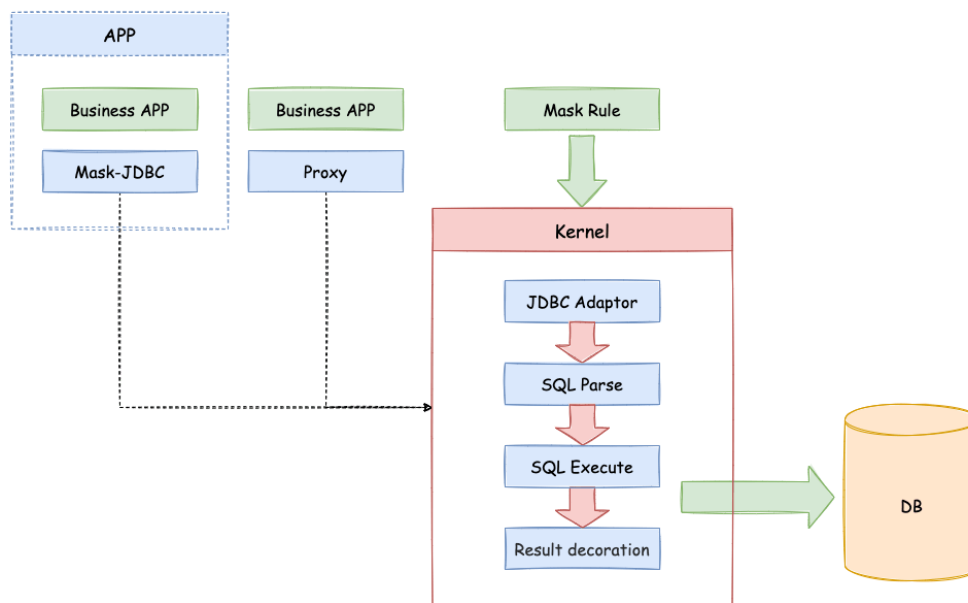
It will also use `encrypt()` to encrypt data and store them in the database. When using SELECT, they will decrypt sensitive data from the database with `decrypt()` and finally return the original data to users.

Currently, Apache ShardingSphere provides three types of implementations for this kind of encryption solution, including MD5 (irreversible), AES (reversible) and RC4 (reversible), which can be used after configuration.

12.8 Mask

Apache ShardingSphere achieves the desensitization of the original data by parsing the SQL queried by users and masking the SQL execution results according to the desensitization rules provided by users.

12.8.1 Overall Architecture

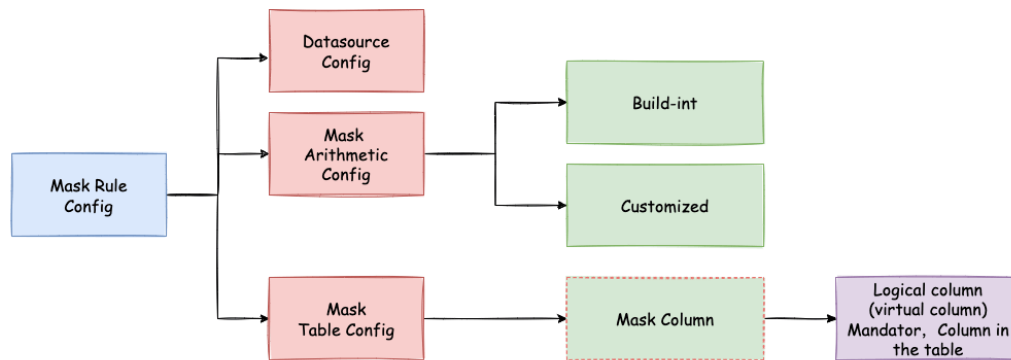


The desensitization module intercepts the SQL initiated by the user, analyzes and executes it through the SQL syntax parser. It then masks the query results by finding out the fields to be desensitized and the desensitization algorithm to be used according to the rules passed specified by the user, and returns to the client.

12.8.2 Mask Rules

Before explaining the whole process in detail, we need to first understand the desensitization rules and configuration, which is the basis of understanding the whole process.

Desensitization configuration is mainly divided into three parts: data source configuration, desensitization algorithm configuration, desensitization table configuration:



Data source configuration: the configuration of the data source.

Mask algorithm configuration: currently, ShardingSphere has a variety of built-in desensitization algorithms: MD5, KEEP_FIRST_N_LAST_M, KEEP_FROM_X_TO_Y , MASK_FIRST_N_LAST_M, MASK_FROM_X_TO_Y, MASK_BEFORE_SPECIAL_CHARS, MASK_AFTER_SPECIAL_CHARS and GENERIC_TABLE_RANDOM_REPLACE.

Users can also implement a set of desensitization algorithms by implementing the interface provided by ShardingSphere.

Mask table configuration: used to tell ShardingSphere which column in the data table is used for data desensitization and which algorithm is used for desensitization.

The mask rule takes effect after it is created

Query attribute configuration: if both plaintext and ciphertext data are stored in the underlying database table, this attribute can be used to determine whether to query the plaintext data in the database table and return it directly, or query the ciphertext data and return it after decryption through Apache ShardingSphere.

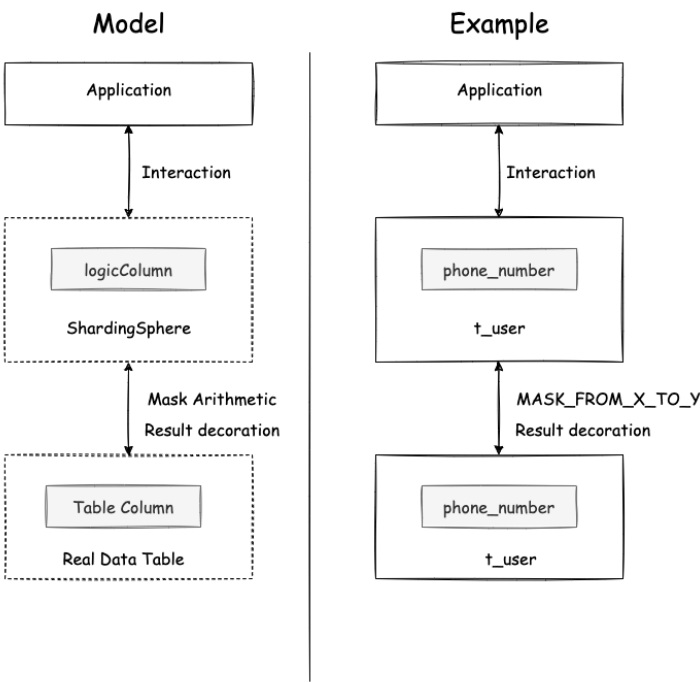
This attribute can be configured at the table level and the entire rule level. The table-level has the highest priority.

12.8.3 Mask Process

For example, if there is a table in the database called `t_user` and there is a field in the table called `phone_number` that uses `MASK_FROM_X_TO_Y`, Apache ShardingSphere does not change the data store.

It' ll only mask the result according to the desensitization algorithm, to achieve the desensitization effect.

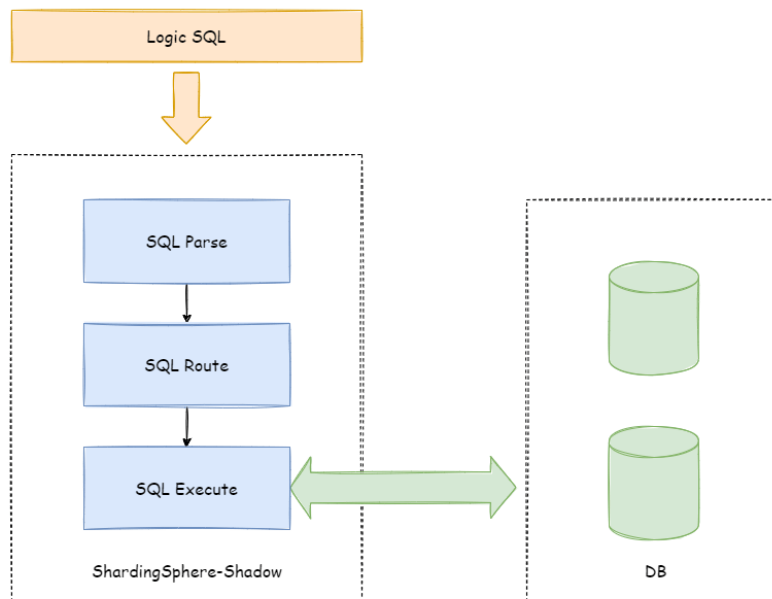
As shown in the picture below:



12.9 Shadow

12.9.1 How it works

Apache ShardingSphere determines the incoming SQL via shadow by parsing the SQL and routing it to the production or shadow database based on the shadow rules set by the user in the configuration file.



In the example of an INSERT statement, when writing data, Apache ShardingSphere parses the SQL and then constructs a routing chain based on the rules in the configuration file. In the current version, the shadow feature is at the last execution unit in the routing chain, i.e. if other rules exist that require routing, such as sharding, Apache ShardingSphere will first route to a particular database according to the sharding rules, and then run the shadow routing determination process to determine that the execution SQL meets the configuration set by shadow rules. Then data is routed to the corresponding shadow database, while the production data remains unchanged.

DML sentence

Two algorithms are supported. Shadow determination first determines whether the execution SQL-related table intersects with the configured shadow table. If the result is positive, the shadow algorithm within the part of intersection associated with the shadow table will be determined sequentially. If any of the determination is successful, the SQL statement is routed to the shadow library. If there is no intersection or the shadow algorithm determination is unsuccessful, the SQL statement is routed to the production database.

DDL sentence

Only supports shadow algorithm with comments attached. In stress testing scenarios, DDL statements are generally not required for testing, and are used mainly when initializing or modifying shadow tables in the shadow database. The shadow determination will first determine whether the execution SQL contains comments or not. If the result is a yes, the HINT shadow algorithm configured in the shadow rules determines them in order. The SQL statement is routed to the shadow database if any of the determinations are successful. If the execution SQL does not contain comments or the HINT shadow algorithm determination is unsuccessful, the SQL statements are routed to the production database.

12.9.2 References

[JAVA API: shadow database configuration](#)

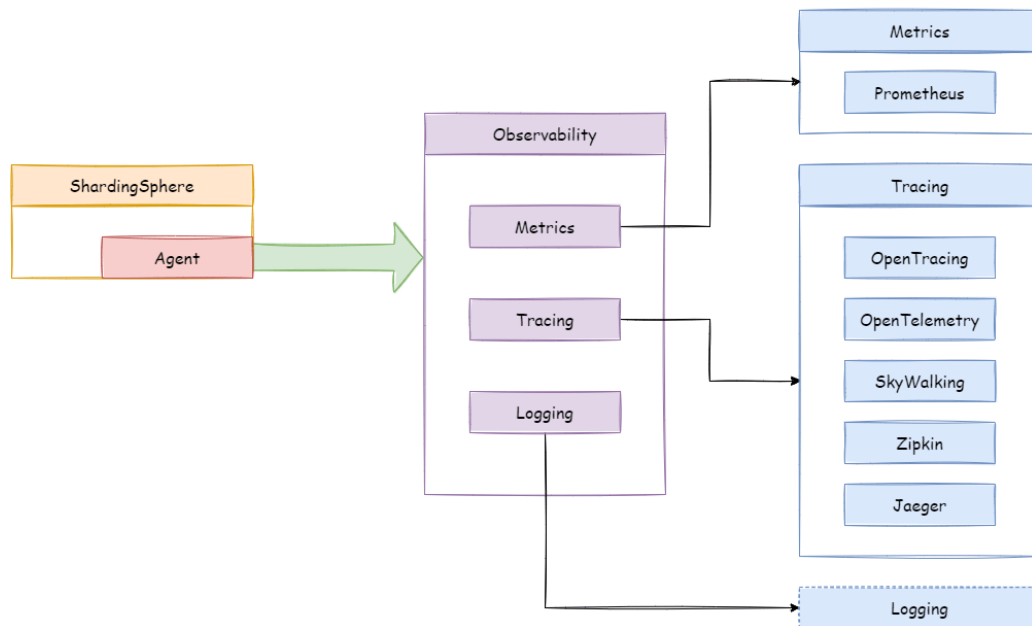
[YAMLconfiguration: shadow database](#)

12.10 Observability

12.10.1 How it works

ShardingSphere-Agent module provides an observable framework for ShardingSphere, which is implemented based on Java Agent.

Metrics, tracing and logging functions are integrated into the agent through plugins, as shown in the following figure:



- The metrics plugin is used to collect and display statistical indicators for the entire cluster. Apache ShardingSphere supports Prometheus by default.
- The tracing plugin is used to obtain the link trace information of SQL parsing and SQL execution. Apache ShardingSphere provides support for exporting tracing data to Jaeger and Zipkin by default. It also supports users developing customized tracing components through plugin.
- The default logging plugin shows how to record additional logs in ShardingSphere. In practical applications, users need to explore according to their own needs.

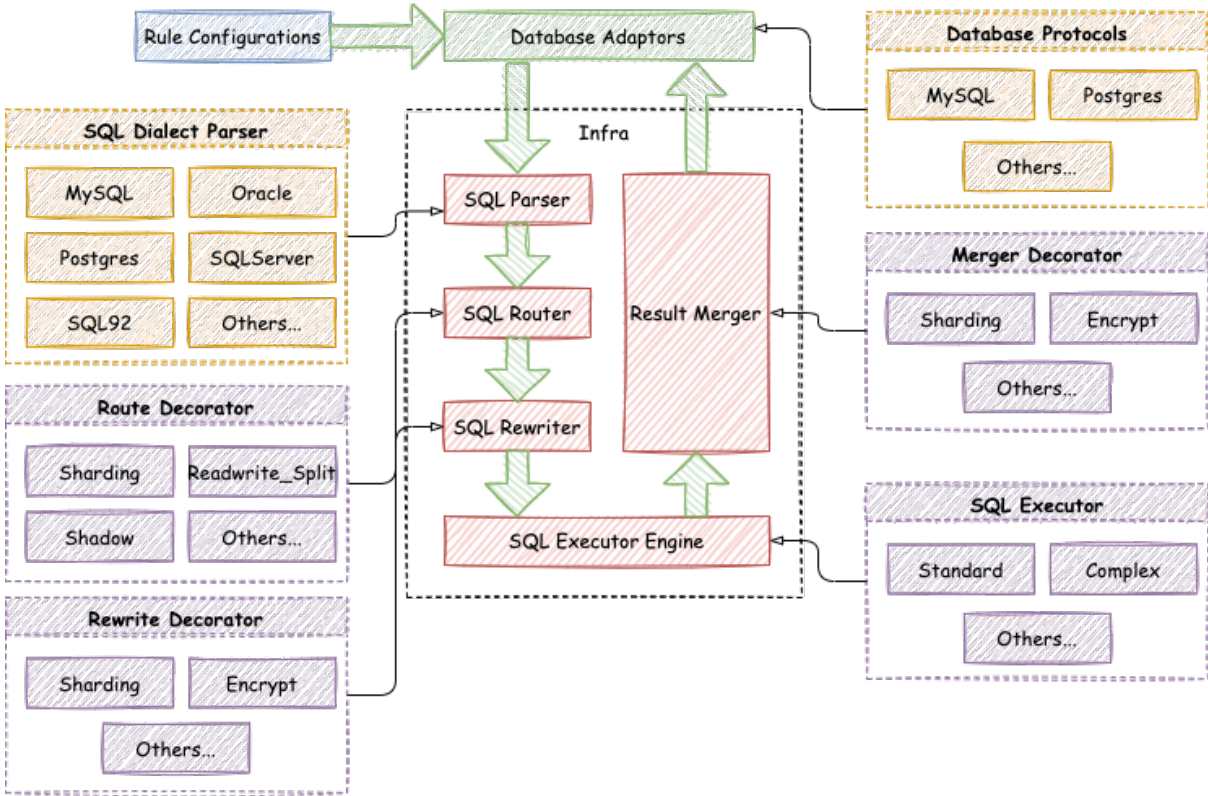
12.11 Architecture

Apache ShardingSphere's pluggable architecture is designed to enable developers to customize their own unique systems by adding the desired features, just like adding building blocks.

A plugin-oriented architecture has very high requirements for program architecture design. It requires making each module independent, and using a pluggable kernel to combine various functions in an overlapping manner. Designing an architecture system that completely isolates the feature development not only fosters an active open source community, but also ensures the quality of the project.

Apache ShardingSphere began to focus on the pluggable architecture since version 5.X, and the functional components of the project can be flexibly extended in a pluggable manner. Currently, features such as data sharding, read/write splitting, database high availability, data encryption, shadow DB stress testing, and support for SQL and protocols such as MySQL, PostgreSQL, SQLServer, Oracle, etc.

are woven into the project through plugins. Apache ShardingSphere has provided dozens of SPIs (service provider interfaces) as extension points of the system, with the total number still increasing.



13.1 JDBC

13.1.1 Found a JtaTransactionManager in spring boot project when integrating with XAtransaction.

Answer:

1. shardingsphere-transaction-xa-core include atomikos, it will trigger auto-configuration mechanism in spring-boot, add @SpringBootApplication(exclude = JtaAutoConfiguration.class) will solve it.

13.1.2 The tableName and columnName configured in yaml or properties leading incorrect result when loading Oracle metadata?

Answer:

Note that, in Oracle's metadata, the tableName and columnName is default UPPERCASE, while double-quoted such as CREATE TABLE "TableName" ("Id" number) the tableName and columnName is the actual content double-quoted, refer to the following SQL for the reality in metadata:

```
SELECT OWNER, TABLE_NAME, COLUMN_NAME, DATA_TYPE FROM ALL_TAB_COLUMNS WHERE TABLE_NAME IN ('TableName')
```

ShardingSphere uses the OracleTableMetaDataLoader to load the metadata, keep the tableName and columnName in the yaml or properties consistent with the metadata. ShardingSphere assembled the SQL using the following code:

```
private String getTableMetaDataSQL(final Collection<String> tables, final DatabaseMetaData metaData) throws SQLException {
    StringBuilder stringBuilder = new StringBuilder(28);
    if (versionContainsIdentityColumn(metaData)) {
        stringBuilder.append(", IDENTITY_COLUMN");
    }
}
```

```

    }
    if (versionContainsCollation(metaData)) {
        stringBuilder.append(", COLLATION");
    }
    String collation = stringBuilder.toString();
    return tables.isEmpty() ? String.format(TABLE_META_DATA_SQL, collation)
        : String.format(TABLE_META_DATA_SQL_IN_TABLES, collation, tables.
stream().map(each -> String.format("%s", each)).collect(Collectors.joining(", ")));
}

```

13.1.3 SQLException: Unable to unwrap to interface com.mysql.jdbc.Connection exception thrown when using MySQL XA transaction

Answer:

Incompatibility between multiple MySQL drivers. Because the MySQL5 version of the driver class under the class path is loaded first, when trying to call the unwrap method in the MySQL8 driver, the type conversion exception occurs.

The solutions: Check whether there are both MySQL5 and MySQL8 drivers in the class path, and only keep one driver package of the corresponding version.

The exception stack is as follows:

```

Caused by: java.sql.SQLException: Unable to unwrap to interface com.mysql.jdbc.
Connection
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:129)
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:97)
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:89)
    at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:63)
    at com.mysql.cj.jdbc.ConnectionImpl.unwrap(ConnectionImpl.java:2650)
    at com.zaxxer.hikari.pool.ProxyConnection.unwrap(ProxyConnection.java:481)
    at org.apache.shardingsphere.transaction.xa.jta.connection.dialect.
MySQLXAConnectionWrapper.wrap(MySQLXAConnectionWrapper.java:46)
    at org.apache.shardingsphere.transaction.xa.jta.datasource.
XATransactionDataSource.getConnection(XATransactionDataSource.java:89)
    at org.apache.shardingsphere.transaction.xa.XAShardingSphereTransactionManager.
getConnection(XAShardingSphereTransactionManager.java:96)

```

13.2 Proxy

13.2.1 In Windows environment, could not find or load main class org.apache.shardingsphere.proxy.Bootstrap, how to solve it?

Answer:

Some decompression tools may truncate the file name when decompressing the ShardingSphere-Proxy binary package, resulting in some classes not being found. The solutions: Open cmd.exe and execute the following command:

```
tar zxvf apache-shardingsphere-${RELEASE.VERSION}-shardingsphere-proxy-bin.tar.gz
```

13.2.2 How to add a new logic database dynamically when use ShardingSphere-Proxy?

Answer:

When using ShardingSphere-Proxy, users can dynamically create or drop logic database through Dist-SQL, the syntax is as follows:

```
CREATE DATABASE [IF NOT EXISTS] databaseName;  
DROP DATABASE [IF EXISTS] databaseName;
```

Example:

```
CREATE DATABASE sharding_db;  
DROP DATABASE sharding_db;
```

13.2.3 How to use suitable database tools connecting ShardingSphere-Proxy?

Answer:

1. ShardingSphere-Proxy could be considered as a MySQL server, so we recommend using MySQL command line tool to connect to and operate it.
2. If users would like to use a third-party database tool, there may be some errors cause of the certain implementation/options.
3. The currently tested third-party database tools are as follows:
 - DataGrip: 2020.1, 2021.1 (turn on “introspect using jdbc metadata” in idea or datagrip).
 - MySQLWorkBench: 8.0.25.

13.2.4 When using a client to connect to ShardingSphere-Proxy, if ShardingSphere-Proxy does not create a database or does not register a storage unit, the client connection will fail?

Answer:

1. Third-party database tools will send some SQL query metadata when connecting to ShardingSphere-Proxy. When ShardingSphere-Proxy does not create a Database or does not register a Storage Unit, ShardingSphere-Proxy cannot execute SQL.
2. It is recommended to create database and register storage unit first, and then use third-party database tools to connect.
3. Please refer to [Related introduction](#) the details about storage unit.

13.3 Sharding

13.3.1 How to solve Cloud not resolve placeholder ...in string value ... error?

Answer:

`${...}` or `$->{...}` can be used in inline expression identifiers using the default implementation of the `InlineExpressionParser` SPI, but the former one clashes with place holders in Spring property files, so `$->{...}` is recommended to be used in Spring as inline expression identifiers.

13.3.2 Why does float number appear in the return result of inline expression?

Answer:

The division result of Java integers is also integer, but in Groovy syntax of inline expression, the division result of integers is float number. To obtain integer division result, `A/B` needs to be modified as `A.intdiv(B)`.

13.3.3 If sharding database is partial, should tables without sharding database & table configured in sharding rules?

Answer:

A table that does not use sharding is called single table in ShardingSphere, and you can use [LOAD statements](#) or [SINGLE rule](#) to configure the single table that needs to be loaded.

13.3.4 When generic Long type SingleKeyTableShardingAlgorithm is used, why does the ClassCastException: Integer can not cast to Long exception appear?

Answer:

You must make sure the field in the database table is consistent with that in the sharding algorithms. For example, the field type in database is int(11) and the sharding type corresponds to genetic type is Integer. If you want to configure Long type, please make sure the field type in the database is bigint.

13.3.5 [Sharding:raw-latex:PROXY] When implementing the Standard-ShardingAlgorithm custom algorithm, the specific type of Comparable is specified as Long, and the field type in the database table is bigint, a ClassCastException: Integer can not cast to Long exception occurs.

Answer:

When implementing the doSharding method, it is not recommended to specify the specific type of Comparable in the method declaration, but to convert the type in the implementation of the doSharding method. You can refer to the ModShardingAlgorithm#doSharding method.

13.3.6 Why is the default distributed auto-increment key strategy provided by ShardingSphere not continuous and most of them end with even numbers?

Answer:

ShardingSphere uses snowflake algorithms as the default distributed auto-increment key strategy to make sure unrepeated and decentralized auto-increment sequence is generated under the distributed situations. Therefore, auto-increment keys can be incremental but not continuous. But the last four numbers of snowflake algorithm are incremental value within one millisecond. Thus, if concurrency degree in one millisecond is not high, the last four numbers are likely to be zero, which explains why the rate of even end number is higher. In 3.1.0 version, the problem of ending with even numbers has been totally solved, please refer to: <https://github.com/apache/shardingsphere/issues/1617>

13.3.7 How to allow range query with using inline sharding strategy (BETWEEN AND, >, <, >=, <=)?

Answer:

1. Update to 4.1.0 above.
2. Configure(A tip here: then each range query will be broadcast to every sharding table):
 - Version 4.x: `allow.range.query.with.inline.sharding` to true (Default value is false).

- Version 5.x: `allow-range-query-with-inline-sharding` to true in `InlineShardingStrategy` (Default value is false).

13.3.8 Why does my custom distributed primary key do not work after implementing `KeyGenerateAlgorithm` interface and configuring type property?

Answer:

`Service Provider Interface (SPI)` is a kind of API for the third party to implement or expand. Except implementing interface, you also need to create a corresponding file in `META-INF/services` to make the JVM load these SPI implementations. More detail for SPI usage, please search by yourself. Other ShardingSphere functionality implementation will take effect in the same way.

13.3.9 In addition to internal distributed primary key, does ShardingSphere support other native auto-increment keys?

Answer:

Yes. But there is restriction to the use of native auto-increment keys, which means they cannot be used as sharding keys at the same time. Since ShardingSphere does not have the database table structure and native auto-increment key is not included in original SQL, it cannot parse that field to the sharding field. If the auto-increment key is not sharding key, it can be returned normally and is needless to be cared. But if the auto-increment key is also used as sharding key, ShardingSphere cannot parse its sharding value, which will make SQL routed to multiple tables and influence the rightness of the application. The premise for returning native auto-increment key is that INSERT SQL is eventually routed to one table. Therefore, auto-increment key will return zero when INSERT SQL returns multiple tables.

13.4 Single table

13.4.1 Table or view %s does not exist. How to solve the exception?

Answer:

In versions before ShardingSphere 5.4.0, single tables used automatic loading. This way has many problems in actual use:

1. After a large number of data sources are registered in the logical database, too many automatically loaded single tables will cause ShardingSphere-Proxy/JDBC to start slowly;
2. When users use DistSQL, they will operate in the order of: **Register storage unit -> Create sharding, encryption, read-write separation and other rules -> Create table**. Due to the existence of the single-table automatic loading mechanism, the database will be accessed multiple times for loading during the operation, and when multiple rules are mixed and used, the single-table metadata will be confused;

3. Automatically load single tables from all data sources. Users cannot exclude single tables or abandoned tables that they do not want to be managed by ShardingSphere.

In order to solve the above problems, starting from ShardingSphere 5.4.0 version, the loading method of single tables has been adjusted. Users need to manually load a single table in the database through YAML configuration or DistSQL. It should be noted that when using the DistSQL LOAD statement to load a single table, you need to ensure that all data sources are registered. Therefore, after the rules are created, the single table LOAD operation is performed based on the logical data source (if there is no logical data source, use the physical data source).

- YAML loading single table example:

```
rules:
  - !SINGLE
    tables:
      - "*"
  - !READWRITE_SPLITTING
    dataSourceGroups:
      readwrite_ds:
        writeDataSourceName: write_ds
        readDataSourceNames:
          - read_ds_0
          - read_ds_1
        loadBalancerName: random
    loadBalancers:
      random:
        type: RANDOM
```

For more YAML configuration of loading single table, please refer to [Single](#).

- DistSQL loading single table example:

```
LOAD SINGLE TABLE *.*;
```

For more LOAD single table DistSQL, please refer to [Load Single Table](#).

13.5 DistSQL

13.5.1 How to set custom JDBC connection properties or connection pool properties when adding a data source using DistSQL?

Answer:

1. If you need to customize JDBC connection properties, please take the `urlSource` way to define `dataSource`.
2. ShardingSphere presets necessary connection pool properties, such as `maxPoolSize`, `idleTimeout`, etc. If you need to add or overwrite the properties, please specify it with `PROPERTIES` in the `dataSource`.

3. Please refer to [Related introduction](#) for above rules.

13.5.2 How to solve Storage unit [xxx] is still used by [SingleRule]. exception when dropping a data source using DistSQL?

Answer:

1. Storage units referenced by rules cannot be deleted
2. If the storage unit is only referenced by single rule, and the user confirms that the restriction can be ignored, the optional parameter ignore single tables can be added to perform forced deletion

```
UNREGISTER STORAGE UNIT storageUnitName [, storageUnitName] ... [ignore single tables]
```

13.5.3 How to solve Failed to get driver instance for jdbcURL=xxx. exception when adding a data source using DistSQL?

Answer:

ShardingSphere Proxy do not have jdbc driver during deployment. Some example of this include mysql-connector. To use it otherwise following syntax can be used:

```
REGISTER STORAGE UNIT storageUnit [..., storageUnit]
```

13.6 Other

13.6.1 How to debug when SQL can not be executed rightly in ShardingSphere?

Answer:

sql.show configuration is provided in ShardingSphere-Proxy and post-1.5.0 version of ShardingSphere-JDBC, enabling the context parsing, rewritten SQL and the routed data source printed to info log. sql.show configuration is off in default, and users can turn it on in configurations. A Tip: Property sql.show has changed to sql-show in version 5.x.

13.6.2 Why do some compiling errors appear? Why did not the IDEA index the generated codes?

Answer:

ShardingSphere uses lombok to enable minimal coding. For more details about using and installment, please refer to the official website of [lombok](#). The codes under the package org.apache.shardingsphere.sql.parser.autogen are generated by ANTLR. You may execute the following command to generate codes:

```
./mvnw -DskipITs -DskipTests install -T1C
```

The generated codes such as `org.apache.shardingsphere.sql.parser.autogen.PostgreSQLStatementParser` may be too large to be indexed by the IDEA. You may configure the IDEA's property `idea.max.intellisense.filesize=10000`.

13.6.3 In SQLServer and PostgreSQL, why does the aggregation column without alias throw exception?

Answer:

SQLServer and PostgreSQL will rename aggregation columns acquired without alias, such as the following SQL:

```
SELECT SUM(num), SUM(num2) FROM tablexxx;
```

Columns acquired by SQLServer are empty string and (2); columns acquired by PostgreSQL are empty sum and sum(2). It will cause error because ShardingSphere is unable to find the corresponding column. The right SQL should be written as:

```
SELECT SUM(num) AS sum_num, SUM(num2) AS sum_num2 FROM tablexxx;
```

13.6.4 Why does Oracle database throw “Order by value must implements Comparable” exception when using Timestamp Order By?

Answer:

There are two solutions for the above problem: 1. Configure JVM parameter “`-oracle.jdbc.J2EE13Compliant=true`” 2. Set `System.getProperties().setProperty(“oracle.jdbc.J2EE13Compliant”, “true”)` codes in the initialization of the project. Reasons: `org.apache.shardingsphere.sharding.merge.dql.orderby.OrderByValue#getOrderValues()`:

```
private List<Comparable<?>> getOrderValues() throws SQLException {
    List<Comparable<?>> result = new ArrayList<>(orderByItems.size());
    for (OrderByItem each : orderByItems) {
        Object value = queryResult.getValue(each.getIndex(), Object.class);
        Preconditions.checkState(null == value || value instanceof Comparable,
            "Order by value must implements Comparable");
        result.add((Comparable<?>) value);
    }
    return result;
}
```

After using `resultSet.getObject(int index)`, for Timestamp oracle, the system will decide whether to return `java.sql.Timestamp` or define `oracle.sql.TIMESTAMP` according to the property of `oracle.jdbc.J2EE13Compliant`. See `oracle.jdbc.driver.TimestampAccessor#getObject(int var1)` method in `ojdbc` codes for more detail:

```

Object getObject(int var1) throws SQLException {
    Object var2 = null;
    if(this.rowSpaceIndicator == null) {
        DatabaseError.throwSQLException(21);
    }
    if(this.rowSpaceIndicator[this.indicatorIndex + var1] != -1) {
        if(this.externalType != 0) {
            switch(this.externalType) {
                case 93:
                    return this.getTimestamp(var1);
                default:
                    DatabaseError.throwSQLException(4);
                    return null;
            }
        }
        if(this.statement.connection.j2ee13Compliant) {
            var2 = this.getTimestamp(var1);
        } else {
            var2 = this.getTIMESTAMP(var1);
        }
    }
    return var2;
}

```

13.6.5 In Windows environment,when cloning ShardingSphere source code through Git, why prompt filename too long and how to solve it?

Answer:

To ensure the readability of source code,the ShardingSphere Coding Specification requires that the naming of classes,methods and variables be literal and avoid abbreviations,which may result in some source files have long names. Since the Git version of Windows is compiled using msys,it uses the old version of Windows Api,limiting the file name to no more than 260 characters. The solutions are as follows: Open cmd.exe (you need to add git to environment variables) and execute the following command to allow git supporting log paths:

```
git config --global core.longpaths true
```

If we use windows 10, also need enable win32 log paths in registry editor or group strategy(need reboot):
 > Create the registry key HKLM\SYSTEM\CurrentControlSet\Control\FileSystem LongPath-enabled (Type: REG_DWORD) in registry editor, and be set to 1. > Or click “setting” button in system menu, print “Group Policy” to open a new window “Edit Group Policy”, and then click ‘Computer Configuration’ > ‘Administrative Templates’ > ‘System’ > ‘Filesystem’, and then turn on ‘Enable Win32 long paths’ option. Reference material: <https://docs.microsoft.com/zh-cn/windows/desktop/FileIO/naming-a-file> <https://ourcodeworld.com/articles/read/109/how-to-solve-filename-too-long-error-in-git-powershell-and-github-application-for-windows>

13.6.6 How to solve `Type is required` error?

Answer:

In Apache ShardingSphere, many functionality implementation are uploaded through SPI, such as Distributed Primary Key. These functions load SPI implementation by configuring the type, so the type must be specified in the configuration file.

13.6.7 How to speed up the metadata loading when service starts up?

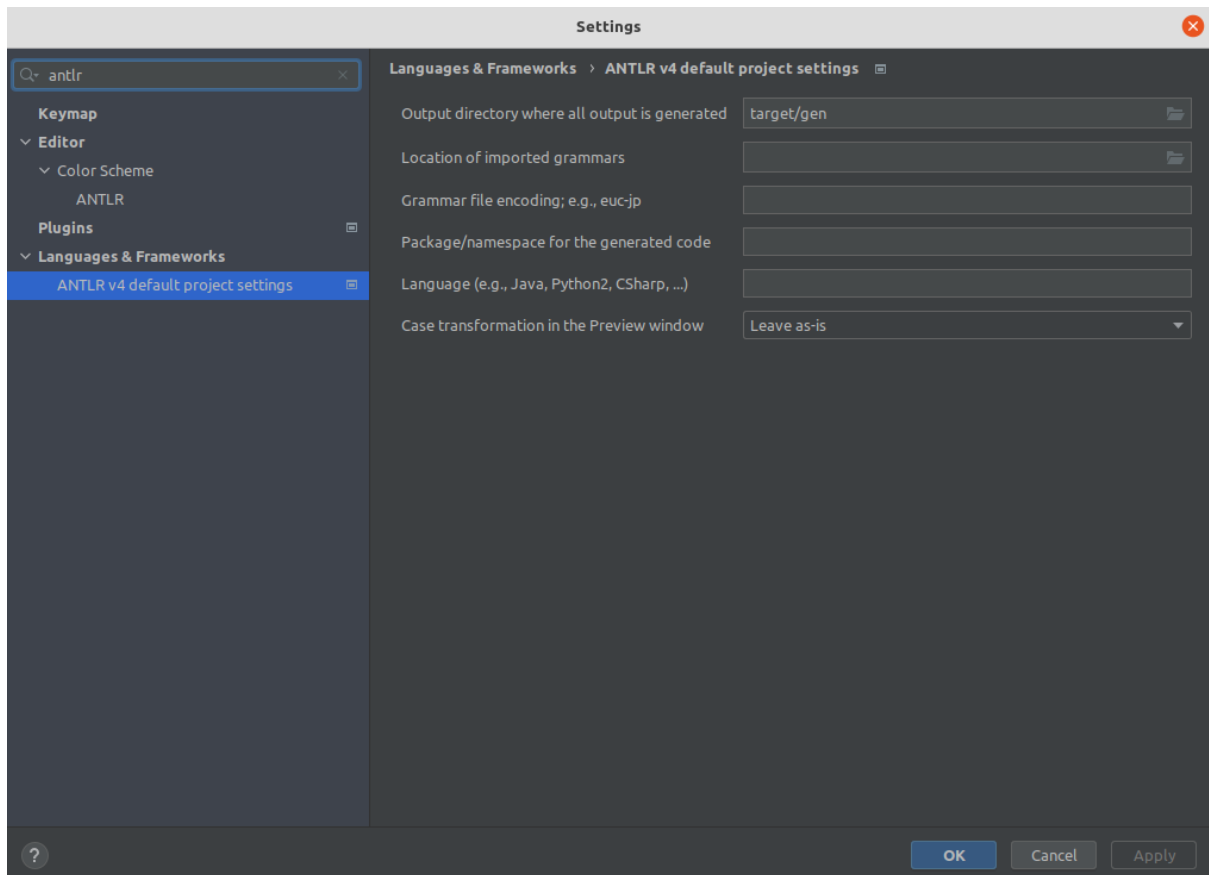
Answer:

1. Update to 4.0.1 above, which helps speed up the process of loading table metadata.
2. Configure:
 - `max.connections.size.per.query`(Default value is 1) higher referring to connection pool you adopt(Version $\geq$ 3.0.0.M3 & Version $<$ 5.0.0).
 - `max-connections-size-per-query`(Default value is 1) higher referring to connection pool you adopt(Version $\geq$ 5.0.0).

13.6.8 The ANTLR plugin generates codes in the same level directory as `src`, which is easy to commit by mistake. How to avoid it?

Answer:

Goto `Settings` -> `Languages & Frameworks` -> `ANTLR v4 default project settings` and configure the output directory of the generated code as `target/gen` as shown:



13.6.9 Why is the database sharding result not correct when using Proxool?

Answer:

When using Proxool to configure multiple data sources, each one of them should be configured with alias. It is because Proxool would check whether existing alias is included in the connection pool or not when acquiring connections, so without alias, each connection will be acquired from the same data source. The followings are core codes from ProxoolDataSource getConnection method in Proxool:

```
if(!ConnectionPoolManager.getInstance().isPoolExists(this.alias)) {
    this.registerPool();
}
```

For more alias usages, please refer to [Proxool](#) official website.

14.1 Latest Releases

Apache ShardingSphere is released as source code tarballs with corresponding binary tarballs for convenience. The downloads are distributed via mirror sites and should be checked for tampering using GPG or SHA-512.

14.1.1 Apache ShardingSphere - Version: 5.5.2 (Release Date: January 16th, 2025)

- Source Codes: [SRC](#) ([ASC](#), [SHA512](#))
- ShardingSphere-JDBC Binary Distribution: [TAR](#) ([ASC](#), [SHA512](#))
- ShardingSphere-Proxy Binary Distribution: [TAR](#) ([ASC](#), [SHA512](#))
- ShardingSphere-Agent Binary Distribution: [TAR](#) ([ASC](#), [SHA512](#))

14.2 All Releases

Find all releases in the [Archive repository](#). Find all incubator releases in the [Archive incubator repository](#).

14.3 Verify the Releases

PGP signatures KEYS

It is essential that you verify the integrity of the downloaded files using the PGP or SHA signatures. The PGP signatures can be verified using GPG or PGP. Please download the KEYS as well as the asc signature files for relevant distribution. It is recommended to get these files from the main distribution directory and not from the mirrors.

```
gpg -i KEYS
```

or

```
pgpk -a KEYS
```

or

```
gpg -ka KEYS
```

To verify the binaries/sources you can download the relevant asc files for it from main distribution directory and follow the below guide.

```
gpg --verify apache-shardingsphere-*****.asc apache-shardingsphere-*****
```

or

```
pgpv apache-shardingsphere-*****.asc
```

or

```
gpg apache-shardingsphere-*****.asc
```